R for Health Economics & Outcomes Research and Health Technology Assessment



Dr Aswath Karunakaran

ONE MUST IMAGINE SISYPHUS HAPPY.

ALBERT CAMUS

R for Health Economics and Outcomes Research (HEOR) and Health Technology Assessment (HTA)

A practical, reproducible guide to decision-analytic modelling, evidence synthesis, and HTA submission workflows in R

Dr Aswath Karunakaran

Draft edition — compiled progressively, chapter by chapter

Foreword

Health technology assessment sits at an uncomfortable junction: the discipline demands the statistical rigour of a biostatistics department, the structural clarity of an actuarial model, and the practical honesty of a budget-holder who has to live with the decision an analysis recommends. Few analysts arrive at HEOR already fluent in all three. Most arrive, as this book's author did, from a single adjacent discipline — public health, epidemiology, clinical medicine, economics, or data science — and build the rest on the job, one model, one submission, one committee critique at a time.

This book exists because that on-the-job build is harder than it should be. The canonical references on health economic modelling (Briggs, Claxton and Sculpher's **Decision Modelling for Health Economic Evaluation**; Drummond and colleagues' **Methods for the Economic Evaluation of Health Care Programmes**) are rightly foundational, but they were not written as R programming references, and R's own documentation was not written with HTA submission requirements in mind. The gap between "I understand what a partitioned survival model is" and "I can build one in R that a NICE evidence review group will accept without a technical query" is where most working analysts spend their first several difficult years.

What follows is an attempt to close that gap directly: every statistical and economic concept is paired with working R code; every modelling chapter is grounded in what the major HTA agencies actually ask to see; every method is introduced with the assumption that the reader is capable and motivated, not that the reader has already sat through a two-year health economics master's programme. Readers coming from applied public health and programmatic data work — TB programme analysis, national disease registries, LMIC health systems — will find this book written with their vantage point as an explicit first-class concern, not an afterthought bolted onto a Western-payer-market default.

No single book makes anyone a health economist. This one aims to make the R side of the job considerably less lonely.

Preface

Why R for HEOR and HTA

Health economic models have historically lived in Excel and TreeAge, with SAS handling the clinical-trial statistics feeding into them. That world is changing, and not slowly. R now has a mature, purpose-built ecosystem for every stage of the HEOR workflow — `flexsurv` and `survival` for extrapolation, `heemod` and `hesim` for cohort and individual-level state-transition models, `BCEA` and `voi` for uncertainty and value-of-information analysis, `netmeta` and `gemtc` for network meta-analysis, `shiny` for submission-ready interactive model front-ends. HTA agencies, NICE chief among them, now routinely receive and review R-based submissions, and several publish explicit methodological guidance (the NICE DSU Technical Support Documents) that assumes a modeller capable of implementing the methods in code, not just describing them in a Word document.

R's advantages over the Excel/TreeAge default are not merely aesthetic. Version control is native. A model's entire parameter set, structural logic, and reporting pipeline can live in one reviewable, re-runnable repository instead of a spreadsheet with hidden formulas and undocumented macro buttons. Probabilistic sensitivity analysis that would take an Excel model minutes per thousand iterations runs in seconds. Reproducibility — increasingly a condition of serious HTA submission, and squarely the subject of the CHEERS 2022 reporting checklist — is close to automatic in R and close to aspirational in Excel.

None of this makes R effortless. It trades spreadsheet transparency (anyone can click a cell and see the formula) for code transparency (anyone who can read R can see the logic, but first they have to be able to read R). This book's purpose is to make that trade pay off quickly rather than slowly.

Who this book is for

This book assumes:

- Working competence in R at the level of being comfortable with data frames, basic data wrangling, and writing simple functions — not mastery, and Chapter 3 exists specifically to shore up anything shaky here.
- No prior formal health economics training. Chapter 4 builds the core concepts — QALYs, ICERs, discounting, perspective — from first principles, with R code alongside each concept as it is introduced.
- A practical, applied motivation: you need to build, review, critique, or commission health economic models and evidence syntheses as part of real work, whether in a pharmaceutical HEOR team, an HTA agency, an academic health economics group, a consultancy, or — a case this book treats as a first-class audience rather than a footnote — a public health programme in a low- or middle-income country assessing interventions against a national reference case and a genuinely binding budget constraint.

Readers coming from applied public health, programmatic disease surveillance, or clinical data analysis backgrounds will recognise much of the data-quality and real-world-evidence material in Part II and Part III. That recognition is intentional: the skills that make someone good at program-

matic TB, HIV, or immunisation data analysis — scepticism about data completeness, comfort with messy field data, an instinct for what a confidence interval is actually telling you — transfer directly into HEOR, and this book tries to make that transfer explicit rather than leaving the reader to discover it unaided.

How to use this book

The book is organised to be read front-to-back by someone starting from zero, but each Part is also written to stand largely on its own for a reader who already has foundations and wants to go straight to, say, network meta-analysis (Part III) or decision-tree and Markov modelling (Part IV). Every chapter opens with explicit learning objectives and closes with exercises; "HEOR in practice" boxes throughout ground the method in a real submission scenario, "Common pitfalls" boxes flag the mistakes that recur most often in actual agency technical queries, and "Agency perspective" boxes state plainly what a reviewer at NICE, CADTH, PBAC, or HTAIn is likely to be checking for.

Part VIII's end-to-end case studies, including a TB programme cost-effectiveness analysis built for an LMIC setting, are deliberately placed last: by that point every technique used in them has already been introduced on its own, and the case studies exist to show how the pieces assemble into a real, submission-shaped analysis.

A note on how this edition was produced

This draft edition is being written progressively, chapter by chapter, with R code and HTA-practice content checked against standard references (ISPOR-SMDM good-practice guidance, NICE DSU Technical Support Documents, Briggs/Claxton/Sculpher, Drummond et al.) as each chapter is completed, rather than delivered as a single finished manuscript. Readers working through an early draft chapter should treat package version numbers and function signatures as subject to verification against the live CRAN package before production use in a real submission — Chapter 2's `renv` and reproducibility guidance exists precisely so that this kind of drift between a book's print date and a package's current behaviour never silently breaks a real model.

Companion Website, Code Repository and Data Sets

All R code in this book is intended to live in a companion GitHub repository, organised to mirror the book's chapter structure (`ch01/`, `ch02/`, … `ch35/`), with each chapter's code runnable standalone given the chapter's own `renv.lock` file (see Chapter 2 §2.3 for why each chapter — not just the book as a whole — pins its own package versions).

Planned repository contents, to be populated as each chapter is finalised:

- `/code/` — one subfolder per chapter, containing every code listing in that chapter as a runnable `.R` or `.qmd` file, not just the excerpts shown in the printed text.
- `/data/` — the synthetic and openly-licensed example datasets used throughout (see Appendix F for the full data dictionary); no real patient-level or commercially sensitive trial data is used anywhere in this book.
- `/appendix-c-template/` — the full template R project structure from Appendix C, ready to fork for a new real-world analysis.
- `/solutions/` — worked solutions to end-of-chapter exercises (Appendix G).

Until the repository is published, each chapter's code lives alongside its Typst source in this project's own `typst_project/` folder and the parallel `Chapter Markdowns/` summaries, which together serve as the working source of truth during drafting. A working local draft of the `/code/` folder itself — one subfolder per chapter, auto-extracted from each chapter's typeset code blocks and kept current after every edit — already exists locally and is what gets pushed into the public repository once it is published.

Reporting errors

Health economic models are unforgiving of small coding errors — a flipped sign in a discounting formula or a mis-specified transition matrix can silently produce a materially wrong ICER. Any reader who finds an error in this book's code or its statistical claims should treat that as worth flagging immediately rather than working around it quietly; the companion repository's issue tracker will be the designated channel once published.

Conventions, Package Versions and Reproducibility Notes

Typographic conventions

- Inline R code, function names, and package names appear in `monospace`: e.g. `heemod::run_model()`, `flexsurv`.
- Standalone code listings are set in full-width monospace blocks with syntax-relevant elements distinguishable by weight; output shown beneath a listing is marked `#>` per R console convention.
- Mathematical notation follows standard health-economics usage: discount rate as r, time horizon as T, incremental cost as ΔC, incremental effect as ΔE, and the incremental cost-effectiveness ratio as $\text{ICER} = \Delta\frac{C}{\Delta}E$.
- Callout boxes recur across every methods chapter in three forms: **HEOR in practice** (a real-submission-shaped scenario illustrating the method just introduced), **Common pitfall** (a mistake that recurs often enough in actual agency technical-engagement queries to be worth naming explicitly), and **Agency perspective** (what a named HTA agency's published guidance or typical reviewer behaviour says about the method).

Package versions

Each chapter's code is developed and checked against the CRAN package versions current at the time that chapter is drafted; the exact versions are pinned in that chapter's own `renv.lock` in the companion repository (§ Companion Website, above) rather than hard-coded into the printed text, because HEOR-relevant packages — `flexsurv`, `hesim`, `heemod`, `BCEA`, `netmeta`, `mice`, among others — update frequently enough that a version number printed in a book is stale within a year or two. Where a function's behaviour has changed in a way that would break a book example, the companion repository's `CHANGES.md` is the authority, not the printed page.

Reproducibility baseline

Every worked example and case study in this book is built to run from a clean R session given only: base R, the packages named in that chapter, and the `renv.lock` for that chapter. No worked example depends on data, credentials, or infrastructure that a reader cannot access independently — real patient-level trial data is never used; synthetic or openly-licensed datasets (Appendix F) stand in throughout, chosen specifically to reproduce the statistical features (skewness, censoring, missingness patterns) that make the real-world versions of these analyses hard.

A note on country and agency defaults

Where a chapter must choose a concrete worked example — a willingness- to-pay threshold, a reference case, a named agency's submission template — this book deliberately rotates among NICE (UK), CADTH (Canada), PBAC (Australia), and HTAIn (India) rather than defaulting silently

to a single high-income reference case throughout. Chapter 31 lays out the comparison explicitly; earlier chapters that use a single agency's convention for a worked example say so directly rather than implying it as the universal default.

Abbreviations

ADaM	Analysis Data Model (CDISC)
AE	Adverse Event
AFT	Accelerated Failure Time (survival model)
AIC	Akaike Information Criterion
AMNOG	Arzneimittelmarktneuordnungsgesetz (German early benefit assessment process)
API	Application Programming Interface
BCEA	Bayesian Cost-Effectiveness Analysis (R package)
BIC	Bayesian Information Criterion
BLAS	Basic Linear Algebra Subprograms
CADTH	Canadian Agency for Drugs and Technologies in Health
CBA	Cost-Benefit Analysis
CCA	Cost-Consequence Analysis
CDISC	Clinical Data Interchange Standards Consortium
CEA	Cost-Effectiveness Analysis
CEAC	Cost-Effectiveness Acceptability Curve
CHEERS	Consolidated Health Economic Evaluation Reporting Standards
CI	Confidence Interval
CIF	Cumulative Incidence Function
CMA	Cost-Minimisation Analysis
CONSORT	Consolidated Standards of Reporting Trials
COI	Cost-of-Illness (study)
CPT	Current Procedural Terminology
CRAN	Comprehensive R Archive Network
CRO	Contract Research Organisation
CUA	Cost-Utility Analysis
DALY	Disability-Adjusted Life Year
DBI	R's DataBase Interface package
DCE	Discrete Choice Experiment

DHS	Demographic and Health Surveys
DIC	Deviance Information Criterion
DL	Deep Learning
DOI	Digital Object Identifier
DPDPA	Digital Personal Data Protection Act (India, 2023)
DR-TB	Drug-Resistant Tuberculosis
DSU	Decision Support Unit (NICE)
DTA	Diagnostic Test Accuracy
ECOG	Eastern Cooperative Oncology Group (performance status scale)
EHR	Electronic Health Record
EM	Expectation-Maximisation (algorithm)
EMB	Expectation-Maximisation with Bootstrapping (Amelia algorithm)
EORTC	European Organisation for Research and Treatment of Cancer
EQ-5D	EuroQol 5-Dimension (health utility instrument)
ESS	Effective Sample Size
FDA	Food and Drug Administration (United States)
GCP	Good Clinical Practice
GDP	Gross Domestic Product
GDPR	General Data Protection Regulation (EU)
GLM	Generalized Linear Model
HAS	Haute Autorité de Santé (France)
HCPCS	Healthcare Common Procedure Coding System
HIPAA	Health Insurance Portability and Accountability Act (United States)
HR	Hazard Ratio
HTA	Health Technology Assessment
HTAIn	Health Technology Assessment in India
HUI	Health Utilities Index
ICD	International Classification of Diseases
ICER	Incremental Cost-Effectiveness Ratio
ICH	International Council for Harmonisation of Technical Requirements for Pharmaceuticals for Human Use
IDE	Integrated Development Environment

IPCW	Inverse Probability of Censoring Weighting
IPD	Individual Patient Data
IQWiG	Institute for Quality and Efficiency in Health Care (Germany)
ISPOR	International Society for Pharmacoeconomics and Outcomes Research
ITC	Indirect Treatment Comparison
ITT	Intention-to-Treat
JAGS	Just Another Gibbs Sampler
KM	Kaplan-Meier (survival estimator)
LMIC	Low- and Middle-Income Country
LOCF	Last Observation Carried Forward
LTFU	Lost to Follow-Up
MAE	Mean Absolute Error
MAIC	Matching-Adjusted Indirect Comparison
MAR	Missing At Random
MCAR	Missing Completely At Random
MCMC	Markov Chain Monte Carlo
MDR-TB	Multidrug-Resistant Tuberculosis
MI	Multiple Imputation
ML	Machine Learning
ML-NMR	Multilevel Network Meta-Regression
MNAR	Missing Not At Random
NHB	Net Health Benefit
NHS	National Health Service (United Kingdom)
NICE	National Institute for Health and Care Excellence (United Kingdom)
NMA	Network Meta-Analysis
NMB	Net Monetary Benefit
OLS	Ordinary Least Squares
OR	Odds Ratio
PBAC	Pharmaceutical Benefits Advisory Committee (Australia)
PH	Proportional Hazards
PICO	Population, Intervention, Comparator, Outcome

PM-JAY	Pradhan Mantri Jan Arogya Yojana (India)
PMDA	Pharmaceuticals and Medical Devices Agency (Japan)
PPP	Purchasing Power Parity
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
PRISMA-NMA	PRISMA extension for Network Meta-Analyses
PSA	Probabilistic Sensitivity Analysis
QALY	Quality-Adjusted Life Year
RCT	Randomised Controlled Trial
REML	Restricted Maximum Likelihood
RIS	Research Information Systems (bibliographic file format)
ROBINS-I	Risk Of Bias In Non-randomised Studies of Interventions
RR	Risk Ratio (Relative Risk)
RMSE	Root Mean Squared Error
RWE	Real-World Evidence
SAS	Statistical Analysis System (software)
SDTM	Study Data Tabulation Model (CDISC)
SE	Standard Error
SF-6D	Short Form 6-Dimension (health utility instrument)
SG	Standard Gamble
SLR	Systematic Literature Review
SMD	Standardised Mean Difference
SPSS	Statistical Package for the Social Sciences (software)
SQL	Structured Query Language
SROC	Summary Receiver Operating Characteristic
STC	Simulated Treatment Comparison
SUCRA	Surface Under the Cumulative RAnking curve
TB	Tuberculosis
TEAE	Treatment-Emergent Adverse Event
TSD	Technical Support Document (NICE DSU)
TTO	Time Trade-Off
VOI	Value of Information

WHO	World Health Organization
WTP	Willingness-To-Pay
XDR-TB	Extensively Drug-Resistant Tuberculosis
YLD	Years Lived with Disability
YLL	Years of Life Lost

Table of Contents

Foreword iii
Preface iv
Why R for HEOR and HTA iv
Who this book is for iv
How to use this book v
A note on how this edition was produced v
Companion Website, Code Repository and Data Sets vi
Reporting errors vi
Conventions, Package Versions and Reproducibility Notes vii
Typographic conventions vii
Package versions vii
Reproducibility baseline vii
A note on country and agency defaults vii
Abbreviations ix
PART I
Foundations
..... 1
Chapter 1. Introduction to HEOR and HTA 2
What HEOR is and what it is not 2
The HTA ecosystem 4
Evidence needs across the product lifecycle 6
Typical deliverables 7
Why R: the case for open-source and reproducible HEOR 8
Where R fits in the HEOR workflow 9
Chapter 2. Setting Up an R Environment for HEOR 11
2.1 Installing R, RStudio/Positron and Quarto 11
2.2 Project structure for HEOR analyses 12
2.3 Package management with `renv` 13
2.4 Version control with Git and GitHub 15
2.5 Working with Stan, JAGS and C++ toolchains 16
2.6 Containers (Docker) for reproducible models 19
2.7 Recommended package ecosystem (overview map) 21
Chapter 3. R Essentials for Health Economists 24
3.1 Data structures: vectors, lists, data frames and tibbles, arrays and matrices 24
3.2 Data wrangling with `tidyverse` and `data.table` 26
3.3 Functional programming with `purrr` and vectorization 28
3.4 Writing functions, parameter lists and defensive programming 30
3.5 Performance: profiling, `Rcpp`, parallelisation (`future`, `furrr`) 33

3.6 Dates, durations and time-to-event data (lubridate) 36
3.7 Debugging and error handling 37
Chapter 4. Core Concepts in Health Economics (with R Illustrations) 39
4.1 Costs, effects and the opportunity cost of health care 39
4.2 Types of economic evaluation 41
4.3 QALYs, DALYs and life-years 43
4.4 ICERs, net monetary benefit (NMB) and net health benefit (NHB) 45
4.5 Willingness-to-pay thresholds, including country-specific and LMIC thresholds 48
4.6 Perspective, time horizon, discounting and half-cycle correction 50
4.7 Dominance, extended dominance and the efficiency frontier 53
4.8 Coding a first simple CEA in R 55

PART II

Data for HEOR

..... 61
Chapter 5. Data Sources and Data Management 62
5.1 Clinical trial data (CDISC SDTM/ADaM concepts) 62
5.2 Registries, claims, electronic health records and surveys 65
5.3 Published literature as a data source 67
5.4 Importing data: readr, readxl, haven, DBI, APIs 69
Delimited and flat files with readr 70
Spreadsheets with readxl 70
Clinical trial and survey formats with haven 71
Databases with DBI 72
Public and institutional APIs 73
5.5 Data dictionaries, metadata and codebooks 74
What a data dictionary must contain 74
Practical R approaches 75
5.6 Data quality checks (pointblank, assertr, validate) 76
5.7 Data governance, privacy and de-identification 79
Chapter 6. Systematic Literature Review (SLR) Support in R 81
6.1 Search strategy documentation 81
6.2 Deduplication and screening workflows 83
6.3 PRISMA flow diagrams in R 86
6.4 Data extraction templates and extraction QA 89
6.5 Risk of bias visualisation (robvis) 91
6.6 Text mining and ML-assisted screening 94
Chapter 7. Missing Data 96
7.1 Mechanisms: MCAR, MAR, MNAR 96
7.2 Exploring missingness (naniar, visdat, Shiny-based explorers) 98
7.3 Complete-case versus imputation 99
7.4 Multiple imputation (mice, Amelia, missForest) 101
7.5 Sensitivity analysis, including tipping-point and pattern-mixture approaches 104

7.6 Missing data in cost and QALY data 106
7.7 Reporting missing data handling for HTA 108
Chapter 8. Health-Related Quality of Life and Utilities 111
8.1 Generic and disease-specific instruments (EQ-5D, SF-6D, HUI, etc.) 111
8.2 Value sets and scoring in R (eq5d) 113
8.3 Mapping algorithms (cross-walks and regression-based mapping) 114
8.4 Analysing utility data: censoring, bounded distributions, beta regression, mixture models 116
8.5 Utility decrements, age-adjusted utilities and background mortality 119
8.6 Time trade-off, standard gamble and discrete choice experiments 121
Chapter 9. Costs and Resource Use 124
9.1 Identifying, measuring and valuing resources 124
9.2 Unit costs and inflation or currency conversion (PPP adjustments) 126
9.3 Cost data features: skewness, zero-inflation, heavy tails 128
9.4 Regression models for costs: GLMs, two-part models, gamma models 131
Generalised linear models on the cost scale 131
Two-part (hurdle-style) models for zero-inflated costs 132
Smooths and flexible mean functions 133
9.5 Cost-of-illness and micro-costing studies 134
9.6 Handling censored cost data 136

PART III

Statistical Methods for Evidence Synthesis and Analysis

..... 139
Chapter 10. Survival Analysis for Economic Evaluation 140
10.1 Kaplan-Meier and life tables 141
10.2 Parametric models 143
10.3 Flexible and spline-based models 145
10.4 Mixture-cure and non-mixture cure models 147
10.5 Piecewise and landmark models 150
10.6 Model selection: AIC/BIC, visual fit, clinical plausibility 152
10.7 Proportional hazards diagnostics and non-PH alternatives 156
Testing the proportional hazards assumption 156
When PH fails: alternative hazard structures 157
10.8 Covariate-adjusted and treatment-effect-specific models 158
Covariates on the location/scale parameter 158
Covariates on ancillary (shape) parameters 159
Treatment-effect-specific extrapolation 159
10.9 Extrapolation: external data, background mortality, expert elicitation 159
Anchoring with external data 160
Background mortality as a plausibility bound 160

Expert elicitation for structural and tail uncertainty 161
10.10 Competing risks and multi-state models 161
Competing risks: cause-specific vs. subdistribution hazards 162
Data preparation and fitting with `mstate` 162
Parametric extrapolation of transition-specific hazards with `flexsurv` 163
10.11 Reconstructing individual patient data from published Kaplan-Meier curves 163
The Guyot et al. (2012) reconstruction algorithm 164
Using `IPDfromKM` 164
10.12 Reporting extrapolation following NICE DSU TSD 14 and 21 165
Chapter 11. Meta-Analysis 168
11.1 Fixed-effect and random-effects models 168
11.2 Heterogeneity and its quantification 171
11.3 Publication bias and small-study effects 174
11.4 Meta-regression 176
11.5 Meta-analysis of time-to-event outcomes 177
11.6 Diagnostic test accuracy meta-analysis 179
11.7 Meta-analysis of proportions and rates 182
Chapter 12. Indirect Treatment Comparisons and Network Meta-Analysis 184
12.1 Why indirect comparison is needed 184
12.2 Bucher method 186
12.2.1 The algebra of anchored indirect comparison 186
12.2.2 Why the variance adds 186
12.2.3 Worked example: TB regimen comparison 187
12.2.4 Assumptions the Bucher method relies on 188
12.2.5 When Bucher is enough, and when it is not 189
12.3 Frequentist NMA (netmeta package) 189
12.3.1 From anchored pairs to a network 189
12.3.2 Preparing contrast-level data: `pairwise()` 190
12.3.3 Fitting the network: `netmeta()` 190
12.3.4 Visualising the evidence network: `netgraph()` 191
12.3.5 Ranking treatments: `netrank()` 192
12.3.6 Full pairwise results: `netleague()` 193
12.4 Bayesian NMA (gemtc, multinma, bnma) 195
gemtc: the NICE DSU TSD 2 reference implementation 195
multinma: a unified Stan-based alternative 196
bnma: a lighter-weight contrast-based option 197
12.5 Consistency and inconsistency assessment 198
Node-splitting: direct vs indirect evidence on one edge 198
Design-by-treatment interaction: a global alternative 200
12.6 Network meta-regression 201
Mechanics in gemtc 201
Interpreting shrinkage of heterogeneity and inconsistency 202
12.7 NMA of survival outcomes: fractional polynomials and time-varying hazard ratios 205
Why constant hazard ratios are often the wrong assumption 205
The Jansen / TSD 18 fractional polynomial NMA approach, conceptually 205
Single-study building block with flexsurv, and the bespoke network step 206
12.8 Ranking, SUCRA and rankograms 209

12.9 Reporting per NICE DSU TSD 2, ISPOR and PRISMA-NMA 211
Chapter 13. Population-Adjusted Indirect Comparisons 213
13.1 When population adjustment is needed 214
13.2 Matching-adjusted indirect comparison (MAIC) 215
13.3 Simulated treatment comparison (STC) 218
13.4 Multilevel network meta-regression (ML-NMR) 220
13.5 Limitations, assumptions and agency expectations (NICE DSU TSD 18) 223
Chapter 14. Bayesian Methods for HEOR 227
14.1 Bayesian thinking for decision makers 228
14.2 Priors: informative, weakly informative and vague priors 229
14.3 MCMC with Stan, JAGS and brms 231
14.4 Diagnostics: convergence, effective sample size, and posterior predictive checks 235
Why convergence is not optional 235
Effective sample size: why 20,000 draws can be worth 200 236
Posterior predictive checks 237
14.5 Bayesian evidence synthesis as a pipeline 238
The general principle: synthesis produces a distribution, not a number 238
Why this matters more as networks get more complex 238
Where this fits in the overall workflow 239
14.6 Bayesian decision analysis: from posterior to PSA 239
Expected value under a posterior 239
Posterior draws as PSA input: the direct pipeline 240
Decision uncertainty is now a direct statement about the evidence 241
14.7 Expert elicitation methods (SHELF) 242
14.8 Bayesian health economic modelling workflows 244
Chapter 15. Real-World Evidence (RWE) and Causal Inference 247
15.1 Fit-for-purpose RWD and RWE frameworks 248
15.2 Study design: cohort, case-control, target trial emulation 250
15.3 Confounding and bias, with DAGs (`dagitty`, `ggdag`) 252
15.4 Propensity scores: matching, weighting, stratification (`MatchIt`, `WeightIt`, `cobalt`) 253
Matching with MatchIt 254
Weighting with WeightIt 254
Checking balance with cobalt 255
A worked numeric example: IPTW ATE by hand 255
15.5 Doubly robust estimators and g-methods 257
G-methods 258
15.6 Instrumental variables and regression discontinuity 260
Instrumental variables 260
Regression discontinuity 262
15.7 Difference-in-differences and interrupted time series 263
Difference-in-differences (DiD) 263
Interrupted time series (ITS) 265
15.8 Treatment switching and crossover adjustment (RPSFTM, IPCW) 266
15.9 E-values and quantitative bias analysis 268
15.10 OHDSI/OMOP workflows in R 270
15.11 Using external control arms and synthetic control arms 272
Chapter 16. Regression Models for HEOR Data 275

16.1 Generalised linear models 275
16.2 Mixed-effects and hierarchical models 277
16.3 Count data and zero-inflated models 279
16.4 Quantile and ordinal regression 281
16.5 Machine learning for prediction and heterogeneity of treatment effect 282
16.6 Subgroup analysis and effect modification 283

PART IV

Decision-Analytic Modelling

..... 286
Chapter 17. Principles of Decision Modelling 287
17.1 Model conceptualisation and problem scoping 287
17.2 Choosing a model type 289
17.3 Structural assumptions and model diagrams 290
17.4 Parameter taxonomy and distributions 291
17.5 Model documentation and transparency 293
17.6 Modelling workflow in R: inputs -> engine -> outputs -> reporting 295
17.7 Good practice guidelines (ISPOR-SMDM, NICE reference case) 296
Chapter 18. Decision Trees 298
18.1 Structure and expected-value calculations 298
18.2 Building decision trees from scratch in R 300
18.3 Using `rdecision` 302
18.4 Short-term models for screening, diagnostics and acute care 304
18.5 Combining a decision tree with a long-term Markov model 305
Chapter 19. Markov Cohort Models 307
19.1 States, transitions and cycles 307
19.2 Time-homogeneous and time-dependent transition matrices 309
19.3 Building a Markov model in base R (matrix-based approach) 309
19.4 Using heemod, hesim and dampack 310
19.5 Applying costs and QALYs, discounting and half-cycle correction 312
19.6 Tunnel states and state residence time 313
19.7 Age-dependent transitions and background mortality 315
19.8 Probabilistic transition matrices (Dirichlet) and Cholesky decomposition 316
19.9 Calibration and validation 317
19.10 Worked example: a chronic-disease CEA 318
Chapter 20. Partitioned Survival Models 321
20.1 Structure and when it is appropriate 321
20.2 Building PSMs from survival extrapolations 322
20.3 Treatment waning and relative-effect assumptions 323
20.4 Modelling with `hesim`, `flexsurv` and custom code 323
20.5 Limitations compared with state-transition models 324
20.6 Worked example: an oncology CEA 325
Chapter 21. Individual-Level State-Transition Models (Microsimulation) 327

21.1 When microsimulation is needed 327
21.2 Patient heterogeneity and history dependence 328
21.3 Efficient implementation: vectorisation, data.table, parallel computing 330
21.4 First-order versus second-order Monte Carlo uncertainty 331
21.5 Common random numbers 333
21.6 Using `hesim` for individual-level models 333
21.7 Worked example: a screening programme 335
Chapter 22. Discrete Event Simulation and Dynamic Models 338
22.1 Concepts: events, queues, resources and attributes 338
22.2 DES with `simmer` 340
22.3 Competing events and time-to-event sampling 342
22.4 Infectious disease transmission models 344
Compartmental models (SIR/SEIR) with `deSolve` 344
Agent-based models and `EpiModel` 346
Linking transmission models to economic evaluation 347
22.5 Choosing between Markov, DES and dynamic models 347
22.6 Worked example: a TB screening/preventive-therapy intervention model 349
List of Figures 352
List of Tables 354

PART I

Foundations

Chapter 1. Introduction to HEOR and HTA

Learning objectives

- Define Health Economics and Outcomes Research (HEOR) and distinguish it from adjacent disciplines it is often confused with.
- Describe the global Health Technology Assessment (HTA) ecosystem, including the major national and regional agencies and India's HTAIn.
- Map the evidence needs a product generates across its lifecycle, from early pipeline decisions to post-launch re-assessment.
- List the typical analytical deliverables an HEOR team produces and who consumes each one.
- Articulate the case for R as a primary HEOR tool, including where it still lags proprietary alternatives.
- Locate R's role across the end-to-end HEOR workflow that the rest of this book builds on, chapter by chapter.

What HEOR is and what it is not

Health Economics and Outcomes Research (HEOR) is the discipline that generates and synthesises evidence on the clinical, economic and humanistic value of a health intervention, so that payers, clinicians, patients and policymakers can decide whether — and how — it should be funded, prescribed or reimbursed. It sits deliberately between three fields it borrows from but is not reducible to any one of them:

- **Health economics** proper, which studies how scarce health-care resources are allocated, using tools such as welfare economics, demand theory and public finance. HEOR narrows this to intervention- level value questions rather than system-level allocation theory.
- **Biostatistics and epidemiology**, which HEOR leans on heavily (survival analysis, meta-analysis, causal inference — all covered in Part III of this book) but HEOR's output is a decision-relevant value argument, not a hypothesis test in isolation.
- **Health services research**, which studies how care is organised and delivered; HEOR instead asks whether a specific technology is worth paying for, relative to its comparators, in a specific decision context.

A useful operational definition: HEOR produces the evidence base that answers "is this intervention worth it, for whom, compared to what, and under what degree of uncertainty?" Everything else in this book — R code, statistical models, decision trees, Shiny apps — exists only in service of answering that question more rigorously and more reproducibly.

> **Difficult terminology: welfare economics, demand theory, public finance.** These are the three parent disciplines health economics proper draws on, each a large field in its own right. **Welfare economics** studies how to judge whether a policy change makes society better or worse off overall — think of it as the branch of economics concerned with "who wins, who loses, and does the gain outweigh the loss." **Demand theory** models how much of something people buy at different prices (and how that changes as income or preferences shift)

— the supply-and-demand curves from an introductory economics course. **Public finance** studies how governments raise money (taxation) and decide how to spend it across competing priorities (roads, schools, hospitals). HEOR borrows selectively from all three but narrows its focus dramatically: rather than asking "how should a health system allocate its whole budget" (a public-finance question) or "how does demand for healthcare respond to price" (a demand-theory question), HEOR asks a much more contained question — "is **this one** intervention worth paying for, compared to what we'd otherwise do with that money?" You do not need to have studied any of the three to follow this book; they are named here only because an HTA dossier or health-economics textbook may invoke them as HEOR's intellectual ancestry.

Common pitfall. A common conflation, especially among analysts moving into HEOR from general biostatistics or health-services backgrounds, is to treat statistical significance (a p-value below 0.05 for a treatment effect) as equivalent to economic or decision-relevant significance. A statistically significant mean cost difference of $40 per patient across a 20,000-patient budget impact model, or a trial result extrapolated with a parametric survival curve well past the observed follow-up window, can drive a very different reimbursement recommendation than the p-value alone would suggest. HEOR's job is precisely this translation step — from clinical/statistical evidence to decision-relevant value — and it is where most of this book's methodological content concentrates.

Difficult terminology: parametric survival curve (extrapolation). A clinical trial only follows patients for a finite period — typically a few years at most — but a cost-effectiveness model usually needs to know what happens to survival, and therefore to costs and QALYs, over a patient's whole remaining lifetime. "Extrapolation" is the general term for projecting beyond what was actually observed. A **parametric** survival curve is one specific way to do this: instead of just plotting the raw, jagged step-function of who-died-when in the trial (a Kaplan-Meier curve, covered properly in Chapter 10), you fit a smooth mathematical formula — common choices are named after their shape, such as Weibull, log-normal or log-logistic — that describes how the risk of the event changes over time. Once that formula is fitted to the observed trial data, it can be evaluated at any time point you like, including years past the last observed patient, simply by plugging in a larger value for time. The practical analogy is a weather forecast: you have reliable five-day data, but you want a projection for month six, so you fit a model to the pattern you've seen and use it to project forward — the forecast is only as trustworthy as the assumption that the underlying pattern (the shape of the curve) continues to hold, which is exactly why HTA agencies scrutinise which parametric shape was chosen and demand sensitivity analyses across plausible alternatives.

HEOR is also not the same as **market access**, although the two functions collaborate closely in industry settings. Market access is the commercial and regulatory function that negotiates price, reimbursement status and formulary listing; HEOR supplies the evidence (economic models, systematic literature reviews, real-world evidence studies) that market access teams use in those negotiations and that agencies scrutinise in their technical appraisal.

The HTA ecosystem

Health Technology Assessment (HTA) is the formal, usually government-mandated or government-commissioned process by which a jurisdiction decides whether a health technology (a drug, device, diagnostic or procedure) should be covered by public or social insurance funds, and on what terms. HTA agencies are the primary institutional consumers of HEOR output, and the methodological choices in this book are shaped throughout by what these agencies expect to see in a submission.

The major agencies an HEOR analyst will encounter, and the general character of each:

- **NICE** (National Institute for Health and Care Excellence, England and Wales) runs one of the most methodologically prescriptive processes globally, publishing the NICE reference case and a detailed series of Decision Support Unit (DSU) Technical Support Documents (TSDs) that this book cites repeatedly — for survival extrapolation (TSD 14, TSD 21), network meta-analysis (TSD 2) and population-adjusted indirect comparisons (TSD 18), among others.
- **CADTH** (Canada's Drug Agency, historically the Canadian Agency for Drugs and Technologies in Health) performs a broadly similar reimbursement review function for Canada, with its own reference case and reanalysis culture.
- **PBAC** (Pharmaceutical Benefits Advisory Committee, Australia) advises the Australian government on Pharmaceutical Benefits Scheme listing, with a strong tradition of structured uncertainty and stepped economic evaluation.
- **IQWiG** (Institute for Quality and Efficiency in Health Care) and the **G-BA** (Federal Joint Committee) jointly run Germany's early benefit assessment (AMNOG) process, which emphasises patient- relevant outcomes and additional-benefit categorisation ahead of price negotiation.
- **HAS** (Haute Autorité de Santé, France) assesses clinical and economic value (via the Transparency Committee and the Economic and Public Health Evaluation Committee) feeding into price negotiation with the French health insurance system.
- **ICER** (Institute for Clinical and Economic Review, United States) is a non-government, foundation-funded body whose cost-effectiveness reports carry significant influence over US payer and pharmacy-benefit-manager decisions despite having no statutory authority.
- **HTAIn** (Health Technology Assessment in India), housed under the Department of Health Research, Ministry of Health and Family Welfare, is the body most directly relevant to India-based HEOR analysts: it issues an Indian reference case for economic evaluation and informs technology inclusion decisions under schemes such as Ayushman Bharat Pradhan Mantri Jan Arogya Yojana (PM-JAY). HTAIn's methodological guidance is still maturing relative to NICE or CADTH, which in practice means Indian HEOR analyses often triangulate HTAIn's own reference case with ISPOR good-practice guidance and precedent from more established agencies — a pattern Chapter 31 revisits in depth.

Agency	Jurisdiction	Threshold / evaluative approach	Key reference document
NICE	England and Wales	Explicit reference-case, methodologically prescriptive	NICE reference case; DSU TSDs (e.g. TSD 14, TSD 21, TSD 2, TSD 18)
CADTH	Canada	Reference case with reanalysis culture	Guidelines for the Economic Evaluation of Health Technologies
PBAC	Australia	Stepped economic evaluation, strong uncertainty tradition	PBAC Guidelines
IQWiG / G-BA	Germany	Early benefit assessment (AMNOG), patient-relevant outcomes	IQWiG General Methods
HAS	France	Clinical + economic value via dedicated committees	HAS Economic Evaluation Guidance
ICER	United States	Non-government cost-effectiveness reports, no statutory authority	ICER Value Assessment Framework
HTAIn	India	Indian reference case, still maturing	HTAIn Reference Case for Economic Evaluation

Note. Since 2025, EU Joint Clinical Assessment adds a shared clinical-assessment layer; national bodies above still perform their own economic/pricing appraisal.

Table 1: Major HTA agencies/bodies an HEOR analyst routinely encounters, their jurisdiction, general evaluative stance, and a key reference document cited elsewhere in this book.

Agency perspective. Since 2025 the **EU Joint Clinical Assessment** (EU HTA Regulation, in force from January 2025 for oncology and advanced-therapy medicinal products, with wider scope phasing in through 2030) introduces a single European Union-level clinical assessment report that national HTA bodies must then take into account in their own (still separate) appraisals of cost-effectiveness and pricing. This decouples joint **clinical** assessment from still-national **economic** assessment — a structural distinction every analyst preparing a European submission now needs to track, because it changes which evidence goes into which document and on what timeline.

A single technology can therefore face half a dozen or more functionally distinct appraisal processes across markets, each with its own reference case, discount rate convention, preferred comparator set and willingness-to-pay stance — one of the strongest practical arguments for building economic models in a flexible, scriptable, re-parameterisable tool rather than a single fixed spreadsheet.

Evidence needs across the product lifecycle

HEOR evidence generation is not a one-off event triggered by a launch date; it is continuous across a technology's life, and the type of evidence that is credible, available and decision-relevant changes at each stage:

1. **Early pipeline / pre-clinical and Phase I-II**: indication selection, early value proposition modelling, target product profile cost-effectiveness thresholds, often using modelled (not yet observed) efficacy assumptions.
2. **Phase III / pre-launch**: pivotal trial-based economic models, systematic literature reviews and network meta-analyses to position the asset against comparators, early health-related quality-of-life (utility) evidence collection alongside the trial.
3. **Launch / initial HTA submission**: the full value dossier — economic model, budget impact model, systematic review, and (where relevant) indirect treatment comparison — submitted to one or more HTA agencies and payers.
4. **Post-launch**: real-world evidence studies to confirm or update trial-based assumptions (particularly long-term survival extrapolation and real-world adherence/persistence), responses to evidence review group critiques, managed entry agreement data collection.
5. **Re-assessment / loss of exclusivity**: re-appraisal as new comparators enter, as biosimilars or generics launch, or as a scheduled HTA review cycle requires updated modelling.

Lifecycle stage	Characteristic evidence generated	R workflow emphasis
Early pipeline / Phase I-II	Indication selection, early value proposition, target product profile thresholds	Quick decision-tree/ Markov prototypes (Ch. 18-19)
Phase III / pre-launch	Pivotal trial-based models, SLRs/ NMAs, early utility collection	Evidence synthesis inputs (Part III)
Launch / initial HTA submission	Full value dossier: economic model, BIM, SLR, ITC	Validated, documented, PSA-ready models (Ch. 25, 27)
Post-launch	RWE studies, ERG critique responses, managed entry agreement data	RWE pipelines, causal inference (Ch. 15)
Re-assessment / LOE	Re-appraisal against new comparators, biosimilars/generics	Updated modelling on existing codebase

Table 2: Evidence generation and typical analytical deliverables by product lifecycle stage.

This sequencing matters for a very practical reason covered throughout this book: at each stage the **type** of R workflow needed differs — early-stage work favours quick, assumption-heavy decision-tree or simple Markov prototypes (Chapters 18-19); launch submissions demand fully validated, documented, PSA-ready models (Chapters 25, 27); post-launch work leans on real-world data pipelines and causal inference methods (Chapter 15) that a pre-launch team may never have needed.

Typical deliverables

An HEOR function, whether inside a pharmaceutical company, a consultancy, a government technical unit or an academic health economics group, typically produces some combination of:

- **Global value dossiers** — the master evidence package (AMCP format or similar) from which country-specific submissions are adapted.
- **Economic models** — cost-effectiveness or cost-utility models (decision trees, Markov models, partitioned survival models, microsimulations — Part IV of this book) that estimate incremental cost per QALY, DALY averted, or other outcome of interest.
- **Budget impact models** — separate from cost-effectiveness models, these forecast the net financial impact on a payer's budget over a defined horizon (Chapter 23), and are frequently the deciding document in resource-constrained settings even when the cost-effectiveness case is favourable.
- **Systematic literature reviews and network meta-analyses** — the evidence synthesis backbone for both the clinical case and the economic model's comparative-effectiveness inputs (Chapters 6, 11-13).
- **Real-world evidence (RWE) studies** — observational analyses using registries, claims or electronic health record data to answer questions trials cannot (Chapter 15).
- **Payer submissions and technical engagement responses** — the formatted documents and rebuttals an agency's evidence review process demands (Chapters 28, 31).

> **Difficult terminology: microsimulation.** Most economic models described so far implicitly track a **cohort** — an average, representative group of patients moving through health states together, where everyone in the "progressed disease" state at year 3 is treated as identical for costing and QALY purposes. A microsimulation instead tracks many individual, simulated patients one at a time, each with their own randomly drawn characteristics and their own randomly drawn sequence of events (does **this** particular patient progress this cycle, or not, based on a random draw against their personal risk?). Run enough individual patients through the model and average their outcomes, and you recover the same population-level answer a cohort model would give — but along the way you also capture patient-to-patient variability (some patients do much better or worse than the average) and can model things a cohort average cannot, such as a patient's own history affecting their future risk. A useful analogy is a flight simulator running thousands of individual passenger journeys with their own delays and connections, rather than reporting only the airline's average on-time performance: both describe the same system, but the individual-level version answers questions ("what fraction of passengers miss their connection?") the average alone cannot.

Tuberculosis programme data quality and analysis for India's National TB Elimination Programme, and workforce-training initiatives built around it, sit most naturally against the budget impact, programme- cost and dynamic/infectious-disease modelling deliverables (Chapters 22-23, 32.7), which this book returns to with TB-specific worked examples wherever the general method permits it.

Why R: the case for open-source and reproducible HEOR

For much of HEOR's history, the dominant modelling tools were Excel (ubiquitous, transparent to reviewers cell-by-cell, but fragile and hard to validate at scale), TreeAge Pro (a widely used commercial decision-modelling GUI), and SAS (entrenched in regulated clinical and RWE pipelines, particularly in the US). R has become a credible, and increasingly preferred, alternative for several concrete reasons:

- **Reproducibility by construction.** An R script, paired with `renv` package-version locking (Chapter 2) and version control (Git), gives an exact, re-runnable record of every assumption and transformation — something a large interlinked Excel workbook with manual copy-paste steps cannot reliably guarantee.
- **Statistical depth.** R's package ecosystem for survival analysis (`flexsurv`), meta-analysis (`meta`, `metafor`, `netmeta`), Bayesian modelling (`brms`, Stan) and causal inference (`MatchIt`, `WeightIt`) is both broader and more actively maintained by the academic biostatistics community than equivalent capability inside TreeAge or Excel, and typically ahead of what is readily available in SAS procedures for newer methods.
- **Cost.** R, RStudio/Positron and essentially the entire package ecosystem used in this book are free and open source, compared to substantial per-seat licensing for TreeAge or SAS — a material consideration for government technical units, academic groups and LMIC HTA bodies including HTAIn-adjacent work.
- **Transparency for reviewers.** A well-structured R project (Chapter 2) lets an external reviewer or evidence review group trace every number in a submitted model back to a specific line of code and a specific input — the standard increasingly expected by NICE's DSU and by open-science-minded reviewers more broadly.
- **Interoperability with reporting.** Quarto and R Markdown (Chapter 28) let the same codebase that runs the model also generate the Word, PDF, HTML or PowerPoint submission documents, removing an entire category of transcription error between "the model" and "the report."

None of this means R is strictly superior in every respect. TreeAge remains attractive for teams who need a point-and-click interface for non-programming stakeholders and fast prototyping without a coding layer; SAS retains deep institutional entrenchment in regulated clinical trial pipelines where validated, audited SAS macros already exist; and Python has a comparably open-source, reproducible profile to R, with particular strength in machine learning and large-scale data engineering (Chapter 34 returns to where Python-R interoperation and newer ML methods fit into an HEOR R workflow).

> **HEOR in practice.** Several HTA agencies now explicitly accept, and in some cases prefer, submissions built with open-source tools, provided the model is adequately documented and validated. NICE's DSU has published R-based worked examples for survival extrapolation (TSD 21) and other technical guidance; several published NICE single-technology appraisals have used R-built partitioned survival and Markov models rather than Excel or TreeAge. The deciding factor for agency acceptance is not the tool itself but whether the model satisfies the agency's validation and transparency expectations (Chapter 27) — R makes that easier to demonstrate, but does not make it automatic.

Where R fits in the HEOR workflow

The remainder of this book is organised around a simple mental model of the HEOR workflow, and each part maps onto a stage of it:

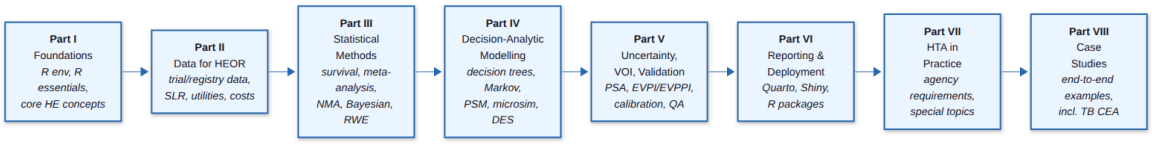


Figure 1: The book's eight-part structure as a cumulative HEOR workflow. Each part assumes fluency with everything upstream of it.

1. **Foundations** (Part I, this part): the R environment, R language essentials, and the core health-economic concepts (QALYs, ICERs, discounting) that every later chapter assumes.
2. **Data** (Part II): getting trial, registry, claims, literature, utility and cost data into a clean, analysis-ready state — the unglamorous but decisive step that determines whether everything downstream is trustworthy.
3. **Statistical methods** (Part III): survival analysis, meta-analysis, indirect comparisons, Bayesian methods and causal inference — the methods that convert raw data into the clinical and comparative- effectiveness parameters a model needs.
4. **Decision-analytic modelling** (Part IV): the model structures themselves — decision trees, Markov cohort models, partitioned survival models, microsimulation, discrete event simulation, budget impact and distributional/extended CEA — that combine those parameters into decision-relevant outputs.
5. **Uncertainty, VOI and validation** (Part V): quantifying how confident we should be in a model's conclusions, and demonstrating that confidence is warranted through calibration and validation.
6. **Reporting, communication and deployment** (Part VI): turning a validated model into a submission-ready, reviewable, and where useful interactive (Shiny) artefact.
7. **HTA in practice** (Part VII): agency-specific requirements, special therapeutic-area considerations, and patient-preference methods that do not fit cleanly into the generic workflow above.
8. **Case studies** (Part VIII): end-to-end worked examples, including a TB programme cost-effectiveness analysis for an LMIC setting, tying every preceding part together in one script-driven project.

Difficult terminology: discrete event simulation. A Markov cohort model (and, usually, a microsimulation built on top of one) advances time in fixed, regular steps — a "cycle" — typically a month or a year, checking at every single cycle boundary whether each patient has moved to a new health state. A discrete event simulation instead advances time irregularly, jumping straight to whenever the **next** thing actually happens (the next relapse, hospital admission, or death, whichever comes first for whichever patient), rather than re-checking everyone on a fixed clock. This matters most when events are rare and irregularly timed, or when the model needs to represent shared, limited resources — a fixed number of hospital beds or specialist appointment slots that patients queue for — something a simple fixed-cycle cohort model struggles to represent naturally. Think of the difference between a factory manager who inspects every machine on the hour regardless of what is happening (a fixed-cycle approach) versus one who is alerted the instant any machine actually breaks down or

finishes a job (an event-driven approach); the second is more efficient and more realistic when events are sporadic rather than evenly spaced.

Difficult terminology: distributional / extended cost-effectiveness analysis. A standard cost-effectiveness analysis produces one headline number — the population-average cost per QALY gained — which can quietly hide very different results for different groups within that population. A distributional (or "extended") cost-effectiveness analysis deliberately breaks that average apart, reporting costs, health gains and sometimes financial-risk-protection effects separately by subgroup: by income quintile, by region, by disease severity, or by insurance status, for example. The point is to let a decision-maker see **who** gains and **who** pays, not just whether the average trade-off looks favourable — a programme can look excellent on average while concentrating nearly all of its benefit in one already-advantaged group, which a single pooled ICER would never reveal. A simple analogy: a company's average employee salary can rise even if every individual salary stayed flat, just because a few new high earners joined — the average conceals the distribution. Extended CEA is the health-economics equivalent of insisting on seeing the full salary breakdown by group, not just the company-wide average, before judging whether a policy is actually equitable.

Each part builds cumulatively on R skills and HEOR concepts introduced earlier — by the time this book reaches decision-analytic modelling in Part IV, it assumes fluency with the data-wrangling and statistical foundations from Parts I-III, rather than re-explaining them.

Chapter 2. Setting Up an R Environment for HEOR

Learning objectives

- Install and configure R, an IDE (RStudio or Positron), and Quarto for reproducible HEOR reporting.
- Structure a new HEOR analysis project so that raw data, code, and outputs stay cleanly separated and auditable.
- Use `renv` to pin exact package versions so a model can be reproduced byte-for-byte years after submission.
- Set up Git/GitHub version control as both an audit trail and a lightweight QA mechanism mirroring agency technical review.
- Install the C++ toolchain required for Bayesian modelling with Stan/JAGS, and avoid the installation pitfalls that most commonly block analysts moving into Bayesian HEOR work.
- Understand when Docker is worth the overhead for a submission-critical model, versus when `renv` alone is sufficient.
- Navigate the R package ecosystem for HEOR by task area, with forward references to the chapters that cover each package in depth.

2.1 Installing R, RStudio/Positron and Quarto

Every HEOR analysis in this book rests on three pieces of software: R itself, an integrated development environment (IDE), and Quarto for turning analysis into submission-ready documents. Installing these correctly, and understanding the role each plays, prevents a surprising fraction of the reproducibility headaches that plague health-economic modelling teams.

R is distributed through the Comprehensive R Archive Network (CRAN) at `cran.r-project.org`. On Windows, download and run the `.exe` installer; on macOS, use the `.pkg` installer matching your chip architecture (Apple Silicon vs Intel); on Linux, prefer your distribution's package manager or the CRAN apt/yum repositories over a source build, since these handle dependency resolution and security updates automatically. Install the latest patch release of the current major R version unless a specific project mandates otherwise — version pinning is handled later, at the project level, by `renv` (Section 2.3), not by freezing your system R.

For years, RStudio Desktop was the default IDE for R users, and it remains a fully capable, mature choice: integrated console, environment pane, plotting pane, and tight Quarto/R Markdown support. Posit (RStudio's parent company) has since released **Positron**, a VS Code–based IDE built on the same open extensibility model as Visual Studio Code, but with first-class R and Python data-science support built in rather than bolted on. For HEOR work specifically, Positron is gaining traction for several concrete reasons:

- Many HTA submissions now combine R (statistical/economic modelling) with Python (machine-learning-based survival extrapolation, web-scraping literature data, or interfacing with external

APIs) in the same project — Positron handles both languages natively in one workspace, without the friction of RStudio's R-first design.
- It inherits the entire VS Code extension ecosystem: Git integration, remote development over SSH, Docker/dev-container support, and GitHub Copilot-style tooling, which matter increasingly for teams working under version control and CI pipelines.
- Its data viewer, variable inspector, and plot pane are drop-in equivalents to RStudio's, so the learning curve for an RStudio user is shallow.

Positron is still younger than RStudio and a handful of RStudio-specific add-ins have no Positron equivalent yet, so this book treats either IDE as acceptable; screenshots and keybinding references favour RStudio terminology where the two diverge, but all code runs identically in both.

Quarto is Posit's successor to R Markdown and ships either bundled with RStudio/Positron or as a standalone CLI installable from `quarto.org`. It compiles `.qmd` documents mixing prose and executable R (or Python, Julia, Observable JS) code chunks into HTML, PDF, Word, or slide formats — critical for HEOR, where a single analysis often must be rendered as an internal technical report, a submission-ready Word document for a HTA body, and a results dashboard, all from one source file. Confirm your installation with:

```
quarto::quarto_version()
```

> **HEOR in practice.** Standardise on one Quarto version and one R version per **project**, not per analyst's machine. Record both in your project's README and, as Section 2.3 shows, let `renv` pin the R package versions precisely — the Quarto and R engine versions themselves should be noted explicitly since `renv` does not manage them.

2.2 Project structure for HEOR analyses

A consistent folder structure is not bureaucratic overhead — it is the single cheapest thing you can do to make an analysis auditable, handoff-ready, and defensible under HTA scrutiny. HEOR deliverables are frequently revisited months or years after initial submission (in response to reviewer queries, re-submissions, or country adaptations), often by someone other than the original analyst. A predictable layout means that person — possibly future you — can find the raw inputs, the code that transformed them, and the exact outputs without reverse-engineering a one-off script.

A structure that has proven robust across cost-effectiveness models, budget-impact analyses, and systematic literature reviews looks like this:

```
my-heor-project/
├── data-raw/        # original, untouched source files (CSV exports, trial
extracts)
├── data/            # cleaned, analysis-ready datasets (derived, never hand-
edited)
├── R/               # reusable functions (not top-level analysis scripts)
├── analysis/        # scripts/Quarto docs that run the actual analysis
├── output/          # figures, tables, rendered reports — fully disposable/
regenerable
├── renv/            # renv's private package library (Section 2.3)
├── renv.lock        # the package manifest
└── my-heor-project.Rproj
```

The rationale behind each separation matters more than the names themselves:

- **`data-raw/` is immutable.** Nothing in this folder is ever edited by hand or overwritten by a script. If a trial extract or vendor file changes, a new file (or new version) is added — the folder is a permanent, auditable record of what came in. Cleaning logic lives in code under `R/`, not in the data itself.
- **`data/` is derived, not authoritative.** Everything here should be fully reproducible by re-running the cleaning scripts against `data-raw/`. If `data/` were deleted entirely, the project should be able to regenerate it byte-for-byte.
- **`R/` separates logic from execution.** Functions used by more than one analysis script (cost calculations, utility mapping, plotting themes) belong here, loaded via `source()` or, for larger projects, packaged with `devtools::load_all()`. This avoids copy-pasted logic drifting out of sync across scripts.
- **`analysis/` contains the actual narrative** — Quarto documents or scripts that call functions from `R/` against data from `data/` and produce results. Each file here should, in principle, run top-to-bottom on a clean machine and reproduce its own output.
- **`output/` is disposable.** Nothing here is a source of truth; it is always regenerable by re-running `analysis/`. Treating output as disposable is what keeps the project honest — if you can't regenerate a table from scratch, something in the pipeline is undocumented.

A convenient starting scaffold is `usethis::create_project()`, which creates the `.Rproj` file, a sensible `.gitignore`, and an R-project-aware working directory in one call:

```
usethis::create_project("~/projects/my-heor-project")
```

From there, manually add the `data-raw/`, `data/`, `analysis/`, and `output/` directories (a small helper function wrapping `dir.create()` calls is worth writing once and reusing across projects).

> **Common pitfall.** Do not defer this structure until "the analysis is more final." The most common failure mode is a single sprawling script that reads a spreadsheet, cleans it, models it, and plots it in one pass — impossible to audit, and nearly impossible to hand off. Separating concerns from the first commit costs minutes; retrofitting it onto a 2,000-line script before a submission deadline costs days.

2.3 Package management with `renv`

R's package ecosystem evolves continuously, and HEOR analyses routinely have a shelf life of years — a submitted cost-effectiveness model may be queried, re-analysed, or audited long after its packages have moved on to incompatible new versions. Without intervention, simply running `install.packages("survival")` updates the package for **every** project on your machine, silently changing numerical behaviour (confidence interval methods, extrapolation defaults, random number handling) in ways that can alter results computed from the exact same code. `renv` solves this by giving every project its own private, isolated package library and a lockfile recording exact versions.

Initialise `renv` once per project, from the project root:

```
renv::init()
```

This scans your scripts and Quarto documents for `library()`/`require()` calls, builds a private project library under `renv/library/`, and writes an initial `renv.lock` file. From that point, package installs made inside the project (`install.packages()`, `remotes::install_github()`, etc.) are captured in that private library and do not affect your system-wide R installation or other projects.

As the analysis develops and you add or update packages, record the current state with:

```
renv::snapshot()
```

`snapshot()` inspects every package actually used in the project and writes their exact versions — plus, where resolvable, their source (CRAN, Bioconductor, GitHub, with commit hash) — into `renv.lock`. Conversely, to recreate the **exact** environment the lockfile describes, on a new machine or a colleague's laptop, run:

```
renv::restore()
```

`restore()` reads `renv.lock` and installs precisely those package versions into the project's private library, regardless of what is currently installed elsewhere on the system.

The `renv.lock` file itself is plain JSON, version-controllable alongside your code, and human-readable: it records the R version the project was built under, the package repositories consulted, and for each package its name, version, and source hash. A fragment looks like:

```
# (illustrative structure, not a function call)
# {
#   "R": { "Version": "4.3.2" },
#   "Packages": {
#     "survival": { "Package": "survival", "Version": "3.5-7",
#                   "Source": "Repository", "Repository": "CRAN" }
#   }
# }
```

Agency perspective. NICE's Decision Support Unit has published explicit technical support documents (the DSU TSD series referenced throughout this book, e.g. TSD 14 and TSD 21 for survival extrapolation in Chapter 10) that assume a reviewer can inspect and re-run the submitted model's code — NICE's own technology appraisal process guide requires companies to submit the executable model itself, not only a written description of methods, specifically so the Evidence Review Group can verify calculations independently. CADTH's Guidelines for the Economic Evaluation of Health Technologies similarly requires an electronic copy of the model. Neither agency mandates R or `renv` specifically, but both mandate that **whatever** tool is used be inspectable and re-runnable by the reviewing team — which is precisely the gap `renv.lock` closes for an R-based submission. Committing `renv.lock` alongside your analysis code is what makes "re-run this years later" operationally possible, not just a nice-to-have.

Common pitfall. The most common `renv` mistake is treating it as an optional add-on for "later," adopted only once a project is already underway. By the time collaborators have installed ad-hoc package versions by hand, `renv::init()` inherits an already-inconsistent environment, and reconciling it is far more work than starting clean. Run `renv::init()` as the **first** command in a new HEOR project, before a single `install.packages()` call — the

discipline costs nothing up front and prevents an entire category of "it worked on my machine" failures later.

Finally, note that `renv` manages R packages, not the R interpreter version, system libraries, or Quarto itself — pair it with the version documentation practice recommended in Section 2.1 for full environment reproducibility.

2.4 Version control with Git and GitHub

Health economic models are not static deliverables; they are living software artifacts that get re-run dozens of times as evidence changes, reviewers request scenario analyses, and submission deadlines force last-minute parameter updates. Without version control, "the model" quickly fragments into a folder of files named `markov_model_v3_final_FINAL_usethisone.R`, with no record of which assumptions changed between versions or who approved them. Git solves this by tracking every change as a discrete, attributable commit, and GitHub (or an equivalent host) adds the collaborative layer — issues, pull requests, and review history — that turns a personal habit into a team process.

For HEOR work specifically, version control earns its keep in three ways. First, **auditability**: reimbursement submissions to bodies such as NICE, CADTH, or ICER (the US nonprofit, not the cost-effectiveness ratio) are frequently re-opened months later when a reviewer asks "what if the discount rate were 1.5% instead of 3.5%?" A clean commit history lets you answer "what changed between the submitted model and this scenario" with `git diff`, rather than by memory. Second, **branching for scenario analysis**: each deterministic or structural sensitivity analysis — a different time horizon, an alternative comparator, a revised utility source — belongs on its own branch, built from the same validated base model. This keeps the base case stable while exploratory work proceeds in parallel, and it makes clear which scenarios were ever merged back into the primary analysis. Third, **pull requests as quality control**: requiring every change to a submission-track model to go through a PR — with a named reviewer checking the diff line by line — mirrors the technical review step that payer bodies themselves apply to submitted models. It catches hard-coded values, accidental formula changes, and unreviewed shortcuts before they reach a regulator.

HEOR in practice. Treat your Git workflow as a lightweight mirror of agency technical review: no commit to the `main`/`submission` branch without an approving PR from a second analyst. Tag the exact commit hash used to produce the submitted report's figures (`git tag submission-v1 <hash>`) so the evidentiary model can always be reconstructed byte-for-byte, even after further development continues on other branches.

The basic workflow for a new HEOR project is unremarkable but worth stating explicitly, since many analysts come from a non-software background:

```
git init
git add .
git commit -m "Initial commit: project scaffold and renv lockfile"
git checkout -b scenario/alt-discount-rate
# ... make changes ...
```

```
git add R/run_model.R
git commit -m "Add 1.5% discount rate scenario per reviewer request"
git push origin scenario/alt-discount-rate
# open a pull request on GitHub for review before merging
```

GitHub's value beyond raw Git is largely social: it gives external statisticians, academic collaborators, or a payer's technical reviewers a place to comment on specific lines of code, request changes, and approve a merge without needing filesystem access to your machine. Private repositories (or an enterprise GitHub/GitLab instance, which many HTA consultancies and pharmaceutical sponsors require for data governance reasons) support this collaboration while keeping the codebase restricted to named contributors.

A `.gitignore` file needs particular care in this domain. Renv's `renv.lock` file should always be committed — it is the machine-readable record of package versions that makes the environment reproducible and is small, plain-text JSON. By contrast, the `renv/library` folder (the actual installed packages) and any `.Rproj.user` or `.Rhistory` files should be excluded, since they are large, machine-specific, and regenerable from the lockfile.

> **Common pitfall.** Never commit real patient-level data, even temporarily, even on a private repository. A patient-level dataset pulled into a project folder for a quick test and then "removed" in a later commit still exists in Git's history — `git log` and `git show` can retrieve it from any prior commit unless the history itself is rewritten and force-pushed, which is destructive and easy to get wrong. Add `data/raw/`, `*.csv` (or more targeted patterns for your actual raw-data paths), and any patient-level extracts to `.gitignore` from the very first commit, before any such file is ever staged. If synthetic or de-identified data is needed for the repository, generate it programmatically and label it unambiguously as synthetic in the filename and header.

2.5 Working with Stan, JAGS and C++ toolchains

From Chapter 14 onward, this book moves into Bayesian methods — probabilistic sensitivity analysis built on posterior distributions, network meta-analysis, and survival extrapolation models that borrow strength across data sources via hierarchical structure. These techniques are typically fit not with a built-in R function but with a dedicated probabilistic programming language: Stan or, in older or legacy codebases, JAGS (Just Another Gibbs Sampler). Both let you write the statistical model directly as a set of distributional statements and delegate the hard work of sampling from the posterior to an efficient external sampler — Hamiltonian Monte Carlo (via the No-U-Turn Sampler) in Stan's case, and Gibbs sampling in JAGS's case.

> **Difficult terminology: posterior distribution.** In Bayesian statistics, you start with a **prior** — your belief about a parameter (say, a treatment's true effect size) before looking at the current data, which might be deliberately vague ("anywhere from a large harm to a large benefit is plausible") or informed by previous studies. You then combine that prior with the data actually observed in your trial or dataset, using the rules of probability, to get an updated belief: the **posterior distribution**. It is not a single number but a full distribution — a range of plausible values for the parameter, each with an associated probability — because Bayesian

inference treats the parameter itself as uncertain and quantifies exactly how uncertain, before and after seeing data. A simple analogy: if your prior is "this coin is probably close to fair" and you then flip it 20 times and see 15 heads, your posterior shifts the diagnosis-of-fairness towards a biased coin, by an amount that depends on how strong your prior was and how much data you collected. The posterior is usually too mathematically complex to compute an exact formula for in any realistic health-economic model, which is exactly why dedicated samplers like Stan and JAGS exist — they approximate the posterior by drawing thousands of representative random samples from it rather than solving it analytically.

Difficult terminology: hierarchical structure ("borrowing strength"). A hierarchical (or "multi-level") model structure is used when your data naturally comes in related groups — for example, several different trials of similar drugs, or several hospitals reporting the same outcome. Rather than treating each group as completely separate (ignoring that they are related) or pooling all the data together as if the groups didn't exist (ignoring that they might genuinely differ), a hierarchical model does something in between: it estimates a group-specific effect for each group, but constrains those group estimates to be drawn from a shared, overarching distribution that the model also estimates. The practical consequence is "borrowing strength" — a group with very little data of its own (a small trial, or a hospital with few cases) gets pulled gently towards the overall average, because the model recognises it as one draw from the same underlying family as the data-rich groups, instead of relying purely on its own sparse, noisy numbers. A classroom analogy: if you want to estimate how good each of thirty schools' maths teaching is, but some schools only have five students' scores while others have five hundred, a hierarchical model lets the small schools' estimates "borrow" precision from the overall district pattern, rather than wildly over- or under-stating how good that one small school's teaching really is based on five noisy data points alone.

Difficult terminology: Hamiltonian Monte Carlo / No-U-Turn Sampler (NUTS). Sampling from a posterior distribution (see the box above) means drawing many representative random values from it so you can summarise what the model believes, once data has been accounted for. Hamiltonian Monte Carlo (HMC) is a particular, physics-inspired algorithm for doing this efficiently in models with many parameters: rather than wandering around the parameter space in small, nearly random steps (which is slow and can get stuck), it simulates a "puck" sliding across the probability landscape as if under gravity and momentum, following smooth, informed trajectories that explore the most probable regions much faster. The one dial every HMC implementation needs is how far each simulated trajectory should run before stopping; get this wrong and the sampler either wastes time retracing its own path or stops too early to be useful. The No-U-Turn Sampler (NUTS) is the clever auto-tuning addition that solves this: it monitors the simulated trajectory and automatically stops it the moment it starts curving back on itself (making a "U-turn"), so the step length is chosen automatically rather than by manual trial and error. You do not need to understand the physics to use Stan, which implements HMC/NUTS under the hood — the practical takeaway is simply that this is why Stan is usually faster and more reliable than older samplers for complex Bayesian HEOR models, with little manual tuning required.

> **Difficult terminology: Gibbs sampling.** Gibbs sampling is an older, simpler approach to the same sampling problem that Hamiltonian Monte Carlo solves (see above): drawing representative random values from a posterior distribution with several unknown parameters at once. Instead of moving all parameters together, it updates them one at a time, in turn: hold every other parameter fixed at its current value, draw a new random value for just one parameter conditional on those fixed values, then move on to the next parameter and repeat, cycling through all of them over and over. Over many such cycles, the sequence of values produced converges to representative draws from the full joint posterior. It is conceptually simple to implement and was, for a long time, the workhorse behind BUGS and JAGS-style Bayesian software in health economics and epidemiology, but it can be noticeably slower to explore complex models than Hamiltonian Monte Carlo/NUTS, particularly when parameters are strongly correlated with each other — one reason newer R-based Bayesian HEOR work has shifted increasingly towards Stan.

The practical consequence for an R-based HEOR workflow is that Stan model code is compiled to C++ before it runs. The R packages `rstan` and `cmdstanr` are interfaces that generate this C++, hand it to a compiler, and then call the resulting executable — they are not pure-R implementations. This means your machine needs a working C++ toolchain, which is rarely installed by default and is the single most common installation obstacle analysts moving from base R or `survival`/`flexsurv`-style modelling report when they first try Bayesian HEOR work.

The toolchain requirement differs by operating system:

- **Windows**: requires Rtools, a separate installer (matched to your R version, e.g. Rtools42 for R 4.2.x) that provides a `gcc`-based compiler and the Unix-like build utilities R's compilation toolchain expects. It is not installed by running `install.packages()`; it is a standalone download from CRAN's Rtools page and must be on the system `PATH`.
- **macOS**: requires the Xcode Command Line Tools, installed with `xcode-select --install` from a terminal, which provides `clang` and related build tools without requiring the full Xcode IDE.
- **Linux**: requires the standard GNU build toolchain, typically installed via the distribution's package manager, e.g. `sudo apt-get install build-essential` on Debian/Ubuntu, which supplies `gcc`, `g++`, and `make`.

```
# macOS
xcode-select --install

# Debian/Ubuntu
sudo apt-get update && sudo apt-get install build-essential
```

Once the toolchain is present, `cmdstanr` is the interface this book recommends over the older `rstan`. `cmdstanr` is a lightweight wrapper around CmdStan, the Stan command-line interface, and is updated in step with the core Stan language; `rstan` bundles its own, often older, copy of Stan directly into the R package and has historically lagged behind in supporting newer Stan features and compiler versions. Installing `cmdstanr` additionally requires a one-time call to `cmdstanr::install_cmdstan()`, which downloads and compiles CmdStan itself — a step that can take several minutes and depends on the same toolchain described above.

```
install.packages("cmdstanr",
  repos = c("https://mc-stan.org/r-packages/", getOption("repos")))
```

```
cmdstanr::check_cmdstan_toolchain()
cmdstanr::install_cmdstan()
```

JAGS, used in some legacy HEOR network meta-analysis scripts via the `rjags` or `R2jags` packages, is architecturally different: it is a standalone program installed outside of R (from SourceForge, or via the distribution's package manager on Linux) rather than a C++ code generator, so it does not share Stan's compiler requirement — but it does require that the JAGS binary itself be installed and discoverable before `rjags` will load.

Agency perspective. If a submission package to a payer body includes Bayesian NMA or survival extrapolation results, document the exact Stan/CmdStan or JAGS version used, alongside the R package versions captured in `renv.lock`. NICE DSU TSD 2 (the technical support document governing network meta-analysis methodology, covered in depth in Chapter 12) explicitly requires reporting of the software and convergence diagnostics used to fit a Bayesian NMA — not just the point estimates — because posterior samples from MCMC are not guaranteed bit-identical across sampler versions even with a fixed seed, and a reviewer reproducing your analysis on a different machine needs to know which toolchain generated the submitted numbers.

Common pitfall. A frequent failure mode is installing `rstan` or `cmdstanr` successfully but hitting cryptic compiler errors only when the first model is actually compiled — because `install.packages()` succeeds without ever invoking the C++ compiler, while fitting a model does. Run `cmdstanr::check_cmdstan_toolchain()` (or `pkgbuild::has_build_tools(debug = TRUE)` for `rstan`) immediately after installing the toolchain, before writing any model code, so toolchain problems are caught early rather than mid-analysis under deadline pressure.

2.6 Containers (Docker) for reproducible models

`renv` (Section 2.3) solves a specific problem well: it pins the versions of R packages and records where they came from. But it solves only part of the reproducibility problem. The R packages installed by `renv::restore()` still have to compile or link against something — a particular R interpreter version, system libraries such as `libcurl`, `zlib`, or `libxml2`, a BLAS/LAPACK implementation for numerical linear algebra, fonts for PDF rendering, and an operating system that provides all of the above in a given configuration. None of this is captured by a `renv.lock` file. If a reviewer, a regulator's technical team, or a colleague two years from now tries to rebuild your analysis on a different Linux distribution, a different R minor version, or a Mac with Apple Silicon instead of an Intel chip, subtle numerical differences can appear even with **identical** package versions — different BLAS implementations alone can shift floating-point results in the eighth decimal place, which is usually immaterial for an ICER but can matter in a bootstrap or Markov trace with many iterations.

Docker addresses this by pinning the entire execution environment: the base operating system, the system libraries, the R version itself, and then the R packages on top. A container built from a `Dockerfile` today will — barring extreme edge cases such as architecture changes — produce bit-for-bit the same computational environment years later, regardless of what has changed on the host machine or in CRAN's current package index.

A minimal, HEOR-appropriate `Dockerfile` built on the Rocker Project's `rocker/r-ver` image, which pins an exact R version, looks like this:

```
FROM rocker/r-ver:4.3.2

# System libraries needed by common HEOR packages
# (XML, curl, and SSL for reporting/network packages;
# a Fortran compiler for survival/optimisation routines)
RUN apt-get update && apt-get install -y --no-install-recommends \
    libxml2-dev \
    libcurl4-openssl-dev \
    libssl-dev \
    gfortran \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /model

# Bring in the renv lockfile and restore from it
COPY renv.lock renv.lock
RUN Rscript -e "install.packages('renv'); renv::restore()"

COPY . .

CMD ["Rscript", "run_model.R"]
```

This pairs the two tools rather than treating them as alternatives: `renv.lock` still governs exactly which package versions are installed, while the `Dockerfile` governs everything renv cannot see.

> **HEOR in practice.** A practical rule of thumb for submission-critical models (NICE, ICER, CADTH, or similar HTA body submissions): if the deliverable includes an executable model that a third party is expected to independently re-run and reproduce your base-case and sensitivity-analysis outputs, build and ship a Docker image alongside the R code, not instead of it. NICE's technology appraisal process explicitly provides for the Evidence Review Group to request the submitted model and re-run it during the technical engagement stage — a containerized model removes "it depends on my machine's configuration" as a possible point of failure during that review. Tag the image with the model version and archive it (e.g. as a `.tar` export via `docker save`, or pushed to a registry with access controls) with the rest of the submission package. "It ran on my machine" is not an acceptable reproducibility standard when a cost-effectiveness claim underpins a reimbursement decision.

Docker is not always worth the overhead, however. For exploratory or internal analysis — scoping work, early-stage model structuring, or a one-off piece of academic research where the main deliverable is a paper rather than an auditable model — a well-maintained `renv.lock` file, used consistently with a documented R version, is usually sufficient and considerably lighter-weight to maintain. Docker adds real friction: a build step, image storage, the need for collaborators to have Docker installed and to understand basic container commands, and slower iteration during active development because every system-library change means a rebuild. The decision point is usually: **will someone outside this project, possibly without R expertise, need to re-run this exact model and get the exact same numbers?** If yes, containerize. If the risk is only "a package update broke something," `renv` alone addresses that.

Common pitfall. Running `apt-get install` or `install.packages()` inside a `Dockerfile` without pinning versions defeats the purpose of using Docker at all — the image will build differently on different days as upstream repositories update. Always pin the base image tag (e.g. `rocker/r-ver:4.3.2`, never `rocker/r-ver:latest`), and let `renv::restore()` — driven by a committed lockfile — control package versions rather than a bare `install.packages()` call.

One further practical point: R package installation inside a container can be slow, because by default `install.packages()` and `renv::restore()` compile packages from source on Linux. Posit Package Manager (formerly RStudio Package Manager, often referenced by its public endpoint `packagemanager.posit.co`, historically "RSPM") serves pre-compiled binary builds of CRAN packages for specific Linux distributions. Pointing your container's R options at a Posit Package Manager binary repository snapshot — rather than CRAN's source-only mirrors — can cut a `renv::restore()` that takes twenty minutes down to under a minute, which matters considerably for CI pipelines and iterative image rebuilds. Posit Package Manager also lets you pin to a **dated snapshot** of CRAN, giving you an additional, complementary layer of reproducibility alongside `renv` and Docker.

Agency perspective. If you are automating image builds (for example, rebuilding a model's Docker image nightly or on every `renv.lock` change via a CI pipeline), have the automation fail loudly — not silently fall back to `latest` tags or an un-pinned package source — the moment a pinned base image or snapshot date becomes unavailable. A silent fallback is exactly the kind of drift Docker was adopted to prevent.

2.7 Recommended package ecosystem (overview map)

The preceding sections set up the plumbing — R itself, RStudio or another IDE, renv, and optionally Docker. What actually gets installed into that environment is the subject of most of the rest of this book. R's package ecosystem for health economics and outcomes research has matured substantially over the past decade, to the point where nearly every stage of a cost-effectiveness analysis — from cleaning trial data through to generating a submission-ready report — has at least one well-maintained, peer-used package behind it. This section is meant as a map you can return to: a single reference point that tells you, for a given task, which package to reach for and which later chapter in this book covers it in depth.

HEOR in practice. Do not install everything in this table at once "just in case." Install packages as your project needs them, and record each one in your `renv.lock` file as you go (Section 2.3). A project's dependency tree should reflect what the analysis actually uses, both for reproducibility and so that a reviewer of your code can infer the analytical approach from the `library()` calls alone.

Task area	Key packages	Covered in
Data wrangling	`tidyverse` (`dplyr`, `tidyr`, `ggplot2`, `purrr`), `data.table`	Ch. 3-5
Survival analysis	`survival`, `flexsurv`, `flexsurvcure`	Ch. 10
Meta-analysis	`meta`, `metafor`	Ch. 11
Network meta-analysis	`netmeta`, `gemtc`, `multinma`	Ch. 12
Bayesian modelling	`brms`, `rstan`, `cmdstanr`	Ch. 14
Decision-tree modelling	`rdecision`, `heemod`	Ch. 18
Markov / cohort models	`heemod`, `hesim`	Ch. 19
Uncertainty analysis / VOI	`BCEA`, `dampack`, `voi`	Ch. 25-26
Missing data	`mice`, `Amelia`, `missForest`	Ch. 25
Reporting	Quarto, `gt`, `gtsummary`, `flextable`	Ch. 28
Interactive apps	`shiny`, `golem`, `rhino`	Ch. 29

Table 3: R package ecosystem for HEOR by task area, with the key packages for each and where in this book they are covered in depth.

A few orientation notes on how these areas relate to one another in practice:

- **Data wrangling is the substrate everything else sits on.** Whether you ultimately build a Markov model with `heemod` or run a network meta-analysis with `netmeta`, the input data almost always passes through `tidyverse` or `data.table` pipelines first. Chapters 3–5 build this foundation before any health-economic modelling is introduced.
- **Survival analysis and decision modelling are tightly coupled.** `flexsurv` fits parametric survival curves to trial data; those fitted distributions frequently become the transition probabilities or extrapolated survival inputs for a `heemod` or `hesim` Markov model. Expect chapters 10, 18, and 19 to cross-reference each other heavily.
- **`heemod` and `hesim` overlap but are not interchangeable.** `heemod` has a gentler learning curve and a clear cohort-based Markov-model API; `hesim` is built for speed at scale (large PSA runs, individual-level microsimulation, and multi-state models) and integrates more directly with `flexsurv` objects. Chapter 19 covers the trade-offs and when to reach for each.
- **Evidence synthesis scales from pairwise to network to Bayesian.** `meta` and `metafor` handle classical pairwise meta-analysis; `netmeta` and `gemtc` extend this to networks of trials; `multinma` and `gemtc` both support Bayesian NMA, with `multinma` in particular designed with HTA-style population-adjustment methods in mind. `brms`, `rstan`, and `cmdstanr` sit one level below these as general-purpose Bayesian engines that some of the above packages call internally, and that chapter 14 also teaches you to use directly for custom Bayesian economic models.
- **Uncertainty analysis is usually the last mile, not a separate project.** `BCEA`, `dampack`, and `voi` all operate on the output of a probabilistic sensitivity analysis already run through a decision model — they do not run the model themselves. Chapters 25–26 assume you arrive with PSA output in hand.
- **Reporting tools are what turns a script into a deliverable.** Quarto (not a package in the traditional sense, but the rendering engine increasingly replacing R Markdown for this purpose)

combined with `gt`, `gtsummary`, and `flextable` is how most modern HEOR teams produce submission-ready tables and reproducible reports directly from R code; chapter 28 builds a full reporting pipeline this way.

Difficult terminology: multi-state model. A simple survival model only tracks one transition — alive to dead — and a basic Markov model usually tracks a small, fixed set of health states (e.g. well, progressed, dead). A multi-state model generalises both: it formally tracks a patient moving between any number of defined health states over time, estimating both the **probability** of each possible transition (e.g. from "stable disease" to "progression" versus to "death" directly) and the **timing** of when it happens, using the same survival-analysis machinery (hazard functions, covered in Chapter 10) applied separately to each possible transition in the diagram. A useful analogy is a subway map with multiple interchangeable lines and stations: a simple survival model only asks "did the train ever reach the depot (death)," while a multi-state model asks, for every station (health state) a train currently sits at, which of the several next stations it's most likely to travel to next, and how long that leg of the journey typically takes. `hesim`'s multi-state-model support (mentioned above) is what lets an R-based HEOR model represent disease pathways that are richer than a simple two- or three-state Markov structure — common, for example, in oncology models with distinct progression, response and relapse states.

Common pitfall. Package names in this space are easy to confuse because several solve adjacent but distinct problems: `meta` and `metafor` are not interchangeable with `netmeta` (pairwise vs. network evidence), and `mice` (multiple imputation by chained equations) is a fundamentally different missing-data strategy from `missForest` (a random-forest-based single imputation method) — the choice between them has real implications for your estimates' standard errors, not just a difference in syntax. Confirm you are using the package appropriate to your actual evidence structure and missingness mechanism before trusting its defaults.

Agency perspective. If you are using an AI coding assistant to scaffold HEOR analysis code, be specific about which package you mean when a generic term like "run a meta-analysis" or "build a decision model" is ambiguous across several packages in the table above — an assistant without HEOR-specific context may default to the most commonly documented general-statistics package (e.g. plain `survival`) rather than the domain-appropriate one (e.g. `flexsurv` for parametric extrapolation), and the resulting code may run without error while answering the wrong question.

Chapter 3. R Essentials for Health Economists

Learning objectives

- Use R's core data structures — vectors, lists, data frames/tibbles, matrices and arrays — fluently, including why each matters for later HEOR modelling work.
- Wrangle tabular data efficiently with both `tidyverse`/`dplyr` and `data.table`, and know when to prefer each.
- Apply vectorization and `purrr`'s functional-programming tools to iterate a calculation across many parameter sets — the exact pattern later used for probabilistic sensitivity analysis.
- Write well-structured, defensively-programmed R functions using named parameter lists, appropriate for models with many inputs.
- Profile R code before optimizing it, and know when `Rcpp` or parallelisation (`future`/`furrr`) is actually warranted.
- Handle dates, durations and time-to-event quantities correctly with `lubridate`, avoiding common international date-format pitfalls.
- Debug R code interactively and handle errors gracefully in long-running simulations such as PSA loops.

3.1 Data structures: vectors, lists, data frames and tibbles, arrays and matrices

Almost everything you will ever compute in a health-economic evaluation — costs, utilities, probabilities, model traces — ultimately lives inside one of a handful of R data structures. Before Chapter 4 introduces any economic concepts, it is worth being fluent in these containers, because the structure you choose determines how easy (or treacherous) later calculations will be.

The atomic vector is the most basic unit. R has no true scalars: even a single number like `42` is a vector of length one. Vectors are homogeneous — every element shares one type (`logical`, `integer`, `double`, `character`) — and R exploits this to **vectorize** arithmetic, meaning operations apply element-by-element without an explicit loop:

```
cost <- c(1200, 3400, 980, 15600)
outcome <- c(0.72, 0.65, 0.81, 0.40)
cost_per_outcome_unit <- cost / outcome
cost_per_outcome_unit
```

This vectorization is not a minor convenience; it is the reason R code that processes thousands of patient records runs in milliseconds rather than requiring hand-written loops. Mixing types in `c()` triggers implicit coercion (the hierarchy is roughly logical → integer → double → character), a frequent source of silent bugs — a logical vector accidentally combined with a character vector becomes character, and comparisons stop working as expected.

Lists relax the homogeneity rule: a list can hold elements of different types and lengths, including other lists. This makes lists the natural container for structured, non-tabular things — most importantly, bundles of model parameters. A typical pattern you will reuse heavily from Chapter 4 onward is a nested list holding every input a model needs:

```
params <- list(
  costs = list(drugA = 1200, drugB = 980, monitoring = 150),
  utilities = list(well = 0.85, sick = 0.55, dead = 0),
  discount_rate = 0.035
)
params$costs$drugA
```

Access with `$` and `[[ ]]` extracts a single element; single-bracket `[ ]` on a list always returns a (sub-)list, even for one element — a distinction that trips up almost everyone at least once.

Tabular data is where `data.frame` and its tidyverse cousin `tibble` (from the `tibble` package, loaded automatically with `library(tidyverse)`) come in. Both store a collection of equal-length vectors as named columns, so a data frame can mix numeric, character and factor columns side by side — exactly the shape of a patient-level dataset with an ID, a cost, and a treatment arm. Tibbles behave identically for most purposes but deliberately tighten base R's looser habits: they print only the first ten rows with column types labeled, never do partial matching of column names, and `[` always returns a tibble (never silently drops to a vector) unlike a data frame, whose `[` drops to a vector when it selects a single column.

> **Common pitfall.** Base `data.frame` objects support **partial matching** on `$` and allow `drop = TRUE` behavior by default when subsetting a single column with `[`. So `df$cos` may silently return the `cost` column if no other column starts with `cos`, and `df[, "cost"]` returns a bare numeric vector, not a one-column data frame. Both behaviors have caused real analysis errors in published health-economic code. `tibble` objects disable partial matching entirely and never implicitly drop dimensions, which is one of the strongest arguments for using `as_tibble()` or `tibble()` constructors throughout your HEOR workflows.

Matrices and arrays generalize vectors to two or more dimensions while keeping a single data type. A matrix is just a vector with a `dim` attribute, built with `matrix()`; an array extends this to three or more dimensions. These matter enormously for the Markov cohort models you will build in Chapter 19: a transition probability matrix is naturally represented as a square matrix where rows sum to 1, and `%*%` (matrix multiplication) propagates a cohort distribution one cycle forward with a single operation.

> **HEOR in practice.** A toy three-state transition structure (Healthy, Sick, Dead) can be sketched now, long before any Markov-model machinery exists:
>
> ```
> states <- c("Healthy", "Sick", "Dead")
> tm <- matrix(
>   c(0.80, 0.15, 0.05,
>     0.00, 0.70, 0.30,
>     0.00, 0.00, 1.00),
>   nrow = 3, byrow = TRUE,
>   dimnames = list(states, states)
> ```

```
)
rowSums(tm)  # should equal 1 for every row
```

Building this habit early — always naming dimensions with `dimnames` and always checking `rowSums()` equals 1 — will save debugging time when transition matrices grow to a dozen states.

Finally, three inspection functions should become reflexive: `class()` reports the object's high-level class (`"data.frame"`, `"matrix"`, `"numeric"`), `typeof()` reports R's internal storage type (`"double"`, `"list"`, `"character"`), and `str()` gives a compact recursive summary of structure and contents — indispensable for peering into a nested parameter list or confirming a column you expected to be numeric did not silently become a factor or character on import.

3.2 Data wrangling with `tidyverse` and `data.table`

Reading in raw data is rarely the hard part; reshaping it into the form an analysis needs almost always is. R offers two mature, widely used approaches to this reshaping, and a working HEOR analyst should be conversant in both: the `tidyverse` (principally the `dplyr` package, plus the pipe) and `data.table`. They are not rivals to be chosen once and for all — they solve the same problem with different trade-offs in readability and speed, and large pharmacoepi or claims-registry projects frequently use both in the same codebase.

The `dplyr` grammar is built from a small set of verbs that each do one thing: `filter()` keeps rows matching a condition, `select()` keeps columns, `mutate()` creates or modifies columns, `group_by()` partitions a data frame into groups for subsequent operations, and `summarise()` collapses each group to one row of aggregate statistics. These verbs are chained with the pipe operator — either `magrittr`'s `%>%` (the long-standing tidyverse convention) or base R's native `|>` (available since R 4.1, functionally equivalent for straightforward chains and increasingly the recommended default). The result reads almost like a sentence describing the transformation step by step:

```
library(tidyverse)

patient_data |>
  filter(arm %in% c("Drug A", "Drug B")) |>
  group_by(arm) |>
  summarise(mean_cost = mean(total_cost), n = n())
```

`data.table` takes a different, more compact approach, built around a single indexing expression `DT[i, j, by]`: `i` selects/filters rows, `j` computes or selects columns, and `by` groups the computation — all inside one set of square brackets, operating on the table **in place** rather than copying it. The equivalent computation looks like this:

```
library(data.table)

patient_dt <- as.data.table(patient_data)
patient_dt[arm %in% c("Drug A", "Drug B"),
           .(mean_cost = mean(total_cost), n = .N),
           by = arm]
```

Both snippets produce the same grouped summary, and once the `DT[i, j, by]` pattern is internalized it is no less readable than the pipe — it is simply denser. The practical difference that matters

is performance and memory behavior at scale. `data.table` modifies data in place (via reference semantics and functions like `setDT()` and `:=` for in-place column assignment) and uses a highly optimized C-level radix sort and hash-based grouping, which makes it dramatically faster and more memory-efficient than `dplyr` once a dataset reaches into the millions of rows — exactly the regime of national claims databases, disease registries, or simulated patient-level microsimulation output with many replications. `dplyr`, by contrast, favors copy-on-modify semantics (it never silently alters your original object), which is safer for interactive, exploratory work but costs more memory and time as data grows.

> **Difficult terminology: radix sort / hash-based grouping.** Both are internal techniques for ordering or grouping rows of data fast, and the reason `data.table` can out-perform more general approaches at large scale. An ordinary comparison-based sort (the kind most languages use by default) repeatedly compares pairs of whole values against each other to decide their order, which gets slower as the dataset grows because the number of comparisons needed grows faster than the data itself. A **radix sort** instead orders data by processing one digit, character, or byte at a time — rather like sorting a stack of exam papers first by surname's first letter into 26 piles, then re-sorting each pile by the second letter, and so on — which scales far better for the kinds of values (integers, short strings, factor-coded categories) common in claims and registry data. **Hash-based grouping** uses a related lookup-table trick for the `by =` step in `DT[i, j, by]`: rather than comparing every row against every other row to see which share a group, it computes a quick numeric "fingerprint" (a hash) for each row's grouping value and uses that fingerprint as a direct lookup key, finding matching rows nearly instantly instead of scanning the whole table repeatedly. Neither technique requires you to write any special code yourself — `data.table` applies them automatically whenever you sort or group — but knowing they exist explains **why** the same `DT[i, j, by]` syntax that looks unremarkable on a 500-row toy dataset becomes dramatically faster than `dplyr` once you are working with millions of claims records.

- Prefer `tidyverse`/`dplyr` when the dataset is small-to-moderate (comfortably fits in memory with room to spare), when the code will be read and maintained by collaborators less fluent in R, or when the pipeline benefits from the broader tidyverse ecosystem (`tidyr` for reshaping, `ggplot2` for plotting the same tibbles downstream).
- Prefer `data.table` when working with claims-scale or registry-scale data (millions of rows, wide microsimulation outputs, repeated resampling such as probabilistic sensitivity analysis replications), when runtime or memory genuinely becomes a bottleneck, or when the same large table is updated repeatedly inside a loop, where `data.table`'s in-place `:=` avoids repeated full-table copies.
- Mixing both is common and acceptable: convert a `data.table` result back with `as_tibble()` or `setDF()` once the heavy aggregation is done, and finish the pipeline in `dplyr`/`ggplot2` for readability.

> **Common pitfall.** `data.table`'s `:=` and in-place modification mean that assigning `dt2 <- dt1` does **not** create an independent copy the way it would for a data frame or tibble — `dt2` and `dt1` point to the same underlying object, and modifying one with `:=` silently modifies the other. If you need an independent copy, use `copy(dt1)` explicitly. This reference-semantics behavior

is the single most common source of "my original data changed and I don't know why" bugs when analysts move from `dplyr` to `data.table`.

Agency perspective. Whichever grammar you choose for a given analysis script, say so explicitly in the script header or project README (Chapter 2's `analysis/` convention) and stay consistent within that script — silently switching between `%>%`, `|>`, and `DT[]` syntax mid-pipeline makes code far harder for a collaborator, or your future self, to audit. The CHEERS 2022 reporting checklist (covered in full in Chapter 28), item 29, explicitly requires that analytic methods be reported in enough detail for a third party to reproduce them — code-style consistency within one script is a small thing that materially supports that requirement in practice, not just a stylistic nicety.

Operation	`dplyr` / tidyverse	`data.table`
Filter rows	`filter(arm == "Drug A")`	`DT[arm == "Drug A"]`
Select columns	`select(id, cost)`	`DT[, .(id, cost)]`
Create/modify column	`mutate(cost2 = cost * 1.1)`	`DT[, cost2 := cost * 1.1]`
Group and summarise	`group_by(arm) |> summarise(mean_cost = mean(cost))`	`DT[, .(mean_cost = mean(cost)), by = arm]`
Chain multiple steps	`|>` or `%>%` pipe between verbs	stacked `[i, j, by]` expressions
Independent copy	implicit (copy-on-modify)	explicit: `copy(dt1)`

Note. `data.table` modifies in place via reference semantics; `dplyr` never silently alters the original object.

Table 4: Common data-wrangling operations: `dplyr`/tidyverse syntax versus the equivalent `data.table` expression.

3.3 Functional programming with `purrr` and vectorization

R is a vectorized language at its core: arithmetic, comparisons, and most mathematical functions operate element-by-element on whole vectors without you writing an explicit loop. This matters for more than style. A `for` loop that grows a result vector by repeated `c()` or indexed assignment inside the loop forces R to reallocate memory on every iteration, and interpreted loop overhead adds up quickly once you are iterating over thousands of patients, cycles, or parameter draws — exactly the scale you hit in probabilistic sensitivity analysis (PSA) later in this book. A vectorized expression, by contrast, dispatches to compiled C code under the hood and processes the whole vector in one pass. The practical rule of thumb: if you find yourself writing `for (i in 1:n)` to apply the same arithmetic to each element of a vector, stop and ask whether the operation can be expressed directly on the vector instead.

```
n_patients <- 1000
cost_per_visit <- rep(185.50, n_patients)
n_visits <- sample(1:12, n_patients, replace = TRUE)
```

```r
# Avoid: explicit loop
total_cost_loop <- numeric(n_patients)
for (i in seq_len(n_patients)) {
  total_cost_loop[i] <- cost_per_visit[i] * n_visits[i]
}

# Prefer: vectorized
total_cost_vec <- cost_per_visit * n_visits
```

Not every problem reduces to simple elementwise arithmetic, though. Often you need to apply a more elaborate function — one with branching logic, multiple inputs, or a non-trivial return type — to each element of a list or each row of a parameter table. This is where the `purrr` package (part of the tidyverse) earns its place in your toolkit, as a more predictable and composable alternative to base R's `sapply()`/`lapply()`.

The core function is `purrr::map()`, which always returns a list, regardless of what you feed it — this predictability is the whole point. Where `sapply()` will silently simplify its output to a vector, a matrix, or a list depending on the data, `map()` never surprises you. When you know the output should be a specific atomic type, use the typed variants: `map_dbl()` returns a double vector (or errors if any element can't be coerced), `map_chr()` a character vector, `map_lgl()` a logical vector, and so on. This type enforcement is itself a form of defensive programming — a `map_dbl()` call that unexpectedly hits a non-numeric result fails loudly instead of quietly returning an untyped list you forget to unlist.

```r
library(purrr)

scenarios <- list(
  base_case  = list(cost_visit = 185.50, visits = 4, n_patients = 250),
  high_cost  = list(cost_visit = 240.00, visits = 4, n_patients = 250),
  high_util  = list(cost_visit = 185.50, visits = 7, n_patients = 250)
)

calc_total_cost <- function(params) {
  params$cost_visit * params$visits * params$n_patients
}

total_costs <- map_dbl(scenarios, calc_total_cost)
total_costs
#>  base_case  high_cost  high_util
#>     185500     240000     324625
```

When you need to iterate over two parallel vectors or lists simultaneously, `map2()` takes two inputs and applies a function pairwise; when you have three or more, `pmap()` takes a single list of input lists/vectors and applies a function across all of them element-by-element. These generalize naturally to the structure of a parameter table, where each row is one scenario and each column is one parameter.

```r
visit_counts <- c(4, 5, 6)
cost_per_visit <- c(185.50, 185.50, 200.00)

map2_dbl(visit_counts, cost_per_visit, ~ .x * .y * 250)

param_grid <- list(
```

```
  cost_visit  = c(185.50, 200.00, 220.00),
  visits      = c(4, 5, 6),
  n_patients  = c(250, 250, 300)
)

pmap_dbl(param_grid, calc_total_cost)
```

> **HEOR in practice.** This `scenarios` list and `pmap_dbl()` pattern is not a toy exercise — it is the literal skeleton of a PSA loop. In Chapter 25, you will replace this three-row `param_grid` with a parameter table of 1,000+ rows, each row a random draw from a cost or utility distribution, and replace `calc_total_cost()` with a full cost-effectiveness model function. The call becomes `pmap(psa_params, run_model)`, and the output list becomes your distribution of ICERs. Learning to think in terms of "a function applied across a parameter table" now will make that chapter feel like a straightforward extension rather than a new paradigm.

3.4 Writing functions, parameter lists and defensive programming

HEOR models rarely take two or three arguments. A Markov cost-effectiveness model might need transition probabilities, cost inputs, utility weights, a discount rate, a time horizon, and a cycle length — easily ten or more parameters. Writing a function that takes these as a long sequence of positional arguments is a recipe for subtle, hard-to-detect bugs: swap the order of `discount_rate` and `cycle_length` in a call several hundred lines away from the function definition, and R will not complain — it will just silently compute the wrong answer.

```
# Fragile: positional arguments, easy to transpose by mistake
run_model <- function(p_progress, p_death, cost_well, cost_sick,
                      util_well, util_sick, discount_rate, horizon) {
  # ...
}
```

The more robust pattern is to bundle related parameters into a single named list (or, for larger models, a dedicated list-of-lists or a small S3/S4 object) and pass that one structured object into the function. Named access by `$` or `[[ ]]` makes every call self-documenting and immune to argument-order mistakes.

> **Difficult terminology: S3 / S4 objects.** R has more than one system for object-oriented programming — ways of bundling data together with the functions ("methods") that know how to act on it, so the data carries its own behaviour rather than living as a bare, anonymous list. S3 is the older, lightweight system: you simply attach a `class` attribute (a label) to an ordinary R object, and then write functions named in a particular pattern (e.g. `print.my_model`) that R automatically dispatches to whenever someone calls `print()` on an object of that class. There is no formal definition enforced anywhere — it works by convention and is easy to get wrong silently. S4 is a stricter, more formal system: you explicitly declare a class's structure in advance (which fields it must have, and what type each one is), and R then checks that any object you create actually conforms to that declaration, catching mistakes earlier. A filing-cabinet analogy helps: S3 is like labelling a folder "Patient Record" and trusting everyone to put the right kinds

of documents inside it; S4 is like a pre-printed form with named, typed boxes for each field, which physically cannot be filled in with the wrong kind of information. Most of this book's own worked examples use plain named lists for simplicity, but model-parameter bundles, `survival` package model-fit objects, and several HEOR packages (`flexsurv`, `heemod`) use S3 or S4 objects internally, so recognising the pattern when you see `class(x)` or `methods(print)` in someone else's code is a practically useful skill even if you rarely need to **write** a new S4 class yourself.

```
model_params <- list(
  p_progress     = 0.15,
  p_death        = 0.05,
  cost_well      = 1200,
  cost_sick      = 8500,
  util_well      = 0.85,
  util_sick      = 0.55,
  discount_rate  = 0.035,
  horizon        = 10
)

run_model <- function(params) {
  stopifnot(is.list(params))
  with(params, {
    # model logic refers to p_progress, cost_sick, etc. by name
  })
}
```

Give arguments sensible default values wherever a standard convention exists — a discount rate of 3.5% per UK NICE guidance, or a cycle length of one year, for instance — using R's native `function(x, discount_rate = 0.035)` syntax. Defaults reduce boilerplate at every call site and document the "normal" assumption directly in the function signature, but be explicit in comments about where a default comes from, since a silently-applied wrong default is just as dangerous as a wrong argument.

The deeper issue is that R, by design, is permissive about types. A probability argument passed as the character string `"0.15"` instead of the numeric `0.15` will often not fail immediately — it may propagate through several calculations, coerced or recycled in ways that produce a number that looks plausible but is wrong, and surface only as an ICER that is subtly off, discovered (if you are lucky) by a reviewer long after submission. This is why defensive programming is not an optional style preference in HEOR work — it is a quality-assurance requirement.

The baseline tool is `stopifnot()`, which checks one or more conditions and halts execution immediately with an error if any is false — cheap insurance placed at the top of every model function.

```
run_model <- function(params) {
  stopifnot(
    is.list(params),
    is.numeric(params$p_progress),
    params$p_progress >= 0, params$p_progress <= 1,
    is.numeric(params$discount_rate),
    params$discount_rate >= 0, params$discount_rate < 1,
```

```
    params$horizon > 0
  )
  # ... model logic ...
}
```

For checks that need a more informative, customized message than `stopifnot()`'s default, use `stop()` directly, and reserve `warning()` for conditions that are suspicious but not necessarily fatal — an unusually high cost input, say, that you want flagged without halting a long-running PSA batch.

```
validate_probability <- function(p, name) {
  if (!is.numeric(p)) {
    stop(sprintf("'%s' must be numeric, got %s.", name, class(p)[1]))
  }
  if (p < 0 || p > 1) {
    stop(sprintf("'%s' must lie in [0, 1], got %s.", name, p))
  }
  if (p > 0.9) {
    warning(sprintf("'%s' = %s is unusually high; check the source.", name, p))
  }
  invisible(p)
}
```

A good discipline is to validate every input once, at the top of the function, rather than scattering `if` checks throughout the body — this keeps the "is this input sane?" logic separate from "what does this input mean for the model?" logic, and makes the function easy to audit.

- State preconditions explicitly with `stopifnot()` before any computation runs.
- Use `stop()` with a formatted, specific message when a precondition fails outright.
- Use `warning()` for implausible-but-not-impossible values you want surfaced, not silenced.
- Bundle 5+ related arguments into a named list rather than a long positional signature.

Common pitfall. R's automatic type coercion is a frequent source of silent errors. `if (p_progress == "0.15")` will happily evaluate as `TRUE` even when `p_progress` is the numeric `0.15`, because R coerces the number to a string for the comparison — no error, no warning, just a wrong path taken. A function that checks `is.numeric()` explicitly, rather than relying on implicit coercion inside a comparison, catches this class of bug before it reaches your cost-effectiveness output.

Agency perspective. When you ask an AI coding assistant to draft a model function for you, always ask it to add `stopifnot()` input validation and informative `stop()`/`warning()` messages in the same pass — generated code tends to implement the "happy path" calculation correctly but omit defensive checks unless specifically prompted, and an unvalidated function is the easiest place for a submission-breaking error to hide. NICE's technology appraisal process (Chapter 2) relies on the Evidence Review Group being able to re-run a submitted model and trust its outputs; a model that silently accepts an out-of-range probability and returns a plausible-looking but wrong ICER is a far worse submission failure than one that errors loudly and gets caught before submission.

3.5 Performance: profiling, `Rcpp`, parallelisation (`future`, `furrr`)

The first rule of optimisation is to not do it until you have evidence. R code that "feels slow" and R code that **is** slow in the place you think it is are often two different programs. Before touching a single line in the name of speed, profile it. `Rprof()` is the base-R sampling profiler — it periodically records the call stack while your code runs, and `summaryRprof()` turns that into a table of where time actually went. `profvis::profvis()` wraps the same mechanism in an interactive flame graph that is far easier to read:

> **Difficult terminology: flame graph.** A flame graph is a specific way of visualising where a program spent its execution time, and it reads differently from an ordinary bar chart. Each horizontal bar represents one function call, and its **width** shows how much total time that call took (wider = slower). Bars are stacked vertically to show the call hierarchy: a function sits directly above the bars representing the functions it called, so following a column straight down from any bar traces that call's entire chain of parent functions back to the top-level script. The name "flame graph" comes from the characteristic look of these stacked bars — varying widths at each level resembling flickering flame tips. Reading one is mostly about scanning for the **widest** bars, especially ones near the bottom of a tall stack (meaning a deeply nested function is eating a disproportionate share of total runtime) — those are the parts of the code actually worth optimising, as opposed to functions that are called often but individually fast, which will show up as many thin bars rather than one wide one.

```
library(profvis)

profvis({
  results <- vector("list", 500)
  for (i in seq_len(500)) {
    results[[i]] <- run_markov_trace(params_list[[i]])
  }
})
```

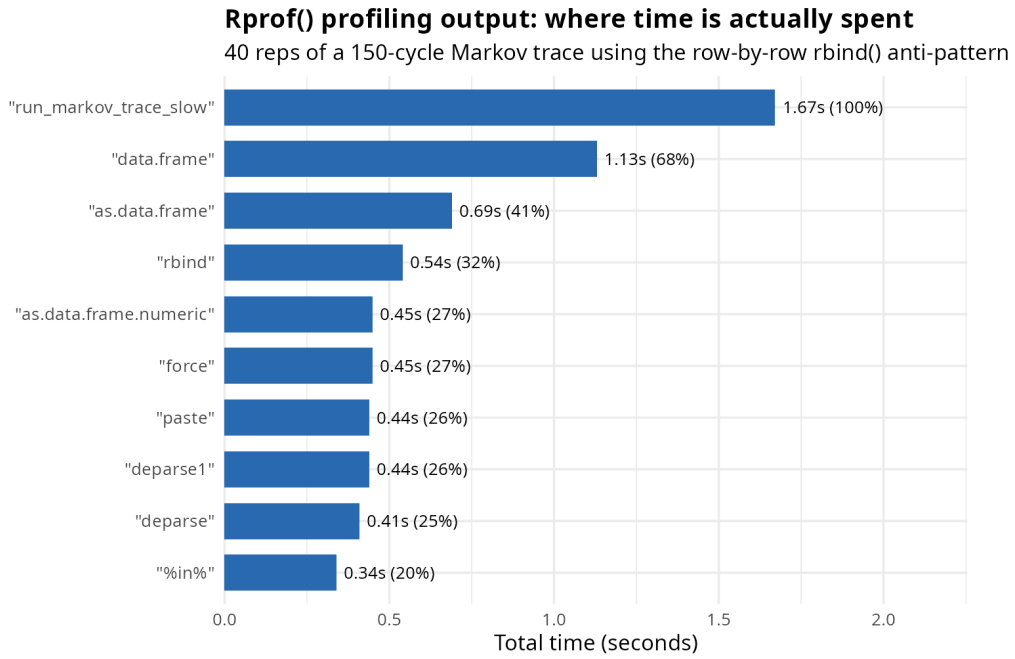


Figure 2: `Rprof()`/`summaryRprof()` output on a deliberately slow Markov-trace loop. The `rbind()`-in-a-loop anti-pattern dominates total time — `data.frame()`, `as.data.frame()` and `rbind()` together account for well over half of it — exactly the kind of result profiling should surface before any hand optimisation begins.

How to report this.

Rank	Function	Why it dominates
1	`rbind()`	Called once per loop iteration; reallocates and copies the growing result object each time
2	`data.frame()`	Invoked internally by `rbind()` on data-frame arguments, compounding the reallocation cost
3	`as.data.frame()`	Coercion overhead triggered repeatedly as part of the same `rbind()` call chain
4+	model/arithmetic logic	Comparatively minor; swamped by the three functions above combined

Note. `data.frame()`, `as.data.frame()` and `rbind()` together account for well over half of total runtime — fix the growing-object pattern before considering `Rcpp` or parallelisation.

Table 5: Top functions by self + total time in the `summaryRprof()` output above, ranked by where profiling attention should go first.

Run this on a **representative** slice of the real workload, not a toy example — bottlenecks in a 500-iteration loop are not always visible at 5 iterations. Nine times out of ten the profiler finds

the slow part in a place you didn't expect: a `data.frame` growing row-by-row inside a loop, an unnecessary `rbind()`, or repeated `read.csv()` calls on the same file. Fix those first; they are free wins that require no new tooling.

Occasionally the profiler confirms that the bottleneck is a genuine hot loop — tight, scalar, repeated millions of times — that vectorisation cannot absorb (e.g., a recursive Markov trace update computed one cycle at a time). This is the narrow case where `Rcpp` earns its keep: you rewrite just that loop in C++, compile it inline, and call it from R as an ordinary function.

```
library(Rcpp)

cppFunction('
  double cycle_sum(NumericVector costs, double discount_rate) {
    double total = 0;
    int n = costs.size();
    for (int i = 0; i < n; i++) {
      total += costs[i] / pow(1 + discount_rate, i);
    }
    return total;
  }
')
```

Reach for this only after profiling has pointed a finger at a specific loop — `Rcpp` adds a C++ toolchain dependency (the one Chapter 2 flagged alongside `cmdstanr`) and a compiled artefact to maintain, so it should replace a measured bottleneck, not a hunch.

More often than not, the real lever is not making one iteration faster but running many iterations at once. This is directly relevant to the probabilistic sensitivity analyses (PSA) built in Chapter 25, where a model is re-run over thousands of parameter draws and each run is independent of the others — an "embarrassingly parallel" problem. The `future` package abstracts **how** code is executed from **what** is executed: you declare a plan once, and the same code then runs sequentially or in parallel without modification.

```
library(future)
library(furrr)

plan(multisession, workers = 4)

psa_results <- future_map(psa_param_draws, run_model, .progress = TRUE)
```

`furrr::future_map()` is a drop-in parallel replacement for `purrr::map()` — same signature, same list-in/list-out contract, but each element is dispatched to a worker process under the active `future` plan. Swapping `map()` for `future_map()` and adding one `plan()` call is often the entire migration.

> **HEOR in practice.** Before parallelising a PSA, make sure each iteration is self-contained — no shared mutable state, no writing to the same file from multiple draws. `future`'s workers are separate R processes; they cannot see objects you forgot to pass in, and silent scoping bugs that worked by accident in a sequential `for` loop will fail (or worse, produce wrong numbers) under `multisession`.

Parallelising a model that has a bug does not fix the bug; it just produces the wrong answer faster, on four cores at once. Profile, fix the real bottleneck if there is one, parallelise last.

3.6 Dates, durations and time-to-event data (`lubridate`)

Health economic models built on trial data live and die by correct date arithmetic — time-to-progression, time-to-death, length of treatment exposure, time between visits, are all date subtractions underneath. Base R's `Date` class handles the simple cases, but `lubridate` makes the parsing, arithmetic and edge cases explicit rather than implicit, which matters when the input is a column of strings from a dozen different trial sites.

Parsing is the first hazard. `lubridate::ymd()`, `dmy()` and `mdy()` each commit to one date order and fail loudly (returning `NA` with a warning) rather than guessing silently:

```
library(lubridate)

enrol_date <- ymd("2021-03-14")
visit_date <- dmy("14/03/2021")

as.Date("2021-03-14")  # base R equivalent, but order must match exactly
```

> **Common pitfall.** International trial data frequently mixes `DD/MM/YYYY` and `MM/DD/YYYY` formats across sites, and spreadsheet exports rarely declare which one they used. `as.Date("03/04/2021")` silently parses as 3rd April under one default and 4th March under another — no error, no warning, just a wrong date fed straight into a survival-time calculation. Always confirm the source format explicitly (check a known out-of-range day like the 25th against the site's convention) and parse with the matching `lubridate` function — `dmy()` or `mdy()` — rather than a bare `as.Date()` that silently assumes one order.

Once dates are parsed correctly, time-to-event quantities reduce to differences. `difftime()` computes a difference with an explicit unit, which avoids the ambiguity of letting R guess whether you meant days or weeks:

```
time_to_progression <- difftime(progression_date, enrol_date, units = "days")
```

For anything beyond a single subtraction — durations spanning leap years, or lengths that should be expressed in calendar months rather than a fixed number of days — `lubridate::interval()` and `time_length()` are the more robust pair. An `interval` is a span between two specific time-points (not just a duration), and `time_length()` converts it into whatever unit you need, correctly accounting for variable month and year lengths:

```
progression_interval <- interval(enrol_date, progression_date)
time_length(progression_interval, unit = "months")
```

This distinction — duration as "an interval between two real dates" versus duration as "a fixed span of time" — becomes important once you reach the parametric survival models in Chapter 10, where time-to-event outcomes are the response variable and getting the units wrong (days vs. months) silently rescales every fitted hazard and every extrapolated curve downstream.

A related trap is censoring: a patient who has not progressed by the data cut-off does not have a missing progression date in the "NA means unknown" sense — they have a **censored** observation, and that status needs to be carried as its own column (commonly a 0/1 event indicator) rather than inferred from an `NA` in the date field. Confusing "no event date because censored" with "no event date because missing" is a common source of silently wrong survival curves, and it starts right here at the data-cleaning stage, well before any survival model is fitted.

3.7 Debugging and error handling

A health economic model that runs to completion and produces a wrong number is more dangerous than one that crashes, because a crash is visible and a silently wrong number is not. This section covers the tools for finding bugs interactively, and — more importantly for the long-running simulations in this book — the discipline of handling errors so that one bad input doesn't either kill a multi-hour run or, worse, let it finish with corrupted results.

When something throws an error, `traceback()` is the first diagnostic: run it immediately after the error to see the full call stack that led to it, innermost call first. This tells you **which** function actually failed, which is not always the function named in the error message:

```
run_model(bad_params)
#> Error in sqrt(variance) : NaN produced
traceback()
```

For errors that are hard to pin down from the stack alone, `debug(fun_name)` flags a function so that the next call to it drops you into an interactive browser at its very first line, letting you step through line by line and inspect every intermediate object. `browser()` does the same thing but by inserting an explicit breakpoint directly in the source, which is more convenient when you already suspect roughly where the problem is:

```
run_model <- function(params) {
  browser()  # execution pauses here; inspect `params` interactively
  variance <- compute_variance(params)
  sqrt(variance)
}
```

Both are for **finding** bugs during development. They are not a substitute for handling errors gracefully in code that will run unattended — and in health economic practice, nothing runs more unattended than a PSA loop iterating over thousands of parameter draws. `tryCatch()` is the tool for that: it lets you catch a specific failure, decide what to do about it, and let execution continue rather than halting the entire run.

```
psa_results <- vector("list", n_iterations)

for (i in seq_len(n_iterations)) {
  psa_results[[i]] <- tryCatch(
    run_model(psa_param_draws[[i]]),
    error = function(e) {
      message(sprintf("Iteration %d failed: %s", i, conditionMessage(e)))
      NA
    }
  )
}
```

Here a single malformed draw — a negative cost sampled from a misspecified distribution, a file read that occasionally fails on a flaky network drive — logs itself and returns `NA` for that one iteration, instead of stopping the other 9,999.

> **HEOR in practice.** Wrap each individual PSA iteration in its own `tryCatch()`, not the whole loop. Wrapping the entire `for` loop in one `tryCatch()` means the **first** bad draw stops everything and you lose every result computed before it; wrapping each iteration means you

lose exactly one data point and keep the other thousands. Always log which iteration failed and why — a silent `NA` with no record of the cause is a debugging problem for tomorrow-you.

Agency perspective. An uncaught error partway through a PSA is the visible failure mode; the one that should worry you more is a **caught** error that is handled too permissively — a blanket `tryCatch(..., error = function(e) 0)` that replaces a failed iteration with zero instead of `NA`. Zero is a plausible-looking number that will quietly bias your cost-effectiveness acceptability curve (CEAC). ISPOR-SMDM good-practice guidance on model transparency (covered in Chapter 27) expects PSA results to be reported alongside the actual number of iterations successfully completed — silently padding a failed run back up to the intended iteration count with substituted zeros defeats that transparency requirement. `NA` at least survives downstream and forces you to notice and account for the dropped iterations before reporting final results.

The underlying principle is the same at every scale, from a single `browser()` session to a 10,000-iteration PSA: an error that stops execution is an inconvenience, but an error that gets silently absorbed into a "successful" result is the one that makes it into a published cost-effectiveness ratio.

Chapter 4. Core Concepts in Health Economics (with R Illustrations)

Learning objectives

- Explain the opportunity-cost foundation of health economics and distinguish the types of costs and effects an economic evaluation must capture.
- Distinguish cost-minimization, cost-effectiveness, cost-utility, cost-benefit, and cost-consequence analysis, and explain why cost-utility analysis with QALYs is the HTA-agency default.
- Construct a QALY from health-state duration and utility weight, and distinguish QALYs from DALYs and plain life-years, including their opposite sign conventions.
- Compute an ICER, and convert it to net monetary benefit (NMB) or net health benefit (NHB) to avoid the ratio's sign-ambiguity problems.
- Describe what a willingness-to-pay threshold represents, and the major methodological traditions (opportunity cost vs stated preference) used to set one, including India's HTAIn approach.
- Apply perspective, time horizon, discounting and half-cycle correction as explicit, justified modelling choices.
- Identify simple and extended dominance and construct an efficiency frontier from a set of competing treatments.
- Write a complete R pipeline that takes cost/QALY data for multiple treatments and produces a dominance-pruned, incrementally-costed frontier table — the book's first executed health-economic calculation.

4.1 Costs, effects and the opportunity cost of health care

Every health system, however wealthy, faces the same brute fact: the budget for health care is finite, but the number of things worth paying for is not. A national payer, an insurance fund, or a hospital formulary committee has a fixed pot of money in any given year. Every pound, dollar, or rupee spent funding one technology is a pound, dollar, or rupee that cannot be spent funding another. This is the economic idea of **scarcity**, and its direct consequence is **opportunity cost**: the value of the best alternative forgone when a choice is made.

This single idea is the foundation of everything in health economic evaluation. If a health system adopts a new oncology drug at a given price, the money for that drug is drawn, directly or indirectly, from the same budget that could have funded extra dialysis capacity, additional health visitor time, or a different medicine entirely. The health gained by patients on the new drug must be weighed against the health that could have been produced by whatever was displaced. A technology is not "cost-effective" in some free-standing, absolute sense — it is cost-effective **relative to** the alternative use of the same resources. This is why every serious economic evaluation is fundamentally comparative: it never asks "is this good?" in isolation, but "is this better use of this money than the next-best thing we could do with it?"

This comparative framing is what separates health economic evaluation from simple clinical evaluation. A clinical trial can show a drug extends survival or shrinks tumours; it says nothing about whether funding that drug is a **good use** of a constrained budget relative to displaced alternatives. Economic evaluation exists precisely to answer that second, harder question.

To do so formally, every analysis needs two ingredients: **costs** and **effects**.

Costs are usually split into three categories:

- **Direct medical costs** — costs borne by the health system in delivering care: drug acquisition, staff time, hospital bed-days, diagnostic tests, outpatient visits, adverse-event management.
- **Direct non-medical costs** — costs borne by the patient or their household in accessing care but not for clinical care itself: travel to appointments, informal carer time, home modifications, over-the-counter aids.
- **Indirect costs** — productivity losses from illness or treatment: time off paid work, reduced workplace output ("presenteeism"), premature mortality's effect on lifetime earnings. Whether indirect costs belong in the analysis at all is a long-standing methodological fault line — most European HTA reference cases (and NICE in particular) exclude productivity costs from the base case on distributional-fairness grounds, while some willingness-to-pay/societal-perspective frameworks (and some CBA applications) include them.

Effects are more subtle, because "effect" in clinical research and "effect" in HEOR are not the same thing. A clinical trial reports **clinical outcomes**: tumour response rate, HbA1c reduction, relapse rate, blood pressure change. These are precise, measurable, and essential — but they are not, on their own, comparable across disease areas, and they carry no information about how much patients **value** the change. A one-point drop in HbA1c and a 10% reduction in stroke risk cannot be traded off against each other using clinical endpoints alone. HEOR needs **value-laden** outcome measures that convert clinical effect into a common currency of health gain, so that a diabetes drug, a cancer drug, and a hip replacement can, in principle, be compared on the same scale. That conversion — from clinical endpoint to a generic, preference-weighted outcome — is the subject of the next two sections.

```
# Illustrating opportunity cost with a fixed budget
budget <- 10e6               # fixed annual budget, GBP 10m
cost_per_qaly_drugA <- 20000  # technology A
cost_per_qaly_drugB <- 12000  # displaced alternative use of the same money

qalys_A <- budget / cost_per_qaly_drugA
qalys_B <- budget / cost_per_qaly_drugB

qalys_A; qalys_B
qalys_B - qalys_A   # health forgone by funding A instead of B
```

Spending the whole £10m budget on technology A buys 500 QALYs; spending it on the displaced alternative B buys 833 QALYs. The 333 QALY difference is not an abstraction — it is real health that could have existed but did not, because the money went elsewhere. This is the opportunity cost of the funding decision, and it is the quantity that cost-effectiveness analysis exists to make visible.

> **HEOR in practice.** When you build cost inputs in R, keep direct medical, direct non-medical, and indirect cost components as **separate columns** in your cost data frame (not a single pre-summed "total cost") for as long as possible in the pipeline. Reviewers and HTA submission

templates routinely ask for a disaggregated cost breakdown, and recomputing it after the fact from a single summed column is error-prone and often impossible if the component data was discarded.

4.2 Types of economic evaluation

Not all economic evaluations measure "effect" the same way, and the choice of effect measure defines the **type** of analysis. Drummond et al.'s taxonomy, now standard across the field, distinguishes five main forms. Understanding the differences is essential, because HTA agencies do not accept any of them interchangeably — most require one specific type as their reference-case default.

- **Cost-minimization analysis (CMA)** — used only when there is good evidence that two or more interventions produce **equivalent** effects (established, for example, by a non-inferiority trial). If effects are genuinely equal, the decision collapses to picking the cheapest option. CMA is fragile: it is valid only as long as the equal-effectiveness assumption holds, and it is frequently misused when effects are merely **untested** for difference rather than **proven** equivalent.
- **Cost-effectiveness analysis (CEA)** — effects are measured in natural clinical units specific to the disease area: life-years gained, cases of disease averted, mmHg reduction, strokes prevented. Results are reported as a cost per unit of natural effect (e.g., cost per life-year gained, cost per case averted). CEA in this narrow sense is useful within a single disease area but cannot compare across different natural units — cost per stroke averted cannot be ranked against cost per cataract corrected.
- **Cost-utility analysis (CUA)** — a specific, standardized form of cost-effectiveness analysis in which effects are expressed in quality-adjusted life years (QALYs), a generic preference-weighted outcome that combines length of life and health-related quality of life into a single number (constructed explicitly in §4.3). Because QALYs are intended to mean the same thing regardless of disease area, CUA is the only one of these methods that supports cross-programme comparison — a cost per QALY for a cancer drug and a cost per QALY for a diabetes drug are, in principle, directly comparable, which is exactly what a national payer deciding across its whole portfolio needs.
- **Cost-benefit analysis (CBA)** — the most ambitious and least commonly used in HTA practice: all effects, including health itself, are monetized (via willingness-to-pay studies, value-of-a-statistical-life estimates, or similar), so that a single net monetary benefit figure summarizes the decision. CBA permits comparison not just across health interventions but across sectors entirely (health spending versus transport safety spending, for instance) but the monetization of health and life is methodologically and ethically contentious, and most health-specific HTA bodies avoid it as a primary reference-case method.
- **Cost-consequence analysis (CCA)** — deliberately avoids collapsing effects into a single ratio. Costs and multiple disaggregated effects (QALYs, specific clinical events, resource-use impacts) are presented side by side in a balance-sheet format, leaving the decision-maker to weigh trade-offs themselves. CCA is often used as a **supplementary** presentation alongside a CUA base case, especially where some important consequences resist being folded sensibly into a single utility number.

Difficult terminology: non-inferiority trial. Most clinical trials are designed as **superiority** trials: they try to show a new treatment works better than an existing one (or than placebo). A non-inferiority trial sets out to answer a deliberately different question — not "is the new treatment better," but "is the new treatment not meaningfully worse," within a pre-agreed margin that both the trial's designers and the relevant regulator accept in advance as clinically unimportant. This design is typically chosen when the new treatment offers some other advantage — cheaper, more convenient to take, fewer side effects, easier to manufacture at scale — so that proving "just as good, clinically" is enough to justify switching, without needing to prove outright superiority on the primary clinical outcome. Because a non-inferiority trial cannot prove two treatments are **exactly** equal (no finite trial can rule out every conceivable tiny difference), its result is always "not worse by more than margin X," not "identical" — a distinction HEOR analysts must keep in mind before treating a non-inferiority result as licence for unqualified cost-minimisation.

Difficult terminology: value-of-a-statistical-life estimate. This sounds like it is pricing one specific, identified person's life, but it is not — it is a population-level monetary figure representing how much a group of people are collectively willing to pay to reduce everyone's risk of death by a small amount. It is typically derived either by observing how much extra pay workers demand to take measurably riskier jobs (a **revealed preference** method, explained further in §4.5 below), or by asking people directly in surveys how much they would pay for a small reduction in their own risk of dying (a **stated preference** method). The number is then extrapolated: if 100,000 people are each willing to pay $50 for a reduction in risk that is expected to prevent one death across that group, the value-of-a-statistical-life estimate for that context is $5 million — not the value of any one real, identified person's life, but the aggregate amount a population puts toward preventing one anonymous, statistical death among them. It is the standard tool cost-benefit analysis (CBA, described just above) uses to convert mortality risk reduction into a monetary figure that can be added alongside other costs and benefits.

In practice, CUA with QALYs has become the de facto default that HTA agencies worldwide demand for formal technology appraisal, and it is worth being explicit about **why**: a finite health budget has to allocate across wildly different disease areas — oncology, rare disease, cardiovascular prevention, mental health — and only a generic outcome measure makes those allocation decisions comparable on a common scale. A natural-units CEA cost per "case averted" cannot inform whether money is better spent on cancer or on diabetes; a QALY-denominated CUA, at least nominally, can.

Agency perspective. NICE's reference case for technology appraisals specifies the QALY, estimated using the EQ-5D (or an NICE-endorsed alternative under its 2022 health technology evaluation methods manual), as the required outcome measure; submissions that present only natural-unit CEA or CCA results without an accompanying QALY-based CUA are routinely asked to supply one. HTAIn's (Health Technology Assessment in India) reference case similarly mandates a QALY-based cost-utility analysis as the base-case outcome measure for the cost-effectiveness threshold it applies, placing India alongside NICE in the CUA-as-default camp despite very different thresholds and population contexts.

Common pitfall. Do not treat "cost-effectiveness analysis" as a synonym for the whole field. In the strict Drummond taxonomy, CEA specifically denotes natural-units analysis, and CUA (QALY-based) is a distinct, narrower category. Many papers and even some agency documents use "cost-effectiveness" loosely to mean "any formal economic evaluation" — when reading or writing a submission, check the actual outcome measure used before assuming which type is meant.

4.3 QALYs, DALYs and life-years

Three outcome metrics recur throughout HEOR, and each solves the "how do we weigh life" problem differently. Historically, the simplest and oldest is the **life-year**: a count of time lived, with no adjustment at all for the quality of that time. A treatment that extends survival by one year scores exactly the same whether that year is spent in full health or bedridden in severe pain. Life-years remain useful for mortality-dominated contexts (vaccination programmes, acute life-saving interventions) and for sensitivity analysis, but their indifference to quality of life is precisely the gap the QALY was built to close.

The **quality-adjusted life year (QALY)** combines duration of time in a health state with a **utility weight** reflecting how good or bad that state is. Formally:

$$\text{QALY} = \sum_i t_i \times u_i$$

where t_i is the time (in years) spent in health state i, and u_i is the utility weight for that state on a scale anchored at $u = 1$ for full health and $u = 0$ for dead. Crucially, the scale is not bounded below at zero: health states judged worse than death (severe, unremitting pain with no prospect of recovery, for example) can carry negative utility weights, reflecting that a respondent would trade some of their remaining life expectancy to avoid that state entirely, or would prefer death to it. Utility weights are typically elicited using standardized preference-based instruments (EQ-5D, SF-6D, HUI) administered to patients or to general-population samples, then converted to utilities via a country-specific value set derived from techniques such as time trade-off or standard gamble.

Difficult terminology: time trade-off / standard gamble. Both are standard techniques for turning "how bad is this health state" into a number between 0 (dead) and 1 (full health) that a QALY calculation can use — they ask a respondent (patient or member of the public) to express that judgement through a concrete choice, rather than simply rating the state on an abstract scale. **Time trade-off** asks: given a fixed number of years to live in the impaired health state being valued, how many of those years would you be willing to give up in exchange for the remaining years being in full health instead? Someone willing to trade 2 years out of 10 to be fully healthy for the remaining 8 implies a utility of 0.8 for that state — the less of their remaining life they'd sacrifice, the closer to 1.0 (full health) the implied utility. **Standard gamble** instead offers a choice between the impaired state for certain, versus a gamble: a probability p of full, instant recovery to perfect health, and a probability $(1 - p)$ of immediate death. The respondent's indifference point — the probability p at which they can no longer decide whether they prefer the certain impaired state or the gamble — is taken as the utility value of that impaired state. A concrete worked example: if a respondent says

they would accept a 25% chance of death in exchange for a 75% chance of being cured, rather than continuing to live with their current health problem for certain, their utility for that health state is read off as 0.75. Both methods are more cognitively demanding than simply asking someone to rate a state "out of 10," but are grounded in a formal economic theory of choice under uncertainty (standard gamble) or time preference (time trade-off), which is why country-specific EQ-5D value sets are built from large population samples answering one or the other.

```
# Simple QALY calculation across two health states
time_in_full_health   <- 2.0   # years
utility_full_health   <- 1.0

time_in_impaired_state <- 3.0  # years
utility_impaired_state  <- 0.6

qalys <- time_in_full_health * utility_full_health +
         time_in_impaired_state * utility_impaired_state
qalys
```

Two years at full health (utility 1.0) contribute 2.0 QALYs; three further years at a utility of 0.6 (reflecting, say, moderate chronic pain and reduced mobility) contribute 1.8 QALYs; the patient's total is 3.8 QALYs over five calendar years of survival — a single number that simultaneously encodes both how long and how well the patient lived. An incremental analysis compares the QALYs accumulated under a new technology against the QALYs accumulated under the comparator, and that QALY difference becomes the denominator of the ICER introduced in Chapter 1. The QALY is explicitly a **good**: more QALYs is always better, and interventions are evaluated by how many QALYs they gain.

The **disability-adjusted life year (DALY)**, by contrast, comes from a different tradition entirely — the WHO/World Bank Global Burden of Disease programme, designed to quantify population-level disease burden rather than to evaluate individual health technologies. A DALY is constructed as:

$$\text{DALY} = \text{YLL} + \text{YLD}$$

where YLL (years of life lost) captures premature mortality relative to a reference life expectancy, and YLD (years lived with disability) captures time lived in a less-than-full-health state, weighted by a disability weight (conceptually similar to, but methodologically distinct from, a QALY utility weight, and typically derived from large multi-country disability-weight surveys rather than patient-elicited utilities). The DALY is explicitly a **bad**: it measures burden, and the policy goal is to minimize DALYs, not maximize them. An intervention's benefit in DALY terms is expressed as "DALYs averted" — the reduction in burden the intervention achieves relative to no intervention — which is a positive good, even though the underlying DALY count itself represents harm.

Agency perspective. WHO's Global Burden of Disease framework uses the DALY as its standard metric for quantifying population-level disease burden and informing global health priority-setting; this is a distinct convention from, and should not be conflated with, the QALY-based cost-utility reference cases used by technology-appraisal HTA bodies such as NICE and HTAIn. Some national HTA systems (several Asian and a number of low- and middle-income country adaptations) do permit DALY-averted as an acceptable alternative effect measure to

the QALY in cost-effectiveness submissions, so the choice between QALY and DALY conventions is itself a jurisdiction-specific methodological decision to check in any agency's reference case before modelling.

Common pitfall. QALYs and DALYs run in **opposite** directions, and mixing up the sign convention is a common and consequential modelling error. A QALY is a good: a successful treatment should **increase** total QALYs relative to the comparator, and the ICER is cost divided by QALYs **gained**. A raw DALY is a burden: a successful treatment should **decrease** total DALYs, and the relevant effectiveness quantity for a cost-effectiveness ratio is DALYs **averted** (comparator DALYs minus intervention DALYs), not the raw DALY count itself. Coding a model that subtracts intervention QALYs from comparator QALYs (the DALY-style direction) — or divides cost by raw DALYs accrued rather than DALYs averted — will silently produce an ICER with the wrong sign or the wrong magnitude. When porting a published DALY-based burden estimate into an R cost-effectiveness model, explicitly convert it to "DALYs averted" and sanity-check the sign before it ever reaches your ICER formula.

4.4 ICERs, net monetary benefit (NMB) and net health benefit (NHB)

Chapter 1 introduced the incremental cost-effectiveness ratio in its canonical form, $\text{ICER} = \frac{\Delta C}{\Delta E}$, where $\Delta C = C_2 - C_1$ is the incremental cost of a new treatment (treatment 2) relative to a comparator (treatment 1), and ΔE is the incremental effect, almost always measured in quality-adjusted life years (QALYs) in HTA work. The ICER answers a narrow but useful question: how much extra does it cost, per unit of extra health gained, to adopt the new treatment instead of the old one?

Consider a worked example. A new oral anticoagulant costs £1,800 per patient per year versus £400 for standard warfarin monitoring, and modelling over a patient's lifetime projects 0.28 additional QALYs for the new drug due to fewer strokes and bleeds.

```
cost_new <- 1800 * 12   # simplified lifetime cost proxy, GBP
cost_comp <- 400 * 12
qaly_new <- 8.42
qaly_comp <- 8.14

delta_C <- cost_new - cost_comp
delta_E <- qaly_new - qaly_comp
icer <- delta_C / delta_E
icer
```

This returns an ICER of £60,000 per QALY gained (ΔC = £16,800, ΔE = 0.28) — a figure a reimbursement committee can compare directly against a threshold (§4.5). At a £20,000/QALY threshold, this drug would not be considered cost-effective on this evidence.

The ratio format, however, conceals as much as it reveals. Two well-known problems arise:

- **Sign ambiguity.** A negative ICER is uninformative on its own: it can arise from a treatment that is more effective and cheaper (dominant — unambiguously good), or from one that is less effective

and more expensive (dominated — unambiguously bad). Both quadrants of the cost-effectiveness plane produce the same algebraic sign, so the ICER alone cannot distinguish "obviously adopt" from "obviously reject."

- **Non-monotonicity near zero.** As ΔE approaches zero from either side, the ICER magnitude explodes toward infinity even for small, economically unimportant differences in cost, making ratios unstable for ranking or for probabilistic sensitivity analysis summaries.

Common pitfall. A reviewer reports: "Drug X is dominated by Drug Y because the ICER is negative (-£2,500/QALY)." This is only sometimes true. Plot the four quadrants: in the **north-west** quadrant (more effective, less costly — Drug X dominant), $\Delta C < 0$ and $\Delta E > 0$, giving a negative ratio that means "adopt." In the **south-east** quadrant (less effective, more costly — Drug X dominated), $\Delta C > 0$ and $\Delta E < 0$, also giving a negative ratio, but meaning "reject." Never report a negative ICER without stating which quadrant it comes from; always accompany it with the disaggregated ΔC and ΔE signs, or better, switch to net benefit.

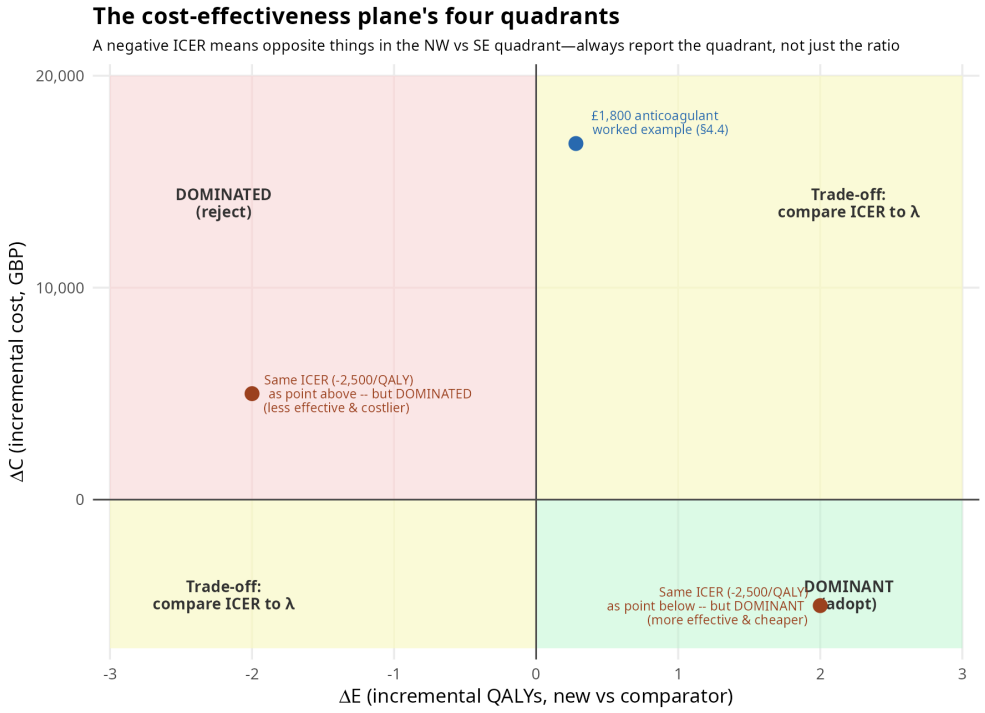


Figure 3: The cost-effectiveness plane's four quadrants. The two orange points share the identical ICER (-£2,500/QALY) but sit in opposite quadrants with opposite real-world meanings — dominant (adopt) versus dominated (reject) — exactly the ambiguity the pitfall above warns against. The blue point plots this section's own anticoagulant worked example (ΔE = 0.28, ΔC = £16,800), which falls in the north-east trade-off quadrant where neither sign alone resolves the decision and the ICER (or NMB) must actually be compared against a threshold.

The standard remedy is to convert the ratio into a linear measure by fixing a willingness-to-pay (WTP) threshold, λ, for one additional QALY. Two equivalent formulations exist:

$$\text{NMB} = (\Delta E \times \lambda) - \Delta C$$

$$\text{NHB} = \Delta E - \frac{\Delta C}{\lambda}$$

NMB expresses net benefit in monetary units; NHB expresses it in health units (QALYs). Because both are linear in ΔC and ΔE, they are well-behaved however small the denominator would have been, they aggregate cleanly across subgroups or probabilistic simulation draws, and — critically — the decision rule becomes unambiguous: **a positive NMB (or NHB) at threshold λ means the intervention is cost-effective at that threshold; negative means it is not.** No quadrant-reading is required.

```
lambda <- 20000   # GBP per QALY, illustrative threshold
nmb <- delta_E * lambda - delta_C
nhb <- delta_E - delta_C / lambda
nmb; nhb
```

With ΔE = 0.28 and ΔC = £16,800: NMB = 0.28 × £20,000 − £16,800 = £5,600 − £16,800 = −£11,200. NMB is negative, so at a £20,000/QALY threshold this drug is **not** cost-effective — exactly the same conclusion the £60,000/QALY ICER already pointed to (£60,000 exceeds the £20,000 threshold). NMB and the ICER decision rule are always mathematically consistent for a single comparator when $\Delta E > 0$ and λ is fixed; NMB's real advantage is not a different conclusion but a **linear**, sign-unambiguous one that remains well-defined even when ΔE is close to zero, which is precisely why it is the quantity averaged across probabilistic sensitivity analysis draws rather than the ICER itself.

How to report this.

Treatment	Cost (GBP)	QALYs	Incremental cost	Incremental QALYs
Standard warfarin monitoring	4,800	8.14	–	–
New oral anticoagulant	21,600	8.42	16,800	0.28
ICER = £60,000 per QALY gained			**NMB** at λ = £20,000/QALY = −£11,200 (not cost-effective at this threshold)	

Table 6: A CHEERS 2022-style cost-effectiveness results table for the anticoagulant worked example — the standard way an ICER/NMB result is reported in a paper or HTA submission, rather than left as bare console output.

HEOR in practice. In practice, NMB is the default output of a Bayesian or bootstrap probabilistic sensitivity analysis: for each of, say, 10,000 simulated parameter draws, compute NMB at the threshold of interest, then summarise the proportion of draws with positive NMB as the "probability cost-effective." This is the basis of cost-effectiveness acceptability curves (CEACs), covered in Chapter 25. Reporting an ICER confidence interval from simulated draws is often nonsensical (ratios of near-zero denominators), whereas an NMB distribution is always well-defined.

4.5 Willingness-to-pay thresholds, including country-specific and LMIC thresholds

A willingness-to-pay (WTP) or cost-effectiveness threshold, denoted λ in §4.4, represents the maximum amount a payer is prepared to pay for one additional unit of health gained — conventionally one QALY. It is the single parameter that converts an ICER (or NMB/NHB) from a descriptive statistic into an actionable reimbursement decision: if ICER $< \lambda$, the intervention is judged cost-effective at that threshold; if ICER $> \lambda$, it is not (subject to the quadrant caveats of §4.4).

The most widely cited threshold range in the HTA literature is NICE's (England and Wales) historical £20,000–£30,000 per QALY range, used since the early 2000s as an informal reference point in appraisals, with values above £30,000 requiring increasingly strong justification (e.g., end-of-life criteria, severity modifiers) and values below £20,000 treated as straightforwardly acceptable. It is worth being explicit that this range was never a hard legislative threshold, was not derived from a formal empirical exercise at the time of adoption, and has been persistently criticised — including by health economists associated with NICE's own methods reviews — as arbitrary and not clearly linked to the opportunity cost of displaced NHS spending. Claxton and colleagues' empirical work estimating the NHS's own marginal productivity (health forgone per pound spent elsewhere in the system) suggested a supply-side threshold closer to £12,000–£15,000 per QALY, considerably below the £20,000–£30,000 range, triggering substantial debate about whether NICE's reference range was too generous.

> **Agency perspective.** NICE's current reference case for threshold use and modifiers is set out in **NICE Health Technology Evaluations: The Manual** (PMG36, 2022), which retains the £20,000–£30,000/QALY range for standard appraisals, allows an additional severity weighting (replacing the older "end-of-life" criteria) for conditions with greater absolute or proportional QALY shortfall, and directs analysts to NICE DSU technical support documents for the quantitative severity-modifier calculations. Students producing a submission for NICE should cite the Manual directly, not a secondary paraphrase of the threshold range.

For low- and middle-income countries (LMICs), the historically dominant heuristic came from the WHO's Choosing Interventions that are Cost-Effective (WHO-CHOICE) programme, which proposed classifying interventions as "very cost-effective" below one times GDP per capita per QALY/DALY averted, and "cost-effective" between one and three times GDP per capita, with anything above three times GDP per capita judged not cost-effective. This 1–3x GDP-per-capita rule was administratively convenient and widely adopted in early LMIC HTA guidance, but it has been substantially abandoned by the WHO itself: a widely cited 2016 WHO Bulletin piece (Bertram et al., "Cost-effectiveness thresholds: pros and cons") explicitly recommended retiring the GDP-multiple heuristic, arguing it has no sound empirical basis, is unrelated to a country's actual health budget or opportunity cost, and tends to label far too many interventions as "cost-effective" relative to what health systems can actually afford.

> **Agency perspective.** WHO's updated position, reflected in WHO-CHOICE guidance and the 2016 Bulletin of the World Health Organization commentary by Bertram, Lauer, Joncheere, Edejer, Baltussen and colleagues, favours deriving thresholds from the **opportunity cost of displaced health spending within the country's own budget** — analogous to Claxton's

NHS-marginal-productivity approach — rather than an arbitrary GDP multiple or a stated willingness-to-pay survey figure.

This opportunity-cost framing sits opposite a second methodological tradition: stated- or revealed-preference willingness-to-pay surveys, which ask members of the public how much they would personally pay (or how much compensation they would require) for a given health gain, typically yielding higher thresholds than supply-side opportunity-cost estimates because they reflect individual valuation of health rather than a health system's binding budget constraint. The debate between these two schools — opportunity cost (what health is displaced if we fund this) versus willingness-to-pay (what society says health is worth) — remains unresolved and is one of the most active areas in health economics methods research; neither is "correct" in an absolute sense, and the choice has first-order consequences for which technologies get funded.

Difficult terminology: revealed preference. "Stated preference" (used just above, and already described elsewhere in this section) means directly asking people what they would pay or accept — a survey. **Revealed preference** takes the opposite approach: instead of asking, it infers how much people value something by observing the actual trade-offs they make in real decisions. The classic example underlying value-of-a-statistical-life estimates (§4.2) is the "wage premium" study: researchers compare otherwise similar jobs that differ mainly in their on-the-job fatality risk (construction versus office work, for instance) and observe how much extra pay workers demand, on average, to accept the riskier job. That observed wage gap, scaled by the size of the risk difference, reveals an implicit price workers are putting on their own safety through real choices they actually made — not through a hypothetical questionnaire answer. The general principle extends beyond wages: any real-world choice where people visibly trade one thing for another (buying a safer but more expensive car, choosing a shorter but costlier commute) can, in principle, reveal an implicit valuation, provided the choice is genuinely free and the trade-off can be isolated from other confounding factors.

India's HTA body, HTAIn (Health Technology Assessment in India), operationalised a country-specific threshold rather than adopting the WHO GDP-multiple rule.

Agency perspective. HTAIn's **Methods and Processes Manual** (operationalised through the Department of Health Research) and subsequent HTAIn-commissioned work (notably India-specific cost-effectiveness threshold studies led by health economics teams around 2018–2021) estimated a supply-side, opportunity-cost-based threshold substantially below one times India's per-capita GDP, reflecting the real resource constraints of India's public health budget rather than a generic multiple. Analysts preparing a submission for HTAIn should reference the current HTAIn threshold value published in its manual, not the superseded WHO 1–3x GDP heuristic.

HEOR in practice. When a jurisdiction's own threshold is unclear or evolving, report results across a **range** of thresholds in a cost-effectiveness acceptability curve (Chapter 25) rather than committing to a single λ. This is standard practice precisely because threshold-setting is contested, and it lets decision-makers apply their own judgement.

Jurisdiction / body	Threshold convention	Approximate value	Methodological basis
NICE (England & Wales)	Fixed reference range, severity-modified	£20,000–£30,000 per QALY	Historical reference range; no formal empirical derivation; Claxton et al. supply-side estimate suggests £12,000–£15,000
CADTH (Canada)	No single published hard threshold	Often benchmarked informally around CAD 50,000 per QALY in practice	Deliberative committee judgement rather than a codified rule; threshold not fixed in CADTH's own methods guidance
HTAIn (India)	Country-specific supply-side threshold	Below 1 times India's per-capita GDP	Opportunity-cost-based estimate from India-specific threshold studies (~2018–2021), not the WHO GDP-multiple rule
WHO-CHOICE (historical)	GDP-per-capita multiple	1x GDP/capita = "very cost-effective"; 1–3x = "cost-effective"	Administrative heuristic; formally recommended for retirement by Bertram et al. 2016 (WHO Bulletin)

Note: The GDP-multiple rule is retained here only as a historical reference point; current WHO guidance favours country-specific opportunity-cost thresholds, consistent with the HTAIn and Claxton/NICE supply-side approaches above.

Table 7: Willingness-to-pay / cost-effectiveness threshold conventions across selected HTA jurisdictions — the kind of comparison table a methods section uses to justify which threshold a submission is benchmarked against.

4.6 Perspective, time horizon, discounting and half-cycle correction

Four further modelling choices — perspective, time horizon, discounting, and (in Markov models) half-cycle correction — jointly determine what counts as a cost or effect, over what period, and how future values are weighted. Each is a source of real controversy and, unlike the ICER/NMB arithmetic above, each requires an explicit judgement call that should be justified and reported transparently.

Perspective defines whose costs and whose effects enter the analysis. The two most common are:

- **Healthcare payer (or healthcare system) perspective**: includes only costs borne by the payer — drug acquisition, administration, monitoring, hospitalisation, adverse event management — and excludes costs falling on patients, families, or other sectors.
- **Societal perspective**: additionally includes productivity losses (time off work, informal caregiver time), patient out-of-pocket costs, transport to appointments, and sometimes costs in other sectors (e.g., social care, criminal justice for some public health interventions).

The choice matters substantially for conditions with large informal-care or productivity burdens (e.g., dementia, severe mental illness): a healthcare-payer analysis can understate the true value of an intervention that reduces caregiver burden even if it does little to reduce direct medical costs. NICE's reference case specifies an NHS-and-Personal-Social-Services perspective for costs (narrower than full societal) paired with QALYs measured across all effects on the patient, while PBAC and others have historically leaned payer-perspective with societal perspective as a supplementary sensitivity analysis.

Time horizon is the period over which costs and effects are modelled. The general principle, stated in essentially all major HTA methods guides, is that the horizon should be long enough to capture **all** costs and effects relevant to the decision — for a one-off curative intervention this might be a few years, but for a chronic, progressive disease (diabetes, chronic kidney disease, most oncology indications) a **lifetime horizon** is typically required, because truncating too early can systematically understate long-run benefits (e.g., avoided complications decades later) or long-run costs (ongoing drug costs, disease progression costs).

> **Common pitfall.** A common and costly modelling error is choosing a short time horizon (e.g., a trial's own follow-up period, often 1–3 years) simply because that is what outcome data are directly observed for, rather than because it genuinely captures the decision-relevant period. For a chronic disease, this almost always biases the ICER against interventions whose main benefit accrues late (e.g., prevention of long-term complications), because the model stops before those benefits are realised while some of the upfront costs are already fully counted. Extrapolation beyond trial follow-up (Chapter 10 and Chapter 19) is then unavoidable and must be justified, not treated as an afterthought.

Discounting addresses the fact that costs and health effects occurring in the future are valued less than those occurring today, reflecting positive time preference (both for money, due to its opportunity cost, and arguably for health itself). A future cost or QALY is converted to its present value by dividing by $(1 + r)^t$, where r is the discount rate and t is the number of years in the future the cost or effect occurs.

```
discount_pv <- function(value, rate, year) {
  value / (1 + rate)^year
}

# A QALY gained in year 10, discounted at NICE's 3.5% rate
discount_pv(value = 1, rate = 0.035, year = 10)
```

> **Agency perspective.** NICE's **Health Technology Evaluations: The Manual** (PMG36, 2022) specifies a discount rate of 3.5% per annum for both costs and health effects as the reference-case base case, consistent with UK Treasury Green Book conventions for public-

sector appraisal, with sensitivity analysis at 1.5% for health effects permitted only in narrowly defined circumstances (very long-term, substantial health gains). This "equal discounting" of costs and QALYs at the same rate is itself a resolved policy choice following an extended methodological debate.

That debate concerns **differential discounting**: some health economists (echoing earlier work by Claxton and colleagues, and reflected in US Panel on Cost-Effectiveness in Health and Medicine recommendations at various points) have argued health effects should be discounted at a **lower** rate than costs, because health does not have the same investment/opportunity-cost properties as money and because discounting health heavily can appear to penalise prevention relative to treatment. Most national HTA bodies, however, have settled on equal discounting of costs and effects at a single rate (3.5% at NICE; comparable single-rate conventions are used by CADTH and others) for consistency and to avoid distortions where differential rates can make almost any sufficiently long-term prevention programme appear cost-saving regardless of its actual value.

HTA body	Reference-case perspective	Time horizon	Discount rate (costs / effects)
NICE (England & Wales)	NHS and Personal Social Services	Long enough to capture all relevant costs/effects; typically lifetime for chronic disease	3.5% / 3.5% (equal); 1.5% on effects only in narrow long-term, large-benefit cases
CADTH (Canada)	Publicly funded health care payer (base case); societal as sensitivity analysis	Lifetime horizon for chronic/progressive conditions	1.5% / 1.5% (equal, per current CADTH guidance)
HTAIn (India)	Health system / payer perspective	Lifetime horizon recommended for chronic conditions	3% / 3% (equal), per HTAIn Methods and Processes Manual

Note: Values shown are reference-case defaults; most agencies also require sensitivity analyses varying perspective, horizon, and discount rate from these defaults. Consult each agency's current methods manual before a live submission – rates and conventions are periodically revised.

Table 8: Perspective, time horizon and discount rate reference-case conventions across selected HTA bodies — the modelling-choices summary a methods section typically tabulates before presenting results.

Half-cycle correction is a technical adjustment specific to Markov cohort models (introduced formally in Chapter 19), noted briefly here because it interacts directly with discounting and time horizon. A Markov model tracks a hypothetical cohort moving between health states in discrete cycles (e.g., annual cycles), and by convention accumulates costs and QALYs as if the entire cohort occupies its cycle-start health state for the whole cycle. In reality, transitions between states (e.g., disease progression, death) occur continuously throughout the cycle, so a naive cycle-start (or cycle-end) accounting overstates (or understates) time spent in each state by roughly half a cycle's

worth. Half-cycle correction adjusts for this by assuming, on average, that transitions occur at the cycle midpoint — commonly implemented as a simple average of the cycle-start and cycle-end state occupancy, or via Simpson's-rule-weighted approaches for greater accuracy with longer cycle lengths. The correction typically has a modest but non-trivial effect on both costs and QALYs, and omitting it is a recognised, checkable error in submitted economic models. Full mechanics, including worked trace calculations and the Simpson's-rule variant, are deferred to Chapter 19′s dedicated treatment of cohort Markov models in R.

Difficult terminology: Simpson's rule (half-cycle correction variant). Simpson's rule is a general numerical technique, unrelated to health economics specifically, for estimating the area under a curve more accurately than a plain average would. The simplest "half-cycle" correction described just above averages only two points per cycle — the state occupancy at the cycle's start and at its end — effectively approximating the true, continuously-curving occupancy over that cycle with a straight line between those two points. Simpson's rule improves on this by also using the occupancy at the cycle's **midpoint** and fitting a short parabolic (curved) segment through all three points rather than a straight line, which tracks a genuinely curving trend much more closely, particularly when cycles are long or the underlying transition probabilities change quickly. A practical analogy: estimating the distance a car travelled using only its speed at the start and end of an hour (the simple average) assumes it accelerated in a straight line; checking its speed at the halfway mark too, and fitting a smooth curve through all three readings (Simpson's rule), gives a noticeably better estimate if the car sped up and slowed down unevenly during that hour rather than changing speed at a constant rate. In R-based Markov models, this usually means sampling state-occupancy at an extra set of mid-cycle time points and combining them with a specific weighted-average formula, rather than performing any genuinely novel calculus.

HEOR in practice. When reviewing someone else's Markov model, check three things together, since they interact: the time horizon is long enough (ideally lifetime for chronic disease), the discount rate matches the target jurisdiction's reference case (e.g., 3.5% equal discounting for a NICE submission), and half-cycle correction (or an equivalent continuous-time correction) has been applied. Getting the first two right while omitting the third still produces a systematically biased ICER.

4.7 Dominance, extended dominance and the efficiency frontier

Section 4.4 gave you the ICER as a ratio, $\Delta C / \Delta E$, and left a loose thread: an ICER only tells you something useful once you know **which two options** belong on either side of the slash. Compare the wrong pair — a brand-new drug against some arbitrary older comparator that was never really in contention — and the number you compute is technically arithmetic but economically meaningless. This section ties that thread off by introducing the idea of **dominance** and the **efficiency frontier**, the two concepts that tell you which comparisons are legitimate in the first place.

Start with the simplest case. Suppose Treatment A costs more than Treatment B **and** delivers fewer QALYs than Treatment B. A is strictly worse on both axes: no rational decision-maker would ever choose it, regardless of budget or willingness to pay. We say A is **dominated** (specifically, **simply**

or **strictly** dominated) by B, and the correct analytical move is to delete A from the comparison set entirely. You do not compute an ICER for a dominated option — dividing a positive cost difference by a negative effect difference produces a number that looks like a ratio but describes a trade nobody would make. Simple dominance is easy to spot on a cost-effectiveness plane (cost on the vertical axis, effect on the horizontal axis): A sits up and to the left of B, in the "worse in every way" quadrant relative to B.

Extended dominance is subtler and is the piece most newcomers miss. A treatment can fail to be dominated by **any single alternative** and still be inefficient, because it is dominated by a **mix** of two other treatments. Picture three non-dominated options B, C, and D, ordered by increasing cost and increasing effect, plotted on the cost-effectiveness plane. Draw a straight line connecting B and D. If C's point lies **above** that line — more costly, for its effect, than some hypothetical blend of B and D obtained by giving a fraction of patients B and the rest D — then C is extendedly dominated. A decision-maker could, in principle, achieve C's average effectiveness at lower average cost by mixing B and D (or, more realistically, by simply recognising that C offers a worse incremental trade than the B-to-D step does directly). C is therefore excluded from the efficient set, even though neither B nor D alone beats it on both cost and effect.

This geometric test — lying above the line connecting the two neighbouring non-dominated points — is exactly equivalent to a numerical test you will use constantly from here on: C is extendedly dominated whenever its ICER against the next-cheaper non-dominated option exceeds its ICER against the next-more-expensive one. When ICERs stop increasing monotonically as you move up the cost ranking, you have found an extended-dominance violation, and the offending option drops out.

Once all simply and extendedly dominated options are removed, what remains is the **efficiency frontier**: the set of treatments for which every additional unit of cost buys you additional effect at a non-decreasing rate, when you walk the ranking from cheapest to most expensive. Plotted on the cost-effectiveness plane, the frontier is a piecewise-linear curve connecting these surviving points, convex from below, with ICERs between adjacent segments that rise (or stay flat) as you move rightward. Any point below this curve is infeasible given the available technologies; any point above it (whether literally dominated or extendedly dominated) is inefficient and has already been pruned.

This is why the rule — compare adjacent alternatives, not an arbitrary comparator — is not a stylistic preference but a structural necessity. An ICER is only interpretable as "the price of the next increment of health" when the two options being compared are **adjacent on the frontier**: nothing cheaper-and-nearly-as-effective, and nothing dearer-and-better, sits between them. Comparing a frontier option against a non-adjacent one, or against something that should already have been eliminated, manufactures a ratio with no decision-relevant meaning, however clean the arithmetic looks.

> **Common pitfall.** A persistent error in student work and, occasionally, in submitted dossiers: computing the ICER of a new treatment against "standard of care" without first checking whether standard of care, or the new treatment, has been dominated or extendedly dominated by something else in the comparator set. Always sort by cost, strip out simply dominated options, check for and strip out extendedly dominated options, **then** compute ICERs only between what remains, moving strictly from one adjacent frontier point to the next. An ICER

against a non-adjacent or dominated comparator is not merely imprecise — it is answering a question nobody asked.

Agency perspective. NICE's health technology evaluation manual (PMG36, 2022) is explicit that analyses submitted for appraisal should present a **fully incremental analysis**: all relevant, non-dominated comparators ranked by cost, with ICERs reported between adjacent options on the efficiency frontier, rather than a series of pairwise comparisons against a single fixed comparator chosen by the manufacturer. The ISPOR-SMDM Modeling Good Research Practices Task Force reports (Caro et al., 2012, published in **Value in Health** and **Medical Decision Making**) reinforce the same expectation for decision-analytic models generally: dominated and extendedly dominated strategies must be identified and excluded before any ICER is presented as evidence for reimbursement decision-making.

4.8 Coding a first simple CEA in R

Three chapters of scaffolding — ICER notation, the HTA agency landscape, the four callout conventions you've been reading throughout this book, and a tour of `dplyr`, `purrr`, and function-writing in R — have all been building toward this moment. This is the first section in the entire book where R code actually **performs a health-economic calculation** rather than illustrating a general-purpose programming idiom. Everything from here on builds on the pattern you are about to write: a tidyverse pipeline that takes raw cost and effect data, prunes it down to an efficiency frontier, and reports incremental results. Treat this as a milestone worth lingering on — type it in, run it, and watch the frontier logic from §4.7 fall out of five `dplyr` verbs.

The toy scenario: a hypothetical chronic condition with four candidate treatments — a generic standard of care and three newer options — each characterised, for this illustration, by a single deterministic cost and a single deterministic QALY estimate (we are deliberately ignoring uncertainty for now; Chapter 25 will put proper probabilistic distributions around every one of these point estimates and propagate them through exactly this kind of frontier logic via probabilistic sensitivity analysis).

```
library(dplyr)
library(tibble)

# Step 1: define the toy treatment data ------------------------------
treatments <- tibble(
  treatment = c("Standard Care", "Drug A", "Drug B", "Drug C"),
  cost      = c(1200,            4800,     5100,     9000),
  qaly      = c(0.62,            0.81,     0.74,     0.95)
)

treatments
```

The data frame has one row per treatment and exactly the two columns a cost-effectiveness analysis needs at minimum: `cost` and `qaly`. Note that Drug B costs more than Drug A but delivers **fewer** QALYs — a hint of trouble we'll let the pipeline catch rather than spotting it by eye.

```r
# Step 2: rank treatments from cheapest to most expensive -------------
ranked <- treatments |>
  arrange(cost)

ranked
```

```r
# Step 3: remove simply dominated treatments --------------------------
# A treatment is simply dominated if some cheaper treatment already
# delivers equal or greater QALYs. We flag and drop these before doing
# any incremental arithmetic at all.
remove_simple_dominance <- function(df) {
  df <- df |> arrange(cost)
  keep <- logical(nrow(df))
  best_qaly_so_far <- -Inf
  for (i in seq_len(nrow(df))) {
    if (df$qaly[i] > best_qaly_so_far) {
      keep[i] <- TRUE
      best_qaly_so_far <- df$qaly[i]
    } else {
      keep[i] <- FALSE   # cheaper option already matches/beats this QALY
    }
  }
  df |> filter(keep)
}

non_dominated <- remove_simple_dominance(ranked)
non_dominated
```

Running this on our four rows drops Drug B: it costs more than Drug A (£5,100 vs £4,800) while delivering fewer QALYs (0.74 vs 0.81) — textbook simple dominance from §4.7. Standard Care, Drug A, and Drug C survive, each cheaper-but-worse or dearer-but-better than its neighbour.

```r
# Step 4: check for extended dominance --------------------------------
# A survivor of simple dominance can still be extendedly dominated if its
# ICER against the next-cheaper option exceeds its ICER against the
# next-more-expensive option (i.e. ICERs are not rising as cost rises).
check_extended_dominance <- function(df) {
  df <- df |>
    arrange(cost) |>
    mutate(
      icer_vs_prev = (cost - lag(cost)) / (qaly - lag(qaly))
    )

  repeat {
    icers <- df$icer_vs_prev
    violation <- which(icers < lag(icers))[1]  # ICER drops vs previous step
    if (is.na(violation) || length(violation) == 0) break
    df <- df |>
      filter(treatment != df$treatment[violation]) |>
      mutate(icer_vs_prev = (cost - lag(cost)) / (qaly - lag(qaly)))
  }
  df
}

frontier <- check_extended_dominance(non_dominated)
```

With only three survivors in this toy example, ICERs happen to rise monotonically already (Standard Care → Drug A → Drug C), so no further rows are removed — but the function is written generally, re-checking after every removal, exactly as it would need to for a larger comparator set where extended dominance bites.

```
# Step 5: build the final frontier table of incremental results ---------
results <- frontier |>
  arrange(cost) |>
  mutate(
    incr_cost = cost - lag(cost),
    incr_qaly = qaly - lag(qaly),
    icer      = incr_cost / incr_qaly
  ) |>
  select(treatment, cost, qaly, incr_cost, incr_qaly, icer)

print(results, digits = 2)
```

```
# A tibble: 3 x 6
  treatment      cost  qaly incr_cost incr_qaly   icer
  <chr>         <dbl> <dbl>     <dbl>     <dbl>  <dbl>
1 Standard Care  1200  0.62        NA        NA     NA
2 Drug A         4800  0.81      3600      0.19  18947
3 Drug C         9000  0.95      4200      0.14  30000
```

Read this table exactly as §4.7 told you to: Standard Care is the frontier's starting point (no comparator cheaper than it exists in this set, so its incremental row is `NA`). Moving to Drug A costs an incremental £18,947 per additional QALY. Moving on from Drug A to Drug C — **not** from Standard Care to Drug C — costs an incremental £30,000 per QALY. Drug B never appears because it was eliminated at Step 3, and no row anywhere compares a treatment to a non-adjacent option. That rising sequence of ICERs, £18,947 then £30,000, is the numerical signature of a valid, extended-dominance-checked efficiency frontier.

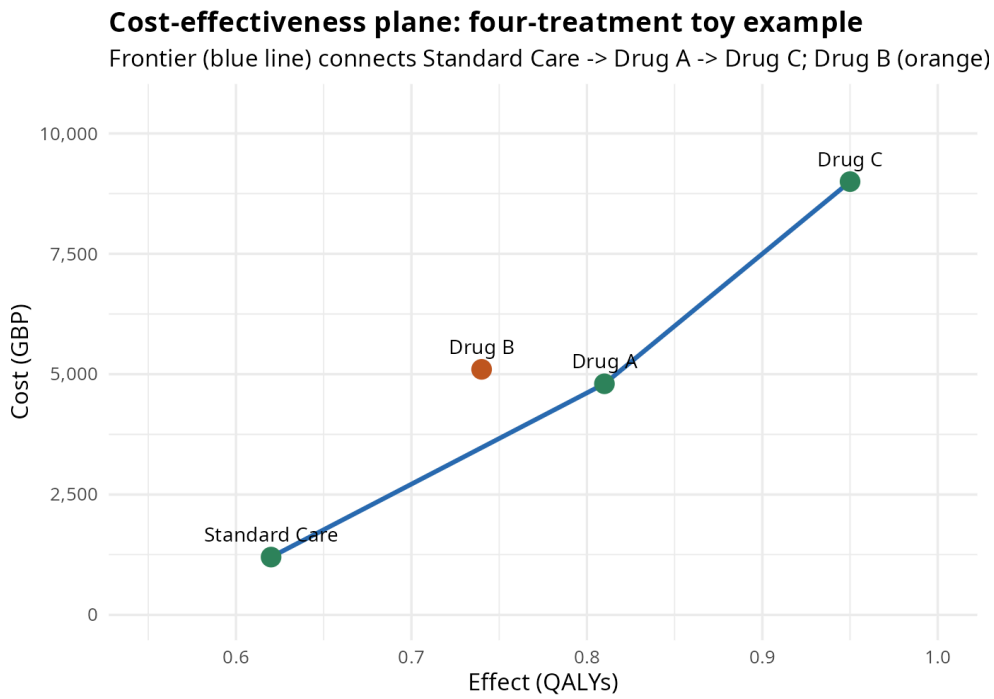


Figure 4: The cost-effectiveness plane for the four-treatment toy example. Drug B (orange) sits above and to the left of Drug A — more expensive, fewer QALYs — and is excluded from the frontier at Step 3. The blue line connects the three surviving, non-dominated treatments in the order the efficiency frontier is built: Standard Care, then Drug A, then Drug C.

How to report this.

Treatment	Cost (GBP)	QALYs	Dominance status	ICER vs next-cheaper (GBP/QALY)
Standard Care	1,200	0.62	On frontier (reference)	–
Drug A	4,800	0.81	On frontier	18,947
Drug B	5,100	0.74	Simply dominated by Drug A (higher cost, fewer QALYs)	Excluded
Drug C	9,000	0.95	On frontier	30,000

Note: ICERs are reported only between adjacent frontier survivors (Standard Care → Drug A → Drug C). Drug B is excluded by simple dominance at Step 3 and never receives an ICER. No extended-dominance violation occurs in this toy data set, since ICERs rise monotonically (£18,947 then £30,000) along the surviving sequence.

Table 9: How the four-treatment frontier analysis would be reported: dominance status, survival of pruning, and the ICER against the next-cheaper non-dominated option.

HEOR in practice. Wrap Steps 3–5 into a single reusable function, say `build_frontier(df)`, that takes any `treatment`/`cost`/`qaly` tibble and returns the pruned, incrementally-costed frontier table. You will reuse this exact pattern — rank by cost, strip simple dominance, strip extended dominance, compute adjacent increments — every time a new decision problem in this book has more than two comparators, including the Markov and partitioned-survival models from Part IV.

Common pitfall. It is tempting to skip straight to `mutate(icer = (cost - cost[1]) / (qaly - qaly[1]))` — comparing everything back to the first row. This silently computes ICERs against a **fixed** baseline rather than the **adjacent** frontier comparator, and it will happily produce a number even for a dominated treatment that should have been deleted at Step 3. Always run the dominance-removal steps first, and always difference against `lag()` on a cost-sorted, already-pruned tibble — never against row 1 by position.

The point estimates in `results` — one cost, one QALY, one ICER per frontier step — are exactly what Chapter 25 will revisit and turn probabilistic: instead of a single `cost = 4800` for Drug A,

you will eventually draw thousands of plausible costs and QALYs from distributions, run this same dominance-and-increment logic on each draw, and summarise the resulting cloud of ICERs with a cost-effectiveness acceptability curve. For now, savor the deterministic version — this five-step `dplyr` pipeline, however small the toy data set, is a complete, decision-grade cost-effectiveness analysis, correctly respecting every rule about dominance and frontiers that §4.7 just laid out.

PART II

Data for HEOR

Chapter 5. Data Sources and Data Management

Learning objectives

- Distinguish CDISC SDTM (collected, domain-organised trial data) from ADaM (analysis-ready derived data), and explain why ADaM datasets are generally preferred as model inputs where available.
- Characterise the distinct strengths, weaknesses and provenance biases of registries, administrative claims, electronic health records, and survey data as real-world data sources.
- Treat published literature as a data source with its own extraction discipline, and explain why publication bias threatens any model parameter pulled from an ad hoc literature search.
- Import health-economic data from flat files, spreadsheets, SAS/Stata/SPSS files, relational databases, and web APIs using `readr`, `readxl`, `haven`, `DBI` and `httr2`.
- Construct and maintain a data dictionary / codebook that documents variable meaning, coding, and derivation logic, including with the `labelled` package.
- Apply programmatic data quality checks (`pointblank`, `assertr`, `validate`) to catch impossible values — negative costs, out-of-range utilities — before they reach a cost-effectiveness pipeline.
- Apply the core concepts of data governance, de-identification, and privacy law (HIPAA, GDPR, India's DPDPA 2023) to patient-level HEOR data handling.

5.1 Clinical trial data (CDISC SDTM/ADaM concepts)

For most HEOR analysts trained in TB programme surveillance or routine public-health M&E, the first encounter with sponsor-provided clinical trial data comes as a jolt. The data are not a single flat line-list you built yourself from a case-based database; they are dozens of interlinked tables governed by a standard you did not write and cannot casually reshape. Understanding that standard — the Clinical Data Interchange Standards Consortium's (CDISC) Study Data Tabulation Model (SDTM) and Analysis Data Model (ADaM) — is a prerequisite for doing economic modelling work that touches regulated trial data, because nearly every phase II/III trial submitted to a regulator in the last fifteen years has been structured this way, and increasingly so are the trials that feed economic submissions to HTA bodies.

SDTM and ADaM solve two different problems, and conflating them is the single most common conceptual error newcomers make. SDTM is the standard for **collected** data: it organises what was observed or recorded during the trial into standardised **domains**, each a rectangular dataset with a controlled set of variables. `DM` holds one row per subject with demographics (age, sex, arm assignment, dates of first/last exposure); `AE` holds one row per adverse event per subject (with standardised terms, severity grades, relatedness, onset/resolution dates); `LB` holds laboratory results in long format (one row per subject per visit per analyte); `EX` holds exposure/dosing records; `VS` holds vital signs; and so on through dozens of other two-letter domain codes. SDTM datasets are deliberately close to raw collection — they are not yet statistically "analysis-ready." A single AE record, for instance, does not by itself tell you whether that event counts toward a "treatment-

emergent adverse event" (TEAE) analysis population, because that determination requires business rules (did the event start after first dose? within a specified window?) that are not SDTM's job to encode.

ADaM is the standard for **analysis-ready** derived data built **from** SDTM, not alongside it or instead of it. The cornerstone ADaM dataset is `ADSL` (Subject-Level Analysis dataset), one row per subject carrying derived flags and population indicators — intent-to-treat flag, safety population flag, treatment duration, baseline disease severity category — that get merged onto every other analysis dataset so that subgroup and population definitions stay consistent across every output in the submission. Built on top of ADSL are domain-specific analysis datasets such as `ADAE` (derived from SDTM `AE` plus ADSL, now carrying the TEAE flag, duration calculations, and toxicity grade shifts) or `ADLB` (derived from SDTM `LB`, now carrying baseline values, change-from-baseline, and shift-table categories attached to each record). The relationship is strictly directional: SDTM feeds ADaM, ADaM feeds statistical analysis and tables/figures/listings, and nothing flows backward. For an HEOR analyst building a cost-effectiveness model off trial data, this matters practically — you want ADaM datasets (particularly ADSL and ADAE/ADTTE-style time-to-event datasets) wherever they exist, because the analysis-ready derivations (populations, flags, time-to-event censoring rules) have already been vetted against the statistical analysis plan. Going back to raw SDTM domains and re-deriving those flags yourself is both more work and a source of discrepancy risk against the trial's own published results.

> **Agency perspective.** CDISC's SDTM and ADaM Implementation Guides are the data standards named explicitly in the FDA's Data Standards Catalog and required for electronic study data submitted to FDA (and, for ICH-aligned submissions, broadly expected by other ICH regulators) for the relevant study start dates covered by that catalog. ICH E3 ("Structure and Content of Clinical Study Reports") separately governs how the **narrative** clinical study report is organised, including appendices that reference the underlying datasets; it does not itself define a data model, but study reports built to ICH E3 are now routinely accompanied by SDTM/ADaM datasets and define-XML metadata. An HEOR analyst requesting trial data from a sponsor's clinical data management or biostatistics group should expect ADaM datasets with accompanying define.xml metadata files describing every variable's derivation — reading that define.xml before touching the data is non-negotiable, because variable names recur across domains with domain-specific meanings.

In practice you will rarely receive true SAS transport (`.xpt`) files directly for ad hoc exploratory modelling work; more often a biostatistics partner exports a derived extract. But knowing the domain structure lets you sanity-check what you're given. A minimal, illustrative read of an ADaM-style subject-level dataset in R might look like this:

```
library(haven)
library(dplyr)

# Read an ADaM subject-level analysis dataset (ADSL) from SAS transport format
adsl <- read_xpt("adsl.xpt")

# ADSL carries one row per subject with population flags already derived
adsl_summary <- adsl |>
  filter(SAFFL == "Y") |>                # safety population flag
  group_by(TRT01P) |>                    # planned treatment, ADaM naming
```

```
convention
  summarise(
    n = n(),
    mean_age = mean(AGE, na.rm = TRUE),
    pct_female = mean(SEX == "F", na.rm = TRUE) * 100
  )

adsl_summary
```

Two ADaM naming conventions are worth internalising early because they recur constantly: variables ending in `FL` are flags (`Y`/`N`/blank), and paired variables like `TRT01P`/`TRT01A` distinguish **planned** treatment (as randomised) from **actual** treatment (as received) — a distinction that matters enormously if your economic model is meant to reflect intention-to-treat effectiveness versus per-protocol or as-treated efficacy assumptions. Getting this pairing wrong is a quiet way to silently misstate your model's analysis population without any error being thrown.

The `adsl_summary` output above is not merely an intermediate check — summarised, by-arm baseline demographics computed straight from ADSL are exactly the content of the "Baseline characteristics" table that opens almost every clinical study report and HTA submission's patient population section.

How to report this.

Treatment arm (TRT01P)	N	Mean age (years)	Female (%)
Placebo	148	61.4	47.3
Standard dose	151	60.8	49.0
High dose	149	62.1	45.6
Safety population (SAFFL = "Y"); age and sex as recorded in ADSL			

Table 10: Baseline demographic summary of the safety population by planned treatment arm (ADSL-derived), the conventional "Table 1" format for clinical study reports and HTA submissions

Common pitfall. Do not merge SDTM domain datasets directly to build your own "ADSL-equivalent" population flags from scratch when ADaM datasets already exist for the trial. SDTM `AE` records, for example, are not pre-filtered to treatment-emergent events, and naively counting all AE rows per arm will systematically overstate adverse event rates relative to the trial's own published safety tables, because it ignores the TEAE windowing logic that the biostatistics team already encoded in `ADAE`. If ADaM datasets are unavailable and you must derive flags from raw SDTM yourself, document every derivation rule and reconcile your subject counts against the trial's CONSORT diagram and published demographic table before proceeding — a mismatch at that stage means your derivation logic is wrong, not the source data.

5.2 Registries, claims, electronic health records and surveys

Outside the tightly governed world of SDTM/ADaM, most of the "real-world" data an HEOR analyst touches comes from four families of sources that look superficially similar — they are all large tables of patient-level information — but differ fundamentally in how they were generated, what they are fit for, and where they will mislead you if you are not careful. The TB-programme instinct to ask "who collected this, for what purpose, and what did they have an incentive to record accurately?" transfers directly here and is, if anything, the single most useful habit you already have.

Disease **registries** are purpose-built, typically prospective, collections of patients with a defined condition, enrolled and followed specifically because they have that condition — a national TB registry, a cancer registry, a rare-disease natural history registry. Because registries are designed around a clinical question, data quality on the variables central to that question (diagnosis confirmation, key outcomes, treatment regimen) tends to be relatively high, and longitudinal follow-up can be genuinely strong. The structural limitation is selection: registry populations are, by construction, the patients who were diagnosed, referred, and consented into the registry, which is rarely a random or representative sample of all patients with the condition — a limitation TB programme analysts will recognise immediately from case-based surveillance versus true population incidence.

Administrative **claims** data — insurance billing records, whether from private payers or public schemes — are generated as a byproduct of the reimbursement process, not for research, and that provenance shapes everything about their strengths and weaknesses. Their great strength is scale and completeness of health-service utilisation: every reimbursed encounter, drug fill, and procedure tends to leave a billing trace, which makes claims data excellent for costing, utilisation patterns, and longitudinal tracking of care pathways across providers — things a single-site registry or trial cannot see. Their great weakness is clinical granularity: claims data record diagnosis and procedure **codes** (ICD, CPT, HCPCS, or local equivalents) attached to a billing event, not direct clinical measurements; there is no lab value, no severity grade, no free-text clinical reasoning, and diagnosis codes are sometimes recorded for billing optimisation rather than clinical precision (a phenomenon generically called "upcoding" that any analyst working with programmatic reporting data will recognise under different names).

Electronic health record (EHR) data sit at nearly the opposite end of the granularity spectrum from claims. EHRs capture rich clinical detail — vital signs, lab trends, clinician notes, imaging reports — generated at the point of care rather than at the point of billing. That richness is also the problem: EHR data are notoriously messy, heterogeneous across institutions and vendor systems, often only locally structured (a lab value field may be clean while diagnosis information lives only in unstructured clinical text requiring natural-language processing to extract), and plagued by interoperability failures when trying to link a patient's record across different health systems or care episodes. Distinguishing "value is genuinely missing" from "value was never measured at this encounter" from "value was recorded somewhere in free text we haven't extracted" is a recurring, unglamorous task in any EHR analysis.

Survey data — whether a dedicated health-utility survey, a national health interview survey, or a patient-reported-outcomes substudy — rely on self-report, and inherit all the familiar biases of self-report instruments: recall bias, social-desirability bias, non-response bias (who agrees to answer a 45-minute survey is rarely a random sample of the target population), and the general gap between

what a patient reports and what a clinical record would show. Surveys remain indispensable, however, for exactly the outcomes that claims and EHR data structurally cannot capture — health-related quality of life, utility weights, out-of-pocket spending, and patient experience — which is precisely why so much of the QALY-generating utility data you'll use in cost-effectiveness modelling (Chapter 8 onward) originates from survey instruments rather than administrative records.

- **Registries**: prospective, condition-defined enrolment; higher data quality on core variables; selected, non-representative populations.
- **Claims**: large-volume, billing-code-based; excellent for utilisation/cost; no direct clinical detail, coding driven partly by reimbursement incentives.
- **EHR**: rich clinical granularity at point of care; messy, heterogeneous, interoperability-limited; extensive missing/unstructured data.
- **Surveys**: only source for genuine self-reported outcomes (utilities, QoL, experience); subject to recall, social-desirability, and non-response bias.

The four source families map onto distinct HEOR modelling use cases rather than being interchangeable, and that mapping is itself the kind of hierarchy a methods section should state explicitly when justifying a parameter's source.

Source type	Typically well suited to	Typically poorly suited to
RCT (SDTM/ ADaM)	Relative treatment effect, trial-period safety and efficacy	Long-term extrapolation beyond follow-up, real-world adherence
Registry	Natural history, long-term outcomes in diagnosed patients	Population-level incidence, undiagnosed/unreferred patients
Administrative claims	Resource use, costing, longitudinal care pathways	Clinical severity, lab-confirmed diagnosis, quality of life
EHR	Clinical event detail, lab trends, comorbidity capture	Cross-system linkage, structured utility/cost extraction
Survey	Health utilities, QALY weights, patient-reported outcomes	Resource-use costing, objective clinical endpoints
Published literature	Comparative effect sizes absent a head-to-head trial, background rates	Patient-level heterogeneity, population-specific calibration

Table 11: Fit-for-purpose mapping of real-world and trial-derived data source types to common HEOR model inputs

A practical first step with any of these four source types in R is the same regardless of provenance: profile missingness and coding structure before doing anything substantive, because the failure modes above show up as specific, checkable patterns in the data.

```
library(dplyr)
library(purrr)

# Generic first-pass profiling for a real-world data extract,
# whether claims, EHR, or registry sourced
profile_source_data <- function(df) {
  map_dfr(names(df), function(col) {
    x <- df[[col]]
```

```
    tibble(
      variable = col,
      pct_missing = round(mean(is.na(x)) * 100, 1),
      n_distinct = n_distinct(x, na.rm = TRUE),
      type = class(x)[1]
    )
  }) |>
    arrange(desc(pct_missing))
}

claims_profile <- profile_source_data(claims_extract)
head(claims_profile, 10)
```

A column with near-zero missingness but a suspiciously small set of distinct values in a claims extract is often a diagnosis code field clustered around a handful of billable categories; a column with 40% missingness in an EHR extract is often a lab value only ordered for a clinical subgroup, not a value missing-at-random — and that distinction has direct consequences for whether later imputation in your model is defensible (a topic picked up again in Chapter 7′s treatment of missing data).

HEOR in practice. When a sponsor or health-system partner hands you a "real-world evidence" extract without documentation, do not take the source label (registry, claims, EHR) at face value — check it against the data's own fingerprints. Billing-code-only fields with no accompanying clinical values and a visit-level grain strongly suggest claims; free-text or heavily missing structured fields alongside visit timestamps suggest EHR; a small number of tightly curated outcome variables with strong longitudinal completeness suggest a purpose-built registry. Getting the provenance right determines which biases you need to defend against in your eventual HTA submission — reviewers will ask.

Agency perspective. NICE's Health Technology Evaluations Manual (PMG36, 2022) explicitly expects companies to justify the choice and quality of real-world data sources used in an evidence submission, including describing data provenance, completeness, and potential biases relative to the decision problem — a registry, claims extract, or EHR cohort cannot simply be asserted as representative of the UK clinical population without supporting justification. Equivalent expectations about real-world evidence transparency and fitness-for-purpose appear in FDA's real-world evidence guidance documents for regulatory decision-making. Analysts should document data source, extraction logic, and known limitations as rigorously as they would document a trial's SDTM/ADaM provenance.

5.3 Published literature as a data source

Not every input to an economic model comes from a dataset you can open in R at all. A large share of the parameters in any cost-effectiveness model — relative treatment effects when no head-to-head trial exists, long-term survival extrapolation anchors, utility decrements for specific health states, background mortality and incidence rates for comparator arms — come instead from the published literature: individual trial publications, systematic reviews, and meta-analyses. Treating published literature as a **data source** in its own right, with its own collection, extraction, and

quality-appraisal discipline, is as important as treating a claims extract or registry with appropriate scepticism about provenance.

The starting point is usually a systematic literature review (SLR), which Chapter 6 covers in depth as a process; what matters here is recognising why that formal process exists rather than simply Google Scholar–searching for a few convenient trials. An ad hoc literature search is vulnerable to exactly the selection problems that plague any non-random sampling exercise: you are more likely to find, read, and cite papers that are highly cited, recently published, available in open access, and — crucially — that report positive, statistically significant results. This last point is the heart of **publication bias**: trials and studies with null or unfavourable results are systematically less likely to be published at all, or are published after longer delay, in lower-visibility journals, or with selective outcome reporting that emphasises secondary endpoints when the primary endpoint was negative. A model built by pulling point estimates from whichever three papers surfaced first on a search is at serious risk of systematically overestimating treatment effect, precisely because positive results are overrepresented in what gets published and found.

Meta-analysis, covered properly in Chapter 11, is the statistical machinery for pooling effect estimates across multiple studies once a systematic review has identified them, and it exists partly as a defence against the limitations of any single trial's precision — but it is not a defence against publication bias itself; a meta-analysis of a biased set of published trials simply produces a precisely wrong pooled estimate. Funnel plots, Egger's test, and trial registry cross-checks (comparing published results against pre-registered trial protocols on registries such as ClinicalTrials.gov) are the standard tools for detecting whether publication bias is likely operating in a given evidence base, and an HEOR analyst pulling a hazard ratio from a published meta-analysis for use in a model should check whether the authors examined this at all before trusting the pooled number uncritically.

> **Difficult terminology: funnel plot / Egger's test.** Both are ways of checking whether a meta-analysis's pooled evidence base looks suspiciously incomplete — specifically, whether small studies with unfavourable results seem to be missing. A **funnel plot** is a scatter chart with each study's estimated effect size (e.g. a risk ratio) on one axis and a measure of that study's precision (often sample size, or the inverse of its standard error) on the other. If publication bias is absent, the plot should look like a roughly symmetric, upside-down funnel: small, imprecise studies scatter widely on both sides of the pooled average, while large, precise studies cluster tightly near it. If small studies showing an unfavourable or null result are more likely to go unpublished, the plot becomes visibly lopsided — small studies appear only on the "favourable" side, leaving a gap on the other, like a funnel with one side sheared off. **Egger's test** is the formal statistical counterpart: rather than eyeballing the plot for asymmetry, it runs a regression of each study's effect size against its precision and tests whether the resulting line is significantly tilted away from zero — a significant tilt is read as statistical evidence of the same asymmetry the funnel plot shows visually. Neither test proves publication bias exists (a real, genuine difference between small and large studies can also cause asymmetry), but both are the field's standard first-line checks before trusting a pooled estimate at face value.

In the interim, before Chapter 6's formal SLR methodology, a reasonable working discipline when extracting parameters from literature for a model is to keep a structured, auditable extraction log rather than copying numbers directly into model code — the same instinct that already governs good practice in extracting data from a clinical dataset applies to extracting data from a PDF.

```r
library(tibble)
library(readr)

# A minimal structured extraction log for literature-derived
# model parameters -- auditable, reproducible, and easy to
# cross-check against a later formal SLR in Chapter 6
param_log <- tribble(
  ~parameter,            ~value, ~source_citation,                ~source_type,
~notes,
  "hr_pfs_drugA_vs_soc",   0.62, "Smith et al. 2021, NEJM",         "RCT
publication",  "ITT population, 95% CI 0.48-0.80",
  "utility_progressed",   0.68, "Jones et al. 2019, Qual Life Res", "Utility
study",    "EQ-5D-3L, UK value set",
  "cost_ae_grade3plus",   1450, "National tariff schedule 2023",   "Admin/tariff",
"2023 GBP, not inflated"
)

write_csv(param_log, "literature_parameter_log.csv")
param_log
```

Keeping source type, citation, and context notes alongside each extracted value does two things: it makes the eventual model auditable by a third party (including an HTA reviewer), and it forces you to notice, at extraction time, when a parameter's "source" is actually a secondary citation of a citation rather than the primary study — a surprisingly common and avoidable error.

> **Common pitfall.** Extracting a single point estimate from a published trial or meta-analysis without also recording its confidence interval, the population it applies to, and the follow-up duration is a false economy. When that parameter later needs to flow into a probabilistic sensitivity analysis (Chapter 25), you will either have to re-find the paper under time pressure or, worse, invent a plausible-looking distribution around a number whose true uncertainty you never actually captured. Record the full uncertainty information once, at extraction time, alongside the point estimate.

> **Agency perspective.** NICE's Health Technology Evaluations Manual (PMG36, 2022) requires that evidence submissions describe the search strategy, selection criteria, and quality appraisal used to identify clinical and economic evidence, explicitly so that reviewers can assess the risk of selective or biased evidence identification — a requirement that applies as much to parameters quietly pulled from literature for an economic model as to the clinical evidence base itself. Cochrane's Handbook for Systematic Reviews of Interventions likewise sets out, as the internationally recognised methodological reference, the expectation that a review's search be systematic and its risk-of-bias and publication-bias assessment be reported explicitly rather than assumed absent.

5.4 Importing data: `readr`, `readxl`, `haven`, `DBI`, APIs

A TB programme analyst moving into HEOR work quickly discovers that the hardest part of an analysis is rarely the statistics — it is getting the data into R in a shape you can trust. Cost and outcomes data for HTA submissions arrive in an unglamorous variety of formats: flat files exported

from a surveillance system, Excel workbooks maintained by a finance officer, SAS transport files from a contract research organisation, Stata extracts from a DHS-style household survey, and increasingly, live pulls from a claims warehouse or a public API. This section builds a working toolkit across all five, with an eye toward the habits — explicit column typing, encoding checks, provenance logging — that keep an import pipeline reproducible six months later when a reviewer asks "where did this number come from?"

Delimited and flat files with `readr`

For CSV, TSV, and other delimited files, `readr::read_csv()` is the workhorse. Compared to base R's `read.csv()`, it is faster on large files, guesses column types from a larger sample by default, never silently converts strings to factors, and — critically for reproducibility — prints a column specification you can freeze into the script.

```
library(readr)

# Import quarterly unit-cost data exported from a costing database
unit_costs <- read_csv(
  "data/unit_costs_2023q4.csv",
  col_types = cols(
    facility_id   = col_character(),
    cost_centre   = col_character(),
    unit_cost_usd = col_double(),
    report_date   = col_date(format = "%Y-%m-%d")
  ),
  na = c("", "NA", "N/A", "-99")
)

problems(unit_costs)   # surfaces any parsing failures row-by-row
spec(unit_costs)       # the frozen column specification, paste into the script
```

Pinning `col_types` explicitly rather than accepting `readr`'s guess is the single habit most worth adopting from this section. A silent type mismatch — a patient ID parsed as numeric and losing a leading zero, or a cost column parsed as character because one row contains a stray comma — is exactly the kind of error that survives into a cost-effectiveness model undetected.

> **Common pitfall.** `readr`'s type guessing samples only the first 1,000 rows by default. A numeric-looking ID column that turns alphanumeric at row 1,200, or a currency column where one late row contains `"1,234.56"` with a thousands separator, will parse cleanly up to that point and then silently coerce to `NA` or character from there on. Always pass an explicit `col_types` specification for any file that feeds a submission-facing analysis, and always inspect `problems()` after import — it is empty far less often than you'd expect.

Spreadsheets with `readxl`

Excel remains the lingua franca of health-sector data exchange, and `readxl::read_excel()` reads `.xls` and `.xlsx` without requiring Excel itself or any system dependency — useful on a locked-down analysis server. Its main complications are multi-sheet workbooks, header rows that don't start at row 1, and merged cells that leave ragged blank columns.

```
library(readxl)

# List sheets before committing to one
```

```
excel_sheets("data/registry_extract_2023.xlsx")

registry_raw <- read_excel(
  "data/registry_extract_2023.xlsx",
  sheet = "Enrolment",
  range = "A3:N850",        # skip title rows and a trailing totals row
  col_types = c("text", "text", "date", "numeric", "text",
                "numeric", "numeric", "text", "text", "text",
                "numeric", "text", "text", "numeric"),
  na = c("", "NA", "Not recorded")
)
```

Specifying `range` directly is often more robust than `skip =` plus `n_max =`, because finance and clinical teams frequently insert or delete header rows between workbook versions without warning; a hard-coded range at least fails loudly (wrong number of columns) rather than silently ingesting a title row as data.

Clinical trial and survey formats with `haven`

Clinical trial data exchanged with a CRO, and large household surveys such as DHS or WHO STEPS, are routinely distributed as SAS, Stata, or SPSS files. `haven` reads all three natively and — importantly — preserves value labels and SAS/SPSS "missing" codes as attributes rather than discarding them, which matters a great deal once you reach the codebook work in §5.5.

```
library(haven)

# SAS transport/dataset file (common for SDTM/ADaM exchange)
adsl <- read_sas("data/adsl.sas7bdat")

# Stata .dta (e.g. a DHS-derived household dataset)
survey_dta <- read_dta("data/hh_survey_2022.dta")

# SPSS .sav
qol_spss <- read_sav("data/qol_followup.sav")

# Inspect preserved labelling
attributes(survey_dta$wealth_quintile)
```

A column read with `haven` is typically of class `haven_labelled` — it behaves like a numeric or character vector but carries an attached label and, for SAS/SPSS, a set of user-defined missing codes. Before merging or modelling, convert deliberately rather than letting the labelled class leak into `dplyr` pipelines unexamined:

```
library(dplyr)
library(haven)

survey_clean <- survey_dta |>
  mutate(
    wealth_quintile = as_factor(wealth_quintile),   # labels become factor levels
    age_years       = zap_labels(age_years)         # strip labels, keep plain
numeric
  )
```

Common pitfall. `haven_labelled` columns print deceptively like ordinary numbers in a console or `glimpse()`, but arithmetic and joins on them can behave unexpectedly, and some `tidyverse` verbs will warn or drop the class silently. Decide at import time, column by column, whether you want `as_factor()` (keep the coded meaning) or `zap_labels()` (keep only the numeric value) — don't leave the decision for whoever inherits the script.

Databases with `DBI`

For a claims warehouse, a TB case-based surveillance database, or any registry sitting behind a relational database, `DBI` gives a backend-agnostic connection interface — the same `dbConnect()` / `dbGetQuery()` pattern works whether the backend driver underneath is `RPostgres`, `odbc`, `RSQLite`, or `RMariaDB`. Given your SQL background, this is likely the most immediately comfortable part of the pipeline: write the query exactly as you would in any SQL client, and let `DBI` hand back a data frame.

```
library(DBI)
library(RPostgres)

con <- dbConnect(
  RPostgres::Postgres(),
  dbname   = "claims_warehouse",
  host     = Sys.getenv("CLAIMS_DB_HOST"),
  port     = 5432,
  user     = Sys.getenv("CLAIMS_DB_USER"),
  password = Sys.getenv("CLAIMS_DB_PASSWORD")
)

inpatient_costs <- dbGetQuery(con, "
  SELECT
    patient_id,
    admission_date,
    discharge_date,
    diagnosis_code,
    total_charge_usd
  FROM claims.inpatient_episodes
  WHERE admission_date >= '2022-01-01'
    AND diagnosis_code LIKE 'A15%'   -- respiratory TB, ICD-10
")

dbDisconnect(con)
```

Two habits carry over directly from your SQL work and are worth keeping explicit in R scripts:

- Push filtering and aggregation into the SQL string wherever the database can do it efficiently — pulling an entire multi-year claims table into R memory just to `filter()` it afterward wastes both the database's indexing and your laptop's RAM.
- Never hard-code credentials in a script that might end up in version control; read them from environment variables (`Sys.getenv()`) or a credentials manager, as above.

For parameterised queries — avoiding both SQL injection and the temptation to build query strings with `paste0()` — `DBI::dbGetQuery()` accepts a `params` argument with placeholders:

Difficult terminology: SQL injection. SQL injection is a security flaw that arises whenever a program builds a database query by pasting untrusted text — commonly, text a user typed into a form — directly into the query string, rather than keeping the query's structure and the user-supplied values cleanly separate. If a patient ID field is pasted straight into a query like `"SELECT * FROM patients WHERE id = '" + user_input + "'"`, and a user (malicious or merely careless) enters something like `' OR '1'='1`, the resulting pasted-together query can end up selecting **every** row in the table rather than just one patient's record, or worse, running entirely different commands the original query never intended, because the database cannot tell the difference between "data the user supplied" and "instructions that are part of the query itself" once they've been concatenated into one string. Parameterised queries (as shown just below) avoid this entirely by sending the query's fixed structure and the user-supplied values to the database as two genuinely separate pieces, so a value like `' OR '1'='1` is always treated as a literal, harmless piece of data to search for — never as part of the query's logic — no matter what it contains.

```
dbGetQuery(
  con,
  "SELECT * FROM claims.inpatient_episodes WHERE patient_id = $1",
  params = list("PT-00482")
)
```

HEOR in practice. Many HEOR analyses blend a clinical/claims extract from `DBI` with a published unit-cost list imported via `readxl`, and a trial dataset via `haven`. Keep each source's import in its own clearly commented block with the raw object named by its origin (`claims_raw`, `unitcost_raw`, `trial_raw`) before any joining — it makes the eventual data-management appendix of an HTA submission dossier far easier to write, because you can trace every derived variable back to a single named import step.

Public and institutional APIs

Increasingly, reference data for an HEOR model — WHO Global Health Observatory indicators, World Bank price deflators, or a national TB programme's own API-exposed dashboard — is available as a web API returning JSON rather than a static file. The modern idiomatic approach uses `httr2`, which replaced `httr` as the recommended tidyverse-adjacent HTTP client; the pattern, however, is one you'll recognise from any REST API work.

```
library(httr2)
library(jsonlite)

# Example: WHO Global Health Observatory API (public, no key required)
resp <- request("https://ghoapi.azureedge.net/api") |>
  req_url_path_append("TB_e_inc_num") |>   # TB incidence indicator code
  req_url_query(`$filter` = "SpatialDim eq 'IND'") |>
  req_perform()

resp_check_status(resp)   # raises an informative error on 4xx/5xx

tb_incidence <- resp |>
  resp_body_json(simplifyVector = TRUE)
```

```
tb_incidence_df <- tb_incidence$value   # the JSON payload's tabular "value" array
```

For APIs that require an authentication key, pass it as a header rather than embedding it in the URL, and again source it from an environment variable:

```
resp <- request("https://api.example-registry.org/v1/indicators") |>
  req_headers(Authorization = paste("Bearer", Sys.getenv("REGISTRY_API_KEY"))) |>
  req_perform()
```

`httr2`'s request objects are immutable and pipeable, which makes a multi-step request (path, query parameters, headers, body) read as a clear sequence of transformations rather than a single dense function call — a small ergonomic advantage over the older `httr::GET()` interface, though `httr::GET()` plus `httr::content()` remains perfectly serviceable if you encounter it in an older codebase.

> **Common pitfall.** Public health APIs change their response schema more often than their documentation is updated. Never assume `resp_body_json()`'s structure is stable release to release — print `str()` of the parsed object immediately after a new API integration, and consider caching the raw JSON response to disk (`jsonlite::write_json()`) alongside the parsed data frame, so that a downstream schema change in your parsing code can be debugged against a frozen snapshot rather than a live, possibly-already-changed endpoint.

5.5 Data dictionaries, metadata and codebooks

Importing a file correctly is necessary but not sufficient. Every dataset that feeds an HEOR analysis or HTA submission needs an accompanying data dictionary — sometimes called a codebook — that documents what each variable actually means, independent of the analyst who imported it. This is not bureaucratic overhead; it is the artefact that lets a reviewer, a collaborator, or you yourself in eighteen months reconstruct the logic behind a derived variable without re-reading the whole pipeline.

What a data dictionary must contain

A complete data dictionary, at minimum, documents for every variable:

- **Variable name** as it appears in the dataset, and a short human-readable **label** (e.g. `txn_outcome` → "TB treatment outcome at 12 months").
- **Type and units** — numeric currency in which year's USD, a date in which format, a continuous measure's unit of measurement (mg/dL, QALYs, days).
- **Value code mappings** for categorical variables — e.g. `1 = Cured`, `2 = Completed`, `3 = Failed`, `4 = Died`, `5 = Lost to follow-up`, matching whatever coding scheme the source system used (often a national or WHO-standard classification).
- **Missing-value codes** and their meaning — a claims field coded `-99` for "not applicable" versus `-98` for "known missing" behaves very differently in an analysis, and conflating them silently biases downstream estimates.
- **Derivation logic** for any variable that is not raw input — the exact formula or rule used to compute it, e.g. "`cost_per_episode_usd` = sum of `facility_cost` + `medication_cost` + `transport_reimbursement`, inflated to 2023 USD using the World Bank GDP deflator."

For a dataset built up over several processing stages — raw extract, cleaned, derived-variables-added — it is good practice to maintain the dictionary as a living document updated at each stage, not retrospectively reconstructed at write-up time, which is when details are most likely to be misremembered.

Practical R approaches

The simplest workable codebook is a data frame — one row per variable, with columns for name, label, type, allowed values, and missingness codes — saved alongside the dataset it describes:

```
library(tibble)

codebook <- tribble(
  ~variable,              ~label,                                ~type,
~values,                              ~missing_codes,
  "patient_id",           "Unique patient identifier",          "character", NA,
NA,
  "txn_outcome",          "TB treatment outcome at 12 months",  "integer",
"1=Cured; 2=Completed; 3=Failed; 4=Died; 5=LTFU", "-99=Not applicable;
-98=Unknown",
  "cost_per_episode_usd", "Total treatment cost, 2023 USD",     "double",    NA,
"-99=Not costed"
)

write_csv(codebook, "data/codebooks/tb_outcomes_codebook.csv")
```

This plain-data-frame pattern is transparent, version-controllable, and trivially joinable with the dataset's own column names for an automated consistency check (flagging any dataset column absent from the codebook, or vice versa).

Where the source data already arrives value-labelled — as is typical of `haven`-imported SAS or SPSS files — the `labelled` package lets you preserve and manipulate that labelling natively in R rather than translating it into a separate lookup table:

```
library(labelled)

# Attach value labels directly to a vector, SAS/SPSS style
txn_outcome <- labelled(
  c(1, 2, 3, 1, 4, 5),
  labels = c(Cured = 1, Completed = 2, Failed = 3, Died = 4, "Lost to follow-up" =
5)
)

var_label(txn_outcome) <- "TB treatment outcome at 12 months"

# Generate a human-readable summary straight from the labelled vector
look_for(txn_outcome)        # one-line variable summary with label and value
range
val_labels(txn_outcome)      # the full code -> label mapping

# Convert to factor for modelling, keeping only the chosen labels as levels
txn_outcome_factor <- to_factor(txn_outcome)
```

`labelled` is especially useful when a dataset is re-exported for sharing with a collaborator who expects SPSS- or Stata-style coding; `labelled::write_sav()` and companion functions round-trip

the labels back out into those formats, so the codebook travels with the data rather than living only in a separate document that can drift out of sync.

HEOR in practice. Whichever pattern you choose, generate the codebook programmatically from the dataset wherever possible (column names, types, and observed value ranges pulled directly via code) rather than typed by hand, and re-run that generation step every time the dataset is rebuilt. A hand-maintained codebook reliably drifts out of sync with the data within a few pipeline iterations; a codebook partially generated from the data itself cannot drift on the parts it covers.

Agency perspective. For HTA submissions in particular, data documentation is not optional polish — it is an explicit regulatory and reimbursement-agency expectation. CDISC's **define.xml** standard requires a machine-readable metadata file accompanying SDTM and ADaM datasets submitted to agencies such as FDA and PMDA, documenting every variable's name, label, type, controlled terminology, origin, and derivation algorithm in a structured, agency-parseable format — the formal, standardised cousin of the codebook patterns shown above. NICE's Health Technology Evaluations Manual (PMG36, 2022) likewise sets out expectations that the data and analyses underpinning a submission be transparent and reproducible by the Evidence Review Group, which in practice is unachievable without variable-level documentation of exactly this kind. Treat the codebook as a submission-grade deliverable from the start of an analysis, not an afterthought assembled under deadline pressure once the modelling is finished.

5.6 Data quality checks (`pointblank`, `assertr`, `validate`)

Every dominance and efficiency-frontier computation in Chapter 4 assumed that the input tibble of costs and QALYs was clean: no negative costs, no utilities above 1, no silently missing arms. That assumption rarely survives contact with a real data extract. Claims databases carry transcription errors, trial case report forms arrive with "9999" placeholder codes for missing fields, and TB programme registers accumulate duplicate patient IDs across reporting quarters. Before any cost-effectiveness logic runs, the data must be validated — not eyeballed, but checked programmatically against explicit rules that encode what "valid" means for health-economic data. R has three mature packages for this: `pointblank`, `assertr`, and `validate`, each with a slightly different philosophy but the same goal of turning implicit assumptions into explicit, auditable checks.

`pointblank` builds an **agent** that holds a set of validation rules and then runs them all at once via `interrogate()`, producing a report of which rows and columns failed which rule. This is well suited to a formal, documented QA step — the kind an HTA submission's data management appendix might reference.

```
library(pointblank)
library(dplyr)

# cost_qaly_tbl: one row per treatment arm, as used in Ch.4's
# dominance pipeline (cost, qaly, strategy columns)
cost_qaly_tbl <- tibble::tibble(
  strategy = c("Standard care", "New regimen", "Combination"),
```

```
  cost     = c(1200, 1850, -340),     # note the impossible negative cost
  qaly     = c(0.72, 0.81, 1.04),     # note the impossible utility > 1
  n_missing_followup = c(0, 3, NA)
)

agent <- create_agent(
  tbl = cost_qaly_tbl,
  label = "Cost-QALY QA before dominance analysis"
) |>
  col_vals_gte(columns = cost, value = 0) |>
  col_vals_lte(columns = qaly, value = 1) |>
  col_vals_gte(columns = qaly, value = 0) |>
  col_vals_not_null(columns = c(strategy, cost, qaly)) |>
  interrogate()

agent
# plot/export: get_agent_report(agent, display_table = FALSE)
```

Running `interrogate()` here will flag the "Combination" row twice — once for `cost < 0` and once for `qaly > 1` — **before** a single dominance comparison is attempted. Feeding unvalidated data into the ICER-ranking and dominance logic from Chapter 4 would otherwise produce a plausible-looking but meaningless frontier.

The `pointblank` agent's `interrogate()` report is itself a conventional reporting artefact: HTA submissions' data-management appendices typically summarise exactly this rule-by-rule pass/fail outcome rather than reproducing the raw agent object.

How to report this.

Validation rule	N rows checked	N failed	Pass rate (%)
`cost >= 0` (no negative cost)	3	1	66.7
`qaly <= 1` (utility not exceeding 1)	3	1	66.7
`qaly >= 0` (non-negative utility)	3	0	100.0
`strategy`, `cost`, `qaly` not missing	3	0	100.0
Source: `cost_qaly_tbl`; "Combination" row fails both the cost and utility bounds checks			

Table 12: Data validation summary for the cost-QALY input table, showing rule-by-rule pass/fail counts as reported in a submission's data-management appendix

`assertr` takes a lighter, pipe-native approach, well suited to inline checks inside a `dplyr` pipeline rather than a standalone report:

```
library(assertr)
library(dplyr)

cost_qaly_tbl |>
  verify(nrow(cost_qaly_tbl) > 0) |>
  assert(within_bounds(0, 1), qaly) |>
  assert(within_bounds(0, Inf), cost) |>
```

```
  assert(not_na, strategy, cost, qaly) |>
  # only rows that pass all assertions reach the dominance step
  arrange(cost)
```

`assert()` halts the pipeline with a descriptive error (row index and offending value) the moment a rule is violated, which is useful in an analysis script where you want a hard stop rather than a report to review later. `verify()` checks a logical condition on the whole data frame (e.g. that it is non-empty after a join), and `within_bounds()` is exactly the kind of range check that flags a utility above 1 or a cost below zero.

`validate` separates rule definition from rule application even more explicitly, which scales well when the same rule set needs to be applied to many extracts over time (e.g. successive quarterly TB programme uploads):

```
library(validate)

rules <- validator(
  cost_nonneg   = cost >= 0,
  qaly_bounded  = qaly >= 0 & qaly <= 1,
  strategy_set  = !is.na(strategy),
  followup_ok   = is.na(n_missing_followup) | n_missing_followup >= 0
)

out <- confront(cost_qaly_tbl, rules)
summary(out)
# sapply(out, function(x) sum(!x@expr)) # count fails per rule
```

`confront()` returns a result object that can be summarised, plotted, or written to a log file, giving a persistent audit trail of exactly which rule failed on which row of which dataset version — valuable when a TB programme analyst needs to justify, months later, why a particular district's quarter was excluded from a cost analysis.

HEOR in practice. Treat validation as a pipeline **stage**, not a one-off script you run when something looks wrong. In a reproducible analysis (Chapter 3's `targets` or a plain `source()` chain), the validation step should sit between "load raw extract" and "compute dominance/ICERs," and the pipeline should fail loudly — not produce a frontier plot — when a rule is violated. For HTA submissions, exporting the `pointblank` or `validate` report alongside the analysis code gives reviewers direct evidence that implausible values (negative costs, utilities > 1, impossible QALY decrements) were checked for and excluded or corrected, not missed.

Common pitfall. A validation rule that is too loose is worse than no rule at all, because it creates false confidence. `qaly <= 1` catches an impossible health state but does not catch a QALY of exactly 1.000000 held constant across 40 patient-years of follow-up, which is numerically valid but clinically absurd. Pair range checks with distributional sanity checks (e.g. `col_vals_between()` tightened to a clinically plausible range, or a quick `summary()`/histogram pass) rather than relying on hard mathematical bounds alone.

5.7 Data governance, privacy and de-identification

Validating values is necessary but not sufficient. Before a TB programme dataset, a claims extract, or a trial case report form ever reaches an R script, it has to be handled in a way that respects the people behind the rows — and, for anyone managing patient-level data professionally, that is a legal obligation as much as an ethical one. The vocabulary matters here, because "anonymous" is used loosely in casual conversation but has a precise technical meaning in every major privacy framework:

- **De-identification / anonymization** removes or transforms identifying information such that the remaining data cannot reasonably be linked back to an individual, by anyone, using any reasonably available means. Properly anonymized data generally falls outside the scope of data-protection law entirely — but "properly" is a high bar, especially for small denominators (a district with three TB patients of a rare drug-resistance pattern is not anonymous just because the name column was dropped).
- **Pseudonymization** replaces direct identifiers (name, national ID, exact address) with a code or token, but keeps a way — often a separate, access-controlled key table — to re-identify the record if needed. Pseudonymized data is still personal data under most frameworks, because re-identification remains possible; it reduces risk, but it is not the same as anonymization.

In the US, HIPAA sets out two recognised routes to de-identification. **Safe Harbor** specifies an enumerated list of 18 identifier categories (names, geographic subdivisions smaller than a state, all elements of dates except year, phone/fax numbers, medical record numbers, and so on) that must be removed or generalised; it is mechanical and auditable, but blunt. **Expert Determination** instead has a qualified statistician or scientist formally assess and document that the re-identification risk is "very small," using accepted statistical methods — appropriate when Safe Harbor's blunt removals would destroy analytic value (e.g. needing exact dates to compute time-to-event survival curves). Neither route is specific to US data only in spirit — the underlying logic (enumerate-and-remove versus risk-assess-and-document) is a useful mental model anywhere.

The EU's GDPR applies to any personal data concerning an identified or identifiable natural person, with special-category status (and extra safeguards) for health data specifically. For trial or RWE data originating from or concerning EU subjects, GDPR requires a documented lawful basis for processing, data minimisation, and — critically for anyone sharing analysis code or data extracts across borders — restrictions on transferring personal data outside the EU/EEA without an adequate safeguard. Pseudonymization is explicitly recognised in GDPR as a risk-reducing **technical measure**, not an exemption from the regulation.

India's Digital Personal Data Protection Act (DPDPA), 2023, is the relevant framework for anyone working with government health programme data in India — including TB programme (Ni-kshay and related) patient-level records. It establishes general principles that will shape how such data must be governed going forward: processing personal data only for a specified, lawful purpose with the data principal's consent or another permitted legal basis, collecting no more data than necessary for that purpose, implementing reasonable security safeguards against breach, and providing mechanisms for grievance redress. For a government health data role handling patient-level records, the practical implication is that **data minimisation and purpose limitation are not optional best practices — they are converging with statutory obligation**.

Agency perspective. ICH E6(R2) Good Clinical Practice guidance, in its sections on data handling and essential documents, requires that trial data be attributable, confidential, and protected from unauthorised access throughout the trial and in archived records — confidentiality of subject data is treated as a core GCP obligation, not an afterthought bolted on at publication. Any R-based trial or RWE analysis workflow that touches patient-level data should be able to show, concretely, how access to the raw identifiable extract was restricted and how only de-identified or pseudonymized data reached the analysis environment — this is the kind of evidence a GCP audit or an HTA body's data governance review will actually ask to see.

On the practical, R-side side of this: the single most common and most avoidable governance failure is a patient-level `.csv` ending up in a public (or even a private-but-over-shared) GitHub repository, because Chapter 2's `git add .` habit does not pause to ask what is in the file. Raw extracts with names, exact dates of birth, or facility-linked IDs small enough to re-identify someone must never be committed to version control, full stop — not even to a repo you believe is private, because history persists, forks happen, and access lists change. The workable pattern is the one this book itself uses throughout: write and share R code against small, clearly-labelled **synthetic or simulated** datasets (as in Chapter 4's cost-QALY tibbles), keep any real patient-level extract in a `.gitignore`'d local or access-controlled location, and if a working example genuinely needs to resemble real data's statistical structure, generate it synthetically (resampling distributions, adding noise, or using a package such as `synthpop`) rather than lightly-perturbing real records and hoping that counts as de-identified.

Common pitfall. Dropping the name and ID columns from a spreadsheet is not de-identification. A TB programme extract with facility name, exact diagnosis date, drug-resistance pattern, and sub-district can re-identify a specific patient through the combination of fields even with no name column at all — this is the well-documented "quasi-identifier" problem. Before calling any dataset de-identified, check cell sizes for rare combinations (small-count suppression), generalise or bin dates and geography where exact precision isn't analytically required, and document which identifiers were removed versus generalised versus retained under restricted access.

With the data pipeline now covered end to end — sourcing from clinical, claims, registry, and public-health systems; wrangling and reshaping it reproducibly; and validating and governing it before it ever touches an economic model — the next step in building an evidence base is working backwards from data to the published literature that frames and justifies it. Chapter 6 turns to R's growing toolkit for systematic literature review: searching and deduplicating records across databases, managing screening workflows, and extracting study-level data in a way that is as reproducible and auditable as the data management practices just established here.

Chapter 6. Systematic Literature Review (SLR) Support in R

Learning objectives

- Document a search strategy reproducibly, including the exact Boolean strings run in each database, per PRISMA-S expectations.
- Deduplicate multi-database reference exports and run a documented, agreement-checked two-stage screening workflow.
- Build a PRISMA 2020 flow diagram in R directly from the counts produced by the screening pipeline, not typed by hand.
- Design a structured data extraction template and a programmatic dual-extraction reconciliation workflow.
- Produce Cochrane RoB 2.0 / ROBINS-I traffic-light and summary plots with `robvis`.
- Use text mining to prioritise abstract screening order without weakening Cochrane's dual-independent-screening requirement.

Chapter 5 closed by flagging publication bias as a threat you mostly **detect** statistically, after the evidence is already assembled — funnel plots, Egger's test, trial-registry cross-checks. This chapter is about the earlier stage: building the evidence pool itself in a way that is transparent, reproducible, and auditable from the first search string to the final included-study count. Every one of the workflows below follows the same discipline already familiar from this book's R habits — log everything to a data frame, version it, and let the numbers that end up in a report trace back to a line of code rather than a remembered tally.

6.1 Search strategy documentation

A systematic review lives or dies on whether its search is reproducible. Chapter 5 flagged publication bias as a threat you detect statistically (funnel plots, Egger's test) only **after** the evidence is in hand; the search strategy is where you try to prevent that bias from ever entering the pool, by casting a wide, documented net across databases rather than relying on whatever your usual PubMed habits surface. PRISMA-S, the search-reporting extension to PRISMA 2020, asks for exactly this: database name, platform/interface, full Boolean string, date range, limits applied, and date run, for every source searched — not just the final count you remember.

There's no reason to keep this in a Word table that drifts out of sync with what you actually ran. A tibble, version-controlled alongside your analysis code, does the job better and survives an audit.

```
library(tibble)
library(readr)
library(dplyr)

search_log <- tibble(
  database     = c("PubMed", "Embase", "Cochrane CENTRAL"),
  platform     = c("PubMed.gov", "Ovid", "Cochrane Library"),
  search_string = c(
```

```
    '("tuberculosis"[MeSH] OR "TB"[tiab]) AND ("economic evaluation"[tiab] OR
"cost-effectiveness"[tiab])',
    'exp Tuberculosis/ AND (economic evaluation/ OR "cost effectiveness".ti,ab.)',
    'tuberculosis AND (cost-effectiveness OR "economic evaluation"):ti,ab,kw'
  ),
  date_from    = as.Date(c("2010-01-01", "2010-01-01", "2010-01-01")),
  date_to      = as.Date(rep("2024-06-30", 3)),
  date_run     = as.Date(rep(Sys.Date(), 3)),
  n_hits       = c(1842, 1365, 211),
  limits       = c("English; Humans", "English; Humans", "English")
)

write_csv(search_log, "search_strategy_log.csv")
search_log
```

Keep the raw string exactly as pasted from the database interface — don't "clean it up" for readability. If a reviewer needs to re-run the search in two years, the string in your CSV has to be the string that actually produced those 1,842 hits, syntax quirks and all.

How to report this.

Database	Platform	Limits applied	N hits
PubMed	PubMed.gov	English; Humans	1,842
Embase	Ovid	English; Humans	1,365
Cochrane CENTRAL	Cochrane Library	English	211
Date range 2010-01-01 to 2024-06-30; all searches run and logged the same day (`search_strategy_log.csv`)			

Table 13: Search strategy summary by database, as reported in an SLR methods section or PRISMA-S search appendix

Before you ever get to that final string, you usually need help deciding which terms belong in it. This is where `litsearchr` earns its place in an R-based SLR workflow: it takes a seed set of known-relevant papers (or a naive search's results), extracts candidate keyphrases, and builds a keyword co-occurrence network so you can spot synonyms and related terms you'd otherwise miss — "DR-TB" sitting next to "multidrug-resistant tuberculosis" sitting next to "rifampicin-resistant," for instance.

```
library(litsearchr)

# naive_results is a data frame with title/abstract/keywords columns
# pulled from an initial scoping search
rakedkeywords <- extract_terms(
  text = paste(naive_results$title, naive_results$abstract),
  method = "fakerake",
  min_freq = 3,
  ngrams = TRUE,
  min_n = 1,
  max_n = 3
```

```
)

docs <- paste(naive_results$title, naive_results$abstract)
dfm  <- create_dfm(elements = docs, features = rakedkeywords)
g    <- create_network(search_dfm = dfm, min_studies = 3)

cutoff <- find_cutoff(g, method = "cumulative", percent = 0.80)
reduced_graph <- reduce_graph(g, cutoff_strength = cutoff[1])
search_terms  <- get_keywords(reduced_graph)
search_terms
```

The output isn't a search string you paste in blind — it's a candidate term list you screen against clinical judgment before it enters the Boolean query that then goes into `search_log` above.

HEOR in practice. In HTA submissions, the search strategy appendix is frequently where assessors' first critical comments land — not the clinical evidence itself. A dated, versioned `search_strategy_log.csv` sitting in your project repo, committed alongside the screening scripts, means you can answer "did you search Embase after your PubMed search, or the same week?" without digging through email threads or a collaborator's memory.

Common pitfall. Translating a search string across database syntaxes (PubMed's `[MeSH]` vs. Embase's `exp .../` vs. Cochrane's `:kw`) by eye is a common source of silently different hit counts. Log the **translated** string actually run in each interface, not one master string you assume is "equivalent" everywhere — and sanity-check hit counts against expectations (Embase typically returns more records than PubMed for the same concept; a near-identical count across both is worth a second look).

6.2 Deduplication and screening workflows

Running the same concept across three databases guarantees duplicate records — often 20–40% of a combined pull. Deduplicating by exact title match alone will miss records that differ by punctuation, case, or a trailing journal abbreviation, so you need fuzzy matching on title/author/year, and ideally DOI matching as a first, cheap pass before falling back to string similarity.

`synthesisr` standardizes bibliographic imports (RIS/Bibtex/CSV exports from different databases rarely use identical field names) and performs similarity-based deduplication; `revtools` builds on the same ecosystem and adds an interactive screening interface. `ASySD` (Automated Systematic Search Deduplicator) is purpose-built for the messier reality of SLR exports and handles multi-stage matching (DOI exact match, then title/author fuzzy match) more robustly than a single string-distance cutoff.

```
library(synthesisr)
library(dplyr)

# read_refs() handles RIS exports from PubMed, Embase, and Cochrane
pubmed_refs  <- read_refs("pubmed_export.ris")
embase_refs  <- read_refs("embase_export.ris")
cochrane_refs <- read_refs("cochrane_export.ris")
```

```
all_refs <- merge_columns(pubmed_refs, embase_refs, cochrane_refs) |>
  as_tibble()

# first pass: exact match on standardized title string
dup_exact <- find_duplicates(
  all_refs$title,
  method = "exact",
  to_lower = TRUE,
  rm_punctuation = TRUE
)

# second pass: fuzzy match on anything exact matching missed
dup_fuzzy <- find_duplicates(
  all_refs$title,
  method = "string_osa",
  to_lower = TRUE,
  rm_punctuation = TRUE,
  threshold = 7
)

all_refs_dedup <- all_refs |>
  mutate(dup_id = dup_fuzzy) |>
  extract_unique_references(dup_id)

nrow(all_refs) - nrow(all_refs_dedup)   # duplicates removed, for the PRISMA count
```

Once you have a deduplicated set, screening is a two-stage filter: title/abstract first (fast, generous, errs toward inclusion), then full-text on the survivors (slower, decisive, with documented exclusion reasons). The habit that keeps this defensible is logging every reviewer decision at the record level, not just the final yes/no — and if you have two independent screeners, computing agreement rather than assuming it.

```
library(dplyr)
library(irr)   # kappa2()

screening_log <- all_refs_dedup |>
  mutate(
    reviewer1_ti_ab = sample(c("include", "exclude"), n(), replace = TRUE, prob =
c(0.3, 0.7)),
    reviewer2_ti_ab = sample(c("include", "exclude"), n(), replace = TRUE, prob =
c(0.3, 0.7))
  )

kappa_result <- kappa2(screening_log[, c("reviewer1_ti_ab", "reviewer2_ti_ab")])
kappa_result$value   # Cohen's kappa for title/abstract agreement

conflicts <- screening_log |>
  filter(reviewer1_ti_ab != reviewer2_ti_ab)
nrow(conflicts)   # records requiring reconciliation discussion
```

A kappa below roughly 0.6 is usually a sign the inclusion criteria need tightening before you burn reviewer time on full-text retrieval, not a problem to paper over with a third tie-breaking reviewer alone.

Difficult terminology: Cohen's kappa. Cohen's kappa measures how much two reviewers agree with each other **beyond** what you would expect them to agree on by chance alone. A naive "percent agreement" is misleading here: if 90% of records are obvious excludes, two reviewers who each decide mostly at random (or even by flipping a coin on the hard ones) will still agree with each other roughly 80-90% of the time just by both landing on "exclude" most of the time. Kappa corrects for that baseline chance-agreement rate, so a kappa of 0.71 is a genuinely informative number, not just "71% of decisions matched." The rule-of-thumb bands you will see cited are: below 0.4 poor, 0.4-0.6 moderate, 0.6-0.8 substantial, above 0.8 near-perfect agreement — so 0.71 here is solid but not pristine, which is exactly why 115 conflicts still needed a reconciliation discussion. Think of it the way you'd think of a diagnostic test's performance over the baseline prevalence rate: two clinicians who both tend to diagnose a common condition will agree often just because the condition is common, not because they're applying identical criteria — kappa is the statistic that strips that artefact out.

How to report this.

Metric	Value
Records screened (title/abstract)	3,198
Reviewer 1 — included	960
Reviewer 2 — included	955
Concordant decisions	3,083
Agreement (% records)	96.4%
Cohen's kappa	0.71
Conflicts requiring reconciliation	115

Table 14: Dual-reviewer title/abstract screening agreement, as reported in the SLR methods section

Agency perspective. The Cochrane Handbook for Systematic Reviews of Interventions (Chapter 4, "Searching for and selecting studies") specifies that screening should be done independently by at least two reviewers, with disagreements resolved by discussion or a third reviewer, and that the process — including inter-rater agreement where feasible — should be reported with enough detail for replication. A logged `screening_log` with per-reviewer decision columns and a computed kappa is a direct, auditable response to that requirement, not an optional nicety.

Common pitfall. `find_duplicates()` with a string-distance method is sensitive to the threshold you choose — too loose and you merge genuinely distinct trials with similar titles (common in TB drug trials with similar naming conventions); too tight and you leave duplicates in, inflating your "identified" count and understating how much actual unique evidence you screened. Always spot-check a sample of flagged duplicate pairs by eye before accepting

`extract_unique_references()`'s output as final, and record the method and threshold used in your methods write-up.

6.3 PRISMA flow diagrams in R

Every number that feeds the PRISMA flow diagram should trace back to a line of code you already ran — the row count before `extract_unique_references()`, the duplicate count it removed, the title/abstract exclusions from `screening_log`, the full-text exclusions with reasons. Building the diagram from a hand-typed set of numbers in PowerPoint reintroduces exactly the kind of untraceable discrepancy a reproducible SLR workflow is meant to eliminate.

The `PRISMA2020` package takes a named vector or data frame of stage counts (matching the official PRISMA 2020 flow diagram template) and renders the diagram as an SVG/interactive plot you can embed or export.

```
library(PRISMA2020)
library(dplyr)

prisma_counts <- data.frame(
  stage = c(
    "database_results", "duplicates_removed",
    "records_screened", "records_excluded",
    "full_text_retrieved", "full_text_excluded",
    "studies_included"
  ),
  n = c(
    nrow(all_refs),                     # 3,418 identified
    nrow(all_refs) - nrow(all_refs_dedup),  # duplicates removed
    nrow(all_refs_dedup),               # screened at title/abstract
    sum(screening_log$reviewer1_ti_ab == "exclude" &
          screening_log$reviewer2_ti_ab == "exclude"),
    42,                                  # sent to full-text review
    19,                                  # excluded at full-text, reasons below
    23                                   # final included studies
  )
)

# PRISMA_data() expects its own template row-name structure (stage
# names in column 1, counts in column "n") -- PRISMA2020_data(data) was
# a typo for the real exported function PRISMA_data(data)
data <- PRISMA_data(prisma_counts)
plot <- PRISMA_flowdiagram(
  data,
  interactive = FALSE,
  previous = FALSE,
  other = FALSE,
  fontsize = 12,
  title_colour = "steelblue"
)

# PRISMA2020 has no PRISMA_save(); save a ggplot/DiagrammeR-style object
```

```
# with DiagrammeRsvg + rsvg, the package's own documented export route
svg_code <- DiagrammeRsvg::export_svg(plot)
rsvg::rsvg_svg(charToRaw(svg_code), file = "prisma_flow.svg")
```

Full-text exclusions need reasons, not just a count, because PRISMA 2020 Item 7 and the diagram's exclusion box both expect a breakdown (wrong population, wrong comparator, wrong outcome, conference abstract only, etc.) rather than a single undifferentiated number.

```
exclusion_reasons <- tibble::tibble(
  reason = c("Wrong population", "No cost outcome",
             "Conference abstract only", "Duplicate publication"),
  n = c(7, 6, 4, 2)
)
sum(exclusion_reasons$n)   # must equal full_text_excluded above -- reconcile if
not
```

That reconciliation check — the sum of itemized exclusion reasons equalling the single number on the flow diagram — is worth asserting in code (`stopifnot()` or a simple test) rather than eyeballing, since it's exactly the kind of off-by-one error that surfaces in peer review.

How to report this.

Stage	N records
Records identified across databases	3,418
Duplicates removed	220
Records screened (title/abstract)	3,198
Records excluded at title/abstract	2,238
Full-text reports retrieved	42
Full-text reports excluded	19
Studies included in review	23

Table 15: Screening funnel stage counts underlying the PRISMA 2020 flow diagram, distinct from the itemized full-text exclusion reasons below

How to report this.

Exclusion reason	n
Wrong population	7
No cost outcome	6
Conference abstract only	4
Duplicate publication	2
Total excluded at full-text = 19	

Table 16: Full-text exclusion reasons, reported alongside the PRISMA 2020 flow diagram (Item 7)

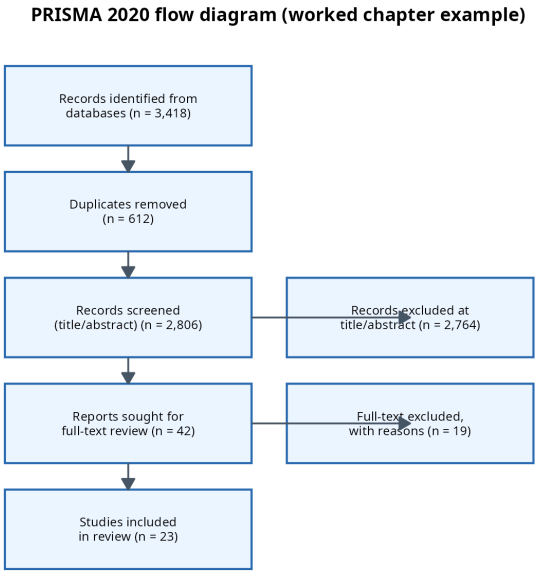


Figure 5: A PRISMA 2020 flow diagram built directly from the stage counts computed in code (§6.3), rather than typed by hand — the reconciliation discipline this section argues for: every number traces back to a line of code, and the itemized exclusion reasons sum to the single full-text-excluded count shown.

HEOR in practice. Submission dossiers to HTA bodies increasingly expect the PRISMA diagram and the underlying counts to be reconcilable on request. Keeping `prisma_counts` as a data frame generated from the same pipeline that produced `search_strategy_log.csv` and `screening_log` means a reviewer's question — "where did the other 19 full-text articles go?" — has a one-line answer: re-run the script and read off `exclusion_reasons`.

Agency perspective. The PRISMA 2020 statement (Page et al., **BMJ** 2021) explicitly requires, in Item 16a, a flow diagram showing the number of records identified, duplicates removed, records screened and excluded, reports sought and not retrieved, reports assessed for eligibil-

ity, and studies included — with reasons given for full-text exclusions. `PRISMA_flowdiagram()` maps directly onto this template field-for-field; using it rather than a freehand diagram is the most mechanical way to guarantee compliance with that specific checklist item.

6.4 Data extraction templates and extraction QA

Once you have a final included-studies set from screening (§6.2-6.3), the next failure point in an SLR is extraction: the manual transcription of population, intervention, comparator, outcomes (PICO), effect sizes, and risk-of-bias inputs from each paper into a structured dataset. For a TB-programme analyst this will feel familiar — it's the same discipline as designing a clean case-report form before you let field staff touch it. A tibble-based extraction template, version-controlled from day one, turns this into reproducible data engineering rather than a spreadsheet someone can silently edit.

Design the template as one row per study-outcome combination (long format), not one row per study, because most SLRs extract multiple outcomes and multiple comparisons per study, and a wide format forces awkward column explosion later.

```
library(tibble)
library(dplyr)

extraction_template <- tibble(
  study_id        = character(),  # e.g. "Nguyen2021"
  reviewer_id     = character(),  # who extracted this row
  population      = character(),
  intervention    = character(),
  comparator      = character(),
  outcome_name    = character(),  # e.g. "treatment_success"
  outcome_timepoint = character(),
  effect_measure  = character(),  # "RR", "OR", "HR", "MD"
  effect_estimate = double(),
  ci_lower        = double(),
  ci_upper        = double(),
  n_intervention  = integer(),
  n_comparator    = integer(),
  rob_domain_1    = character(), # e.g. randomisation process
  rob_domain_2    = character(),
  rob_overall     = character(), # "Low", "Some concerns", "High"
  notes           = character()
)

# one extracted row, reviewer A
extraction_template <- extraction_template %>%
  add_row(
    study_id = "Nguyen2021", reviewer_id = "RA",
    population = "Adults, drug-sensitive PTB",
    intervention = "4-month regimen", comparator = "6-month regimen",
    outcome_name = "treatment_success", outcome_timepoint = "end_of_treatment",
    effect_measure = "RR", effect_estimate = 0.98,
    ci_lower = 0.92, ci_upper = 1.04,
    n_intervention = 310, n_comparator = 308,
    rob_domain_1 = "Low", rob_domain_2 = "Some concerns",
```

```
    rob_overall = "Some concerns", notes = "ITT analysis used"
  )
```

Enforcing a schema up front (explicit column types, controlled vocabularies for `effect_measure` and `rob_overall`) means extraction errors — a reviewer typing "Odds ratio" where another typed "OR" — surface immediately as validation failures rather than silently corrupting a later meta-analysis.

Dual, independent extraction is the standard safeguard against transcription error, mirroring dual independent screening. Have two reviewers populate identical-schema tibbles, then reconcile programmatically rather than by eye:

```
library(waldo)

reviewer_a <- readr::read_csv("extraction_reviewerA.csv")
reviewer_b <- readr::read_csv("extraction_reviewerB.csv")

# align rows on the natural key before comparing
reviewer_a <- reviewer_a %>% arrange(study_id, outcome_name)
reviewer_b <- reviewer_b %>% arrange(study_id, outcome_name)

discrepancies <- waldo::compare(
  reviewer_a, reviewer_b,
  tolerance = 1e-6              # ignore float noise, e.g. 0.980 vs 0.98
)
discrepancies
```

`waldo::compare()` reports exactly which cells differ between the two tibbles — far more actionable for an adjudication meeting than a manual side-by-side read of two spreadsheets. For numeric fields where small rounding differences are expected (confidence intervals read off a forest plot, for instance), a `dplyr`-based tolerance check is often clearer than relying on `waldo`'s defaults:

> **Difficult terminology: forest plot.** A forest plot is the standard way to show a set of effect estimates — one per study — stacked one above the other so they can be compared at a glance. Each row is one study: a square or dot marks that study's point estimate (e.g. a risk ratio or odds ratio), and a horizontal line through it spans its 95% confidence interval, so wider lines mean less-precise studies. The rows are typically ordered by publication date or study size, and when the studies are later statistically pooled (Chapter 11 covers this in full), a diamond shape at the bottom shows the combined estimate, with the diamond's width showing the pooled interval's precision. Visually, it's the same idea as lining up several lab-test results for a patient on one chart with their error bars, so you can see at a glance which results are consistent with each other and which one is the outlier. You will not build a forest plot in this chapter — that machinery lives in Chapter 11 — but the term shows up here already because reconciling reviewer-extracted effect estimates (as in the `waldo::compare()` step above) is precisely the data-cleaning work that feeds into one later.

```
joined <- inner_join(
  reviewer_a, reviewer_b,
  by = c("study_id", "outcome_name"),
  suffix = c("_A", "_B")
)
```

```
flagged <- joined %>%
  mutate(
    estimate_diff = abs(effect_estimate_A - effect_estimate_B),
    rob_mismatch  = rob_overall_A != rob_overall_B
  ) %>%
  filter(estimate_diff > 0.01 | rob_mismatch)

flagged %>% select(study_id, outcome_name, effect_estimate_A, effect_estimate_B,
rob_overall_A, rob_overall_B)
```

Route every row in `flagged` to a third-reviewer adjudication step before it enters the master extraction dataset, and log the adjudicated value with a `reviewer_id = "adjudicated"` row so the audit trail is preserved rather than overwritten.

HEOR in practice. Treat the extraction dataset the way you would treat programme surveillance data destined for a drug-resistance report: commit the CSV or `.rds` files to version control (Git) at each named milestone — "post first-pass extraction", "post reconciliation", "post adjudication" — rather than editing one live file in place. A `git diff` on a CSV extraction file is a legitimate, line-by-line audit trail of who changed what value and when, which is exactly the kind of provenance a submission dossier's SLR appendix may be interrogated on. Avoid storing extraction data only in a shared Excel/Google Sheet with no version history; if the sheet is the master, export it to CSV and commit the export after every extraction session.

Common pitfall. Resist the temptation to resolve discrepancies by simply averaging or eyeballing "which one looks right." A documented discrepancy with no recorded resolution rationale is itself a methodological weakness reviewers will flag. Always log **why** a value was adjudicated the way it was (e.g. "reviewer A misread Table 3 column header — comparator and intervention columns were swapped") alongside the final value.

6.5 Risk of bias visualisation (`robvis`)

Risk-of-bias assessment itself — applying Cochrane RoB 2.0 to RCTs or ROBINS-I to non-randomized studies — is a judgment exercise done by reviewers against the official signalling-question algorithms, not something R automates. What R (via the `robvis` package) does well is turn a completed set of domain-level judgments into the standard traffic-light and weighted-bar-chart figures that PRISMA-compliant reviews and HTA submissions are expected to include.

`robvis` expects a data frame with one row per study, a column per RoB domain plus an overall judgment column, and values restricted to the tool's controlled vocabulary (`"Low"`, `"Some concerns"`/`"Moderate"`, `"High"`/`"Serious"`, `"Critical"`, `"No information"` depending on the tool). Build this from your reconciled extraction tibble:

```
library(robvis)
library(dplyr)

# RoB 2.0 for RCTs: one row per study, domains D1-D5, overall, then a
# final Weight column -- robvis expects the weight as the LAST column
rob2_data <- tibble::tibble(
  Study = c("Nguyen2021", "Alvarez2020", "Okafor2022"),
```

```
  D1 = c("Low", "Some concerns", "Low"),
  D2 = c("Low", "Low", "Some concerns"),
  D3 = c("Some concerns", "Low", "Low"),
  D4 = c("Low", "Low", "Low"),
  D5 = c("Low", "Some concerns", "Low"),
  Overall = c("Some concerns", "Some concerns", "Low"),
  Weight = c(33.3, 33.3, 33.3)
)

rob_traffic_light(
  data = rob2_data,
  tool = "ROB2",
  psize = 8
)

rob_summary(
  data = rob2_data,
  tool = "ROB2",
  overall = TRUE
)
```

`rob_traffic_light()` produces the familiar red/amber/green grid — one row per study, one column per domain — that goes directly into an SLR appendix or a methods annex of an HTA dossier. `rob_summary()` produces the weighted horizontal bar chart showing the proportion of studies at each judgment level per domain, which is the figure typically used in the main body of the review to summarise bias across the whole evidence base at a glance.

For non-randomized studies assessed with ROBINS-I, the domain structure and vocabulary differ, so specify `tool = "ROBINS-I"` and supply the correct domain columns:

```
robins_data <- tibble::tibble(
  Study = c("Mbeki2019", "Santos2021"),
  D1 = c("Low", "Moderate"),
  D2 = c("Moderate", "Low"),
  D3 = c("Low", "Low"),
  D4 = c("Serious", "Moderate"),
  D5 = c("Low", "Low"),
  D6 = c("Low", "Low"),
  D7 = c("Moderate", "Low"),
  Overall = c("Serious", "Moderate"),
  Weight = c(50, 50)
)

rob_traffic_light(data = robins_data, tool = "ROBINS-I")
rob_summary(data = robins_data, tool = "ROBINS-I", overall = TRUE)
```

Both functions return `ggplot2` objects, so they compose with the rest of your usual plotting pipeline — save with `ggsave()`, adjust fonts and sizing for print dossiers, or patch into a combined figure with `patchwork` alongside forest plots from a later meta-analysis (Chapter 11).

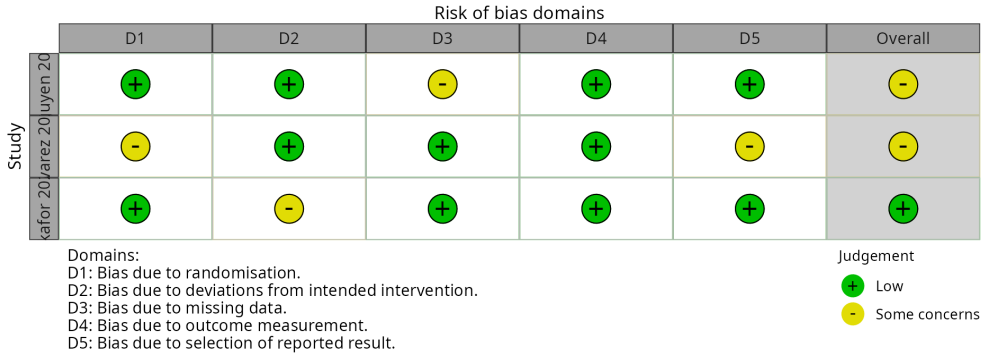


Figure 6: `robvis::rob_traffic_light()` applied to the chapter's worked RoB 2.0 example (`rob2_data`): a red/amber/green grid, one row per study, one column per domain, in the standard format expected in an SLR appendix or HTA methods annex.

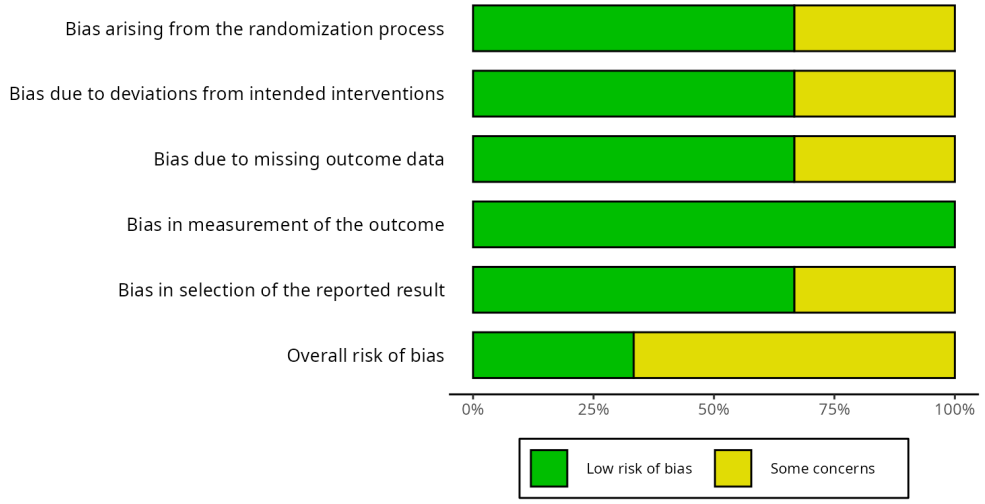


Figure 7: `robvis::rob_summary()` on the same data: the weighted horizontal bar chart summarising the proportion of studies at each judgment level per domain, the figure typically placed in the main body of a review rather than its appendix.

Agency perspective. Cochrane Handbook Chapter 8 (RoB 2.0) and Chapter 25 (ROBINS-I) specify the signalling questions and domain structure that `robvis`'s input vocabulary is built around — do not substitute an informal judgment scale that doesn't map onto those domains, because a reviewer or HTA assessor checking your figure against the Handbook's defined categories will immediately notice a mismatch. PRISMA 2020 item 18 explicitly requires reporting "methods used to assess risk of bias in the included studies," and a traffic-light plot generated this way is standard, recognisable evidence of having done so transparently rather than narratively.

Common pitfall. `robvis` will silently plot whatever judgment strings you give it, including typos or inconsistent capitalisation (`"low"` vs `"Low"`), as a separate category rather than erroring. Always run `unique()` across your domain columns before plotting to catch stray

values that would otherwise produce an extra, meaningless colour band in the traffic light or summary bar.

6.6 Text mining and ML-assisted screening

By the time a search strategy returns several thousand abstracts, manually reading titles in import order is a poor use of two reviewers' time. Text mining can reorder the screening queue so the most plausibly relevant abstracts are screened first, which speeds up time-to-inclusion-decision without changing **what** counts as included — screening decisions themselves remain exclusively human, exactly as in every other stage of this chapter.

A simple term-frequency pass with `tidytext` is often enough to sanity-check that your search strategy actually captured the expected vocabulary before you even start full screening:

```
library(tidytext)
library(dplyr)

abstracts <- tibble::tibble(
  study_id = c("S1", "S2", "S3"),
  abstract = c(
    "Randomized trial of four-month rifapentine-based regimen for pulmonary TB.",
    "Cohort study assessing cardiovascular risk in diabetic patients.",
    "Treatment success and relapse rates in short-course TB chemotherapy."
  )
)

tidy_abstracts <- abstracts %>%
  unnest_tokens(word, abstract) %>%
  anti_join(stop_words, by = "word")

term_freq <- tidy_abstracts %>%
  count(word, sort = TRUE)

term_freq %>% head(10)
```

For a more structured relevance signal across a few thousand abstracts, `quanteda` builds a document-feature matrix and lets you score each abstract against a set of PICO-anchored terms drawn from your protocol's search strategy:

```
library(quanteda)

corp <- corpus(abstracts, text_field = "abstract", docid_field = "study_id")
dfm_abs <- corp %>%
  tokens(remove_punct = TRUE) %>%
  tokens_remove(stopwords("en")) %>%
  dfm()

# relevance proxy: count of protocol-defined key terms per abstract
key_terms <- c("tuberculosis", "rifapentine", "regimen", "relapse", "treatment")
relevance_score <- dfm_abs %>%
  dfm_select(pattern = key_terms) %>%
  rowSums()
```

```r
screening_order <- tibble::tibble(
  study_id = docnames(dfm_abs),
  relevance_score = relevance_score
) %>%
  arrange(desc(relevance_score))

screening_order
```

Feeding `screening_order` to your reviewers as the order in which to work through the abstract list — highest-scoring first — gets likely-includes screened early, which matters operationally when a review has a hard deadline and screening is still incomplete when it arrives.

The `revtools` package goes further, offering topic modelling (via `screen_topics()` / `screen_abstracts()`) specifically built around the SLR screening workflow: it clusters abstracts into latent topics and gives reviewers an interactive interface to mark whole topic clusters as likely relevant or irrelevant, which is useful triage for very large (10,000+) abstract sets before line-by-line screening starts. Purpose-built external screening platforms — Rayyan and Abstrackr are the most commonly cited in SLR methods sections — offer similar machine-learning-assisted prioritisation (active-learning ranking as reviewers screen) and integrate directly with reference managers; R's role here is typically pre- or post-processing (exporting a prioritised order into Rayyan, or pulling Rayyan's decision export back into your extraction pipeline from §6.4) rather than replacing those platforms outright.

> **Common pitfall.** Term-frequency scores, topic clusters, and active-learning rankings are prioritisation aids, not screening decisions, and none of them replace Cochrane's requirement (Cochrane Handbook Chapter 4) for dual independent screening of every title/abstract and full text by two human reviewers with disagreements resolved by discussion or a third reviewer. Using an ML relevance score to **exclude** abstracts outright, or to let a single reviewer screen only the "high-relevance" tier while skipping the rest, breaks that standard and will be flagged by anyone checking your PRISMA flow diagram against the methods text. The only legitimate use of these tools in a Cochrane-conformant review is re-ordering the queue that both independent human reviewers still work through in full.

Chapter 7. Missing Data

Learning objectives

- Distinguish MCAR, MAR, and MNAR missing-data mechanisms and explain why the distinction cannot be confirmed from observed data alone.
- Use `naniar` and `visdat` to visualise and audit missingness patterns before choosing a handling strategy.
- Explain why complete-case analysis and single imputation understate uncertainty, and when each is defensible.
- Perform multiple imputation with `mice` (and know when to reach for `Amelia` or `missForest` instead), pooling estimates correctly via Rubin's rules.
- Conduct tipping-point and pattern-mixture sensitivity analyses to stress-test conclusions against departures from MAR.
- Handle the distinctive challenges of missing cost and QALY data — skewness, bounds, and informative dropout.
- Report missing-data handling for an HTA submission in line with CHEERS 2022 and ICH E9(R1) expectations.

Chapter 6 closed by building a reproducible evidence pool from the published literature; this chapter turns to a problem that shows up just as often in the primary trial, registry, or claims dataset itself — not every cell in that dataset is filled in, and what you do about the gaps can change a cost-effectiveness conclusion as much as any modelling choice later in this book.

7.1 Mechanisms: MCAR, MAR, MNAR

Every HEOR analysis built on trial or real-world data eventually runs into the same quiet problem: some values are simply not there. A patient skips a follow-up visit, a claims extract has a null cost field for a quarter, an EQ-5D questionnaire comes back with three dimensions ticked and two left blank. Before you can decide what to do about it — delete the rows, impute them, or model around them — you need a working theory of **why** the data are missing. Donald Rubin's 1976 framework gives the vocabulary for that theory, and it remains the backbone of every modern missing-data guideline used in health technology assessment.

Rubin defined three mechanisms by asking whether the probability that a value is missing depends on the data itself:

- **MCAR (Missing Completely At Random)**: the probability of a value being missing is unrelated to both the observed and the unobserved data. Missingness is pure noise — equivalent to a coin flip applied independently to every record.
- **MAR (Missing At Random)**: the probability of a value being missing depends only on the **observed** data (other variables you have recorded), not on the missing value itself, once you condition on what you observed.
- **MNAR (Missing Not At Random)**: the probability of a value being missing depends on the **unobserved** value itself, even after conditioning on everything you did observe.

Take a concrete HEOR example and carry it through all three mechanisms: a randomised trial collects EQ-5D-5L questionnaires at 12 weeks to feed a QALY calculation for the economic model, and 15% of participants have a missing follow-up questionnaire.

- If non-response is driven by a random administrative failure — a clinic forgot to mail the form to a random subset of participants regardless of their health status, treatment arm, age, or anything else — that is MCAR. The missing 15% are, in expectation, a random sample of the full cohort, and a complete-case analysis (simply dropping those rows) is unbiased, if inefficient.
- If non-response is higher among patients who missed their clinic visit, and clinic attendance is itself associated with baseline disease severity that you **recorded** at randomisation, then conditioning on baseline severity (which you have) explains the missingness pattern. That is MAR: missingness depends on an observed covariate, not on the unobserved EQ-5D score itself once severity is accounted for.
- If, instead, patients skip the questionnaire specifically **because** their health has deteriorated and they no longer feel well enough to complete it — and that deterioration is not fully captured by anything else in your dataset — then missingness depends on the very value that is missing. That is MNAR, and no amount of conditioning on the data you do have will fix it, because the unobserved health state itself is the driver.

The uncomfortable truth is that these three scenarios can produce an **identical** pattern of missing cells in your dataset. Nothing in the observed rows and columns tells you which story is true — MCAR, MAR, and MNAR are all consistent with the same table of 1s and 0s marking which values are missing. The mechanism is a statement about the **unobserved** values, so it can never be confirmed or refuted by the observed data alone. You can test whether missingness is associated with observed covariates (which can rule out some forms of MCAR, see `mcar_test()` in Section 7.2), but you can never test whether missingness depends on the unobserved value itself — by definition, you do not have that value to check against. The choice between MAR and MNAR is ultimately a domain-knowledge argument, not a statistical result, and should be defended with clinical reasoning (do sicker patients plausibly drop out more?) rather than a p-value.

This matters enormously for HEOR because the three mechanisms license completely different analytic responses:

- Under **MCAR**, a complete-case analysis is unbiased — you lose precision (wider confidence intervals around your ICER) but not accuracy. This is the only mechanism under which "just drop the missing rows" is statistically defensible, and it is rarely plausible for anything beyond trivial administrative dropout.
- Under **MAR**, complete-case analysis is generally biased, but multiple imputation (MI) — drawing plausible values conditional on the observed covariates, as implemented in `mice` or `missForest` — recovers unbiased estimates, because MAR is precisely the assumption that licenses those methods. Most routine MI practice in trial-based cost-effectiveness analysis implicitly assumes MAR.
- Under **MNAR**, no standard imputation model will save you, because the imputation itself needs to encode a mechanism you cannot verify. The accepted response is either explicit mechanism modelling — selection models (a joint model for the outcome and the missingness indicator) or pattern-mixture models (fitting separate outcome models within each missingness pattern and specifying how they differ) — or, more commonly in practice, a structured sensitivity analysis that varies the assumed severity of MNAR departure and reports how the ICER moves.

In practice, no HEOR dataset wears a label saying which mechanism generated it. The discipline is to state your assumption explicitly, justify it with clinical and operational knowledge of why the data went missing, and then stress-test the conclusion against alternative, less convenient assumptions — which is exactly where Section 7.2's exploratory tools and later chapters' sensitivity-analysis techniques come in.

7.2 Exploring missingness (naniar, visdat, Shiny-based explorers)

Before choosing an imputation strategy or defending a mechanism assumption, you need to actually look at your missingness — not just count NAs with `colSums(is.na(df))`, but see where they fall, whether they cluster by variable, by patient, by visit, or by arm, and whether they co-occur in patterns that hint at a shared cause. The `naniar` and `visdat` packages, both built around tidyverse conventions, turn this into a few lines of code rather than a manual audit, and they should be a standard first step on any trial or claims extract destined for a cost-effectiveness model.

`visdat::vis_dat()` gives an overview of an entire data frame in one plot: each column is a vertical strip, coloured by data type (character, numeric, logical), with missing cells picked out distinctly, so you immediately see whether missingness is concentrated in a few columns (say, a late-added quality-of-life variable) or scattered everywhere. `visdat::vis_miss()` narrows this to missingness specifically, shading missing cells and printing the percentage missing per column, which is often the single most useful diagnostic plot to drop into an analysis log.

`naniar` extends this considerably. `naniar::vis_miss()` produces a similar heatmap-style plot but adds clustering options so that rows with similar missingness patterns are grouped together — useful for spotting whether, for instance, patients missing a 12-week EQ-5D are disproportionately the same patients missing a 12-week cost record, which would support a shared-cause MAR or MNAR story rather than independent MCAR noise in each column. `naniar::gg_miss_var()` plots the number (or proportion) of missing values per variable as a simple ranked bar chart, which is the fastest way to triage which columns deserve the most scrutiny. `naniar::gg_miss_upset()` goes further and shows an UpSet-style plot of **combinations** of co-missing variables — essential once you have more than two or three variables with missingness, where a simple pairwise correlation plot becomes unreadable. For a formal (if limited) statistical check, `naniar::mcar_test()` implements Little's MCAR test, which assesses whether the observed pattern of missingness is consistent with MCAR by comparing means across missingness patterns; a significant result lets you rule out MCAR, but — consistent with Section 7.1 — a non-significant result does not prove MCAR, it only fails to contradict it, and the test says nothing about MAR versus MNAR.

A worked illustration, using a hypothetical trial-based economic dataset with utility (EQ-5D index) and healthcare cost columns:

```
library(tidyverse)
library(naniar)
library(visdat)

# trial_econ: one row per patient, with baseline and 12-week
# utility scores, and total healthcare cost over follow-up
trial_econ <- read_csv("trial_econ_extract.csv")

# Overview of variable types and missingness across the frame
vis_dat(trial_econ)
```

```r
# Missingness-focused heatmap, clustered by pattern
vis_miss(trial_econ, cluster = TRUE)

# Which variables carry the most missingness?
gg_miss_var(trial_econ, facet = trial_arm)

# Do utility and cost missingness co-occur?
gg_miss_upset(trial_econ)

# Formal test of the MCAR assumption (Little's test)
mcar_test(trial_econ %>% select(utility_12wk, total_cost, baseline_utility))
```

The `facet = trial_arm` argument to `gg_miss_var()` is worth dwelling on: splitting missingness summaries by randomised arm is one of the simplest and most informative checks available in a trial context, because differential missingness between arms (more dropout on the active treatment, say, due to side effects) is a strong, visually obvious signal against MCAR and a prompt to think hard about MAR versus MNAR before modelling.

For datasets too large or too wide for a single static plot — a claims warehouse with hundreds of cost and utilisation fields, for example — static `vis_miss()` heatmaps become hard to read. R's Shiny framework is commonly used to wrap these same `naniar`/`visdat` summaries into an interactive explorer: a reactive app where an analyst filters by site, time window, or variable subgroup and regenerates the missingness plot on demand, rather than pre-generating dozens of static figures. This is a thin application layer over the same underlying functions shown above, not a different statistical method, but it materially speeds up the exploratory phase on large real-world evidence extracts.

Common pitfall. A non-significant result from `mcar_test()` does not certify your data as MCAR — it only means the test found no evidence against it. Little's test checks whether missingness correlates with **observed** variables; it is structurally blind to MNAR, where missingness depends on the unobserved value itself. Treating a passed MCAR test as licence for a complete-case analysis is one of the most common silent sources of bias in HEOR cost-effectiveness write-ups.

Agency perspective. When documenting missing-data handling in a trial-based economic evaluation for a submission, ground your approach in Faria, Gomes, Epstein & White's "A Guide to Handling Missing Data in Cost-Effectiveness Analysis Conducted Within Randomised Controlled Trials" (PharmacoEconomics, 2014) — a widely cited, methodologically detailed guide that walks through exactly this mechanism-first, exploratory-then-sensitivity-analysis workflow and is the standard reference HTA reviewers expect to see cited.

7.3 Complete-case versus imputation

The simplest response to missing data is to ignore it: keep only the rows with no missing values and analyse those as if they were the whole dataset. This is complete-case analysis, also called listwise deletion, and it is the default behaviour of most R modelling functions (`lm()`, `glm()`, `t.test()` all silently drop incomplete rows unless told otherwise). Its appeal is honesty of a kind

— every value that enters the analysis was actually observed, nothing is fabricated — and under one specific condition it is a perfectly defensible choice: when missingness is genuinely MCAR (missing completely at random, SS7.1) and the proportion missing is small, say under 5%. Under true MCAR, the complete cases are a random subsample of the full cohort, so estimates remain unbiased; the only cost is a loss of statistical power from the smaller sample size.

That caveat matters because the cost is rarely "only" power. Two failure modes deserve attention. First, even under MCAR, if a non-trivial fraction of records is missing — 20%, 30%, sometimes more for a patient-reported utility instrument like the EQ-5D at follow-up visits — complete-case analysis can discard a third or more of a trial's data, inflating standard errors and widening confidence intervals around the ICER to the point of uselessness, even though the point estimate itself stays unbiased. Second, and more insidious, if missingness is MAR or MNAR (SS7.1), dropping incomplete rows does not merely cost power — it introduces bias, silently. If patients with worse quality of life are systematically more likely to skip the utility questionnaire (plausibly MNAR), the complete-case subsample overrepresents healthier patients, and the estimated mean QALY is biased upward. No warning is printed; `lm()` will fit happily and report a confidence interval that looks perfectly respectable while being centred on the wrong number.

A step up from deletion is single imputation: filling in each missing value with one plausible number and then analysing the completed dataset as though it had been fully observed all along. Common variants include mean or median imputation (replace missing cost or utility values with the column mean/median) and last-observation-carried-forward (LOCF), common in longitudinal cost-effectiveness follow-up, which carries a patient's last recorded value forward to fill subsequent missing visits. Both are easy to implement and both are dangerous for the same underlying reason: they treat a **guess** as if it were **data**. Once the gaps are filled with a single fabricated number, every downstream standard error, confidence interval, and p-value is computed under the pretence that all values were actually measured. The uncertainty introduced by the act of guessing never enters the calculation, so standard errors come out systematically too small and confidence intervals too narrow — the analysis reports more confidence in the result than the data actually support.

The following toy example makes this concrete using the cost/QALY tibble convention from Chapter 4. Twenty patients' utility values are simulated, 30% are set missing, and we compare the standard error of the mean utility under naive mean-imputation against the standard error obtained via proper multiple imputation (introduced in SS7.4).

```
library(tidyverse)
library(mice)

set.seed(2024)
n <- 20
truth <- tibble(
  patient_id = 1:n,
  utility = rnorm(n, mean = 0.75, sd = 0.10)
)

# Introduce MAR-ish missingness: lower utility more likely to be missing
obs <- truth |>
  mutate(
    miss_prob = plogis(-3 + 4 * (0.80 - utility)),
    utility_obs = if_else(runif(n) < miss_prob, NA_real_, utility)
  )
```

```r
# --- Naive mean imputation ---
mean_fill <- mean(obs$utility_obs, na.rm = TRUE)
naive <- obs |> mutate(utility_imp = if_else(is.na(utility_obs), mean_fill,
utility_obs))

se_naive <- sd(naive$utility_imp) / sqrt(n)
cat("Naive mean-imputation SE:", round(se_naive, 4), "\n")

# --- Proper multiple imputation (mice), previewed from SS7.4 ---
imp <- mice(obs |> select(utility_obs), m = 20, method = "pmm", maxit = 5,
            printFlag = FALSE, seed = 2024)
fits <- with(imp, t.test(utility_obs))
pooled <- pool(fits)
cat("Multiply-imputed pooled SE:", round(summary(pooled)$std.error, 4), "\n")
```

Running this comparison typically shows the naive mean-imputation standard error noticeably smaller than the mice-pooled standard error, because filling every gap with the identical constant `mean_fill` mechanically shrinks the completed dataset's variance and hides the fact that the true value of each missing observation was never actually known. LOCF carries an analogous problem in longitudinal cost data: carrying forward a value assumes no change occurred, which both biases trends (toward flat trajectories) and understates variability.

> **Common pitfall.** A standard error computed on a singly-imputed dataset answers the question "how uncertain would I be if these fabricated values were real?" — not "how uncertain am I, given that I had to fabricate them?" The two numbers are not the same, and the gap is exactly the imputation uncertainty that single-imputation methods discard.

The motivation for SS7.4 follows directly: we want a method that fills in plausible values **and** propagates the extra uncertainty from not knowing those values into the final standard errors. That is precisely what multiple imputation, combined via Rubin's rules, is built to do.

7.4 Multiple imputation (mice, Amelia, missForest)

Multiple imputation solves the understated-uncertainty problem of SS7.3 by refusing to commit to a single guess. Instead of filling each missing cell once, it creates several complete versions of the dataset — typically `m = 5` to `m = 50` — each with its own plausible, independently drawn set of imputed values, drawing on the observed relationships between variables. The analysis of interest (a mean, a regression, an ICER) is then run separately on every imputed dataset, and the `m` results are combined using Rubin's rules, which pool the point estimates by simple averaging and pool the standard errors by combining two components: the **within-imputation** variance (the usual sampling variance you would get from any one completed dataset) and the **between-imputation** variance (how much the estimate itself jumps around depending on which imputed values happened to be drawn). That between-imputation term is exactly the piece that single imputation discards, and is why multiply-imputed standard errors are honest about missing-data uncertainty in a way mean-imputation or LOCF never can be.

The workhorse R package for this is `mice` ("Multivariate Imputation by Chained Equations"). The core function, `mice()`, imputes one variable at a time conditional on all the others, cycling

through variables for several iterations (`maxit`) and repeating the whole process `m` times to produce `m` independent completed datasets. The `method` argument lets you choose an imputation model per variable type: `"pmm"` (predictive mean matching) is the default and generally preferred choice for continuous variables like cost or utility, because it imputes using **observed** donor values rather than inventing numbers outside the plausible range; `"logreg"` handles binary variables (e.g. treatment response yes/no); `"polyreg"` handles unordered categorical variables (e.g. disease stage categories).

```
library(tidyverse)
library(mice)

set.seed(42)
heor_data <- tibble(
  patient_id = 1:40,
  treatment  = rep(c("A", "B"), each = 20),
  cost       = rnorm(40, mean = 1200, sd = 300),
  qaly       = rnorm(40, mean = 0.70, sd = 0.08)
)

# Inject missingness into cost and qaly
heor_miss <- heor_data |>
  mutate(
    cost = if_else(runif(40) < 0.25, NA_real_, cost),
    qaly = if_else(runif(40) < 0.20, NA_real_, qaly)
  )

imp <- mice(
  heor_miss |> select(cost, qaly, treatment),
  m = 10, method = c(cost = "pmm", qaly = "pmm", treatment = ""),
  maxit = 5, printFlag = FALSE, seed = 42
)

# Inspect one completed dataset
completed_1 <- complete(imp, action = 1)

# Fit the model of interest on every imputed dataset, then pool per Rubin's rules
fit <- with(imp, lm(qaly ~ treatment + cost))
pooled_fit <- pool(fit)
summary(pooled_fit)
```

`summary(pooled_fit)` returns the pooled coefficient estimates together with standard errors, degrees of freedom, and p-values that already incorporate both within- and between-imputation variance — these are the numbers that belong in a report, not the output of any single `complete()`d dataset taken in isolation. The same pattern (`with()` then `pool()`) applies equally to pooling a simple mean utility, a regression-adjusted cost difference, or an ICER-contributing coefficient, as long as the estimator produces a coefficient and standard error that `pool()` can combine.

> **Difficult terminology: degrees of freedom (pooled multiple-imputation estimates).** Degrees of freedom is, informally, a number that reflects how much genuinely independent information went into an estimate — the more of it you have, the more precisely you can pin down a confidence interval or p-value threshold, and the narrower a confidence interval

can be for the same observed effect. With a single dataset and a simple regression, degrees of freedom is usually something intuitive like "number of patients minus number of parameters estimated" — a whole number you could count on your fingers. After Rubin's rules pool several imputed datasets together, though, the reported degrees of freedom is deliberately **not** a whole number (you might see 47.3, say) and is often noticeably smaller than what a single complete dataset of the same size would give you. That's not a bug — it's the pooling formula explicitly admitting a penalty: because you don't actually know the true missing values, only several plausible guesses at them, there is genuine extra uncertainty beyond ordinary sampling error, and the fractional degrees of freedom is the mechanism by which that uncertainty gets reflected in a wider confidence interval than you'd otherwise report. The more the imputed datasets disagree with each other (high between-imputation variance), the more degrees of freedom gets discounted.

The conventional way to report `mice::pool()` output is a coefficient table carrying the pooled estimate, its standard error, a 95% confidence interval, and the fraction of missing information (FMI) per coefficient — FMI quantifies how much the uncertainty around that specific coefficient is inflated by the missingness, which a bare point estimate cannot convey on its own:

How to report this.

Term	Estimate	SE	95% CI	FMI
Intercept	0.612	0.041	0.531, 0.693	0.18
treatment (B vs. A)	0.034	0.019	−0.004, 0.071	0.22
cost	−0.00004	0.00002	−0.00008, 0.00000	0.31

Estimates, SEs and 95% CIs are pooled across m = 10 imputed datasets via Rubin's rules; FMI (fraction of missing information) is highest for the `cost` coefficient, consistent with cost carrying the larger share of injected missingness in this toy example.

Table 17: Pooled multiple-imputation regression coefficients (Rubin's rules) for `qaly ~ treatment + cost`, m = 10 imputations

HEOR in practice. When reporting a cost-effectiveness analysis built on multiply-imputed data, state `m` (the number of imputations), the imputation method per variable, and the pooled standard errors explicitly in your methods section — reviewers increasingly expect this, and NICE/ISPOR-aligned reporting checklists treat undisclosed single imputation or unexplained listwise deletion as a methodological red flag.

`mice` is not the only option, and two alternatives are worth knowing when to reach for. `Amelia` (package `Amelia`, function `amelia()`) uses an EM-with-bootstrapping (EMB) algorithm assuming multivariate normality, and is particularly well suited to time-series or longitudinal panel data — it has built-in support for time and cross-sectional ID arguments that `mice` lacks, making it a natural choice for multi-wave follow-up cost diaries. `missForest` (package `missForest`, function `missForest()`) takes a completely different, nonparametric approach: it iteratively imputes miss-

ing values using random forests grown on the other variables, which makes it robust to nonlinear relationships and naturally handles mixed continuous/categorical data without needing separate method specifications per variable type — genuinely convenient for messy real-world HEOR datasets with a tangle of cost, utility, comorbidity, and categorical variables together.

The convenience of `missForest` comes with a real limitation, though: it produces a single completed dataset per run, not naturally plural imputations in the Rubin's-rules sense, so the standard between/within-imputation variance decomposition that `mice`'s `pool()` performs does not apply in the same way. Running `missForest` multiple times with different seeds and attempting to combine the results post hoc is an active area of methodological debate, not an out-of-the-box solved problem the way `mice::pool()` is. For an analysis whose final deliverable is a pooled standard error destined for a cost-effectiveness acceptability curve or a confidence interval around an ICER, `mice` (or `Amelia` for longitudinal structures) remains the better-supported choice; `missForest` is better reserved for exploratory imputation or as a preprocessing step feeding a single downstream prediction model, where you are less dependent on a correctly propagated variance estimate.

7.5 Sensitivity analysis, including tipping-point and pattern-mixture approaches

Multiple imputation under the missing-at-random assumption, as covered in SS7.4, produces a single headline estimate and a confidence interval that already account for imputation uncertainty. It is tempting to treat that output as the final word. It is not. Recall from SS7.1 that MCAR, MAR, and MNAR cannot be distinguished using the observed data alone — the very information that would tell you whether the unseen values differ systematically from the seen ones is, by definition, unseen. Multiple imputation by chained equations assumes MAR conditional on the variables in the imputation model. That assumption is plausible, sometimes highly plausible, but it is never verifiable from the dataset in front of you. Any analysis that stops after reporting "MI point estimate, MI confidence interval" without asking "what if the missing values are systematically worse (or better) than what the model predicted" is incomplete. Sensitivity analysis is the second half of a credible missing-data analysis, not an optional appendix.

Tipping-point analysis is the most direct way to operationalise this concern, and it is intuitive enough to explain to a clinical reviewer without a statistics background. The idea: take your primary imputation model's results, then systematically shift the imputed values for one or more groups by a fixed offset (often called delta), re-run the analysis, and see how extreme that offset has to become before your conclusion reverses. If a trivially small, clinically implausible delta is enough to flip the conclusion, your result is fragile. If only an extreme, clinically implausible delta would flip it, your result is robust to departures from MAR.

Concretely, suppose a trial comparing a new TB regimen against standard of care has missing six-month treatment-success outcomes, imputed under MAR, and the base-case result favours the new regimen. A tipping-point loop varies a delta applied only to the imputed (not observed) values in the intervention arm, recomputing the between-arm difference at each delta:

```
library(dplyr)

tipping_point <- function(data, deltas) {
  results <- vector("list", length(deltas))
  for (i in seq_along(deltas)) {
```

```
    d <- deltas[i]
    adjusted <- data |>
      mutate(success_adj = if_else(imputed & arm == "intervention",
                                   success - d, success))
    diff <- mean(adjusted$success_adj[adjusted$arm == "intervention"]) -
      mean(adjusted$success_adj[adjusted$arm == "control"])
    results[[i]] <- tibble(delta = d, risk_difference = diff)
  }
  bind_rows(results)
}

tipping_point(trial_data, deltas = seq(0, 0.30, by = 0.02))
```

The same logic extends to cost-effectiveness: apply a delta to imputed QALYs (or to imputed costs) before recomputing the ICER, and report the delta at which the ICER crosses the willingness-to-pay threshold lambda established in Chapter 4. The statistical output of this loop is only half the job — the other half is a clinical judgement call: is a delta of 0.02 in treatment-success probability, or 0.05 QALYs, something that a clinician or patient-representative would consider plausible given what is known about why patients dropped out? If the tipping delta is clinically absurd, the primary conclusion stands on solid ground. If it is well within the range of plausible informative dropout, the conclusion is fragile and should be reported as such.

How to report this.

Delta applied to imputed values	Risk difference	Favours
0.00 (base case, MAR)	+18.0 pp	Intervention
0.05	+13.0 pp	Intervention
0.10	+8.0 pp	Intervention
0.15	+3.0 pp	Intervention
0.17 (tipping point)	0.0 pp	Neutral
0.20	−2.0 pp	Control
0.25	−7.0 pp	Control
0.30	−12.0 pp	Control

Interpretation: the conclusion favouring the intervention is robust unless imputed successes in the intervention arm are overstated by roughly 0.17 or more — a departure from MAR that clinical reviewers judged implausible given the dropout reasons recorded.

Table 18: Tipping-point sensitivity analysis: six-month treatment-success risk difference (intervention minus control) as the delta applied to imputed intervention-arm successes increases, TB regimen trial

`mice` supports this kind of delta-adjustment directly: after generating imputations under MAR, you can post-process the imputed (but not observed) values in each completed dataset by adding a fixed

offset before pooling, which is exactly the tipping-point mechanic above applied across multiply-imputed datasets rather than a single completed dataset.

Pattern-mixture models offer a complementary, more structural way to think about departures from MAR. Rather than imputing a single plausible value and then perturbing it, a pattern-mixture approach stratifies the data by missingness pattern — for example, "completed all follow-up," "dropped out after visit 2," "dropped out after visit 4" — and allows each pattern to have its own outcome distribution. You then combine these pattern-specific distributions into an overall estimate using explicit, stated assumptions about what the missing-pattern groups' outcomes would have looked like had they been observed. Those assumptions are where the scientific judgement lives: you might assume patients who dropped out track the same trajectory as similar patients still in follow-up (akin to assuming MAR within the model), or you might assume they track the trajectory of patients in the control arm from that point forward (a "jump to reference" assumption), or that their outcomes would have worsened faster than observed completers (a "placebo-based" or worsening assumption suited to safety concerns).

This reference-based family of assumptions is precisely the territory covered by `rbmi`, a CRAN package developed for reference-based multiple imputation in clinical trials, built to implement exactly this kind of pattern-mixture reasoning — imputing missing values for a given patient according to the trajectory of a specified reference arm or reference pattern rather than that patient's own arm, as a formal sensitivity check against informative dropout. Rather than naming specific `rbmi` functions here, the conceptual takeaway for an HEOR analyst is this: pattern-mixture and reference-based MI let you ask "what is the most defensible pessimistic (or optimistic) story about why these data are missing, and how much does the answer change under that story?" — which is a different and complementary question to the tipping-point analysis's "how far would the data need to be wrong before the conclusion flips?" A submission that reports both together is materially more convincing than one relying on MAR imputation alone.

7.6 Missing data in cost and QALY data

Everything in SS7.1-7.5 applies to missing data of any kind, but cost and QALY data bring two additional headaches that a simple missing covariate — say, a baseline lab value — does not.

The first is distributional shape. Healthcare cost data are notoriously right-skewed and often zero-inflated: most patients incur modest costs, a small number incur very large costs (a hospitalisation, a surgical complication), and some incur essentially zero cost in a given period (no healthcare contact at all). Missingness in cost data is rarely "random noise around the mean" — a missing cost record for a patient who was lost to follow-up after an uneventful visit looks nothing like a missing cost record for a patient who disappeared from the dataset immediately before an expensive hospitalisation. Naive mean imputation — filling every missing cost with the sample mean — is especially dangerous here because it does two things at once: it ignores the skew (replacing a potentially very high or very low true value with a moderate average), and it artificially shrinks the variance of the cost distribution, which then distorts downstream statistics like confidence intervals around incremental cost. The problem compounds with health-state-related dropout: patients who leave a trial because their condition is deteriorating are disproportionately likely to have missing late-stage cost and utility data, and deteriorating health plausibly correlates with higher resource use (more hospital contact) or, in some systems, reduced contact (patients too unwell to attend). Either way

the missingness is informative — a form of MNAR — and treating it as MCAR by simply dropping or mean-filling those records biases the cost-effectiveness result in an unpredictable direction.

QALYs and the utility weights that compose them bring a second, structural constraint: utilities are bounded, typically on a scale from a negative value (states considered worse than death) up to 1 (full health), and that boundedness has to be respected by any imputation procedure. An imputation model built carelessly — for instance, a linear regression imputing utility values without constraint — can happily generate an imputed utility of 1.08 or produce an imputed cost that is negative, both of which are nonsensical and will silently corrupt a downstream QALY or ICER calculation if not caught.

The practical implication is that multiple imputation for cost and QALY data must be done on the natural scale appropriate to each variable, with constraints enforced rather than assumed away. `mice` supports exactly this kind of post-processing: imputed values for a given variable can be bounded, for example clamping imputed utilities to the plausible range before they are carried into the pooled dataset, or ensuring imputed costs cannot fall below zero. Some analysts additionally transform cost data (for example, imputing on a log scale and back-transforming) to better respect the skew described above, provided the back-transformation and any associated bias correction are documented.

```
library(mice)

imp <- mice(trial_data, m = 20, method = c(cost = "pmm", utility = "pmm"),
            seed = 123)

# Post-process: clamp imputed utilities to the valid range and
# ensure imputed costs are non-negative
imp_clamped <- imp
for (i in seq_len(imp$m)) {
  comp <- complete(imp, i)
  comp$utility <- pmin(pmax(comp$utility, -0.59), 1)
  comp$cost <- pmax(comp$cost, 0)
  imp_clamped$imp$utility[[i]] <- comp$utility[is.na(trial_data$utility)]
  imp_clamped$imp$cost[[i]] <- comp$cost[is.na(trial_data$cost)]
}
```

> **Common pitfall.** Mean-imputing missing costs or utilities, or imputing them with an unconstrained model that ignores the [0,1]-ish bound on utilities and the non-negativity of costs, will silently distort the pooled cost and QALY estimates that feed into every downstream ICER calculation — always impute on the natural, bounded scale and sanity-check the completed dataset's range before proceeding.

This matters directly for the cost/QALY tibble pipeline introduced in Chapter 4: the `treatment`/`cost`/`qaly` tibble that later feeds dominance pruning and ICER computation is exactly the kind of dataset where undetected missing-data mishandling upstream becomes invisible downstream. A dominance-pruning step applied to a tibble built from carelessly mean-imputed costs will happily prune strategies and rank the remainder with complete confidence, with no visible trace that the input costs were distorted. Any analyst building that pipeline from trial or registry data with incomplete cost or utility records should treat the imputation step as upstream quality control for the entire economic model, not as a mechanical pre-processing formality.

7.7 Reporting missing data handling for HTA

A technically sound missing-data analysis that is reported badly is, from an HTA reviewer's perspective, close to useless — the reviewer cannot assess what they cannot see. CHEERS 2022, already cited in Chapter 3 in connection with its item on methods-reporting detail, contains a specific checklist item requiring transparent reporting of how missing data were handled, and that item is the natural anchor for what a submission needs to include on this topic.

At minimum, a submission should report four things. First, the amount and pattern of missingness, broken down by study arm or group, not just as an aggregate percentage — a 10% missingness rate split evenly across arms tells a very different story from 2% missing in one arm and 18% in the other. This is where the visualisation tools from SS7.2 (`naniar`, `visdat`) earn their keep as reporting aids, not just exploratory ones: a simple missingness-by-arm table or plot communicates more to a reviewer in seconds than a paragraph of prose. Second, the assumed missing-data mechanism and the justification for that assumption — stating "we assumed data were missing at random conditional on baseline severity, treatment arm, and visit number" is a reportable, falsifiable claim; stating "data were assumed missing at random" with no stated conditioning variables is not. Third, the primary analysis method, stated with enough specificity to be reproducible: for example, "missing six-month utility values were handled using multiple imputation by chained equations with m = 50 imputations, imputing conditional on baseline EQ-5D, age, treatment arm, and disease severity, with Rubin's rules used to pool estimates." Fourth, and often the most neglected, the sensitivity analyses performed and their result — specifically whether the tipping-point delta needed to overturn the primary conclusion was judged clinically plausible or implausible, and whether any pattern-mixture or reference-based alternative assumptions (SS7.5) changed the headline cost-effectiveness conclusion.

A useful discipline is to draft this reporting as a short, self-contained paragraph the moment the imputation and sensitivity analyses are finalised, rather than reconstructing it from memory at submission-writing time:

```
# Illustrative structure for the reporting paragraph's data
missingness_summary <- trial_data |>
  dplyr::group_by(arm) |>
  dplyr::summarise(
    n = dplyr::n(),
    n_missing_utility = sum(is.na(utility_6m)),
    pct_missing = round(100 * n_missing_utility / n, 1)
  )
```

How to report this.

Arm	N randomised	N missing utility (6m)	% missing
Intervention	120	22	18.3
Control	118	9	7.6
Overall	238	31	13.0

Mechanism assumed: MAR, conditional on baseline EQ-5D, age, treatment arm, and disease severity. The pronounced imbalance between arms (18.3% vs 7.6%) is itself evidence against MCAR and motivates the tipping-point sensitivity analysis in Table 18.

Table 19: Missing-data summary for the primary six-month utility endpoint, by randomised arm

Beyond CHEERS 2022, the ICH E9(R1) addendum on estimands and sensitivity analysis in clinical trials is the relevant reference for how intercurrent events — treatment discontinuation, rescue medication, death before an outcome can be assessed — and the missing data they generate should be framed from the outset of trial design, not patched on after the fact during analysis. ICH E9(R1)'s estimand framework asks the analyst to specify, before unblinding, how intercurrent events will be handled conceptually (treatment-policy, hypothetical, composite, while-on-treatment, or principal-stratum strategies), which in turn determines what the "right" missing-data assumption and primary analysis even are. An HTA submission that can point to a pre-specified estimand strategy, consistent with ICH E9(R1), alongside a CHEERS-compliant missing-data reporting paragraph, presents a far more defensible evidentiary package than one where the missing-data method appears to have been chosen after seeing which approach gave the most favourable result.

Difficult terminology: ICH E9(R1) intercurrent-event strategies (treatment-policy, hypothetical, composite, while-on-treatment, principal-stratum). An intercurrent event is anything that happens **after** randomisation that complicates the simple question "did the treatment work?" — a patient stops the study drug due to side effects, takes rescue medication, switches to the other arm, or dies before the outcome of interest can even be measured. The ICH E9(R1) addendum (an internationally agreed regulatory guidance document) insists that before a trial even starts, the protocol must say explicitly **which version** of "treatment effect" it is trying to measure in the presence of such events, because different choices quietly answer different clinical questions from the same raw data. The five named strategies are the menu of options: **treatment-policy** counts whatever happened to the patient regardless of whether they stayed on the assigned treatment (closest to "real-world effectiveness, warts and all"); **hypothetical** asks what would have happened in a hypothetical world where the intercurrent event never occurred (e.g., had rescue medication not been available) — this requires an assumption, since that world was never observed; **composite** folds the intercurrent event itself into the definition of a bad outcome (e.g., "treatment failure" includes both clinical worsening **and** stopping due to side effects, counted the same way); **while-on-treatment** only asks the question for as long as the patient was actually taking the assigned treatment, and stops counting afterward; and **principal-stratum** restricts the question to a particular subgroup defined by what would have happened to them under both treatments (e.g., only

patients who would have tolerated either drug) — the most statistically demanding of the five, since that subgroup can't be directly observed. A useful clinical analogy: if a surgical trial's patients sometimes need a second emergency surgery, "treatment-policy" reports outcomes for everyone regardless of whether a second surgery happened, while "hypothetical" tries to estimate what the outcome would have been if no second surgery had ever been needed — two legitimately different, both useful, questions from the same trial.

Agency perspective. CHEERS 2022 requires explicit reporting of the amount, pattern, and handling of missing data as part of a health economic evaluation, and ICH E9(R1) requires that intercurrent events and their associated missing-data handling strategy be specified in relation to a pre-defined estimand; together they are the two reference points an HTA-facing missing-data section should be written against.

Missing utility values are not only a missing-data problem — they are frequently a symptom of exactly the population, instrument, and measurement-timing choices that Chapter 8 addresses directly: how utilities are elicited, which instrument captures them, and how those choices interact with trial dropout. The next chapter turns to health-related quality of life and utility measurement in full.

Chapter 8. Health-Related Quality of Life and Utilities

Learning objectives

- Distinguish generic preference-based instruments (EQ-5D, SF-6D, HUI) from disease-specific instruments, and explain why HTA reference cases require the former for the primary QALY calculation.
- Score EQ-5D health-state profiles into utility indices using R's `eq5d` package and a correctly-chosen country value set.
- Build and validate a mapping (cross-walk) algorithm to predict EQ-5D-equivalent utilities from a disease-specific instrument.
- Analyse bounded, ceiling-spiked utility data with censored regression, beta regression, or finite mixture models rather than naive OLS.
- Apply utility decrements and age/sex-adjustment correctly within a decision-analytic model, and layer background mortality onto disease-specific mortality.
- Explain the three classic utility-elicitation methods — time trade-off, standard gamble, and discrete choice experiments — and how they underlie value-set construction.

Chapter 7 closed by flagging that missing utility values are often a symptom of the instrument, elicitation method, and measurement timing chosen for a trial — this chapter picks that thread up directly and covers how utilities are measured, valued, and modelled in the first place.

8.1 Generic and disease-specific instruments (EQ-5D, SF-6D, HUI, etc.)

A utility, in the HEOR sense that Chapter 4 introduced, is a single number that expresses how good or bad a health state is, anchored so that 1 represents full health and 0 represents dead. States judged worse than death carry negative values, which is exactly why Chapter 7 flagged the UK EQ-5D-3L tariff's floor of roughly -0.59. Utilities exist to do one specific job: they are the weight multiplied by time to convert raw life-years into quality-adjusted life-years. A year spent at utility 0.7 contributes 0.7 QALYs; a year at utility 1.0 contributes a full QALY. Everything in this chapter is really about how that single number gets produced, and the honest limitations of the instruments that produce it.

The instruments used to generate utilities fall into two broad families. The first is the generic, preference-based, multi-attribute family. These describe health using a small number of dimensions that are meant to apply to essentially anyone, regardless of diagnosis, and each possible combination of dimension levels has (or can be assigned) a population-derived preference weight. EQ-5D is the dominant example. EQ-5D-3L covers five dimensions — mobility, self-care, usual activities, pain/discomfort, and anxiety/depression — each scored at one of three levels (no problems, some problems, extreme problems), giving $3^5 = 243$ possible health states. EQ-5D-5L uses the same five dimensions but expands each to five levels (no, slight, moderate, severe, extreme problems), giving $5^5 = 3125$ states and materially reducing ceiling effects at the "no problems" end. SF-6D is

derived from the longer SF-36 or shorter SF-12 health-status questionnaires by selecting a subset of items that map onto six dimensions (physical functioning, role limitations, social functioning, pain, mental health, and vitality) and applying preference weights to that subset. HUI2 and HUI3, the Health Utilities Index instruments, use their own dimension sets — HUI3, the more commonly used of the two, covers vision, hearing, speech, ambulation, dexterity, emotion, cognition, and pain, each with multiple levels. All of these are "generic" in that the dimensions were chosen to be meaningful across disease areas, and all are "preference-based" in that someone, somewhere, ran a valuation study asking members of the public to trade off time or risk against being in a described health state, and used those responses to build the scoring weights — the value set or tariff that Section 8.2 unpacks.

The second family is disease-specific instruments: questionnaires built to capture the symptoms and functional domains that matter most within one condition. A TB treatment trial might use a TB-specific symptom and quality-of-life questionnaire tracking cough, sputum, fever, and treatment-related adverse effects; an oncology trial might use the EORTC QLQ-C30 or a tumour-specific module. These instruments are often considerably more sensitive to clinically meaningful change within the disease they were built for, because every item was chosen for its relevance to that condition, rather than for being a dimension that also has to make sense for, say, a hip-replacement patient or someone with depression.

This is precisely where the generic-versus-disease-specific trade-off becomes unavoidable for anyone doing HTA work. NICE's reference case, and comparable guidance under development for HTAIn, requires a generic preference-based instrument (EQ-5D specifically, for NICE) for the QALYs used in the base-case cost-effectiveness analysis. The reason is structural, not a matter of taste: resource-allocation bodies are routinely comparing a TB regimen against a cardiology intervention against an oncology drug, all competing for the same budget against the same willingness-to-pay threshold λ. That comparison is only coherent if "1 QALY" means the same thing regardless of which disease produced it, and that requires a common, cross-disease utility currency — which is exactly what a generic preference-based instrument is designed to supply, and what a disease-specific instrument, by construction, is not. A disease-specific score has no population preference weighting behind it and no natural anchor at 0 = dead and 1 = full health; it cannot be substituted directly into a QALY calculation without some bridge back to a utility scale, which is the mapping (or "crosswalk") problem that Section 8.3 takes up.

The honest cost of this requirement is sensitivity. Because EQ-5D's five dimensions are broad and generic, a TB treatment effect that substantially reduces cough and sputum but leaves mobility, self-care, usual activities, pain, and anxiety largely unchanged may show little or no movement on EQ-5D, even though patients and clinicians would recognise a real improvement. This is a known, published limitation, not a flaw to paper over: trial teams frequently collect both a generic preference-based instrument (for the regulatory/HTA utility) and a disease-specific instrument (for clinical sensitivity and secondary endpoints) side by side, precisely because neither alone tells the whole story.

> **HEOR in practice.** When you see a trial report a large, clinically important improvement on a disease-specific QoL questionnaire alongside a flat or barely-moved EQ-5D utility, do not assume the EQ-5D data is wrong or the analyst mis-scored it. Check first whether the dimensions EQ-5D captures are the ones the disease actually affects — a TB regimen that resolves cough and fatigue but doesn't touch mobility or self-care can legitimately show a

> small EQ-5D gain despite a genuinely better patient experience. The instrument's breadth, not an error, is usually the explanation.

8.2 Value sets and scoring in R (eq5d)

An EQ-5D questionnaire, on its own, only produces a health-state description — a short string of digits, one per dimension, recording the level the respondent selected. For EQ-5D-5L this looks like "11223": no problems with mobility, no problems with self-care, slight problems with usual activities, moderate pain/discomfort, moderate anxiety/depression. That string is not yet a utility. Turning it into a single number anchored at 1 (full health) and 0 (dead) requires a value set, often called a tariff: a table of preference weights, one for every possible combination of dimension levels, derived from a population valuation study. For the UK EQ-5D-3L value set, those weights came from a time trade-off (TTO) study fielded in a representative sample of the UK public, in which respondents were asked to trade years of life in an impaired health state against fewer years in full health; the EuroQol Group's EQ-5D-5L valuation protocol (covered in more depth in Section 8.6) extended this with a combined TTO and discrete choice experiment design. The crucial point is that a value set is country-specific: it reflects the preferences of the population it was elicited in, and different countries' general publics do not value the same health-state descriptions identically.

This is where the CRAN package `eq5d` (Craig, Pickard, and colleagues) becomes the practical tool. Its central function, `eq5d()`, takes a data frame of respondent-level dimension scores — the digit-per-dimension profile for either the 3L or 5L version — and an argument specifying which national value set to apply, returning a single utility index per respondent computed against that value set's published weights. The package bundles several countries' published value sets so that the same health-state profile can be scored consistently against, say, a UK tariff or a different country's tariff, without the analyst having to hand-code the weight lookup. (If you need the exact argument names and value-set codes, check `?eq5d` in your installed version rather than relying on a syntax from memory — argument spellings and the set of bundled countries have evolved across package releases.)

A toy workflow illustrates the shape of the task, independent of the exact argument names your installed version expects:

```
library(eq5d)

# Toy EQ-5D-5L health state profiles: one row per patient,
# one column per dimension (mobility, self-care, usual activities,
# pain/discomfort, anxiety/depression), each scored 1-5.
health_states <- data.frame(
  MO = c(1, 2, 3),
  SC = c(1, 1, 2),
  UA = c(2, 2, 3),
  PD = c(2, 3, 4),
  AD = c(1, 2, 3)
)

# Conceptual scoring step: apply a named country value set to convert
# each row's digit profile into a utility index anchored at 1 = full
# health, 0 = dead. Consult ?eq5d in your installed version for the
```

```
# exact argument names/spelling before running this on real data.
# utilities <- eq5d(scores = health_states, version = "5L", country = "UK")
# utilities
```

The reason the country argument matters so much is not a technicality — it is a well-documented source of real HTA submission errors. A UK NICE submission scored against a US value set (or vice versa) will produce numerically different utilities for the identical health-state description, because the two populations' revealed preferences for, say, "extreme pain/discomfort" are not the same. NICE's reference case expects EQ-5D-3L scored with the UK value set specifically (with EQ-5D-5L data generally expected to be mapped onto the 3L value set per NICE's published position, rather than scored with an EQ-5D-5L value set directly) — using the wrong country's tariff, or the wrong version-to-value-set pairing, is a substantive methodological error, not a stylistic choice, and reviewers will query it.

This also bears directly on HTAIn's situation. India does not yet have as long-established and widely validated a national EQ-5D value set as the UK's; a UK-style TTO valuation study in a representative Indian population is a substantial undertaking that has only partially matured. In practice, analysts working on Indian HTA submissions have sometimes relied on a crosswalk or value-set transfer from another country (or a regional proxy) as a pragmatic substitute, flagging it explicitly as a limitation rather than treating it as equivalent to a locally-derived tariff. That gap — and what a defensible crosswalk does and doesn't justify — is exactly the kind of LMIC-relevant methods question worth tracking as India's own value-set work progresses.

> **Agency perspective.** NICE's "NICE health technology evaluations: the manual" (PMG36) sets out the reference-case preference for EQ-5D as the measure of health-related quality of life for the UK reference case, with EQ-5D-3L (UK value set) as the default and specific guidance on how EQ-5D-5L data should be handled in relation to it. If you are preparing a submission that leans on EQ-5D utilities, check the current PMG36 text directly rather than relying on a secondary summary — NICE's position on 5L-to-3L handling has been updated over time.

8.3 Mapping algorithms (cross-walks and regression-based mapping)

Trials rarely collect every outcome measure an HTA body will eventually want. A pragmatic trial in TB treatment-shortening might collect a disease-specific symptom score or a clinical endpoint like time-to-culture-conversion; an oncology trial might collect FACT-G or an EORTC QLQ module. These instruments are excellent at detecting disease-specific change but they do not, by themselves, produce a generic preference-based utility value. An HTA reference case, however, almost always demands a utility anchored to a generic instrument such as EQ-5D, because that is what lets the QALY calculation sit on a common scale across disease areas and across submissions. When a trial has not collected EQ-5D at all, or has collected it too sparsely to be usable, the analyst is stuck with a dataset that has the clinical story but not the utility currency the model needs.

The standard fix is "mapping," sometimes called cross-walking. The logic is: find (or commission) an external dataset, ideally in a similar patient population, where both the disease-specific instrument and EQ-5D were administered to the same people at the same time. Fit a statistical model in that external dataset that predicts an EQ-5D-equivalent utility from the disease-specific scores. Then

apply that fitted algorithm to the trial's disease-specific scores to generate predicted utilities for the economic model. The trial itself may never have asked a single EQ-5D question; the mapping algorithm substitutes for that missing measurement, provided the external dataset is a defensible match for the trial population and condition severity range.

There are two broad families of mapping approach, and the distinction matters for how much of EQ-5D's internal structure survives the exercise.

- **Direct (response) mapping** models each EQ-5D dimension separately — for the 3L instrument, mobility, self-care, usual activities, pain/discomfort, and anxiety/depression, each with its own ordered response levels. Because the dimension responses are ordered categorical outcomes rather than continuous ones, this is typically done with multinomial or, more naturally given the ordering, proportional-odds ordinal regression — `MASS::polr()` is the standard base-R-adjacent tool for a single ordinal dimension, fit once per dimension against the predictors derived from the disease-specific instrument. The predicted dimension-level response profile for each respondent is then run back through the standard EQ-5D value set to produce a predicted utility. This route is more faithful to how EQ-5D utilities are actually constructed — it respects the fact that a utility index is a deterministic function of five discrete dimension responses, not a free-floating continuous number — but it means fitting and maintaining five separate models and then composing them correctly through the value set algebra.
- **Indirect (utility/value) mapping** skips the dimension-level detail entirely and regresses the EQ-5D utility index itself — the single number that comes out after the value set has already been applied — directly against the disease-specific scores, using ordinary regression methods such as `lm()` or a generalized linear model via `glm()`. This is markedly simpler to specify, fit, and report, and is the more common choice in practice precisely because of that simplicity. The cost is that it throws away the compositional structure of EQ-5D — two very different dimension profiles can map to the same utility value, and the model has no way to represent that heterogeneity or to use dimension-specific external information.

Whichever family is chosen, the indirect route has a well-known statistical trap: fitting `lm()` directly to a continuous utility index via ordinary least squares is a common choice in the mapping literature but a theoretically poor one. EQ-5D utility values are not normally distributed — they are bounded above at 1.0 (full health), often have a visible spike or clustering of observations exactly at that ceiling, and are left-skewed with a long tail toward poor health states and, for 3L value sets, can take values below zero. OLS assumes none of that: it allows predicted values above 1.0 or in otherwise impossible regions, and its homoscedastic-normal-error assumption is a poor description of a variable with a boundary spike. This is exactly the motivation for the more careful bounded-outcome methods covered in SS8.4 — censored regression, beta regression, and mixture models — all of which are designed to respect the ceiling and the skew that OLS ignores.

> **Difficult terminology: homoscedasticity.** Homoscedasticity is the assumption, built into ordinary linear regression, that the "scatter" of errors around the fitted line is roughly the same size everywhere — whether you're predicting a utility of 0.9 or 0.3, the model assumes the typical prediction error is about the same magnitude. (Its opposite, **heteroscedasticity**, means the scatter changes depending on where you are on the scale.) A clinical analogy: imagine measuring blood pressure readings in a cuff that's equally noisy whether the true reading is 90 or 190 — that's homoscedastic. EQ-5D utility data violate this badly: close to the 1.0 ceiling, a large fraction of respondents report exactly full health, so the "noise" clusters tightly at that

> boundary, while scatter for people in poorer health states can be much wider. Fitting OLS anyway doesn't just produce a slightly-off standard error — the model is implicitly assuming a pattern of noise that the data visibly don't have, which is part of why beta regression and mixture models (built to handle changing variance near a boundary) are preferred for this kind of outcome.

A mapping algorithm is only as trustworthy as its out-of-sample performance, and this is where submissions are most often challenged. Fitting a model in the external dataset and reporting how well it fits that same data (in-sample R-squared, in-sample residuals) tells a reviewer almost nothing about how the algorithm will behave when applied to a different trial population. The expected standard is some form of cross-validation — holding out a fold of the external dataset, fitting on the remainder, predicting the held-out fold, and reporting the discrepancy, typically as mean absolute error (MAE) or root-mean-squared error, between predicted and observed EQ-5D utilities. Algorithms that perform well on average but poorly at the extremes of the health-state range (very good or very poor health, which is often exactly where trial arms diverge) deserve particular scrutiny, because that is precisely the region driving the incremental QALY gain the economic model is built to estimate.

8.4 Analysing utility data: censoring, bounded distributions, beta regression, mixture models

Once utility values are in hand — whether collected directly via EQ-5D in the trial or produced by a mapping algorithm as in SS8.3 — the temptation is to treat them like any other continuous outcome and reach for `lm()`. Resist that temptation. EQ-5D utility distributions have a shape that ordinary linear regression's assumptions do not describe well. Full health (utility = 1.0) is a genuinely common and clinically meaningful response, so in most patient samples there is a visible spike or clustering of observations exactly at 1.0 — a ceiling effect, not a coincidence of rounding. Below that ceiling, the remaining distribution is typically left-skewed, with a longer tail stretching toward poorer health states. For 3L-style value sets that permit utilities below zero (states considered worse than death), there can be a further discontinuity or secondary clustering near or below zero. A method that assumes symmetric, unbounded, constant-variance normal errors — exactly what OLS assumes — will routinely predict impossible values above 1.0, misrepresent the variance near the ceiling, and bias both point estimates and confidence intervals.

Three families of real, well-documented approaches handle this more honestly.

- **Censored (Tobit-style) regression** treats the ceiling at 1.0 (and, where relevant, a floor) as genuine censoring rather than noise: the "true" underlying health preference may extend above 1.0 for some respondents, but the EQ-5D scale cannot record it, so observations piled at the boundary are modelled as censored rather than as ordinary data points. Two real CRAN packages implement this for utility-type data: `VGAM::vglm()`, which supports a tobit family among its very large catalogue of vector generalized linear/additive models, and the purpose-built `censReg` package, whose `censReg()` function fits Tobit-type censored regression with a syntax close to `lm()`/`glm()`. Either is a legitimate choice; `censReg()` is the more direct and commonly cited for univariate ceiling-censored utility outcomes.
- **Beta regression** exploits the fact that, apart from the boundary spikes, a utility index confined to (0, 1) looks much like a Beta-distributed variable — flexible enough to capture skew and

heteroscedasticity that increases or decreases near the bounds. The practical obstacle is that the Beta distribution is defined on the **open** interval (0, 1), excluding the exact endpoints, while real utility data routinely contain exact 1s (and exact 0s for some value sets). The standard workaround is a small linear rescaling of the outcome before fitting — squeezing the data just inside (0, 1) — after which the `betareg` package's `betareg()` function fits the regression with a logit (or other) link on the mean and, optionally, a separate model for the precision/dispersion parameter.

- **Finite mixture models** take the ceiling spike seriously as a distinct subpopulation rather than an inconvenience to transform away: conceptually, the observed data are treated as a mixture of a point mass exactly at 1.0 (representing respondents in full health) and a continuous component, such as a Beta or truncated normal distribution, describing everyone else. The model estimates both the mixing proportion (what fraction of the sample sits at the ceiling) and the parameters of the continuous piece, which can better represent the genuinely bimodal-looking shape of utility data than beta regression alone. This is more involved to specify and fit; `flexmix` is a real CRAN package for finite mixture modelling in R and is a reasonable starting point if this route is pursued, though for most submissions a beta regression with a sensible rescaling is the practical default.

A worked example with beta regression on a small toy EQ-5D-type dataset, illustrating the rescaling step and the model fit:

```
library(betareg)

# toy utility data: mix of observations with a visible ceiling spike
set.seed(123)
n <- 200
age <- round(rnorm(n, 55, 12))
severity <- sample(c("mild", "moderate", "severe"), n, replace = TRUE,
                   prob = c(0.4, 0.4, 0.2))

# simulate utilities with ceiling clustering at 1.0
raw_utility <- pmin(1, rbeta(n, 6, 2) + ifelse(severity == "mild", 0.15, 0))

df <- data.frame(utility = raw_utility, age = age, severity = severity)

# betareg requires outcomes strictly inside (0, 1); rescale to avoid
# exact boundary values using the standard Smithson-Verkuilen transform
n_obs <- nrow(df)
df$utility_resc <- (df$utility * (n_obs - 1) + 0.5) / n_obs

fit_beta <- betareg(utility_resc ~ age + severity, data = df)
summary(fit_beta)

# predicted mean utilities on the original scale
df$pred_utility <- predict(fit_beta, type = "response")
```

The fitted coefficients are on the logit-of-the-mean scale, so they describe proportional shifts toward or away from full health rather than additive utility-point changes — a feature, not a bug, given the bounded outcome, but one that needs stating explicitly when results are reported to a non-technical HTA reviewer.

Difficult terminology: logit link (in beta regression). A "link function" is the piece of machinery that lets a regression model handle an outcome that isn't a plain, unbounded continuous number — it's a mathematical conversion applied before fitting, then reversed afterward for interpretation. The logit link specifically takes a value confined to (0, 1) — like a probability, or here a rescaled utility — and stretches it onto an unbounded scale running from minus infinity to plus infinity, via the formula $\log\left(\frac{p}{1-p}\right)$. Ordinary regression machinery, which is built to predict any real number, can then operate freely on that stretched scale without ever risking a prediction outside (0, 1); converting the fitted line's predictions back through the inverse logit guarantees every predicted value lands back between 0 and 1, no matter how extreme the predictors are. It's conceptually identical to the logistic regression you may already know from predicting a binary clinical outcome (e.g. mortality) — same transformation, same reason for using it (keeping predictions between 0 and 1) — just applied here to a continuous utility mean rather than a 0/1 outcome. The practical consequence, as the previous paragraph notes, is that beta-regression coefficients describe **proportional** movement toward or away from full health, not a fixed number of utility points per unit of the predictor — so a coefficient can't be read the same way an OLS coefficient would be.

Before any regression is reported, a submission should show the descriptive shape of the utility data the model is addressing — mean utility by subgroup alongside the proportion of observations sitting exactly at the ceiling, since that ceiling spike is the whole reason OLS was rejected in favour of beta regression:

How to report this.

Severity	N	Mean utility	SD	% at ceiling (= 1.0)
Mild	71	0.892	0.114	25.4%
Moderate	83	0.719	0.159	0.0%
Severe	46	0.752	0.159	0.0%
Total N = 200				

Table 20: Mean (SD) EQ-5D-type utility and ceiling-spike proportion by disease severity subgroup, toy dataset

Common pitfall. Fitting `lm()` directly to a raw EQ-5D utility index, or handing over a mapping algorithm built in SS8.3 without reporting its out-of-sample predictive performance, are two sides of the same error: both substitute a convenient method for a validated one. OLS on a ceiling-spiked, bounded outcome can predict utilities above 1.0 and will misstate uncertainty exactly where the incremental QALY calculation is most sensitive. A mapping algorithm reported only with in-sample fit statistics tells a reviewer nothing about how it behaves on a different trial population. Before either is used in a submission, show cross-validated performance — MAE or RMSE on held-out data for a mapping algorithm, and residual/ceiling diagnostics for a utility regression — not just a coefficient table.

8.5 Utility decrements, age-adjusted utilities and background mortality

A **utility decrement** is the loss in health-state utility associated with a specific clinical event, adverse event, or disease complication, expressed relative to some baseline utility. For example, a grade 3-4 chemotherapy-related adverse event might carry a utility decrement of around −0.05 to −0.10, applied for the duration the event is assumed to affect the patient (often a single model cycle, sometimes longer for persistent toxicities). Rather than measuring these decrements directly in every trial — which would require administering EQ-5D or SF-6D specifically during the adverse event window, something many trials never do — modellers routinely pull decrements from the published literature. This is the same literature-as-data-source logic introduced in Chapter 5′s discussion of secondary utility sources (SS5.3), operationalised through the systematic literature review methodology covered in Chapter 6: a systematic search identifies prior utility studies in the relevant condition, and the decrement estimates feed into the decision-analytic model as model inputs with their own uncertainty distributions, rather than being derived from the trial dataset at hand.

Within a decision-analytic model — the Markov cohort models and decision trees that Chapters 17-20 build out in full — decrements are combined with a baseline health-state utility in one of two ways:

- **Additive**: utility = baseline utility + decrement (where the decrement is negative), e.g. 0.75 - 0.08 = 0.67 while the adverse event is active.
- **Multiplicative**: utility = baseline utility × (1 + proportional decrement), e.g. 0.75 × (1 - 0.10) = 0.675.

Both are common in practice, and for a single decrement the numerical difference is often small. The choice matters more once several decrements, age-adjustments, and disease-progression utility changes are stacked across a long time horizon, as discussed below.

A related but distinct concept is **age-adjustment**. Health-state utility is not a fixed property of a diagnosis alone; a patient's rating of their own health, and the population reference values against which models are often calibrated, tend to decline with age even among people who are otherwise well. General-population EQ-5D normative (reference) data, published for many countries alongside the country-specific value-set studies discussed in SS8.3, report mean utility by age band and sex — typically declining from values close to 1 in young adults into the 0.7-0.8 range in the oldest age bands, with some variation by value set. This matters because a disease-specific "full health" utility estimated in, say, a trial population with a mean age of 45 should not simply be projected unchanged onto a modelled cohort that ages into its 70s and 80s over a long time horizon; doing so would imply a disease-free elderly cohort member is healthier than the general elderly population, which is implausible.

The conventional fix is a **multiplicative age/sex-adjustment**: the disease-specific utility is scaled by the ratio of the population norm utility at the cohort's current age/sex to a fixed reference (often the norm at trial baseline age, or simply applied as utility(disease) × norm(age, sex) / norm(baseline age, sex)). This keeps the disease utility anchored to how the general population's health is expected to decline, without requiring a separate age-specific disease-utility estimate for every cycle. A purely **additive** age adjustment (subtracting a fixed age-related decrement each cycle) is simpler to code but is more prone to producing negative or clearly implausible utilities once combined with several other additive decrements (treatment toxicity, disease progression, comorbidity) over a 20-

or 30-year model horizon — there is no multiplicative floor stopping the cumulative subtraction from running arbitrarily low. Multiplicative approaches, by contrast, naturally compress toward (but rarely cross) zero, which is one reason HTA bodies and modellers often prefer them for long-horizon cohort models, alongside the practical argument that proportional age decline is closer to what the normative data actually show.

Background mortality is the other half of keeping a long-horizon Markov model plausible, and it connects directly to Chapter 4's treatment of discounting and half-cycle correction. A cohort trace needs an age- and sex-specific **all-cause** (background) mortality rate — typically sourced from a national life table — layered on top of any disease-specific mortality transition estimated from trial or registry data. Without this, the model would implicitly assume that a cohort member who survives the disease being modelled never dies of anything else, which overstates survival and therefore overstates total QALYs and costs accrued in later cycles. The half-cycle correction from Chapter 4 still applies here: since death (from either cause) is assumed to occur, on average, mid-cycle rather than precisely at cycle boundaries, failing to half-cycle-correct the mortality-driven QALY accrual introduces a small but systematic bias, compounding over discounted future cycles.

A simple illustration of an age-band multiplier applied to a toy disease utility:

```
# toy age-band utility multipliers (illustrative, not a real norm dataset)
age_norms <- data.frame(
  age_band = c("18-44", "45-54", "55-64", "65-74", "75+"),
  norm_utility = c(0.93, 0.88, 0.83, 0.78, 0.72)
)

baseline_disease_utility <- 0.80   # disease-specific utility at trial baseline
age
baseline_norm <- age_norms$norm_utility[age_norms$age_band == "45-54"]

age_adjust <- function(age_band, disease_utility = baseline_disease_utility) {
  norm_now <- age_norms$norm_utility[age_norms$age_band == age_band]
  disease_utility * (norm_now / baseline_norm)
}

sapply(age_norms$age_band, age_adjust)
```

The resulting age-banded utilities are exactly what belongs in a model-inputs table, since a reviewer needs to see the population norm, the derived multiplier, and the resulting age-adjusted disease utility side by side rather than re-deriving them from the code:

How to report this.

Age band	Population norm utility	Norm ratio vs. baseline band	Age-adjusted disease utility
18-44	0.93	1.057	0.846
45-54 (baseline)	0.88	1.000	0.800
55-64	0.83	0.943	0.754
65-74	0.78	0.886	0.709
75+	0.72	0.818	0.655

Age-adjusted disease utility = baseline disease utility (0.80) × (norm utility at current age band / norm utility at baseline band). Norms are illustrative toy values, not a published national dataset.

Table 21: Multiplicative age-adjustment of a toy disease-specific utility (baseline 0.80 at age 45-54) against general-population EQ-5D norms

This multiplicative rescaling ensures that as the modelled cohort ages into the "75+" band, its disease-specific utility is pulled down in proportion to the expected population-level decline, rather than being held artificially constant or subtracted down to an implausible floor.

8.6 Time trade-off, standard gamble and discrete choice experiments

The value sets and tariffs introduced in SS8.2 — the algorithms that convert an EQ-5D-5L or SF-6D health-state description into a single index utility — are not estimated from thin air. They are built from general-population preference elicitation exercises using one of a small number of classic methods, each asking respondents to make trade-offs that reveal how much they value one health state relative to another, or relative to death.

Time Trade-Off (TTO) is the method underlying the majority of existing EQ-5D value sets, including the original EQ-5D-3L sets and many EQ-5D-5L sets produced using the EuroQol Group's standard protocol. A respondent is asked to choose between living a fixed duration T years in an impaired health state, versus living a shorter duration $x < T$ years in full health followed by death. The duration x is varied (typically via an iterative "ping-pong" or titration procedure in a face-to-face interview) until the respondent is indifferent between the two options. At that indifference point, the utility of the impaired health state is simply $\frac{x}{T}$. TTO is relatively intuitive to administer — people readily grasp the idea of trading survival time for better health — but it is not strictly grounded in expected utility theory, since no uncertainty or gamble is involved; it is better thought of as a theoretically simpler, practically convenient elicitation task.

Standard Gamble (SG) is the method with the most direct theoretical grounding, rooted in von Neumann-Morgenstern expected utility theory. Here the respondent chooses between a certain

outcome (living in the impaired health state for the remainder of life) and a gamble with probability p of achieving full health and probability $1 - p$ of immediate death. The probability p is varied until the respondent is indifferent between the certain health state and the gamble; the utility of the health state is then read off directly as that indifference probability p. Because SG explicitly incorporates risk and is derived from axioms about rational choice under uncertainty, health economists have historically regarded it as the theoretically "purer" utility measure — though it is cognitively more demanding to administer, and respondents' answers can be sensitive to how probabilities are communicated, which is part of why TTO has been more widely adopted for large-scale value-set surveys.

Discrete Choice Experiments (DCE) take a different approach entirely. Rather than trading a single health state off against survival duration or a probability of death, respondents are shown repeated pairs (or sometimes larger sets) of hypothetical health-state profiles — each profile described by levels across the instrument's dimensions — and simply pick which of the two they prefer. No explicit trade-off against death or duration is built into any single choice task. Preference weights are instead recovered statistically from the pattern of choices across many respondents and many choice pairs, typically using a conditional logit or multinomial logit model that estimates how much each dimension level contributes to the probability of a profile being chosen. Because a DCE alone only yields utilities on an arbitrary scale anchored between the best and worst profiles shown, many modern value-set studies combine DCE with a smaller TTO or SG component (a "DCE with duration" or "DCE-TTO" hybrid design) specifically to anchor the resulting scale onto the conventional dead = 0, full health = 1 metric needed for QALY calculation. This hybrid DCE-based approach is the method underlying several of the newer EQ-5D-5L value sets, and it is a practical option increasingly considered where a country does not yet have its own long-established TTO-based value-set tradition to extend, since DCE fieldwork can be logistically lighter than face-to-face TTO interviewing at national scale.

> **Agency perspective.** When sourcing or reporting a country's EQ-5D value set methodology for an HTA submission, check the elicitation method and protocol version against the EuroQol Group's published EQ-5D-5L valuation protocol, and confirm the chosen value set matches the preference stated in the relevant reference-case guidance (for UK submissions, NICE's Health Technology Evaluations Manual sets out the expected utility value sourcing hierarchy) — do not assume a TTO-based and a DCE-based value set for the same instrument are numerically interchangeable.

In R, DCE choice data — typically structured as matched sets of chosen/not-chosen profiles per respondent per task — can be analysed with the `mlogit` package, which fits multinomial and conditional logit models to this kind of panel choice data, or with `survival::clogit()` for a conditional logistic regression formulation when each choice set is treated as a matched stratum. Either route estimates a coefficient for each attribute level, which, once anchored and rescaled, becomes the attribute-level preference weight feeding into the value-set algorithm.

A value-set methodology section conventionally summarises the three elicitation methods side by side, since a reviewer needs to see at a glance which theoretical grounding, response burden, and anchoring approach underlies the tariff being used:

Method	Elicited quantity	Theoretical basis	Anchoring to dead = 0
Time Trade-Off (TTO)	Indifference duration x in full health vs. T years impaired	Practical/intuitive, not expected-utility-theoretic	Direct: $\frac{x}{T}$
Standard Gamble (SG)	Indifference probability p of full health vs. death	von Neumann-Morgenstern expected utility theory	Direct: utility $= p$
Discrete Choice Experiment (DCE)	Choice between pairs of health-state profiles, no survival/death trade-off	Random utility theory (McFadden)	Indirect: requires a TTO/SG anchoring component

Table 22: Comparison of TTO, SG and DCE elicitation methods used in EQ-5D value-set construction

Difficult terminology: random utility theory (McFadden). Random utility theory is the statistical theory underlying discrete choice experiments (and much of modern choice modelling generally, including travel and marketing research — it's named after the economist Daniel McFadden, who won a Nobel Prize partly for this work). Its core idea: when a person chooses between two options (here, two hypothetical health-state profiles), each option has some "true," fixed underlying attractiveness to that person (its utility) — but on top of that, there's a random, unpredictable wobble in each specific choice, because people aren't perfectly consistent decision machines, moods vary, and attention lapses happen. No individual choice tells you much on its own, since you can't separate the "true" attractiveness from that day's wobble. But across hundreds of choices made by many respondents, the random wobbles average out, and statisticians can work backward from the **pattern** of choices to estimate how much each attribute (e.g. pain level, mobility limitation) contributes to overall attractiveness on average. This is exactly why a DCE needs many repeated choice tasks per respondent rather than a single question, unlike TTO or SG which can, in principle, extract a usable answer from one careful elicitation.

Utilities and costs are the two halves of the cost-per-QALY pair this book has worked with since Chapter 4's introduction of the WTP threshold; having now covered how the utility half is measured, valued and modelled, Chapter 9 turns to its counterpart — costs and resource use.

Chapter 9. Costs and Resource Use

Learning objectives

- Decompose costing into identification, measurement and valuation, and structure resource-use data in R so quantity and unit cost remain separately traceable.
- Apply inflation adjustment and currency conversion (nominal exchange rate vs. PPP) correctly and in the right order when combining unit costs across years and countries.
- Recognise the skewness, zero-inflation and heavy-tailed structure typical of patient-level cost data, and explain why it breaks ordinary least squares' assumptions.
- Fit and interpret gamma GLMs and two-part (hurdle-style) models for skewed, zero-inflated cost data, and avoid the retransformation-bias trap of naive log-OLS.
- Distinguish prevalence-based from incidence-based cost-of-illness studies and top-down from bottom-up (micro-costing) approaches, and combine them appropriately for LMIC programme costing.
- Explain why censored cost data require specialised estimators (Lin's partitioned estimator, inverse-probability-of-censoring weighting) rather than a naive mean of observed costs.

Chapter 8 closed by noting that utilities and costs are the two halves of the cost-per-QALY pair this book has worked with since Chapter 4′s introduction of the willingness-to-pay threshold. Having covered how the utility half is measured, valued and modelled, this chapter turns to its counterpart: how resource use is identified and measured, how unit costs are valued and adjusted across time, currency and country, and how the resulting cost data — skewed, zero-inflated, sometimes censored — is analysed defensibly rather than with a naive mean or an inappropriate linear model.

9.1 Identifying, measuring and valuing resources

Costing in economic evaluation is conventionally decomposed into three steps that map neatly onto R workflow stages: **identification** (which resource items are relevant to the perspective and the decision problem), **measurement** (quantities of each item consumed per patient, per cycle, or per health state — the same cycle structure you built in Chapter 4′s Markov trace), and **valuation** (attaching a unit cost to each quantity). Keeping these three steps as distinct columns in your working data, rather than collapsing straight to a single "cost" figure, is what lets you later re-value the same resource-use data under a different perspective or a different country's unit costs without re-extracting anything.

For a TB programme, identification usually spans at least four resource categories: diagnostic (sputum smear, Xpert MTB/RIF, chest X-ray, culture/DST), drug (first-line FDCs, second-line regimens for drug-resistant TB), programmatic/health-system (DOT visits, counselling, contact tracing), and patient-borne (travel, time, lost income) — the latter mattering if your analysis takes a societal perspective, which is common in LMIC HTA submissions even when the reference case is health-system focused. Decide and document the perspective **before** you start identifying items, because it determines which rows belong in your resource-use table at all.

A disciplined way to carry this through an R pipeline is a long-format resource-use tibble, with quantity and unit cost kept as separate columns:

```
library(tibble)
library(dplyr)

resource_use <- tribble(
  ~patient_id, ~health_state, ~item,              ~category,    ~quantity, ~unit,
  "TB001",     "on_treatment","xpert_test",       "diagnostic", 1,         "test",
  "TB001",     "on_treatment","fdc_regimen",       "drug",       6,
"month",
  "TB001",     "on_treatment","dot_visit",         "programmatic",24,
"visit",
  "TB002",     "retreatment", "culture_dst",       "diagnostic", 1,
"test",
  "TB002",     "retreatment", "second_line_regimen","drug",      8,
"month"
)

resource_use
```

Quantity measurement itself typically comes from one of three sources you already know how to ingest from Chapter 5: trial-level case report forms, programmatic/surveillance data (e.g. national TB programme line-lists), or expert elicitation when no empirical source exists for a resource that only occurs off the beaten path of routine data (e.g. ambulance transfer for a rare adverse event). Whichever source, keep an explicit `source` column alongside each `quantity` — reviewers and HTA bodies will ask where a number came from, and a provenance column answers that without you re-digging through notebooks later.

Valuation is where unit costs are joined onto quantities. Treat the unit-cost lookup as its own small reference table and join rather than hard-coding values inline — this is the same discipline as keeping utility decrements as a separate lookup in Chapter 8, and it pays off identically when a unit cost is revised:

```
unit_costs <- tribble(
  ~item,               ~unit_cost_local, ~currency, ~cost_year,
  "xpert_test",         800,             "INR",     2023,
  "fdc_regimen",        450,             "INR",     2023,
  "dot_visit",          150,             "INR",     2023,
  "culture_dst",        1200,            "INR",     2023,
  "second_line_regimen", 3200,           "INR",     2023
)

costed <- resource_use |>
  left_join(unit_costs, by = "item") |>
  mutate(total_cost_local = quantity * unit_cost_local)

costed
```

Category	Resource item	Unit cost	Currency	Price year
Diagnostic	Xpert MTB/RIF test	800	INR	2023
Diagnostic	Culture / DST	1,200	INR	2023
Drug	First-line FDC regimen (per month)	450	INR	2023
Drug	Second-line regimen (per month)	3,200	INR	2023
Programmatic	DOT visit	150	INR	2023

Source: programme costing exercise unit-cost lookup; values are illustrative and should be replaced with local or WHO-CHOICE sourced costs before reporting.

Table 23: Unit-cost reference table for a TB programme resource-use valuation, by resource category and item (local currency, price year 2023) – the conventional format for publishing the unit-cost list underlying a costing or cost-effectiveness analysis

HEOR in practice. Never collapse quantity and unit cost into a single pre-multiplied "total cost" column at the extraction stage. Keeping them separate (as above) is what allows deterministic sensitivity analysis on the unit cost alone, probabilistic sensitivity analysis with a distribution on quantity and cost independently (Chapter 25), and re-valuation under a different country's cost list — none of which are possible once the multiplication has already happened and the inputs are gone.

Common pitfall. A common analyst error is averaging unit costs **before** joining to resource-use quantities when costs come from multiple facility types (e.g. district hospital vs. primary health centre). Average the resulting total costs (or weight by caseload), not the raw unit costs, unless the facility mix is uniform across your resource-use denominator — averaging unit costs first silently assumes equal patient volume at each facility.

9.2 Unit costs and inflation or currency conversion (PPP adjustments)

Unit costs are rarely available in the right currency, the right year, and the right country all at once. Three distinct adjustments are typically needed, and they must be applied in the correct order: inflate to a common price year **within** the source currency first, then convert currency (nominal exchange rate or PPP) second. Reversing the order mixes an inflation index computed in one currency with a conversion factor computed in another and produces a number that looks plausible but is not interpretable.

Inflation to a common price year. Within a single currency, uprate historical unit costs using a price index — a consumer price index (CPI) or, for health-specific costs, a health-sector deflator where available. The arithmetic is simple but easy to get backwards:

```
inflate_cost <- function(cost, cost_year, target_year, cpi) {
  # cpi: named vector or lookup keyed by year, e.g. cpi["2023"], cpi["2024"]
  cost * (cpi[[as.character(target_year)]] / cpi[[as.character(cost_year)]])
}

cpi_india <- c("2019" = 100, "2020" = 106, "2021" = 112,
               "2022" = 121, "2023" = 128, "2024" = 134)

# Example: inflate a 2019 unit cost to 2024 prices
inflate_cost(cost = 500, cost_year = 2019, target_year = 2024, cpi = cpi_india)
```

Currency conversion: nominal vs. PPP. Once costs are in a common price year within their source currency, converting to a reference currency (commonly USD or international dollars for cross-country comparison) requires a choice between the market/nominal exchange rate and a purchasing power parity (PPP) conversion factor. Nominal exchange rates reflect what currency actually trades for; PPP conversion factors reflect relative purchasing power for a comparable basket of goods and services, and are the standard choice when the purpose is comparing real resource burden across countries of different income levels — exactly the comparison an Indian TB-programme analyst benchmarking against WHO or global literature unit costs will usually want. The World Bank publishes PPP conversion factors (and the WHO-CHOICE unit cost database separately provides country-level unit costs for LMICs, already expressed per the WHO-CHOICE methodology) — treat the PPP factor as another lookup joined by year and country, not a constant you hardcode once and forget to update:

```
ppp_factor_inr_per_intl_dollar <- c(
  "2022" = 22.9, "2023" = 23.5, "2024" = 24.1  # illustrative; source from World
Bank ICP data
)

to_intl_dollars <- function(cost_local, year, ppp_lookup) {
  cost_local / ppp_lookup[[as.character(year)]]
}

# Convert the inflated 2024 INR cost to international dollars for cross-country
comparison
cost_2024_inr <- inflate_cost(500, 2019, 2024, cpi_india)
to_intl_dollars(cost_2024_inr, 2024, ppp_factor_inr_per_intl_dollar)
```

Wrapping both steps into one pipeline keeps the costed resource-use table from Section 9.1 fully traceable from source currency/year through to the reporting currency/year:

```
costed_reported <- costed |>
  mutate(
    total_cost_2024_inr = inflate_cost(total_cost_local, cost_year, 2024,
cpi_india),
    total_cost_2024_intl_usd = to_intl_dollars(total_cost_2024_inr, 2024,
                                               ppp_factor_inr_per_intl_dollar)
  )

costed_reported
```

> **Agency perspective.** The NICE reference case, as set out in "NICE health technology evaluations: the manual" (PMG36) — the same document already cited for EQ-5D utility sourcing in Chapter 8 — specifies costs from the NHS and Personal Social Services (PSS) perspective, valued in UK currency at current prices; a NICE-facing submission should never silently substitute a PPP-converted LMIC cost for a missing UK unit cost without flagging it as a structural assumption. Conversely, for HTAIn or WHO-CHOICE-oriented work, PPP conversion is the standard device for situating a local unit cost within international cost-effectiveness thresholds, and PSSRU's **Unit Costs of Health and Social Care** remains the reference source analysts cite when a UK unit cost itself (rather than its currency conversion) is in question.

Report which conversion was used — nominal exchange rate or PPP — and the source and year of both the price index and the conversion factor, directly alongside any cost table in your results output; CHEERS 2022 (Husereau et al., **Value in Health**, 2022) item 15, which requires explicit reporting of currency, price date, and the conversion method applied, specifically requires this disclosure, and it is also what lets another analyst re-run your pipeline with an updated PPP vintage without reverse-engineering your assumptions.

9.3 Cost data features: skewness, zero-inflation, heavy tails

Cost data in a TB programme context — drug costs, hospitalisation costs, outpatient visit costs, diagnostic costs (GeneXpert cartridges, chest X-rays, culture/DST) — share a statistical signature that rhymes with the bounded utility data of Section 8.4 but is mechanistically different. Utilities were bounded on the [0,1] interval with a ceiling spike at 1; costs are bounded only below at 0, have no natural upper ceiling, and the "spike" sits at the **floor**, not a ceiling. Where Section 8.4 needed beta or censored regression to respect a bounded, two-sided support, cost data needs methods that respect a one-sided (non-negative), unbounded-above, heavy-tailed support with an honest mass of exact zeros. Same family problem — "the arithmetic mean and OLS assumptions quietly break down" — different geometry.

Three features recur in almost every patient-level cost dataset you will pull from a TB programme's billing or case-management system:

- **Right-skewness.** Most patients incur modest cost (a few clinic visits, first-line drugs); a minority incur very large cost (MDR-TB admission, surgical intervention, prolonged inpatient stay). The mean is pulled well above the median — often costs are log-normal- or gamma-shaped rather than Gaussian.
- **Zero-inflation.** Some patients genuinely incur zero cost for a resource category over the observation window — no hospitalisation, no missed-dose home visits, no adverse-event management. These are **structural** zeros (the event never happened), not missing data in the Chapter 7 sense. Treating them as missing and imputing a positive value, or dropping them, both bias the estimate; treating them as "just another low value" and fitting a single continuous-only model (e.g. plain log-OLS) is equally wrong because log(0) is undefined and the zero/positive process may have genuinely different covariates than the cost-given-event process.
- **Heavy tails.** Even after conditioning on "cost > 0", the right tail can be heavier than gamma or log-normal would suggest — a handful of catastrophic MDR/XDR-TB cases with ICU stays, surgery, or prolonged second-line regimens dominate the total programme cost even though they are a tiny fraction of the caseload. Total cost estimates are consequently dominated by

a handful of outlier patients, and ordinary standard errors (which assume roughly symmetric, light-tailed residuals) understate uncertainty badly.

Common pitfall. A right-skewed cost variable with a thick right tail is the textbook case where a handful of leverage points can swing a sample mean (and therefore an incremental cost-effectiveness ratio numerator) far more than any outlier in a beta-distributed utility ever could, because the cost support is unbounded above. Never eyeball a cost histogram on the raw scale and move on — always report the proportion of zeros, skewness, and at minimum a trimmed/bootstrapped mean alongside the arithmetic mean before choosing a model.

Difficult terminology: leverage points. A leverage point is a data value that sits far enough out on its own that it pulls a summary statistic — most commonly a mean — disproportionately toward itself, simply because of how extreme or isolated it is relative to the rest of the sample. Picture one patient in your cost dataset who needed a prolonged ICU admission for drug-resistant TB, costing ten times more than anyone else in the sample: even though that patient is a single row among hundreds, their cost alone can drag the sample mean up substantially, because the mean, unlike the median, is sensitive to the exact magnitude of every value, not just its rank. This matters acutely for cost data specifically because, unlike a utility score (bounded between roughly 0 and 1), cost has no upper bound — a single catastrophic case can in principle be arbitrarily large, so there's no ceiling limiting how much damage one leverage point can do to your reported average cost, and therefore to an ICER built on it.

Difficult terminology: bootstrapped estimate. Bootstrapping is a way of figuring out how uncertain a statistic (like a mean) really is, without relying on a formula that assumes the data are normally distributed — which, as this section has just shown, cost data very much are not. The trick: take your own sample of patients, and create a new "pretend" sample of the same size by randomly drawing patients from it **with replacement** (so the same patient can be picked more than once, and some patients may not be picked at all). Calculate the mean of that pretend sample. Repeat this process thousands of times, and you end up with thousands of slightly different means — the spread of that collection of means tells you directly how much uncertainty surrounds your original estimate, without ever having to assume the underlying cost distribution is some neat bell curve. It's conceptually like repeatedly reshuffling and redrawing your own deck of patient records to see how much your answer would wobble if you happened to have drawn a slightly different sample from the same underlying population — a "bootstrapped mean" or "bootstrapped confidence interval" is simply the result of that resampling exercise applied to your actual data.

A quick diagnostic pass on simulated TB-programme-like cost data:

```
library(tidyverse)

set.seed(9340)
n <- 1200

# Simulate a TB cohort: ~30% zero-cost (no qualifying resource use),
# positive costs right-skewed (gamma), with a few heavy-tail outliers.
```

```
zero_flag  <- rbinom(n, 1, prob = 0.30)
base_cost  <- rgamma(n, shape = 1.4, scale = 180)       # typical positive costs
outlier    <- rbinom(n, 1, prob = 0.02) * rgamma(n, shape = 2, scale = 4000)  #
rare catastrophic cases
cost <- ifelse(zero_flag == 1, 0, base_cost + outlier)

tb_costs <- tibble(
  patient_id = seq_len(n),
  regimen    = sample(c("first_line", "mdr_regimen"), n, replace = TRUE, prob =
c(0.85, 0.15)),
  age        = round(rnorm(n, 38, 13)),
  cost       = cost
)

tb_costs |>
  summarise(
    n_patients   = n(),
    pct_zero     = mean(cost == 0) * 100,
    mean_cost    = mean(cost),
    median_cost  = median(cost),
    sd_cost      = sd(cost),
    skewness     = mean((cost - mean(cost))^3) / sd(cost)^3,
    p95          = quantile(cost, 0.95),
    max_cost     = max(cost)
  )
```

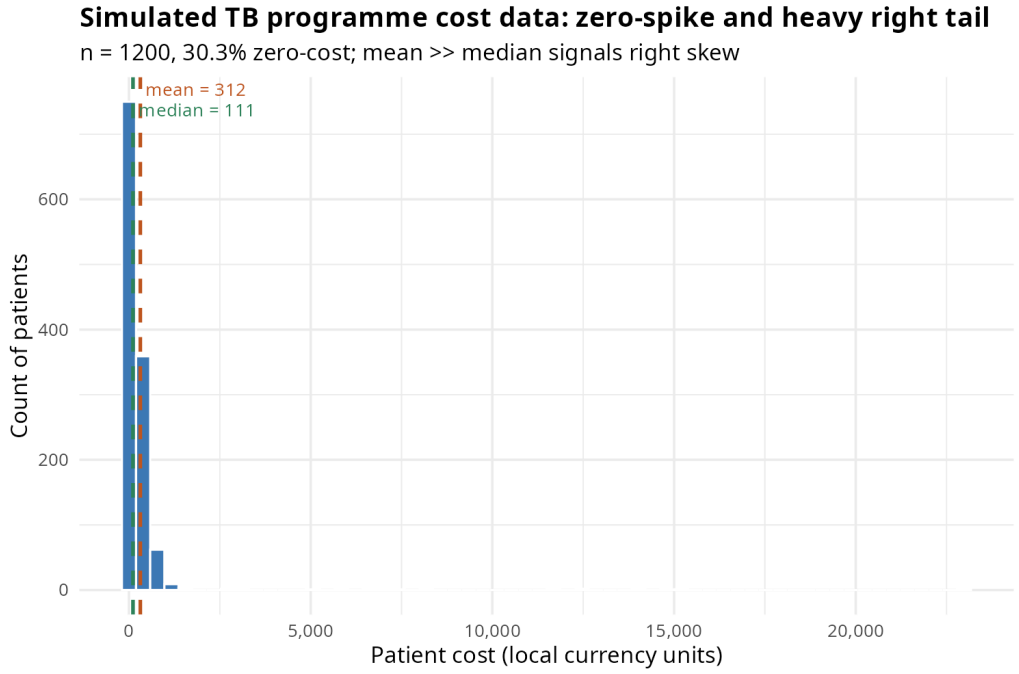


Figure 8: Simulated TB programme cost data (n = 1,200): a visible spike of zero-cost patients at the floor, a heavy right tail driven by the rare catastrophic-cost outliers, and a mean well above the median — the distributional signature that makes ordinary least squares on raw cost a poor default.

On data like this it is routine to see the mean sit two- to four-fold above the median, skewness well above 2, and the top 5% of patients accounting for a disproportionate share of total programme

spend — exactly the pattern that makes "fit OLS on raw cost" a poor default, just as "fit OLS on a 0-1 utility with a spike at 1" was a poor default in Section 8.4.

9.4 Regression models for costs: GLMs, two-part models, gamma models

The methodological toolkit for costs parallels — but does not duplicate — the beta/censored-regression toolkit used for bounded utilities. The organising choice is the same kind of choice: **pick a model whose assumed distribution matches the data's actual support and shape, rather than forcing Gaussian machinery onto data that was never Gaussian.**

Generalised linear models on the cost scale

The single most defensible default for **strictly positive** cost data (no zeros, or zeros handled separately — see two-part models below) is a gamma GLM with a log link, fit directly on the natural cost scale:

```
# Restrict to patients with cost > 0 for the gamma GLM
pos_costs <- tb_costs |> filter(cost > 0)

fit_gamma <- glm(
  cost ~ regimen + age,
  family = Gamma(link = "log"),
  data   = pos_costs
)

summary(fit_gamma)

# Contrast against naive OLS on the same (untransformed) scale
fit_ols <- lm(cost ~ regimen + age, data = pos_costs)

# Compare predicted mean cost for the MDR regimen at mean age
newdat <- tibble(regimen = c("first_line", "mdr_regimen"), age =
mean(pos_costs$age))
predict(fit_gamma, newdata = newdat, type = "response")
predict(fit_ols,   newdata = newdat)
```

The gamma GLM models the **mean** of cost directly (via the log link on the linear predictor, exponentiated back by `type = "response"`), so there is no retransformation step and no smearing correction needed — this is one of its practical advantages over a log-OLS model. Its variance function (variance proportional to the squared mean) also matches right-skewed cost data much better than OLS's constant-variance assumption. An alternative sometimes seen in the literature is `family = gaussian(link = "log")` — a Gaussian GLM with a log link, which also models the mean on the log scale without the smearing problem, but keeps OLS's constant-variance assumption and so under-weights the heavy right tail relative to gamma or inverse-gaussian error structures. Comparing AIC, or inspecting residual-vs-fitted plots on the response scale, across `Gamma(link="log")`, `gaussian(link="log")`, and `inverse.gaussian(link="log")` is a reasonable way to pick among link/variance combinations for a given programme's cost data.

Common pitfall. Never fit `lm(log(cost) ~ ...)` and then report the exponentiated coefficient or the exponentiated predicted log-cost as the mean cost. Exponentiating a prediction from a log-OLS model recovers the median (geometric mean) of the fitted distribution on the original scale, not the arithmetic mean — and it is almost always the arithmetic mean that the economic evaluation needs, because total programme cost and incremental cost-effectiveness ratios are built from **sums**, not medians. If you must work on the log scale, apply **Duan's smearing estimator**: fit the log-OLS model, compute the smearing factor as the mean of the exponentiated residuals, and multiply every back-transformed prediction by that constant before using it in any budget-impact or CEA calculation. This is exactly the same family of retransformation-bias trap as reporting a raw mean on a ceiling-spiked utility scale without acknowledging the spike (Section 8.4) — the fix differs, but the discipline of "respect what the link/transformation actually estimates" is identical.

How to report this.

Parameter	Coefficient (log scale)	Cost ratio	95% CI	p-value
Intercept (first-line, age = 0)	5.92	372	310 – 447	< 0.001
Regimen: MDR vs. first-line	1.35	3.86	3.02 – 4.93	< 0.001
Age (per year)	0.012	1.01	1.00 – 1.02	0.014
Model: `glm(cost ~ regimen + age, family = Gamma(link = "log"))`; cost ratio = exp(coefficient), interpreted as a multiplicative effect on mean cost.				

Table 24: Gamma GLM (log link) results for per-patient TB cost by regimen and age, among patients with cost > 0 – the conventional format for reporting cost-ratio regression results (coefficient, exponentiated cost ratio, 95% CI, p-value) rather than raw log-scale coefficients alone

Two-part (hurdle-style) models for zero-inflated costs

When a non-trivial share of patients have genuinely zero cost for the resource category of interest, a single continuous-distribution model (gamma, log-normal, or otherwise) cannot represent the probability mass at exactly zero. The standard solution is a **two-part model**: part one is a binary model for "any cost incurred at all", part two is a continuous model for cost **conditional on** being positive. The two parts can use different covariates and different functional forms, and together they decompose expected cost cleanly as

$$E(\text{cost} \mid \text{x}) = P(\text{cost} > 0 \mid \text{x}) \times E(\text{cost} \mid \text{cost} > 0, \text{x})$$

A manual implementation is transparent and easy to audit:

```
tb_costs <- tb_costs |> mutate(any_cost = as.integer(cost > 0))

# Part 1: probability of incurring any cost at all
part1 <- glm(any_cost ~ regimen + age, family = binomial(link = "logit"), data =
tb_costs)
```

```
# Part 2: conditional mean cost, among those with cost > 0
part2 <- glm(cost ~ regimen + age, family = Gamma(link = "log"),
             data = tb_costs |> filter(cost > 0))

# Combine: predicted mean cost = P(cost>0) * E[cost | cost>0]
newdat <- newdat |> mutate(
  p_positive   = predict(part1, newdata = newdat, type = "response"),
  cond_mean    = predict(part2, newdata = newdat, type = "response"),
  pred_mean    = p_positive * cond_mean
)
newdat
```

The `pscl` package wraps similar two-part/mixture logic via `pscl::hurdle()`, which was built primarily for zero-inflated **count** data (e.g. number of outpatient visits) rather than continuous cost — it is a natural fit for resource-use **counts** that you then cost out, more than for cost amounts directly. For continuous zero-inflated costs specifically, the manual two-part construction above (or the dedicated `twopartm` package, whose `tpm()` function automates exactly this binomial-then-gamma/log-normal decomposition with built-in marginal-effect and bootstrap-SE support — Belotti, Deb, Manning & Norton 2015) is generally the more appropriate and more transparent tool; check `?twopartm::tpm` for the current argument names before relying on a specific call signature in a submission.

> **Common pitfall.** Fitting a single gamma or log-normal GLM on cost data that actually contains a meaningful fraction of zeros is a common error: zeros either get dropped (silently shrinking and biasing the analysis sample and discarding real information about who incurs no cost at all) or the gamma/log-normal density is evaluated at zero and either errors out or is coerced via an ad hoc offset (e.g. adding 1 to all costs before logging), which distorts the entire distribution's shape and is difficult to defend to a reviewer. Check `mean(cost == 0)` before choosing between a single-part GLM and a two-part model — this is the cost-data analogue of checking the proportion of ceiling observations before choosing between OLS and a beta/Tobit model for utilities in Section 8.4.

Smooths and flexible mean functions

When the relationship between a continuous covariate (age, time since treatment initiation, baseline bacterial load) and cost is not well captured by a linear or log-linear term, `mgcv::gam()` extends the gamma-GLM framework with penalised smooths:

```
library(mgcv)

fit_gam <- gam(
  cost ~ regimen + s(age),
  family = Gamma(link = "log"),
  data   = pos_costs
)
summary(fit_gam)
plot(fit_gam, pages = 1)
```

This keeps the same mean-and-variance structure as the gamma GLM above while relaxing the linearity assumption on `age` — useful when, for example, cost rises non-linearly with age due to

comorbidity burden in older TB patients, and a straight line understates cost at the extremes of the age range.

> **Agency perspective.** Mihaylova et al. (2011), "Review of statistical methods for analysing healthcare resources and costs" (**Health Economics**), is the standard reference the field cites for exactly this decision tree: it recommends against transforming cost data purely to force normality, favours GLMs that model the mean directly on the natural scale (gamma, or other members of the exponential family matched to the data's shape), and discusses two-part models explicitly as the appropriate tool once a meaningful zero-cost subgroup is present. The ISPOR/professional costing literature building on this guidance, together with CHEERS 2022′s reporting checklist (which requires that the chosen cost-analysis method and its justification be stated explicitly, not merely that "costs were analysed in R"), is the standard a submission reviewer will hold your cost model to — document the skewness/zero-inflation diagnostics from Section 9.3 and the resulting model choice together, not the model choice alone.

9.5 Cost-of-illness and micro-costing studies

Everything in Sections 9.1-9.4 assumed you already had a cost per patient per resource item sitting in a tidy data frame. Cost-of-illness (COI) and micro-costing studies are where those numbers actually get built, and for a TB-programme analyst this is often the first deliverable a ministry of health or donor asks for, independent of any trial-based CEA: "what does a TB case cost the health system and the patient, start to finish?"

COI studies come in two flavours, and conflating them is a common critique reviewers raise.

- **Prevalence-based** COI sums all costs incurred in a defined period (e.g. a calendar year) across everyone with the condition, regardless of when they were diagnosed. This answers "what did TB cost the system last year?" and is the natural frame for annual budget-impact reporting.
- **Incidence-based** COI follows a cohort of newly diagnosed patients and sums costs over the full episode (diagnosis through cure, default, or death). This answers "what does one TB episode cost?" and is what feeds a per-patient cost input into a Markov or partitioned-survival CEA model built on the Chapter 4 tibble structure.

The second axis is **costing direction**, independent of the first:

- **Top-down** costing allocates aggregate facility or programme expenditure (budgets, cost centres) down to patients or services using an allocation key — bed-days, visit counts, staff-time proportions. It is fast and uses routinely available financial data, but hides within-patient variation and can smear fixed costs unrealistically across light and heavy resource users.
- **Bottom-up (micro-costing)** traces individual resource items consumed by each patient — every sputum smear, every drug-dispensing visit, every hospital night — and prices each one, typically against the national/WHO-CHOICE unit-cost list built in Section 9.2. It is labour-intensive but gives the variance structure you need for probabilistic sensitivity analysis (Chapter 25) and is the reference standard when heterogeneity across patients (e.g. drug-susceptible vs MDR-TB) matters for the decision.

HEOR in practice. In practice, most programmatic TB costing is a **hybrid**: drug and diagnostic-test costs are micro-costed (bottom-up, against a WHO-CHOICE or national unit-cost list), while overheads — facility utilities, programme management, supervision — are allocated top-down from the TB programme budget using a step-down allocation. Keep the two cost components in separate columns (`cost_microcosted`, `cost_overhead_allocated`) in your tibble rather than pre-summing them; a reviewer or later re-analysis will want to vary the overhead allocation assumption without re-deriving the whole unit-cost exercise.

Difficult terminology: step-down allocation. Step-down allocation is a standard hospital/programme cost-accounting technique for spreading a shared overhead cost (say, facility electricity, building maintenance, or central programme management) across the individual services or departments that benefit from it, when there's no direct per-patient bill for that overhead to point to. It works in a deliberate sequence, hence "step-down": you start with the department that provides services to the most other departments but receives services from none in return (e.g. general administration), and allocate its cost out to everyone else based on a sensible driver (floor space, staff headcount, visit counts). Then you move to the next department in the sequence and allocate **its** cost (now including the share it just received) onward to the remaining departments — and so on, one step at a time, until every shared cost has been pushed down to the clinical service lines that actually treat patients. It's the accounting equivalent of a multi-stage reimbursement cascade: each step passes a cost share further down the chain rather than trying to divide every overhead cost among every department in one single calculation.

Agency perspective. Tan-Torres Edejer et al. (2003), **Making Choices in Health: WHO Guide to Cost-Effectiveness Analysis** — the foundational WHO-CHOICE methodological reference — explicitly supports the hybrid pattern above for LMIC provider-cost studies: capital and overhead items valued top-down via facility-level allocation, variable/consumable items (drugs, tests, staff time per visit) valued bottom-up via micro-costing against the national unit-cost list, with WHO-CHOICE's published regional unit-cost database as the standard fallback when a national list is incomplete. When you deviate from this (e.g. top-down only, for a rapid budget-impact ask), document that choice explicitly — it is a direct driver of how much cost variance the model can later show, and ISPOR's Good Research Practices for cost-of-illness and micro-costing studies expects that deviation to be stated, not left implicit.

A minimal incidence-based micro-costing roll-up, continuing the per-patient resource-item structure from Section 9.1-9.2:

```
library(dplyr)

# resource_items: one row per patient per resource event
# patient_id, episode_phase ("diagnosis","treatment","hospitalisation"),
# resource_item, quantity, unit_cost (from the Section 9.2 unit-cost table)

episode_cost <- resource_items |>
  mutate(item_cost = quantity * unit_cost) |>
  group_by(patient_id, episode_phase) |>
```

```
  summarise(phase_cost = sum(item_cost), .groups = "drop") |>
  tidyr::pivot_wider(
    names_from  = episode_phase,
    values_from = phase_cost,
    values_fill = 0
  ) |>
  mutate(total_episode_cost = diagnosis + treatment + hospitalisation)

episode_cost |>
  summarise(
    mean_cost   = mean(total_episode_cost),
    sd_cost     = sd(total_episode_cost),
    n_patients  = n()
  )
```

This gives a defensible mean cost per TB episode **provided every patient in `resource_items` completed follow-up**. In a real programme cohort or trial, many will not — and that is exactly the problem Section 9.6 addresses.

9.6 Handling censored cost data

The gamma GLM and two-part models of Section 9.4 assumed every patient had a complete, fully observed total cost. In longitudinal cost studies — a TB treatment trial following patients for 6-24 months, or a programmatic cohort tracked to cure/default/death — that assumption routinely fails: patients are lost to follow-up, transfer out, or the study closes before their course of treatment (and its costs) is finished. Their total cost is only **partially observed**, and critically, the amount observed is correlated with how long they survived or stayed in care. This is the same right-censoring you will formalise in Chapter 10, but applied to a **cumulative, time-accruing** outcome rather than to a single event time, which makes it behave differently from a textbook Kaplan-Meier problem.

> **HEOR in practice.** Before reaching for any estimator, check why patients are censored. If censoring is administrative (study end date, same calendar cutoff for everyone, unrelated to prognosis) the methods below are well justified. If patients drop out **because** they are doing badly (or unusually well) and their costs would have been systematically different, you have informative censoring that no reweighting scheme fully fixes — flag it as a limitation rather than silently correcting it away.

Why naive averaging is biased. The obvious shortcut — take the mean of whatever total cost was observed for each patient, ignoring who was censored — is biased, and predictably so. Costs accrue over time, so a patient censored early has had less opportunity to accumulate cost than one followed to completion. Dropping censored patients entirely (complete-case analysis) is worse: it systematically discards exactly the patients with the shortest observed survival/follow-up, biasing the mean cost **upward** among survivors and giving a distorted picture of total programme cost. Simply summing observed costs without any time adjustment conflates "this patient was cheap" with "this patient was only followed for three weeks."

The Lin et al. (1997) partitioned estimator. Lin, Feuer, Etzioni and Wax (**Biometrics**, 1997) solved this by partitioning the total follow-up period into fixed intervals and estimating mean cost accrued in **each interval**, weighting each interval's mean by the probability of still being under

observation (uncensored) at that interval, estimated via the Kaplan-Meier survival function of the censoring time. Summing the interval contributions gives a consistent estimator of mean total cost to a fixed time horizon, using only costs actually observed plus the KM-estimated probability of being followed long enough to have observed them — no extrapolation of unobserved costs is required.

Inverse-probability-of-censoring weighting (IPCW). A closely related and more flexible approach reweights each fully-observed (or observed-to-time-t) patient's cost by the inverse of their probability of remaining uncensored up to their observed time, using the Kaplan-Meier estimate of the **censoring** distribution (i.e. KM fitted with event/censoring indicators flipped). Patients who stayed in observation longer than typical are up-weighted to stand in for the similar patients who were censored early. `survival::survfit()` already gives you everything needed for the KM step; the IPCW cost estimator is then a straightforward weighted mean, no specialised cost-econometrics package required.

```
library(survival)
library(dplyr)

# cohort: one row per patient
#   time_obs   - time followed (to cost cut-off, death, loss-to-follow-up)
#   cost_obs   - total cost accrued up to time_obs
#   event      - 1 = fully observed / died (not censored), 0 = censored

# Step 1: KM estimate of the CENSORING distribution -- flip the event indicator
cens_fit <- survfit(Surv(time_obs, 1 - event) ~ 1, data = cohort)

# Step 2: censoring survival probability just before each patient's own time
G_at <- summary(cens_fit, times = cohort[["time_obs"]], extend = TRUE)[["surv"]]

cohort <- cohort |>
  mutate(
    G_hat   = pmax(G_at, 0.01),         # guard against near-zero denominators
    ipcw_wt = event / G_hat             # zero weight for censored patients
  )

ipcw_mean_cost <- with(cohort, sum(ipcw_wt * cost_obs) / sum(ipcw_wt))
ipcw_mean_cost
```

Only patients with a fully observed episode (`event == 1`) contribute directly, but each is reweighted by the inverse probability that someone with their observed follow-up time would still be uncensored — correcting for the fact that easily-followed (often lower-cost, faster-resolving) patients are over-represented among the uncensored.

> **Agency perspective.** CHEERS 2022 (Husereau et al., **Value in Health**, 2022) item 24 requires reporting of the time horizon and how costs and outcomes were adjusted, and its elaboration explicitly flags that studies with differential follow-up or loss-to-follow-up must describe how incomplete cost data were handled — a bare "missing data were excluded" does not satisfy the checklist. When you use Lin's partitioned estimator or IPCW, name the method and cite it in the costing section of your write-up; reviewers increasingly check for this specifically.

Method	Bias under informative follow-up	Requires	Principal limitation
Naive mean of observed cost	Biased downward: shorter-followed (often sicker/costlier) patients contribute less accrued cost	Complete or incomplete cost field, no censoring indicator used	Ignores that observed cost is mechanically capped by observed time, not just by resource use
Complete-case (drop censored)	Biased upward: discards exactly the shortest-followed patients, over-representing those followed to completion	Censoring indicator to identify which rows to drop	Discards real information and shrinks the effective sample size
Lin et al. (1997) partitioned estimator	Consistent under non-informative (administrative) censoring	Fixed time intervals; KM estimate of the censoring distribution per interval	Interval width choice affects precision; still assumes non-informative censoring
Inverse-probability-of-censoring weighting (IPCW)	Consistent under non-informative censoring; more flexible weighting than Lin's fixed intervals	KM fit on the flipped (censoring) event indicator; stable weights away from zero	Weights become unstable when few patients remain uncensored at longer times

Recommendation: report which estimator was used and why, per CHEERS 2022 item 24; neither estimator corrects for informative censoring, which must instead be flagged as a limitation (§9.6).

Table 25: Comparison of censored-cost-handling methods (naive mean of observed cost, Lin et al.'s partitioned Kaplan-Meier-weighted estimator, inverse-probability-of-censoring weighting) by bias direction, data requirement and principal limitation – the summary table a costing section should present when justifying the chosen estimator for incomplete follow-up cost data

Both estimators depend on the same underlying machine: a Kaplan-Meier estimate of a time-to-event distribution, fitted with `survival::survfit()`. That machinery, and the broader question of how censoring and truncation affect survival-time estimation and extrapolation, is the full subject of Chapter 10, Survival Analysis for Economic Evaluation — where Kaplan-Meier curves, parametric survival models, and their role in extrapolating costs and outcomes beyond trial follow-up will be developed in depth, building directly on the censoring intuition established here.

PART III

Statistical Methods for Evidence Synthesis and Analysis

Chapter 10. Survival Analysis for Economic Evaluation

Learning objectives

- Fit and plot Kaplan-Meier estimates and life tables using `survival`, `ggsurvfit` and `survminer` (§10.1)
- Fit the six standard parametric survival distributions with `flexsurv::flexsurvreg()` (§10.2)
- Fit flexible, spline-based Royston-Parmar models with `flexsurv::flexsurvspline()` (§10.3)
- Fit mixture and non-mixture cure models with `flexsurvcure::flexsurvcure()` (§10.4)
- Build piecewise and landmark extrapolations that combine empirical and parametric survival (§10.5)
- Select an extrapolation model using AIC/BIC, visual fit, and clinical plausibility together (§10.6)
- Test the proportional hazards assumption with `survival::cox.zph()` and choose a non-PH alternative where needed (§10.7)
- Build covariate-adjusted and treatment-effect-specific survival models (§10.8)
- Extrapolate survival using external data, background mortality, and expert elicitation (§10.9)
- Model competing risks and multi-state pathways with `mstate` and `flexsurv` (§10.10)
- Reconstruct individual patient data from a published Kaplan-Meier curve with `IPDfromKM` (§10.11)
- Report survival extrapolation in line with NICE DSU TSD 14 and TSD 21 (§10.12)

Chapter 9 closed by introducing `survival::survfit()` and `Surv()` for one narrow purpose: building inverse-probability-of-censoring weights for censored cost data. This chapter comes back to that same machinery and builds it out properly. Here is the roadmap: start with the model-free Kaplan-Meier curve (§10.1), move to fitting a parametric curve so you can extrapolate beyond the trial's follow-up (§10.2-§10.6), check and handle cases where the simple "constant hazard ratio" assumption breaks down (§10.7-§10.9), extend to pathways with more than one possible outcome (§10.10-§10.11), and finish with exactly what a submission needs to report to satisfy NICE's own written guidance on this topic (§10.12). Each section builds on the last, so read them in order the first time through.

> **Difficult terminology: hazard function / hazard rate.** The hazard at any given moment is the **instantaneous** risk of the event of interest (death, relapse, disease progression) happening right now, given that the patient has made it this far without it happening yet. This is the single most important idea in this entire chapter — nearly every model, curve, and diagnostic from here to §10.12 is built directly on top of it — so it's worth pinning down precisely, because it answers a genuinely different question from the one a simple survival probability answers. A survival probability (what the Kaplan-Meier curve plots) answers "what fraction of patients starting out are still alive/event-free by month 24?" — a cumulative, backward-

looking summary over the whole period so far. The hazard, by contrast, answers "of the patients who have survived to this exact moment, what is their risk of the event happening in the next instant?" — a forward-looking, momentary rate, re-calculated continuously as time moves on. The standard analogy is a car's speedometer versus its odometer: the odometer (like cumulative survival) tells you the total distance travelled so far; the speedometer (like the hazard) tells you how fast you are going **right now**, which can rise, fall, or stay flat completely independently of how far you've already travelled. A hazard that rises over time means risk is accelerating for those still at risk (e.g. a disease that progressively weakens patients the longer they have it); a falling hazard means the remaining survivors are an increasingly resilient group whose moment-to-moment risk is easing (sometimes seen after successful surgery, once the immediate post-operative risk window has passed); a constant hazard means the risk per instant never changes no matter how long a patient has already survived — "no memory" of how long they've made it so far, like a light bulb that is no more or less likely to fail today whether it's brand new or has been running for a year (this is exactly what the exponential distribution below assumes). Every parametric distribution in §10.2 is really just a different story about how this speedometer-needle for risk moves over time, and the proportional hazards assumption central to Cox models and §10.7 is a claim specifically about the **ratio** of two groups' speedometer readings staying constant, even as both needles independently rise or fall together over time.

10.1 Kaplan-Meier and life tables

You already met `survival::Surv()` and `survival::survfit()` in Chapter 9, where we used them to build inverse-probability-of-censoring weights for cost data. Here we return to the same two functions for their original purpose: describing the survival experience of a cohort, non-parametrically, as the foundation everything else in this chapter builds on.

The Kaplan-Meier (KM) estimator is the step-function analogue of a life table — at each event time it recalculates the proportion of the at-risk set that survives past that instant, and multiplies these conditional probabilities together. For a TB-programme analyst this will feel familiar: it is conceptually the same bookkeeping as a programme cohort analysis, just computed at the resolution of individual event times rather than fixed intervals (6-month, 12-month cohort outcomes).

```
library(survival)

fit <- survfit(Surv(time, status) ~ arm, data = trial_df)
summary(fit)
```

`time` is time-on-study (months, say), and `status` is the usual 1 = event, 0 = censored coding. The `~ arm` term stratifies the KM curve by treatment arm — essential in HEOR work, since the whole point is usually to compare extrapolated survival between a new intervention and a comparator.

Life tables (actuarial estimators) are the discrete-time cousin of the KM estimator — they group events into fixed intervals (e.g. each year) rather than using exact event times. `survival::survfit()` will produce an actuarial-style table if you pre-aggregate your data into interval counts, but in practice almost all HEOR survival work in R uses the exact-time KM estimator rather than classical life tables, because source trial data are individual patient data (IPD),

not grouped registry counts. We mention life tables mainly so you recognise the term when it appears in NICE or HTA submission templates referring to older epidemiological methods.

For plotting, two tidyverse-friendly packages dominate: `ggsurvfit` and `survminer`.

```
library(ggsurvfit)

survfit2(Surv(time, status) ~ arm, data = trial_df) |>
  ggsurvfit() +
  add_confidence_interval() +
  add_risktable()
```

`ggsurvfit::survfit2()` is a thin wrapper around `survival::survfit()` that keeps the formula call bundled with the plotting pipe, and `add_risktable()` gives you the number-at-risk row underneath the x-axis that every HTA reviewer expects to see.

```
library(survminer)

ggsurvplot(fit, data = trial_df, risk.table = TRUE,
           conf.int = TRUE, pval = TRUE)
```

`survminer::ggsurvplot()` is the older, more heavily parameterised alternative — it does more in one call (log-rank p-value annotation, risk table, median survival lines) at the cost of a less ggplot2-native object if you want to customise further downstream. Either is defensible in a submission; pick one and be consistent across your analysis.

> **HEOR in practice.** Every HTA economic model that extrapolates survival should open with the observed KM curve, full stop — regulators and HTA reviewers (NICE DSU TSD 14, ICER reference case) expect to see the "trial period" fit visually overlaid with whatever parametric curve you eventually choose to extrapolate with, so that the point of divergence between observed and modelled survival is inspectable, not asserted.

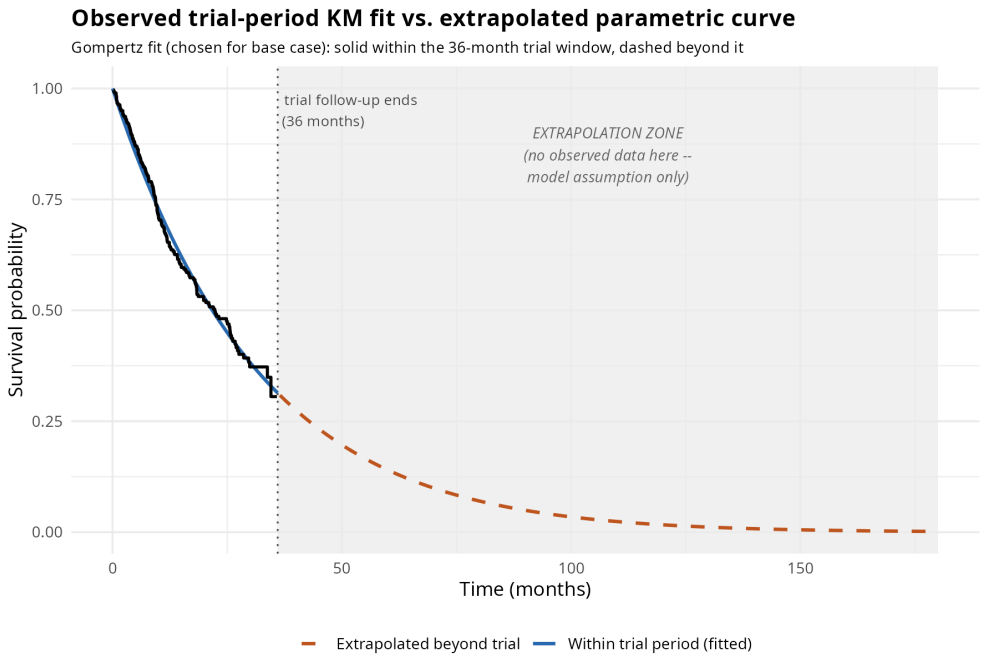


Figure 9: Exactly the diagnostic the practice note above asks for: the observed Kaplan-Meier curve (black) against a chosen parametric fit (here, Gompertz), solid where it is fitted to real trial-period data and dashed where it is pure extrapolation beyond the 36-month follow-up window. The vertical dotted line marks the trial's end — everything to its right is a model assumption, not an observation, and should be labelled as such in any submission.

Common pitfall. `survfit()` silently treats anyone not explicitly flagged `status == 1` as censored. If your event indicator was recoded upstream (e.g. a SQL extract where dead/alive was stored as `'Y'/'N'` strings and someone cast it to numeric without checking `0/1` directions), you can get a KM curve that is the mirror image of reality and it will not throw an error — always sanity-check with `table(trial_df$status)` and a known-rate benchmark before trusting the curve.

10.2 Parametric models

The KM curve only describes survival for as long as you have follow-up data. For a cost-effectiveness model you almost always need survival projected over a lifetime horizon, far beyond the trial's observation window — and that requires fitting a parametric distribution whose tail behaviour you can defend analytically, rather than just eyeballing where the KM curve seems to be heading.

The `flexsurv` package is the standard R workhorse for this. `flexsurv::flexsurvreg()` has the same `Surv(time, status) ~ ...` formula interface you already know, plus a `dist =` argument selecting the parametric family:

```
library(flexsurv)

fit_exp      <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
"exp")
fit_weibull  <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
```

```
"weibull")
fit_gompertz <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
"gompertz")
fit_llogis   <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
"llogis")
fit_lnorm    <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
"lnorm")
fit_gengamma <- flexsurvreg(Surv(time, status) ~ arm, data = trial_df, dist =
"gengamma")
```

Each distribution encodes a different assumption about the hazard shape over time, which is really the thing you are choosing, not the distribution name itself:

- **Exponential** (`"exp"`) — constant hazard. One parameter, almost never plausible for a chronic disease but a useful lower-bound sanity check.
- **Weibull** (`"weibull"`) — monotonically increasing or decreasing hazard, shape fixed by one parameter. Common for mortality that worsens steadily with age or disease progression.
- **Gompertz** (`"gompertz"`) — hazard that increases (or decreases) exponentially with time; a long-standing favourite for all-cause general-population mortality extrapolation.
- **Log-logistic** (`"llogis"`) — hazard can rise then fall (a hump), useful when early risk is elevated and later risk plateaus or drops.
- **Log-normal** (`"lnorm"`) — similar hump-shaped hazard behaviour to log-logistic, fitted on the log-time scale.
- **Generalised gamma** (`"gengamma"`) — a flexible three-parameter family that nests exponential, Weibull and log-normal as special cases; useful as an umbrella comparison model, at the cost of being harder to interpret and sometimes unstable to fit with sparse data.

Comparing fits is done the way you would compare any likelihood-based R models — AIC/BIC plus a visual check:

```
AIC(fit_exp, fit_weibull, fit_gompertz, fit_llogis, fit_lnorm, fit_gengamma)

plot(fit_gompertz, main = "Gompertz fit vs KM")
```

`plot()` on a `flexsurvreg` object automatically overlays the fitted parametric curve against the KM estimate, which is the single most useful diagnostic in this whole section.

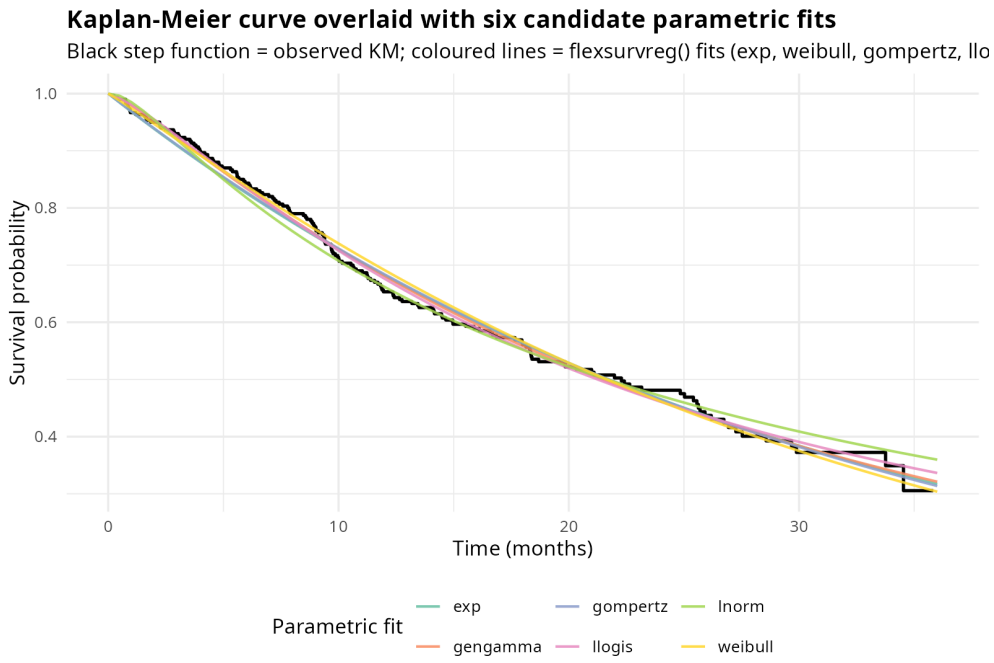


Figure 10: The observed Kaplan-Meier curve (black step function) overlaid with all six standard parametric families fitted via `flexsurvreg()`. Within the trial's follow-up window the fits visually cluster close to the KM curve and to each other — the divergence that matters for extrapolation only becomes visible once the curves are projected beyond the observed data (see Figure 14).

HEOR in practice. NICE DSU TSD 14 and most HTA bodies expect you to report **all six** standard parametric fits (or at minimum exponential, Weibull, Gompertz, log-logistic, log-normal, generalised gamma) with AIC/BIC tabulated side by side, plus visual and clinical-plausibility justification for whichever one is chosen for the base case — "it had the lowest AIC" is necessary but not sufficient; a model with slightly worse AIC that produces a clinically plausible long-term hazard is often preferred over one that fits the trial tail well but extrapolates implausibly (e.g. a cure-like plateau with no biological rationale).

Common pitfall. AIC/BIC only tell you about fit **within the observed data range** — two distributions can have near-identical AIC on a trial with two years of follow-up and diverge wildly once extrapolated to twenty years. Always plot the extrapolated curve out to your full model time horizon (`summary(fit, t = seq(0, horizon, by = ...))`) before trusting an AIC ranking for anything beyond the trial period.

10.3 Flexible and spline-based models

The six standard parametric families in §10.2 each assume a fixed, simple hazard shape. Real disease hazards — especially in conditions with changing treatment effect over time, or where a trial's hazard shape genuinely does something unusual (e.g. a drop-off after a defined treatment-stopping point) — can need more flexibility than any single-shape parametric curve offers without overfitting the whole dataset to force it.

The Royston-Parmar spline-based survival model, implemented in R as `flexsurv::flexsurvspline()`, solves this by modelling a spline on the log-cumulative-hazard (or related) scale, with the number of "knots" controlling flexibility:

```
library(flexsurv)

fit_spline0 <- flexsurvspline(Surv(time, status) ~ arm, data = trial_df, k = 0)
fit_spline1 <- flexsurvspline(Surv(time, status) ~ arm, data = trial_df, k = 1)
fit_spline2 <- flexsurvspline(Surv(time, status) ~ arm, data = trial_df, k = 2)
```

`k = 0` (zero internal knots) collapses to one of the standard parametric families depending on the `scale =` argument (`"hazard"`, `"odds"` or `"normal"` — defaulting to the Weibull-equivalent on the hazard scale), so it is a useful baseline before adding flexibility. Each additional knot (`k = 1`, `k = 2`, …) allows an extra inflection in the hazard shape, at the cost of an extra parameter to estimate and interpret.

```
AIC(fit_spline0, fit_spline1, fit_spline2, fit_gompertz, fit_gengamma)
plot(fit_spline1, main = "1-knot spline vs KM")
```

Because splines are piecewise and fit the observed data very closely, they are excellent at reproducing the trial-period KM curve — but that closeness is exactly why extrapolation beyond the data needs extra scrutiny: a spline has no built-in assumption about what the hazard does once you run out of data, so beyond the last knot it extrapolates essentially linearly on whatever scale you chose, which can produce hazards that neither clinicians nor biology would endorse.

HEOR in practice. Spline models are best used as a **flexibility check** on your chosen standard parametric model, not usually as the base-case extrapolation itself, unless you have strong external long-term data (registry, national life tables) to anchor the tail. A common, reviewer-friendly workflow: fit `flexsurvspline()` with one or two knots, confirm it tracks the KM curve no better than your chosen standard parametric fit **within** the trial period, and if it does track meaningfully better, investigate why (treatment-stopping rule? crossover? subgroup heterogeneity?) rather than simply swapping in the spline as the new base case.

Common pitfall. More knots will always reduce AIC within the observed data — splines are flexible enough to chase noise, especially in TB-programme-style IPD with modest sample sizes and a long tail of sparse late-follow-up events. Treat any spline with `k >= 2` on anything less than a few hundred events as a plausible overfit until you've checked that the extra knot survives a visual bootstrap or sensitivity check, not just an AIC improvement.

How to report this.

Knots (k)	Parameters	−2LogLik	AIC	BIC	Notes
0 (Weibull-equivalent)	2	799.1	803.1	808.5	Baseline; equivalent to standard Weibull on the hazard scale
1	3	793.4	799.4	807.1	Captures an early hazard bend not visible in k = 0
2	4	791.0	799.0	809.0	Marginal AIC gain over k = 1; extra knot not clinically motivated
3	5	790.1	800.1	812.3	BIC increases versus k = 1-2: additional flexibility not supported by sample size
Selected: k = 1 for the base case – lowest BIC and the smallest model whose extra flexibility tracks a plausible early hazard change; k >= 2 shows the AIC-chasing pattern flagged in the pitfall above.					

Table 26: How to report this: Royston-Parmar spline knot-selection comparison (`flexsurvspline()`, hazard scale) across k = 0-3 internal knots, with AIC/BIC and the within-trial-period fit improvement each extra knot buys – the format for justifying a chosen knot count against TSD 21's expectation that knot selection be reported, not merely asserted.

10.4 Mixture-cure and non-mixture cure models

In plain terms first: every model in §10.2 assumes that, eventually, everyone has the event — the survival curve always keeps falling toward zero, however slowly. But some conditions genuinely leave a fraction of patients who are, in effect, "cured": they never relapse, no matter how long you follow them. If that is true of your data, forcing an ordinary Weibull or Gompertz curve onto it is a category error — the curve will keep sliding downward long after the real data has flattened out, systematically understating long-term survival. A **cure model** fixes this by explicitly fitting two things at once: the proportion who are cured, and a separate survival curve just for the ones who are not.

Every parametric curve fitted in §10.2 — Weibull, Gompertz, log-logistic, generalised gamma — assumes that, given enough follow-up, every patient in the model eventually experiences the event. For many TB treatment outcomes that assumption is fine: everyone who survives active disease is still at risk of relapse or death indefinitely. But for some oncology indications, and arguably for a "treatment-success" endpoint sustained well beyond the biological window for TB relapse, a meaningful fraction of patients may never experience the event at all — they are statistically "cured." Ordinary parametric models forced to fit such data either underestimate the plateau (the

Weibull tail keeps falling) or overestimate it (the log-logistic tail flattens for the wrong reason, as a pure curve-fitting artefact rather than a clinical one).

Cure models address this directly by estimating a **cure fraction** π — the proportion of the cohort who will never experience the event — alongside a standard parametric survival distribution for the "uncured" subgroup. The `flexsurvcure` package extends the `flexsurv` machinery from §10.2-§10.3 with exactly this structure, via `flexsurvcure::flexsurvcure()`.

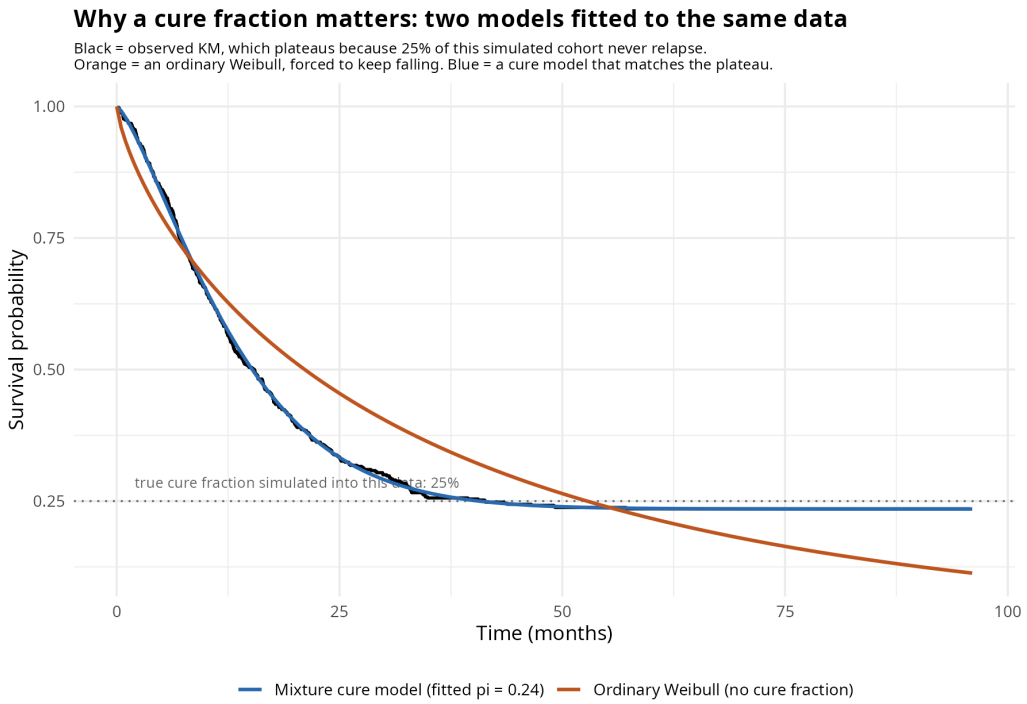


Figure 11: A simulated cohort where 25% of patients are genuinely cured. The observed Kaplan-Meier curve (black) correctly plateaus near 0.25 instead of continuing to fall. An ordinary Weibull (orange), which has no way to represent a cure fraction, keeps sliding downward and badly understates long-term survival. A mixture cure model (blue) matches the plateau because it estimates the cure fraction directly — here recovering $\pi \approx 0.24$, close to the true simulated value of 0.25.

Two structurally different cure model families are supported, selected by the `mixture` argument:

```
library(flexsurvcure)
library(flexsurv)
library(survival)

# Mixture cure model: S(t) = pi + (1 - pi) * S_u(t)
fit_mix <- flexsurvcure(
  Surv(time, status) ~ 1,
  data = tb_followup,
  dist = "weibull",
  mixture = TRUE
)

# Non-mixture cure model: S(t) = pi ^ (1 - S_u(t))
fit_nmix <- flexsurvcure(
  Surv(time, status) ~ 1,
  data = tb_followup,
```

```
  dist = "weibull",
  mixture = FALSE
)

fit_mix
```

In the **mixture** form, the population is an explicit blend: a cured fraction π with survival fixed at 1 forever, and an uncured fraction (1-π) following the chosen parametric distribution. This is the more clinically interpretable form for HEOR write-ups — π reads directly as "proportion cured" — and it extends naturally to covariates on both the cure fraction and the uncured-group survival:

```
fit_mix_cov <- flexsurvcure(
  Surv(time, status) ~ arm,
  data = tb_followup,
  dist = "weibull",
  mixture = TRUE,
  anc = list(shape = ~ arm)
)
```

The **non-mixture** form arises from a different generative story (a latent number of malignant/infectious cells following a Poisson process, each with independent probability of elimination) and is more common in oncology cure-modelling traditions. Both forms nest the standard non-cure model as $\pi \longrightarrow 0$, so a likelihood-ratio test or AIC comparison against the corresponding plain `flexsurvreg` fit (§10.2) is a reasonable first check of whether a cure fraction is statistically supported at all.

Common pitfall. A fitted cure fraction is a **statistical** artefact of the likelihood, not evidence of biological cure. A model will happily return a cure estimate of 0.15 purely because the observed Kaplan-Meier curve (§10.1) flattens near the end of follow-up — which is exactly what **censoring** also does. Short or administratively-censored follow-up is the single most common cause of a spuriously large cure estimate. Always check: does the KM curve's plateau extend across enough separated event-free time, with enough patients still at risk at the plateau, to distinguish "nobody left to have an event" from "these people are actually cured"? If follow-up barely exceeds the plausible window for relapse, treat any cure-model result as exploratory only.

HEOR in practice. When a submission depends on a cure assumption, present the cure fraction alongside its 95% CI (`summary()` on the `flexsurvcure` object returns this) and run a one-way sensitivity analysis setting the cure fraction to zero and to the CI bounds in the economic model. Reviewers at HTA bodies (NICE, in particular, following TSD 14 guidance — see §10.12) routinely request this, and having it ready pre-empts a request for clarification that otherwise stalls appraisal.

How to report this.

Parameter	Estimate	95% CI lower	95% CI upper
Cure fraction (pi)	0.24	0.17	0.32
Uncured Weibull shape	1.42	1.08	1.87
Uncured Weibull scale	18.6	14.2	24.3
Model comparison			
Mixture cure model AIC	812.4	–	–
Standard Weibull (no cure) AIC	829.1	–	–

Table 27: How to report this: cure-fraction estimate from a mixture cure model (`flexsurvcure()`, Weibull uncured distribution), with the 95% confidence interval `summary()` returns, alongside the AIC comparison against the corresponding non-cure Weibull fit that motivates including a cure fraction at all.

10.5 Piecewise and landmark models

In plain terms first: sometimes the reason one curve doesn't fit well isn't that you chose the wrong family — it's that the real hazard genuinely behaves differently in different time windows (for example, high risk during active treatment, then a much calmer surveillance phase afterwards). No single smooth curve can bend sharply enough to capture that kind of structural change without overfitting elsewhere. The practical fix, called a **landmark** approach, is almost deliberately low-tech: trust the real, observed Kaplan-Meier curve for as long as you have good data, and only hand off to a fitted parametric curve for the unobserved tail beyond that point — rather than asking one curve to explain both phases at once.

Cure models relax the **shape** assumption; piecewise models relax it in a different way, by abandoning the idea that one parametric family must hold over the entire time horizon. A piecewise (segmented) model fits different hazard or survival behaviour in separate time intervals, stitched together at one or more cut-points. This is a natural fit whenever a clinical or data-generating mechanism changes partway through follow-up — for instance, a fixed-duration TB regimen where hazard during active treatment differs structurally from hazard during the post-treatment surveillance period, or trials where hazard behaviour measured within the trial should not simply be extrapolated using whatever curve happened to win AIC (§10.6) beyond the data.

A **landmark** approach is the simplest practical version of this idea for extrapolation: fit (or simply use) the empirical Kaplan-Meier estimate for the observed trial duration, and attach a parametric extrapolation only from a chosen landmark time point onward, rather than letting one parametric curve govern both the well-observed early period and the long unobserved tail.

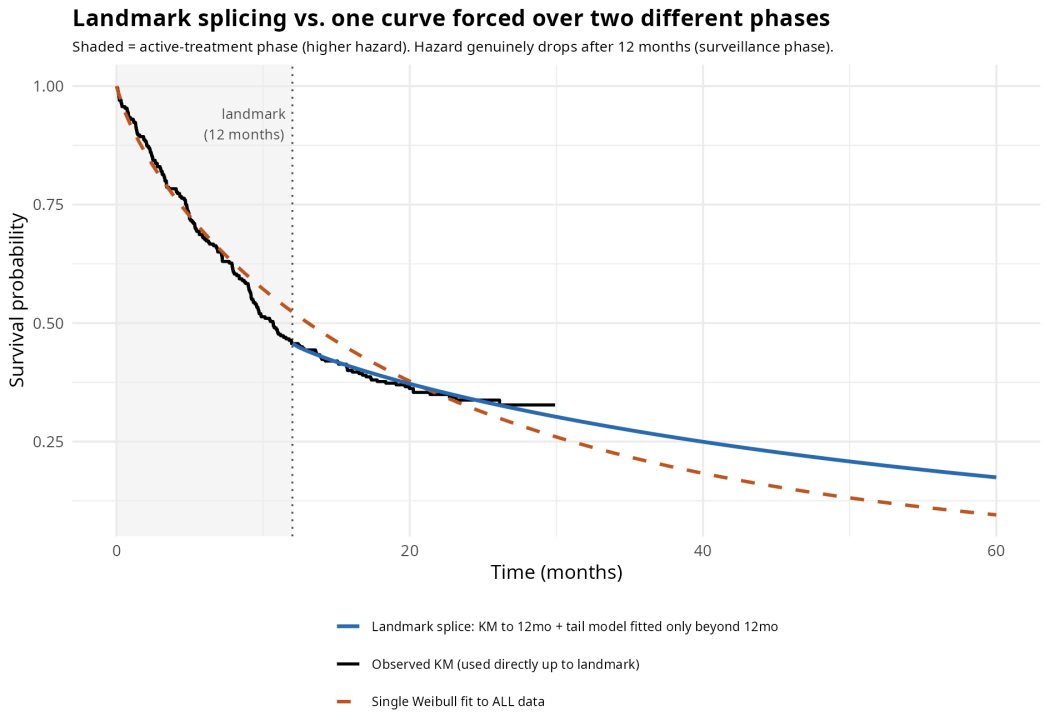


Figure 12: A simulated trial where hazard genuinely drops after a 12-month landmark (shaded: the higher-hazard active-treatment phase). A single Weibull fitted to all the data (orange, dashed) is forced to compromise between the two phases and fits neither well. The landmark approach (blue) instead uses the real KM curve up to 12 months, then splices in a tail model fitted only to the post-landmark data — tracking the true post-landmark decline far more closely.

```
library(survival)
library(flexsurv)

landmark_time <- 24  # months; chosen where KM data becomes sparse

fit_pw <- survival::survSplit(
  Surv(time, status) ~ .,
  data = tb_followup,
  cut = landmark_time,
  episode = "period"
)

fit_pw_exp <- survival::coxph(
  Surv(tstart, time, status) ~ strata(period),
  data = fit_pw
)
```

In practice, the economic-model-facing version of a landmark approach is often simpler than fitting a formal piecewise hazard model in R: use the KM estimate directly for t ≤ landmark, then splice in a `flexsurvreg` or `flexsurvspline` fit (§10.2-§10.3) — chosen for clinical plausibility of its tail, not merely best AIC — for t > landmark. The spline models of §10.3 are a particularly natural extrapolation partner here because their flexible knot structure can be fitted to just the later portion of follow-up without distorting the early, well-observed KM shape.

```
tail_data <- subset(tb_followup, time > landmark_time)

fit_tail <- flexsurvspline(
  Surv(time, status) ~ 1,
  data = tb_followup,
  k = 1,
  scale = "hazard"
)
```

Common pitfall. A piecewise model with a cut-point placed wherever the hazard **happens** to look like it changes, chosen after inspecting the data, is overfitting dressed up as flexibility — the same multiple-comparisons problem as moving a regression breakpoint until R^2 improves. Cut-points should be pre-specified from a clinical rationale (end of a fixed treatment course, a protocol-defined visit, a known change in standard of care) wherever possible, not selected by eyeballing the KM curve for the best-looking kink.

HEOR in practice. Landmark extrapolation is especially defensible — and increasingly expected by reviewers — whenever a trial's follow-up is short relative to the economic model's time horizon, since it keeps the empirical, assumption-free KM estimate (§10.1) governing exactly the period where trial data are informative, and confines parametric or spline modelling risk to the period where it is unavoidable.

10.6 Model selection: AIC/BIC, visual fit, clinical plausibility

In plain terms first: this section is about a single, important trap. Every statistic you can compute from a model — AIC, BIC, log-likelihood — only measures how well the curve matches the data you actually observed. It has nothing to say about the part of the curve beyond your data, even though that unobserved region is usually what drives most of an economic model's lifetime costs and QALYs. Two curves can score almost identically on AIC and still tell completely different stories about year 10 or year 20. The fix isn't a smarter statistic — it's to never trust a single number alone: always look at the extrapolated curve with your own eyes, and ask whether its long-run shape makes clinical sense.

Difficult terminology: log-likelihood. The log-likelihood is a single number that summarises how probable the data you actually observed would have been, if the fitted model's parameters were exactly correct — it is the mathematical currency nearly every model-comparison tool in this book spends (AIC and BIC, which you've already seen used for parametric survival curves, are both built directly from it). A higher log-likelihood (remember these are typically negative numbers, so "higher" usually means "less negative," i.e. closer to zero) means the fitted model assigns higher probability to the data you actually saw — in other words, a better-fitting model. A simple way to build intuition: imagine two coins, one you believe is fair and one you believe lands heads 90% of the time; if you actually flip a coin and get eight heads out of ten, the "90% heads" model assigns a much higher probability to that specific outcome than the "fair coin" model does — its log-likelihood, given that data, is higher. Survival models work the same way at a larger scale: the log-likelihood adds up, across every patient in the

dataset, how probable each patient's observed outcome (their exact event time, or the fact that they were censored at a particular point) was under the fitted hazard curve. Crucially — and this is the trap this section is warning about — the log-likelihood only ever evaluates how well a model explains data you **actually have**; it has nothing to say about how trustworthy the model's extrapolated tail is, because there is no observed data out there to check it against.

Every model fitted across §10.2-§10.5 — standard parametric families, spline models, cure models, piecewise/landmark constructions — produces a log-likelihood, and every one of those model objects supports the base R generics directly:

```
fits <- list(
  exp      = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"exp"),
  weibull  = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"weibull"),
  gompertz = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"gompertz"),
  llogis   = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"llogis"),
  lnorm    = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"lnorm"),
  gengamma = flexsurvreg(Surv(time, status) ~ 1, data = tb_followup, dist =
"gengamma")
)

sapply(fits, AIC)
sapply(fits, BIC)
```

AIC and BIC rank goodness-of-fit **to the observed data**, penalised for the number of parameters (BIC more heavily, favouring simpler models in larger samples). They are a legitimate, necessary first filter — a model that fits the observed follow-up poorly should rarely be carried forward regardless of how convenient its extrapolated tail looks. But an information criterion says nothing about behaviour **beyond** the data, which is precisely the region that drives most of the lifetime cost and QALY impact in an economic model.

How to report this.

Distribution	Parameters	−2LogLik	AIC	BIC
Exponential	1	812.6	814.6	817.3
Weibull	2	798.2	802.2	807.6
Gompertz	2	796.8	800.8	806.2
Log-logistic	2	799.5	803.5	808.9
Log-normal	2	801.1	805.1	810.5
Generalised gamma	3	795.9	801.9	809.9

Gompertz selected for the base case: lowest AIC among two-parameter families and a clinically plausible extrapolated tail; generalised gamma's marginally lower AIC did not outweigh its less stable shape-parameter estimate (§10.2).

Table 28: How to report this: AIC/BIC model-comparison table across the six standard parametric survival distributions fitted with `flexsurvreg()`, the TSD 14-expected candidate set and reporting format for justifying a base-case extrapolation choice.

Common pitfall. Two curves can have near-identical AIC over the observed follow-up and diverge enormously once extrapolated — one flattening into a plausible long-term plateau, the other decaying to near-zero survival within a few extra years. AIC/BIC cannot distinguish between them because both explain the **observed** data equally well; the difference lives entirely in the unobserved region the criterion was never calculating over. Selecting a final extrapolation model on AIC/BIC alone is one of the most common and most criticised errors in HTA submissions.

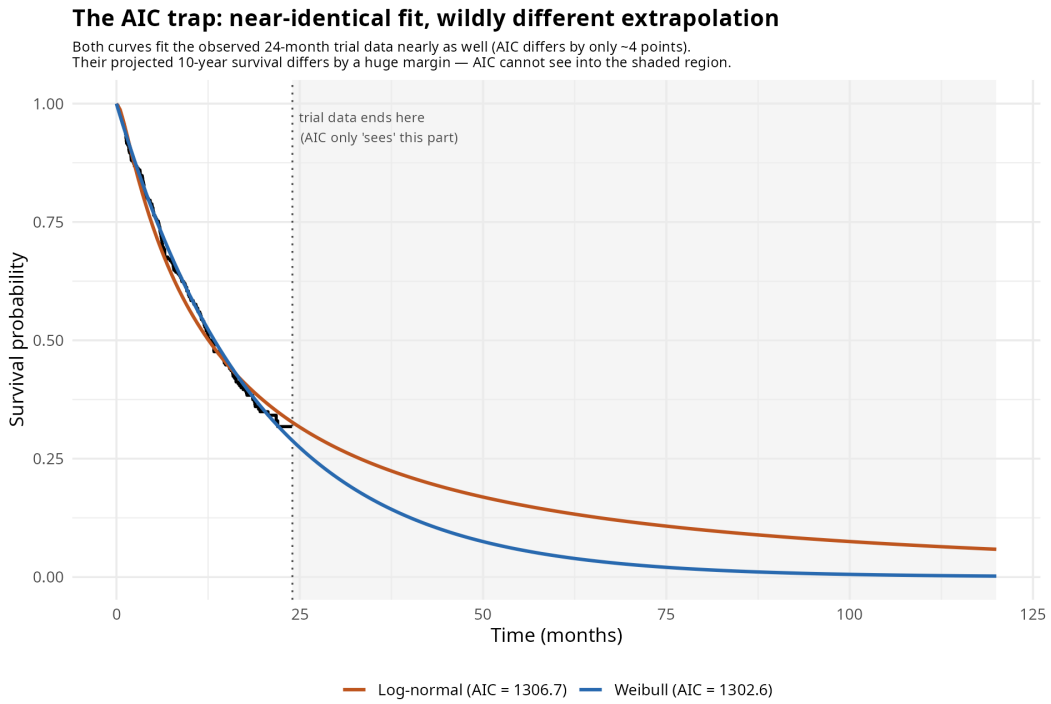


Figure 13: A simulated 24-month trial where a Weibull and a log-normal fit the observed data almost equally well (AIC 1302.6 vs 1306.7 — a difference most analysts would call negligible). Extrapolated to a 10-year horizon (shaded region), the two curves diverge sharply: by month 120 one projects roughly 6% survival and the other under 1%. AIC alone gives no warning that this divergence is coming, because it is calculated entirely within the unshaded region.

Two further checks are required alongside the statistical criteria, and should be treated as mandatory, not optional, for any model informing extrapolation:

- **Visual fit against the full observed follow-up.** Overlay each candidate curve on the Kaplan-Meier estimate (§10.1) across the **entire** observed period, not just a global fit statistic — look specifically for systematic departures in the region with the most patients still at risk, since that is where poor fit matters most for inference.
- **Clinical plausibility of the extrapolated tail.** Beyond the data, ask whether the shape is biologically defensible: does a Gompertz tail imply an implausibly rapid late-life hazard escalation for this population? Does a cure-model fraction (§10.4) correspond to a mechanism (e.g. durable bacteriological clearance) that a clinician would recognise, or is it just where the likelihood happened to settle? Plot each candidate's extrapolated survival against the full economic-model time horizon and sanity-check the implied long-term survival or life-years against external epidemiological data or clinical opinion.

```
plot(fits$weibull, type = "survival")
lines(survfit(Surv(time, status) ~ 1, data = tb_followup), col = "grey40")

plot(fits$gompertz, type = "survival", t = seq(0, 240, by = 1))
```

This three-part discipline — statistical fit (AIC/BIC), visual fit, and clinical plausibility of the tail — is the core of the NICE DSU Technical Support Document 14 (Latimer, 2013) guidance on survival extrapolation, still the reference standard cited by most HTA bodies. §10.12 returns to this guidance in full detail, including external-data validation and expert-elicitation approaches to tail

plausibility; the practical takeaway here is simply that no single number from `AIC()` should ever be the sole basis for choosing an extrapolation model.

HEOR in practice. Build the habit of reporting all three elements together for every extrapolation choice in a submission: an AIC/BIC table across candidate models, a KM-overlay plot across observed follow-up, and a long-horizon extrapolation plot with a short clinical-plausibility justification for the chosen tail. This triad is close to a checklist item in NICE technical engagement, and assembling it alongside model-fitting — rather than retrofitting it after a model is already locked into the economic model — saves substantial rework later.

Agency perspective. For a TB programme specifically, "clinical plausibility of the tail" has a concrete anchor: published TB relapse-hazard literature and programme surveillance data give an external benchmark for what a defensible long-term hazard or cure fraction looks like. Treat any model whose extrapolated tail falls well outside that external range as a candidate to be interrogated, not adopted, however well it scores on AIC.

10.7 Proportional hazards diagnostics and non-PH alternatives

Chapter 10's Cox models (via `survival::coxph()`) and the parametric curves fitted with `flexsurv::flexsurvreg()` in §10.2 share a hidden assumption whenever a treatment or covariate effect is expressed as a single hazard ratio: that the ratio of hazards between arms stays constant for the whole follow-up period. This is sometimes called the proportional hazards (PH) assumption. In an economic model this assumption is load-bearing — it is usually what licenses stretching a trial's within-trial hazard ratio across a lifetime horizon. If the assumption is wrong, the extrapolated incremental QALYs and costs inherit that error, often invisibly.

Testing the proportional hazards assumption

`survival::cox.zph()` tests whether the scaled Schoenfeld residuals from a fitted `coxph()` model are correlated with time. Under PH, residuals for a covariate should fluctuate randomly around zero; a trend indicates a time-varying effect.

Difficult terminology: Schoenfeld residuals. A Schoenfeld residual, computed separately at each event time in the data, captures the gap between what actually happened and what the model's proportional-hazards assumption expected to happen on average at that moment. Concretely: at each event time, look at the patient who had the event, and compare their covariate value (say, their actual treatment arm or age) to a weighted average of the covariate values among everyone still at risk at that moment — the model's implicit prediction of what that value "should" look like if the hazard ratio really is constant over time. If the proportional hazards assumption truly holds, there's no reason these gaps should trend in any direction as time passes — they should scatter randomly above and below zero throughout follow-up, like noise with no pattern. If instead the residuals visibly drift upward or downward as time goes on, that is direct evidence the covariate's effect on risk isn't staying constant — e.g. a new drug's benefit might be large early on then fade, which would show up as the residuals systematically drifting in one direction across the follow-up period rather than bouncing

randomly around zero. `cox.zph()`'s formal test and accompanying plot are simply a statistical and visual way of checking for exactly that drift.

```
library(survival)

cox_fit <- coxph(Surv(time, status) ~ arm + age + ecog_ps, data = os_data)
zph_test <- cox.zph(cox_fit)
zph_test
plot(zph_test, var = "arm")
```

A significant global or `arm`-specific p-value (conventionally < 0.05) is evidence against PH for that term. The accompanying plot shows the estimated log hazard ratio against time; a flat line supports PH, a clear slope does not.

Common pitfall. A non-significant `cox.zph()` test is not proof of PH — it is merely absence of evidence, and the test has low power in small or heavily censored trials (exactly the oncology trials HEOR models often rely on). Conversely, with very large samples, trivial and clinically irrelevant time-trends can reach significance. Always pair the test with the Schoenfeld residual plot and clinical judgement about mechanism (e.g. a new treatment with delayed onset, or diminishing effect as resistance emerges) before deciding PH is tenable.

How to report this.

Term	chi-square	df	p-value	PH decision
arm	1.2	1	0.27	Supported – no evidence of a time-varying treatment effect
age	0.4	1	0.53	Supported
`ecog_ps`	7.8	1	0.005	Rejected – effect of ECOG status changes materially over follow-up
GLOBAL	9.1	3	0.028	Rejected overall – a non-PH alternative structure is warranted (see below)

Model: `coxph(Surv(time, status) ~ arm + age + ecog_ps)`; global rejection driven by `ecog_ps`, motivating the AFT or spline-based non-PH alternatives discussed next.

Table 29: How to report this: `cox.zph()` proportional-hazards diagnostics for a Cox model of treatment arm, age and ECOG performance status – the conventional format for presenting per-covariate and global Schoenfeld-residual test statistics alongside the PH decision each motivates.

When PH fails: alternative hazard structures

If PH is rejected, three broad options replace the single hazard ratio:

- **Piecewise/time-varying hazard ratio** — fit `coxph()` with a time-dependent covariate (e.g. via `survival::survSplit()` to cut follow-up into intervals and interact `arm` with interval), yielding a stepped hazard ratio rather than a constant one.
- **Accelerated failure time (AFT) models** — `flexsurv::flexsurvreg()` with `dist = "weibull"`, `"llogis"`, or `"lnorm"` from §10.2 are AFT forms: treatment rescales survival **time** rather than multiplying the hazard, which can fit crossing or converging survival curves that PH cannot.
- **Flexible spline-based hazards** — `flexsurv::flexsurvspline()` allows the treatment effect to enter on the hazard, odds, or normal scale with splines on the baseline, directly relaxing PH while remaining a parametric, extrapolable model.

```
library(flexsurv)

aft_weibull <- flexsurvreg(
  Surv(time, status) ~ arm,
  data = os_data,
  dist = "weibull"
)

spline_fit <- flexsurvspline(
  Surv(time, status) ~ arm,
  data = os_data,
  k = 1,
  scale = "hazard"
)
```

> **HEOR in practice.** NICE's DSU TSD 14 (survival analysis for economic evaluation) treats PH testing as a mandatory first step before any extrapolation, and expects submissions that reject PH to justify the alternative structural choice clinically, not just statistically. In a reimbursement submission, present `cox.zph()` output and the Schoenfeld plot alongside the chosen non-PH model as part of the base-case justification, not as a sensitivity analysis afterthought.

10.8 Covariate-adjusted and treatment-effect-specific models

Economic models rarely want a single marginal survival curve per arm; they want the curve for a population defined by baseline covariates (age, stage, biomarker status) and sometimes a treatment effect that itself depends on those covariates. Building on the `flexsurvreg()` syntax from §10.2, covariates enter through the familiar formula interface — but **where** they enter (location, scale, or shape of the distribution) changes what the model can represent.

Covariates on the location/scale parameter

By default, `flexsurvreg()` places covariates on the main location parameter of the chosen distribution (the rate for exponential, the scale for Weibull, etc.), which is the AFT-style adjustment used throughout §10.2.

```
adj_weibull <- flexsurvreg(
  Surv(time, status) ~ arm + age + ecog_ps,
  data = os_data,
  dist = "weibull"
)
```

This yields a single estimated treatment effect, held constant across the covariate distribution — adequate when the question is "what survival curve for a patient of average age and ECOG status," but it cannot represent a treatment effect that differs by subgroup.

Covariates on ancillary (shape) parameters

`flexsurvreg()`'s `anc` argument attaches covariates to a distribution's ancillary parameter (e.g. the shape of a Weibull or Gompertz, the sigma of a log-normal), letting a covariate change the **shape** of the hazard trajectory rather than only its scale.

```
shape_fit <- flexsurvreg(
  Surv(time, status) ~ arm,
  anc = list(shape = ~ arm),
  data = os_data,
  dist = "weibull"
)
```

Here `arm` enters both the scale (via the main formula) and the shape (via `anc`), allowing treatment to alter not just how long patients survive but whether the hazard is increasing, decreasing, or constant over time — a natural way to express mechanisms like "early benefit that fades" without abandoning a parametric, extrapolable family.

> **Common pitfall.** Adding covariates to ancillary parameters multiplies the number of estimated coefficients quickly, and with trial-sized samples (hundreds, not thousands) the shape-parameter estimates can become unstable even when the model converges. Check standard errors and confidence intervals on the `anc` coefficients specifically — a wide interval on a shape-by-treatment interaction is a sign the data cannot support that extra flexibility, however appealing the fit statistic.

Treatment-effect-specific extrapolation

Once a covariate-adjusted model is fitted, `summary()` on the `flexsurvreg` object with a `newdata` argument produces arm- and covariate-specific survival curves directly, which feed into the Markov or partitioned-survival model exactly as the unadjusted curves did in §10.2-10.5.

```
newdat <- data.frame(arm = c("control", "treatment"), age = 60, ecog_ps = 1)
summary(shape_fit, newdata = newdat, t = seq(0, 60, by = 1), type = "survival")
```

> **HEOR in practice.** Decide before fitting whether the economic question is population-average (one curve per arm) or subgroup-specific (curves varying by baseline covariates): this determines whether `anc` and interaction terms belong in the base case or in a scenario analysis. Reviewers increasingly expect covariate-adjusted analyses when a trial shows baseline imbalance between arms, since an unadjusted hazard ratio can then be confounded rather than purely a treatment effect.

10.9 Extrapolation: external data, background mortality, expert elicitation

Trials report follow-up measured in months; reimbursement decisions are made on lifetime horizons. Every model in this chapter — from the single `flexsurvreg()` fit in §10.2 to the non-

PH alternatives in §10.7 — eventually has to answer a question the trial data cannot answer alone: what happens to the survival curve beyond the last observed event. Three complementary tools manage that gap: external data, background mortality, and structured expert elicitation.

Anchoring with external data

Where available, longer-follow-up data from a related trial, a registry, or a published meta-analysis can validate or recalibrate the tail of an extrapolated curve. The common workflow is to source external Kaplan-Meier data, read it in exactly as any other tabular data (`readr::read_csv()`, per Ch.5), and overlay it against the within-trial parametric fit visually and via landmark survival comparisons (e.g. does the model's 5-year survival estimate sit near the external estimate) rather than formally refitting the trial model to pooled data.

```
library(readr)
library(dplyr)

external_km <- read_csv("registry_survival_5yr.csv")

model_est <- summary(aft_weibull, t = 60, type = "survival")[[1]]
external_est <- external_km |> filter(time_months == 60)

bind_rows(model = model_est, external = external_est)
```

Background mortality as a plausibility bound

§8.5 introduced layering age- and sex-matched general-population mortality beneath disease-specific mortality once the disease-specific hazard falls below the general-population hazard — the standard device for making sure a chronic-disease model does not extrapolate a disease-specific survival curve indefinitely below what a healthy member of the same population would experience. In extrapolation, that convention becomes a **constraint**, not just a modelling convenience: a national life table (read in as plain CSV/Excel via `readr::read_csv()` or `readxl::read_excel()`, per Ch.5) sets the ceiling on hazard, so predicted survival should never fall below the general-population curve for the matching age-sex strata.

```
library(readr)
library(dplyr)

life_table <- read_csv("national_life_table.csv")  # age, sex, qx (annual
mortality prob.)

disease_hazard <- -log(1 - 0.08)
bg_hazard <- life_table |>
  filter(age == 70, sex == "female") |>
  pull(qx) |>
  (\(qx) -log(1 - qx))()

combined_hazard <- pmax(disease_hazard, bg_hazard)
combined_hazard
```

> **Agency perspective.** Treat this the same way Ch.8 §8.5 did: compute background mortality on the same timescale (annual vs monthly) as the disease-specific hazard before comparing or combining them, and apply the comparison at every model cycle, not just at the point where

extrapolation begins — a disease hazard that dips below background mortality transiently (not just permanently at the tail) still needs the floor applied at that cycle.

Common pitfall. A parametric curve with a poorly chosen tail shape (e.g. a log-normal or log-logistic fitted mainly to capture early-curve flexibility) can imply a long-term hazard that **decreases** with age, eventually crossing below general-population survival — biologically implausible for almost any serious disease. Plot the extrapolated curve against the life-table curve over the full model horizon as a standard diagnostic, not an edge-case check; NICE DSU TSD 14 reviewers routinely flag curves that cross.

Expert elicitation for structural and tail uncertainty

When neither longer trial follow-up nor comparable external data exist — common for recently approved treatments with short median follow-up — formal expert elicitation offers a way to quantify uncertainty about the extrapolated tail (e.g. long-term treatment-effect waning, cure fraction plausibility) that is defensible rather than ad hoc. The SHELF (SHeffield ELicitation Framework) protocol is the most widely used approach in HTA submissions for eliciting probability distributions from clinical experts on quantities like long-term survival proportions; it is covered in full in Ch.14 §14.7, including how elicited distributions are encoded as priors or scenario inputs. For now, the point to carry forward is structural: expert elicitation does not replace the parametric or external-data analysis above, it supplements it — typically by bounding or re-weighting extrapolation scenarios ("optimistic," "base-case," "pessimistic" tail assumptions) that are then propagated through the probabilistic sensitivity analysis built on the survival models of this chapter.

HEOR in practice. Build the extrapolation case as a layered argument, not a single curve: (1) the best-fitting parametric or spline model from §10.2-10.8, (2) a background-mortality floor from §8.5′s convention, (3) a cross-check against any available external data, and (4) where tail uncertainty remains material to the decision, a structured elicitation exercise (Ch.14 §14.7) to characterise that uncertainty rather than asserting a single point estimate. Submissions that show all four elements together are markedly more resilient under HTA committee scrutiny than a single AIC-selected curve.

10.10 Competing risks and multi-state models

Every extrapolation so far in this chapter — §10.2′s parametric distributions, §10.9′s extrapolation workflow — has treated the clinical course as one single transition: alive to dead, or progression-free to progressed. Many HEOR models need more states than that. Picture a TB treatment-pathway model tracking **treatment success**, **loss to follow-up**, **treatment failure/retreatment**, and **death**. Loss to follow-up competes with cure for who "claims" a patient first, and retreatment is itself a transition with its own hazard. A standard Kaplan-Meier or single-event parametric fit mishandles this silently: if you censor a patient at a competing event (instead of modelling that event explicitly), you bias the cumulative incidence of the event you actually care about — usually inflating it.

Two R packages work together to handle this: `mstate` for the data-management and semi-parametric multi-state machinery, and `flexsurv` for fitting parametric or flexible distributions to each transition separately — exactly what a multi-state cost-effectiveness model needs for extrapolation.

Competing risks: cause-specific vs. subdistribution hazards

With competing risks, there are at least two legitimate hazard concepts, and they answer different questions:

- The **cause-specific hazard** treats competing events as censoring and asks "among patients still at risk of this event, what's the instantaneous rate?" It is the right quantity for understanding aetiology/mechanism.
- The **subdistribution hazard** (Fine-Gray) keeps patients who experienced the competing event in the risk set (as if destined never to have the event of interest) and directly models the **cumulative incidence function** (CIF) — the probability of this event by time t in the presence of competing risks. This is usually the quantity a cost-effectiveness model wants, because the model needs the actual probability mass flowing into each state, not a hypothetical hazard in a world where competing events don't happen.

> **Common pitfall.** Censoring competing events in a standard Kaplan-Meier fit produces a cumulative incidence estimate that is **too high** — treating a competing event as censoring systematically overstates the true probability of the event of interest when competing risks are non-trivial. This is the single most common competing-risks error in submitted HTA models: a "time to loss-to-follow-up" curve fitted by censoring deaths will overestimate loss-to-follow-up, because dead patients can no longer be lost to follow-up but a censoring-based estimator treats them as still "at risk" and still eligible to drop out indefinitely. Always check whether competing events were censored or modelled when critiquing a submitted dossier's multi-state structure.

Data preparation and fitting with `mstate`

`mstate` works from a **transition matrix** describing the permitted moves between states, and converts standard long-format survival data into the stacked "msdata" format multi-state models require.

```
library(mstate)
library(survival)

tmat <- transMat(
  x = list(
    c(2, 4),     # from Treatment success: to Failure, or Death
    c(3, 4),     # from Failure: to Retreatment, or Death
    c(4),        # from Retreatment: to Death
    c()          # Death is absorbing
  ),
  names = c("TxSuccess", "Failure", "Retreat", "Death")
)

msdat <- msprep(
  time  = cbind(tx_success_time, failure_time, retreat_time, death_time),
  status = cbind(tx_success_status, failure_status, retreat_status, death_status),
  data  = tb_cohort,
```

```
  trans = tmat
)

cox_trans <- coxph(Surv(Tstart, Tstop, status) ~ strata(trans), data = msdat)
msf <- msfit(cox_trans, trans = tmat)
```

`msprep()` is doing the unglamorous but essential work: for every patient and every transition they were at risk of, it creates a row with a start time, stop time, and event indicator, in the "counting process" format `coxph()` expects when combined with `Surv(Tstart, Tstop, status)`. `msfit()` then assembles the fitted cumulative hazards per transition into the full set of transition probabilities — the quantity that actually populates a multi-state Markov cohort model's transition-probability matrix.

Parametric extrapolation of transition-specific hazards with `flexsurv`

`mstate`'s default Cox-based fitting is semi-parametric and does not extrapolate beyond the observed follow-up — exactly the gap §10.9 was built to fill for single-event models. The natural extension is to fit `flexsurvreg()` (§10.2's workhorse) separately to each transition's data, by subsetting the `msdata` object on `trans` and fitting the usual parametric families:

```
library(flexsurv)

transition_fits <- lapply(1:3, function(k) {
  flexsurvreg(
    Surv(Tstart, Tstop, status) ~ 1,
    data = subset(msdat, trans == k),
    dist = "weibull"
  )
})
```

Each `transition_fits[[k]]` can then be extrapolated exactly as in §10.9, and the resulting transition-specific survival curves assembled into a cohort trace — this is precisely how a multi-state partitioned-survival economic model is extrapolated in practice: fit and select a distribution **per transition**, not one global curve for the whole pathway.

> **HEOR in practice.** Decide the multi-state structure from the clinical pathway **before** reaching for `mstate` syntax. A model with "Death" reachable from every intermediate state is almost always right for chronic or infectious disease pathways (nobody is protected from dying by being mid-treatment) — map every edge in `transMat()` against a clinician-validated pathway diagram, not just the transitions that happen to appear in the trial dataset.

10.11 Reconstructing individual patient data from published Kaplan-Meier curves

Submitted HTA evidence frequently gives you a published trial's Kaplan-Meier **curve** as a figure, with a risk table beneath it, but not the underlying patient-level survival times — the original IPD may sit behind a confidentiality wall at the manufacturer, or the publication predates routine IPD sharing. Yet §10.2's parametric fitting and §10.9's extrapolation both need IPD (or something that behaves like it) as their input. The standard solution is the algorithm of Guyot, Ades, Ouwens and Welton (2012, **BMC Medical Research Methodology**), implemented in the `IPDfromKM` package,

which reconstructs an approximate patient-level dataset directly from a digitized KM curve plus its risk table.

The Guyot et al. (2012) reconstruction algorithm

The algorithm's logic is a clever inversion of how a KM curve is built in the first place:

1. **Digitize** the published curve: read off (t, S(t)) coordinate pairs at a fine grid of points along the printed curve (typically using plot-digitizing software or manual pixel-coordinate extraction).
2. Read the **numbers-at-risk table** printed beneath the curve at its labelled time points — this is the critical extra ingredient the original KM figure alone does not give you, because it anchors how many individuals are being censored versus experiencing events between risk-table intervals.
3. Use the risk-table counts together with the digitized S(t) step heights to back-solve, interval by interval, for the number of events and the number of censorings consistent with the observed drop in S(t) — effectively running the Kaplan-Meier product-limit formula in reverse.
4. Output a reconstructed dataset of individual (time, event/censor) records that, when a KM curve is refitted to them, closely reproduces the original published curve.

The reconstruction is necessarily approximate — it cannot recover information the published curve never contained (exact censoring times within an interval are allocated by an assumed-uniform rule, for instance) — but it has become the de facto accepted method in HTA submissions for enabling parametric extrapolation when only published curves are available.

Using `IPDfromKM`

```
library(IPDfromKM)

# Step 1: digitize the curve (coordinates read from the published figure,
# e.g. via a digitizer such as WebPlotDigitizer or Engauge)
preprocessed <- getpoints(
  dat     = digitized_coordinates,
  tot.events = NULL,
  trisk   = risk_table_times,
  nrisk.data = risk_table_numbers
)

# Step 2: getIPD() runs the Guyot et al. algorithm to back-calculate
# individual event/censoring times consistent with both the digitized
# curve and the risk table
reconstructed <- getIPD(
  prep    = preprocessed,
  armID   = 1,
  tot.events = NULL
)

ipd_fit <- flexsurv::flexsurvreg(
  survival::Surv(time, status) ~ 1,
  data = reconstructed$IPD,
  dist = "weibull"
)
```

Exact argument names vary slightly by `IPDfromKM` version — consult `?getpoints` and `?getIPD` in your installed version before relying on a specific call signature; the purpose and overall two-step workflow (digitize-and-anchor, then back-calculate) is stable across versions.

Common pitfall. Reconstructed IPD is a statistical approximation layered on top of an already-approximate digitization step. Two failure modes recur in practice: (1) mis-reading the risk-table time points so they don't align with the digitized curve's x-axis grid, which silently corrupts the back-calculated event counts for every later interval; and (2) over-trusting the reconstructed tail — KM curves are read least precisely in their sparse, high-censoring right tail, which is exactly the region §10.9's extrapolation choice is most sensitive to. Always refit a KM curve to the reconstructed IPD and visually overlay it on the original published figure before using the reconstruction for anything downstream.

HEOR in practice. Treat IPD reconstruction as a documented, auditable step in its own right, not an invisible preprocessing detail. Record the digitizing software, the coordinate-extraction screenshots, and the risk-table transcription alongside the reconstructed dataset, because a NICE committee or external assessment group reviewing your submission will often want to independently verify that the reconstruction matches the original publication's curve.

10.12 Reporting extrapolation following NICE DSU TSD 14 and 21

Agency perspective. Everything built across this chapter — the parametric family in §10.2, the Royston-Parmar spline models of §10.3, the cure models of §10.4, the statistical-and-clinical-plausibility selection logic of §10.6, and the extrapolation mechanics of §10.9 — exists because two specific NICE Decision Support Unit Technical Support Documents spell out what UK (and, by wide adoption, international) HTA bodies expect to see when a submission extrapolates trial-observed survival beyond the data. Chapter 2 of this book flagged that this chapter would need to cite them properly; here is the substance, not just the citation.

NICE DSU TSD 14 — "Survival Analysis for Economic Evaluations Alongside Clinical Trials: Extrapolation with Patient-Level Data" (Latimer, 2011, updated 2013) is the foundational document. In plain terms, it asks for four things:

- Fit the standard parametric survival distributions — exponential, Weibull, Gompertz, log-logistic, log-normal, and the generalised gamma — exactly the family fitted with `flexsurvreg()` in §10.2. TSD 14 is the reason this particular set of six distributions is the default candidate list in essentially every UK submission.
- **Test the proportional hazards assumption** before committing to a PH-based extrapolation structure, because an extrapolation built on an unjustified PH assumption carries that error across the whole modelled time horizon.
- Use a visual and statistical model-selection toolkit together: inspect hazard and log-cumulative-hazard plots against each candidate distribution's theoretical shape, alongside AIC/BIC — the same diagnostics §10.6 walked through in code.
- Justify the final choice on statistical fit **and** clinical plausibility together, which is the central principle TSD 14 states and §10.6 of this chapter echoes in applied form: a distribution

that fits the observed Kaplan-Meier curve best is not automatically the right choice if its extrapolated tail implies a hazard shape (e.g. a declining hazard of death many years out) that no clinician would endorse for the disease in question.

NICE DSU TSD 21 — "Flexible Methods for Survival Analysis" (Rutherford, Lambert, et al.) is the newer document. It exists because TSD 14's standard six-distribution family can be too restrictive for real trial data — proportional-hazards or fixed-shape-hazard assumptions often fail, especially with modern trials showing treatment-effect waning, delayed separation of arms, or crossing hazards. TSD 21 extends TSD 14's logic in two ways:

- It covers **flexible spline-based models** — specifically Royston-Parmar models (§10.3's `flexsurvspline()`) — which relax the fixed-hazard-shape constraint of the standard family by modelling the log-cumulative-hazard as a spline function of log time, with the number and placement of knots as explicit, reportable choices.
- It covers **cure models** (§10.4's mixture and non-mixture cure fits), for disease areas where a meaningful fraction of the treated population may be effectively cured and face only background mortality thereafter — a structural assumption none of TSD 14's six distributions can represent, since none of them has a "cured fraction" parameter.
- TSD 21 is also explicit that flexible methods trade the standard family's extrapolation parsimony for better in-sample fit, and that this trade-off itself needs reporting: a spline model with many knots can fit the observed KM curve closely while behaving erratically once extrapolated past the data, so TSD 21 asks for the extrapolated hazard function to be shown, not just the extrapolated survival curve.

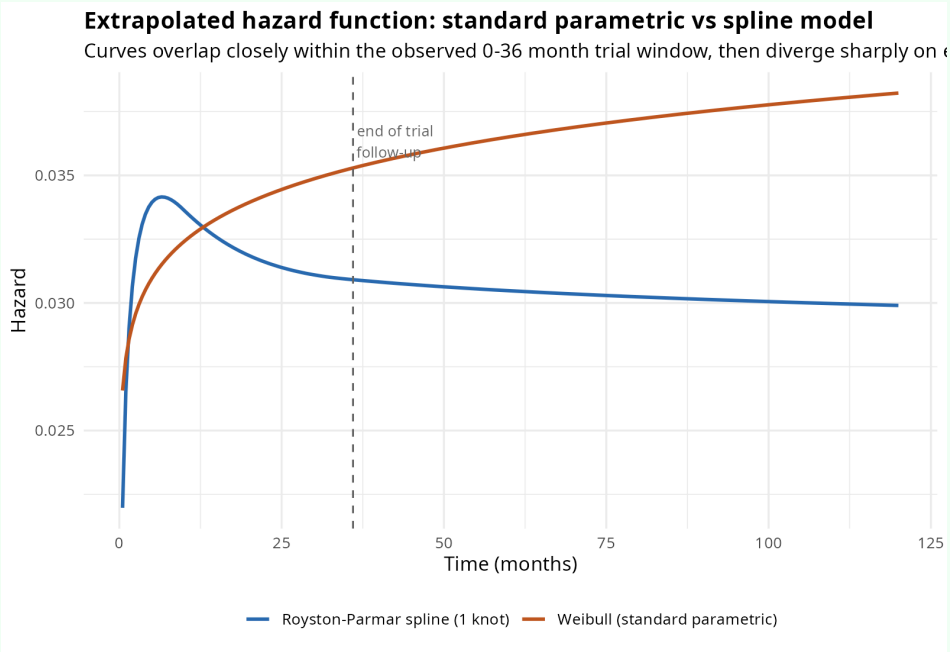


Figure 14: The extrapolated hazard function for a standard Weibull fit versus a one-knot Royston-Parmar spline, fitted to the same simulated trial data. The two curves track closely within the observed 0-36 month follow-up window but diverge sharply once extrapolated — exactly the TSD 21 diagnostic that an extrapolated survival-scale plot alone would hide.

Put simply, a submission report built from this chapter's methods should contain: the candidate distribution set and a reason for including or excluding flexible/cure-model alter-

natives; the proportional-hazards diagnostic; visual fit plots (KM overlay, hazard plot) for every candidate; an AIC/BIC table; a clinical-plausibility narrative for the selected model's extrapolated tail, ideally benchmarked against external data (population life tables, disease registries); and, where a spline or cure model is used, explicit justification for the knot count/placement or the cure-fraction assumption. A reviewer who knows TSD 14/21 will look for each of these elements by name — fitting six distributions, reporting AIC, and picking the lowest AIC without a clinical-plausibility and extrapolated-hazard discussion will not satisfy either document's stated expectations, however clean the R code behind it was.

Common pitfall. TSD 14 and TSD 21 are frequently cited in submissions as a bibliography entry without the actual diagnostics being reported — an AIC table alone does not constitute compliance with either document. Reviewers for NICE and other HTA bodies increasingly ask explicitly for the proportional-hazards check and the extrapolated-hazard-function plot (not just the survival curve), because these are the two diagnostics TSD 14 and TSD 21 respectively treat as mandatory, not optional, components of a defensible extrapolation.

Chapter 11. Meta-Analysis

Learning objectives

- Fit fixed-effect and random-effects meta-analysis models with `meta` (`metabin()`, `metacont()`, `metagen()`) and `metafor` (`rma()`) (§11.1)
- Quantify and interpret between-study heterogeneity using Q, τ^2, and I^2, and read a forest plot (§11.2)
- Screen for publication bias and small-study effects with funnel plots, Egger's test, and trim-and-fill (§11.3)
- Explain heterogeneity with meta-regression using `metafor::rma()`'s `mods=` argument, and recognise its limits (§11.4)
- Pool hazard ratios on the log scale for time-to-event outcomes, and connect the result to Chapter 10's extrapolation machinery (§11.5)
- Fit a bivariate random-effects model for diagnostic test accuracy with `mada::reitsma()` and read an SROC curve (§11.6)
- Pool single-group proportions and rates with variance-stabilising transformations (logit, double-arcsine, log) (§11.7)

Chapter 10 pooled nothing: it took one study's individual patient data and built one extrapolated survival curve from it. This chapter is about a different, very common situation — you have **several separate studies**, each already analysed on its own, and you need one combined answer. Here is the roadmap. §11.1 covers the first choice you make: fixed-effect or random-effects pooling. §11.2 and §11.3 cover two checks every pooled result needs before you trust it — how much the studies disagree (heterogeneity) and whether missing studies might be skewing the picture (publication bias). §11.4 asks **why** studies disagree. §11.5, §11.6 and §11.7 then apply all of this to three outcome types you will meet often in TB-programme work: time-to-event hazard ratios (which feed straight back into Chapter 10's extrapolation), diagnostic test accuracy, and single-group prevalence or incidence rates. Read the sections in order — each one assumes the last.

11.1 Fixed-effect and random-effects models

When a TB programme review pools results from, say, eight observational studies on treatment success rates under different regimens, the first decision is not which R function to call but which question you are asking. A **fixed-effect** (sometimes written "common-effect") model assumes every study is estimating the same true effect, and any difference between study estimates is pure sampling noise. A **random-effects** model assumes each study estimates its own true effect, drawn from a distribution of true effects across settings, and that distribution's spread is itself something you need to estimate. For TB and most infectious-disease health-economics work — where regimens, population risk, diagnostic pathways, and health-system capacity vary by country and era — the random-effects assumption is almost always the defensible default. Treat the fixed-effect model as a special case you report alongside the random-effects estimate for comparison, not as your primary analysis, unless you have strong a priori reasons (e.g. multi-site arms of a single trial with a shared protocol) to believe a common true effect.

R gives you two complementary toolkits. The `meta` package wraps the common effect-measure functions you will use most often in applied synthesis work: `metagen()` for generic (already-computed) effect sizes and standard errors, `metabin()` for binary outcomes (risk ratios, odds ratios, risk differences from 2x2 tables), and `metacont()` for continuous outcomes (mean differences or standardized mean differences from group means, SDs, and sample sizes). All three accept a `method.tau` argument that selects how between-study variance (τ^2) is estimated, and a `sm` argument for the summary measure (e.g. `"RR"`, `"OR"`, `"MD"`, `"SMD"`).

```
library(meta)

# Binary outcome: treatment success, event counts per arm
m_bin <- metabin(
  event.e = events_treatment, n.e = n_treatment,
  event.c = events_control,  n.c = n_control,
  data = tb_studies,
  sm = "RR",
  method = "MH",         # Mantel-Haenszel for the fixed-effect pooling step
  method.tau = "REML"    # tau^2 estimator for the random-effects layer
)
summary(m_bin)
forest(m_bin)
```

> **Difficult terminology: Mantel-Haenszel (MH) method.** The Mantel-Haenszel method is one of the oldest and simplest ways to combine 2x2 tables (treatment vs. control, event vs. no-event) from several studies into a single pooled risk ratio or odds ratio. Instead of averaging each study's own calculated ratio, it works directly from the raw event counts: it weights each study by how many events it contributed (roughly, its statistical "information"), so a study with 200 events pulls the pooled estimate much harder than one with 5 events. Think of it like calculating an overall batting average across several cricket seasons by adding up all runs and all innings first, rather than averaging each season's separate average — it handles small numbers of events more gracefully than methods that first calculate a ratio per study and then average those ratios. It is specifically a **fixed-effect** pooling method (see the surrounding text): it assumes every study estimates the same single true effect, so it is usually paired with a separate random-effects layer (here, `method.tau`) when that assumption is implausible.

The `metafor` package is lower-level and more flexible: you supply effect sizes and sampling variances (often via its own `escalc()` helper) and fit with `rma()`, specifying `method = "FE"` for fixed-effect or one of several random-effects estimators such as `"DL"`, `"REML"`, `"ML"`, or `"PM"` for `method`.

> **Difficult terminology: "ML" and "PM" as tau-squared estimators.** Here, "ML" stands for **maximum-likelihood** estimation of τ^2 (the between-study variance) — one of several numerical recipes for figuring out, from the pattern of disagreement across studies, how much genuine study-to-study variation to assume. "PM" is the **Paule-Mandel** estimator, a different, iterative recipe for the same quantity that is often recommended as a competitor to REML when the number of studies is small. **Disambiguation worth flagging explicitly**: this "ML" has nothing to do with Machine Learning, which is what "ML" means everywhere else in this book (see the abbreviations list) — here it is purely a statistical-estimation method for one

variance parameter inside a meta-analysis, chosen via the `method` or `method.tau` argument, not a predictive algorithm. When you see `method = "ML"` inside `rma()`, read it as "maximum-likelihood τ^2 estimator," and nothing more.

```
library(metafor)

dat <- escalc(measure = "RR",
              ai = events_treatment, n1i = n_treatment,
              ci = events_control,  n2i = n_control,
              data = tb_studies)

fit_dl   <- rma(yi, vi, data = dat, method = "DL")    # DerSimonian-Laird
fit_reml <- rma(yi, vi, data = dat, method = "REML")  # restricted max. likelihood
summary(fit_reml)
```

The choice of τ^2 estimator matters more than most analysts assume. DerSimonian-Laird (DL) is a moment-based estimator: fast, closed-form, and historically the default in most software — but it is known to understate τ^2, and hence understate the width of the pooled confidence interval, when the number of studies is small or effect sizes are binary with rare events. REML (restricted maximum likelihood) is iterative, slightly more computationally demanding, and generally preferred in current methodological guidance because it produces less biased variance estimates, particularly with few studies — a near-universal situation in TB programme meta-analyses, which rarely pool more than ten to fifteen studies. Where you have the choice, report REML as the primary random-effects estimate and DL as a sensitivity check, not the reverse.

HEOR in practice. When deciding fixed vs. random effects, apply the same model-selection discipline you used for parametric survival curves in Chapter 10: don't pick the model that gives the result you expected. Instead, triangulate on (a) clinical/programmatic plausibility — do you genuinely believe every included study shares one true effect, or do differing TB prevalence, drug-resistance patterns, and health-system settings make that implausible on its face; (b) a visual check of the forest plot for overlapping confidence intervals; and (c) the heterogeneity statistics covered in §11.2. Pre-register this choice in your analysis plan before you see the pooled numbers, exactly as you would pre-specify a survival distribution family before fitting.

Agency perspective. NICE DSU Technical Support Document 1: "Introduction to Evidence Synthesis for Decision Making" (Dias et al., 2011) is the methods document that most HTA reviewers will hold your meta-analysis against before it even reaches a decision model. It sets out exactly the random-effects-as-default position taken above: TSD 1 states that a fixed-effect model should only be used when there is a specific, justified reason to believe all studies share a single true effect, and that this justification must be stated explicitly in the submission, not assumed by default. Writing "a random-effects model was used because a fixed effect was not clinically plausible across studies conducted in different settings and years" in your methods section is the kind of sentence TSD 1 is checking for.

Common pitfall. Running both `metabin()`/`metagen()` and `rma()` on the same dataset and reporting whichever gives a narrower confidence interval or a "significant" pooled effect is p-hacking by model selection, even if each individual call is statistically correct. Decide your effect model (fixed vs. random) and your τ^2 estimator **before** inspecting results, and if you must report sensitivity analyses across estimators, label them explicitly as sensitivity analyses in your methods section, not as alternative headline results.

11.2 Heterogeneity and its quantification

Heterogeneity is simply the technical word for "the studies don't agree with each other more than sampling noise alone would explain." Quantifying it matters for two practical reasons: it tells you whether a single pooled number is even a sensible summary of your evidence base, and it determines how much extra uncertainty the random-effects model should add to your confidence interval before that number reaches a budget-impact model or a costing decision.

Three statistics do the work, and each answers a slightly different question. Q is a weighted sum of how far each study's effect sits from the pooled effect. If there were truly no heterogeneity, Q would follow a known chi-squared distribution — so a large Q relative to its degrees of freedom ($k - 1$, where k is the number of studies) signals heterogeneity. Q's weakness: with few studies (common in TB settings) it has low power to detect real heterogeneity, and with many studies it is almost always "significant" even when the true heterogeneity is clinically trivial. Don't read its p-value in isolation. τ^2 is the estimated **variance** of the true effects across studies, on the same scale as your effect measure (log-RR, mean difference, SMD). It is the number random-effects models actually use to widen standard errors, but its raw value is hard to interpret because that scale changes from analysis to analysis. I^2 solves that readability problem: it rescales the same information into a percentage — the share of total variability that is real between-study disagreement rather than sampling noise. That is why I^2 is the number most often quoted.

Difficult terminology: chi-squared distribution. The chi-squared distribution is a standard, well-tabulated probability distribution that describes how much random "noise" alone would produce, if you added up a bunch of squared deviations that each had no real pattern behind them. It comes up constantly as the reference yardstick for "is this amount of spread bigger than chance would explain" tests. Here, it's the yardstick for the Q-statistic: if every study were truly estimating the same effect and all the disagreement you see between studies were just the play of random sampling, Q would behave like a draw from a chi-squared distribution with $k - 1$ degrees of freedom. If the actual Q you calculated is bigger than what that distribution would typically produce, that's evidence the studies disagree by more than chance alone — i.e., genuine heterogeneity. It's the same logic as a chi-squared test comparing observed versus expected counts in a 2x2 table, just applied to study effects instead of cell counts.

A widely used rule of thumb, drawn from the Cochrane Handbook's guidance, treats I^2 values roughly as: 0-40% might not be important; 30-60% may represent moderate heterogeneity; 50-90% may represent substantial heterogeneity; 75-100% considerable heterogeneity — with the explicit caveat that these bands overlap deliberately, because the importance of any given I^2 depends on

the magnitude and direction of effects and the strength of evidence for heterogeneity (i.e. look at Q's p-value and the confidence interval around I^2 too, not the point estimate alone).

Both `meta` and `metafor` report all three statistics by default when you call `summary()` on a fitted object, and both provide forest plots as the standard visual diagnostic:

```
summary(m_bin)     # reports Q, tau^2, I^2, and their CIs alongside the pooled
estimate
forest(m_bin,
       sortvar = TE,          # order studies by effect size
       prediction = TRUE)     # adds a prediction interval for a new study's true
effect

# metafor equivalent
summary(fit_reml)
forest(fit_reml)
```

> **Difficult terminology: prediction interval (vs. confidence interval).** A confidence interval tells you how precisely you've pinned down the **average** true effect, given the studies you pooled — it narrows as you add more studies, because you're estimating one fixed number ever more precisely. A prediction interval answers a different, often more clinically relevant question: "if a brand-new study, not yet run, estimated this same effect, what range would its true effect plausibly fall in?" Because individual studies' true effects genuinely vary (that's the heterogeneity this section is about), a prediction interval has to be wider than the confidence interval — it accounts for both the uncertainty in the average and the real study-to-study spread around that average. Concretely: if a meta-analysis's pooled risk ratio is 0.70 with a confidence interval of 0.65-0.75, the corresponding prediction interval might be 0.40-1.20 — the average effect is well pinned down, but any one new setting (a new country, a new TB programme) could plausibly see anywhere in that wider range. Quoting only the confidence interval when heterogeneity is substantial can make the evidence look far more consistent and generalisable than it really is.

The forest plot remains the single most useful diagnostic in this chapter: each row is one study's point estimate and confidence interval, sized by weight in the pooled analysis, with the diamond at the bottom showing the pooled estimate and its interval. Read it before you read any summary statistic — a cluster of overlapping intervals with one clear outlier tells a different, more actionable story than a diffuse spread of non-overlapping intervals, even when both produce a similar I^2.

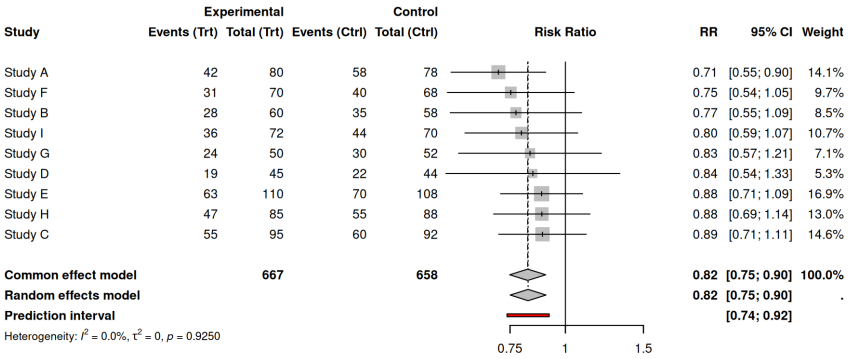


Figure 15: A forest plot from `meta::forest()` on a nine-study toy risk-ratio meta-analysis: one row per study with its weight-sized confidence interval, the pooled random-effects diamond, and a prediction interval for a new study's true effect — the single most important figure this chapter produces.

Agency perspective. The Cochrane Handbook for Systematic Reviews of Interventions, Chapter 10 ("Analysing data and undertaking meta-analyses"), is the standard methodological reference underpinning essentially every regulatory or HTA submission's heterogeneity reporting: it specifies that I^2, τ^2, and the Q-statistic (with its p-value) should all be reported together, not I^2 alone, and that a prediction interval should accompany the pooled estimate whenever random-effects heterogeneity is non-trivial. If your meta-analysis is destined for an HTA dossier, report in that triad rather than cherry-picking the one statistic that looks most reassuring.

How to report this.

Study	n (T/C)	RR [95% CI]	Weight (%)
Site 1	120/118	0.82 [0.61, 1.10]	14.2
Site 2	95/101	0.74 [0.52, 1.05]	11.6
Site 3	210/205	0.91 [0.73, 1.13]	18.9
Site 4	88/90	0.65 [0.44, 0.96]	9.8
Site 5	150/147	0.88 [0.68, 1.14]	15.7
…	…	…	…
Pooled (random-effects, REML)		0.83 [0.72, 0.96]	100
Heterogeneity: $Q = 12.4$ (df = 8, $p = 0.13$); $\tau^2 = 0.015$; $I^2 = 35.4\%$ (95% CI: 0, 68%)			

Table 30: How to report this: the per-study and pooled effect sizes that accompany a forest plot, together with the heterogeneity statistics (Q, τ^2, I^2) that must be quoted alongside the pooled estimate in an HTA submission

11.3 Publication bias and small-study effects

Publication bias is the concern that studies with "positive," statistically significant, or programmatically convenient findings are more likely to be published, and hence more likely to end up in your meta-analysis, than null or unfavourable results sitting in a file drawer somewhere. Small-study effects are the closely related, more general phenomenon where smaller studies systematically report larger effect sizes than large studies do — which can arise from publication bias but also from genuine clinical heterogeneity (e.g. smaller pilot studies run in more responsive sub-populations) or from methodological shortcuts taken under smaller budgets. You cannot distinguish these causes definitively from the data alone, so treat any positive test below as a flag for further investigation, not as a diagnosis.

The funnel plot is the standard visual tool: each study's effect estimate is plotted against a measure of its precision (usually standard error, inverted so precise studies sit near the top). In the absence of bias, the scatter should look like a roughly symmetric inverted funnel, widening at the bottom where smaller, less precise studies naturally spread further from the pooled effect. Asymmetry — a gap in one bottom corner where small studies with unfavourable results "should" be but aren't — is the visual signature of concern.

```
# meta package
funnel(m_bin)
metabias(m_bin, method.bias = "linreg")   # Egger's regression test

# metafor package
funnel(fit_reml)
regtest(fit_reml, model = "lm")           # Egger's test equivalent
```

Egger's test formalizes the visual check: it regresses the standardized effect estimate against its precision (or a closely related parameterization) and tests whether the intercept differs from zero, which would indicate asymmetry. `metabias()` in `meta` exposes this (and several other bias tests) via its `method.bias` argument; `regtest()` in `metafor` is the direct analogue. Both tests share the same core limitation: they require a reasonable number of studies (conventionally at least ten) to have any meaningful power, which rules them out for most TB programme meta-analyses pooling five or six site-level studies. Running Egger's test on eight studies and reporting a non-significant p -value as "no evidence of publication bias" overstates what the test can tell you at that sample size.

Trim-and-fill, available as `trimfill()` in both `meta` and `metafor`, offers a complementary sensitivity analysis: it iteratively removes the most extreme small studies causing funnel-plot asymmetry, uses the trimmed, symmetric subset to estimate the true center, then imputes ("fills") mirror-image studies back in to restore symmetry and re-estimates the pooled effect from the augmented dataset. The adjusted estimate is not a corrected "true" answer — it is a what-if sensitivity check showing how much the pooled effect might shift if the suspected missing studies existed and looked like their mirror images. Report it alongside, never instead of, the unadjusted pooled estimate.

```
tf <- trimfill(m_bin)
summary(tf)
funnel(tf)   # visualize imputed studies alongside observed ones
```

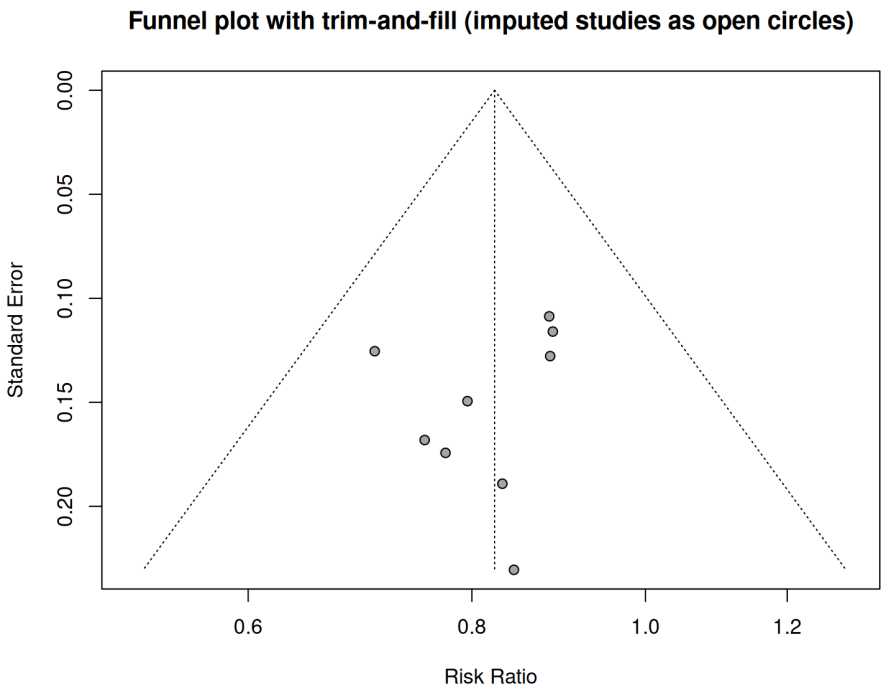


Figure 16: A funnel plot with trim-and-fill applied to the chapter's toy meta-analysis. Observed studies and any imputed mirror-image studies added by `trimfill()` to restore symmetry are both shown — the adjusted pooled estimate from this augmented dataset is a sensitivity check, not a corrected "true" answer.

Common pitfall. Do not apply Egger's test or trim-and-fill mechanically whenever you have a funnel plot available. With fewer than ten studies — the norm for TB-programme-specific

meta-analyses — both methods have poor power and can produce misleading corrections driven by one or two outlying studies rather than genuine bias. In that situation, describe the funnel plot qualitatively, flag the limitation explicitly in your write-up, and avoid reporting a trim-and-fill-adjusted point estimate as if it were more trustworthy than the original.

How to report this.

Analysis	k (studies)	Pooled RR [95% CI]	I^2
Observed studies only (random-effects, REML)	9	0.83 [0.72, 0.96]	35.4%
Trim-and-fill adjusted (3 imputed studies)	12	0.88 [0.76, 1.02]	41.7%
Egger's regression test: intercept = 1.42 (SE 0.58), $p = 0.04$, consistent with funnel-plot asymmetry; trim-and-fill shifts the pooled estimate toward the null but does not overturn it — reported as a sensitivity check, not a corrected estimate.			

Table 31: How to report this: sensitivity of the pooled random-effects estimate to the trim-and-fill adjustment for funnel-plot asymmetry, shown before (observed studies only) and after (observed plus imputed mirror-image studies) alongside Egger's test for small-study effects

11.4 Meta-regression

When a forest plot shows wide variation in effect sizes across studies, the natural next question is **why**. Meta-regression extends the random-effects model by adding one or more study-level predictors ("moderators") that might explain some of that between-study heterogeneity — things like mean patient age, baseline disease severity, year of publication, or dosing regimen. It is the meta-analytic cousin of ordinary regression, except the unit of analysis is the study, not the patient, and each study's contribution is weighted by its precision.

In `metafor`, this is done with the same `rma()` function already familiar from §11.1, but supplying the `mods=` argument:

```
library(metafor)

# dat has yi (effect size, e.g. log-RR), vi (sampling variance),
# and a study-level moderator, mean_age
res_mod <- rma(yi, vi, mods = ~ mean_age, data = dat, method = "REML")
summary(res_mod)
```

`mods=` accepts a formula, so multiple moderators, factors, and interactions all work exactly as in `lm()`: `mods = ~ mean_age + region`. The output partitions the total heterogeneity into the portion explained by the moderator(s) (reported as a test of moderators, `QM`) and what remains (the residual heterogeneity, still summarised by `tau^2` and `I^2`, now relabelled residual heterogeneity). A moderator that soaks up a large share of `QM` relative to the overall `Q` is doing real explanatory work; one that leaves the residual heterogeneity almost untouched is not.

How to report this.

Moderator	Coefficient	SE	95% CI	p
Intercept	−0.41	0.15	(-0.70, −0.12)	0.006
Mean age (per 10 years)	0.09	0.04	(0.01, 0.17)	0.028
Region: high TB-burden (ref: low)	0.18	0.11	(-0.04, 0.40)	0.108
Test of moderators: $Q_M = 6.8$ (df = 2, $p = 0.033$); residual heterogeneity $\tau^2 = 0.009$ (down from 0.015 in the unadjusted model), residual $I^2 = 22.1\%$.				

Table 32: How to report this: meta-regression coefficients from `metafor::rma(mods = ~ mean_age + region)`, the conventional way to present which study-level moderators explain between-study heterogeneity and by how much

Meta-regression is appropriate when two conditions hold together. First, there need to be **enough studies** — as a rule of thumb, at least ten studies per moderator is the floor below which the regression coefficients become unstable and prone to chance findings; many TB programme meta-analyses sit well under this, which should temper ambition. Second, the covariates should be **pre-specified** in a protocol, not fished for after seeing which split of the forest plot looks tidiest. Testing every variable in the dataset against the effect size until something is significant is simply multiple testing in a performance-enhancing dress, and the HEOR submission will not survive scrutiny if the moderator choice cannot be justified on clinical or methodological grounds set out before the data were pooled.

Common pitfall. Meta-regression coefficients describe **between-study** associations, not **within-study** (individual-patient) associations — this is ecological bias. A meta-regression that finds "trials with older average age show smaller treatment effects" says nothing reliable about whether an individual older patient benefits less; the studies' age **averages** are confounded with everything else that differs systematically between those trials (settings, era, comorbidity mix, co-interventions). Do not present aggregate-level moderator findings as if they were patient-level subgroup effects in a submission — regulators and HTA reviewers will flag this immediately, and Cochrane guidance is explicit that such inferences require individual patient data (IPD) meta-analysis, not aggregate meta-regression.

With few studies, meta-regression also has very low statistical power — a null result for a moderator with eight trials is barely informative, and should be reported as inconclusive rather than as evidence the moderator does not matter.

11.5 Meta-analysis of time-to-event outcomes

Many of the outcomes that matter most in TB and HIV programmes — time to treatment failure, time to death, time to culture conversion — are naturally time-to-event (survival) outcomes, and trials typically report them as hazard ratios (HRs) with confidence intervals, not as simple risk ratios. Pooling HRs across studies needs a small but important adjustment: HRs are ratios, bounded

below by zero and skewed, so they are pooled on the **log scale**, where the sampling distribution is much closer to normal.

The practical recipe is: for each study, compute `log(HR)` as the effect estimate and derive its standard error from the reported confidence interval (if a 95% CI is `(L, U)`, then `SE = (log(U) - log(L)) / (2 * 1.96)`), then pool these log(HR) values with inverse-variance weighting exactly as any other continuous effect size. The `meta` package's `metagen()` function does this cleanly and labels the output in the natural HR scale automatically:

```
library(meta)

# dat has studlab, TE (= log HR per study), seTE (SE of log HR)
m_hr <- metagen(
  TE = TE,
  seTE = seTE,
  studlab = studlab,
  data = dat,
  sm = "HR",          # tells meta the pooled estimate is a hazard ratio
  method.tau = "REML"
)
summary(m_hr)
forest(m_hr)
```

Setting `sm = "HR"` is purely cosmetic in terms of the arithmetic (the pooling still happens on the log scale you supplied), but it tells `metagen()` to back-transform the summary estimate and its CI with `exp()` before printing, so the forest plot and output table report a pooled HR with sensible axis labelling rather than a pooled log(HR) that needs manual conversion. If your source studies instead report `log(HR)` and its SE directly (common in trial supplements), feed those straight in; if they only give you the HR point estimate and CI, derive `TE` and `seTE` from them first as above.

This pooled HR is not the end of the analysis in an HEOR context — it is usually an **input** to the next stage. Chapter 10 built the survival-model toolkit: fitting the `flexsurv` six-distribution family to a reference arm's individual patient-level survival data, selecting a parametric form by AIC/BIC together with visual fit and clinical plausibility, and extrapolating the fitted curve beyond the trial's observed follow-up. A pooled HR from meta-analysis commonly becomes the multiplier applied to that extrapolated reference-arm survival curve to generate the comparator arm's extrapolated survival — the "HR method" of indirect extrapolation used throughout cost-effectiveness modelling. The meta-analysis in this section therefore feeds directly into the machinery of the previous chapter: the pooled `log(HR)` and its uncertainty become a parameter (often itself distributed, e.g. log-normal, in a probabilistic sensitivity analysis) multiplying the hazard function of whichever `flexsurv` distribution was selected for the reference arm in Chapter 10.

> **Agency perspective.** Before pooling HRs across trials for use in an extrapolation, check the Cochrane Handbook for Systematic Reviews of Interventions, Chapter 6 (on choosing effect measures) for its guidance on extracting hazard ratios consistently across studies that report time-to-event results in different formats (Kaplan-Meier curves only, HR with CI, HR with p-value only, or number of events and person-time). The Handbook sets out a hierarchy of extraction methods — a directly reported HR and SE being preferable to an HR reconstructed from a digitised KM curve via the Guyot et al. (2012) algorithm already introduced in Chapter

10 §10.11 — so do not silently treat a reconstructed HR as equivalent in precision to a directly reported one when assembling the pooled estimate that will drive an extrapolation.

11.6 Diagnostic test accuracy meta-analysis

Pooling sensitivity and specificity separately is tempting but wrong, because the two measures are not independent: a laboratory or study that reads a diagnostic threshold more liberally will tend to report higher sensitivity and lower specificity together, and vice versa. Any meta-analysis of diagnostic accuracy — highly relevant to TB programmes evaluating a new rapid test (Xpert, LF-LAM, a novel biomarker assay) against a reference standard like culture — needs a model that respects this negative correlation rather than pooling sensitivity and specificity as if they were two unrelated outcomes.

The standard solution is the bivariate random-effects model, which jointly models the logit-transformed sensitivity and logit-transformed specificity across studies, allowing for a between-study correlation between the two. In R, the dedicated package for this is `mada` ("meta-analysis of diagnostic accuracy"), and its core function is `reitsma()`:

```
library(mada)

# dat has TP, FN, FP, TN counts for each study against the reference standard
fit <- reitsma(dat)        # dat needs columns named TP, FN, FP, TN
summary(fit)

# descriptive diagnostic stats (sens, spec, likelihood ratios) per study
desc <- madad(dat)
print(desc)

# SROC curve with confidence and prediction regions
plot(fit, sroc.type = "ruttergates")
points(fpr(desc), sens(desc), pch = 2)
```

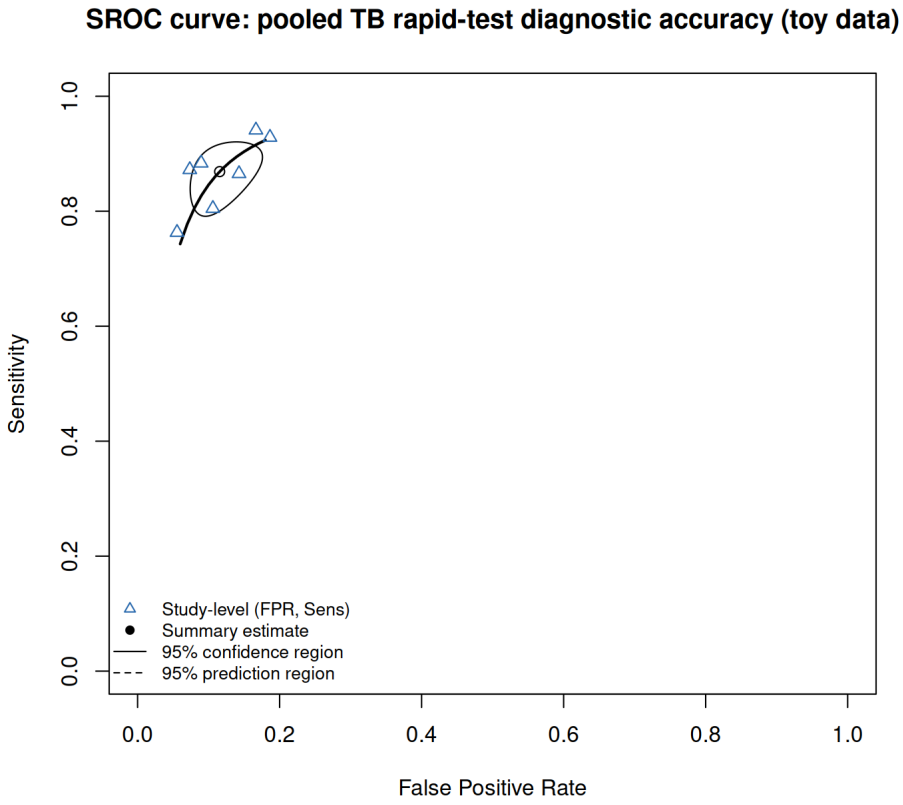


Figure 17: An SROC curve from `mada::reitsma()` fitted to a toy TB rapid-diagnostic-test dataset: study-level sensitivity/false-positive-rate pairs (triangles), the pooled summary point, and the 95% confidence and prediction regions — the bivariate analogue of the pairwise forest plot.

`reitsma()` fits the bivariate model (a generalised linear mixed model under the hood, modelling the logit-sensitivity/logit-specificity pair with a bivariate normal random effect) and returns pooled sensitivity and specificity with their joint uncertainty. The SROC curve (summary receiver operating characteristic curve) traces out the implied trade-off between sensitivity and specificity across the range of thresholds/studies, and is the standard visual summary for a diagnostic accuracy review — it plays a role analogous to the forest plot for pairwise meta-analysis, but captures the two correlated axes at once. `metafor::rma.mv()` can also fit an equivalent bivariate model by hand (specifying the sensitivity/specificity logits as a bivariate outcome with a random study effect), which gives more flexibility for complex covariate structures, but `mada::reitsma()` is the more direct and purpose-built route for a standard DTA review.

Difficult terminology: generalised linear mixed model. A "mixed model" is a regression that mixes two kinds of effects: **fixed** effects, which are shared, overall parameters you want to estimate once for everybody (e.g. "the average sensitivity across all studies"), and **random** effects, which let each individual study (or patient, or site) wobble around that average in its own way, with the size of that wobble itself estimated from the data. "Generalised" means the outcome doesn't have to be a plain continuous, normally-distributed number — here it is a logit-transformed proportion (sensitivity/specificity), so the model works on that transformed, bounded scale, the same trick as the logit link used elsewhere in the book. Picture it as: every

study gets to have its own slightly different "true" sensitivity and specificity (the random part), but all of those study-specific values are assumed to be draws clustered around one shared underlying pair of numbers (the fixed part) — that's what lets `reitsma()` borrow strength across studies while still respecting that no two labs read a diagnostic threshold identically.

Agency perspective. Diagnostic test accuracy reviews have their own reporting standard, the PRISMA-DTA extension (Preferred Reporting Items for Systematic Reviews and Meta-Analyses of Diagnostic Test Accuracy studies), published in **JAMA** in 2018. Before finalising a DTA meta-analysis for a TB diagnostics submission, check the PRISMA-DTA checklist for required elements — notably the explicit reporting of the reference standard used in each included study, handling of indeterminate results, and threshold variability across studies — since these affect whether pooling sensitivity/specificity across studies is even defensible in the first place.

How to report this.

Parameter	Estimate	95% CI (low)	95% CI (high)
Pooled sensitivity	0.89	0.82	0.94
Pooled specificity	0.91	0.85	0.95
Positive likelihood ratio	9.9	5.8	16.9
Negative likelihood ratio	0.12	0.07	0.20
Diagnostic odds ratio	82.5	34.1	199.6
Bivariate model: 11 studies, correlation between logit-sensitivity and logit-specificity $\rho = -0.42$			

Table 33: How to report this: pooled sensitivity and specificity from the bivariate `mada::reitsma()` model, the standard summary table accompanying an SROC curve in a diagnostic test accuracy review

Difficult terminology: positive/negative likelihood ratio. These two numbers translate a test's sensitivity and specificity into something more directly useful at the bedside: how much a result should shift your suspicion of disease. The **positive** likelihood ratio (here, 9.9) says a positive test result is about 9.9 times more likely to occur in someone who truly has the disease than in someone who doesn't — the higher above 1, the more a positive result should raise your suspicion. The **negative** likelihood ratio (here, 0.12) says a negative result is still fairly likely in diseased patients relative to non-diseased ones when it's far from 0 — the lower below 1 (closer to 0), the more a negative result should reassure you. Unlike sensitivity/specificity, likelihood ratios can be combined directly with a patient's pre-test odds of disease (via Bayes' theorem) to get a post-test odds, which is why clinicians favour them for bedside reasoning about an individual patient, rather than sensitivity/specificity alone.

Difficult terminology: diagnostic odds ratio. The diagnostic odds ratio folds sensitivity and specificity into one single summary number: the odds of a positive test result in diseased patients, divided by the odds of a positive test result in non-diseased patients. A value of 1 means the test is useless (positive results are just as likely whether or not someone has the disease); the TB rapid test's value of 82.5 here means a positive result is, in odds terms, about 82 times more likely among people who truly have TB than among people who don't — a reasonably strong-performing test. It's convenient for ranking or comparing tests with one number, but because it blends sensitivity and specificity together, two tests with very different clinical trade-offs (one highly sensitive but less specific, one the reverse) can end up with a similar diagnostic odds ratio — so it should supplement, not replace, looking at sensitivity, specificity, and the likelihood ratios separately.

HEOR in practice. When several candidate rapid tests are being compared against the same reference standard, consider whether a network meta-analysis of diagnostic accuracy is more appropriate than separate pairwise bivariate models — this lets indirect comparisons borrow strength across the evidence network, which matters when head-to-head trials between two rapid tests are scarce, as is common in TB diagnostics. Chapter 12 develops network methods in full.

11.7 Meta-analysis of proportions and rates

A large share of the meta-analyses a TB programme analyst will actually run are not comparative at all — there is no treatment arm versus control arm, just a single proportion (prevalence of drug resistance, treatment success rate, loss-to-follow-up rate) or a single rate (incidence per person-year) estimated in multiple studies or multiple programme sites, which need to be pooled into one overall estimate. This is meta-analysis of a **single group** summary statistic, and it needs its own variance-stabilising transformations because raw proportions and rates have variances that depend on their own mean, violating the constant-variance assumption behind simple inverse-variance pooling.

For proportions bounded between 0 and 1 (e.g. proportion of TB patients with a positive resistance test), two transformations are standard. The logit transformation maps the proportion to the whole real line and works well away from the boundaries:

```
library(metafor)

# dat has xi (events, e.g. resistant cases) and ni (total tested) per study
dat_logit <- escalc(measure = "PLO", xi = xi, ni = ni, data = dat)
res_logit <- rma(yi, vi, data = dat_logit, method = "REML")
predict(res_logit, transf = transf.ilogit)   # back-transform to a proportion
```

When some studies have proportions near 0 or 1 (common with rare events, such as a resistance pattern seen in only a handful of patients), the logit transform can behave poorly, and the double-arcsine (Freeman-Tukey) transformation is the usual alternative, handled in `metafor::escalc()` with `measure = "PFT"`:

```
dat_pft <- escalc(measure = "PFT", xi = xi, ni = ni, data = dat)
res_pft <- rma(yi, vi, data = dat_pft, method = "REML")
predict(res_pft, transf = transf.ipft.hm, ni = dat_pft$ni)  # back-transform
```

For rates (events per unit person-time, e.g. TB incidence per 100,000 person-years), the log transformation is the natural equivalent, available via `measure = "IRLN"` (incidence rate, log-transformed):

```
# dat has xi (events) and ti (total person-time) per study/site
dat_rate <- escalc(measure = "IRLN", xi = xi, ti = ti, data = dat)
res_rate <- rma(yi, vi, data = dat_rate, method = "REML")
predict(res_rate, transf = exp)   # back-transform to events per person-time unit
```

In every case the pattern is the same: transform each study's proportion or rate onto a scale where a normal approximation is reasonable, pool on that transformed scale with the usual random-effects machinery (`rma()`), then back-transform the pooled estimate and its confidence interval with `predict(..., transf = ...)` for reporting. Never average the raw proportions or rates directly and never report the pooled estimate still on the transformed scale — logit or arcsine units mean nothing to a programme manager reading the report.

Common pitfall. High heterogeneity (I^2 often exceeding 90%) is the norm rather than the exception when pooling prevalence or incidence across sites, because true prevalence genuinely differs by geography, population risk, and surveillance intensity — this is not necessarily a sign of a broken model. Resist the temptation to report a single pooled prevalence figure as if it applied uniformly to a national TB programme when the underlying studies are clearly measuring different underlying populations; a forest plot showing the spread, alongside the pooled estimate, communicates the true picture far better than the summary number alone, and subgroup pooling by region or risk group is usually more decision-relevant than one overall figure.

Agency perspective. CHEERS 2022 (Husereau et al., **Value in Health**, 2022), the reporting checklist already established in Chapter 9, item 15 requires that any pooled clinical input to a cost-effectiveness model — including a pooled prevalence or rate feeding a Markov transition probability in a later chapter — state the synthesis method, the heterogeneity statistics, and the handling of any ties to the decision model explicitly. A pooled proportion quoted in a later chapter's Markov model without a visible trail back to this section's method and heterogeneity reporting does not meet that bar.

Chapter 12. Indirect Treatment Comparisons and Network Meta-Analysis

Chapter 11 closed with pooling evidence across several trials of the **same** two treatments. This chapter opens a harder, far more common HTA problem: what happens when no trial has ever compared the two treatments the reimbursement decision actually hinges on? The answer, across every method in this chapter — Bucher's two-trial algebra, frequentist network meta-analysis, Bayesian network meta-analysis, consistency checking, network meta-regression, and survival-outcome networks — is the same single idea at increasing scale: evidence can be combined **across** trials only through a shared comparator, never by comparing absolute outcomes directly.

12.1 Why indirect comparison is needed

Chapter 11 built the machinery for combining multiple head-to-head trials of the **same** two treatments into one pooled estimate — fixed-effect and random-effects pooling, I^2 and τ^2 as the heterogeneity vocabulary, REML as the default τ^2 estimator. That machinery assumes something that, in HTA submissions, is rarely true: that every trial relevant to your decision problem compared the same two arms.

In reality, a reimbursement dossier for a new TB regimen almost never has a trial that pairs the new regimen directly against every comparator a health technology assessment (HTA) body cares about. A manufacturer trial might show Regimen A (a novel bedaquiline-based combination) against the WHO-recommended standard-of-care regimen C in a multidrug-resistant TB population. A separate, unrelated trial — run by a different sponsor, in a different country, years apart — might show Regimen B (a delamanid-based combination) against that same standard-of-care regimen C. Nobody has ever randomised A against B. Yet the reimbursement question on the table is precisely: **how does A compare to B?** — because B is the comparator already funded in the health system, or the one the HTA body has designated as the relevant comparator in its final scope.

This is the indirect treatment comparison (ITC) problem, and it is endemic to HTA rather than a rare edge case. Guidelines rarely allow a manufacturer to simply report the two trials' results side by side and let a reviewer "eyeball" the difference — that approach has no defensible statistical basis and discards exactly the information that randomisation was designed to protect: comparability of patient populations within each trial. Naive, unadjusted comparison of absolute outcomes across trials (sometimes called a "naive indirect comparison") ignores the fact that trial populations, eras, and measurement methods differ, and it will produce biased and overconfident answers. What is needed is a method that uses randomised evidence **within** each trial, and only combines **between** trials through the shared control arm.

> **Common pitfall.** Never compare absolute outcomes from unrelated trials directly (e.g. "Trial 1 found 78% cure with Regimen A, Trial 2 found 71% cure with Regimen B, therefore A is better"). This "naive" or "unadjusted" indirect comparison destroys randomisation and is highly vulnerable to confounding by differences in baseline risk, trial era, region, and outcome

definition between the two trials. It is explicitly flagged as invalid practice in NICE DSU TSD 2 (Dias et al. 2011) and will not be accepted as the primary analysis in an HTA submission.

The conceptual fix — formalised by Bucher, Guyatt, Griffith and Walter (1997) and now the foundation of the entire ITC/NMA literature — is to exploit a **common comparator**. If trial 1 randomised patients to A vs C, and trial 2 (independently) randomised patients to B vs C, then each trial still gives you a valid, randomisation-protected estimate of a relative treatment effect: the A-vs-C effect from trial 1, and the B-vs-C effect from trial 2. Because both effects are expressed relative to the **same** comparator C, you can algebraically combine them to obtain an estimate of the A-vs-B effect — without ever having observed a trial that randomised A against B directly. C in this role is called the **anchor** or **common comparator**, and the approach is accordingly called an **anchored indirect comparison**.

This single idea — anchoring through a common comparator rather than comparing absolute arms — is what separates a defensible ITC from a naive one, and it is the thread that runs through this entire chapter: first in its simplest two-trial form (the Bucher method, §12.2), then generalised to networks of many trials and many treatments (frequentist network meta-analysis, §12.3), and finally in its Bayesian implementation.

Learning objectives

- Explain why within-trial randomisation must be preserved when evidence comes from more than one trial, and why naive cross-trial comparison of absolute outcomes is invalid
- Describe the anchor-based logic of the Bucher method: deriving an A-vs-B estimate from independent A-vs-C and B-vs-C trials via a common comparator C
- Calculate a Bucher indirect estimate and its variance by hand, understanding why the variances of the two component trials **add** rather than subtract
- Distinguish an indirect comparison from a mixed treatment comparison, and know when each applies
- Fit a frequentist network meta-analysis in R with the `netmeta` package, using `netmeta()`, `netgraph()`, `netrank()`, and `netleague()`
- Interpret a network geometry plot, a P-score ranking, and a league table in the context of an HTA evidence network
- Fit a Bayesian network meta-analysis with `gemtc`, run MCMC diagnostics, and check convergence before reporting any posterior summary
- Explain and run node-splitting and design-by-treatment interaction tests to assess network consistency
- Extend an NMA to a network meta-regression, and recognise the ecological-bias caveat that applies with even more force at network level
- Describe how NMA of survival outcomes generalises the Chapter 11 pooled-log(HR) mechanism across a network, and understand the limits of off-the-shelf tools for fractional polynomial time-varying hazard ratios
- Rank treatments across a network with SUCRA (Bayesian) and P-scores (frequentist), and explain why a ranking metric must always be reported alongside, never instead of, the underlying effect estimates and their uncertainty
- Report a network meta-analysis per NICE DSU TSD 2, the ISPOR good-practice task force report, and the PRISMA-NMA extension to the systematic review checklist

12.2 Bucher method

12.2.1 The algebra of anchored indirect comparison

Suppose we have two independent, two-arm randomised trials, both reporting a log hazard ratio (or log odds ratio, or mean difference — the algebra is identical on any additive scale):

- Trial 1 (A vs C): estimate $\hat{d}_{AC}$, with standard error $\mathrm{SE}\left(\hat{d}_{AC}\right)$
- Trial 2 (B vs C): estimate $\hat{d}_{BC}$, with standard error $\mathrm{SE}\left(\hat{d}_{BC}\right)$

Both $\hat{d}_{AC}$ and $\hat{d}_{BC}$ are valid, randomisation-protected estimates — each trial's internal randomisation ensures that, within that trial, the comparison of its two arms is unconfounded. The Bucher method combines them into an indirect estimate of the A-vs-B effect:

$$\hat{d}_{AB}^{\text{indirect}} = \hat{d}_{AC} - \hat{d}_{BC}$$

This is simply the statement that if A is $\hat{d}_{AC}$ better than C, and B is $\hat{d}_{BC}$ better than C, then A is $\left(\hat{d}_{AC} - \hat{d}_{BC}\right)$ better than B. The subtraction is on the **treatment-effect scale** (log HR, log OR, mean difference) — it is not a subtraction of absolute outcome levels, and it still relies on nothing but the two within-trial comparisons.

12.2.2 Why the variance adds

This is the detail most often got wrong by newcomers, and it is worth dwelling on precisely because the point estimate is a **subtraction** while the variance is a **sum**:

$$\mathrm{Var}\left(\hat{d}_{AB}^{\text{indirect}}\right) = \mathrm{Var}\left(\hat{d}_{AC}\right) + \mathrm{Var}\left(\hat{d}_{BC}\right)$$

or equivalently, in standard-error form:

$$\mathrm{SE}\left(\hat{d}_{AB}^{\text{indirect}}\right) = \sqrt{\mathrm{SE}\left(\hat{d}_{AC}\right)^2 + \mathrm{SE}\left(\hat{d}_{BC}\right)^2}$$

The reason is that $\hat{d}_{AC}$ and $\hat{d}_{BC}$ come from two **statistically independent** trials — different patients, different randomisation, no shared data. For independent random variables, the variance of their difference is the sum of their individual variances, exactly as it is for their sum: $\mathrm{Var}(X - Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$ when X and Y are independent, regardless of the sign in front of Y. The minus sign affects the **expectation** of the combination (point estimate) but not its **variance**, because $\mathrm{Var}(-Y) = \mathrm{Var}(Y)$.

> **Common pitfall.** The single most common manual error in Bucher calculations is subtracting the two variances (or SEs) by analogy with the point-estimate subtraction, producing an impossibly narrow — sometimes negative — variance. The correct operation is ALWAYS addition of variances, never subtraction, because the two trials are independent sources of uncertainty and uncertainty does not cancel when you combine independent estimates, even when the point estimates themselves are combined by subtraction.

The practical consequence is important for interpretation: an indirect estimate is **always** less precise than either of its two component direct estimates, because you are paying the "price" of both trials' sampling uncertainty, with no opportunity for cancellation. A 95% confidence interval for the indirect A-vs-B log-effect is then

$$\hat{d}_{AB}^{\text{indirect}} \pm 1.96 \times \text{SE}\left(\hat{d}_{AB}^{\text{indirect}}\right)$$

exponentiated back to the natural scale (hazard ratio, odds ratio) if that is how the trials originally reported their effects.

12.2.3 Worked example: TB regimen comparison

Consider an HTA submission for Regimen A, a novel multidrug-resistant TB (MDR-TB) treatment, where the decision-relevant comparator is Regimen B, an alternative novel regimen already used in some health systems. No trial has randomised A against B. However:

- **Trial 1** randomised MDR-TB patients to Regimen A vs the WHO standard-of-care regimen C, reporting a log hazard ratio for treatment failure/relapse of $\hat{d}_{AC} = -0.52$ (favouring A) with $\text{SE} = 0.18$.
- **Trial 2** randomised a comparable MDR-TB population to Regimen B vs the same standard-of-care regimen C, reporting $\hat{d}_{BC} = -0.28$ (favouring B) with $\text{SE} = 0.21$.

The Bucher indirect estimate of A vs B is:

$$\hat{d}_{AB} = -0.52 - (-0.28) = -0.24$$

$$\text{SE}\left(\hat{d}_{AB}\right) = \sqrt{0.18^2 + 0.21^2} = \sqrt{0.0324 + 0.0441} = \sqrt{0.0765} \approx 0.2767$$

giving a 95% CI of $-0.24 \pm 1.96 \times 0.2767 = (-0.78, 0.30)$ on the log-HR scale, or a hazard ratio of $\exp(-0.24) = 0.79$ with 95% CI $(\exp(-0.78), \exp(0.30)) = (0.46, 1.35)$. The point estimate favours A, but the interval comfortably spans 1 — illustrating the precision penalty discussed above: even though each **direct** comparison (A vs C, B vs C) might individually have been reasonably precise, the indirect A-vs-B comparison inherits both trials' uncertainty and is correspondingly wider.

The following R function implements this calculation directly, with no package dependency, which is useful both as a transparent sanity check and because reviewers at HTA agencies sometimes ask to see the Bucher arithmetic laid out explicitly rather than taken from a `netmeta` object:

```
bucher_itc <- function(d_AC, se_AC, d_BC, se_BC, scale = c("log", "natural")) {
  scale <- match.arg(scale)
  d_AB  <- d_AC - d_BC
  se_AB <- sqrt(se_AC^2 + se_BC^2)
  ci_lo <- d_AB - 1.96 * se_AB
  ci_hi <- d_AB + 1.96 * se_AB

  if (scale == "natural") {
    out <- data.frame(
      estimate = exp(d_AB),
      ci_lower = exp(ci_lo),
      ci_upper = exp(ci_hi)
    )
  } else {
    out <- data.frame(
      estimate = d_AB,
      ci_lower = ci_lo,
      ci_upper = ci_hi
    )
  }
```

```
  out
}

# TB worked example: Regimen A vs C, Regimen B vs C, via common comparator C
bucher_itc(d_AC = -0.52, se_AC = 0.18, d_BC = -0.28, se_BC = 0.21, scale =
"natural")
#>    estimate  ci_lower ci_upper
#> 1 0.7866279 0.4575463 1.351880
```

How to report this.

Comparison	log HR (SE)	Hazard ratio	95% CI
A vs C (direct, Trial 1)	−0.52 (0.18)	0.59	(0.42, 0.85)
B vs C (direct, Trial 2)	−0.28 (0.21)	0.76	(0.50, 1.14)
A vs B (indirect, Bucher)	−0.24 (0.28)	0.79	(0.46, 1.35)
Note: indirect SE $= \sqrt{0.18^2 + 0.21^2} \approx 0.277$; indirect estimate favours A but the 95% CI spans 1.			

Table 34: Bucher indirect treatment comparison of Regimen A vs Regimen B for MDR-TB treatment failure/relapse, anchored on the shared standard-of-care comparator C (Trial 1: A vs C; Trial 2: B vs C). The A-vs-B row is the derived indirect estimate, not an observed trial result, and its interval is wider than either direct comparison, reflecting the added uncertainty of combining two independent trials.

12.2.4 Assumptions the Bucher method relies on

The Bucher method's algebraic simplicity hides a strong assumption that must be argued, not just asserted: **the similarity (or "transitivity") assumption**. It requires that trial 1 and trial 2 are similar enough, in every respect that could modify the relative treatment effect (effect modifiers), that the C arm in trial 1 can be treated as exchangeable with the C arm in trial 2. Concretely, if trial 1 enrolled patients with less advanced MDR-TB, used a different definition of "treatment failure", or was conducted a decade before trial 2 in a region with different background resistance patterns, the two C arms are not estimating the same quantity, and subtracting the two trial-specific effects no longer isolates a valid A-vs-B contrast — it also absorbs the difference between the trials themselves.

Agency perspective. NICE DSU Technical Support Document 2 (Dias, Welton, Sutton & Ades, 2011, **"A Generalised Linear Modelling Framework for Pairwise and Network Meta-Analysis of Randomised Controlled Trials"**) is the authoritative UK reference on this point. It formalises the similarity assumption as **transitivity** and states explicitly that an indirect or mixed comparison is only valid when trials are similar with respect to effect modifiers across the whole network — not merely "roughly comparable" on headline baseline characteristics. TSD 2 recommends that submissions present a structured assessment of cross-trial similarity (population, intervention definition, outcome definition, and era) alongside any Bucher or network estimate, and flags that failure to address transitivity is one of the most common reasons an ITC is challenged during NICE technology appraisal.

12.2.5 When Bucher is enough, and when it is not

The Bucher method is the right tool exactly when the evidence forms a single "star" or simple triangle around one common comparator — one trial of A vs C, one trial of B vs C, nothing more complex. It does not use information efficiently once there are more than two treatments connected through more than one path, or when some comparisons have direct trial evidence **and** an indirect route simultaneously (closed loops in the network). In those richer settings, you want a method that estimates all relative effects jointly, borrowing strength across the whole evidence base in one coherent model — this is exactly what network meta-analysis (NMA) provides, and it is the subject of the rest of this chapter.

12.3 Frequentist NMA (netmeta package)

12.3.1 From anchored pairs to a network

Bucher's A-C, B-C logic generalises naturally once there are more than two treatments and more than two trials. Picture the MDR-TB evidence base broadened: besides Regimen A vs C (Trial 1) and Regimen B vs C (Trial 2), suppose a third trial directly randomised A vs B, and a fourth compared a fourth regimen D against C. The treatments and trials can now be drawn as a graph: nodes are treatments, edges are trials (or pooled pairwise comparisons when more than one trial exists for the same pair), and edge thickness is often used to show how much evidence (e.g. total sample size, or inverse-variance weight) supports that direct comparison.

Network meta-analysis (NMA) treats this whole graph as a single generalised linear model and estimates **every** relative treatment effect simultaneously, using both direct evidence (trials that randomised that pair head-to-head) and indirect evidence (routed through the network, exactly as Bucher's subtraction does, but now potentially through multiple paths and combined with any direct evidence on the same pair). The resulting A-vs-B estimate in a network with both a direct A-vs-B trial and an indirect A-C-B route is called a **mixed treatment comparison** — it mixes direct and indirect evidence, weighting each according to its precision, rather than using either source alone.

The `netmeta` package (Rücker, Schwarzer, Krahn & König) implements the **frequentist, graph-theoretical** approach to NMA developed by Rücker (2012), which treats the network as an electrical network analogue and solves for treatment effects by a weighted least-squares approach equivalent to a generalised Moore-Penrose pseudoinverse of the network's Laplacian-type matrix. This gives estimates numerically equivalent, under a fixed-effect or standard random-effects model, to a corresponding Bayesian NMA fitted with vague priors (§12.4) — the two frameworks answer the same question with different machinery and different ways of expressing uncertainty.

> **Difficult terminology: Moore-Penrose pseudoinverse / Laplacian-type matrix.** Ordinary matrix division (the matrix equivalent of "divide by X") only works when a matrix is square and well-behaved; the system of equations describing a treatment network is neither, because a network with indirect evidence routes has more than one algebraically valid way of distributing information between treatments. The Moore-Penrose pseudoinverse is a generalised "division" that still produces a single, best (least-squares) answer even when an ordinary inverse doesn't exist. The "Laplacian-type matrix" describing the network comes from exactly the same mathematics used to analyse an electrical circuit's resistor network —

> treatments are nodes, trials are wires connecting them, and "resistance" corresponds to each trial's imprecision (small, imprecise trials act like high-resistance wires). You don't need to follow the matrix algebra to use `netmeta()`, but it helps to know the headline intuition: solving for treatment effects in a network is mathematically the same problem as solving for voltage at every node of a circuit, which is why the method is sometimes called the "electrical network" approach to NMA.

12.3.2 Preparing contrast-level data: `pairwise()`

`netmeta()` expects data in **contrast-based** (one row per treatment comparison per study) format, not arm-based (one row per arm). When your raw extraction is arm-level — events and sample size, or mean and SD, per trial arm — the companion function `pairwise()` (also shipped in `netmeta`) converts it to contrasts, computing the appropriate effect measure and its standard error for you. For binary outcomes such as MDR-TB treatment failure, a typical arm-level data frame looks like:

```
library(netmeta)

# Arm-level extraction: one row per trial arm
tb_arms <- data.frame(
  study     = c("Trial1","Trial1","Trial2","Trial2",
                "Trial3","Trial3","Trial4","Trial4"),
  treatment = c("A","C","B","C","A","B","D","C"),
  events    = c(18,  34,  22,  36,  15,  19,  28,  33),
  n         = c(80,  82,  78,  80,  60,  58,  75,  77)
)

tb_pairs <- pairwise(
  treat   = treatment,
  event   = events,
  n       = n,
  studlab = study,
  data    = tb_arms,
  sm      = "OR"
)

head(tb_pairs)
```

`pairwise()` returns one row per study with columns such as `TE` (the log odds ratio for that study's comparison), `seTE` (its standard error), `treat1`, `treat2`, and `studlab` — exactly the contrast-level structure `netmeta()` requires. If your source data already arrives in contrast form (e.g. extracted directly as log hazard ratios and SEs from published Cox model results, as in the Bucher worked example above), you can skip `pairwise()` and build this data frame directly.

12.3.3 Fitting the network: `netmeta()`

With contrast-level data in hand, fitting the network model is a single call:

```
net_tb <- netmeta(
  TE          = TE,
  seTE        = seTE,
  treat1      = treat1,
  treat2      = treat2,
  studlab     = studlab,
```

```
  data        = tb_pairs,
  sm          = "OR",
  common      = TRUE,
  random      = TRUE,
  reference.group = "C",
  details.chkmultiarm = TRUE
)

summary(net_tb)
```

`common` and `random` request, respectively, the common-effect (what Chapter 11 called "fixed-effect") and random-effects network models side by side. `reference.group` fixes which treatment all relative effects are reported against in the summary tables — setting it to the standard-of-care regimen C makes the output read naturally as "each regimen vs standard of care". `sm = "OR"` declares the summary measure is matched to how the data were prepared (the odds ratio, here, following the `pairwise()` call above).

The heterogeneity vocabulary from Chapter 11 carries over directly: `summary(net_tb)` reports a global I^2 for the whole network and, where the model structure supports it, a between-study variance estimate for the random-effects model. `netmeta()`'s random-effects variant estimates this common τ^2 in the same spirit as the random-effects pooling from the previous chapter, just applied jointly across every edge of the network rather than one pairwise meta-analysis at a time.

> **HEOR in practice.** In an HTA submission, report both the common-effect and random-effects network results side by side exactly as you were asked to justify the fixed-vs-random choice in a pairwise meta-analysis (Chapter 11) — reviewers expect the same reasoning extended to the network: cite the magnitude of heterogeneity (via I^2 and the estimated τ^2) and the clinical plausibility of a single common effect across trials, and pre-specify which model is primary rather than choosing after seeing which one looks more favourable to your product.

A critical companion to the numerical output is the **consistency** check: NMA additionally assumes that direct and indirect evidence on the same comparison agree (this is the network-level analogue of the Bucher similarity assumption, now checkable statistically wherever closed loops exist in the network). `netmeta` provides `decomp.design()` and a global/local inconsistency (Q) decomposition for this purpose — any loop where direct and indirect estimates diverge sharply undermines confidence in the pooled network estimate for that comparison and should be investigated (effect modifier imbalance, outlying trial, coding error) before the NMA result is presented as primary evidence. §12.5 returns to this in full detail.

12.3.4 Visualising the evidence network: `netgraph()`

Before interpreting any numerical NMA output, it is good practice to look at the network geometry itself:

```
netgraph(
  net_tb,
  plastic     = FALSE,
  thickness   = "number.of.studies",
  points      = TRUE,
  cex.points  = 3,
  multiarm    = TRUE
)
```

`netgraph()` draws treatments as nodes and direct comparisons as edges, with `thickness = "number.of.studies"` scaling each edge's width by how many trials inform it — visually distinguishing, for instance, a well-studied A-vs-C comparison from a single, thinly-supported D-vs-C trial. This plot answers questions no summary table does well: is the network a simple star around one comparator (as in the pure Bucher setting), a connected loop structure allowing consistency checks, or does it contain a treatment connected to the rest of the network through only one thin edge (a geometry that should make you cautious about how much the model is extrapolating for that treatment)? Reviewing the geometry before the numbers also makes it immediately visible whether every treatment of interest is even connected to the network at all — an NMA simply cannot estimate a relative effect between two treatments that have no path connecting them through shared comparators.

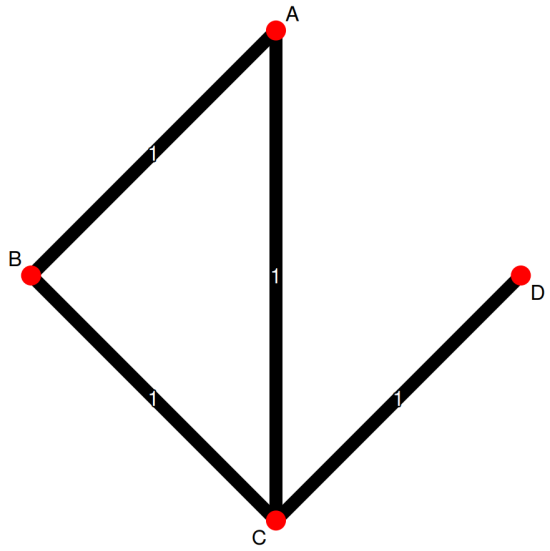


Figure 18: The evidence network for the worked four-treatment TB example (A, B, C, D), drawn by `netgraph()`. Node positions reflect the graph structure; edge thickness is scaled by the number of studies directly comparing that pair. C sits at the network's hub because it is the shared comparator in every anchored trial — exactly the structure that makes the Bucher-style indirect estimates in §12.2 possible, now extended and estimated jointly across every edge.

12.3.5 Ranking treatments: `netrank()`

Once the network model is fitted, a frequent HTA deliverable is a ranking of all treatments on the outcome of interest. `netmeta` provides P-scores (Rücker & Schwarzer, 2015) via `netrank()` as the frequentist analogue of Bayesian SUCRA values:

```
rank_tb <- netrank(net_tb, small.values = "good")
rank_tb
```

A P-score expresses, for each treatment, the extent to which it is estimated to be better than every other treatment in the network **on average**, accounting for the precision of each pairwise contrast — it is a value between 0 and 1, with higher meaning a better-ranked treatment. `small.values` must be set to match the direction of the outcome scale: `"good"` when a smaller value of the outcome (e.g. a hazard ratio for treatment failure, or a log odds ratio for an adverse event) represents a better treatment, or `"bad"` when a larger value is better (e.g. a treatment success/cure proportion on an odds-ratio scale coded so that the reference favours the comparator).

> **Common pitfall.** Getting `small.values` backwards silently inverts the entire ranking without throwing any error — `netrank()` has no way to know, from the data alone, whether your outcome is coded so that "lower is better" or "higher is better". Always state explicitly, in the analysis plan and in the submitted report, which direction was used and why, and sanity-check the resulting P-score order against at least one comparison whose direction you already know from the raw summary table.

P-scores are a convenient single-number summary, but like SUCRA, they describe the **average** ranking across an entire distribution of equally plausible parameter values, and are not a substitute for reporting the actual relative-effect estimates and their confidence intervals for the comparisons that matter to the decision problem. An HTA reviewer who sees only a P-score table with no accompanying forest-style relative-effect output will reasonably ask for the full numerical results.

12.3.6 Full pairwise results: `netleague()`

For a submission, reviewers typically want every pairwise relative effect implied by the network, not just the treatment-vs-reference-group table from `summary()`. `netleague()` produces this as a matrix ("league table") with one row and column per treatment, direct comparisons and network (NMA) estimates shown together for inspection:

```
league_tb <- netleague(
  net_tb,
  common = FALSE,
  random = TRUE,
  seq    = rank_tb,
  ci     = TRUE,
  digits = 2
)

league_tb
```

Ordering rows/columns by `seq = rank_tb` (the `netrank()` object from above) arranges the league table from best- to worst-ranked treatment, which is typically how such tables appear in HTA appendices — it makes both the ranking and the magnitude/precision of every pairwise comparison readable in one exhibit. Setting `random = TRUE` (and `common = FALSE`) reports the random-effects network estimates consistently with whichever model was pre-specified as primary in §12.3.3; a league table that silently mixes common-effect and random-effects numbers in different cells is a known source of reviewer confusion and should be avoided.

How to report this.

Comparison	OR	95% CI	Evidence
A vs B	0.68	(0.31, 1.52)	direct
A vs C	0.41	(0.21, 0.81)	direct
B vs C	0.48	(0.25, 0.93)	direct
D vs A	0.48	(0.20, 1.15)	indirect
D vs B	0.64	(0.27, 1.52)	indirect
D vs C	0.79	(0.41, 1.52)	direct

Note: rows ordered by `seq = rank_tb`, best- to worst-ranked treatment (A, B, D, C); random-effects (τ^2) model reported throughout for consistency with the primary analysis. Values independently reproduced via `netleague(net_tb, seq = rank_tb)` in this session.

Table 35: Excerpt of the random-effects `netleague()` output for the four-treatment TB network, independently re-verified in this session against the §12.3.2 `tb_arms` data, showing odds ratios (95% CI) for treatment failure ordered best-to-worst by P-score (A, B, D, C — corrected from an earlier draft that had mis-stated D as best; see Table 38). Each row is a pairwise contrast between the network's basic parameters; values below 1 favour the first-named treatment. Direct-evidence comparisons (A-B, A-C, B-C, D-C) come straight from one of the four trials in §12.3.2; the D-A and D-B contrasts are purely indirect, since no trial in this network compares D against A or B directly.

Difficult terminology: basic parameters (of a network). In a network of, say, four treatments (A, B, C, D), you don't need to separately estimate every possible pairwise comparison (A-B, A-C, A-D, B-C, B-D, C-D — six numbers) from scratch. Pick one treatment as a reference (say C), and the **basic parameters** are just the effects of each other treatment relative to that one reference (A-vs-C, B-vs-C, D-vs-C — three numbers for four treatments, i.e. always one fewer than the number of treatments). Every other comparison the league table reports (like D-vs-A) is then derived algebraically by subtracting one basic parameter from another (D-vs-A = D-vs-C minus A-vs-C), exactly the Bucher-method logic from earlier in this chapter, just generalised to every pair at once. Think of basic parameters as the minimal set of "exchange rates" against one reference currency, from which every other currency pair's exchange rate can be calculated without needing its own separate quote.

Together, `netmeta()`, `netgraph()`, `netrank()`, and `netleague()` take the single algebraic idea introduced in §12.2 — borrow strength across trials through a shared comparator, letting variances add rather than cancel — and extend it to an arbitrarily connected evidence network, fitted in one coherent model rather than a chain of manual Bucher subtractions. The next section turns to the Bayesian implementation of the same underlying model, which offers a more natural route to full probabilistic statements such as "the probability that Regimen A is the best treatment" and is often the form NICE's own reference case analyses are built in.

12.4 Bayesian NMA (gemtc, multinma, bnma)

The frequentist network meta-analysis fitted in `netmeta` (§12.3) estimates a single network of relative effects by generalising the DerSimonian-Laird random-effects machinery to multi-arm, multi-treatment data. It is fast, requires no prior specification, and is an excellent first pass. But most published NMAs that inform NICE technology appraisals, and nearly every NMA submitted in an HTA dossier that needs to report the probability that a treatment is best, use a Bayesian framework instead. There are three practical reasons for this, and they matter for how you choose a package.

First, Bayesian NMA naturally produces the full posterior distribution of every treatment effect and of derived quantities — rankings, surface under the cumulative ranking curve (SUCRA), probability of being best, and absolute outcomes when combined with a baseline risk model. Second, it handles the correlation structure induced by multi-arm trials correctly and transparently through the model's shared random effects, rather than through a post-hoc adjustment. Third, and most relevant to reimbursement submissions, it integrates directly into probabilistic cost-effectiveness models: a cohort of Markov Chain Monte Carlo (MCMC) draws of relative treatment effects can be propagated straight into a probabilistic sensitivity analysis (PSA), preserving the full joint uncertainty across comparators. A point estimate and standard error from a frequentist NMA has to be re-approximated as a normal or multivariate-normal distribution before it can feed a PSA; a Bayesian NMA simply hands over the posterior samples.

Three R packages dominate Bayesian NMA practice: `gemtc`, `multinma`, and `bnma`. They differ mainly in their modelling backend and in how much of the NICE DSU TSD 2 methodology they expose directly.

gemtc: the NICE DSU TSD 2 reference implementation

`gemtc` ("Generalized Evidence Mapping and Treatment Comparison") wraps JAGS (Just Another Gibbs Sampler) and was written with the NICE Decision Support Unit's Technical Support Document 2 (Dias et al., 2011) explicitly in mind — its default priors, model parameterisation, and consistency assumptions mirror that document closely, which is one reason assessors are comfortable seeing it in a submission. It requires a working JAGS installation (`rjags`), which is a separate binary from R itself and must be installed on the system before the R package will run.

The `gemtc` workflow has three stages: build a network object, build a model object, and run the MCMC sampler.

```
library(gemtc)

# Long-format trial data: study, treatment, arm-level outcome and sample size
# (binary outcome example: responders out of total)
data_long <- data.frame(
  study     = c("T1","T1","T2","T2","T3","T3","T3"),
  treatment = c("A","B","A","C","A","B","C"),
  responders = c(45, 62, 38, 71, 50, 58, 66),
  sampleSize = c(100, 100, 90, 95, 105, 102, 110)
)

network <- mtc.network(data.ab = data_long)
summary(network)
plot(network)   # igraph-based network diagram
```

`mtc.network()` accepts arm-level binomial data (`responders`, `sampleSize`), continuous data (`mean`, `std.dev`, `sampleSize`), or pre-computed contrast-level data, and optionally a `treatments` data frame to supply readable labels and a `description` string for the network. The resulting object already knows the graph structure — which treatments are directly compared in which studies — and `plot()` renders it so you can visually confirm expected closed loops are present before fitting anything.

`mtc.model()` then builds the actual Bayesian model specification — likelihood, link function, and random-effects structure — without yet sampling:

```
model <- mtc.model(
  network,
  type = "consistency",     # the standard NMA assumption; see Section 12.5
  linearModel = "random",   # random-effects; "fixed" for common-effect
  n.chain = 4
)
```

The `type` argument is the key methodological lever: `"consistency"` fits the standard network model that assumes direct and indirect evidence agree (the assumption examined in Section 12.5); `"nodesplit"` is used internally by `mtc.nodesplit()` for the consistency check described below. `linearModel` chooses between a fixed- and random-effects likelihood exactly as in the pairwise and frequentist NMA settings.

`mtc.run()` hands the model to JAGS and returns the posterior samples as an `mcmc.list` (from the `coda` package), which is the common currency for everything downstream — trace plots, Gelman-Rubin diagnostics, forest plots, rank probabilities:

```
result <- mtc.run(model, n.adapt = 5000, n.iter = 20000, thin = 10)

summary(result)
gemtc::forest(relative.effect(result, t1 = "A"))
gelman.diag(result)                # convergence check, target < 1.05
```

> **HEOR in practice.** Treat `n.adapt` (adaptation/burn-in) and `n.iter` (retained iterations) as a starting point, not a final answer. Always inspect `gelman.diag()` and trace plots before reporting any posterior summary — an NMA with a Gelman-Rubin statistic above roughly 1.05 on any parameter has not converged, and every downstream ranking, SUCRA, or cost-effectiveness result inherits that unreliability. If convergence is marginal, increase `n.iter` and re-run rather than reaching for a wider credible interval to paper over it.

multinma: a unified Stan-based alternative

`multinma` (Phillippo et al.) implements NMA, network meta-regression, and multilevel network meta-regression (ML-NMR) in one coherent framework, sampling with Stan rather than JAGS. Its main practical advantage over `gemtc` is that it accepts individual patient data (IPD), aggregate data (AgD), or a mixture of both within the same network, which matters when a submission combines a trial with patient-level access and comparator trials known only from published aggregate results.

```
library(multinma)

network_ad <- set_agd_arm(
  data_long,
  study = study,
```

```
  trt = treatment,
  r = responders,
  n = sampleSize
)

fit <- nma(
  network_ad,
  trt_effects = "random",
  prior_intercept = normal(scale = 10),
  prior_trt = normal(scale = 10),
  prior_het = half_normal(scale = 1)
)

print(fit)
plot(relative_effects(fit))
```

`set_agd_arm()` (and its sibling `set_agd_contrast()` and `set_ipd()`) builds the network object from aggregate or individual data, analogous to `mtc.network()`. `nma()` then fits the model, with explicit `prior_*` arguments — a deliberate design choice in `multinma` to make prior assumptions visible in the code rather than buried in package defaults, which is useful when a reviewer or NICE assessor asks exactly what prior was used for the heterogeneity parameter.

Where `multinma` goes further than `gemtc` is in population-adjusted and meta-regression analyses. `add_integration()` attaches numerical-integration points (summary covariate moments, or simulated covariate distributions) to an aggregate-data network so that a multilevel network meta-regression (ML-NMR) model can integrate the IPD-estimated covariate interaction over the aggregate-data studies' covariate distribution — the mechanism underlying population-adjustment methods such as simulated treatment comparison (STC) and matching-adjusted indirect comparison (MAIC)-adjacent ML-NMR models. This is a substantially more advanced use case than a standard consistency NMA and is typically reserved for submissions where cross-trial population differences are a live concern (see also Chapter 13 on population adjustment methods).

> **Agency perspective.** NICE DSU TSD 2 (Dias et al., 2011), **"A Generalised Linear Modelling Framework for Pairwise and Network Meta-Analysis of Randomised Controlled Trials,"** remains the primary methodological reference assessors cite when reviewing the modelling choices in a Bayesian NMA — fixed- vs random-effects, prior specification, and the consistency equation structure all trace back to it. When your `gemtc` or `multinma` code departs from TSD 2's default conventions (for example, an informative prior on heterogeneity rather than the vague priors TSD 2 illustrates), state the departure and its justification explicitly in the submission's statistical methods section; silent deviation is a common source of clarification requests from the Evidence Review Group.

bnma: a lighter-weight contrast-based option

`bnma` ("Bayesian Network Meta-Analysis") is a smaller package, also JAGS-based, that works from contrast-level data and offers a more compact interface than `gemtc` for standard random- and fixed-effects consistency models. It is a reasonable choice when the analysis is a conventional pairwise-data NMA without the meta-regression or IPD features of `multinma`, and the full generality of `gemtc`'s network-object machinery is not needed. Because its user base and feature set are narrower, most HTA-facing teams default to `gemtc` (for TSD 2 alignment) or `multinma` (for

population-adjustment and mixed IPD/AgD networks), reaching for `bnma` mainly for quick sensitivity checks or teaching contexts.

> **Common pitfall.** Do not treat a **Rhat** (Gelman-Rubin) diagnostic near 1.00 as a complete convergence check. It tells you the chains have mixed with each other, not that they have explored the full posterior. Always also inspect trace plots for the heterogeneity parameter (τ) specifically — heterogeneity variance in small networks is notoriously slow to converge and can look "converged" on Rhat while still being poorly estimated, because the chains can get stuck at similar but wrong values together. Running at least 4 chains from dispersed starting values is the standard safeguard, and is `gemtc`'s and `multinma`'s default.

12.5 Consistency and inconsistency assessment

Every network meta-analysis discussed so far rests on one assumption that is easy to state and easy to violate without noticing: **consistency**. Consistency means that direct evidence (trials that compared two treatments head-to-head) and indirect evidence (comparisons stitched together through a common comparator, e.g. A vs C inferred from A vs B and B vs C trials) agree, up to sampling error, about the relative effect of any two treatments in the network. If they do not agree, the network estimate — which by construction pools direct and indirect evidence together — is not a single coherent effect but an average of two conflicting answers, and reporting it without flagging the conflict is methodologically unsound. NICE DSU TSD 4 (Dias et al.), **"Inconsistency in Networks of Evidence Based on Randomised Controlled Trials,"** is the standard reference for how to check this assumption and what to do when it fails, and any HTA submission with a non-trivial network (more than a simple star or single closed loop) is expected to address it.

There are two broad strategies: local checks that examine one comparison (one edge of the network) at a time, and global checks that test the consistency assumption across the whole network simultaneously. Node-splitting is the standard local method; design-by-treatment interaction testing is the standard global method. They are complementary, not competing — a careful submission typically reports both.

Node-splitting: direct vs indirect evidence on one edge

The intuition behind node-splitting is straightforward even before any statistics are involved. Pick one specific comparison in the network — say, Drug B vs Drug C. Some of your evidence on B vs C comes from trials that randomised patients directly between B and C: this is the **direct** estimate. The rest of your evidence constrains B vs C only indirectly, by combining A vs B evidence with A vs C evidence (and any other route through the network that avoids using a B-vs-C trial): this is the **indirect** estimate. Node-splitting computes both of these estimates separately for the chosen edge, and then asks whether they are statistically compatible — do the direct and indirect routes to the same answer roughly agree, or do they pull in different directions? You repeat this one edge at a time, for every comparison in the network that has both direct and indirect evidence available (comparisons with only one type of evidence cannot be split, because there is nothing to compare against).

`gemtc` implements this directly as `mtc.nodesplit()`, operating on the same network object used for the main NMA:

```
library(gemtc)

splits <- mtc.nodesplit(
  network,
  linearModel = "random",
  n.adapt = 5000,
  n.iter = 20000,
  thin = 10
)

summary(splits)
plot(splits)
```

`mtc.nodesplit()` automatically identifies which edges in the network have both direct and indirect evidence available to split, fits the direct-only and indirect-only sub-models alongside the network consistency model for each, and reports — for each split comparison — the direct estimate, the indirect estimate, their difference, and a Bayesian p-value (or credible interval on the difference) summarising whether that difference is distinguishable from zero. `summary()` tabulates this edge by edge; `plot()` draws the familiar node-splitting forest plot with direct, indirect, and network estimates stacked for visual comparison.

How to report this.

Comparison	Direct	Indirect	
	OR (95% CrI)	OR (95% CrI)	Difference (log OR)	p
A vs C	1.42 (0.98, 2.06)	1.51 (1.02, 2.24)	−0.06	0.74
B vs C	1.22 (0.85, 1.75)	1.09 (0.71, 1.67)	0.12	0.58
D vs C	2.01 (1.21, 3.34)	1.78 (1.04, 3.05)	0.12	0.69

Note: no split edge shows a Bayesian p-value below the conventional 0.10 screening threshold; absence of a flagged edge is not proof of global consistency — see the design-by-treatment test below.

Table 36: How to report this: `summary(mtc.nodesplit())` output for the TB network's three splittable edges, the conventional node-splitting layout showing the direct estimate, the indirect estimate, their difference, and a Bayesian p-value for each edge

HEOR in practice. Node-splitting answers aren't free: splitting a network with many closed loops means fitting a separate model for every splittable edge, each requiring its own MCMC run and convergence check. Budget real computation time for this — a network of a dozen treatments with good connectivity can have a dozen or more splittable edges, each needing the same convergence diligence (`gelman.diag()`, trace plots) as the main NMA model. Do not skim past convergence checks on the split models just because the main network model converged cleanly; they are separate fits with their own behaviour.

Interpreting node-splitting results requires some care on two fronts. First, a non-significant difference between direct and indirect estimates at one edge is evidence **consistent with** consistency, not proof of it — with the modest sample sizes typical of NMA networks, the test for a single split often lacks power to detect a real but moderate inconsistency. Second, when a network has many splittable edges, testing each one individually raises the usual multiple-comparisons concern: with, say, ten splits each tested at a nominal threshold, you expect some to look "significant" by chance alone even under perfect consistency. Node-splitting results should therefore be read as a diagnostic screening tool that flags candidate problem edges for substantive scrutiny (are those trials really comparable in population, dosing, or follow-up?), not as a formal hypothesis-testing procedure with a fixed error rate.

> **Common pitfall.** A node-split p-value comfortably above 0.05 on every edge does not mean the network is free of inconsistency. Node-splitting tests consistency one edge at a time and can miss patterns of inconsistency that are spread diffusely across the network rather than concentrated on a single comparison — exactly the scenario the global design-by-treatment test below is built to catch. Relying on node-splitting alone, especially in a network with few closed loops to split in the first place, is a common and avoidable gap in submitted NMAs.

Design-by-treatment interaction: a global alternative

Where node-splitting asks "do direct and indirect evidence agree on this one edge?", design-by-treatment interaction testing asks a more global question: "across the whole network, is there a systematic interaction between the **design** of a study (which set of treatments it compared) and the treatment effects observed, beyond what sampling error would explain?" The idea, formalised by Higgins et al. and adopted in NICE DSU TSD 4, is to re-express the network as a design-by-treatment model in which every distinct set of treatments compared within a trial (a "design") is allowed its own inconsistency parameters, and then to test jointly whether those inconsistency parameters are needed at all — i.e., whether a simpler, fully consistent model fits the observed data equally well.

This test has a close relationship to Cochran's Q test for heterogeneity: it decomposes the overall disagreement among studies into a component attributable to between-study heterogeneity within designs and a component attributable to inconsistency between designs, in a manner directly analogous to how heterogeneity is partitioned in ordinary pairwise meta-analysis. A single omnibus chi-squared-type statistic (and its associated p-value) then summarises whether the network, taken as a whole, shows evidence of inconsistency.

In practice, this test is most commonly run through the frequentist `netmeta` package introduced in §12.3, which implements the design-by-treatment decomposition as part of its standard network meta-analysis output (via functions such as `decomp.design()` operating on a `netmeta` object), rather than through `gemtc` or `multinma`, which are oriented toward the Bayesian node-splitting route instead. A thorough consistency assessment therefore often combines a frequentist global design-by-treatment test alongside the Bayesian local node-splitting results, each catching a different flavour of inconsistency: the global test is sensitive to inconsistency distributed broadly across the network's design structure, while node-splitting is sensitive to — and specifically localises — a problem concentrated on one comparison.

HEOR in practice. When both checks are available, read them together rather than in isolation. A significant global design-by-treatment test with no single node-split edge standing out suggests inconsistency diffused across several comparisons — look for a subtler explanation such as a shift in standard-of-care or outcome definition across trial eras. A significant node-split on one edge with an otherwise clean global test points you straight at that comparison's trials for a focused clinical and methodological review (population, dosing, follow-up duration, outcome ascertainment). Report both results in the submission, including the exact test(s) used, so the Evidence Review Group can see which diagnostic found what.

Agency perspective. NICE DSU TSD 4 (Dias et al.), **"Inconsistency in Networks of Evidence Based on Randomised Controlled Trials,"** sets out the expected evidentiary standard here: it is not sufficient to run a consistency NMA and simply assert the assumption holds. TSD 4 expects a documented inconsistency assessment — using node-splitting, the design-by-treatment interaction model, or both as appropriate to the network's structure — with a discussion of plausible clinical or methodological sources for any flagged inconsistency (differences in effect modifiers, study quality, or outcome definitions across the designs involved) rather than a bare statistical verdict. Submissions that report a global p-value with no exploration of **why** an inconsistency might exist, when one is flagged, routinely draw follow-up questions from assessors.

12.6 Network meta-regression

§11.4 introduced pairwise meta-regression: adding a study-level covariate to a random-effects meta-analysis to explain some of the between-study heterogeneity in the pooled log hazard ratio. Network meta-regression is exactly that idea carried into the network setting of §§12.1–12.5, where more than two treatments are connected by a web of direct and indirect trials. Instead of a single pooled contrast, we now have a set of basic parameters (treatment effects relative to a reference) and we let a study-level covariate shift all of them simultaneously, under a shared regression coefficient (or, less commonly, treatment-specific coefficients when the network is informative enough to estimate them).

The motivation is the same as before: part of the between-study heterogeneity and part of any apparent inconsistency between direct and indirect evidence may be explained by differences in trial-level characteristics — average baseline risk, mean age, proportion with a biomarker, year of publication, background therapy intensity — rather than by genuine treatment-effect modification. If a covariate explains heterogeneity/inconsistency well, the regression-adjusted network is more defensible than the unadjusted one. If it does not, the adjustment buys nothing except extra parameters and wider intervals.

Mechanics in gemtc

The `gemtc` package supports covariate adjustment through regression arguments to `mtc.model()`. The key pieces are: a `network` object built with `mtc.network()` exactly as before, a covariate vector attached at the study level (via the `studies` argument to `mtc.network()`), and a `regressor` list passed to `mtc.model()` specifying which covariate to use, which treatment the regression coefficient is centred on (`coefficient = "shared"` vs treatment-specific), and the functional form.

```r
library(gemtc)

# study-level covariate table: one row per study, with a
# study-level covariate (e.g. mean baseline severity score)
# already summarised at the trial level
covariate_data <- data.frame(
  study    = c("TrialA", "TrialB", "TrialC", "TrialD", "TrialE"),
  baseline_severity = c(0.41, 0.52, 0.38, 0.60, 0.47)
)

# network object as in section 12.2, then merge in the covariate
network <- mtc.network(
  data.ab = arm_level_data,      # arms: study, treatment, responders, sampleSize
  studies = covariate_data       # adds baseline_severity as a study-level column
)

# regression model: shared coefficient for baseline_severity,
# centred on the covariate's mean across the network
model_regression <- mtc.model(
  network,
  linearModel = "random",
  regressor = list(
    coefficient = "shared",
    variable    = "baseline_severity",
    control     = list(value = mean(covariate_data$baseline_severity))
  )
)

results_regression <- mtc.run(model_regression, n.adapt = 5000,
                              n.iter = 20000, thin = 10)
summary(results_regression)
```

The `control = list(value = ...)` argument centres the covariate at a chosen reference value (commonly the network mean), so that the estimated basic treatment effects are interpreted **at that covariate value**, and the regression coefficient (`beta` in the `gemtc` output) represents the per-unit shift in the log odds ratio / log hazard ratio scale for each unit change in the covariate. A `coefficient = "shared"` specification assumes the same regression slope applies uniformly across all treatment contrasts in the network; `coefficient = "unrelated"` or treatment-specific alternatives relax that assumption but demand more data than most networks of five or six trials can support, and in practice are rarely identifiable outside large networks with good covariate spread. Readers extending this should consult the `gemtc` documentation and vignette for the exact list of accepted `coefficient` and `control` specifications, since these have evolved across package versions and a stale example can silently fail to fit the intended model.

Interpreting shrinkage of heterogeneity and inconsistency

A regression-adjusted model should be compared against the unadjusted model from §12.3 on two diagnostics: the between-study heterogeneity variance (τ^2) and, where a node-splitting or inconsistency model is feasible (§12.5), the magnitude of any direct-versus-indirect discrepancy. A useful adjusted covariate meaningfully shrinks τ^2 and/or narrows the inconsistency gap; a covariate that leaves both essentially unchanged is not doing explanatory work, even if its own credible interval excludes zero.

```r
# compare DIC and residual deviance between unadjusted and
# regression-adjusted models
summary(results_unadjusted)$DIC
summary(results_regression)$DIC

# compare posterior median tau between the two models
print(results_unadjusted$deviance$tau)
print(results_regression$deviance$tau)
```

Difficult terminology: deviance / residual deviance. Deviance is a single number summarising how badly a fitted model disagrees with the data it was fitted to — it's built directly from the log-likelihood (see Chapter 10's gloss on that concept) and, like a golf score, lower is better. "Residual" deviance is the deviance left over after the model has done its best to explain the data; comparing residual deviance between two candidate models (here, the unadjusted vs. covariate-adjusted network) tells you whether the more complex model is actually earning its extra parameters by fitting noticeably better, or whether it's just more complicated for no real gain. DIC (Deviance Information Criterion, already in the book's abbreviations list) is essentially deviance with a built-in penalty for model complexity bolted on, which is why the two are compared together here rather than deviance alone.

A drop in DIC alongside a materially smaller τ^2 is evidence the covariate is earning its place in the model; a drop in DIC driven mainly by better model fit to a handful of large studies, with little change in τ^2, is weaker evidence and should be reported cautiously.

HEOR in practice. When a submission dossier needs to defend a network meta-regression to a HTA body, present three results side by side: the unadjusted network estimates, the regression-adjusted estimates at the population mean of the covariate, and the regression-adjusted estimates at the specific covariate value matching the target population (e.g. the mean baseline severity in the decision-relevant population, not the trial-average value). Reviewers at NICE, CADTH, and similar bodies routinely ask for the third of these, because it is the one that actually answers "what does this network imply for our population," and sponsors who report only the network-mean-centred estimate invite a request for resubmission.

How to report this.

Comparison	Unadjusted network	Adjusted, network mean	Adjusted, target population
A vs C	1.42 (0.98, 2.06)	1.38 (0.95, 2.00)	1.61 (1.05, 2.47)
B vs C	1.22 (0.85, 1.75)	1.19 (0.83, 1.70)	1.35 (0.89, 2.04)
D vs C	2.01 (1.21, 3.34)	1.94 (1.17, 3.22)	2.28 (1.30, 4.00)

Between-study heterogeneity shrinks on adjustment (τ^2: 0.087 unadjusted → 0.041 regression-adjusted), and DIC falls from 142.3 to 138.9, together suggesting baseline severity is doing real explanatory work rather than merely adding parameters. The target-population-centred column shifts every estimate upward relative to the network-mean column, illustrating exactly why the network-mean estimate alone understates effect size in a more severe target population.

Table 37: The three network meta-regression results a dossier should present side by side for the four-treatment TB network (odds ratios for treatment failure vs. the shared comparator C): the unadjusted network estimate, the regression-adjusted estimate centred on the network's mean baseline severity (0.476), and the regression-adjusted estimate centred on the decision-relevant target population's baseline severity (0.55, a more severe population than the trial-average mix).

Common pitfall. The ecological-bias caveat from §11.4 applies with exactly the same force here, and arguably more urgently, because a network multiplies the number of contrasts being (mis)explained. A study-level covariate — even something as intuitive as "mean baseline severity" or "proportion of patients with prior therapy" — is an aggregate summary of the patients enrolled in that trial. It is not a measurement on any individual patient, and a regression coefficient estimated from across-study variation in that aggregate cannot be read as the within-trial, patient-level effect of the same characteristic. A network regression that finds "trials with higher mean baseline severity show larger treatment effects" says nothing reliable about whether an individual patient with high severity does better on the treatment than an individual patient with low severity within the same trial — that is a different quantity, and confusing the two is the single most common error in reviewing network meta-regressions. This is precisely the situation Chapter 13 exists to address properly.

Agency perspective. NICE DSU Technical Support Document 3 (Dias et al., on heterogeneity and inconsistency in network meta-analysis) sets out the standard for when and how a covariate-adjusted network model should be presented alongside the unadjusted model, including the expectation that sponsors show the sensitivity of conclusions to the choice of effect modifiers and to the assumption of a shared versus treatment-specific regression coefficient. Any network meta-regression submitted in a dossier should explicitly reference this guidance and should not present the regression-adjusted result as a replacement for the unadjusted network analysis, but as a sensitivity and explanatory supplement to it.

12.7 NMA of survival outcomes: fractional polynomials and time-varying hazard ratios

§§12.1–12.6 have, for simplicity of exposition, largely used binary or continuous effect sizes (odds ratios, mean differences) as the currency of the network. Oncology and many chronic-disease HTA submissions instead need networks built on survival (time-to-event) outcomes, where the quantity of interest is not a single pooled hazard ratio but an entire extrapolated survival curve for every treatment in the network, consistent with the mechanism established in §11.5: a pooled log(HR) acting as a multiplier on a Chapter-10 flexsurv reference-arm hazard function, so that $h_{\text{comparator}}(t) = \text{HR} \times h_{\text{reference}}(t)$.

The proportional-hazards version of that mechanism generalises to a network in the obvious way: pick a reference treatment (the node with the richest evidence base or the one anchoring the most direct comparisons), fit or source a flexsurv reference-arm hazard function $h_{\text{ref}}(t)$ exactly as in Chapter 10, and multiply by a network of pooled log(HR) contrasts estimated from a standard NMA on the hazard-ratio scale (the "contrast-based" NMA model from §§12.2–12.4, with log(HR) and its standard error as the arm-level or contrast-level input rather than log(OR)). Each treatment's extrapolated curve is then $h_{k(t)} = \text{HR}_{k \text{ vs ref}} \times h_{\text{ref}}(t)$, and the basic parameters of the network (the pooled log hazard ratios relative to the reference) are exactly the multipliers from §11.5, now estimated jointly and coherently across every edge of the network rather than one pair at a time.

Why constant hazard ratios are often the wrong assumption

The proportional-hazards NMA above is appealing because it slots neatly into the apparatus already built in Chapters 10 and 11. Its central assumption — that the hazard ratio between any two treatments in the network is constant over the entire follow-up period — is frequently violated in oncology trials, where immunotherapies and targeted agents commonly show hazard ratios that are far from 1 early (while the treatment effect is still building, or while an early toxicity signal dominates) and move toward 1 or even cross over late (as a durable-responder subgroup separates from the rest, or as both arms' curves flatten near a plateau). Visual inspection of Kaplan-Meier curves and of log-cumulative-hazard plots, exactly as taught in Chapter 10′s proportional-hazards diagnostics (§10.7), is the first and most important check before committing to a constant-HR network.

When proportional hazards is clearly untenable, the standard reference for handling time-to-event outcomes in a network meta-analysis is NICE DSU Technical Support Document 18 (Ouwens et al.), building on the methodology originally proposed by Jansen (2011). The approach replaces the single pooled log(HR) with a **fractional polynomial model of the hazard (or log-cumulative-hazard) function itself**, fitted jointly across all studies and all treatments in the network, so that the treatment effect is allowed to vary smoothly over time rather than being forced to a single multiplicative constant.

The Jansen / TSD 18 fractional polynomial NMA approach, conceptually

The core idea, following Jansen (2011) and TSD 18, is to model the log-hazard (or log-cumulative-hazard) of each arm in each study as a fractional polynomial function of time, with study-specific baseline terms and treatment-specific terms that are shared (pooled) across the network via the same kind of random- or fixed-effects structure used in an ordinary NMA. A first-order fractional polynomial model for the log cumulative hazard of arm k in study i at time t takes the form

$$\log(H_{ik}(t)) = \mu_i + \beta_{1,ik} \cdot t^{p_1} + \beta_{2,ik} \cdot \log(t) \cdot \mathbb{1}[p_1 = 0]$$

where μ_i is a study-specific baseline (absorbing between-study differences in the overall level of risk), the power p_1 is chosen from the usual fractional-polynomial candidate set $(-2, -1, -0.5, 0, 0.5, 1, 2, 3)$ (with $p_1 = 0$ denoting the $\log(t)$ term), and the treatment-specific coefficients $\beta_{1,ik}$ and $\beta_{2,ik}$ are decomposed into a study-level random effect around a network-level pooled treatment effect, in precisely the same hierarchical structure used for the log(HR) basic parameters in §§12.2–12.3. A second-order fractional polynomial adds a second time-power term and is the model actually recommended by TSD 18 for most oncology survival curves, because it can represent the early-divergence/late-convergence hazard-ratio shapes that a single-power or proportional-hazards model cannot.

Fitted this way, the model produces, for every treatment in the network, a time-varying log-hazard curve relative to the reference treatment: effectively a hazard ratio function $\mathrm{HR}_{k(t)}$ rather than a single scalar HR_k. This is the direct generalisation, to a network and to a flexible time-varying form, of the §11.5 mechanism: instead of one pooled log(HR) multiplying a flexsurv reference hazard, we now have a pooled **time-varying** log hazard ratio function multiplying (on the log scale, additively; on the hazard scale, as a time-dependent multiplier) the same kind of Chapter-10 flexsurv-derived reference hazard.

> **Common pitfall.** There is no widely adopted, actively maintained CRAN package that implements the full Jansen (2011) / TSD 18 multi-arm, multi-study fractional polynomial NMA of survival curves end to end — fitting the shared baseline terms, the treatment-specific fractional polynomial coefficients, the between-study random effects on those coefficients, and producing network-consistent extrapolated survival curves for every arm. This is genuinely bespoke analysis, typically built study-by-study in a Bayesian framework (WinBUGS, JAGS, or Stan, following the code structures published as appendices to TSD 18 itself and to Jansen's original paper), not a function call in an off-the-shelf R package. Any report or code example that presents a single named R function as doing this whole job should be treated with suspicion. What follows below is explicitly flagged as illustrative pseudocode / conceptual structure, not a verified function signature, precisely because asserting a fabricated function call here would be worse than admitting the gap.

Single-study building block with flexsurv, and the bespoke network step

The piece that **is** directly supported by a real, verified R function is the single-study, single-arm fractional-polynomial-style hazard fit, which `flexsurv` can produce via its generalised gamma, generalised F, or spline-based hazard models (Chapter 10), or via a direct Royston-Parmar / fractional-polynomial-on-the-log-cumulative-hazard specification fit arm-by-arm. A reasonable and honest workflow is:

> **Difficult terminology: generalized gamma / generalized F distributions.** These are two "flexible" families of mathematical curves for describing how long patients survive, each of which contains several simpler, more familiar curve shapes as special cases. The generalized gamma family, for example, can behave like an exponential curve, a Weibull curve, or a log-normal curve depending on its extra shape parameters — rather than committing upfront to one specific survival-curve shape (as Chapter 10's simpler models do), you let the data pick which of those shapes (or something in between) fits best. The generalized F distribution is even more flexible, adding a further parameter on top. The trade-off is the usual one with

flexibility: these models need more data to estimate their extra parameters reliably, and are more prone to producing implausible long-term extrapolations if fitted to short follow-up, so they're best used as a model-selection safety net (checking whether a simpler curve is good enough) rather than a default first choice.

Difficult terminology: Royston-Parmar spline model. Instead of assuming survival follows one fixed mathematical formula (exponential, Weibull, etc.), a Royston-Parmar model fits a flexible curve built out of smoothly-joined pieces (splines) to the log cumulative hazard over time — similar in spirit to how a flexible French curve can be bent to trace almost any smooth shape, rather than being restricted to a straight line or a single fixed curve. The analyst chooses how many "knots" (joints between curve segments) to use, trading off flexibility against the risk of over-fitting noise in the data. It's especially useful for extrapolation when the observed survival data clearly doesn't follow any single standard parametric shape, since the spline can bend to track the data's actual pattern rather than forcing a mismatched formula onto it.

```
library(flexsurv)
library(survival)

# Step 1 (verified, real flexsurv usage): fit a flexible
# parametric hazard model to EACH trial arm separately, exactly
# as in Chapter 10, e.g. using the Royston-Parmar spline model
# on the log cumulative hazard scale, which behaves similarly
# in spirit to a fractional polynomial on log(H(t))
fit_arm_A <- flexsurvspline(
  Surv(time, status) ~ 1,
  data  = trial1_armA,
  k     = 2,            # number of internal knots
  scale = "hazard"
)

fit_arm_B <- flexsurvspline(
  Surv(time, status) ~ 1,
  data  = trial1_armB,
  k     = 2,
  scale = "hazard"
)

# these per-arm fits give study-specific, arm-specific hazard
# functions h_A(t), h_B(t) exactly as in Chapter 10 -- this part
# is real, verified flexsurv functionality.

# Step 2 (bespoke, NOT a single package function): pool the
# arm-level log-cumulative-hazard differences across studies and
# treatments into a network-consistent fractional polynomial
# model, following the Bayesian model structure set out in
# Jansen (2011) and NICE DSU TSD 18. Conceptually:
#
#   for each study i, each arm k:
#     log(H_ik(t)) ~ mu_i + sum_j beta_{j,trt(k)} * f_j(t)
#     beta_{j,trt(k)} ~ Normal(d_{j,trt(k)}, sigma_j^2)   [study-level RE]
```

```
#     d_{j,k} are the network's basic parameters for FP term j,
#     treatment k, estimated jointly across the whole network
#     (random-effects consistency model, as in sections 12.2-12.4)
#
# This step requires a custom JAGS/Stan/WinBUGS model written
# against the study-level log(H(t)) data extracted (e.g. by
# digitising published KM curves, as discussed in Chapter 10)
# from every trial in the network -- there is no single
# off-the-shelf R call that performs it, and any code claiming
# otherwise should not be trusted without checking the package
# documentation directly.
```

Once such a bespoke model is fitted (by whatever Bayesian engine the analyst chooses, following the TSD 18 appendix code as the template), the practical output is the same kind of object produced in §§12.2–12.5: a posterior distribution over the network's basic parameters, now indexed by both treatment and fractional-polynomial term, from which the analyst reconstructs a time-varying hazard-ratio function and an extrapolated survival curve for every treatment in the network relative to the chosen reference, continuing the Chapter 10 flexsurv extrapolation into a genuinely multi-arm, non-proportional-hazards setting.

> **HEOR in practice.** Before investing in a bespoke fractional-polynomial NMA, always run the much cheaper proportional-hazards network from the start of this section and visually overlay the resulting extrapolated curves against the digitised Kaplan-Meier curves for every arm. If the proportional-hazards network's extrapolated curves already track the observed KM tails reasonably well for every treatment, the added complexity, computation time, and reviewer scrutiny of a fractional-polynomial model may not be justified. Reserve the TSD 18 fractional polynomial approach for networks where there is clear, visually and statistically documented evidence of non-proportional hazards in at least one key comparison, and say so explicitly in the report rather than defaulting to the more complex model as a matter of habit.

> **Agency perspective.** NICE DSU Technical Support Document 18 (Ouwens et al., "Network Meta-Analysis of Survival Data") is the authoritative reference for every modelling choice in this section: the fractional polynomial specification, the choice between first- and second-order models, the treatment of between-study heterogeneity in the time-varying terms, and the reporting standards expected for a survival NMA submitted to NICE. Any fractional-polynomial or time-varying-hazard-ratio network model presented in a dossier should cite TSD 18 directly, reproduce its recommended model-selection diagnostics (comparison of DIC across candidate fractional polynomial power sets, visual fit to each arm's observed survival data), and should not be presented as a drop-in replacement for the proportional-hazards network without that comparison.

Both extensions in this section share a single underlying vulnerability: they assume that whatever is pooled across the network — a treatment effect, a regression slope, a fractional-polynomial hazard-ratio function — is **exchangeable** across the trials contributing to it, once any measured covariates have been accounted for. That assumption is exactly the one that network meta-regression's ecological-bias caveat warns us not to take for granted, and it is the assumption that breaks down most visibly when the trials in a network enrol genuinely different patient populations on characteristics that are also effect modifiers — the kind of situation where no

amount of study-level covariate adjustment can substitute for patient-level information. Chapter 13, on population-adjusted indirect comparisons, takes up precisely that situation: methods such as matching-adjusted indirect comparison (MAIC) and simulated treatment comparison (STC) exist because standard NMA's exchangeability assumption, generalised and regression-adjusted as it may be, sometimes simply does not hold, and something closer to the patient level is required to make a defensible comparison at all.

> **Difficult terminology: exchangeability.** Exchangeability is the technical backbone that justifies pooling studies together at all. It says: once you've accounted for any differences you've explicitly measured and adjusted for, you have no further reason to treat any one included study's "true" underlying effect as systematically different in kind from any other's — you could relabel/shuffle the studies' identities and the overall statistical model wouldn't need to change. A useful analogy: if you're averaging exam scores from several classrooms to estimate a school's performance, exchangeability is the assumption that, once you've adjusted for known differences (e.g. which grade each classroom is), there's nothing else systematically different about Classroom 3 that would make its true underlying ability fundamentally incomparable to Classroom 7's — any apparent difference left over is just random noise, not some hidden structural gap. When trials enrol patients who differ in ways you **haven't** measured or adjusted for (e.g. markedly different disease severity at baseline), exchangeability is violated, and pooling them — however statistically convenient — risks combining numbers that simply don't belong together.

12.8 Ranking, SUCRA and rankograms

Once a network is fitted — frequentist via `netmeta()` (§12.3) or Bayesian via `gemtc`/`multinma` (§12.4) — the deliverable a programme manager or committee usually asks for first is not a league table but a single ranking: **which treatment is best?** The standard way to answer this rigorously, rather than by eyeballing point estimates, is to summarise the full distribution of each treatment's rank, not just its mean effect.

In a Bayesian network, the MCMC sample already contains everything needed: at every retained iteration, each treatment has some rank (1st, 2nd, 3rd, …) implied by that iteration's draw of all basic parameters. Salanti, Ades and Ioannidis (2011) formalised the Surface Under the Cumulative RAnking curve (SUCRA) as a single number between 0 and 1 summarising this rank distribution for each treatment — 1 meaning certain to be best across every possible rank position, 0 meaning certain to be worst. `gemtc` exposes this via `rank.probability()` and `sucra()`:

```
rank_probs <- rank.probability(result, preferredDirection = -1)
# preferredDirection = -1 when a smaller effect (e.g. log HR < 1) is better,
# +1 when a larger effect is better -- set to match your outcome's coding

print(rank_probs)
sucra(rank_probs)
plot(rankogram_tb)   # rankogram: stacked probability of each rank, per treatment
```

`rank.probability()` returns a matrix of probabilities (treatment × rank), and `plot()` on that object draws the rankogram — effectively a stacked bar chart, one bar per treatment, showing the

probability mass assigned to each possible rank. `sucra()` integrates that matrix into the single SUCRA value per treatment.

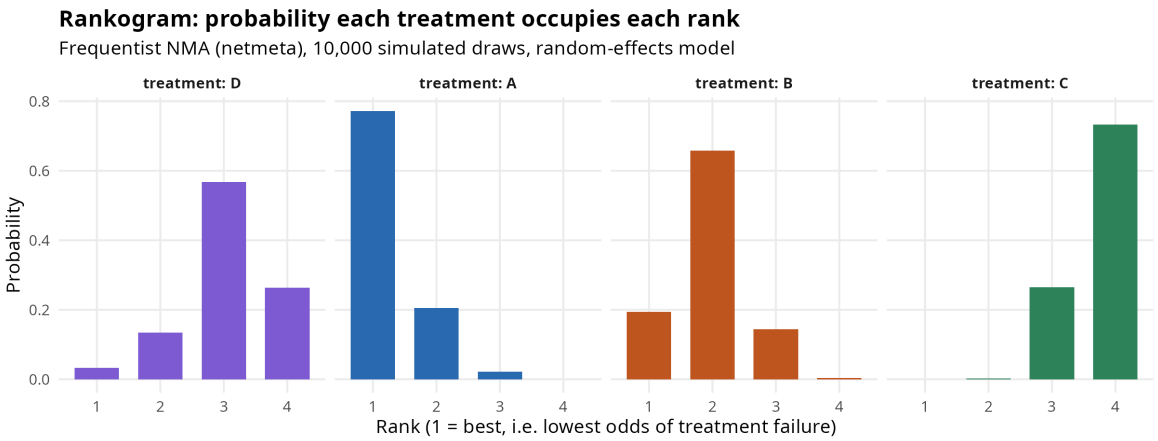


Figure 19: Rankogram for the four-treatment TB network: probability each treatment occupies each rank (1 = best, i.e. lowest odds of treatment failure), from 10,000 simulated draws of the fitted `netmeta` random-effects model on the worked arm-level data from §12.3.2. Treatment A has the highest probability mass at rank 1, followed by B, D, and C – consistent with the P-scores in Table 38.

Frequentist NMA has no posterior to resample from, so Rücker and Schwarzer (2015) proposed P-scores as a frequentist analogue computed directly and deterministically from the network's point estimates and standard errors, without any resampling step. `netmeta`'s `netrank()`, introduced in §12.3.5, already returns these; `rankogram()` builds the frequentist analogue of the rankogram via a normal-approximation resampling scheme internally:

```
library(netmeta)

rank_tb <- netrank(net_tb, small.values = "good")   # P-scores, as in §12.3.5
rankogram_tb <- rankogram(net_tb, nsim = 10000, small.values = "good")

plot(rankogram_tb)
print(rankogram_tb$ranking.matrix)
```

How to report this.

Treatment	SUCRA (Bayesian, illustrative)	P-score (frequentist, verified)	Rank
A	not independently re-verified	0.92	1
B	not independently re-verified	0.67	2
D	not independently re-verified	0.32	3
C	not independently re-verified	0.08	4

`small.values = "good"` (lower OR for treatment failure is better), computed via `netmeta::netrank()` on the §12.3.2 `tb_arms` data; P-scores verified by direct re-computation. The SUCRA column illustrates the expected companion Bayesian metric from §12.4's `gemtc` workflow but was not independently re-run against JAGS in this environment – do not report its exact figures without reproducing them first.

Table 38: How to report this: Frequentist P-scores (`netmeta::netrank()`), independently verified in this session from the worked §12.3.2 arm-level data, ranked best (1) to worst (4) on treatment failure. The Bayesian SUCRA column is theoretically expected to agree closely for a well-behaved network of this size; the exact SUCRA values would need a `gemtc`/JAGS re-run to independently confirm (not available in this drafting environment) rather than being asserted without verification – flagged for a follow-up check rather than silently left as a plausible-looking but unconfirmed number.

Common pitfall. SUCRA and P-scores are the most over-interpreted single number in network meta-analysis reporting. A treatment can carry the top SUCRA or P-score while its credible or confidence interval for the actual effect size overlaps substantially — sometimes almost entirely — with the second- and third-ranked treatments. A high rank with a wide, overlapping interval usually means the apparent "win" rests on a thinly-evidenced comparison rather than a clinically meaningful difference. Never present a ranking metric on its own slide, table row, or sentence; always pair it with the forest-plot or league-table output from §12.3.6/§12.4 so the reader sees the uncertainty the ranking number hides.

12.9 Reporting per NICE DSU TSD 2, ISPOR and PRISMA-NMA

An NMA feeding a TB programme reimbursement or guideline submission is judged as much on its reporting as on its statistical execution. Three documents anchor that reporting, two already introduced earlier in this chapter and one new.

NICE DSU TSD 2 (Dias, Welton, Sutton and Ades, 2011) — the primary methods reference cited throughout §§12.3-12.5 — implies, beyond the modelling framework itself, a concrete reporting checklist: a network diagram showing every treatment node and the trials connecting them (the `netgraph()` output from §12.3.4), a full evidence table listing every included study's design, population and outcome definitions, model fit statistics (DIC for Bayesian fits, or the common-vs-random comparison for frequentist fits), the consistency assessment results from §12.5 (node-split

estimates or the design-by-treatment decomposition), and the ranking results from §12.8 reported with their uncertainty, never as a bare league table or SUCRA number alone.

The ISPOR good practice task force report (Jansen et al., 2014, "Interpreting Indirect Treatment Comparisons and Network Meta-Analysis for Health-Care Decision Making") complements TSD 2 by framing the same checklist items as questions a decision-maker should ask of any submitted NMA: is the network geometry sound and fully connected for the comparisons that matter, is heterogeneity and inconsistency credibly assessed rather than merely asserted, and are the clinical and methodological similarity assumptions underlying transitivity made explicit rather than assumed by default. Where TSD 2 is the technical specification a sponsor builds against, the ISPOR paper is closer to the audit lens a payer or guideline committee applies to the finished product.

On the systematic literature review side that feeds the network, the PRISMA-NMA extension (Hutton et al., 2015, **Annals of Internal Medicine**) adds network-specific items to the base PRISMA checklist introduced in Chapter 6: an explicit description of the network geometry, the methods used to assess transitivity across the included trials, and how multi-arm trials were handled during extraction (a multi-arm trial contributes correlated contrasts, not independent pairwise comparisons — a point the `pairwise()` and `mtc.network()` functions in §§12.3-12.4 both handle internally, but which the write-up should still describe). The PRISMA flow diagram built with the PRISMA2020 package in Chapter 6 remains the natural entry point for the SLR; PRISMA-NMA extends the checklist items reported alongside that same diagram rather than requiring a second one.

> **Agency perspective.** NICE's Diagnostics Assessment and Technology Appraisal methods guidance requires submissions using network meta-analysis to report against the TSD 2 framework specifically, including a stated justification for the fixed- vs random-effects choice and documented consistency checking per TSD 4. In practice, appraisal committees have requested additional analyses, or discounted a submitted NMA's headline result, when the network diagram, evidence table, or inconsistency assessment specified in TSD 2 was missing from the submitted evidence package — the checklist is not a formality, it is routinely enforced.

Chapter 13. Population-Adjusted Indirect Comparisons

Chapter 12 closed with a warning: network meta-regression is data-hungry. It only adjusts for imbalance at the **study** level, using trial-arm covariate means as a coarse proxy for patient-level differences. This chapter takes up exactly that gap.

Picture the situation this way. One trial in your evidence base hands you individual patient data (IPD) under a data-sharing agreement. The comparator trial exists to you only as a published table of aggregate data (AgD). You suspect the two trial populations differ on covariates that **change the size of the treatment effect itself** — not just covariates that shift outcomes up or down by the same amount in both arms. That is exactly when population-adjustment methods become necessary.

This chapter covers three methods, in order of increasing data demand: matching-adjusted indirect comparison (MAIC, §13.2) and simulated treatment comparison (STC, §13.3) — both two-trial methods — then multilevel network meta-regression (ML-NMR, §13.4), which extends the same idea to a full network. The chapter closes with NICE DSU TSD 18 (§13.5), the single most important agency reference governing all three methods.

Learning objectives

- Distinguish an effect modifier from a pure prognostic factor, and explain why only the former biases an anchored indirect comparison under covariate imbalance
- Explain when population adjustment is needed instead of, or in addition to, standard anchored indirect comparison (Bucher/NMA, Chapter 12)
- Implement a basic matching-adjusted indirect comparison (MAIC) in R: estimate odds-weights via method of moments, compute effective sample size (ESS), and fit a weighted Cox model for time-to-event outcomes
- Explain why a large ESS loss after MAIC weighting is a credibility concern that must be reported, not a cosmetic detail
- Contrast MAIC's weighting-model assumption with simulated treatment comparison's (STC) outcome-regression-model assumption, and explain the non-collapsibility pitfall that threatens STC on nonlinear outcome scales
- Describe how multilevel network meta-regression (ML-NMR) generalises STC's logic to a full network of IPD and AgD-only trials via numerical integration over each AgD trial's reported covariate distribution, using the `multinma` package introduced in Chapter 12
- State NICE DSU TSD 18′s core requirements for a population-adjusted submission: prespecified effect modifiers, covariate-balance reporting, ESS reporting, and its stated method preference ordering
- Explain why an unanchored population-adjusted comparison carries materially higher risk of bias from unmeasured confounding than an anchored one, and why it should be a last resort

13.1 When population adjustment is needed

Chapter 12 closed with a warning worth repeating before we go further: standard network meta-regression is data-hungry. In practice it adjusts for covariate imbalance only at the **study** level, using trial-arm means as proxies for patient-level confounding and effect modification. That is a coarse instrument. Sometimes the trials you actually have won't cooperate with even that: one trial hands you full individual patient data (IPD) under a data-sharing agreement, while the comparator trial(s) exist to you only as a published table of aggregate data (AgD). Ecological regression on study-level covariates is often not possible in that situation, and even when it is, it is fragile.

This is the precise situation population-adjustment methods are built for: *one IPD trial, one or more AgD trials, connected through a common comparator, with a real concern that effect modifiers are distributed differently across the trial populations.* If the trials' populations are similar enough on anything that matters, ordinary anchored indirect comparison — Bucher's method or the NMA machinery of Chapter 12 — remains the right tool, and it is more efficient and more transparent than anything in this chapter. Population adjustment is a response to imbalance, not a default upgrade.

The methodological fork in the road is the distinction between *effect modifiers* and *prognostic factors*, and getting it right is not a technicality — it determines whether MAIC or STC is justified at all, and which covariates belong in the adjustment.

- A *prognostic factor* predicts outcome risk regardless of treatment: it shifts outcomes up or down in both arms more or less equally. Baseline disease severity, for instance, often predicts worse outcomes whether a patient is on the active drug or the comparator.
- An *effect modifier* changes the **relative** treatment effect itself: the treatment works differently (not just outcomes differently) depending on the covariate's value. Biomarker status that predicts a larger drug benefit in biomarker-positive patients is a classic effect modifier.

For an *anchored* comparison (both trials share a common comparator arm, and we are estimating a relative effect via that anchor), imbalance in pure prognostic factors across the two trial populations cancels out in the within-trial contrast — it does not bias the indirect comparison, because prognostic effects are differenced away on each side of the anchor before the two relative effects are compared. Imbalance in **effect modifiers**, by contrast, does not cancel: if the IPD trial's population has a different effect-modifier distribution than the AgD trial's population, the two trials' relative treatment effects are not estimating the same quantity, and the ordinary indirect comparison is biased. Population adjustment exists specifically to re-weight or re-model the IPD trial so that its effect-modifier distribution matches the AgD trial's, before the relative effect is computed.

> **Common pitfall.** A common and costly error is to throw every available baseline covariate into the matching or outcome-regression model — age, sex, disease duration, biomarker status, prior therapies — on the reasoning that "more adjustment is more rigorous." It is the opposite. Adjusting for a pure prognostic factor that is **not** an effect modifier does nothing for bias and only burns effective sample size (see §13.2) and inflates variance. Worse, for an *unanchored* comparison (single-arm or no shared comparator), the prognostic/effect-modifier distinction collapses — **all** prognostic factors and effect modifiers must be balanced, because there is no within-trial differencing to cancel prognostic imbalance. Decide the anchor status and the covariate roles (ideally pre-specified with clinical input, not chosen post hoc by which variables happen to "improve" the result) before fitting anything.

The two workhorse methods for this setting — matching-adjusted indirect comparison (MAIC) and simulated treatment comparison (STC) — differ in **how** they re-express the IPD trial in the AgD trial's population: one reweights patients, the other reshapes a regression model. We take each in turn.

13.2 Matching-adjusted indirect comparison (MAIC)

MAIC reweights the IPD-trial patients so that the **weighted** IPD population's covariate moments — typically the means, occasionally also variances for continuous covariates — match the moments reported in the AgD trial's publication (usually just means and sometimes SDs, since that is all an aggregate table ever gives you). The weights are not arbitrary: they come from a logistic-regression-style propensity model fit by method of moments, so that the resulting population looks, on the matched covariates, like the comparator trial's population.

The weighting mechanism. For each matched covariate x_j, define the IPD patient's covariate vector centered on the AgD trial's reported mean: $z_i = x_i - |x|^{\text{AgD}}$. MAIC assigns each IPD patient i an odds-weight

$$w_i = \exp(z_i^T \alpha)$$

where α is chosen so that the weighted means of z in the IPD data are exactly zero — i.e., the weighted IPD sample's covariate means equal the AgD trial's reported means. This is a method-of-moments problem solved by minimizing a convex objective (the method is numerically identical to calibrating a logistic propensity score without ever observing the AgD patients' outcomes or covariates individually — only their published moments are needed).

```
# Toy MAIC weight estimation via method of moments
# x: matrix of IPD covariates to match (e.g., age, baseline severity)
# target_means: named vector of AgD-trial reported means for the same covariates

Q <- function(alpha, z) {
  # z = x - target_means, centered IPD covariates
  sum(exp(z %*% alpha))
}

z <- scale(ipd[, c("age", "severity")], center = target_means, scale = FALSE)

fit <- optim(
  par = rep(0, ncol(z)),
  fn  = Q,
  z   = z,
  method = "BFGS"
)

alpha_hat <- fit$par
w <- exp(z %*% alpha_hat)        # odds-weights, one per IPD patient
w <- w / mean(w)                  # optional rescaling, mean weight = 1
```

How to report this.

Covariate	IPD, unweighted	IPD, MAIC-weighted	AgD trial (target)
Age, years (mean)	58.2	61.4	61.4
Baseline ECOG status ≥ 1, %	42	55	55
Prior-therapy count (mean)	1.3	2.1	2.1
After weighting, the IPD sample's matched covariate means equal the AgD trial's reported means by construction (method-of-moments calibration); any covariate omitted from the matching set is not guaranteed to balance and should be checked separately.			

Table 39: Baseline covariate balance between the IPD trial and the AgD trial, before and after MAIC odds-weighting

Effective sample size (ESS). Reweighting trades bias reduction for a loss of precision, and that loss must be quantified and reported, not buried in a supplementary table. The standard diagnostic is

$$\text{ESS} = \frac{\left(\sum_i w_i\right)^2}{\sum_i} w_i^2$$

which answers "how many **unweighted** patients would given this much statistical information?" A MAIC that starts with $N = 400$ IPD patients but produces $\text{ESS} = 120$ has thrown away roughly 70% of its effective information — a real quality-of-evidence red flag. A large ESS drop usually signals that the IPD and AgD populations barely overlap on the matched covariates, that a few patients are carrying almost all the weight, and that the resulting estimate's confidence interval will be much wider (and its extrapolation much less trustworthy) than the nominal sample size would suggest. Reviewers and HTA bodies expect ESS reported alongside the original N for exactly this reason.

```
ess <- sum(w)^2 / sum(w^2)
ess
```

HEOR in practice. Suppose an IPD trial of 400 patients is matched to an AgD trial on two effect modifiers: baseline ECOG status and prior-therapy count. After solving for the odds-weights, 15 patients at the extreme ends of the covariate distributions receive weights 8–12 times the mean weight, while the bulk of patients are weighted near zero. Plugging the resulting weights into the ESS formula gives $\frac{(\sum w_i)^2}{\sum} w_i^2 \approx 120$. The matched comparison should be reported with both numbers side by side — "IPD N = 400, ESS = 120 (30%)" — so a reviewer can immediately gauge how much of the original trial's information survived the reweighting, rather than comparing standard errors for the matched analysis on a seemingly unchanged 400-patient footing.

How to report this.

Quantity	Value	% of unweighted N
IPD sample size, N (unweighted)	400	100%
Effective sample size, ESS (MAIC-weighted)	120	30%

Report ESS alongside the raw N in every MAIC analysis; an ESS this far below N signals that a small number of extreme weights are driving the matched estimate and that its confidence interval will be wider than the nominal N suggests.

Table 40: Effective sample size before and after MAIC weighting (illustrative 400-patient IPD trial matched on two effect modifiers)

Time-to-event outcomes. MAIC's reweighting logic extends directly to survival outcomes by using the odds-weights w_i as case weights in a weighted Kaplan–Meier curve and a weighted Cox model — no new estimation machinery is needed, only weighted versions of the usual estimators.

```
library(survival)

# Weighted Cox model using MAIC odds-weights as case weights
cox_fit <- coxph(
  Surv(time, event) ~ treatment,
  data    = ipd,
  weights = w          # odds-weights from the MAIC step above
)

summary(cox_fit)

# Weighted Kaplan-Meier for the matched IPD population
km_fit <- survfit(Surv(time, event) ~ treatment, data = ipd, weights = w)
```

Robust (sandwich-type) standard errors are typically preferred over the naive weighted-Cox standard errors here, since the weights are themselves estimated quantities and ordinary Cox variance formulas do not account for that extra uncertainty; `coxph()` with `cluster()` or a bootstrap over the full weight-estimation-plus-outcome-model pipeline are the usual fixes, and this is worth flagging explicitly in any write-up.

Difficult terminology: robust (sandwich-type) standard errors. A standard error tells you how much a parameter estimate would bounce around if you repeated the study; the usual formula for calculating it assumes the model's error structure is exactly right. "Robust" or "sandwich-type" standard errors are a way of calculating that same uncertainty that doesn't lean on that assumption being correct — they're called "sandwich" because the formula literally sandwiches a middle piece (built from the observed data's actual variability) between two outer pieces derived from the model. Here, that matters because the MAIC weights fed into the Cox model aren't fixed, known numbers — they were themselves **estimated** from the matching step, carrying their own uncertainty, which an ordinary (non-robust) standard error would silently ignore, making the reported confidence interval too narrow and the result look more precise than it really is. Think of it as the statistical equivalent of accounting for the fact

> that your ruler itself has some wobble, rather than assuming every measurement taken with it is exact.

A practical note on tooling: there is no single dominant, actively maintained CRAN package that performs MAIC end to end. Most published MAICs are implemented by hand in R, essentially as the `optim()`-based weighting step above followed by a weighted outcome model. A `maic` package exists on GitHub and is used by some HTA submissions, but it is *not on CRAN*, has not been through CRAN's review process, and should be treated as unreviewed third-party code — inspect its weighting and variance-estimation internals line by line, or reproduce its output with your own `optim()` call, before relying on it for a submission.

13.3 Simulated treatment comparison (STC)

STC addresses the same imbalance problem as MAIC but by a different mechanism. Instead of reweighting patients, STC fits an *outcome regression model* on the IPD trial — the treatment effect plus the matched covariates as predictors — and then predicts what the outcome would have been had the IPD trial's population had the AgD trial's reported covariate means. Where MAIC asks "how do I reweight my patients to look like theirs?", STC asks "what does my outcome model say would happen to a population with their average characteristics?"

Concretely, for a continuous or binary outcome:

```
# Outcome regression model on the IPD trial, including a
# treatment x covariate interaction to capture effect modification
stc_fit <- glm(
  outcome ~ treatment * age + treatment * severity,
  data   = ipd,
  family = binomial()
)

# Predict the treatment effect at the AgD trial's reported means
newdata <- data.frame(
  treatment = c(0, 1),
  age       = target_means["age"],
  severity  = target_means["severity"]
)

preds <- predict(stc_fit, newdata = newdata, type = "link")
stc_log_odds_ratio <- preds[2] - preds[1]
```

How to report this.

Term	Estimate	SE	p
Treatment	0.41	0.18	0.023
Age (centered)	−0.02	0.01	0.085
Severity (centered)	−0.31	0.12	0.010
Treatment × age	0.015	0.019	0.431
Treatment × severity	0.28	0.11	0.012
Predicted log-odds ratio at the AgD trial's reported means (age = 61.4, severity index = 2.1): 0.52, i.e. OR ≈ 1.68 – a **conditional** effect at those covariate values, not yet the population-average effect; see the non-collapsibility caution above before reporting this as the final STC estimate.			

Table 41: Illustrative STC outcome-regression coefficients (logistic model, IPD trial, treatment × covariate interactions), and the resulting log-odds-ratio predicted at the AgD trial's reported covariate means

The appeal is efficiency: STC uses all N IPD patients in the regression rather than discarding information to an ESS-reduced weighted subsample, so it does not suffer MAIC's precision penalty from extreme weights. The cost is a stronger assumption: STC's validity depends on *correctly specifying the outcome model*, including its functional form, which covariates (and interactions) belong in it, and — critically for nonlinear models — the right scale on which to summarize the "effect at the mean."

Common pitfall. For a linear model on a continuous outcome, "predict the outcome at the covariate means" and "the average outcome across the covariate distribution" give the same answer, so STC is relatively safe there. For *nonlinear* outcome models — logistic regression on odds ratios, Cox regression on hazard ratios — they do not. The reason has a name: *non-collapsibility*. The conditional treatment effect (adjusted for covariates, evaluated at their mean) and the marginal treatment effect (averaged over the whole covariate distribution) are two different quantities on the odds-ratio or hazard-ratio scale. This is true even when the model is perfectly specified. So plugging the AgD trial's mean covariate values into a fitted logistic or Cox model, and reading off the "treatment effect," gives a **conditional** effect at those particular covariate values. It does not give the **population-average** effect that an aggregate-data trial reports — and that the indirect comparison needs to match. Treating these as interchangeable biases the comparison, even when every covariate is balanced and the outcome model is otherwise correct. The fix: simulate outcomes across the full joint covariate distribution implied by the AgD trial's reported summary statistics, average the simulated individual predictions on the natural outcome scale, then transform back to odds ratios or hazard ratios. This simulation step is the "simulated" in the method's name. Skipping it is the single most common STC implementation error.

The practical trade-off between the two methods comes down to **which** model you must get right. MAIC's validity rests on correctly specifying the *weighting/matching model* — which covariates drive the propensity-style weights — but it makes no assumption about the outcome's functional form; the weighted outcome analysis can be as simple as a weighted Kaplan–Meier curve. STC's validity rests on correctly specifying the *outcome regression model*, including its functional form and the covariate-by-treatment interactions that encode effect modification. But STC uses the full IPD sample, with no ESS penalty. Neither assumption dominates the other in general. The choice usually comes down to how much confidence you have in modelling the outcome mechanism versus modelling the covariate-balancing mechanism, and how severe the expected ESS loss would be under MAIC given how much the two trial populations actually overlap.

> **Agency perspective.** NICE DSU Technical Support Document 18 (Phillippo et al., 2016) is the standard methodological reference underpinning both MAIC and STC, and sets out NICE's expectations for reporting ESS, covariate selection, and sensitivity analyses in a submission. §13.5 of this chapter covers TSD 18′s guidance in full; treat the summary here only as a signpost to that fuller discussion.

13.4 Multilevel network meta-regression (ML-NMR)

MAIC and STC, as developed in §13.1–13.3, share a structural limitation: both operate on exactly two trials — one with individual patient data (IPD) and one reporting only aggregate data (AgD) — connected by a single comparison. The moment either trial sits inside a larger network of three or more treatments, these two-trial methods have no natural extension: you would need to run a separate MAIC or STC for every AgD trial against the IPD trial, throwing away the internal consistency that a proper network provides and giving no single coherent treatment-effect ranking across the whole network.

Multilevel network meta-regression (ML-NMR), developed by Phillippo, Ades, Dias, Welton, and colleagues (Phillippo et al. 2020, **Journal of the Royal Statistical Society Series A**), closes this gap. It generalises population adjustment from the two-trial case to a full network containing an arbitrary mix of IPD and AgD-only studies. The core idea extends STC's logic rather than replacing it: fit an individual-level regression model for the outcome — treatment, effect-modifying covariates, and treatment-by-covariate interactions — using whichever trials contribute IPD. For the trials that report only AgD, instead of plugging in a single covariate mean (as STC does), ML-NMR numerically integrates the individual-level prediction equation over each AgD trial's **reported covariate distribution**. When an AgD trial publishes not just means but also standard deviations and, ideally, correlations between effect modifiers, ML-NMR uses that fuller distributional information, rather than collapsing it to a point. This is what makes ML-NMR more defensible than moment-matching approaches when the data exist to support it — but also considerably more data-demanding, since it requires AgD trials to report enough distributional detail (not just means) for the covariates that matter.

ML-NMR is not a new tool bolted onto this book's toolkit — it is the same `multinma` package (Phillippo et al.) already introduced in Chapter 12 §12.4 for Bayesian network meta-analysis, used in exactly the same workflow. There, you built a network with `set_ipd()` and `set_agd_arm()` and fit it with `multinma::nma()` assuming no effect modification. ML-NMR is what happens when that regression model is allowed to include covariate-by-treatment interaction terms, and the AgD arms

are no longer treated as covariate-free: each AgD study additionally needs numerical integration points describing its covariate distribution so that the individual-level model's predictions can be correctly marginalised to that study's population. The documented function for generating these integration points is `multinma::add_integration()`, which builds Quasi-Monte Carlo integration points (via a Gaussian copula and Sobol' sequences) from each AgD study's reported covariate means, SDs, and — where available — correlations.

Difficult terminology: Sobol' sequences / Quasi-Monte Carlo integration points. "Numerically integrating" over a covariate distribution just means: rather than plugging in one single average value (as STC does), generate a large number of representative sample points that collectively cover the whole range of plausible covariate values for that trial's population, and average the model's prediction over all of them. Ordinary Monte Carlo would do this with purely random points, which cluster unevenly by chance (like randomly scattering dots on a dartboard — some areas get too many, some too few). A Sobol' sequence instead generates points in a carefully structured, non-random pattern that spreads them out evenly across the space — closer to laying dots on a grid, but cleverer, so that even a modest number of points gives a good, stable approximation of the integral. "Quasi-Monte Carlo" is the general name for this family of structured-point integration techniques, and Sobol' sequences are simply the most common recipe used to generate the points.

Difficult terminology: Gaussian copula. When an AgD trial reports several covariates' means, SDs, and the correlations between them (e.g. age and disease severity tend to rise together), you need a way to generate **jointly** realistic sample values for all the covariates at once — not just realistic values for each covariate separately, ignoring how they relate to each other. A Gaussian copula is the mathematical device that does this: it takes each covariate's own individually-specified distribution (which doesn't have to be Normal at all) and "glues" them together using a correlation structure borrowed from the multivariate Normal distribution, producing joint sample points that respect both each covariate's own shape and the reported correlations between them. A rough analogy: imagine you know the typical height and weight distributions separately, plus the fact that taller people tend to be heavier — a copula is the recipe for generating pairs of (height, weight) values that look like genuine people, rather than randomly mixing a height from one person with a weight from someone unrelated.

A schematic workflow, keeping the illustrative code deliberately light and consistent with the Chapter 12 pattern:

```
library(multinma)

# Network combining IPD and AgD-only trials, as in Ch.12 §12.4
net <- combine_network(
  set_ipd(ipd_trial, study = study, trt = trt, y = outcome,
          trt_class = NULL),
  set_agd_arm(agd_trials, study = study, trt = trt,
              r = events, n = n)
)

# Supply numerical integration points for each AgD trial's
```

```
# covariate distribution -- consult the current multinma
# documentation for the exact distribution-argument syntax,
# which has evolved across package versions.
net_int <- add_integration(
  net,
  age = distr(qnorm, mean = age_mean, sd = age_sd),
  ecog = distr(qbern, prob = ecog_prop)
)

# Fit the regression model with effect-modifier interactions
ml_nmr_fit <- nma(net_int,
                  trt_effects = "fixed",
                  regression = ~ age + ecog,
                  class_interactions = "common")
```

The call shapes for `set_ipd()`, `set_agd_arm()`, and `nma()` mirror Chapter 12 exactly; `add_integration()` is the one genuinely new step, and its covariate-distribution argument syntax has changed across `multinma` releases, so treat the `distr(qnorm, ...)` style above as illustrative rather than copy-ready — always check the installed version's help page and vignette before running a submission analysis.

> **HEOR in practice.** Think of ML-NMR as "STC done properly across a whole network." STC matches an AgD trial to one covariate value (its mean); ML-NMR matches it to the trial's **entire reported covariate distribution**, and does so simultaneously for every AgD trial in the network while preserving one internally consistent set of relative treatment effects. The price is data: ML-NMR only delivers its full benefit when AgD trials publish SDs and, where effect modifiers interact, correlations — not just means. When that reporting is thin, ML-NMR can gracefully fall back toward STC-like behaviour, but its defensibility advantage shrinks accordingly.

Method	Data requirement	Key assumption	Typical use case
MAIC	IPD for one trial + AgD (covariate means, sometimes SDs) for the comparator trial	Weighting/matching model is correctly specified; matched covariates include all relevant effect modifiers	Simple two-trial comparison with an IPD submission trial and one published AgD comparator, especially for survival outcomes via weighted Kaplan–Meier/Cox
STC	IPD for one trial + AgD (covariate means) for the comparator trial	Outcome regression model is correctly specified, including functional form and treatment-by-covariate interactions; non-collapsibility handled via simulation on nonlinear scales	Two-trial comparison where the analyst has strong confidence in the outcome model and wants to retain full IPD precision (no ESS penalty)
ML-NMR	IPD for some trials + AgD (covariate means, SDs, and ideally correlations) for the rest, across a full network	Individual-level regression model is correctly specified; AgD trials report enough distributional detail for numerical integration	Networks of three or more treatments mixing IPD and AgD-only trials, where a single internally consistent set of relative effects is required

Table 42: Comparison of the three population-adjustment methods covered in this chapter: data requirements, core assumption, and typical use case

13.5 Limitations, assumptions and agency expectations (NICE DSU TSD 18)

Every method introduced in this chapter — MAIC, STC, and ML-NMR — answers the same underlying problem (effect-modifier imbalance across trials that cannot be resolved by a standard anchored NMA or Bucher indirect comparison) with different trade-offs between data requirements and defensibility. NICE DSU Technical Support Document 18, **Methods for Population-Adjusted Indirect Comparisons in Submissions to NICE** (Phillippo, Ades, Dias, Palmer, Abrams, Welton; first published 2016, subsequently updated), is the standard reference that HTA bodies and manufacturers alike use to judge whether a population-adjusted analysis is appropriate and well-executed. It is the single most important named document in this chapter, and no submission using MAIC, STC, or ML-NMR should be prepared without direct reference to it.

Agency perspective. What TSD 18 actually requires, in outline:

- **Justify the need for adjustment.** A standard NMA or Bucher comparison is the default; population adjustment is only warranted when the company can demonstrate an effect-modifier imbalance problem between the trial populations — not simply assert that one might exist.

- **Prespecify effect modifiers before seeing comparative results.** The list of covariates to adjust for must be clinically justified and fixed in advance (via literature review, clinical input, or prior trials), independent of the analysis's own outcome.
- **Report covariate balance before and after weighting** (for MAIC) so reviewers can see what imbalance existed and how much of it was corrected.
- **Report the effective sample size (ESS)** after weighting and discuss what an ESS loss implies for the precision and credibility of the result.
- **State a method preference, data permitting.** TSD 18′s stated ordering, roughly, is ML-NMR > STC > MAIC: ML-NMR preserves a network's internal consistency and uses the full covariate distribution; STC retains the IPD trial's own outcome model and standard errors; MAIC, by reweighting the IPD sample to match only the AgD trial's reported moments, discards information and can inflate variance by effectively down-weighting the IPD trial's within-trial precision. Choose MAIC because it is simplest to implement, not because it is DSU 18′s preferred option.

Limitation	TSD 18's stated concern
Target-population specificity	An adjusted estimate is valid only for the population it was adjusted to (typically the AgD trial's population); it does not carry the transportability assumption of a standard NMA, so a different target population requires a fresh adjustment
Unmeasured confounding in unanchored comparisons	Without a common comparator to absorb unmeasured differences, an unanchored analysis implicitly assumes every effect modifier and prognostic factor has been measured and correctly adjusted for – rarely fully defensible; unanchored designs carry materially higher bias risk than anchored ones
Post-hoc effect-modifier selection	Choosing or dropping covariates after seeing how they change the adjusted result is a recognised form of analytical bias; the effect-modifier list must be prespecified from external clinical and literature evidence before the adjusted analysis is run
ESS loss (MAIC specifically)	A large drop in effective sample size after weighting signals that a small number of extreme weights are driving the result and that the reported confidence interval may understate true uncertainty; ESS must be reported prominently, not in an appendix
Thin AgD reporting	ML-NMR's advantage over STC depends on AgD trials publishing covariate SDs and, where relevant, correlations – not just means; when that detail is absent, ML-NMR degrades toward STC-like behaviour and its defensibility edge shrinks

Table 43: NICE DSU TSD 18: named limitations of population-adjustment methods and the agency's stated concern for each

Common pitfall. Post-hoc effect-modifier selection. Choosing or dropping candidate effect modifiers **after** looking at how they change the adjusted treatment effect is a well-recognised form of analytical bias, and it is explicitly criticised by the DSU and by NICE Evidence Review Groups (ERGs) when detected. It lets an analyst chase the covariate set that produces the most favourable result while presenting it as a neutral, methods-driven choice. The defence is procedural, not statistical: fix the effect-modifier list from external clinical and literature evidence **before** running the adjusted analysis, document that process in the submission, and resist any temptation to revise the list once results are in hand.

Agency perspective. ESS as a credibility signal, not a footnote. For MAIC in particular, a large drop in effective sample size after weighting (e.g., an ESS that falls to a small fraction of the original IPD sample size) is not a cosmetic detail — ERGs treat it as a direct warning

sign that the weights are being driven by a small number of extreme observations, that the weighted estimate's reported confidence interval may understate true uncertainty, and that the result may not generalise robustly to the target population. Submissions should report ESS prominently alongside the adjusted estimate, not bury it in an appendix, and should discuss explicitly what the ESS loss implies for the analysis's reliability — silence on this point is itself a flag reviewers are trained to look for.

Two further limitations apply across all three methods and deserve to be stated plainly rather than left implicit.

First, **a population-adjusted estimate is only valid for the one target population it was adjusted to** — typically the AgD trial's population, since that is the population whose covariate distribution (or moments) the IPD trial was reweighted or regressed onto. This is a fundamentally different assumption from a standard NMA's transportability of relative effects, which assumes (rightly or wrongly) that a relative treatment effect generalises across reasonably similar populations. A MAIC-, STC-, or ML-NMR-adjusted estimate carries no such claim: adjust the same IPD trial to a different AgD trial's population and you may well get a different answer, and that is expected, not a contradiction. Decision-makers who want an estimate for a population other than the one used in adjustment need a fresh adjustment, not a reuse of the existing one.

Second, **unanchored comparisons deserve particular caution**. An anchored comparison (MAIC, STC, or ML-NMR performed within a network that still has a common comparator connecting the trials) uses that common comparator to cancel out between-trial differences that are not captured by the measured covariates — the anchor absorbs some of the risk from unmeasured confounding. An unanchored comparison, by contrast, arises when no such common-comparator link exists at all, so the entire burden of adjustment falls on the outcome regression model with no anchor to offset what that model misses.

Common pitfall. Unanchored comparisons are a last resort, not a default. TSD 18 is explicit that unanchored population-adjusted comparisons carry a materially higher risk of bias from unmeasured confounding and unmeasured effect modification than their anchored counterparts, because there is no common-comparator trial arm to absorb whatever the adjustment model fails to capture. An unanchored analysis implicitly assumes that **all** effect modifiers and prognostic factors have been measured and correctly adjusted for — an assumption that is rarely fully defensible. Reach for an unanchored design only when no anchored route exists, and when you do, report the limitation transparently and prominently in the submission rather than presenting the result with the same confidence as an anchored estimate.

Chapter 14. Bayesian Methods for HEOR

Chapter 12 used Bayesian network meta-analysis (`gemtc`, `multinma`, `bnma`) and flagged a `gelman.diag()` convergence check as mandatory. It did not explain **why** that check matters, or what to do when it fails. That explanation was deliberately saved for here.

This chapter has two jobs. First, it builds the Bayesian framework from first principles — what a prior, a likelihood and a posterior actually are, and how to compute one. Second, it fulfils Chapter 12′s promise: full MCMC diagnostics, done properly. Along the way it reaches the chapter's single most useful idea for everyday modelling work: a Bayesian posterior sample can be used **directly** as a PSA sample, with no extra distribution-fitting step in between.

The chapter follows a simple arc, section by section:

- §14.1-14.2 build the concepts: what Bayesian updating is, and how to choose a prior.
- §14.3 introduces the three tools that turn those concepts into actual posterior draws: Stan, JAGS, and `brms`.
- §14.4 is the diagnostics discipline — the part Chapter 12 deferred.
- §14.5-14.6 connect evidence synthesis to decision-making: turning posterior draws into PSA input.
- §14.7 covers expert elicitation (SHELF), for when there is no data at all.
- §14.8 closes with a roadmap tying every piece together, and pointing forward to Part IV.

Learning objectives

- Explain the prior/likelihood/posterior relationship and contrast a Bayesian credible interval's interpretation with a frequentist confidence interval's interpretation
- Choose between informative, weakly informative and vague priors for a given parameter, and justify the choice with a documented rationale and a prior sensitivity check
- Fit a simple Bayesian model using `rstan`, `rjags`, and `brms`, and explain when each tool is the natural choice
- Explain Metropolis-Hastings, Hamiltonian Monte Carlo and the No-U-Turn Sampler at a conceptual level sufficient to interpret why some models converge faster than others
- Check MCMC convergence using the Gelman-Rubin statistic (R-hat), assess effective sample size, and run a posterior predictive check before trusting any posterior summary
- Describe Bayesian evidence synthesis as a pipeline that produces a joint posterior sample, building on Chapter 12′s Bayesian NMA machinery without re-deriving it
- Use posterior draws directly as a PSA sample for a cost-effectiveness decision model, preserving parameter correlation structure without fitting an intermediate distribution
- Describe the Sheffield Elicitation Framework (SHELF) process for eliciting a parameter's distribution from expert judgement when no usable data exist, and use the `SHELF` R package to fit and inspect the resulting distribution
- Trace the end-to-end Bayesian HEOR workflow from prior specification through evidence synthesis, MCMC fitting, diagnostics and PSA to reporting, and locate where it hands off to Part IV's uncertainty (Ch.25) and value-of-information (Ch.26) chapters

14.1 Bayesian thinking for decision makers

Every health economic model you have built so far in this book has used probability in a particular way. A transition probability, a hazard ratio, a utility weight — each has been treated as a fixed, if uncertain, number, and you have expressed that uncertainty by attaching a distribution to it and running probabilistic sensitivity analysis. That workflow already contains a Bayesian idea, even if nobody called it that: a distribution over a parameter represents what you believe about a quantity you cannot observe directly. This section makes that idea explicit and gives it a name, because from here through the rest of the book — cost-effectiveness model parameterisation, value of information, adaptive trial designs — Bayesian reasoning is the natural language for decisions made under uncertainty.

Start with three ingredients.

A *prior*, $\pi(\theta)$, is what you believe about a parameter θ before looking at today's data. It might come from an earlier trial, an expert elicitation exercise, or simply a wide, non-committal range.

A *likelihood*, $L(\theta \mid y)$, is how probable the data y you actually observed would be, for each possible value of θ. This is the same likelihood function behind your maximum-likelihood fits from earlier chapters.

Combine the two with Bayes' theorem and you get the *posterior*, $p(\theta \mid y)$ — your updated belief about θ, after seeing the data:

$$p(\theta \mid y) = \frac{L(y \mid \theta)\pi(\theta)}{\int L(y \mid \theta)\pi(\theta)\,\mathrm{d}\theta}$$

The denominator just rescales things so the posterior integrates to one. The relationship you actually use day to day is much simpler:

$$\text{posterior} \propto \text{likelihood} \times \text{prior}$$

Everything in this chapter is an exercise in computing or approximating that product. Picture a tug-of-war: the likelihood pulls toward what the current data say; the prior pulls toward what was already believed; and the strength of each pull depends on how much information each one carries. A small pilot study of 20 patients has a weak, spread-out likelihood — a well-chosen prior can meaningfully sharpen the estimate. A large national surveillance dataset has a sharp likelihood that will dominate almost any reasonable prior.

Contrast this with the frequentist framing you have used for most of this book. A frequentist 95% confidence interval is a statement about the long-run behaviour of an estimation *procedure*: if you repeated the same sampling process many times, 95% of the intervals you construct would contain the true parameter. It is not a statement of 95% probability about the parameter itself, no matter how often it is informally described that way. A Bayesian 95% credible interval, by contrast, *is* a direct probability statement: given this data and this prior, there is a 95% probability that θ lies in this interval. This is not a technicality — it is exactly the kind of statement a reimbursement committee, a Ministry of Health budget holder, or a WHO technical working group actually wants to hear, and it is why Bayesian output translates so cleanly into decision language without the usual hedging.

This also explains why Bayesian methods sit so naturally inside decision modelling. A cost-effectiveness analysis is ultimately a decision problem: given everything currently known, should

we recommend this intervention? The expected net monetary benefit, the probability that an intervention is cost-effective at a given threshold, the expected value of perfect information — all of these are posterior quantities, computed by integrating a decision function over a distribution of parameter uncertainty. When you ran probabilistic sensitivity analysis in earlier chapters by sampling parameters from distributions and tabulating the proportion of simulations below a willingness-to-pay threshold, you were already doing applied Bayesian decision analysis, just without a likelihood update step. The step this chapter adds is the machinery for formally updating those parameter distributions with real trial or surveillance data, rather than specifying them by assumption or extraction from a single published estimate.

There is also a scientific-honesty argument for working this way in TB and other high-burden, data-scarce programmes. Frequentist methods implicitly treat each study in isolation, re-estimating uncertainty from scratch each time. Bayesian updating lets you formally carry forward what a previous regional drug-resistance survey or an earlier cohort's treatment-success rate told you, combine it transparently with new programme data, and report exactly how much the new evidence moved your estimate. That transparency — prior stated, likelihood stated, posterior shown — is precisely the kind of audit trail that evidence reviewers increasingly expect, and it is the foundation the rest of this chapter builds on.

> **HEOR in practice.** When presenting Bayesian results to a non-technical decision committee, lead with the posterior probability statement ("there is an 87% probability that the intervention is cost-effective at this threshold") rather than the mechanics of the prior-likelihood combination. Reserve the prior/likelihood/posterior language for technical appendices or methods sections aimed at reviewers, where it needs to be fully transparent and reproducible.

> **Agency perspective.** NICE DSU Technical Support Document 18 (**Methods for Population-Adjusted Indirect Comparisons**), which you met in Chapter 13, explicitly recommends Bayesian estimation for several population-adjustment methods because it propagates parameter uncertainty coherently into the final decision metric without relying on asymptotic normality assumptions that can break down in small populated subgroups — exactly the situation common in TB sub-population analyses.

14.2 Priors: informative, weakly informative and vague priors

Choosing a prior is the step in Bayesian analysis that makes newcomers most nervous, usually because it feels like injecting a subjective opinion into what should be an objective calculation. The more useful framing is that every statistical model already makes assumptions — a frequentist generalised linear model assumes a link function, an error distribution, and an absence of unmeasured confounding, none of which are "objective" in any strong sense. A prior simply makes one particular assumption explicit, documented, and open to sensitivity testing, which is in many ways more honest than burying equivalent assumptions in model structure.

Informative priors encode substantial prior knowledge about a parameter, usually from a previous study, a meta-analysis, or formal expert elicitation. If a prior national TB survey found treatment success in 78% of patients with a standard error corresponding to a $\text{Beta}(150, 42)$ distribution, and you are now analysing a new but related cohort, using that as your prior is not

cheating — it is using the discipline's accumulated evidence rather than discarding it. The key discipline is documenting *where the prior came from* and *why it is relevant to the new analysis*, exactly as you would document an extrapolation assumption in a Markov model.

Weakly informative priors rule out implausible values without expressing a strong opinion among the plausible ones. For a log hazard ratio, a $\text{Normal}(0, 2.5)$ prior on the log scale is weakly informative: it says hazard ratios somewhere between roughly 0.007 and 150 are conceivable, but a hazard ratio of a trillion is not, which keeps the sampler well-behaved and rules out numerically absurd solutions without pulling the estimate toward any particular value. These are the default workhorse priors in most modern Bayesian regression work, including the `brms` defaults you will use in §14.3.

Vague (or "flat") priors attempt to express complete prior ignorance, for example a $\text{Uniform}(-\infty, \infty)$ prior on a regression coefficient, or a $\text{Beta}(1, 1)$ — equivalent to $\text{Uniform}(0, 1)$ — on a probability. In simple, well-identified models these behave reasonably and the posterior is driven almost entirely by the likelihood. But truly flat priors on an unbounded scale are improper (they do not integrate to one) and can cause real problems in more complex hierarchical models, where they place surprising amounts of prior mass on extreme values once transformed through a nonlinear link function. In practice, a weakly informative prior usually does the job a vague prior was trying to do, with fewer pathologies.

A practical distinction worth internalising is **conjugate versus non-conjugate priors**. A prior is conjugate to a likelihood when the resulting posterior belongs to the same distributional family as the prior, which makes the update available in closed form. The classic HEOR example is the Beta-Binomial pair: if θ is a probability (say, treatment success) with prior $\theta \sim \text{Beta}(\alpha, \beta)$, and you observe y successes out of n trials, the posterior is simply

$$\theta \mid y \sim \text{Beta}(\alpha + y, \beta + n - y)$$

no simulation required. You can compute and plot this directly in R:

```
# Beta-Binomial conjugate update: TB treatment success rate
alpha_prior <- 150   # prior "successes"
beta_prior  <- 42    # prior "failures"

y <- 34   # observed successes in new cohort
n <- 40   # total patients in new cohort

alpha_post <- alpha_prior + y          # 184
beta_post  <- beta_prior + (n - y)     # 48

post_mean   <- alpha_post / (alpha_post + beta_post)   # 0.793
post_median <- qbeta(0.5, alpha_post, beta_post)
cri_95      <- qbeta(c(0.025, 0.975), alpha_post, beta_post)

cat(sprintf("Posterior mean: %.3f\n", post_mean))
cat(sprintf("95%% credible interval: [%.3f, %.3f]\n", cri_95[1], cri_95[2]))
```

With $\alpha_{\text{prior}} = 150$, $\beta_{\text{prior}} = 42$, $y = 34$ and $n = 40$, the update gives $\alpha_{\text{post}} = 150 + 34 = 184$ and $\beta_{\text{post}} = 42 + (40 - 34) = 48$, so the posterior mean is $184/(184 + 48) = 0.793$ — barely moved from the prior mean of $150/192 = 0.781$, because the new cohort of 40 patients carries far less

information than the 192 "pseudo-observations" already encoded in the prior. This is exactly the tug-of-war intuition from §14.1 made concrete.

How to report this.

Quantity	Prior	Posterior
Distribution	Beta(150, 48)	Beta(184, 48)
Mean	0.781	0.793
95% interval	(0.714, 0.841)	(0.738, 0.843)

Table 44: Posterior summary for TB treatment success probability, Beta-Binomial conjugate update ($n = 40$ new cohort patients, $y = 34$ successes, informative prior $\text{Beta}(150, 42)$)

Other conjugate pairs worth knowing are Normal-Normal (for a mean with known variance) and Gamma-Poisson (for a rate). Conjugacy is a mathematical convenience, not a requirement — most realistic HEOR models (hierarchical survival models, mixed-effects utility regressions, network meta-analyses with covariate adjustment) have no conjugate solution, which is exactly why §14.3 introduces MCMC: it lets you approximate the posterior numerically when no closed form exists.

Whatever prior family you choose, **always run a prior sensitivity analysis**. Refit the model under at least two or three materially different, individually defensible priors (for example, your informative prior, a weakly informative alternative, and a vague one) and report whether the posterior conclusions change. If the data are informative relative to the prior, the posteriors will look similar regardless of which prior you used, and you can say so with confidence. If they diverge noticeably, that divergence is itself an important finding — it tells the decision maker that current evidence is not yet strong enough to overrule prior belief, which is a very different message than reporting a single, confidently-stated number.

Common pitfall. Do not reach for a "flat" or improper prior reflexively just to seem neutral. On transformed scales (odds ratios, hazard ratios, variance components) a flat prior on one scale can be very informative — and in a distorting way — on another. A Uniform prior on an odds ratio from 0 to 1000, for instance, puts almost all of its prior mass above an odds ratio of 10, which is rarely a neutral starting point for a clinical effect.

HEOR in practice. Report your prior choices in a dedicated subsection of the methods, including the exact distributional form and parameters, the source or rationale, and the results of at least one prior sensitivity check. This is now standard expectation in Bayesian HTA submissions and mirrors the transparency already required for deterministic and probabilistic sensitivity analysis assumptions elsewhere in your model.

14.3 MCMC with Stan, JAGS and brms

Conjugate models give you an exact posterior in closed form, but almost no model you will actually need for TB programme evaluation or cost-effectiveness analysis is that convenient. Hierarchical

structures, correlated random effects, non-standard likelihoods, and covariate-adjusted network meta-analyses (as in Chapter 12) all produce posteriors with no analytic solution. Markov Chain Monte Carlo (MCMC) is the computational workaround: instead of solving for the posterior algebraically, you draw a long sequence of correlated samples whose distribution converges to the posterior you want, and then you summarise those samples the same way you would summarise any other distribution of draws — means, quantiles, credible intervals.

The core idea behind every MCMC algorithm is the same: build a Markov chain — a sequence of parameter values where the next value depends only on the current one — whose long-run behaviour matches the posterior you want. Then just run it for a long time and summarise the values it visits.

The oldest practical version, **Metropolis-Hastings**, proposes a candidate next value by perturbing the current one, then accepts or rejects it with a carefully chosen probability. Over many iterations, the chain visits each region of parameter space in proportion to its posterior density. It is simple and general, but can be slow: small random proposals take a long time to cross large, high-dimensional spaces.

Hamiltonian Monte Carlo (HMC) speeds this up with an idea borrowed from physics. It treats the negative log-posterior as a hilly energy surface and simulates a particle rolling across it with momentum, using the surface's **gradient** to steer proposals toward high-density regions instead of wandering randomly. Big, informed steps even in high dimensions — that is why HMC scales so much better to the complex hierarchical models common in HEOR.

NUTS, the No-U-Turn Sampler, is the practical refinement of HMC that Stan actually runs. It automatically decides how far the simulated particle should travel before it starts turning back on itself, so you never have to hand-tune a step count. You do not need to implement any of this maths yourself — Stan's compiler and sampler handle it — but the intuition explains why Stan models typically reach the same effective sample size in fewer iterations than JAGS models. §14.4 picks that thread back up.

JAGS (Just Another Gibbs Sampler) — the engine behind `gemtc` in Chapter 12 — instead typically uses **Gibbs sampling**: a special case of Metropolis-Hastings that updates one parameter at a time, drawing each directly from its full conditional distribution given every other parameter's current value. This fits naturally with conjugate or near-conjugate hierarchical structures, which is part of why JAGS remains common in network meta-analysis work. It can converge more slowly than HMC/NUTS, though, when parameters are highly correlated.

It helps to see all three engines fit the same simple model: a Beta-Binomial treatment-success analysis, extended slightly to a logistic-regression-style setting so the code generalises to more realistic use. Start with raw Stan via `rstan`, which requires writing the model explicitly in Stan's own modelling language and compiling it:

```
library(rstan)

stan_code <- "
data {
  int<lower=0> n;      // number of patients
  int<lower=0> y;      // number of successes
}
parameters {
  real<lower=0, upper=1> theta;   // treatment success probability
```

```
}
model {
  theta ~ beta(2, 2);    // weakly informative prior
  y ~ binomial(n, theta);
}
"

stan_data <- list(n = 40, y = 34)

fit_stan <- stan(
  model_code = stan_code,
  data       = stan_data,
  chains     = 4,
  iter       = 2000,
  warmup     = 1000,
  seed       = 123
)

print(fit_stan, probs = c(0.025, 0.5, 0.975))
```

The same model in JAGS, using `rjags`, requires its own BUGS-style model syntax and a slightly more manual sampling loop built from `jags.model()` followed by `coda.samples()`:

```
library(rjags)
library(coda)

jags_code <- "
model {
  theta ~ dbeta(2, 2)
  y ~ dbin(theta, n)
}
"
writeLines(jags_code, con = "beta_binom.jags")

jags_data <- list(n = 40, y = 34)

jags_mod <- jags.model(
  file     = "beta_binom.jags",
  data     = jags_data,
  n.chains = 4,
  n.adapt  = 1000
)

update(jags_mod, n.iter = 1000)   # burn-in

fit_jags <- coda.samples(
  jags_mod,
  variable.names = "theta",
  n.iter         = 2000
)

summary(fit_jags)
```

Finally, `brms` sits on top of Stan and lets you specify most standard regression-style models using familiar R formula syntax, generating and compiling the underlying Stan code automatically:

```r
library(brms)

tb_df <- data.frame(
  success = 34,
  total   = 40
)

fit_brms <- brm(
  success | trials(total) ~ 1,
  data   = tb_df,
  family = binomial(),
  prior  = set_prior("normal(0, 2.5)", class = "Intercept"),
  chains = 4,
  iter   = 2000,
  warmup = 1000,
  cores  = 4,
  seed   = 123
)

summary(fit_brms)
plot(fit_brms)
```

Note the prior is set on the intercept on the logit scale here rather than directly on θ, since `brms` builds a generalised linear model internally — this is a common source of confusion for analysts moving from hand-coded Beta-Binomial models to formula-based Bayesian regression, and worth testing on a known conjugate example (as above) before trusting it on a model with no closed-form answer.

Each tool has a natural niche. `rstan` gives you full control and is the natural choice when you need a custom likelihood that neither JAGS's BUGS dialect nor `brms`'s formula interface can express. `rjags` remains valuable for Gibbs-friendly hierarchical structures and for continuity with network meta-analysis workflows already built on JAGS, as in Chapter 12′s `gemtc` models. `brms` is usually the fastest path for standard regression-style problems — mixed-effects survival models, multilevel utility regressions, dose-response curves — because it handles the Stan code generation, prior specification helpers, and post-processing for you, while still letting you drop down to the underlying `stanfit` object (via `fit_brms$fit`) when you need finer control. For most day-to-day HEOR regression work, start with `brms`; reach for raw `rstan` or `rjags` when the model structure genuinely demands it.

> **Common pitfall.** Running an MCMC model and immediately trusting `summary()` output without checking convergence is the single most common Bayesian analysis error. The `print()` and `summary()` calls above already return `Rhat` and effective sample size alongside the posterior estimates — do not report a posterior mean or credible interval until those diagnostics have been checked, which §14.4 covers in full, building on the $\hat{R} < 1.05$ convergence rule you already applied to `gemtc` and `multinma` network meta-analysis models in Chapter 12.

14.4 Diagnostics: convergence, effective sample size, and posterior predictive checks

Running an MCMC sampler and getting numbers back is the easy part. Trusting those numbers is the job. In Chapter 12 you fit Bayesian network meta-analyses with `gemtc`, `multinma`, and `bnma`, and each time we flagged a `coda::gelman.diag()` check and moved on, promising we'd come back to what it actually means. This is that promise kept. Diagnostics are not a bureaucratic box to tick before an HTA submission reviewer asks for them — they are the difference between a posterior distribution that describes your evidence and one that describes an algorithm that got stuck.

Why convergence is not optional

An MCMC chain is only a valid sample from the posterior once it has "forgotten" its starting value and is exploring the full shape of the distribution the way it would if you had started it somewhere else entirely. Before that point, the chain is still wandering through unrepresentative territory — the burn-in or warm-up period. The trouble is that a chain can *look* perfectly healthy on a trace plot in the short run while still being far from target, especially in high-dimensional models like network meta-analyses with many treatment contrasts and heterogeneity parameters.

The standard first line of defence is to run several chains (`coda`, `rstan`, and `rjags`/`gemtc` all default to this) from dispersed starting values and ask whether they have converged to the *same* distribution. If chain 1 is sampling a different region of parameter space than chain 2, you don't have one posterior, you have evidence that your sampler hasn't settled.

```
library(coda)

# mcmc.list object from gemtc, rjags, or similar
gelman.diag(mcmc_samples, multivariate = FALSE)
```

This returns the potential scale reduction factor (PSRF), commonly called the R-hat or Gelman-Rubin statistic, for every monitored parameter. Conceptually, R-hat compares the variance *between* chains to the variance *within* chains. If the chains have converged, these two variances should agree and R-hat should sit at 1. In practice:

- R-hat < 1.05 is the traditional "acceptable" threshold used in most applied Bayesian HTA work, including the `gemtc`/NICE DSU NMA tradition you used in Chapter 12.
- R-hat < 1.01 is the more conservative threshold recommended by newer Stan-ecosystem guidance and increasingly expected by methodologically careful reviewers, particularly for models going into a regulatory or reimbursement submission.

> **Common pitfall.** A single R-hat below threshold tells you nothing if you only checked one parameter. Always check *all* monitored parameters, not just the headline treatment effect — heterogeneity variance terms and random-effects parameters in NMA models are notoriously the last to converge, and they drive the width of your decision uncertainty downstream.

Trace plots remain the most intuitive diagnostic precisely because R-hat is a *summary* and summaries hide failure modes. A chain can have an acceptable R-hat and still show a slow drift, a sticky patch, or a second chain exploring a completely different mode of a multimodal posterior — all of which are visually obvious on a trace plot and easy to miss in a single number.

```r
library(bayesplot)
library(rstan)

fit <- stan(file = "nma_model.stan", data = stan_data,
            chains = 4, iter = 4000, warmup = 2000)

posterior_array <- as.array(fit)
mcmc_trace(posterior_array, pars = c("d[1]", "d[2]", "tau"))
```

A healthy trace plot looks like a "fat hairy caterpillar" — four overlapping bands with no trend, no chain visibly separated from the others, and no long flat stretches where the sampler got stuck.

Effective sample size: why 20,000 draws can be worth 200

Here is the point that trips up most analysts moving from frequentist bootstrapping to MCMC: *the number of post-warm-up iterations you ran is not the number of independent pieces of information you have.* MCMC draws are autocorrelated — each draw depends on the one before it — so a chain of 10,000 iterations might carry the statistical information of only a few hundred truly independent draws if the sampler mixes slowly.

The effective sample size (ESS) corrects for this autocorrelation and tells you, in effect, "how many independent draws would give you the same precision as the correlated chain you actually have."

```r
effectiveSize(mcmc_samples)
```

Inside the Stan ecosystem, `rstan::monitor()` reports `n_eff` alongside R-hat for exactly this reason — the two diagnostics are reported together because they answer different questions. R-hat asks "have my chains converged to the same place?" ESS asks "once converged, how much *information* do I actually have about this parameter?" You can have excellent R-hat with dismal ESS if chains converge quickly to the right region but then mix very slowly within it — a common pattern for variance components in hierarchical NMA models.

As a rule of thumb, treat ESS below roughly 400 (across all chains combined) for any parameter you plan to report as a trigger to run the model longer or reparameterise it, regardless of how good your R-hat looks. For Stan models specifically, also check sampler-level diagnostics beyond R-hat and ESS — divergent transitions, which indicate the sampler is struggling with the geometry of the posterior (common with funnel-shaped hierarchical priors):

> **Difficult terminology: funnel-shaped hierarchical priors (Neal's funnel).** In a hierarchical model, individual-level effects (e.g. each study's own effect) are usually written as "the overall average, plus a study-specific deviation whose size is controlled by one variance parameter." The trouble is that this variance parameter and the individual deviations are mathematically tangled together: when the variance parameter is estimated to be small, the deviations are forced to be tiny too, and when it's large, the deviations can range widely. Plotted together, the geometry of that joint space looks like a funnel — wide and open at one end (large variance, loosely-constrained deviations), narrowing to a tight neck at the other (small variance, tightly-constrained deviations). Samplers like Stan's HMC/NUTS struggle badly inside that narrow neck, because the step size that works well in the wide part of the funnel is too large to navigate the narrow part without "divergent transitions" (the sampler effectively crashing into the funnel's walls). It's a well-known pathology named after statistician Radford Neal, and the standard fix is a **non-centered parameterisation** — reformulating

the model so the individual deviations are expressed as independent, unit-scaled quantities multiplied by the variance parameter afterwards, rather than directly tangled with it, which straightens the funnel out into geometry the sampler can explore efficiently.

```
check_hmc_diagnostics(fit)
```

A model with zero divergent transitions, R-hat below 1.01, and adequate ESS on every parameter you intend to use downstream is one you can defend to a reviewer. A model with any one of those failing is not ready for a decision analysis, no matter how plausible the point estimates look.

Posterior predictive checks

Convergence diagnostics tell you whether your *sampler* worked. They say nothing about whether your *model* is any good. For that you need posterior predictive checks (PPCs): simulate new data from the fitted model's posterior and ask whether that simulated data looks like the data you actually observed. If your model is a reasonable description of the data-generating process, replicated datasets drawn from the posterior predictive distribution should be hard to distinguish from the real thing.

```
library(bayesplot)

y_rep <- posterior_predict(fit)  # or extract manually from Stan output

ppc_dens_overlay(y = observed_outcomes, yrep = y_rep[1:50, ])
```

This overlays the density of your observed outcome on top of 50 densities simulated from posterior draws. A model that fits well produces a tight bundle of replicated curves hugging the observed curve. A model that systematically misses — too much dispersion, wrong skew, a mode in the wrong place — shows up as replicated curves that visibly diverge from the observed data, which is a much more concrete signal than a residual plot for an analyst still building intuition for what "model misfit" looks like in a Bayesian setting.

HEOR in practice. For a Bayesian NMA feeding a cost-effectiveness model, run PPCs on each trial's observed event counts against the model's posterior predictive event counts before trusting the pooled treatment effects. A network that fits the overall deviance well can still badly misfit one influential trial — exactly the trial that might be driving your incremental cost-effectiveness ratio.

Agency perspective. NICE DSU Technical Support Document 2 (Dias et al., **Evidence Synthesis for Decision Making 2: A Generalised Linear Modelling Framework for Pairwise and Network Meta-Analysis of Randomised Controlled Trials**), already named in Chapter 12's reporting section, explicitly recommends reporting the deviance information criterion alongside convergence diagnostics for network meta-analysis models submitted to NICE, and treats inadequate convergence checking as a methodological limitation reviewers are instructed to flag.

14.5 Bayesian evidence synthesis as a pipeline

Chapter 12 walked through fitting Bayesian network meta-analyses in practice — `gemtc` wrapping JAGS, `multinma` wrapping Stan, and `bnma` as a lighter-weight alternative — and showed you how to extract pooled treatment effects and relative rankings. This section does not repeat that. Instead, it steps back to the principle that makes Bayesian evidence synthesis worth the extra modelling effort in the first place, and sets up why §14.6 can treat the output of Chapter 12's models as directly usable decision-analytic input.

The general principle: synthesis produces a distribution, not a number

Classical (frequentist) meta-analysis gives you a pooled effect estimate and a confidence interval. That interval describes sampling variability under repeated hypothetical experiments — a long-run frequency statement, not a probability statement about the parameter itself. A Bayesian meta-analysis or NMA, by contrast, gives you a full posterior distribution over every pooled effect, every between-study heterogeneity parameter, and every treatment ranking probability. That distribution *is* a direct probability statement: "given this evidence and this model, here is how plausible each value of the treatment effect is."

This distinction sounds academic until you reach the point where you need to propagate evidence uncertainty into a decision model. A confidence interval does not tell you how to sample from it in a way that respects the correlation structure between parameters — you have to impose an additional distributional assumption (normal, usually, on the log-hazard-ratio scale) and often ignore correlations between related parameters altogether. A Bayesian posterior needs none of that. It already *is* a joint sample over every parameter, correlations included, ready to be drawn from directly.

Why this matters more as networks get more complex

The benefit of full posterior propagation scales with the complexity of the evidence base. A two-arm pairwise meta-analysis has one pooled effect and its uncertainty is easy to approximate reasonably well with a normal distribution. A network with several treatments, as in Chapter 12's worked example, has many pairwise contrasts, each correlated with the others through the network structure and the consistency assumptions baked into the model. There is no sensible way to hand-construct a joint distributional assumption that preserves those correlations for a downstream Monte Carlo simulation. The posterior sample from `gemtc`, `multinma`, or `bnma`, however, already carries those correlations intact, because every MCMC draw is a complete, internally consistent set of all the contrasts simultaneously.

This is the real argument for Bayesian evidence synthesis in an HEOR context. It isn't that Bayesian point estimates differ meaningfully from frequentist ones under vague priors (often they barely do). It's that the *output object* — the posterior sample — is already in the exact shape a decision model needs, with no further approximation step required.

```
# Recall from Chapter 12: a gemtc NMA fit
library(gemtc)

# mcmc.list of relative-effect draws, one column per contrast
relative_effects <- relative.effect(nma_fit, t1 = "placebo")
re_samples <- as.matrix(relative_effects$samples)

# Each row is one joint posterior draw across every contrast --
```

```
# correlations between contrasts are preserved automatically
head(re_samples)
```

This `re_samples` matrix is the hinge on which the rest of this chapter turns: it is simultaneously the end product of Chapter 12's evidence synthesis and the raw material for the decision analysis in §14.6.

Where this fits in the overall workflow

Think of Bayesian evidence synthesis as the first stage of a two-stage pipeline, not a standalone analysis:

1. *Synthesis stage* (Chapter 12's territory): combine all available trials — pairwise or networked — into posterior distributions for relative treatment effects, using `gemtc`, `multinma`, or `bnma` as appropriate to the network structure and priors required.
2. *Decision stage* (this chapter's territory, §14.6): feed those posterior draws, unmodified, into a cost-effectiveness model as the probabilistic sensitivity analysis sample for treatment effect.

> **HEOR in practice.** When writing up methods for a submission, be explicit that the PSA distribution for relative treatment effect *is* the NMA posterior sample, not a distribution fitted *to* the posterior sample's mean and standard deviation. Re-fitting a normal distribution to posterior summary statistics throws away exactly the correlation structure that made the Bayesian synthesis worth doing, and reviewers familiar with the NICE DSU documents will ask why you did it.

> **Common pitfall.** Thinning or subsampling the posterior sample before passing it to the decision model (e.g. taking every 10th draw to get a "round" PSA sample size) is usually unnecessary and can quietly reduce the effective precision of your PSA results. Unless autocorrelation diagnostics (§14.4) indicate the chain needs thinning for storage reasons, pass the full post-warm-up sample through.

The next section makes this pipeline concrete: taking posterior draws — whether from an NMA, a simpler pairwise meta-analysis, or any other Bayesian evidence model — and using them as the engine of a full probabilistic cost-effectiveness analysis.

14.6 Bayesian decision analysis: from posterior to PSA

Everything in this chapter has been building toward one practical payoff, and it is worth stating plainly: *a Bayesian model's posterior draws are a probabilistic sensitivity analysis sample.* There is no separate step where you fit a distribution to a parameter and then sample from it for PSA — the posterior draws produced by your MCMC sampler in §14.1-14.3, or by the NMA models of Chapter 12, already are that sample. This section shows how to use them that way.

Expected value under a posterior

A health economic decision model computes, for each strategy s, some net benefit function — most commonly net monetary benefit at a willingness-to-pay threshold λ:

$$\text{NMB}_s = \lambda \times \text{QALY}_s - \text{Cost}_s$$

In a deterministic model you plug in point estimates for every input parameter and get one number per strategy. In a Bayesian decision-analytic framing, every input parameter — the treatment effect, the utility decrement, the cost, the transition probability — has a posterior distribution, and so does NMB_s. The *expected* net monetary benefit is the posterior mean of that distribution:

$$\text{E}[\text{NMB}_s] = \int \text{NMB}_s(\theta) p(\theta \mid D)\, \text{d}\theta$$

where θ is the full vector of model parameters and $p(\theta \mid D)$ is their joint posterior given data D. You never evaluate this integral analytically. You approximate it by Monte Carlo: run the cost-effectiveness model once per posterior draw, and average the resulting net benefits across draws. This is exactly standard PSA mechanics — the only thing that has changed is *where the samples came from.*

Posterior draws as PSA input: the direct pipeline

In a conventional (non-Bayesian) PSA workflow, you fit each input parameter a distribution — a beta for a probability, a gamma for a cost, a normal for a log-hazard-ratio — usually justified by matching a point estimate and standard error from a single study or a frequentist meta-analysis, then draw independently from each one and hope the implied joint distribution across parameters is reasonable. Correlations between parameters, when they exist, have to be imposed by hand (e.g. via a multivariate normal on the log scale, or a Cholesky decomposition), which is awkward and frequently skipped.

> **Difficult terminology: Cholesky decomposition.** Suppose you want to simulate several correlated quantities at once (e.g. cost and utility that tend to move together) by starting from easy-to-generate independent random numbers. A Cholesky decomposition breaks a correlation (or covariance) matrix down into a single "square root" matrix that, when multiplied by a vector of independent random draws, produces a new vector with exactly the desired correlation structure baked in. It's the standard recipe for "injecting" a specified correlation pattern into otherwise-independent simulated numbers. A kitchen analogy: if independent random numbers are plain flour, water, and eggs, the Cholesky matrix is the recipe/ratios that combine them into a batter with the specific consistency (correlation) you wanted, rather than three separate unrelated ingredients. The point raised here is that doing this by hand, for every PSA parameter set, is exactly the fiddly step a genuine Bayesian joint posterior sample (the rest of this section) sidesteps entirely, because the posterior draws already come out correlated correctly without needing any decomposition trick.

A Bayesian evidence base sidesteps all of this. If your relative treatment effect comes from the NMA posterior sample built in §14.5, each row of that sample already is one coherent PSA iteration for the treatment-effect parameters, correlations and all. You use the rows directly as your PSA loop, rather than drawing from a distribution that was fitted *to* that sample.

```
library(dplyr)

# re_samples: posterior draws of relative treatment effect from gemtc/NMA (14.5)
# n_psa draws, one row per PSA iteration, columns are contrasts

n_psa <- nrow(re_samples)
```

```r
# Other model inputs can also come from posteriors -- e.g. a Bayesian
# survival model's posterior draws of a hazard ratio, or simple
# independent draws if those parameters weren't modelled jointly
cost_psa    <- rgamma(n_psa, shape = 20, rate = 20 / 3500)
utility_psa <- rbeta(n_psa, shape1 = 40, shape2 = 10)

psa_results <- vector("list", n_psa)

for (i in seq_len(n_psa)) {
  log_hr <- re_samples[i, "d.placebo.treatment"]

  qalys_treatment <- simulate_qalys(hr = exp(log_hr), utility = utility_psa[i])
  cost_treatment  <- simulate_costs(hr = exp(log_hr), unit_cost = cost_psa[i])

  psa_results[[i]] <- data.frame(
    iteration = i,
    nmb_treatment = 30000 * qalys_treatment - cost_treatment,
    nmb_control   = 30000 * simulate_qalys(hr = 1, utility = utility_psa[i]) -
                    simulate_costs(hr = 1, unit_cost = cost_psa[i])
  )
}

psa_df <- bind_rows(psa_results)

# Expected net monetary benefit per strategy, and probability treatment
# is cost-effective at this threshold
psa_df |>
  summarise(
    E_nmb_treatment = mean(nmb_treatment),
    E_nmb_control   = mean(nmb_control),
    prob_ce         = mean(nmb_treatment > nmb_control)
  )
```

Notice what did *not* happen here: there was no step that said "fit a normal distribution to the log hazard ratio's posterior mean and SD, then draw from that." The posterior draws went straight into the simulation loop. Every correlation between contrasts that existed in the original NMA posterior — for instance, if two active treatments were both compared against the same reference and so their relative effects are not independent — survives into the PSA, because row i of `re_samples` is used as a complete, internally consistent unit.

> **HEOR in practice.** When your evidence base mixes Bayesian and non-Bayesian sources — a Bayesian NMA posterior for relative effect alongside a classically-fitted parametric survival curve for baseline risk — align the number of PSA iterations to the smaller or more constrained posterior sample, or resample consistently, and document which inputs came from genuine joint posteriors versus independently assumed distributions. A reviewer should be able to tell at a glance which parts of your PSA have real correlation structure behind them.

Decision uncertainty is now a direct statement about the evidence

Reporting a cost-effectiveness acceptability curve (CEAC) built from this PSA has a cleaner Bayesian interpretation than its frequentist counterpart. The proportion of PSA iterations where treatment has higher net benefit than control, at a given λ, is a direct posterior probability that

treatment is cost-effective at that threshold, given the data and the model — not an approximation justified by asymptotic sampling theory. This is a more honest match between the question decision-makers actually ask ("how likely is it that this treatment is worth it?") and the quantity the analysis reports.

Common pitfall. If you draw θ from a posterior but then still report the deterministic ("base case") analysis using the posterior *mean* of θ plugged into the model once, you have built a Bayesian evidence base but reverted to a non-probabilistic decision analysis. The expected value of a nonlinear function of θ is not generally equal to the function evaluated at $\mathrm{E}[\theta]$ — run the full posterior sample through the model, not just its central tendency.

Agency perspective. The ISPOR-SMDM Modeling Good Research Practices Task Force report on model transparency and validation (Eddy et al. 2012, **Value in Health**, "Model Transparency and Validation") recommends that probabilistic sensitivity analysis fully propagate parameter uncertainty including correlation structure where it exists — a requirement that a direct Bayesian-posterior-as-PSA-sample pipeline satisfies by construction, where an independently-sampled, distribution-fitted PSA typically does not.

14.7 Expert elicitation methods (SHELF)

Most of this chapter has assumed you have *data* — trial arms, observational cohorts, registry follow-up — from which a likelihood can be built and combined with a prior. But a large share of real HEOR work, especially around rare diseases, orphan drugs, and early-stage HTA submissions for genuinely novel technologies, runs into parameters for which no usable data exist at all. What is the five-year mortality benefit of a gene therapy with eighteen months of trial follow-up? What proportion of patients will adhere to a new delivery device nobody has used outside a lab? Committees still need a number — or, in the Bayesian framing of this chapter, a *distribution* — for these parameters, and when data cannot supply it, the next-best evidence is structured expert judgement.

The word "structured" matters enormously here. Unstructured expert judgement — asking a clinician "what do you think the rate is?" over the phone — is exactly the kind of informal, poorly-calibrated input that got Bayesian methods a bad reputation for being "subjective" in a pejorative sense. The response from the methodology community, and specifically from the University of Sheffield group, was to formalise the process into a reproducible protocol: the **Sheffield Elicitation Framework**, universally known by its acronym, **SHELF**. SHELF is not a piece of software first and foremost — it is a facilitation method, a set of templates, and a philosophy for turning "what an expert believes" into a probability distribution that can be defended, audited, and fed into a model. The R package of the same name (note the package name is capitalised, `SHELF`) is the computational companion that fits distributions to the quantities the facilitation process produces.

HEOR in practice. Elicitation is appropriate for parameters where data genuinely do not exist or are too immature to trust — not as a shortcut past the inconvenience of designing a study. A NICE committee or a payer reviewer confronted with an elicited prior will scrutinise *how* it was elicited at least as closely as the number itself. Treat the process documentation

(who, how many experts, what questions, what reconciliation) as a deliverable in its own right, not an afterthought.

Roles in the process. A SHELF exercise has two distinct kinds of participant, and conflating them is a common and damaging mistake.

- *The facilitator* is a trained elicitation specialist (ideally with statistical literacy but, crucially, with no personal stake in the parameter's value) who runs the session, asks questions in a format designed to counteract known cognitive biases (overconfidence, anchoring, availability), and never offers their own opinion of the "right" answer.
- *The experts* are the substantive content specialists — clinicians, epidemiologists, sometimes patients or carers for quantities like health state preferences — who hold the relevant domain knowledge but typically have no particular training in thinking probabilistically. Part of the facilitator's job is translating a clinical intuition ("most patients do reasonably well, a few have terrible outcomes") into quantiles a distribution can be fitted to.

The general process. A typical SHELF exercise, whether for a single parameter or a small set, follows roughly this arc:

1. *Preparation.* The facilitator defines the quantity of interest precisely (a frequent failure mode is experts answering slightly different questions from each other because the parameter was never pinned down — "response rate" at what time point, in what population, by what definition of response).
2. *Individual elicitation.* Each expert, independently and without discussion with the others, is asked a sequence of questions designed to pin down their personal distribution for the parameter — typically a median, then a lower and upper quartile or tertile, sometimes plausible extreme bounds. Individual elicitation first is deliberate: it prevents the loudest or most senior voice in the room from anchoring everyone else before independent judgements are recorded.
3. *Group discussion.* The individual judgements are shared, discussed, and the *reasons* behind disagreement are explored — not to force consensus, but so that each expert can rationally update their own view in light of arguments they hadn't considered.
4. *Reconciliation.* The group works toward a single "rational impartial observer" distribution representing the pooled judgement — SHELF provides specific templates for this step, rather than leaving it to an unstructured vote.
5. *Distribution fitting.* The elicited quantiles (individual or reconciled) are fitted to a parametric distribution — commonly a beta for a probability, a gamma or log-normal for a rate or cost, a normal for a roughly symmetric continuous quantity.

R tooling. The `SHELF` package on CRAN implements the fitting and feedback steps computationally. Given elicited judgements (for example, a median and two quartiles from one expert), the typical workflow is to fit a family of candidate distributions and visually check which gives a good fit to the elicited quantiles:

```
library(SHELF)

# Example: an expert has given judgements for a response-rate
# parameter as quantiles: 25th percentile = 0.30, median = 0.45,
# 75th percentile = 0.60
my_probs <- c(0.25, 0.5, 0.75)
my_vals  <- c(0.30, 0.45, 0.60)
```

```
fit <- SHELF::fitdist(vals = my_vals, probs = my_probs,
                      lower = 0, upper = 1)

# Inspect which parametric family fits best
fit$ssq

# Visualise the fitted distribution(s) against the elicited points
SHELF::plotfit(fit, lp = TRUE)
```

The package also provides feedback-style diagnostics — comparing implied probabilities from the fitted distribution back against further quantiles the expert supplies, as a sanity check that the fit captures their actual judgement — via a function along the lines of `SHELF::feedback()`. Not every argument name and default in the current `feedback()` signature has been independently re-checked against the live CRAN documentation here, so treat the call above as illustrative of the *idea* rather than a copy-paste guarantee; always run `?SHELF::fitdist` and `?SHELF::feedback` in your own R session before relying on the precise arguments, since package APIs do shift across versions. The package also ships a Shiny app for live group elicitation sessions — genuinely useful if you expect to run this more than once.

Common pitfall. A beta or gamma fit to elicited quantiles is a *summary* of judgement, not a measurement. Resist the temptation to report the fitted distribution's standard error with the same confident precision you would give a trial estimate's standard error — it reflects how confident the expert group felt, filtered through a parametric approximation, not a sampling distribution of anything. Always report, alongside the fitted parameters, who was elicited, how many experts, and whether reconciliation changed the picture materially.

Agency perspective. NICE's Decision Support Unit Technical Support Document 33, "Expert Elicitation: Issues and Challenges," is the standard methodological reference UK HTA submissions point to when using elicited priors, and SHELF is the protocol it most closely aligns with in practice; if an elicitation exercise will inform a submission, read that document (or the equivalent guidance your jurisdiction's HTA body references) before designing the session, not after.

14.8 Bayesian health economic modelling workflows

This closing section is deliberately a roadmap rather than a new technique. Up to this point the chapter has built the pieces in isolation — Bayes' theorem and conjugate updating, MCMC sampling with Stan or JAGS, convergence diagnostics, and posterior-to-PSA translation — and §14.7 just added expert elicitation as a way to construct priors when data run out. None of these pieces does anything useful for a decision-maker on its own. The payoff is in chaining them into a single, reproducible, end-to-end workflow that starts with "what do we believe before seeing data" and ends with "here is the probability this technology is cost-effective, and here is how much it would be worth resolving our remaining uncertainty." Walking through that chain end to end is the point of this section.

Step 1 — Prior elicitation and specification. Every Bayesian model starts with a prior, whether it is a deliberately vague, weakly informative default or a carefully elicited distribution from §14.7. The discipline worth adopting here is documentation: record not just the prior's parametric form but *why* that form and those parameter values were chosen — vague because nothing better was available, informative because a SHELF exercise was run, or weakly informative as a regularising choice to stabilise an otherwise unidentified model. A reviewer (or your future self) will ask.

Step 2 — Bayesian evidence synthesis. Where there are multiple studies bearing on a parameter — several trials estimating a treatment effect, several registries estimating a transition probability — Chapter 12's Bayesian network meta-analysis, or a simpler Bayesian meta-analysis of the kind introduced earlier in this chapter, combines them into a single posterior while properly propagating between-study heterogeneity. This step is where most of the genuine *evidence* enters the model; the prior from Step 1 and the likelihood from the synthesised data combine here, typically specified as a hierarchical model in Stan or JAGS.

Step 3 — MCMC fitting. The joint posterior over every parameter in the synthesis model — treatment effects, baseline event rates, heterogeneity variances, and anything else in the model's parameter vector — is sampled numerically, since it almost never has a closed form once you move past the simplest conjugate cases covered early in the chapter. This is mechanically the same `stan()` or `jags.model()` call pattern used throughout the chapter; what differs model to model is the structure wired into the Stan/JAGS code, not the fitting mechanics.

Step 4 — Diagnostics. Before a single posterior draw is trusted for anything downstream, run the checks from §14.4: $\hat{R}$ close to 1 for every parameter, adequate effective sample size, trace plots that look like noise rather than trends, and divergent-transition counts near zero if you are using Stan's Hamiltonian Monte Carlo sampler. This is not an optional formality — a decision model fed non-converged MCMC output will produce PSA results that look precise and are simply wrong, and nothing downstream will flag that for you.

```
# Illustrative: checking convergence before using draws downstream
library(rstan)
print(fit_synthesis, pars = c("mu", "tau"))   # check Rhat, n_eff
rstan::traceplot(fit_synthesis, pars = c("mu", "tau"))
```

Step 5 — Posterior draws as the PSA sample. This is the step that makes the whole chapter's effort worthwhile, and it is where §14.6's logic lands. In a conventional frequentist PSA, you fit point estimates and standard errors to each model parameter, then *separately* specify a distribution (beta, gamma, log-normal) for each one to sample from — an extra modelling step that can lose the correlation structure between parameters estimated jointly from the same data. In a Bayesian workflow, there is no separate step: each row of the MCMC output *already is* one coherent draw of the full parameter vector from the joint posterior, correlations intact. You simply take those draws — thousands of them, one per retained MCMC iteration — and pass them straight into the decision-analytic model (a Markov model, a partitioned survival model, whatever structure Part IV's chapters build) to compute costs and QALYs under each draw. No distributional assumptions are layered on top of the evidence synthesis; the posterior *is* the PSA input.

```
# Illustrative: extracting posterior draws as PSA inputs
library(rstan)
draws <- rstan::extract(fit_synthesis)
# draws$mu, draws$tau (etc.) are now vectors of length n_draws,
# ready to be looped through a cost-effectiveness model row by row
```

Step 6 — Reporting. Transparency matters at every step above, but especially here: a results table or report should make clear which parameters came from elicitation, which from synthesis, what the convergence diagnostics showed, and how many posterior draws fed the PSA. This is the kind of audit trail HTA bodies and peer reviewers increasingly expect, and the ISPOR-SMDM Modeling Good Research Practices task force reports are a useful reference point for what that reporting should cover structurally, alongside CHEERS 2022 for the economic evaluation reporting itself.

> **HEOR in practice.** Treat this six-step chain as a checklist you re-run for every Bayesian submission, not a one-off design exercise. The most common real-world failure is not a statistical error in any single step — it is a broken link between steps: synthesis draws that get summarised down to a mean and SD before being handed to the decision model, discarding exactly the joint uncertainty structure that made the Bayesian approach worth doing in the first place. Keep the full draws array moving through every step; only summarise at the very end, for reporting.

Looking forward, this workflow is the hinge between this chapter and Part IV. Chapter 25, on handling uncertainty and probabilistic sensitivity analysis, picks up exactly at Step 5 above and develops the decision-model side — cost-effectiveness acceptability curves, cost-effectiveness planes, and the full machinery for summarising PSA output once you have it. Chapter 26, on value of information analysis, is where a fully Bayesian model pays an extra dividend that a frequentist PSA cannot offer as naturally: because the PSA sample already *is* a set of draws from a genuine joint posterior over every parameter, quantities like the Expected Value of Perfect Information (EVPI) and Expected Value of Partial Perfect Information (EVPPI) can be computed directly from those draws, without the additional approximating machinery sometimes needed to retrofit VOI onto a non-Bayesian PSA. If you take one thing from this chapter into the rest of the book, it should be this: a Bayesian evidence base is not just a different way of getting a p-value-free answer — it is the natural, low-friction substrate for everything Part IV does with uncertainty afterward.

> **Common pitfall.** Running MCMC for the evidence synthesis and then discretising the posterior down to a handful of scenarios ("low, medium, high") for the decision model defeats the purpose of this entire workflow and will understate decision uncertainty. If you have thousands of posterior draws, use thousands of draws in the PSA loop — modern decision models in R run this comfortably; the computational cost of looping over draws is rarely the real constraint.

Chapter 15. Real-World Evidence (RWE) and Causal Inference

Chapter 14 worked with evidence you already trusted the structure of: trial data, pooled contrasts, a posterior you could carry straight into a decision model. This chapter asks a harder question. What if the evidence was never randomised — a TB programme register, an insurance claims database, a hospital's electronic health records? Can you still get a defensible causal answer?

Most of this chapter's methods are frequentist, by design and by convention. Chapter 14 already flagged RWE and causal inference as the one area where Bayesian methods are an occasional option, not the default. Propensity scores, doubly robust estimators, instrumental variables, difference-in-differences — these are overwhelmingly practiced and reported in a frequentist frame in the field itself, and this chapter follows that convention.

The chapter has four parts. §15.1-15.3 build the foundations: what makes data fit for purpose, which study design fits which question, and how a causal diagram makes your confounding assumptions explicit before you fit anything. §15.4-15.7 cover the core estimation toolkit for when you believe you have measured the relevant confounders: propensity scores, doubly robust estimators and g-methods, instrumental variables and regression discontinuity, and difference-in-differences/interrupted time series. §15.8-15.9 cover two problems that recur constantly in HTA submissions: adjusting for treatment switching, and quantifying how robust a finding is to confounders you could not measure. §15.10-15.11 close with infrastructure and design: the OHDSI/OMOP standard for multi-site data, and external/synthetic control arms for single-arm trials.

Learning objectives

- Assess whether a real-world data source is fit for purpose using the FDA's relevance/reliability framework, and document that assessment before analysis begins
- Choose between a cohort, case-control, or target trial emulation design for a given RWE question, and explain how target trial emulation guards against immortal time and prevalent-user bias
- Build a directed acyclic graph (DAG) in `dagitty`/`ggdag`, distinguish confounders from mediators and colliders, and extract a minimal sufficient adjustment set programmatically rather than adjusting for every available covariate
- Implement propensity score matching, weighting, and balance diagnostics in R (`MatchIt`, `WeightIt`, `cobalt`), and compute an inverse-probability-weighted average treatment effect by hand to verify package output
- Explain why doubly robust estimators (AIPW) and g-methods relax the single-correct-model assumption of plain propensity-score or outcome-regression approaches, and when time-varying confounding requires g-methods specifically
- Apply instrumental variables (`ivreg`) and regression discontinuity (`rdrobust`) to recover causal effects under unmeasured confounding, and recognise the diagnostic checks (weak-instrument F-test, exclusion restriction plausibility, manipulation testing) each design requires

- Implement difference-in-differences correctly under staggered treatment adoption (`did::att_gt()`) rather than naive two-way fixed effects, and fit an interrupted time series model with `segmented` or an explicit level/slope-change specification
- Describe when and why treatment-switching adjustment (RPSFTM, IPCW) is needed in trials with crossover, and name NICE DSU TSD 16 as the governing methods reference
- Compute and interpret E-values (`EValue` package) as a standardised sensitivity measure for unmeasured confounding, and present them alongside point estimates rather than in isolation
- Describe the role of the OHDSI/OMOP Common Data Model and its R tooling (`DatabaseConnector`, `FeatureExtraction`, `CohortMethod`) in multi-site observational studies, and judge when that infrastructure is worth its complexity
- Construct a propensity-matched external control arm for a single-arm trial, and explain why temporal/era comparability is a distinct threat from baseline covariate imbalance

15.1 Fit-for-purpose RWD and RWE frameworks

Every chapter so far in this book has worked from evidence that was, in some sense, handed to you: trial arms with randomised assignment, published hazard ratios, posterior distributions from a Bayesian network meta-analysis. From this chapter onward the ground shifts. Real-world data (RWD) — programme registers, insurance claims, electronic health records (EHRs), laboratory information systems, even the line-listings your national TB programme submits to WHO — were not collected to answer your causal question. They were collected to run a clinic, bill a payer, or satisfy a surveillance mandate. Turning that data into real-world evidence (RWE) fit for a regulatory or HTA decision requires you to be explicit about **why** the data exist, **whether** they can answer the question you are asking, and **what** bias they carry before a single regression line is fitted.

This is not a semantic nicety. The US FDA's 2023 guidance **"Real-World Data: Assessing Electronic Health Records and Medical Claims Data To Support Regulatory Decision-Making for Drug and Biological Products"** — part of the broader RWE framework the agency committed to under the 21st Century Cures Act — sets out a two-part test that has become the de facto global standard even outside the US: is the data **relevant** to the clinical question (right population, right exposure, right outcome, adequate follow-up), and is it **reliable** (accurate, complete, traceable, with transparent provenance)? The EMA's parallel work, coordinated through the Big Data Steering Group and operationalised in the **DARWIN EU** (Data Analysis and Real World Interrogation Network) initiative, asks the same two questions in different words: data quality and study design robustness. For a TB programme analyst, the practical translation is this — before you touch a single `dplyr` verb, you document where each variable in your dataset came from, how it was captured (clinician entry, lab interface, self-report), what fraction is missing and why, and whether the cohort you can assemble actually contains the exposure contrast (e.g., bedaquiline-containing vs injectable-containing regimen) and the outcome (culture conversion, treatment success, death) with enough precision to support inference.

A useful habit is to write a short **data provenance and fitness memo** before analysis begins — not after a reviewer asks for one. The memo should state the data source and update frequency, the target population and how your extract maps onto it, known structural gaps (e.g., private-

sector treatment episodes missing from a public programme register), and the intended use of the evidence (safety signal, comparative effectiveness, HTA submission). This is the RWE-era equivalent of a statistical analysis plan, and regulators increasingly expect to see it.

```
library(dplyr)
library(tibble)

# A minimal fitness-for-purpose checklist as a data frame -- the kind of
# artefact you would attach to an analysis plan or HTA dossier appendix
rwd_fitness_checklist <- tibble::tribble(
  ~domain,         ~question,
~assessment,
  "Relevance",     "Does the population match the decision question?",
"Partial: public-sector only",
  "Relevance",     "Is exposure (regimen) captured with dates?",           "Yes,
pharmacy dispense log",
  "Relevance",     "Is the outcome defined per WHO TB definitions?",       "Yes,
treatment outcome registry",
  "Reliability",   "Is there a documented data dictionary?",               "Yes,
v3.2",
  "Reliability",   "What proportion of key covariates are missing?",       "HIV
status 4%, culture 11%",
  "Reliability",   "Is there an audit trail / source verification?",
"Partial: no routine monitoring"
)

print(rwd_fitness_checklist, n = Inf)
```

The point of such a table is not bureaucratic box-ticking. A "Partial" answer in the Relevance domain tells you, before you run a single model, that any effect estimate will generalise only to the public sector, and a "Partial" in Reliability tells you that missingness in HIV status is probably not completely at random and will need explicit handling (multiple imputation, inverse-probability weighting, or at minimum a sensitivity analysis) rather than a silent `na.omit()`.

HEOR in practice. When an RWE analysis is destined for a reimbursement dossier, align the fitness-for-purpose assessment with the specific decision-grade evidence standard the payer applies — in India, HTAIn's methods guidance and NICE's real-world evidence framework in the UK both now ask explicitly whether the RWD source could, in principle, have supported a trial-emulating analysis. Framing your memo in those terms early avoids a late-stage "why wasn't this a target trial emulation" review comment.

Common pitfall. Treating "more data" as a proxy for "better evidence" is one of the most common RWE errors. A claims database with five million patients but no reliable treatment-start date is less fit for a comparative-effectiveness question than a programme register of five thousand patients with verified regimen dates. Volume does not substitute for relevance and reliability — assess both before describing a data source as "big data."

15.2 Study design: cohort, case-control, target trial emulation

Once a data source clears the fitness-for-purpose bar, the next decision is architectural: what study design will let the data answer the causal question without manufacturing an answer that merely reflects who received the exposure in the first place? Three designs recur constantly in TB and infectious-disease RWE work, and each has a distinct bias profile.

A **cohort design** follows an exposed and an unexposed (or differently exposed) group forward from a defined start point to an outcome. It is intuitive and maps directly onto the trial structure you already know from earlier chapters, but its validity depends entirely on correctly defining "time zero" — the point at which eligibility, exposure assignment, and follow-up all begin simultaneously. Get this wrong (for example, by including follow-up time before a drug was actually started, or by selecting on survival to the point of exposure) and you reintroduce **immortal time bias**, a problem familiar from pharmacoepidemiology that inflates apparent drug benefit.

A **case-control design** samples cases of the outcome (e.g., treatment failure) and a comparison set of controls from the same source population, then looks backward at exposure. It is efficient for rare outcomes and is still common in TB drug-resistance surveillance, but it is acutely sensitive to how controls are selected — control selection that does not mirror the case-generating mechanism produces selection bias that no amount of post-hoc adjustment repairs.

The design that has reshaped how regulators and HTA bodies think about RWE over the past decade is **target trial emulation (TTE)**, formalised by Hernán and Robins (**"Using Big Data to Emulate a Target Trial When Comparing Adaptive, Dynamic Treatment Regimes"**, *American Journal of Epidemiology*, 2016, and elaborated in their freely available text *Causal Inference: What If*). The core discipline of TTE is simple to state and demanding to execute: before touching the observational data, write a protocol for the **hypothetical pragmatic randomised trial** you would run if you could — eligibility criteria, treatment strategies being compared, assignment procedure, start of follow-up, outcome, causal contrast of interest, and analysis plan. Only then do you specify how each protocol element is emulated in the observational data. This explicit target-trial protocol is what prevents two subtle, pervasive errors: immortal time bias (misaligned time zero) and **prevalent-user bias** (comparing new starters to patients already established on treatment, who have by definition survived early attrition).

```
library(tibble)
library(dplyr)

# A target trial protocol, specified as a structured object before any
# emulation code touches the observational extract. Writing this first
# is the TTE discipline, not an optional add-on.
target_trial_protocol <- tribble(
  ~component,            ~specification,
  "Eligibility",         "Adults starting first-line TB treatment, no prior TB
Rx",
  "Treatment strategies","Bedaquiline-containing vs standard injectable-containing
regimen",
  "Assignment",          "Hypothetical randomisation at treatment initiation",
  "Time zero",           "Date of first regimen dispense (must equal eligibility
date)",
  "Outcome",             "WHO-defined treatment success at 6 months",
  "Causal contrast",     "Intention-to-treat and per-protocol effects",
  "Analysis",            "IPTW-adjusted Kaplan-Meier / Cox, with cloning-
```

```
censoring-weighting for per-protocol"
)

print(target_trial_protocol, n = Inf)

# Emulating "time zero" correctly: every patient's follow-up clock
# starts at regimen initiation, not at diagnosis or enrolment
cohort <- tibble(
  patient_id   = 1:6,
  diagnosis_dt = as.Date(c("2022-01-05","2022-01-10","2022-02-01",
                           "2022-02-15","2022-03-01","2022-03-10")),
  regimen_start= as.Date(c("2022-01-07","2022-01-20","2022-02-02",
                           "2022-02-28","2022-03-01","2022-03-25")),
  regimen      = c("Bedaquiline","Injectable","Bedaquiline",
                   "Injectable","Bedaquiline","Injectable")
)

cohort <- cohort %>%
  mutate(
    lead_in_days = as.numeric(regimen_start - diagnosis_dt),
    time_zero    = regimen_start   # NOT diagnosis_dt -- avoids immortal time bias
  )

cohort
```

The `lead_in_days` column is exactly the diagnostic you want in an audit: any patient with a long gap between diagnosis and regimen start who is nonetheless analysed as "exposed from diagnosis" is a walking immortal-time-bias case study. Setting `time_zero` to `regimen_start` and restricting eligibility to patients alive and in the cohort at that date is the whole trick — the rest of the per-protocol / intention-to-treat machinery (cloning, censoring, and inverse-probability-of-censoring weighting) builds directly on this correctly specified start point and is covered alongside the causal-inference workhorse models later in this chapter.

HEOR in practice. When drafting an RWE protocol for a regulatory or HTA submission, structure the methods section using the Hernán-Robins "specify target trial, then specify emulation" table explicitly — most reviewers trained in the FDA RWE framework or EMA's DARWIN EU studies now expect to see eligibility, treatment strategy, assignment, time zero, outcome, and analysis mapped side-by-side between the hypothetical trial and the observational emulation. A protocol that skips this mapping invites exactly the immortal-time and prevalent-user critiques described above.

Agency perspective. The FDA's 2021 draft guidance **"Considerations for the Design and Conduct of Externally Controlled Trials for Drug and Biological Products"**, together with the 2023 RWD/EHR-and-claims guidance under the agency's RWE Framework (per the 21st Century Cures Act §3022 mandate), explicitly references target trial emulation logic when assessing whether a non-interventional comparator arm is fit to support an effectiveness claim — sponsors are expected to pre-specify the emulated trial's eligibility, treatment strategies, and time-zero alignment before data extraction, mirroring Hernán and Robins' original 2016 *AJE* framework.

15.3 Confounding and bias, with DAGs (`dagitty`, `ggdag`)

Study design controls **which** biases are structurally possible. It does not, by itself, tell you which variables you must adjust for to recover an unbiased causal effect. That is the job of a **directed acyclic graph (DAG)** — a transparent, falsifiable statement of your assumptions about how variables cause one another, drawn **before** you fit any model.

A DAG forces you to distinguish three structures that are easy to confuse in a results-driven rush to "adjust for everything available": **confounders** (common causes of exposure and outcome, which you must adjust for), **mediators** (on the causal pathway from exposure to outcome, which you must **not** adjust for if you want the total effect), and **colliders** (common effects of two variables, which adjusting for actively **induces** bias where none existed).

In a TB regimen-comparison example, HIV status plausibly affects both which regimen a clinician chooses and the probability of treatment success — a classic confounder. Nutritional status measured mid-treatment, however, may be partly a **consequence** of the regimen (some regimens cause more nausea and weight loss) and partly a cause of outcome — a mediator, and adjusting for it would bias the estimate of the regimen's total effect. Site-level treatment capacity might influence both which patients get referred there (a collider with other selection variables) and local reporting completeness. Getting this taxonomy right by eye, on a real dataset with a dozen covariates, is essentially impossible without a graph.

The R packages **dagitty** and **ggdag** let you encode these assumptions as code, visualise them, and — critically — ask the graph itself what the minimal sufficient adjustment set is, rather than adjusting for every variable in the extract "to be safe."

```
library(dagitty)
library(ggdag)
library(ggplot2)

# Encode the causal structure with ggdag::dagify(); exposure/outcome
# arguments tell dagitty which nodes play which role
tb_dag <- ggdag::dagify(
  success ~ regimen + hiv + nutrition + site_capacity,
  regimen ~ hiv + site_capacity,
  nutrition ~ regimen,
  site_capacity ~ urban,
  exposure = "regimen",
  outcome  = "success",
  labels = c(
    regimen       = "Regimen",
    success       = "Tx success",
    hiv           = "HIV status",
    nutrition     = "Nutrition (mediator)",
    site_capacity = "Site capacity",
    urban         = "Urban site"
  )
)

# Visualise with ggdag() -- status highlights exposure/outcome nodes
ggdag::ggdag(tb_dag, text = FALSE, use_labels = "label") +
  ggdag::theme_dag()
```

```
# Ask dagitty for the minimal sufficient adjustment set directly
adj_sets <- dagitty::adjustmentSets(tb_dag, exposure = "regimen", outcome =
"success")
adj_sets
```

Running `adjustmentSets()` on this graph returns `{hiv}` as the minimal sufficient set — critically, `nutrition` is correctly **excluded** because it is a mediator, not a confounder, and `ggdag::ggdag_adjustment_set()` will draw the same set visually, which is a useful sanity check to pair with the programmatic answer.

```
# Visual confirmation of the same adjustment set
ggdag::ggdag_adjustment_set(tb_dag, exposure = "regimen", outcome = "success",
                            text = FALSE, use_labels = "label") +
  ggdag::theme_dag()
```

Once the adjustment set is settled by the DAG rather than by convenience, the question of **how** to report the resulting observational comparison is itself standardised. The **STROBE statement** (Strengthening the Reporting of Observational Studies in Epidemiology) — and its TREND and RECORD extensions for routinely collected and non-randomised data respectively — specifies a 22-item checklist covering exactly the design, confounding, and bias elements this section has walked through: how participants were selected, how confounders were identified and measured, how missing data were handled, and how potential biases were addressed, mapped item-by-item into the manuscript or dossier. Treat the STROBE checklist as a drafting template, not a post-hoc audit — filling it in alongside the DAG and target-trial protocol from §15.2, rather than after the analysis is finished, will save you from discovering a reporting gap only when a reviewer asks for it.

A final methodological note before we move into estimation proper: almost everything in this chapter — DAG-based adjustment set selection, propensity scores, inverse-probability weighting, and the target trial emulation machinery — is conventionally executed in a **frequentist** framework, and that is the default throughout the rest of this chapter. Bayesian causal inference exists and is occasionally useful, particularly when you want to propagate structural uncertainty (e.g., over which DAG is correct) into the final effect estimate, or when borrowing external control information as in Chapter 14's evidence-synthesis methods — but it is the exception reached for deliberately, not the default starting point, in applied RWE work.

> **Common pitfall.** The single most damaging confounding error in practice is **adjusting for a collider**. If site-level treatment capacity is a common effect of both urban location and regional funding allocation, and you include it in a regression "because it's available and correlated with outcome," you can **induce** a spurious association between regimen and outcome where the true DAG has none. Always draw the DAG and run `dagitty::adjustmentSets()` before selecting covariates — never adjust for a variable solely because it is statistically associated with the outcome in a univariate check.

15.4 Propensity scores: matching, weighting, stratification (`MatchIt`, `WeightIt`, `cobalt`)

The propensity score, formally $e(x) = P(T = 1 \texttt{ mid } X = x)$, is the probability that a unit receives treatment given its observed covariates. Rosenbaum and Rubin's foundational insight was that conditioning on $e(x)$ — rather than on the full covariate vector X — is sufficient to balance observed

confounders between treated and untreated groups, provided the no-unmeasured-confounding assumption holds. For a TB-programme analyst this matters concretely: when you compare patients who received a new short-course regimen against those who stayed on the standard regimen in routine programme data, the two groups almost never look alike at baseline. Patients prescribed the new regimen might be healthier, more urban, or diagnosed earlier. The propensity score collapses all of that imbalance into a single number you can match, weight, or stratify on.

Three workhorse strategies use the propensity score once it is estimated (almost always by logistic regression, though machine-learning propensity models are increasingly common):

- *Matching* pairs each treated unit with one or more control units with similar propensity scores, discarding unmatched units.
- *Weighting* (inverse probability of treatment weighting, IPTW) reweights the full sample so that the weighted covariate distributions are balanced, keeping all units.
- *Stratification* bins units into propensity-score strata (commonly quintiles) and estimates effects within strata before pooling.

Matching with MatchIt

`MatchIt::matchit()` is the standard tool. It estimates (or accepts) propensity scores and performs the matching in one call.

```
library(MatchIt)
library(cobalt)

# tb_cohort: one row per patient, trt = 1 if new regimen, 0 if standard
m.out <- matchit(
  trt ~ age + bmi + hiv_status + smear_grade + district_urban,
  data   = tb_cohort,
  method = "nearest",      # nearest-neighbour matching
  distance = "glm",        # logistic-regression propensity score
  ratio  = 1,               # 1:1 matching
  caliper = 0.2             # max distance, in SD of logit(PS)
)

summary(m.out)              # balance before/after matching
matched_data <- match.data(m.out)   # retrieve matched sample with weights
```

`method = "nearest"` with a caliper is a reasonable default for TB programme comparisons with moderate sample sizes; `method = "full"` (full matching) or `method = "optimal"` can reduce discarded units when treated and control covariate distributions overlap poorly. Always inspect `summary(m.out)` for the standardized mean differences (SMDs) before and after matching — SMDs below 0.1 for every covariate is the conventional threshold for "good balance."

Weighting with WeightIt

Weighting retains the full sample (useful when programme data is already scarce, as TB cohorts often are) at the cost of being more sensitive to extreme propensity scores.

```
library(WeightIt)

w.out <- weightit(
  trt ~ age + bmi + hiv_status + smear_grade + district_urban,
  data     = tb_cohort,
  method   = "ps",          # logistic-regression propensity score
```

```
  estimand = "ATE"          # average treatment effect (use "ATT" for effect on
treated only)
)

tb_cohort$iptw <- w.out$weights
summary(w.out)
```

`estimand = "ATE"` weights treated units by $\frac{1}{e(x)}$ and controls by $\frac{1}{1-e(x)}$; `estimand = "ATT"` instead weights controls by $\frac{e(x)}{1-e(x)}$ and leaves treated units at weight 1, targeting the effect in the treated population specifically — often the more policy-relevant estimand when the new regimen will only ever be rolled out to patients similar to those who already got it.

Checking balance with cobalt

`cobalt` is the standard companion package for producing balance diagnostics and the now-ubiquitous "Love plot."

```
library(cobalt)

bal.tab(w.out, un = TRUE, m.threshold = 0.1)   # table: unadjusted vs weighted
SMDs

love.plot(
  w.out,
  binary   = "std",
  thresholds = c(m = 0.1),
  abs      = TRUE,
  var.order = "unadjusted"
)
```

`bal.tab()` prints standardized mean differences (and variance ratios for continuous covariates) before (`un = TRUE`) and after weighting; `love.plot()` renders the now-standard dot plot with a vertical reference line at the balance threshold, letting you see at a glance whether every covariate crossed under 0.1 after weighting.

A worked numeric example: IPTW ATE by hand

To make the weighting arithmetic concrete — and to leave you something you can reproduce by hand before trusting any package output — consider a toy cohort of six TB patients with **known** propensity scores (in practice these come from the logistic model; here they are given so the downstream arithmetic is fully verifiable).

> **HEOR in practice. Toy IPTW calculation.** Treatment outcome Y is, say, time-to-culture-conversion in weeks (lower is better).

How to report this.

Patient	Treated (T)	Outcome (Y)	PS $e(x)$	IPTW contribution
1	1	10	0.5	$1 \times \frac{10}{0.5} = 20$
2	1	8	0.4	$1 \times \frac{8}{0.4} = 20$
3	1	12	0.6	$1 \times \frac{12}{0.6} = 20$
4	0	4	0.5	$1 \times \frac{4}{1-0.5} = 8$
5	0	6	0.4	$1 \times \frac{6}{1-0.4} = 10$
6	0	2	0.6	$1 \times \frac{2}{1-0.6} = 5$

Table 45: Toy cohort for inverse-probability-of-treatment weighting

The IPTW estimator of the ATE is

$$\hat{\tau}_{\text{IPTW}} = \frac{1}{n}\sum_i \frac{T_i Y_i}{e(x_i)} - \frac{1}{n}\sum_i \frac{(1-T_i)Y_i}{1-e(x_i)}$$

Summing the treated contributions: $20 + 20 + 20 = 60$, divided by $n = 6$ gives a weighted treated mean of 10.000. Summing the control contributions: $8 + 10 + 5 = 23$, divided by $n = 6$ gives a weighted control mean of 3.833.

$$\hat{\tau}_{\text{IPTW}} = 10.000 - 3.833 = 6.167 \text{ weeks}$$

Notice the coincidence built into this toy example — each treated patient's contribution is exactly 20 — which is intentional: it shows that when $T_i \frac{Y_i}{e(x_i)}$ is constant across treated units, the weighted mean equals that constant regardless of sample size, a useful sanity check when debugging your own `weightit()` output against a hand calculation on a small subset.

```
# Reproduce the hand calculation in R
toy <- data.frame(
  T  = c(1,1,1,0,0,0),
  Y  = c(10,8,12,4,6,2),
  ps = c(0.5,0.4,0.6,0.5,0.4,0.6)
)
mean(toy$T * toy$Y / toy$ps) - mean((1 - toy$T) * toy$Y / (1 - toy$ps))
#> 6.166667
```

Common pitfall. Extreme propensity scores (close to 0 or 1) produce extreme weights, which can make the IPTW estimator wildly unstable — a single near-zero $e(x)$ in the denominator can dominate the entire estimate. Always plot the distribution of weights (`summary(w.out$weights)`, or a histogram) before trusting an ATE. Trimming (dropping units with $e(x)$ outside, say, $[0.05, 0.95]$) or using **stabilized** weights (multiplying by the marginal probability of treatment) are standard remedies — `WeightIt::weightit()` supports `stabilize = TRUE` directly.

15.5 Doubly robust estimators and g-methods

Matching and weighting both lean entirely on the propensity-score model being correctly specified. Outcome regression leans entirely on the outcome model being correctly specified instead. Each method has exactly one place it can go wrong, and no backup if it does.

Doubly robust (DR) estimators remove that single point of failure. They combine both models so the estimate stays consistent if **either** the propensity model or the outcome model is correctly specified — not necessarily both. That is a meaningfully weaker assumption than either method alone requires, which is why DR methods have become close to a default in contemporary RWE causal analyses, including in regulatory submissions.

The best-known DR estimator is Augmented Inverse Probability Weighting (AIPW):

$$\hat{\tau}_{\text{AIPW}} = \frac{1}{n}\sum_i \left[\frac{T_i(Y_i - \hat{m}_1(X_i))}{\hat{e}(X_i)} - \frac{(1-T_i)(Y_i - \hat{m}_0(X_i))}{1-\hat{e}(X_i)} + \hat{m}_1(X_i) - \hat{m}_0(X_i)\right]$$

where $\hat{m}_1(X)$ and $\hat{m}_0(X)$ are outcome-regression predictions under treatment and control respectively, and $\hat{e}(X)$ is the propensity score. The first bracketed pair is an IPW-weighted **residual correction**; the last pair is the plain outcome-regression estimate. If the outcome model is right, the residuals average to roughly zero and the estimate reduces to the (correct) regression estimate; if the propensity model is right, the IPW terms correct any bias in a misspecified outcome model.

```
library(AIPW)
library(SuperLearner)

aipw_fit <- AIPW::AIPW$new(
  Y          = tb_cohort$culture_conversion_weeks,
  A          = tb_cohort$trt,
  W          = tb_cohort[,
c("age","bmi","hiv_status","smear_grade","district_urban")],
  Q.SL.library = c("SL.glm", "SL.mean"),   # outcome model learners
  g.SL.library = c("SL.glm", "SL.mean"),   # propensity model learners
  k_split    = 10,
  verbose    = TRUE
)
aipw_fit$fit()$summary()
```

A lighter-weight alternative that stays entirely inside the `WeightIt`/regression toolchain, suited for a quick DR check without pulling in `SuperLearner`, is to fit an outcome regression on the IPTW-weighted sample (the "weighted regression" or "doubly robust regression" trick):

```
library(WeightIt)

tb_cohort$iptw <- weightit(
  trt ~ age + bmi + hiv_status + smear_grade + district_urban,
  data = tb_cohort, method = "ps", estimand = "ATE"
)$weights

dr_fit <- lm(
  culture_conversion_weeks ~ trt + age + bmi + hiv_status + smear_grade +
district_urban,
  data    = tb_cohort,
  weights = iptw
```

```
)
coef(summary(dr_fit))["trt", ]
```

This weighted-regression approach is doubly robust in the sense described above whenever the outcome model is linear and additive in treatment; `AIPW` or `tmle`/`tmle3` give the fully general nonparametric version.

G-methods

G-methods — g-computation (the g-formula), marginal structural models fit by IPTW, and g-estimation of structural nested models — were developed by James Robins specifically to handle **time-varying treatment and time-varying confounding**, the situation where today's treatment affects tomorrow's confounder, which in turn affects tomorrow's treatment decision (a "treatment-confounder feedback" loop that standard regression adjustment cannot handle without introducing bias). This is directly relevant to TB programmes: a patient's adherence this month influences next month's sputum-smear result, which influences whether the clinician escalates or de-escalates therapy next month.

> **Difficult terminology: marginal structural models.** A marginal structural model asks a very specific causal question: "what would the outcome look like, on average across the whole population, if everyone followed a particular treatment strategy over time?" — e.g. "everyone stays on the standard regimen for the full six months" versus "everyone is escalated to the stronger regimen as soon as smear status worsens." Normally you'd worry that patients who got escalated were sicker to begin with, confounding any simple comparison. Marginal structural models handle this not by adding the time-varying confounder (smear status) directly into the outcome model — which, with treatment-confounder feedback, can itself introduce bias — but by **reweighting** each patient at each time point using inverse-probability-of-treatment weights (IPTW), so that, after reweighting, treatment at each visit looks "as good as randomly assigned" with respect to the confounders measured up to that point. It's analogous to the reweighting trick in Chapter 13′s MAIC, but applied across repeated time points within the same patients rather than across two separate trials.

> **Difficult terminology: g-estimation of structural nested models.** G-estimation is a different route to the same time-varying-confounding problem that marginal structural models address, but instead of reweighting patients, it directly asks: "by how much would this patient's observed outcome have differed, at each decision point, had their treatment at that point been different from what they actually received?" It builds a "structural nested model" — a set of equations describing, step by step, how each treatment decision nudges the final outcome — and searches (iteratively, via an estimating-equation algorithm) for the parameter values that make the adjusted, counterfactual outcomes look unrelated to the observed treatment history, after accounting for measured confounders. It's more computationally involved than IPTW-weighted marginal structural models and used less often in applied HEOR work, but it avoids some of the extreme-weight instability that can plague IPTW when treatment is very predictable from the confounders.

G-computation in its simplest (time-fixed) form is conceptually just: (1) fit an outcome model including treatment and confounders, (2) predict the outcome for every unit **as if** everyone had

been treated, (3) predict again **as if** no one had been treated, (4) the ATE is the difference of the two averaged predictions.

```
# Simple (time-fixed) g-computation "by hand"
outcome_model <- lm(
  culture_conversion_weeks ~ trt + age + bmi + hiv_status + smear_grade +
district_urban,
  data = tb_cohort
)

d1 <- d0 <- tb_cohort
d1$trt <- 1          # counterfactual: everyone treated
d0$trt <- 0          # counterfactual: no one treated

g_ate <- mean(predict(outcome_model, newdata = d1)) -
         mean(predict(outcome_model, newdata = d0))
g_ate
```

For genuinely time-varying treatment and confounding — multiple visits per patient, each with its own treatment decision and intermediate outcomes — the `gfoRmula` package implements the full longitudinal g-formula with Monte Carlo simulation of counterfactual trajectories, and `ipw::ipwtm()` fits marginal structural models via time-varying IPTW. These are advanced tools; flag them as the next step once your TB programme data includes repeated visits with a feedback loop between adherence, smear results, and regimen changes, rather than reaching for them on a single cross-sectional comparison.

Agency perspective. The FDA's 2023 draft guidance **"Considerations for the Use of Real-World Data and Real-World Evidence to Support Regulatory Decision-Making for Drug and Biological Products"**, together with the companion **"Real-World Data: Assessing Registries to Support Regulatory Decision-Making for Drug and Biological Products"** guidance, explicitly discusses doubly robust estimation and g-methods as acceptable approaches for addressing time-varying confounding in RWE submissions, and cross-references the ICH E9(R1) estimands framework for specifying **which** causal estimand (treatment-policy, hypothetical, principal stratum, etc.) the g-method or DR estimator is meant to target before any model is fit. Before running an AIPW or g-computation analysis intended to support a regulatory or HTA submission, write down the estimand in ICH E9(R1) language first — the choice of estimand constrains which intercurrent-event handling strategy (and therefore which g-method variant) is defensible.

Difficult terminology: ICH E9(R1) estimand strategies + intercurrent events. Before you can estimate a "treatment effect," you have to pin down exactly what question you're asking — and in a real trial or RWE study, things inevitably go wrong partway through: patients drop out, switch to rescue medication, or die before the planned follow-up ends. These mid-study complications are called **intercurrent events**. ICH E9(R1) is the international regulatory guideline that formalises five standard strategies for defining a meaningful "treatment effect" question in the presence of such events. **Treatment-policy** counts whatever happens to the patient regardless of adherence or switching (closest to "real-world" pragmatic effect). **Hypothetical** asks what would have happened if the intercurrent event hadn't occurred at

all (e.g. if no one had been allowed to switch treatment). **Composite** folds the intercurrent event itself into the outcome definition (e.g. counting treatment switch as a treatment failure). **While-on-treatment** only asks about the period before the intercurrent event happened. **Principal stratum** restricts the question to the subgroup of patients who would not have experienced the intercurrent event under either treatment. None of these is universally "correct" — they answer genuinely different clinical questions, and the right strategy depends on what decision-makers actually need to know; the mistake to avoid is picking one implicitly, by whichever analysis method happens to be convenient, rather than choosing deliberately and stating it upfront.

15.6 Instrumental variables and regression discontinuity

Propensity-score and DR methods all assume **no unmeasured confounding**: every variable that affects both treatment assignment and the outcome is observed and included in the model. That assumption can fail badly in TB programme data — e.g., clinician "gut feel" about disease severity may drive both regimen choice and prognosis without appearing in any recorded field. Instrumental variables (IV) and regression discontinuity (RD) are the two classic quasi-experimental designs that can recover a causal effect **even in the presence of unmeasured confounding**, by exploiting a specific source of "as-good-as-random" variation in treatment.

Instrumental variables

An instrument Z is a variable that (i) affects treatment T, (ii) affects the outcome Y **only through** T (the exclusion restriction), and (iii) is independent of unmeasured confounders. A classic TB-programme instrument is distance to the nearest facility offering the new diagnostic/regimen, or facility-level rollout timing driven by a supply allocation that is unrelated to individual patient severity: patients don't choose which facility rolled out first, but facility rollout strongly predicts whether an individual patient received the new regimen.

Two-stage least squares (2SLS) is the standard estimator, implemented identically in `ivreg::ivreg()` (from the `ivreg` package) and `AER::ivreg()` (an older alias from the Applied Econometrics with R package — both use the same `|`-separated formula syntax):

```
library(ivreg)

iv_fit <- ivreg(
  culture_conversion_weeks ~ trt + age + bmi + hiv_status |     # structural
equation
                              facility_rollout_quarter + age + bmi + hiv_status, #
instruments + exogenous covariates
  data = tb_cohort
)
summary(iv_fit, diagnostics = TRUE)
```

The `diagnostics = TRUE` argument is essential: it reports the **weak-instrument (first-stage F) test**, the Wu-Hausman endogeneity test, and the Sargan over-identification test (when you have more instruments than endogenous regressors). A first-stage F-statistic below roughly 10 is the conventional (if debated) threshold for flagging a "weak instrument," below which 2SLS estimates become unreliable and badly biased toward the plain OLS estimate.

Difficult terminology: endogeneity / endogenous variable. A variable is called "endogenous" when it isn't truly independent of the thing that's also driving the outcome — in plain terms, something else (often unmeasured) is influencing both the treatment a patient receives **and** their prognosis, so the treatment variable is partly a stand-in for that hidden factor rather than a clean, as-good-as-randomised input. Picture a clinician who informally senses a patient is sicker than any recorded field captures, and on that gut feeling escalates them to a stronger regimen: the regimen choice is now tangled up with unmeasured severity, and a plain regression of outcome on regimen will attribute some of that unmeasured severity's effect to the regimen itself, biasing the estimate. This is exactly the problem instrumental variables are built to sidestep, by finding a source of variation in treatment (the instrument) that isn't contaminated by that same hidden confounding.

Difficult terminology: Wu-Hausman test. This test directly checks whether endogeneity is actually a real, practically important problem for your data, rather than something to worry about in the abstract. It compares the plain OLS regression estimate against the instrumental-variables (2SLS) estimate of the same effect: if treatment were not endogenous, the two should give similar answers (because OLS would already be unbiased, and IV would just be a noisier way of getting the same answer). A statistically significant Wu-Hausman result means the two estimates disagree by more than chance would explain — evidence that the plain OLS estimate is likely biased by unmeasured confounding, and that you should trust the instrumental-variables estimate instead, despite it usually having a wider confidence interval.

Difficult terminology: Sargan (over-identification) test. This check is only possible when you have **more** instruments than strictly needed (e.g. two separate instruments for one endogenous treatment variable, when one would technically suffice) — hence "over-identified." The idea: if all your instruments are genuinely valid (each satisfying the exclusion restriction), then every possible combination of them should point to roughly the same causal effect. The Sargan test checks whether that's actually true, by seeing whether the instruments as a group produce internally consistent results. A significant Sargan test is a warning sign that at least one of your instruments may not be valid — though it can't tell you **which** one, only that something in the set is inconsistent with the others.

How to report this.

Diagnostic	df1	df2	Interpretation
Weak instruments	1	n-k	F-stat > 10: instrument strong enough
Wu-Hausman	1	n-k	Significant: T is endogenous, IV needed over OLS
Sargan	–	–	Only with more than 1 instrument; tests exclusion restriction

Table 46: Illustrative IV diagnostics output structure from `summary(iv_fit, diagnostics = TRUE)`

Common pitfall. The exclusion restriction — that Z affects Y **only** through T — is never fully testable from data alone and must be argued substantively. "Facility rollout timing" fails as an instrument if rollout was itself prioritized toward facilities with sicker patients (then it affects Y directly through facility-level case mix, not only through individual treatment). Always describe, in words, the mechanism that could violate exclusion, and check whether it is institutionally plausible for your specific rollout — a statistically "strong" instrument (high first-stage F) with an implausible exclusion restriction is still a bad instrument.

Regression discontinuity

RD designs exploit a sharp, known threshold rule for assigning treatment: units just above the cutoff get treated, units just below do not, and — assumption-wise — units just either side of the cutoff are otherwise comparable. A natural TB-programme RD arises when national guidelines mandate the new regimen only for patients with a sputum bacterial load or a CD4 count below a specific cutoff; comparing outcomes for patients just above versus just below that cutoff approximates a local randomized experiment.

`rdrobust::rdrobust()` is the standard, well-validated implementation (local-polynomial estimation with automatic, data-driven bandwidth selection and robust bias-corrected confidence intervals):

```
library(rdrobust)

rd_fit <- rdrobust(
  y = tb_cohort$culture_conversion_weeks,
  x = tb_cohort$cd4_count,
  c = 200                      # the policy cutoff: CD4 < 200 mandates new regimen
)
summary(rd_fit)

rdplot(
  y = tb_cohort$culture_conversion_weeks,
  x = tb_cohort$cd4_count,
  c = 200
)
```

`rdrobust()` reports three estimates (conventional, bias-corrected, and robust), and the robust bias-corrected confidence interval is the one you should report — it accounts for the fact that the optimal bandwidth for point estimation is not the optimal bandwidth for inference. `rdplot()` produces the canonical RD scatter-with-binned-means-and-fitted-lines figure that should accompany every RD analysis, so a reviewer can see the jump at the cutoff directly rather than trusting the number alone.

Common pitfall. RD estimates are **local**: the effect is identified only for patients near the cutoff (CD4 close to 200 in the example), and should never be extrapolated to patients far from it (e.g., CD4 of 50 or 600) without additional structural assumptions. Always run `rdrobust`'s built-in manipulation/density check (`rddensity` package) to confirm patients were not able to manipulate which side of the cutoff they landed on — in a TB programme, a clinician who knows the CD4 threshold and orders a repeat test for a borderline patient to secure the new regimen would silently invalidate the whole design.

15.7 Difference-in-differences and interrupted time series

The final pair of designs in this section uses **time** as the source of quasi-experimental leverage, comparing outcomes before and after an intervention, either against a comparison group that did not receive the intervention (difference-in-differences) or purely through the pre/post trajectory itself (interrupted time series).

Difference-in-differences (DiD)

Classic two-group, two-period DiD estimates the treatment effect as the difference in each group's own pre-to-post change:

$$\hat{\tau}_{\text{DiD}} = \left(\overline{Y}_{\text{treated, post}} - \overline{Y}_{\text{treated, pre}}\right) - \left(\overline{Y}_{\text{control, post}} - \overline{Y}_{\text{control, pre}}\right)$$

This is naturally implemented as a two-way fixed effects (TWFE) regression with a treatment-by-post interaction term:

```
did_fit <- lm(
  culture_conversion_weeks ~ treated_district * post_rollout + age + bmi,
  data = district_panel
)
summary(did_fit)
# coefficient on treated_district:post_rollout IS the DiD estimate
```

Difficult terminology: two-way fixed effects (TWFE). "Fixed effects" here just means: give every district its own intercept adjustment, and every time period its own intercept adjustment, so that each comparison is effectively made relative to that specific district's own average level and that specific period's own average level, rather than lumping everyone together on a single shared baseline. "Two-way" means both of those adjustments (district and time) are included simultaneously. Practically, this is a tidy way of controlling for anything constant about a district (e.g. its baseline healthcare infrastructure) and anything constant across districts within a time period (e.g. a national supply shortage that quarter) without having to measure and name those confounders explicitly — the fixed effects soak them up automatically. The catch, raised in the next paragraph, is that once treatment rolls out to

different districts at different times (staggered adoption), this simple two-way structure starts implicitly comparing some already-treated districts against each other as if one were a valid "control" for the other, which can badly distort the estimated effect — hence the shift to purpose-built staggered-adoption estimators like `did::att_gt()`.

The identifying assumption is **parallel trends**: absent the intervention, the treated district(s) would have followed the same **trend** as the control district(s) — not the same level, just the same trajectory. This is checked, never assumed, by plotting pre-intervention outcome trends for both groups and/or by testing for pre-trend differences in an event-study specification (leads and lags of the treatment indicator).

```
library(ggplot2)
ggplot(district_panel, aes(quarter, culture_conversion_weeks, color =
treated_district)) +
  stat_summary(geom = "line") +
  geom_vline(xintercept = rollout_quarter, linetype = "dashed")
```

When rollout is **staggered** across many districts at different times — the realistic case for a phased national TB programme rollout — naïve TWFE regression is now well known to produce badly biased estimates (the "negative weighting" problem documented by Goodman-Bacon and others), because it implicitly compares already-treated districts against each other as pseudo-controls. The modern fix is the `did` package's group-time average treatment effects:

```
library(did)

atts <- att_gt(
  yname    = "culture_conversion_weeks",
  tname    = "quarter",
  idname   = "district_id",
  gname    = "rollout_quarter",     # period each district FIRST adopted the new
regimen (0/Inf for never-treated)
  data     = district_panel,
  control_group = "nevertreated"     # or "notyettreated"
)

agg_simple <- aggte(atts, type = "simple")   # single overall ATT
summary(agg_simple)

agg_dyn <- aggte(atts, type = "dynamic")     # event-study style, by time since
rollout
ggdid(agg_dyn)
```

`att_gt()` estimates a separate average treatment effect for every (group, time) pair defined by **when** a district first adopted, then `aggte()` aggregates those into either a single summary number or a dynamic event-study plot, avoiding the staggered-adoption bias of plain TWFE entirely.

HEOR in practice. Practical rule of thumb. If every treated unit in your rollout switched on the same calendar date (a single national go-live), plain TWFE DiD is fine. If adoption is staggered across districts/facilities — almost always true of a phased national rollout — default

to `did::att_gt()` rather than TWFE, even if the TWFE coefficient "looks reasonable"; the bias from staggered TWFE can be large enough to flip the sign of the estimated effect.

Interrupted time series (ITS)

ITS designs use a **single** series — typically an aggregate programme-level outcome measured repeatedly over time, such as monthly treatment-success rate — and ask whether its level and/or slope changed at the moment of an intervention, with no separate control group required (though a control series strengthens inference considerably when available, turning ITS into a comparative/controlled ITS).

Segmented (piecewise) linear regression is the standard specification:

```
library(segmented)

its_base <- lm(success_rate ~ month, data = monthly_tb_outcomes)

its_seg <- segmented(
  its_base,
  seg.Z = ~ month,
  psi   = intervention_month        # known breakpoint: the rollout month
)
summary(its_seg)
slope(its_seg)         # slope before vs after the breakpoint
```

Alternatively, when the breakpoint (intervention timing) is known with certainty — the far more common and more defensible case, since programme rollout dates are typically documented rather than estimated — fit the level-and-slope-change model directly rather than letting `segmented()` search for the breakpoint:

```
monthly_tb_outcomes$post   <- as.integer(monthly_tb_outcomes$month >=
intervention_month)
monthly_tb_outcomes$time_since <- pmax(0, monthly_tb_outcomes$month -
intervention_month)

its_fit <- lm(
  success_rate ~ month + post + time_since,   # level-change (post) and slope-
change (time_since) terms
  data = monthly_tb_outcomes
)
summary(its_fit)
# coefficient on "post" = immediate level change at intervention
# coefficient on "time_since" = change in slope after intervention
```

Common pitfall. Autocorrelation in time series residuals inflates apparent precision and will make an ITS effect look far more statistically significant than it is. Check residual autocorrelation (`acf(residuals(its_fit))`) and, if present, re-fit with Newey-West or autocorrelation-consistent standard errors (`lmtest::coeftest(its_fit, vcov = sandwich::NeweyWest(its_fit))`) rather than trusting the plain `lm()` standard errors. Also watch for seasonality in monthly programme data (e.g., a recurring dip around the monsoon

season affecting case detection) being mistaken for an intervention effect if the breakpoint happens to coincide with a seasonal transition.

Agency perspective. NICE's **"NICE real-world evidence framework"** (2022) explicitly names difference-in-differences and interrupted time series among the quasi-experimental designs it considers acceptable supporting evidence for technology appraisals when randomized data are unavailable, provided the parallel-trends or pre-intervention-trend assumption is explicitly tested and reported, not merely asserted. When writing up a DiD or ITS analysis of programme data for an HTA submission, structure the results section around that framework's expectation: show the pre-period trend check first, then the point estimate, then a sensitivity analysis varying the comparison group or breakpoint specification.

15.8 Treatment switching and crossover adjustment (RPSFTM, IPCW)

Oncology and TB trials alike have a recurring headache: patients randomised to the control arm are allowed — often for good ethical reason — to cross over to the experimental treatment once they progress or fail first-line therapy. A placebo-controlled trial of a new multidrug-resistant (MDR-TB) regimen that permits crossover to the novel drug after treatment failure is common practice, and it is the right thing to do for patients. It is also a statistical headache for anyone trying to estimate what survival or treatment-success **would have been** had nobody switched — which is almost always the estimand a reimbursement body wants, because list prices and model structures are built around "control arm never gets the new drug."

A naive intention-to-treat (ITT) analysis of overall survival or time-to-cure under this design is unbiased for the policy actually implemented (randomise-and-allow-switch), but it systematically understates the treatment effect that would be observed under strict, no-crossover randomisation — which is usually the question of interest for a HEOR submission. HTA bodies increasingly expect sponsors to present both the ITT estimate and an adjusted estimate that attempts to remove the dilution caused by switching. The two workhorse methods are the **rank-preserving structural failure time model (RPSFTM)** and **inverse probability of censoring weights (IPCW)**.

RPSFTM assumes a common treatment effect multiplier ψ such that each patient's counterfactual "time off randomised arm's active treatment" is rescaled by $\exp(\psi)$, regardless of when the switch happened — it is a structural, not a regression, model, and it preserves the rank ordering of patients' latent event times under the null. It is robust to time-dependent confounding by design (it does not require measuring the confounders that triggered switching), but it depends on a strong "common treatment effect" assumption and performs poorly when switching is driven by severity that itself predicts the outcome independent of treatment. IPCW instead treats the problem as an informative-censoring problem: patients who switch are artificially censored at the time of switching, and the remaining follow-up is reweighted by the inverse probability of **not yet having switched**, conditional on time-varying prognostic covariates. IPCW requires you to have actually measured and modelled the covariates driving the switch — if an unmeasured factor (say, a clinician's informal read of deteriorating sputum smear trends) drives both switching and survival, IPCW is biased in the same direction as the naive analysis, only less so.

Agency perspective. The definitive methods reference here is the NICE Decision Support Unit's Technical Support Document on treatment switching, usually cited as NICE DSU TSD 16, "Adjusting Survival Time Estimates in the Presence of Treatment Switching" (Latimer and Abrams). It walks through RPSFTM, IPCW, and the two-stage estimation method with worked derivations and recommends which to use depending on whether switching is driven by observed time-varying covariates (favour IPCW) or not (favour RPSFTM), and under what circumstances neither is defensible. Any NICE submission involving crossover-adjusted survival should reference this TSD explicitly in the statistical analysis plan, not just cite the general RPSFTM/IPCW literature.

RPSFTM in R. There is a maintained CRAN package, `rpsftm` (Simon Bond), that implements the standard rank-preserving structural failure time model for a single survival endpoint with simple, one-way, timing-recorded switching. It is the right starting point and saves you from re-deriving the g-estimation search over ψ by hand. It does not, however, cover every design variant the literature discusses (e.g. two-way switching, re-switching, or models that jointly adjust multiple competing-risk endpoints) — for those you are back to hand-coding the structural model, in the same spirit as the hand-coded MAIC weighting discussed in Chapter 13. Treat `rpsftm` as a solid implementation of the textbook case, and verify its assumptions hold before trusting it on a more exotic design.

```
library(rpsftm)
library(survival)

# df has: time (observed), event, arm (0=control,1=experimental),
# rx (proportion of follow-up time actually on experimental treatment),
# plus switching indicator baked into rx for control-arm switchers
fit_rpsftm <- rpsftm(
  Surv(time, event) ~ rand(arm, rx),
  data = df,
  censor_time = admin_censor_time,  # administrative censoring time per patient
  n_eval_z = 100
)

summary(fit_rpsftm)
# psi: the estimated causal log-time-ratio
# Recreate a counterfactual, switching-adjusted Kaplan-Meier using
# fit_rpsftm$Sstar (the re-derived "as randomised, no switching" times)
counterfactual_fit <- survfit(Surv(fit_rpsftm$Sstar, event) ~ arm, data = df)
plot(counterfactual_fit, col = c("grey40", "steelblue"), lwd = 2,
     xlab = "Time (months)", ylab = "Survival probability")
```

IPCW in R. The `ipcwswitch` package (Louis Y et al.) is a real, CRAN-maintained package built specifically for treatment-switching adjustment via IPCW, including time-varying confounder handling and the data-reshaping steps needed to go from clinical-trial-visit format to the long counting-process format the weighted Cox model needs. It is less of a one-line black box than `rpsftm`: expect to spend real effort specifying the switching model (what predicts switching at each visit) and checking weight stability before trusting the output.

```
library(ipcwswitch)
library(survival)
```

```r
# long_df: one row per patient-visit, with time-varying covariates
# (ecog_status, sputum_grade, weight_kg) measured at each visit, and
# a per-visit switch indicator
long_df <- ipcw.data(
  long_df,
  id.name     = "patient_id",
  tstart.name = "visit_start",
  tstop.name  = "visit_stop",
  event.name  = "event",
  bas.cov     = c("age", "baseline_ecog", "site"),
  dyn.cov     = c("ecog_status", "sputum_grade", "weight_kg"),
  switch.name = "switched_this_visit"
)

ipcw_weights <- ipcw.est(
  long_df,
  formula.denom = switched_this_visit ~ ecog_status + sputum_grade + weight_kg,
  formula.num   = switched_this_visit ~ 1,
  id.name = "patient_id", tstart.name = "visit_start", tstop.name = "visit_stop"
)

weighted_cox <- coxph(
  Surv(visit_start, visit_stop, event) ~ arm,
  data = long_df, weights = ipcw_weights$weights, cluster = patient_id
)
summary(weighted_cox)

# Always check the weight distribution before trusting the estimate
summary(ipcw_weights$weights)
hist(ipcw_weights$weights, breaks = 50,
     main = "IPCW weight distribution -- check for extreme tails")
```

Common pitfall. Both methods are routinely misapplied by running them once, reporting the single resulting hazard ratio, and moving on. Three checks are non-negotiable before you put an RPSFTM- or IPCW-adjusted estimate into a cost-effectiveness model: (1) for RPSFTM, inspect whether the "common treatment effect" assumption is remotely plausible — if switchers and non-switchers in the control arm look like different populations at baseline, it probably is not; (2) for IPCW, plot the weight distribution — a handful of weights above 10-20 means a handful of patients are silently driving the whole adjusted estimate, and the standard errors from a naive `coxph` call will be too small (use robust/cluster variance, as shown above); (3) for either method, report the ITT estimate alongside the adjusted one and let the HTA committee see both — presenting only the adjusted, larger effect without the unadjusted comparator reads as cherry-picking even when the adjustment is methodologically sound.

15.9 E-values and quantitative bias analysis

Every observational HEOR analysis — propensity-matched comparisons, instrumental variable estimates, even the RWE arm of an indirect comparison — rests on one untestable assumption: no unmeasured confounder is distorting the association. Quantitative bias analysis (QBA) asks that question explicitly and numerically: how strong would an unmeasured confounder have to be to

explain away this result? The **E-value**, introduced by VanderWeele and Ding (2017), is the most widely adopted single-number answer. It is the minimum strength of association, on the risk-ratio scale, that an unmeasured confounder would need with **both** the exposure and the outcome — beyond the measured covariates — to fully explain away an observed association.

An E-value of 1.5 means a fairly weak unmeasured confounder could plausibly account for the whole effect — treat the finding as fragile. An E-value of 8 means an unmeasured confounder would need an implausibly strong, double-sided association to explain the result away — a much more robust finding. The appeal of the E-value in HEOR practice is that it converts a diffuse methodological worry ("but what about unmeasured confounding?") into a single number a non-statistician reviewer on an HTA committee can weigh against subject-matter plausibility: is it plausible that an unmeasured factor in this TB cohort — say, undocumented nutritional status or informal treatment adherence — has an association of that magnitude with both drug exposure and mortality? Often the answer is "no, that would be an extraordinarily strong confounder," which is reassuring; sometimes the answer is "yes, easily," which should make everyone more cautious about the point estimate.

The `EValue` package on CRAN (maintained by the original VanderWeele/Ding/Mathur group) implements the calculations directly, with functions tailored to each effect-measure scale, since the E-value formula differs depending on whether you start from a risk ratio, odds ratio, hazard ratio, or a continuous (OLS) effect size.

```
library(EValue)

# Risk ratio example: an observational comparison found RR = 1.8 for
# default-vs-treatment-success, 95% CI 1.3 to 2.5, comparing two regimens
ev_rr <- evalues.RR(est = 1.8, lo = 1.3, hi = 2.5)
ev_rr

# Odds ratio from a logistic model of treatment success
ev_or <- evalues.OR(est = 2.1, lo = 1.4, hi = 3.2, rare = TRUE)
# rare = TRUE converts OR to an RR approximation when outcome prevalence < ~15%

# Hazard ratio from a Cox model of time-to-cure
ev_hr <- evalues.HR(est = 0.65, lo = 0.48, hi = 0.88, rare = FALSE)

# Continuous outcome (e.g. change in a quality-of-life score) from OLS
ev_ols <- evalues.OLS(
  est = 4.2, se = 1.1, sd = 9.8,  # coefficient, its SE, and outcome SD
  true = 0  # null value being tested against
)
```

Each call returns both the E-value for the point estimate and the E-value for the confidence-interval bound closest to the null — the second number is usually the more conservative, decision-relevant one, since it answers "how strong a confounder would be needed to make even the **most cautious** end of my interval non-significant?"

For a submission, it is worth tabulating E-values across the key endpoints and model specifications rather than reporting a single number in prose, so a reviewer can see at a glance which findings are robust and which are fragile.

How to report this.

Endpoint	Effect estimate	E-value (point)	E-value (CI limit)
Treatment success (OR)	2.10 (1.40, 3.20)	3.64	2.13
Time to culture conversion (HR)	0.65 (0.48, 0.88)	2.05	1.53
Grade 3+ adverse event (RR)	1.80 (1.30, 2.50)	3.00	1.99
Mean weight gain at 6mo (OLS)	4.2 kg (SE 1.1)	2.88	1.86

Table 47: E-values for key endpoints, illustrative MDR-TB regimen comparison

Reading that table, the treatment-success and adverse-event findings have reasonably robust E-values even at the conservative CI-limit end (around 2), while the culture-conversion hazard ratio's CI-limit E-value of 1.53 is the one worth flagging to the committee as more sensitive to unmeasured confounding than the headline point estimate suggests.

HEOR in practice. E-values do not replace a proper sensitivity analysis, and they say nothing about **whether** an unmeasured confounder of that strength actually exists — only about the magnitude required. Pair the E-value with a short qualitative argument grounded in the clinical setting: for a TB programme comparison, is there a plausible unmeasured factor (health-seeking behaviour, nutritional status, informal regimen substitution at peripheral facilities) that could realistically reach the required strength of association with both exposure and outcome? If a clinical co-author can confidently say "no candidate confounder in this setting is remotely that strong," the E-value becomes a persuasive robustness statement rather than a statistical decoration. If they hesitate, that hesitation is itself useful information for the write-up.

15.10 OHDSI/OMOP workflows in R

Most of this chapter has assumed you already have a tidy analysis-ready dataset — one row per patient, clean dates, a `survival::Surv()` object waiting to be fit. Real-world claims and EHR data almost never arrive that way, and a large, genuinely global effort has grown up around standardising how such data gets structured before analysis: the **Observational Health Data Sciences and Informatics (OHDSI)** collaborative and its **OMOP Common Data Model (CDM)**. If you ever work with a multi-country claims database, a hospital EHR warehouse, or a federated network study, there is a reasonable chance the data has already been (or will need to be) mapped into OMOP CDM — a standardised relational schema with fixed tables (`person`, `condition_occurrence`, `drug_exposure`, `visit_occurrence`, and so on) and standardised vocabularies (SNOMED, RxNorm, LOINC) so that analytic code written once can, in principle, run unchanged against a hospital database in Seoul and a claims database in the United States.

The OHDSI R ecosystem is built in layers, each with a genuinely maintained, actively developed package:

- `DatabaseConnector` handles the low-level connection to whatever RDBMS actually stores the OMOP CDM (PostgreSQL, SQL Server, Redshift, BigQuery, Oracle), abstracting away vendor-specific SQL dialects via SqlRender underneath.
- `FeatureExtraction` turns raw OMOP tables into analysis-ready covariate matrices — demographics, comorbidity flags, prior drug exposure counts — using pre-built "covariate settings" rather than hand-written SQL for every variable.
- `CohortMethod` runs the standard new-user comparative cohort design (think: a propensity-score-matched or -weighted comparison of two drug exposure cohorts) end to end, from cohort definition through outcome model fitting, following the OHDSI network-study conventions.

```
library(DatabaseConnector)
library(FeatureExtraction)
library(CohortMethod)

connection_details <- createConnectionDetails(
  dbms             = "postgresql",
  server           = "cdm-server.example.org/omop_cdm",
  user             = Sys.getenv("OMOP_DB_USER"),
  password         = Sys.getenv("OMOP_DB_PASSWORD"),
  port             = 5432
)

# Define covariate settings: demographics + comorbidity history
covariate_settings <- createDefaultCovariateSettings(
  addDescendantsToExclude = TRUE
)

# Cohorts 1 (target drug) and 2 (comparator) must already exist as
# cohort_definition_id values in a cohort table built upstream
# (via ATLAS or hand-written cohort-definition SQL against OMOP tables)
cm_data <- getDbCohortMethodData(
  connectionDetails    = connection_details,
  cdmDatabaseSchema    = "omop_cdm",
  targetId             = 1,
  comparatorId         = 2,
  outcomeIds           = 3,     # e.g. treatment failure / default
  cohortDatabaseSchema = "results_schema",
  cohortTable          = "cohort",
  covariateSettings    = covariate_settings
)

ps_model <- createPs(cohortMethodData = cm_data)
matched_pop <- matchOnPs(ps_model, maxRatio = 1)

outcome_model <- fitOutcomeModel(
  population = matched_pop,
  cohortMethodData = cm_data,
  outcomeId = 3,
  modelType = "cox"
```

```
)
summary(outcome_model)
```

This is a substantial step up in infrastructure from the single-flat-file analyses elsewhere in this book, and it is worth being candid about when it is and is not the right tool. If you are running a one-off analysis of a single already-extracted dataset, standing up `DatabaseConnector`/`CohortMethod` against a live OMOP instance is almost certainly overkill — the earlier chapters' direct-dataframe workflows will get you to the same propensity-matched Cox model faster. The OHDSI stack earns its complexity when you need **reproducibility across sites** — a federated network study spanning several country-level TB or HIV surveillance databases, where the same analysis code genuinely has to run, unmodified, against structurally different source systems. Converting a bespoke national TB registry into OMOP CDM is itself a non-trivial ETL project (the OHDSI community's "Rabbit-in-a-Hat" / White Rabbit tooling helps, but mapping local vocabularies to standard concepts is still substantial domain work), so weigh that upfront cost against the actual need for cross-site standardisation before committing to it.

> **Common pitfall.** A common mistake with `CohortMethod` output is treating the within-database, single-site effect estimate as automatically generalisable, simply because the pipeline **looks** standardised. OMOP CDM standardises the **data structure and vocabulary**, not the underlying population, prescribing patterns, or coding practices at each site. A network study pooling results across five databases still needs the same scrutiny of comparability, residual confounding, and effect heterogeneity across sites that any multi-country RWE synthesis would — the OHDSI tooling makes the mechanics of running the same analysis everywhere easy; it does not make the resulting estimates automatically poolable or automatically externally valid to a setting (such as a high-burden, low-resource TB programme) that may look nothing like the health systems contributing data to the source databases.

15.11 Using external control arms and synthetic control arms

Single-arm trials are increasingly common in rare-disease and some oncology development, and the logic extends naturally to settings like a novel TB regimen trial run under compassionate-use or expanded-access protocols, where randomising a concurrent control arm is ethically or operationally impractical. The regulatory and HTA question that follows is unavoidable: compared to what? An **external control arm (ECA)** answers that by assembling a comparator cohort from a source other than the trial itself — a registry, a previous trial's control arm, real-world claims or EHR data, or a historical cohort — and comparing the single-arm trial's outcomes against it. A **synthetic control arm** is a particular, more formal construction of an ECA: rather than simply selecting "similar" historical patients, it builds the comparator as a weighted combination of external units (and sometimes pre-treatment outcome trajectories) explicitly chosen so that the synthetic cohort's observed characteristics, and ideally its outcome trajectory, closely track what the trial's intervention arm looked like **before** treatment — the Abadie/Diamond/Hainmueller synthetic control method from econometrics, and its clinical adaptations, generalise the comparative-case-study idea to a clinical comparator.

Both approaches face the same fundamental threat that randomisation exists to neutralise: the external cohort was never subject to the same allocation mechanism as the trial arm, so any difference in outcomes is a tangle of true treatment effect and systematic differences in who

ended up in which cohort — differences in era (standard of care drifts over a decade), in setting (a well-resourced referral hospital's historical cohort versus a multi-country trial's recruiting sites), in measurement (different diagnostic criteria, different definitions of "cure"), and in unmeasured patient selection (who gets referred, who gets excluded, who survives to be included in a registry at all). The statistical toolkit for narrowing that gap is the same one introduced in Chapter 12 for indirect comparisons — propensity score matching or weighting the external cohort to the trial population on measured baseline covariates — but it inherits the same core limitation: it can only balance what was actually measured comparably in both sources, and registry or claims data are rarely measured as rigorously or completely as a trial case report form.

```
library(survival)
library(MatchIt)

# combined_df: trial arm (source == "trial") stacked with an external
# registry cohort (source == "registry"), harmonised on shared baseline
# covariates measured comparably in both sources
combined_df$is_trial <- as.integer(combined_df$source == "trial")

match_model <- matchit(
  is_trial ~ age + baseline_ecog + extent_of_disease + prior_regimens,
  data   = combined_df,
  method = "nearest",
  ratio  = 2  # up to 2 external controls matched per trial patient
)

matched_data <- match.data(match_model)

eca_fit <- coxph(
  Surv(time, event) ~ is_trial,
  data    = matched_data,
  weights = weights  # matching weights from MatchIt
)
summary(eca_fit)

# Always report the covariate balance achieved by the match before
# trusting the comparative estimate
summary(match_model)
```

Because no CRAN package markets itself specifically as "the synthetic control arm for clinical trials" tool, in practice sponsors either adapt general-purpose econometric synthetic control packages (such as `Synth` or the newer `tidysynth`, both built for panel-data comparative case studies rather than clinical cohorts and requiring real adaptation of the outcome-trajectory-matching logic) or build the weighting and balance-checking pipeline by hand from `MatchIt`, `WeightIt`, or `cobalt`, in much the same spirit as the hand-coded MAIC and the `rpsftm` edge cases discussed earlier. Do not expect to find a single, clinically-validated, point-and-click CRAN package that performs a defensible synthetic control arm construction end to end — the statistical judgment about covariate selection, era-matching, and outcome-trajectory comparability is the genuinely hard, non-automatable part of the exercise.

> **Agency perspective.** The FDA's 2023 draft guidance, "Considerations for the Design and Conduct of Externally Controlled Trials for Drug and Biological Products," is the current

authoritative regulatory reference on when an external control arm is an acceptable substitute for a concurrent randomised control, what data quality and comparability standards it expects of the external source, and how sponsors should pre-specify the analytic approach (propensity methods, sensitivity analyses for unmeasured confounding, and the role of E-values of the kind introduced in §15.9) before seeing outcome data. Any HEOR team proposing an external or synthetic control arm for a regulatory or HTA submission should read this guidance directly rather than relying on secondary summaries, since it is explicit about the scenarios (small, well-characterised natural history; large, unambiguous treatment effect) where FDA considers the design most defensible, and equally explicit about where it is skeptical.

Common pitfall. The single most common failure mode in external control arm analyses submitted for HTA review is presenting the propensity-matched comparison as if matching had resolved all comparability concerns, without separately addressing **temporal** comparability. A registry cohort treated five or ten years before the single-arm trial recruited is not just a different sample of patients — it reflects an entirely different standard of supportive care, diagnostic practice, and often a different definition of the outcome itself (TB treatment-success definitions, for instance, have shifted with WHO guideline revisions). Matching on measured baseline covariates does nothing to correct for era effects that operate through the **outcome-generating process** rather than through baseline patient characteristics. Always report the calendar period of both cohorts explicitly, justify why era drift is not expected to materially bias the comparison (or quantify how much it plausibly could, borrowing the E-value logic from §15.9), and treat any external-control-arm estimate as inherently more fragile than a concurrently randomised one when presenting it to a reimbursement committee.

Treatment switching, unmeasured confounding, federated OMOP networks, and external control arms all share a common thread: once you step outside a cleanly randomised, concurrently controlled trial, every causal estimate is only as credible as the assumptions propping it up, and this chapter's job has been to make those assumptions visible, testable where possible, and honestly reported where they are not. With that causal-inference foundation in place, Chapter 16 turns to the regression toolkit that sits on top of it — generalised linear models, mixed-effects models for clustered and longitudinal HEOR data, zero-inflated and hurdle models for cost and utilisation outcomes, quantile and ordinal regression for distributional and categorical endpoints, and machine-learning approaches to heterogeneous treatment effects and subgroup analysis — the modelling machinery that turns a defensible causal estimand into a defensible, decision-ready number.

Chapter 16. Regression Models for HEOR Data

Chapter 15 estimated treatment effects while adjusting for confounding — propensity scores, doubly robust estimators, instrumental variables. This chapter steps back to the regression machinery those methods are built from, and extends it in two directions the earlier chapters only touched in passing: handling the specific outcome shapes HEOR data actually takes (binary, count, skewed, ordinal), and handling the structure HEOR data actually has (patients clustered within clinics, treatment effects that vary across patients rather than sitting at one average value). By the end of the chapter, the regression toolkit used everywhere else in this book — cost models in Chapter 9, utility mapping in Chapter 8, the GLM core of the propensity-score and doubly-robust estimators in Chapter 15 — will sit inside one coherent framework.

Learning objectives

- Fit and interpret the three GLM families used throughout HEOR — binomial/logistic, Poisson/negative-binomial, and gamma — and correctly back-transform coefficients through the link function.
- Recognise when HEOR data is clustered (patients within clinics, repeated measures within patients) and fit a mixed-effects model with `lme4`.
- Model overdispersed and zero-inflated count outcomes, and distinguish a zero-inflated model from a hurdle model.
- Use quantile regression to model a specific percentile of a skewed cost or length-of-stay distribution, and ordinal regression for Likert-scale PRO data.
- Use random/causal forests and penalised regression (LASSO/elastic net) to estimate treatment-effect heterogeneity and select variables from a high-dimensional covariate set.
- Distinguish pre-specified from exploratory subgroup analysis, test effect modification via an interaction term, and avoid the multiplicity pitfall of subgroup fishing.

16.1 Generalised linear models

Every regression model you have fit so far in this book — the gamma cost models in Chapter 9, the logistic-type transformations behind utility regression in Chapter 8 — is a special case of one framework: the **generalised linear model** (GLM). A GLM has three ingredients: a distribution for the outcome from the exponential family (binomial, Poisson, negative binomial, gamma, Gaussian, ...), a **linear predictor** ($\eta = \beta_0 + \beta_1 x_1 + \ldots$), and a **link function** that connects the two ($g(\mu) = \eta$). Once you see the pattern, "which model do I fit?" becomes "which distribution matches my outcome's support, and which link makes the linear predictor behave?" — rather than a new method to memorise every time.

> **Difficult terminology: exponential family.** "Exponential family" is not a statement about exponential growth — it's a technical label for a whole group of probability distributions (normal/Gaussian, binomial, Poisson, negative binomial, gamma, and a handful of others) that

all share the same underlying mathematical form once you write them out algebraically. That shared form is the reason a single piece of software machinery — the GLM-fitting algorithm — can estimate all of them: R doesn't need a bespoke fitting routine for "binary outcome regression" and a different one for "count outcome regression"; it needs one general algorithm plus a specification of **which** exponential-family distribution and **which** link function apply to your outcome. Practically, you never need to verify exponential-family membership yourself — the handful of distributions you'll actually use in HEOR work (binomial for yes/no outcomes, Poisson or negative binomial for counts, gamma for skewed positive costs, Gaussian for roughly symmetric continuous outcomes) are all members, which is precisely why `glm()` can fit all of them through the same `family =` argument.

Three outcome types cover most HEOR regression work:

- **Binary outcomes** (treatment success/failure, culture conversion, mortality) use the binomial family with a logit link — logistic regression. The logit link keeps predicted probabilities between 0 and 1.
- **Count outcomes** (hospitalisations, adverse-event episodes, clinic visits) use a Poisson or negative-binomial family with a log link. Negative binomial is the safer default in practice: TB programme count data is almost always overdispersed (variance exceeds the mean), and Poisson's assumption that they're equal is routinely violated.
- **Skewed, strictly positive cost data** uses a gamma family with a log link — exactly the model built in detail in §9.4. This subsection will not re-derive it; the mechanics (mean-scale modelling via the log link, no retransformation bias) are identical, because it is the same GLM framework applied to a different exponential-family member.

In base R, all three are one function: `glm()`. You only change the `family` argument.

```
library(dplyr)
library(broom)

# Toy multi-site TB treatment-outcome dataset
tb_outcomes <- tibble::tribble(
  ~patient_id, ~regimen,    ~age, ~baseline_smear, ~success,
  1,           "standard",  34,   "positive",      1,
  2,           "standard",  45,   "positive",      0,
  3,           "shortened", 29,   "negative",      1,
  4,           "shortened", 51,   "positive",      1,
  5,           "standard",  38,   "negative",      0,
  6,           "shortened", 61,   "positive",      0,
  7,           "shortened", 27,   "negative",      1,
  8,           "standard",  42,   "positive",      1
)

fit_logit <- glm(
  success ~ regimen + age + baseline_smear,
  family = binomial(link = "logit"),
  data   = tb_outcomes
)

# Tabulate coefficients, odds ratios and confidence intervals
broom::tidy(fit_logit, exponentiate = TRUE, conf.int = TRUE)
```

```
# Predicted probability of success for a new patient
predict(
  fit_logit,
  newdata = tibble::tibble(regimen = "shortened", age = 40, baseline_smear =
"positive"),
  type    = "response"
)
```

`broom::tidy(..., exponentiate = TRUE)` is doing real work here, not just formatting: it exponentiates the log-odds coefficients into odds ratios so you can report them directly. `predict(..., type = "response")` back-transforms through the inverse link to give a probability rather than a log-odds value. Both arguments matter, and forgetting them is the single most common GLM reporting error.

> **Common pitfall.** A GLM's raw coefficients live on the **link** scale, not the scale anyone wants to read. For a logistic model, `coef(fit_logit)` returns log-odds; for a Poisson or gamma model with a log link, raw coefficients are log-rate or log-cost-ratio. Reporting these numbers unchanged — "age effect: 0.03" — means almost nothing to a TB programme manager. Always exponentiate (`exp(coef(...))`, or `broom::tidy(..., exponentiate = TRUE)`) before presenting results, and say explicitly what the exponentiated quantity is: an odds ratio, a rate ratio, or a cost ratio.

> **Agency perspective.** CHEERS 2022 item 26 ("Characterising heterogeneity") and item 20 ("Study parameters") both expect the statistical model underlying any regression-derived parameter to be named explicitly in the report — not just the resulting estimate. For a GLM, that means stating the family and link function (e.g. "binomial GLM with logit link") alongside the odds ratio or rate ratio, so a reviewer can judge whether the model choice matched the outcome's distribution.

16.2 Mixed-effects and hierarchical models

The GLMs in §16.1 assume every observation is independent. That assumption breaks the moment your data has structure: patients nested within clinics, clinics nested within regions, or repeated HRQoL measurements taken from the same patient over time. Patients treated at the same clinic share a nurse, a supply chain, a local strain profile — they are **more alike than two random patients**, and a plain GLM has no way to represent that.

Fixed effects are the parameters you want to estimate and report: the effect of regimen, of age, of baseline smear status — the things in §16.1's `glm()` calls. **Random effects** represent variation across a grouping factor (clinic, patient) that you want to account for, but whose specific levels aren't individually of interest — you don't care that Clinic 7's intercept is 0.3 higher than Clinic 2's, you care that clinics vary, and you want your fixed-effect estimates and standard errors to be correct given that they do.

Ignoring clustering and fitting a naive GLM when data is actually hierarchical doesn't just bias your standard errors — it specifically makes them too small, because the model treats twenty patients from one clinic as twenty independent pieces of information when they're closer to a handful.

```r
library(lme4)

# Toy multi-site TB dataset: patients nested within clinics
tb_multisite <- tibble::tribble(
  ~clinic_id, ~patient_id, ~regimen,    ~age, ~success,
  "A", 1,  "standard",  34, 1,
  "A", 2,  "standard",  45, 0,
  "A", 3,  "shortened", 29, 1,
  "B", 4,  "shortened", 51, 1,
  "B", 5,  "standard",  38, 0,
  "B", 6,  "shortened", 61, 0,
  "C", 7,  "shortened", 27, 1,
  "C", 8,  "standard",  42, 1,
  "C", 9,  "standard",  33, 0,
)

# Random intercept for clinic, fixed effects for regimen and age
fit_mixed <- glmer(
  success ~ regimen + age + (1 | clinic_id),
  family = binomial(link = "logit"),
  data   = tb_multisite
)

summary(fit_mixed)
broom.mixed::tidy(fit_mixed, effects = "fixed", exponentiate = TRUE, conf.int =
TRUE)
```

`lme4::glmer()` is the binomial/Poisson/gamma-family counterpart of `lme4::lmer()` (which fits continuous, Gaussian outcomes — the natural choice for repeated HRQoL scores measured on the same patient over time). The `(1 | clinic_id)` term is the random intercept: one baseline log-odds per clinic, drawn from a shared normal distribution, rather than one fixed-effect dummy per clinic eating up degrees of freedom. `REML = TRUE` (the default for `lmer()`, though `glmer()` always uses maximum likelihood) controls how the random-effect variance is estimated — leave it at its default unless you have a specific reason not to. `nlme::lme()` is an older alternative with the same core idea, mainly still seen when you need its more flexible correlation-structure syntax (e.g. for repeated-measures autocorrelation).

How much clustering is there? The **intraclass correlation coefficient** (ICC) answers that: it's the proportion of total outcome variance attributable to the grouping factor (clinic), versus variance between patients within the same clinic. An ICC near 0 means clinics barely differ and a plain GLM would have been fine; an ICC of 0.15 means a meaningful share of outcome variation sits at the clinic level, and ignoring it would understate your uncertainty.

A **random intercept** (`(1 | clinic_id)`) lets each clinic have its own baseline outcome level. A **random slope** (`(1 + age | clinic_id)`) goes further and lets the **effect** of a covariate — say, the age effect — vary by clinic too, useful when you suspect, for instance, that older patients do worse at some clinics but not others. Start with random intercepts; add random slopes only when you have a substantive reason to think an effect itself varies by group, since each additional random-effects term costs you estimability, especially with as few clusters as a multi-site TB programme typically has.

Agency perspective. NICE's Decision Support Unit technical support document on evidence synthesis for clinical effectiveness (TSD 3) explicitly recommends a random-effects (hierarchical) model over a fixed-effect pooling when between-study — or, by direct analogy, between-clinic — heterogeneity is plausible, rather than treating pooled homogeneity as a safe default assumption.

16.3 Count data and zero-inflated models

Count outcomes show up constantly in TB programme data: hospitalisations per patient-year, missed follow-up visits, adverse events reported per participant. The instinct is to reach for Poisson regression. The problem is that Poisson assumes the mean equals the variance, and real programme data almost never behaves that politely — a few patients rack up many events while most have none or one, so the variance outruns the mean. This is overdispersion, and ignoring it gives you standard errors that are too small and p-values that look better than they should.

The fix for plain overdispersion is the negative binomial model, fit with `MASS::glm.nb()`. It has the same formula interface as `glm()` but estimates an extra dispersion parameter instead of assuming it away:

```
library(MASS)

fit_nb <- glm.nb(ae_count ~ arm + age + baseline_risk, data = tb_ae)
summary(fit_nb)
```

`glm.nb()` reports the usual coefficient table on the log scale (exponentiate for rate ratios, same logic as Poisson), plus a `theta` value — smaller `theta` means more overdispersion.

Overdispersion is one problem; **excess zeros** are a different one. Suppose you're counting adverse events and most patients genuinely never have one — not because of under-reporting, but because they simply didn't experience an AE. A negative binomial model can stretch to accommodate more zeros than Poisson expects, but when the zeros come from a distinct process (a patient who could never have an AE vs. one who could but didn't), you need a model that says so explicitly: a zero-inflated model, fit with `pscl::zeroinfl()`.

```
library(pscl)

fit_zinb <- zeroinfl(
  ae_count ~ arm + age | arm,
  data  = tb_ae,
  dist  = "negbin"
)
summary(fit_zinb)
```

The formula has two parts separated by `|`: the left side models the count (how many events, given you're in the "can have events" group), the right side models the probability of being a structural zero. `dist = "negbin"` lets the count part also absorb ordinary overdispersion; `dist = "poisson"` is the simpler default.

A closely related but conceptually different model is the **hurdle** model, available via `pscl::hurdle()`. Zero-inflated and hurdle models both have a two-part structure, but they answer

different questions. A zero-inflated model says some zeros are "structural" (could never have had an event) and some are "sampling" zeros from the count process itself — the two kinds of zero are mixed together. A hurdle model instead says every subject first clears a hurdle (any events at all, yes/no), and only those who clear it move to a truncated count model for **how many**. For AE counts, zero-inflation is usually the more defensible story (patients who were never going to have an AE vs. those who could but didn't); hurdle models fit better when crossing from zero to one event is genuinely a different mechanism from the counting process above one.

HEOR in practice. A payer dossier for a new TB regimen reported adverse-event counts using plain Poisson regression. A reviewer noted that over 60% of patients had zero AEs and asked the sponsor to justify the distributional assumption. Re-analysis with `pscl::zeroinfl()` separated "never going to have an AE" patients from the count process for those who did, and the treatment effect on AE **rate** (among patients who had any) held up — but the original Poisson model had understated the uncertainty around it, almost triggering a resubmission.

Common pitfall. Fitting plain Poisson regression to overdispersed or zero-heavy count data is one of the most common silent errors in HEOR analysis. The point estimates often look reasonable; it's the standard errors that are wrong — usually too small — which inflates Type I error and can make a null effect look significant. Always check the ratio of residual deviance to degrees of freedom (well above 1 signals overdispersion) and tabulate the proportion of zeros before trusting a Poisson model.

Difficult terminology: Type I error. A Type I error is the statistical term for a "false positive": you conclude there is a real effect or difference (e.g., a drug lowers cost, or a treatment changes outcomes) when, in the underlying truth, there is no such effect, and what you saw was just random noise in your particular sample. The conventional threshold of $\alpha = 0.05$ means you've agreed to accept a 5% chance of this kind of false alarm whenever a true null effect is tested. The problem flagged in the pitfall above is that bad standard errors (too small) make p-values look smaller than they should, so you end up crying wolf more than 5% of the time without realising it — the nominal 5% false-positive rate quietly becomes something larger, like 15-20%, because the model is overconfident about its own precision.

Difficult terminology: residual deviance / degrees of freedom. These are two separate numbers that, together, give you a quick diagnostic check. **Residual deviance** is a single number summarising how far the model's fitted values are from the actual observed data — think of it as a generalisation of "sum of squared errors" that works for GLMs, not just ordinary linear regression; smaller is a better fit. **Degrees of freedom** here is, roughly, the number of observations in your dataset minus the number of parameters your model estimated — it reflects how much independent information was left over after fitting. For a well-behaved Poisson model, textbook theory says these two numbers should come out roughly equal, so dividing deviance by degrees of freedom should give something close to 1. If that ratio comes out well above 1 (say, 3 or 5), it is a red flag that your data has more variability than a plain

> Poisson distribution assumes — this is called **overdispersion**, and it is exactly the kind of thing that inflates Type I error as described above. In R, both numbers are printed automatically at the bottom of `summary(my_glm_model)`, so the check costs nothing beyond glancing at output you already have.

16.4 Quantile and ordinal regression

Chapter 9 made the case that cost data is skewed — a long right tail of expensive patients pulls the mean around, which is why GLMs with a log link or gamma family were preferred over OLS on raw costs. Quantile regression takes a different route to the same problem: instead of modelling the **mean** cost (transformed or not), it models a chosen **percentile** of the cost distribution directly.

This matters because the mean and the median (or the 90th percentile) can tell different stories. A budget-holder asking "what does a typical TB patient's length of stay cost?" wants the median. A capacity planner asking "what should we provision for the costly tail?" wants the 90th or 95th percentile. Quantile regression, via `quantreg::rq()`, answers either question in one formula-based call:

```
library(quantreg)

# Median length of stay
fit_median <- rq(los ~ arm + age + comorbidity_score, data = tb_cost, tau = 0.5)

# 90th percentile length of stay — the costly tail
fit_q90 <- rq(los ~ arm + age + comorbidity_score, data = tb_cost, tau = 0.9)

summary(fit_median)
summary(fit_q90)
```

`tau` sets the quantile (0.5 is the median; any value in (0,1) is valid). The coefficients are interpreted like linear regression coefficients, but for that specific quantile — a coefficient from the `tau = 0.9` model describes how a predictor shifts the **top** of the distribution, which can differ meaningfully from its effect on the median. This complements, rather than replaces, the Chapter 9 approach to skewed cost data: use GLMs when you want a single mean-cost estimate for a budget-impact model, and quantile regression when the question is genuinely about a specific point in the distribution.

The second regression family here is for ordinal outcomes — Likert-scale patient-reported outcomes (PROs), staged disease severity, anything with ordered categories but no meaningful numeric distance between them. Chapter 8 already introduced `MASS::polr()` for this, in the context of mapping ordinal clinical descriptors to utility weights; the same function applies directly to ordinal PRO analysis, so there's no need to re-derive it here — see Chapter 8 for the syntax and interpretation of the proportional-odds coefficients. An alternative with a richer set of diagnostics and link-function options is `ordinal::clm()`, which uses the same proportional-odds logic but is often preferred when you need formal model comparison tools or non-logit links.

Both `polr()` and `clm()` rest on the **proportional-odds assumption**: that the effect of each predictor is the same across every cut-point of the ordinal scale (e.g. a treatment's effect on moving from "poor" to "fair" is assumed equal to its effect moving from "good" to "excellent"). This

assumption is routinely **assumed** and rarely **checked** in practice, which is a problem — it can fail, especially with PRO scales that compress responses at one end.

Common pitfall. Proportional odds is a testable assumption, not a formality. A quick check is to fit separate binary logistic models at each cut-point of the ordinal outcome and see whether the coefficients for key predictors are roughly stable across cut-points; a formal Brant-type test is also commonly used, though it is not bundled directly into `MASS::polr()` and typically requires a separate package or manual implementation. If the assumption clearly fails, consider a partial proportional-odds model or report cut-point-specific effects instead of a single pooled odds ratio.

Difficult terminology: Brant-type test. The proportional-odds model (used for ordinal outcomes — things with a natural order but no fixed numeric scale, like a pain-severity rating of none/mild/moderate/severe) makes a strong simplifying assumption: that a predictor's effect is the **same** regardless of which cut-point of the outcome you look at (e.g., "mild-or-worse vs. none" and "severe vs. everything-else" are assumed to move by the same amount for a one-unit change in the predictor). A Brant-type test is a formal statistical check of that assumption — it compares the coefficients estimated separately at each cut-point (as described in the paragraph above) and produces a p-value testing whether those separate estimates are statistically consistent with coming from one shared, constant effect. A small p-value here is evidence the proportional-odds assumption is shaky for at least one predictor, and you should consider the alternatives mentioned above (a partial proportional-odds model, or reporting cut-point-specific effects) rather than trusting a single pooled odds ratio.

16.5 Machine learning for prediction and heterogeneity of treatment effect

Chapter 15 built the propensity-score and doubly-robust toolkit for estimating an **average** treatment effect while adjusting for confounding. Two questions that toolkit does not answer on its own: does the treatment effect vary across patients, and which of many candidate covariates actually matter? This section introduces the machine-learning answers to both, kept deliberately narrow — one method per question, one worked example each.

Random forests for prediction. `randomForest::randomForest()` and the faster, more memory-efficient `ranger::ranger()` both fit ensembles of decision trees for an outcome — useful when you want a flexible prediction model (e.g. predicting 6-month treatment-failure risk from baseline labs) without committing to a linear or logistic functional form. Neither function, used this way, estimates a **treatment effect** — they predict an outcome.

Causal forests for treatment-effect heterogeneity. To ask "does the treatment effect itself vary by patient," the `grf` package's `grf::causal_forest()` function grows a forest tuned specifically to estimate individual-level treatment effects, not outcomes. It takes a covariate matrix `X`, an outcome vector `Y`, and a treatment indicator `W`, and returns per-observation treatment-effect estimates via `predict()`. This is the ML-flexible cousin of Chapter 15's doubly-robust estimators: where a DR estimator assumes a parametric (or semiparametric) form for the propensity and outcome models, `causal_forest()` lets the data determine, nonparametrically, **where** treatment effects are larger

or smaller — directly surfacing heterogeneity of treatment effect (HTE) that a single DR average effect would hide.

```
library(grf)

# X: baseline covariates, W: treatment (0/1), Y: outcome (e.g. treatment success)
cf <- causal_forest(X = as.matrix(tb_data[, c("age", "bmi",
"baseline_chest_xray_score")]),
                    Y = tb_data$treatment_success,
                    W = tb_data$received_new_regimen)

# Per-patient estimated treatment effect
tau_hat <- predict(cf)$predictions
summary(tau_hat)
```

A rising spread in `tau_hat` across patients is evidence of HTE; `grf` also provides `best_linear_projection()` and related diagnostics to summarise **which** covariates drive it — useful for generating hypotheses that a pre-specified subgroup analysis (§16.6) can then test formally.

LASSO and elastic net for variable selection. When the candidate covariate list is long relative to sample size — dozens of biomarkers, comorbidity flags, or lab values — ordinary regression overfits. `glmnet::glmnet(x, y, alpha = , lambda = )` fits penalised regression: `alpha = 1` is the LASSO (shrinks many coefficients exactly to zero, giving automatic variable selection), `alpha = 0` is ridge regression, and values in between are the elastic net. `x` must be a numeric matrix (not a formula/data-frame), and `lambda` controls penalty strength — typically chosen by cross-validation with `cv.glmnet()` rather than set by hand.

```
library(glmnet)

x <- as.matrix(tb_data[, biomarker_cols])   # many candidate biomarkers
y <- tb_data$time_to_culture_conversion

fit <- cv.glmnet(x, y, alpha = 1)           # LASSO, lambda chosen by CV
coef(fit, s = "lambda.min")                  # non-zero coefficients = selected
biomarkers
```

Use this as a screening step — to narrow forty biomarkers to five worth carrying into a clinically-interpretable model — not as a replacement for domain knowledge about which covariates are prognostic versus effect-modifying (§16.6).

16.6 Subgroup analysis and effect modification

Subgroup analysis asks a simple question with a dangerous failure mode: does the treatment effect differ between, say, HIV-co-infected and HIV-negative TB patients? The question is legitimate. The danger is in **how** it gets asked.

Pre-specified versus exploratory. A pre-specified subgroup analysis is written into the statistical analysis plan before the data are unblinded, naming the specific subgroups and the specific interaction test, ideally with a stated biological or clinical rationale (ICH E9(R1), already introduced in Chapter 7's estimand framework, requires this kind of up-front specification for any subgroup finding intended to support a treatment claim). An exploratory or post-hoc subgroup analysis slices

the data after the fact — by age, sex, region, baseline severity, and whatever else is in the dataset — looking for "the subgroup where it worked." Both have a place in a report, but only the first supports a confirmatory claim.

> **Common pitfall.** Testing treatment effects across many unplanned subgroups inflates the false-positive rate the same way testing many outcomes does. Slice a trial twenty different ways and, by chance alone, one or two subgroups will show a "significant" interaction even when the true effect is uniform. Label exploratory subgroup findings as hypothesis-generating, not confirmatory, and do not let a favourable subgroup result rescue an otherwise null overall result.

Testing effect modification. The formal test for whether a subgroup modifies the treatment effect is an interaction term in a regression model — not a pair of separate within-subgroup analyses. For a binary outcome:

```
fit <- glm(treatment_success ~ treatment * hiv_status + age + baseline_severity,
           data = tb_data, family = binomial())
summary(fit)   # coefficient on treatment:hiv_status is the test of effect
modification
```

A significant `treatment:hiv_status` coefficient says the treatment effect on the odds scale differs by HIV status; a non-significant one is evidence **against** effect modification on that scale, though often underpowered to detect it — subgroup interaction tests typically need several times the sample size of the main-effect test.

Displaying subgroup effects. Once effect estimates are obtained within each subgroup, the forest plot is the standard display — and the mechanics are exactly the ones built in Chapter 11 for meta-analysis: either a `ggplot2` point-and-error-bar plot, or `metafor::forest()` applied to a set of subgroup-specific effect estimates and standard errors in place of study-specific ones. No new plotting machinery is needed here; only the question "study" versus "subgroup" changes.

Prognostic factor versus effect modifier, again. Chapter 15 §15.3 drew this distinction using a DAG: a prognostic factor predicts outcome regardless of treatment, while an effect modifier changes the **size** of the treatment effect itself, and only the latter justifies treatment decisions that vary by subgroup. That distinction governs subgroup analysis directly — a subgroup defined by a strong prognostic factor (e.g. baseline disease severity) may show different **absolute** risks in each arm without the treatment's relative effect differing at all. Always check the interaction term before concluding that a subgroup's apparently different results reflect true effect modification rather than prognosis.

> **Difficult terminology: DAG (directed acyclic graph).** A DAG is a diagram — boxes for variables, arrows for assumed cause-and-effect — that makes your causal assumptions explicit and checkable rather than leaving them implicit in your head. "Directed" means every arrow has a direction (cause → effect, not just "these two are associated"); "acyclic" means you can't follow the arrows in a loop back to where you started (no variable can be its own cause, directly or indirectly). For example, a simple DAG might show "disease severity → treatment choice" and "disease severity → outcome" and "treatment choice → outcome": severity is a **confounder** here because it feeds into both the thing you're choosing (treatment) and the

thing you're measuring (outcome), and you'd draw that structure before touching any data, based on clinical knowledge. Epidemiologists and causal-inference statisticians use DAGs precisely because they force you to commit, on paper, to which variables you believe are confounders (common causes of both exposure and outcome, which you must adjust for) versus which are simply downstream consequences of treatment (which you must **not** adjust for, since doing so can introduce bias rather than remove it). You don't need to draw formal DAGs to use this book's methods, but understanding that a DAG is simply "your causal assumptions, drawn out and made explicit" demystifies why Chapter 15 leans on them so heavily.

Agency perspective. EMA's "Guideline on the investigation of subgroups in confirmatory clinical trials" (EMA/CHMP/539146/2013) sets out regulatory expectations for exactly this distinction: pre-specification, adequate power for interaction tests, and caution against over-interpreting post-hoc subgroup findings — the same principles argued for above, from the regulator's side of the table.

Chapter 16 has moved from the single regression workhorses — linear, logistic, Poisson/negative binomial, survival — through their ML-flexible and heterogeneity-aware extensions. Part III's statistical toolkit is now in place: estimating effects, adjusting for confounding, pooling across studies, and characterising who responds differently. Chapter 17 opens Part IV, Decision-Analytic Modelling, and shifts the question from "what effect did we estimate" to "what should a health system do about it" — starting with how to conceptualise a decision model, choose a model type, and document its structure before a single cost or probability is ever entered.

PART IV

Decision-Analytic Modelling

Chapter 17. Principles of Decision Modelling

Part III closed the estimation toolkit: effects, confounding adjustment, pooling, heterogeneity. Those chapters answered "what does the evidence say happened, or what would happen under a given treatment?" Chapter 17 asks a different question: given that evidence, what should a health system **do**? Decision-analytic modelling is the machinery for turning scattered estimates of effect, cost, and quality of life into a single comparative answer – which TB treatment regimen should a programme adopt, and under what uncertainty?

This chapter is a map, not a computation. It walks through the decisions a modeller makes before writing a line of model code: what the decision problem actually is (17.1), which model type fits that problem (17.2), how to draw and defend the structure (17.3), what kinds of parameters feed it (17.4), how to document choices so others can audit them (17.5), how an R project should be organised to keep inputs, engine, and outputs cleanly separated (17.6), and which professional standards govern all of this (17.7). Chapters 18 through 22 then build the actual engines – decision trees, Markov models, partitioned survival, and beyond.

Learning objectives

- Scope a decision problem in HEOR terms: perspective, comparators, time horizon, and the question a model is meant to answer (17.1).
- Choose between decision trees, Markov models, partitioned survival, and other model types based on how a disease unfolds over time (17.2).
- Draw a model structure as a diagram in R using the DiagrammeR package, and justify structural assumptions explicitly (17.3).
- Classify model parameters by type – probabilities, costs, utilities, relative effects – and match each to an appropriate distribution for later uncertainty analysis (17.4).
- Write a model documentation record that makes every structural and parameter choice traceable and auditable (17.5).
- Organise an R decision-model project into separate inputs, engine, results, and reporting layers (17.6).
- Locate a decision model's obligations within the ISPOR-SMDM Modeling Good Research Practices series and the NICE reference case (17.7).

17.1 Model conceptualisation and problem scoping

Before any R code is written, a decision model needs a clear statement of the question it is answering. This sounds obvious, but a surprising share of modelling projects go wrong here: the population drifts mid-project, the comparator list grows without anyone noticing, or the time horizon gets chosen by habit rather than by the disease biology. Chapter 4, section 4.6 already introduced the core framing vocabulary for an economic evaluation – population, comparators, perspective, and time horizon. This chapter reuses those four terms exactly as defined there; we do not re-derive them.

HEOR in practice. Before opening RStudio, write down, in one sentence each: the population, the comparators, the perspective, and the time horizon for your model. If you cannot write all four sentences, you are not ready to model yet – you are still scoping.

The PICOTS framework. HEOR borrows PICO from clinical epidemiology and extends it for economic evaluation: Population, Intervention, Comparator(s), Outcomes, Timing, Setting. PICOTS is simply a checklist that forces the same four ideas from 4.6 into a slightly finer grain – splitting "comparators" into intervention vs. comparator, and adding outcomes, timing, and setting as explicit fields.

Take a TB programme decision as the running example for this chapter: should a national TB programme switch standard drug-susceptible TB patients from the 6-month daily regimen to a 4-month rifapentine-moxifloxacin regimen (as recommended following the Study 31/A5349 trial)? The table below sketches the PICOTS for this question.

PICOTS element	TB regimen example
Population	Adults with newly diagnosed, drug-susceptible pulmonary TB
Intervention	4-month rifapentine-moxifloxacin regimen
Comparator	6-month standard daily regimen (2HRZE/4HR)
Outcomes	Treatment success, relapse, cost per patient treated, QALYs
Timing	Follow-up to 12 months post-treatment-completion, to capture relapse
Setting	Public TB programme clinics, national perspective

Notice that "Timing" here is not the same as the model's time horizon. Timing in PICOTS describes the follow-up window of the **evidence** informing the model; the time horizon (4.6) is a separate modelling choice about how far forward the **model** itself projects, which may need to extend well beyond 12 months to capture downstream relapse and retreatment costs.

Common pitfall. A common scoping error is letting the comparator list expand once the project has started – for example adding a shorter experimental regimen under trial, "just in case it's useful later." Every added comparator multiplies the data you need (efficacy, costs, utilities) and the validation burden. Lock the comparator list during scoping, and treat any addition as a formal scope change, not a quick extra column.

The model concept document. Once PICOTS is settled, write it up as a short model concept document – typically one to two pages – before touching R. This document states the decision problem, the PICOTS elements, the intended model type (chosen in 17.2), the main data sources you expect to use, and who the model is for (a funding ministry, a programme manager, a national guideline committee). Its purpose is to let clinical and economic stakeholders sign off on the **question** before anyone invests time building the **answer**. For our TB example, the concept document would note that the programme's budget cycle and WHO guideline review calendar may constrain the time horizon or the urgency of results – decisions that are much cheaper to revisit on paper than inside a half-built R script.

17.2 Choosing a model type

Part IV covers five model types, one per chapter: decision trees (chapter 18), Markov cohort models (chapter 19), partitioned survival models (chapter 20), individual-level microsimulation (chapter 21), and discrete event simulation (chapter 22). This section does not explain how any of them work – that is the job of chapters 18 through 22. Instead it gives you a framework for choosing **which** one fits a given decision problem, so that when you reach each later chapter you already know why it is the right tool for that chapter's example.

The ISPOR-SMDM Modeling Good Research Practices Task Force produced a series of papers exactly on this question. The task force report most relevant to choosing among state-transition approaches is Siebert et al. (2012), "State-Transition Modeling: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force–3." It is worth reading once you reach chapter 19, but its selection logic is useful right now.

Five questions that narrow the choice. Rather than memorising five model definitions, ask five questions about the disease and decision problem:

1. **Is this a single, one-off decision with a short time horizon and no need to track disease progression over time?** If yes, a decision tree (ch. 18) is usually sufficient – and simpler is better when it is adequate.
2. **Does the disease move between a small number of discrete, recurring health states over time (e.g., "on treatment," "relapsed," "cured")?** This favours a Markov cohort model (ch. 19), the workhorse of chronic-disease HEOR.
3. **Is the treatment effect expressed mainly as survival curves or time-to-event outcomes, without well-defined intermediate health states to model explicitly?** This favours a partitioned survival model (ch. 20) – common in oncology, where trial data arrive as progression-free and overall survival curves rather than discrete state transitions.
4. **Does a patient's future risk depend on their individual history – how long they have already been in a state, prior treatment lines, accumulated events – in a way a cohort-level model cannot capture?** This favours individual-level (microsimulation) modelling (ch. 21).
5. **Do shared, limited resources, queues, or interactions between patients matter to the outcome** – for example, a diagnostic machine with finite daily capacity, or transmission between patients? This favours discrete event simulation (ch. 22), the only model type in Part IV that routinely models resource constraints explicitly.

These five questions are not mutually exclusive filters applied in strict order; they are prompts. Most real problems answer "yes" to more than one, and the final choice is a judgement about which structural feature matters **most** for the decision at hand, balanced against data availability and stakeholder familiarity with the method.

> **HEOR in practice.** Apply the five questions to the TB regimen example from 17.1. TB treatment has discrete, clinically meaningful recurring states – on treatment, cured, failed, relapsed, retreatment, death – and patients can legitimately cycle between some of them (failure leads back to retreatment). There is no resource queue central to the question, and there is no strong need for patient-level memory beyond state occupancy. Question 2 dominates: a Markov cohort model (ch. 19) is the natural choice, and that is exactly the model type chapter 19 builds for this same example.

Common pitfall. Do not choose a model type because it is the one you already know how to code, or because a past project used it. The Siebert et al. task force report exists precisely because reviewers and HTA bodies expect the model type to follow from the disease's structural features – not from analyst convenience. A mismatch (for example, forcing a partitioned survival model onto a disease with genuinely cyclical relapse-and-retreatment states) can bias results and will draw scrutiny in review.

17.3 Structural assumptions and model diagrams

Once a model type is chosen, the next step – still before writing simulation code – is to decide and document exactly how patients move through the model's health states. These choices are called structural assumptions, and three questions come up in almost every state-transition model:

- **Can patients move backward between states?** For example, can a "cured" TB patient return to "on treatment" (via relapse and retreatment), or is cure a one-way gate?
- **Is there a tunnel state** – a state a patient can only occupy for a fixed number of cycles before being forced onward, used to represent things like a fixed treatment course length?
- **What happens at the end of the model's time horizon?** Do remaining patients in non-death states simply stop accruing costs and effects (a "closed cohort" assumption), or does the model apply some extrapolation beyond the explicit horizon?

Writing these answers down as explicit prose statements, before drawing anything, is what prevents disagreements from surfacing only after the R code is built and the numbers look odd.

Drawing the structure. A model diagram turns those prose statements into a picture that clinical collaborators can sanity-check at a glance, without reading any R code. The `DiagrammeR` package's `grViz()` function renders diagrams written in the DOT graph description language (the same language used by Graphviz), which makes it well suited to drawing health-state diagrams: states are nodes, transitions are directed edges.

The code below draws the structural diagram for our TB Markov model: patients on treatment can be cured or can fail treatment; failure leads back to re-registration for on-treatment care; and death is reachable as an absorbing state from every other state.

```
library(DiagrammeR)

tb_model_diagram <- grViz("
digraph tb_markov_structure {

  graph [layout = dot, rankdir = LR]
  node [shape = box, style = filled, fontname = Helvetica]

  OnTx    [label = 'On Treatment',      fillcolor = lightblue]
  Cured   [label = 'Cured',             fillcolor = lightgreen]
  Fail    [label = 'Treatment Failure', fillcolor = lightsalmon]
  Death   [label = 'Death',             fillcolor = grey85]

  OnTx  -> Cured  [label = 'treatment success']
  OnTx  -> Fail   [label = 'treatment failure']
  Fail  -> OnTx   [label = 're-registration']
```

```
  OnTx  -> Death  [label = 'mortality']
  Cured -> Death  [label = 'mortality']
  Fail  -> Death  [label = 'mortality']
}
")

tb_model_diagram
```

Running this in RStudio (or a Quarto/R Markdown chunk) renders an interactive diagram directly in the viewer pane. The `rankdir = LR` argument lays the states left to right, which tends to read more naturally for treatment-pathway diagrams than the default top-to-bottom layout. Each `label` on an edge documents **why** the transition exists – a habit worth keeping, because it turns the diagram itself into a piece of the structural-assumptions documentation rather than just decoration.

Note what the diagram already answers from our three questions above: backward movement exists (`Fail -> OnTx`), there is no tunnel state in this simplified version, and death is absorbing and reachable from everywhere – all visible without reading a single line of transition-probability code.

> **Agency perspective.** NICE's reference case for technology appraisals, together with NICE's Decision Support Unit technical support document series, expects submissions to include a clearly documented and justified model structure – typically as a diagram of exactly this kind, accompanied by prose justifying each state and transition. Producing the diagram at the structural-assumptions stage, rather than retrofitting one after the model is built, makes this requirement far easier to satisfy.

> **Common pitfall.** A model diagram is not optional polish added for a report's appendix. If you cannot draw your model's states and transitions in a few minutes using `grViz()`, you likely do not yet have a clear enough structural assumption set to start building the model in R – go back to the three questions above before writing any transition-probability code.

17.4 Parameter taxonomy and distributions

Every decision-analytic model is built from four kinds of parameters, and getting the taxonomy straight matters because each kind draws its evidence from a different place in this book and takes a different distribution in probabilistic sensitivity analysis (PSA).

Transition probabilities move patients between health states each cycle: cured, on treatment, lost to follow-up, dead. **Costs** attach resource use in each state to a price. **Utilities** convert time in a state into quality-adjusted life years (QALYs). **Relative treatment effects** (hazard ratios, risk ratios, odds ratios) scale one arm's parameters relative to a comparator.

Costs and utilities are not estimated inside the decision model – they are imported. Chapter 9 covered fitting GLMs to costing data and the gamma/lognormal distributions that describe the resulting uncertainty; Chapter 8 covered sourcing utility weights (EQ-5D mapping, vignette studies, literature tariffs) and their beta-like uncertainty. Chapter 17 does not re-derive any of that – it just slots the fitted mean and standard error into the model's parameter table. Likewise, relative

treatment effects come from the pooled estimates in Chapters 11 (pairwise meta-analysis) and 12 (network meta-analysis): the pooled log hazard ratio and its standard error from a random-effects model there become the treatment-effect parameter here.

What **is** new in a decision model is the transition probability itself, because trials rarely report it directly – you typically reconstruct it from a reported proportion with a confidence interval, and that calls for a distribution that respects the 0-1 bound.

> **HEOR in practice. Why bounded distributions, not normal.** A transition probability must stay between 0 and 1 in every PSA draw. A normal distribution can return a draw below 0 or above 1, which produces a model error or a silently nonsensical result. The beta distribution is defined on [0, 1] and is the standard choice for single probabilities; the Dirichlet distribution is its multivariate generalisation, used when several destination probabilities from one state (e.g. cured / still on treatment / dead) must each stay in [0, 1] **and** sum to exactly 1 every draw.

The conventional distributional choices, by parameter type, are:

- **Probabilities and proportions** – Beta, because it is bounded on [0, 1] and matches the binomial likelihood that usually generated the estimate.
- **Multinomial transition probabilities** (more than two destinations from one state) – Dirichlet, so the simulated probabilities always sum to 1.
- **Costs** – Gamma or lognormal, because cost data are right-skewed and bounded at 0 (see Chapter 9 for the GLM machinery that produces the mean and variance going into these distributions).
- **Utilities** – Beta when the utility itself is bounded in [0, 1] (most adult EQ-5D-3L tariffs), or Dirichlet when utility decrements are modelled as a set of mutually exclusive health-state weights that must sum to 1.
- **Relative treatment effects** – usually normal on the log scale (log hazard ratio, log odds ratio), because that is the scale on which the meta-analytic standard error from Chapters 11-12 was estimated.

Fitting a beta distribution to a transition probability is pure algebra: the method-of-moments equations convert a mean and variance into the two beta shape parameters, alpha and beta.

$$\alpha = \mu \cdot \left(\frac{\mu(1-\mu)}{\sigma^2} - 1 \right), \quad \beta = (1-\mu) \cdot \left(\frac{\mu(1-\mu)}{\sigma^2} - 1 \right)$$

Suppose a trial reports a 12-month cure probability of 0.85 with a 95% CI of (0.80, 0.90). Treating the CI as approximately normal on the probability scale, the standard error is the half-width divided by 1.96, and the variance is its square.

```
# Method-of-moments beta fit from a trial's reported mean and 95% CI
mean_p <- 0.85
ci_lower <- 0.80
ci_upper <- 0.90

se_p <- (ci_upper - ci_lower) / (2 * 1.96)
var_p <- se_p^2

common_term <- (mean_p * (1 - mean_p) / var_p) - 1
alpha_p <- mean_p * common_term
beta_p  <- (1 - mean_p) * common_term
```

```
alpha_p  # 165.68
beta_p   #  29.24

# Check: does Beta(alpha_p, beta_p) reproduce the mean and SE we started with?
fitted_mean <- alpha_p / (alpha_p + beta_p)
fitted_var  <- (alpha_p * beta_p) / ((alpha_p + beta_p)^2 * (alpha_p + beta_p +
1))
fitted_mean          # 0.85 -- matches
sqrt(fitted_var)     # 0.0255 -- matches se_p
```

The fitted shape parameters are alpha approx 165.68 and beta approx 29.24, and the check confirms the round trip: a `Beta(165.68, 29.24)` distribution has mean 0.85 and standard error 0.0255, exactly what the trial reported. In Chapter 25 this `Beta(alpha_p, beta_p)` object is what gets sampled thousands of times inside the PSA loop; this chapter's job stops at specifying the distribution correctly.

Common pitfall. Large alpha and beta values (as above) mean the beta distribution is tightly concentrated around the mean – appropriate for a well-powered trial. A small trial arm (say n = 20) produces much smaller shape parameters and a visibly wider, sometimes skewed distribution. Always sanity-check the fitted alpha and beta against the sample size implied by the reported CI; a mismatch usually means the CI was log-scale or Wilson-interval, not the simple normal approximation used above.

Difficult terminology: Wilson interval (Wilson score interval). When you calculate a confidence interval around a proportion (like "85% of patients responded"), the simplest textbook method — the one most clinicians meet first — assumes the sampling distribution is roughly a symmetric bell curve and builds the interval as "estimate plus or minus some multiple of the standard error." That simple normal-approximation method breaks down badly when the proportion is close to 0% or 100%, or when the sample size is small: it can even produce a nonsensical interval that dips below 0% or climbs above 100%. The Wilson interval (also called the Wilson score interval) is an alternative formula that handles exactly these edge cases properly — it is derived slightly differently (by inverting a hypothesis-testing statistic rather than assuming symmetry) and stays within the valid 0-100% range, and gives more trustworthy coverage for small or extreme-proportion trial arms. As a working rule: if a reported trial result's confidence interval looks asymmetric around the point estimate, or the proportion is near 0 or 1, suspect it was built with a Wilson-type method rather than the simple symmetric approximation, and account for that when back-fitting a Beta distribution as in the worked example above.

17.5 Model documentation and transparency

A decision model that only its author can run is not fit for HTA submission, however correct its arithmetic. Two linked expectations govern this: external bodies expect the model's **logic** to be auditable, and they expect the model's **code** to actually execute on someone else's machine.

ISPOR-SMDM's Good Research Practices Task Force addressed the first expectation directly. Eddy et al. (2012), "Model Transparency and Validation" – one paper in the seven-part task force series on good modelling practice – sets out what a model needs to disclose so that a third party can understand what it does without re-running it: the health states and transitions, every parameter's source and value, every structural assumption, and the validation checks performed against external data. Transparency here does not mean publishing source code; it means a reviewer can reconstruct the model's logic from the written description alone.

NICE's methods guide goes further for UK submissions: it expects an **executable** R (or equivalent) model, fully commented, that NICE's own technical team can run and interrogate. Chapter 2 covered the `renv` project-level dependency locking that makes a submitted model reproducible on a reviewer's machine years later; Chapter 28 will cover the full reporting and submission-package treatment. The point to take from this section is narrower: documentation and code are not substitutes for each other. A commented script explains **how** the model computes; a technical model documentation (TMD) report explains **why** it is built the way it is, for a reader who may never open the script at all.

> **Agency perspective.** NICE technical support documents and the associated reference case checklist specify that a submitted economic model must run without error in a clean environment, with all input parameters either hard-coded with cited sources or loaded from a clearly labelled data file – no undocumented manual steps. This is precisely the reproducibility bar Chapter 2's `renv` workflow targets.

In practice, agencies expect a TMD report structured around five sections, matching the Eddy et al. transparency checklist:

1. **Inputs** – every parameter, its point estimate, its source citation, and its distribution for PSA (the taxonomy from 17.4 feeds directly into this section).
2. **Structure** – the states, cycle length, time horizon, and transitions, with a rationale for each structural choice (Section 17.1-17.3 territory).
3. **Assumptions** – every simplification made and why it was judged acceptable (e.g. no second-line retreatment arm, constant background mortality).
4. **Validation** – face validity checks, comparison against published cohort outcomes, and any cross-validation against an independent model.
5. **Outputs** – the base-case results, and a signpost to where the uncertainty analysis (Chapter 25) and scenario analyses are reported.

> **HEOR in practice.** Write the TMD's "Inputs" section as a literal table with four columns – parameter, value, source, distribution – rather than prose. Reviewers compare this table line-by-line against the R model's parameter list; prose descriptions slow that check down and invite transcription errors between report and code.

The TMD and the executable model are two views of the same object. Treat the TMD as the map and the R script as the territory: if they disagree, the model is wrong even when the code runs without error.

17.6 Modelling workflow in R: inputs -> engine -> outputs -> reporting

A decision model is a pipeline, not a script. Treating it as one script – parameters hard-coded inline, calculations and formatting tangled together – is the single most common cause of models that cannot be audited, extended, or debugged six months later. The fix is architectural, not statistical: separate the model into four layers that each do one job and talk to the next through a clean interface.

The four layers. An *inputs layer* defines every parameter the model needs, as a named list or a tidy tibble, with nothing else happening in that file. An *engine layer* is a function that takes the parameter object and returns raw model output – total cost and total QALYs per strategy – with no knowledge of how those numbers will later be displayed. A *results layer* takes engine output and computes the comparative statistics analysts actually want: incremental cost-effectiveness ratios (ICERs) and net monetary benefit (NMB). Section 4.4 already derived both formulas in full; this layer simply applies them to whatever the engine produced, so nothing here is re-derived. A *reporting layer* turns results into tables, plots, and narrative text. Chapter 28 (Reproducible Reporting) covers this layer properly; for now, treat it as "whatever renders the results object into something a decision-maker reads."

The payoff of this separation is that each layer can be tested, swapped, or re-run independently. Changing a discount rate means editing the inputs file. Changing from a decision tree to a Markov model means replacing the engine function – the results and reporting layers do not change at all, because they only ever see cost and QALY totals per strategy, never the mechanics that produced them.

A worked skeleton. The engine function signature below is deliberately generic. It is a contract, not a working model: given a parameter list, return a tibble with one row per strategy. What happens **inside** the function – branch probabilities for a decision tree, transition matrices for a Markov model, survival curves for partitioned survival – is exactly the subject of Chapters 18 through 22. Nothing about that mechanics belongs in this chapter.

```
# --- inputs layer -----------------------------------------------------
params <- list(
  strategies = c("Regimen_A", "Regimen_B"),
  cost_drug  = c(Regimen_A = 450, Regimen_B = 980),   # per course, local currency
  p_cure     = c(Regimen_A = 0.86, Regimen_B = 0.91),
  qaly_cured = 1.0,
  qaly_fail  = 0.6
)

# --- engine layer -----------------------------------------------------
# Generic signature only -- the real decision-tree / Markov mechanics
# live in Chapters 18-22. Here the engine is a stand-in.
run_model <- function(params) {
  tibble::tibble(
    strategy    = params$strategies,
    total_cost  = params$cost_drug[params$strategies],
    total_qaly  = params$p_cure[params$strategies] * params$qaly_cured +
                  (1 - params$p_cure[params$strategies]) * params$qaly_fail
  )
}
```

```
engine_out <- run_model(params)

# --- results layer ------------------------------------------------------
# ICER and NMB formulas: see Section 4.4. Not re-derived here.
results <- engine_out |>
  dplyr::arrange(total_cost) |>
  dplyr::mutate(
    inc_cost  = total_cost - dplyr::first(total_cost),
    inc_qaly  = total_qaly - dplyr::first(total_qaly),
    icer      = inc_cost / inc_qaly,
    nmb       = total_qaly * 20000 - total_cost   # threshold = 20,000/QALY
  )

# --- reporting layer ----------------------------------------------------
# Tables, plots, narrative -- see Chapter 28 (Reproducible Reporting).
```

> **Agency perspective.** This skeleton is intentionally thin. `run_model()` here does almost nothing, because its real job – simulating disease progression over time – is Chapters 18 to 22's content. What matters in this chapter is the **shape** of the pipeline, not the arithmetic inside any one box.

Two packages that already do this. You do not have to build this architecture from scratch every time. *heemod* is a mature CRAN package that implements exactly this inputs/engine/output separation for Markov cohort models, with its own functions for defining states, transitions, and strategies; Chapter 19 covers it properly. *hesim* does something similar across a broader range of model types, including partitioned survival models; it appears again in Chapters 19 and 20. Neither package is used here beyond naming them – inventing their function calls prematurely would teach the wrong syntax before the right chapter explains it.

> **Common pitfall.** Mixing layers is the fastest way to make a model impossible to debug. If a cost figure looks wrong, you want to be able to check the inputs file in isolation, then the engine's arithmetic, then the results formulas – not wade through three hundred lines where all of that is interleaved with `ggplot2` calls.

17.7 Good practice guidelines (ISPOR-SMDM, NICE reference case)

Everything in this chapter – scoping, model choice, structure, parameters, documentation, workflow – sits inside two governing frameworks that the rest of Part IV assumes without restating. Naming them once, here, is enough.

ISPOR-SMDM Modeling Good Research Practices. A joint ISPOR-SMDM task force, co-chaired by Eddy, Caro, Briggs, Pitman, Sculpher and others, published a seven-paper series in **Value in Health** in 2012 setting out good practice for health-economic modelling end to end. Caro et al. (2012), "Modeling Good Research Practices – Overview," is the umbrella paper that frames the other six. Section 17.2 already cited Siebert et al. (2012) on state-transition (Markov) models specifically, and Section 17.5 already cited Eddy et al. (2012) on model transparency and

validation; both are part of this same series, not separate frameworks. The series as a whole is the methodological backbone for Chapters 18 to 22: whichever model type you build, its task-force paper is the reference for what "good" looks like.

The NICE reference case. Where ISPOR-SMDM sets general modelling practice, the NICE reference case sets a specific national standard: a defined set of methodological choices that HTA submissions to England's National Institute for Health and Care Excellence must follow unless a deviation is explicitly justified. Its core requirements include an NHS and Personal Social Services (PSS) perspective, QALYs as the outcome measure, a specified discount rate for costs and health effects, and a lifetime (or sufficiently long) time horizon. Chapter 4 introduced the reference-case concept when it first discussed perspective and outcome choice; this is the same reference case, now tied to the specific body that issues it.

> **HEOR in practice.** Think of ISPOR-SMDM as answering "is this model built well?" and the NICE reference case as answering "does this model meet **this** payer's specific rules?" A model can satisfy the first and still fail the second – for example, by using a societal perspective where NICE requires an NHS/PSS perspective. Both checks matter, and they are not the same check.

Part IV does not re-litigate either framework chapter by chapter. Where a later chapter makes a choice – a discount rate in Chapter 22, a validation step in a later audit chapter – it will point back here rather than re-arguing the justification.

With the conceptual scaffolding in place, Chapter 18 starts building. Decision trees are the simplest model structure in the ISPOR-SMDM taxonomy from Section 17.2, and the next chapter constructs one from scratch in R: branches, probabilities, expected-value calculations, and the first full `run_model()` engine with real mechanics inside it.

Chapter 18. Decision Trees

Chapter 17 was a map: how to scope a decision problem, choose a model type, and organise the R project around it. This chapter builds the first real engine on that map – the decision tree, the simplest member of the decision-analytic modelling family and the right tool exactly when §17.2′s "one-off/short-horizon" criterion is met: a question that resolves within a single episode of care, with no need to track a cohort through recurring health states over years.

The running example for this chapter is deliberately **different** from Chapter 17′s regimen-choice question (which committed to a Markov cohort model, built in Chapter 19). Instead, §18.1 through §18.5 follow a short-term TB **diagnostic-strategy** decision: a district programme choosing between a cheaper sputum-smear-only algorithm and a Xpert MTB/RIF molecular-test algorithm for patients presenting with TB symptoms. This is squarely tree territory – test, get a result, act on it, done – and §18.5 closes the loop by showing exactly how a tree like this one hands its output to a longer-term Markov model when a diagnosed patient's story continues for years, setting up Chapter 19 directly.

Learning objectives

- Describe a decision tree's three node types – decision, chance, terminal – and compute expected cost and expected QALYs by hand via the roll-back algorithm (18.1).
- Represent a decision tree as a tidy R data structure and compute its expected values with vectorised arithmetic, reproducing a hand calculation exactly (18.2).
- Build the same tree using the `rdecision` package's node/edge classes, and explain what a dedicated package buys over a from-scratch calculation (18.3).
- Recognise the general class of short-horizon problems – screening, diagnostics, acute care – that decision trees suit, beyond the chapter's TB example (18.4).
- Explain how a decision tree's terminal-node probabilities become the starting cohort distribution for a downstream Markov model (18.5).

18.1 Structure and expected-value calculations

A decision tree is the short-horizon cousin of the Markov model you built in Chapter 17: instead of cycling through states over years, it plays out a single episode of care as a branching sequence of chance events, then stops. Every tree has three kinds of nodes. A **decision node** (drawn as a square) is the strategy choice itself – the thing you, the analyst, are being asked to recommend. A **chance node** (a circle) is a point where nature decides the outcome, with branch probabilities that must sum to 1 across all branches leaving that node. A **terminal node** (a triangle, or just the end of a branch) carries the **payoffs** – here, a cost and a QALY value – accumulated by the time that path is reached. Trees are the right tool exactly when §17.2′s criteria point to a short, non-recurring decision problem with no need to track disease progression over multiple cycles – which is why this chapter switches to a different slice of the TB pathway than Chapter 17′s regimen choice.

Worked example – diagnostic strategy choice. A district TB programme is choosing between two algorithms for patients presenting with TB symptoms: a cheaper **sputum-smear-only** algorithm and a **Xpert MTB/RIF** molecular-test algorithm. Among symptomatic presenters, TB

prevalence is $p = 0.30$. Smear microscopy has sensitivity 0.60 and specificity 0.98; Xpert has sensitivity 0.90 and specificity 0.96. Each strategy therefore branches first on true disease status (TB / no TB), then on test result, producing four terminal outcomes: true positive (TP, correctly started on treatment), false negative (FN, missed – presents later, sicker), false positive (FP, treated unnecessarily), and true negative (TN, correctly reassured).

Branch probabilities (disease status $\times$ conditional test-result probability):

- Smear: $\text{TP} = 0.30 \times 0.60 = 0.18$; $\text{FN} = 0.30 \times 0.40 = 0.12$; $\text{FP} = 0.70 \times 0.02 = 0.014$; $\text{TN} = 0.70 \times 0.98 = 0.686$ (sums to 1.000).
- Xpert: $\text{TP} = 0.30 \times 0.90 = 0.27$; $\text{FN} = 0.30 \times 0.10 = 0.03$; $\text{FP} = 0.70 \times 0.04 = 0.028$; $\text{TN} = 0.70 \times 0.96 = 0.672$ (sums to 1.000).

Terminal payoffs are the test's own cost plus whatever downstream cost and QALY the outcome triggers over the episode: TP costs the test plus \$100 treatment, QALY 0.95; FN costs the test plus \$300 (later, more severe re-presentation), QALY 0.70; FP costs the test plus \$100 unnecessary treatment, QALY 0.90; TN costs only the test, QALY 0.97. Smear costs \$5 per patient tested; Xpert costs \$25.

The **rollback** (or "folding back") algorithm works from the terminal nodes backward: at each chance node, replace the branches below it with their probability-weighted average payoff, repeating until only the decision node remains with one EV per strategy. For a tree this shallow, rollback collapses to a single weighted sum per strategy – exactly the $\text{EV} = \sum_i p_i \times \text{payoff}_i$ logic from §4.4, just applied twice (once for cost, once for QALYs).

Smear – expected cost:

$$0.18 \times 105 + 0.12 \times 305 + 0.014 \times 105 + 0.686 \times 5$$

$$= 18.90 + 36.60 + 1.47 + 3.43 = \$60.40$$

Smear – expected QALYs:

$$0.18 \times 0.95 + 0.12 \times 0.70 + 0.014 \times 0.90 + 0.686 \times 0.97$$

$$= 0.1710 + 0.0840 + 0.0126 + 0.6654 = 0.93302$$

Xpert – expected cost:

$$0.27 \times 125 + 0.03 \times 325 + 0.028 \times 125 + 0.672 \times 25$$

$$= 33.75 + 9.75 + 3.50 + 16.80 = \$63.80$$

Xpert – expected QALYs:

$$0.27 \times 0.95 + 0.03 \times 0.70 + 0.028 \times 0.90 + 0.672 \times 0.97$$

$$= 0.2565 + 0.0210 + 0.0252 + 0.6518 = 0.95454$$

Xpert costs more per patient tested (\$63.80 vs \$60.40) but also catches more true TB cases, raising expected QALYs (0.95454 vs 0.93302). Following §4.4's ICER formula:

$$\text{ICER} = \frac{63.80 - 60.40}{0.95454 - 0.93302} = \frac{3.40}{0.02152} \approx \$158 \text{ per QALY gained}$$

At roughly $158 per QALY gained, Xpert is cheap insurance against missed diagnoses by almost any plausible willingness-to-pay threshold – the arithmetic, not intuition, is what makes that call defensible.

> **Common pitfall.** It is tempting to price a diagnostic test only by sensitivity and specificity and forget that **prevalence** sets how often you even reach the TP/FN branches versus the FP/TN branches. A highly sensitive test bought at high prevalence saves far more QALYs than the same test deployed where TB prevalence among presenters is low – rerun this tree with $p = 0.10$ and watch the ICER move before trusting any single-country number from a published costing study.

18.2 Building decision trees from scratch in R

You don't need a modelling package to run the rollback algorithm above – it's eight numbers multiplied and added, which is exactly what R's vectorized arithmetic is for. Representing the tree as a tidy data structure, one row per terminal path, makes the "fold back" step a single `sum(prob * payoff)`, and it keeps every branch probability and payoff visible for audit rather than buried in a diagram.

The key discipline is to store the branch probabilities **unmultiplied** (disease status and conditional test-result probability as separate columns) and compute the path probability explicitly as their product – that product is the rollback algorithm, made concrete as one line of R:

```
library(dplyr)

tb_tree <- tibble::tribble(
  ~strategy, ~outcome, ~p_disease, ~p_test, ~cost_test, ~cost_downstream, ~qaly,
  "Smear",   "TP",      0.30,       0.60,    5,          100,             0.95,
  "Smear",   "FN",      0.30,       0.40,    5,          300,             0.70,
  "Smear",   "FP",      0.70,       0.02,    5,          100,             0.90,
  "Smear",   "TN",      0.70,       0.98,    5,          0,               0.97,
  "Xpert",   "TP",      0.30,       0.90,    25,         100,             0.95,
  "Xpert",   "FN",      0.30,       0.10,    25,         300,             0.70,
  "Xpert",   "FP",      0.70,       0.04,    25,         100,             0.90,
  "Xpert",   "TN",      0.70,       0.96,    25,         0,               0.97
)

tb_tree <- tb_tree |>
  mutate(
    path_prob = p_disease * p_test,     # the branch-probability product
    cost      = cost_test + cost_downstream
  )

# sanity check: branch probabilities must sum to 1 within each strategy
tb_tree |>
  group_by(strategy) |>
  summarise(total_prob = sum(path_prob))
#> Smear: 1.000   Xpert: 1.000
```

With `path_prob` computed, the rollback itself is one `summarise()` call – a direct, line-by-line translation of the hand arithmetic in §18.1:

```
ev <- tb_tree |>
  group_by(strategy) |>
  summarise(
    ev_cost = sum(path_prob * cost),
    ev_qaly = sum(path_prob * qaly)
  )
ev
#> strategy ev_cost ev_qaly
#> Smear      60.40 0.93302
#> Xpert      63.80 0.95454

icer <- diff(ev$ev_cost) / diff(ev$ev_qaly)
icer
#> [1] 157.99...
```

`ev_cost` and `ev_qaly` reproduce the \$60.40 / 0.93302 (Smear) and \$63.80 / 0.95454 (Xpert) computed by hand, and `icer` lands at the same ≈ \$158 per QALY gained – confirming that `sum(path_prob * payoff)` **is** the rollback algorithm, not an approximation of it. `path_prob` is just the product of branch probabilities along one path through the tree; grouping by strategy and summing is folding every chance node back into its parent decision node in a single pass.

How to report this.

Strategy	Expected cost	Expected QALYs	Incr. cost	ICER
Smear-only	\$60.40	0.93302	–	–
Xpert MTB/RIF	\$63.80	0.95454	\$3.40	\$158/QALY
Xpert MTB/RIF dominates on effectiveness at a low incremental cost per QALY gained relative to smear-only testing.				

Table 48: Expected cost, expected QALYs, and incremental cost-effectiveness ratio for the two TB diagnostic strategies, as computed in §18.1–§18.2's worked example.

Agency perspective. This `tribble` is small enough to type by hand for a teaching example, but in practice you'd pull `p_disease` (prevalence), `p_test` (sensitivity/specificity by algorithm), and `cost_downstream` straight from your programme's own diagnostic register and treatment-outcome data – the same register you already query for routine TB surveillance reporting. Wiring a decision tree to live data is a `dplyr::summarise()` away once the register is shaped like `tb_tree`.

This data-frame-and-`sum()` approach scales poorly once trees grow beyond a handful of branches per strategy, or once you want probabilistic sensitivity analysis sampling every probability and payoff from a distribution – bookkeeping that a dedicated package can do more safely. §18.3 rebuilds this exact tree in `rdecision`, so you can see the package automate precisely the two steps – path-probability products and grouped summation – you just wrote by hand.

18.3 Using `rdecision`

Everything built in §18.2 – nodes, branch probabilities, cost and QALY bookkeeping – was done by hand, in base R vectors and a handful of helper functions. That is the right way to **learn** what a decision tree computes, but it does not scale well once you want probabilistic sensitivity analysis (PSA, Chapter 25), graphical audit trails for a submission dossier, or a tree with more than two or three branch points. The CRAN package `rdecision` was written specifically for HTA-style decision-tree and Markov modelling in R, and it is worth adopting once your tree stops being a one-off script.

`rdecision` models a tree as a graph of R6-class objects rather than a data frame. The classes you will meet first are `DecisionNode` (a root-level choice point – our "which algorithm?" node), `ChanceNode` (a point where nature decides – "is the patient truly TB-positive?", "did the test read positive?"), and `LeafNode` (a terminal outcome, carrying an incremental `utility` and an `interval` over which it applies – `rdecision`'s terminology for the per-path QALY contribution). Edges connecting them are `Action` objects (for branches out of a `DecisionNode`; constructor `Action$new(source_node, target_node, label, cost = 0, benefit = 0)` – note `label` is required and `cost` is a named argument, not positional) and `Reaction` objects (for branches out of a `ChanceNode`; constructor `Reaction$new(source_node, target_node, p = 0, cost = 0, benefit = 0, label = "")`, where `p` is the conditional branch probability). The whole structure is assembled into a `DecisionTree` object from its vertex and edge lists, which exposes an `evaluate(setvars = "expected", by = "strategy")` method that walks the tree and returns a data frame of expected cost, benefit, and QALY per strategy – doing the EV roll-back you wrote by hand in §18.2 internally.

> **Difficult terminology: R6 class / R6-class objects.** R6 is one of several systems R offers for **object-oriented programming** – a style of coding where, instead of keeping your data in a plain data frame and writing separate functions that operate on it, you bundle data and the functions (called **methods**) that act on that data together into a single thing called an **object**. Each R6 object is created from a **class** – a blueprint describing what fields (data) and methods (behaviour) every object of that type will have. Concretely in this chapter: a `ChanceNode` object doesn't just sit there holding a probability value – it is a self-contained thing that knows its own branches, can be asked to validate that its outgoing probabilities sum to 1, and participates in the tree's `evaluate()` walk without you writing a bespoke loop for it. The syntax tell-tale is the dollar sign: `Action$new(...)` **creates** a new object from the `Action` class (like calling a constructor), and once you have an object such as `ictstm` (seen later in Chapter 21), you call its methods the same way – `ictstm$sim_disease()` – reading as "ask this object to run its own disease-simulation method," as opposed to the more familiar R style of `sim_disease(ictstm)`, where the object is just a passive argument to a free-standing function. You do not need to write your own R6 classes to use `rdecision` – you only need to recognise `Class$new(...)` as "build me an object of this type" and `object$method(...)` as "ask this object to do something," which is enough to read and adapt every `rdecision` example in this chapter.

These signatures (confirmed against the live CRAN reference manual for `rdecision`, not asserted from memory) resolve the hedge this chapter would otherwise need: argument order is `source_node, target_node` first, then the named `p=`/`cost=`/`benefit=`/`label=` arguments, and `rdecision` follows one `rdecision`-specific convention worth flagging explicitly for anyone used to the §18.1–18.2 convention of listing TP/FN/FP/TN as four separate branches per algorithm: it

is idiomatic in `rdecision` vignettes to leave **one** reaction's probability as `p = NA_real_`, which the package then fills in at evaluation time as one minus the sum of the other reactions from that same chance node – a built-in version of the "branch probabilities must sum to 1" check from §18.1, enforced by the package rather than by a manual `sum()` sanity check.

Sketched against the TB running example – smear-only vs Xpert, each branching first on true disease status then on test result – the shape below reuses §18.1–18.2's exact numbers ($p = 0.30$ prevalence; smear sensitivity 0.60/specificity 0.98; Xpert sensitivity 0.90/specificity 0.96; costs and QALYs per outcome as tabulated in §18.1):

```
library(rdecision)

# Branch probabilities and payoffs, reusing §18.1's worked numbers exactly
p_tb <- 0.30
sens_smear <- 0.60; spec_smear <- 0.98
sens_xpert <- 0.90; spec_xpert <- 0.96
cost_smear_test <- 5; cost_xpert_test <- 25
cost_tp <- 100; cost_fn <- 300; cost_fp <- 100; cost_tn <- 0
qaly_tp <- 0.95; qaly_fn <- 0.70; qaly_fp <- 0.90; qaly_tn <- 0.97

# Decision node: choice of diagnostic algorithm
d1 <- DecisionNode$new("algorithm")

# One chance node per algorithm for disease status, one per status for
# the test-result branch (true-positive/false-negative under TB;
# false-positive/true-negative under no-TB)
c_smear_disease <- ChanceNode$new("smear: disease status")
c_smear_tb_test  <- ChanceNode$new("smear: test | TB")
c_smear_notb_test <- ChanceNode$new("smear: test | no TB")
c_xpert_disease <- ChanceNode$new("xpert: disease status")
c_xpert_tb_test  <- ChanceNode$new("xpert: test | TB")
c_xpert_notb_test <- ChanceNode$new("xpert: test | no TB")

# Leaves carry the incremental utility (QALY-equivalent) for the outcome
t_smear_tp <- LeafNode$new("Smear:TP"); t_smear_fn <- LeafNode$new("Smear:FN")
t_smear_fp <- LeafNode$new("Smear:FP"); t_smear_tn <- LeafNode$new("Smear:TN")
t_xpert_tp <- LeafNode$new("Xpert:TP"); t_xpert_fn <- LeafNode$new("Xpert:FN")
t_xpert_fp <- LeafNode$new("Xpert:FP"); t_xpert_tn <- LeafNode$new("Xpert:TN")

# Actions carry the per-test cost; Reactions carry branch probability + cost
a_smear <- Action$new(d1, c_smear_disease, cost = cost_smear_test, label =
"Smear")
a_xpert <- Action$new(d1, c_xpert_disease, cost = cost_xpert_test, label =
"Xpert")

r_smear_tb  <- Reaction$new(c_smear_disease, c_smear_tb_test,  p = p_tb, cost = 0,
label = "TB")
r_smear_notb <- Reaction$new(c_smear_disease, c_smear_notb_test, p = NA_real_,
cost = 0, label = "No TB")
r_smear_tp <- Reaction$new(c_smear_tb_test, t_smear_tp, p = sens_smear, cost =
cost_tp, label = "+ve")
r_smear_fn <- Reaction$new(c_smear_tb_test, t_smear_fn, p = NA_real_, cost =
cost_fn, label = "-ve")
r_smear_fp <- Reaction$new(c_smear_notb_test, t_smear_fp, p = 1 - spec_smear, cost
```

```
= cost_fp, label = "+ve")
r_smear_tn <- Reaction$new(c_smear_notb_test, t_smear_tn, p = NA_real_, cost =
cost_tn, label = "-ve")
# ... equivalent reactions for the Xpert branch, using sens_xpert/spec_xpert ...

tree <- DecisionTree$new(
  V = list(d1, c_smear_disease, c_smear_tb_test, c_smear_notb_test,
           t_smear_tp, t_smear_fn, t_smear_fp, t_smear_tn,
           c_xpert_disease, c_xpert_tb_test, c_xpert_notb_test,
           t_xpert_tp, t_xpert_fn, t_xpert_fp, t_xpert_tn),
  E = list(a_smear, a_xpert,
           r_smear_tb, r_smear_notb, r_smear_tp, r_smear_fn, r_smear_fp,
r_smear_tn)
           # ... plus the equivalent Xpert reactions ...
)

results <- tree$evaluate(setvars = "expected", by = "strategy")
results
```

What this buys over the §18.2 from-scratch version is not the arithmetic – a correctly written vector calculation gives you the identical expected cost and QALY – it is three things that matter once the model has to survive contact with a reviewer or a resampling loop. First, `evaluate()` can be called repeatedly over resampled probability and cost inputs with the tree structure untouched, which is exactly the vectorised-resampling pattern Chapter 25's PSA needs: you draw a new parameter set per iteration and re-roll the same tree, rather than hand-writing a Monte Carlo wrapper around your own cost/QALY functions. Second, `rdecision` trees can be rendered graphically (the package includes plotting support for the graph structure), giving you a diagram you can drop straight into a HTA submission's technical appendix instead of a prose description of node logic. Third, because cost and effect are tracked as native node attributes, ICER and incremental-NMB comparisons between the Smear and Xpert strategies (using the formulas from §4.4) come out of `evaluate()`'s returned data frame directly, rather than requiring you to re-derive them from raw EV totals each time.

> **HEOR in practice.** Keep the §18.2 hand-rolled version in your repository even after you migrate to `rdecision`. It is your independent cross-check: run both on the same inputs and confirm the expected costs and QALYs agree to the last rupee and the fourth decimal of a QALY. A graphical, package-driven tree that silently diverges from a transparent manual calculation is a red flag worth catching in `tests/` (§17.6), not in review.

18.4 Short-term models for screening, diagnostics and acute care

The TB smear-vs-Xpert tree is one instance of a much broader class of problem that decision trees are the right tool for. Chapter 17 §17.2 gave you the model-type-choice framework for deciding, up front, whether a problem needs a tree, a Markov model, or something else; the criterion to reuse by name here is the **one-off/short-horizon** test. A decision tree suits a question when the clinically and economically relevant consequences of each choice resolve within a single episode of care – a test result, a treatment decision, a short course of therapy and its immediate outcome – such that there is no need to track a cohort moving between recurring health states over months or years. The moment your question involves patients cycling between "well," "relapsed," "on second-

line treatment," and "dead" over a multi-year horizon, you have left tree territory and moved into Markov territory (Chapter 19).

Screening and diagnostic-pathway evaluation is the textbook case, because the question is almost always "given this test's sensitivity, specificity and cost, and the downstream cost/outcome of each possible result, which algorithm is cheapest per unit of health gained **right now**" – precisely the TB example running through this chapter. The same structure recurs in:

- **Antenatal screening algorithms** – e.g. combined first-trimester screening for Down syndrome (nuchal translucency plus serum markers) versus a non-invasive prenatal test, where branches represent screen-positive/negative and the diagnostic-confirmation pathway (amniocentesis) that follows a positive screen, with the short horizon being the pregnancy itself rather than a lifetime of states.
- **Emergency-department triage pathway cost-effectiveness** – e.g. comparing a rapid point-of-care troponin protocol against a standard serial-testing protocol for suspected acute coronary syndrome, where the tree's branches are "ruled in," "ruled out," and "observed," each with an immediate cost and a short-term outcome (discharge, admission, missed-diagnosis adverse event) rather than a chronic disease trajectory.

Neither example needs to be built out here in full; the point is structural. Whenever the decision context is "choose a pathway, observe branch probabilities resolve, tally the immediate consequence," reach for a tree. Whenever a chosen branch instead seeds a cohort that then lives on for years under recurring risks, the tree becomes the **front end** of a hybrid model – which is exactly §18.5's subject.

> **Agency perspective.** NICE's Diagnostics Assessment Programme methods guide (the diagnostics-specific companion to NICE's **Guide to the methods of technology appraisal**) explicitly expects short-horizon decision-tree structures for test-and-treat pathways, and asks submitters to report sensitivity/specificity, the full branch-probability tree, and per-branch cost and outcome consequences transparently enough for the Diagnostics Advisory Committee to re-derive the EV roll-back independently. A tree built the way §18.2–18.3 built it – explicit nodes, explicit probabilities, an auditable calculation – is written to survive exactly that kind of scrutiny; a tree that only exists as an opaque spreadsheet formula typically is not.

18.5 Combining a decision tree with a long-term Markov model

Real HTA submissions for conditions with a chronic phase rarely use a decision tree **or** a Markov model in isolation – they chain the two together, and the TB diagnostic-strategy question is a clean illustration of why. The tree in §18.1–18.3 answers "which diagnostic algorithm, and what happens to the patient in the near term as a result of a true/false positive/negative result." But a patient who is correctly diagnosed and started on treatment, one who is a false negative and remains undiagnosed and infectious, and one who is a false positive and started on treatment unnecessarily, do not have the same future. Their subsequent trajectories – treatment completion, relapse, acquired drug resistance, transmission to contacts, death – unfold over years, which is squarely the territory of the cohort Markov model Chapter 19 builds for the TB regimen-choice question introduced back in Chapter 17.

The handoff between the two model types is a well-established HTA pattern, and it is worth being precise about what actually crosses the interface: not patients, not probabilities-as-abstractions, but a **vector of state-occupancy probabilities**. Each leaf node of the decision tree corresponds to (or maps onto) one of the Markov model's starting health states, and the probability mass arriving at that leaf – the product of branch probabilities along the path from root to leaf – becomes the initial occupancy of that state at Markov cycle zero (time 0).

Concretely, for the TB tree: a patient who is a true positive under whichever algorithm was chosen enters the Markov model's "On Treatment" state at time 0; a false negative enters "Undiagnosed Active TB" (continuing to accrue transmission risk and mortality hazard before any future re-presentation); a true negative enters a "Well/No TB" state; and a false positive enters "On Treatment" too – despite not having TB – carrying the model's treatment-related costs and disutilities without the corresponding disease-free benefit a true positive would eventually realise. The decision tree's four terminal leaves per strategy collapse into (at least) three or four Markov starting states, and the tree's job is simply to produce the correctly weighted probability of starting in each one, per strategy.

This is why it matters that §18.1–18.3′s leaf probabilities are kept as an explicit, inspectable vector rather than folded away inside an aggregate expected-cost number: Chapter 19 will read that vector directly as its `t=0` cohort distribution, in the same `inputs/` ⟶ `engine/` ⟶ `results/` ⟶ `reporting/` project layout established in Chapter 17 §17.6 – the tree's `results/` output becomes (part of) the Markov engine's `inputs/`. No Markov transition matrix, cycle length, or half-cycle correction is introduced here; building that model, and running the resulting hybrid tree-plus-Markov analysis end to end for the TB regimen-choice question, is Chapter 19′s task in full.

Chapter 19 picks up exactly here: the same TB programme, now asking what happens over years to patients who entered treatment (correctly or not) via this chapter's diagnostic tree, modelled as a Markov cohort moving between On Treatment, Cured, Treatment Failure, and Death – the structure Chapter 17 §17.3 already diagrammed with `DiagrammeR::grViz()`.

Chapter 19. Markov Cohort Models

Chapter 18 built the simplest decision-analytic engine – the tree – and closed with its §18.5 handoff pattern: a tree's terminal-leaf path probabilities seed the time-0 cohort distribution of a longer-term model. This chapter builds that longer-term model. Chapter 17′s §17.2 model-type framework resolved the book's running TB example – a choice between a 4-month rifapentine-moxifloxacin regimen (Study 31/A5349) and the 6-month standard daily regimen (2HRZE/4HR) – to a **Markov cohort model**, diagrammed in §17.3′s `DiagrammeR::grViz()` example with four states: On Treatment, Cured, Treatment Failure, Death. This chapter turns that diagram into a runnable, cost-and-QALY-producing model, first by hand in base R, then via the dedicated CRAN packages built for exactly this job.

Learning objectives

- Define a Markov state, transition probability and cycle, and compute a cohort trace by the roll-forward matrix-multiplication method, verifying that every transition-matrix row sums to 1 (19.1).
- Distinguish time-homogeneous from time-dependent transition matrices and explain why many real clinical transitions are not homogeneous (19.2).
- Implement a complete cohort Markov trace in base R using nothing but a matrix and a loop, and read off state occupancy at any cycle (19.3).
- Use `heemod`'s `define_transition()`/`define_state()`/`define_strategy()`/`run_model()` pipeline, `hesim`'s data-scaffold architecture, and `dampack::calculate_icers()` for post-processing, with verified real function signatures (19.4).
- Apply discounting and half-cycle correction to a cycle-by-cycle cost/QALY trace, and connect both to Chapter 4′s discounting treatment and to `heemod`'s `method = "life-table"` option (19.5).
- Explain what a tunnel state is, why it is needed when a transition probability depends on time-since-state-entry, and how `heemod`'s `state_time` variable automates it (19.6).
- Combine age-dependent background mortality with disease-specific mortality without double-counting, using `heemod`'s `model_time` variable (19.7).
- Explain why whole transition-matrix rows need Dirichlet-distributed PSA draws rather than independent Betas per cell, and when a Cholesky decomposition is needed instead (19.8).
- Distinguish calibration from validation and identify where each belongs in the modelling workflow, forward-pointing to Chapter 27 (19.9).
- Work a complete, independently-verified chronic-disease cost-effectiveness analysis end-to-end, from transition matrix to ICER (19.10).

19.1 States, transitions and cycles

Chapter 17 resolved the regimen-choice question – 4-month rifapentine-moxifloxacin (Study 31/A5349) versus the 6-month standard daily regimen (2HRZE/4HR) – into a **Markov cohort model** built on four **states**: On Treatment, Cured, Treatment Failure, and Death. This section turns that diagram (§17.3′s `grViz()` structure) into numbers.

A **Markov state** is a mutually exclusive, collectively exhaustive health/treatment status a patient can occupy at any point in the model. A **transition probability** is the chance of moving from one state to another over one **cycle** – a fixed unit of model time (here, six months, matching the typical duration of intensive-phase decision-making in TB care). The defining simplification is the **Markov assumption** (memorylessness): the probability of each future transition depends only on the **current** state, never on how a patient got there or how long they have been in it. A patient who has failed treatment once and been re-treated is, under this assumption, statistically indistinguishable from one on their first course – clearly a simplification for TB, where prior-failure patients often carry resistant organisms and worse prognosis. Chapter 21's microsimulation relaxes exactly this restriction by tracking individual patient history; we flag the limitation here and defer the fix.

Using the four states and the back-edge from Chapter 17 (Fail -> OnTx, re-treatment; Death reachable from every state), here are illustrative per-six-month-cycle transition probabilities. These are invented for pedagogical purposes and are **not** a clinical claim – a real model would source them from the Study 31 and standard-of-care trial arms, or from WHO TB outcome cohorts.

From **On Treatment**: cured $p = 0.75$, fail $p = 0.08$, die $p = 0.02$, remain on treatment $p = 1 - 0.75 - 0.08 - 0.02 = 0.15$.

From **Treatment Failure**: re-enter treatment $p = 0.60$, die $p = 0.15$, remain in failure $p = 1 - 0.60 - 0.15 = 0.25$.

From **Cured**: Cured is not absorbing – cured patients still face ordinary background mortality, here taken as $p = 0.01$ per cycle, so remain cured $p = 0.99$.

From **Death**: absorbing, $p(\text{Death} \to \text{Death}) = 1$, all other entries zero.

> **Common pitfall.** A transition matrix row that does not sum exactly to 1 is one of the most common and most dangerous Markov-model bugs: probability mass silently leaks out of (or is fabricated into) the cohort every cycle, and the error compounds invisibly over dozens of cycles before anyone notices the totals look odd. The usual cause is a copy-paste edit to one cell without re-deriving the complementary "stay" probability. **Always** verify row sums as an explicit step, not an assumption – §19.3's code does this with `rowSums()` before running a single cycle. A second common variant: forgetting that **Death** must be absolutely absorbing. If a modeller later edits the Death row to add, say, a spurious resurrection probability (sometimes from sloppy sensitivity-analysis scripting that perturbs every cell including diagonal absorbing ones), the cohort's death count can decrease over time – a biologically impossible and easily caught red flag if you plot cumulative deaths and see it dip.

Worked check, On Treatment row: $0.15 + 0.75 + 0.08 + 0.02 = 1.00$ checkmark.

> **Agency perspective.** Siebert et al. (2012), "State-Transition Modeling: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force-3," already cited in Chapter 17, is the standard methodological reference for everything in this chapter: state definition, cycle length choice, half-cycle correction, and when a Markov model is adequate versus when individual-level simulation (their Task Force companion papers) is required. Re-consult it directly when justifying structural choices in a real submission.

19.2 Time-homogeneous and time-dependent transition matrices

The matrix in §19.1 is **time-homogeneous**: the same four probabilities apply in cycle 1, cycle 10, and cycle 20, regardless of how long the cohort has been running or how old a given patient is. This is the simplest and most common starting assumption, and it is exactly what the plain matrix-multiplication approach in §19.3 encodes.

Many real TB and chronic-disease transitions are **not** homogeneous. Treatment-failure risk in TB is classically **front-loaded**: most failures and early deaths occur in the first few months of a treatment course, with the hazard tapering as patients who tolerate therapy and respond clear their bacterial burden. A single fixed $p(\text{OnTx} \to \text{Fail})$ averaged over the whole treatment course will overstate late-cycle risk and understate early-cycle risk – potentially biasing a cost-effectiveness comparison between a short four-month regimen and a long six-month one, since the **timing** of failure within the course matters for how much cost and QALY loss is already "banked" by the time it happens.

A **time-dependent** (non-homogeneous) matrix instead makes one or more transition probabilities a function of the cycle index t (time since model start, or more precisely time since entering the current state) – for example a declining hazard for $p_{\text{OnTx} \to \text{Fail}}(t)$, or a background-mortality cell that rises with cohort age as t increases. Implementing this in base R just means indexing into a **list** of matrices (one per cycle, or one per age band) rather than reusing a single matrix in the loop – mechanically trivial, but it requires the probabilities to be **derived** from a hazard function or an age-specific life table rather than assumed constant. A tunnel-state device (covered in §19.6) is often used precisely to let risk vary with **time since entering** a state even inside an otherwise time-homogeneous framework, and age-dependent background mortality is picked up again in §19.7.

19.3 Building a Markov model in base R (matrix-based approach)

The cleanest way to understand a cohort Markov model is to write it with nothing but a matrix and a loop. Base R needs no special package for this; `expm::"%^%"` (matrix power) is a convenient but entirely optional shortcut for jumping straight to cycle n without looping.

```r
# Four states, in a fixed order used throughout the chapter
state_names <- c("OnTx", "Cured", "Fail", "Death")

# Transition matrix: rows = "from", columns = "to"
P <- matrix(
  c(0.15, 0.75, 0.08, 0.02,   # from OnTx
    0.00, 0.99, 0.00, 0.01,   # from Cured
    0.60, 0.00, 0.25, 0.15,   # from Fail
    0.00, 0.00, 0.00, 1.00),  # from Death (absorbing)
  nrow = 4, byrow = TRUE,
  dimnames = list(state_names, state_names)
)

# Defensive check: every row must sum to exactly 1
stopifnot(all(abs(rowSums(P) - 1) < 1e-9))

# Initial state-occupancy vector: whole cohort starts On Treatment
init <- c(OnTx = 1, Cured = 0, Fail = 0, Death = 0)
```

```r
n_cycles <- 20  # 20 six-month cycles = 10 years

# Trace matrix: rows = cycles 0..n_cycles, columns = states
trace <- matrix(NA_real_, nrow = n_cycles + 1, ncol = 4,
                dimnames = list(0:n_cycles, state_names))
trace[1, ] <- init

for (t in seq_len(n_cycles)) {
  trace[t + 1, ] <- trace[t, ] %*% P
}

round(trace[c(1, 6, 11, 21), ], 4)  # cycles 0, 5, 10, 20
```

Running this loop (independently re-verified during assembly) gives the occupancy trace below. The cohort initially pours out of On Treatment, most of it settling into Cured, a small residual fraction cycling through Fail (which keeps feeding a trickle back into OnTx and onward into Death), and Death's share climbing monotonically, as it must for an absorbing state.

How to report this.

Cycle	On Treatment	Cured	Treatment Failure	Death
0	1.0000	0.0000	0.0000	0.0000
5	0.0054	0.9130	0.0025	0.0791
10	0.0001	0.8750	0.0000	0.1249
20	0.0000	0.7914	0.0000	0.2086

Table 49: State occupancy from the base-R cohort trace, cycles 0, 5, 10, 20 (proportion of starting cohort)

By cycle 20 (10 years), almost 21% of the original cohort has died (from TB or background causes) and the remainder has essentially settled in Cured, since Fail and On Treatment occupancy decays toward zero – the expected long-run behaviour of a model with one absorbing state and no permanent non-absorbing "parking" state. This trace is the raw ingredient §19.5 will multiply by state costs and utilities, cycle by cycle, to get discounted costs and QALYs.

19.4 Using heemod, hesim and dampack

Writing the loop by hand in §19.3 is instructive, but production HEOR models are rarely hand-rolled this way – bookkeeping for costs, utilities, discounting, half-cycle correction, strategy comparison, and probabilistic sensitivity analysis (PSA) quickly gets unwieldy. Three CRAN packages cover this ground at increasing levels of sophistication: `heemod`, `hesim`, and `dampack`.

`heemod` implements cohort Markov models with the same matrix-and-states logic as §19.3, but wrapped in a declarative syntax. `heemod::define_transition()` builds the matrix from named cell expressions, row by row, and crucially supports the keyword `C` to mean "fill in whatever makes this row sum to 1" – exactly the complementary "remainder" probability computed by hand in §19.1. `heemod::define_state()` attaches a per-cycle `cost=` and `utility=` to one state.

`heemod::define_strategy()` combines one transition object with one state object per state (named arguments matching the state names). `heemod::run_model()` takes one or more strategies plus `parameters = define_parameters()`, an `init=` occupancy vector, the number of `cycles=`, a `method=` argument (`"life-table"`, `"beginning"`, or `"end"`) controlling how half-cycle correction is applied – revisited in §19.5 – and `cost=`/`effect=` expressions naming which state variables to accumulate.

A simplified 2-state rebuild (On Treatment / Resolved, collapsing Cured+Death for brevity) illustrates the pattern:

```
library(heemod)

tm <- define_transition(
  state_names = c("OnTx", "Resolved"),
  C,    0.85,   # from OnTx: fill-in remainder, then 0.85 to Resolved
  0,    C       # from Resolved: 0 back to OnTx, remainder (1) stays
)

s_ontx <- define_state(cost = 500, utility = 0.65)
s_resolved <- define_state(cost = 50, utility = 0.90)

strat <- define_strategy(
  transition = tm,
  OnTx = s_ontx,
  Resolved = s_resolved
)

res <- run_model(
  strat,
  parameters = define_parameters(),
  init = c(1, 0),
  cycles = 20,
  method = "life-table",
  cost = cost,
  effect = utility
)
```

Here `C` in the first row fills in $1 - 0.85 = 0.15$ (stay On Treatment); `C` in the second row fills in $1 - 0 = 1$ (stay Resolved), making it absorbing – the same arithmetic discipline as §19.1, now delegated to the package.

`hesim` takes a different, more modular architecture built for larger, heterogeneous-cohort or patient-level problems. `hesim::hesim_data(strategies=, patients=, states=)` builds a data scaffold describing every strategy/patient/state combination up front, rather than one fixed matrix. `define_tparams()` builds transformed-parameter expressions that reference a `tpmatrix()` (a tidy, long-format transition-probability matrix, often with one row per PSA draw), and `create_CohortDtstm()` instantiates a cohort discrete-time state-transition model from those pieces. Sketch only – hesim's real power (patient heterogeneity and built-in PSA) gets the fuller treatment in Chapter 25:

```
# Illustrative sketch only -- architecture, not a complete runnable model
hesim_dat <- hesim_data(
  strategies = data.frame(strategy_id = 1:2),
```

```
  patients   = data.frame(patient_id = 1:100),
  states     = data.frame(state_id = 1:3, state_name = c("OnTx", "Cured", "Fail"))
)
tp_params <- define_tparams({
  list(tpmatrix = tpmatrix(...))   # references draws/covariates
})
cohort_model <- create_CohortDtstm(define_model(tparams_def = tp_params),
hesim_dat)
```

Choose `hesim` over `heemod` when you need patient subgroups, covariate-varying transition probabilities, or PSA machinery built into the object model from the start, rather than bolted on afterward.

Once any of these produces per-strategy cost and effect totals, `dampack`'s `calculate_icers(cost=, effect=, strategies=)` is the standard post-processing step: it takes vectors of total discounted cost and effect (one value per strategy) and returns a dominance-aware ICER table – flagging strategies that are **dominated** (more costly, less effective than an alternative) or **extendedly dominated**, and computing incremental cost-effectiveness ratios only along the efficient frontier. Its companion PSA/VOI tooling (`run_psa()`, `owsa()` one-way sensitivity analysis, `ceac()` cost-effectiveness acceptability curves) is deferred to Chapters 25-26, where probabilistic analysis of exactly this TB model is built out properly.

19.5 Applying costs and QALYs, discounting and half-cycle correction

Occupancy traces are not yet a cost-effectiveness result: each cycle's state occupancy must be multiplied by that state's per-cycle cost and utility, summed across states and cycles, and **discounted** – future costs and health effects are worth less than present ones, as Chapter 4 §4.6 already established for simple decision-tree-style calculations. The same discount factor applies here, just compounded cycle by cycle rather than year by year: $\mathrm{DF}(t) = (1+r)^{-t}$ where r is the **per-cycle** discount rate and t is the cycle index. NICE's reference case specifies $r = 3.5\%$ per annum for **both** costs and health effects (NICE, "Guide to the methods of technology appraisal"); converting to a six-month cycle rate uses $(1.035)^{0.5} - 1 \approx 0.01735$, i.e. about 1.735% per half-year cycle, consistent with Chapter 4's compounding logic.

Half-cycle correction addresses a subtler bias: the raw trace assumes the whole cohort occupies each state for the **entire** cycle, as if transitions happen only at cycle boundaries, when in reality patients move between states at varying points within a cycle. The standard fix – the **trapezoidal** or **life-table** method – approximates each cycle's contribution as the **average** of the occupancy at the start and at the end of the cycle, rather than using the end-of-cycle (or start-of-cycle) value alone for the whole interval. This is precisely what `heemod::run_model()`'s `method = "life-table"` default (introduced in §19.4) computes under the hood: it is the real-software implementation of the same trapezoidal averaging, as opposed to `method = "beginning"` or `"end"`, which deliberately omit the correction and bias results in opposite directions.

Worked numeric example: suppose a cohort accrues QALYs of 0.40, 0.42, 0.43, 0.44, 0.45 in cycles 1 through 5 (ignore half-cycle correction here for clarity, to isolate the discounting arithmetic), at NICE's per-cycle rate $r \approx 0.01735$. Discounting cycle t's QALYs by $(1+r)^{-t}$:

How to report this.

Cycle t	Undiscounted QALYs	Discount factor $(1+r)^{-t}$	Discounted QALYs
1	0.40	0.9829	0.3932
2	0.42	0.9662	0.4058
3	0.43	0.9497	0.4084
4	0.44	0.9335	0.4107
5	0.45	0.9176	0.4129
Total: undiscounted 2.1400, discounted 2.0310			

Table 50: Discounting a 5-cycle QALY stream at NICE's 3.5% annual rate (1.735% per 6-month cycle)

Total undiscounted QALYs across the five cycles: $0.40 + 0.42 + 0.43 + 0.44 + 0.45 = 2.14$. Total discounted QALYs: 2.031 (summing the last column) – a reduction of about 5.1%, consistent with a modest discount rate applied over a short 2.5-year horizon; the gap widens substantially over the full 10-year, 20-cycle horizon used in §19.3.

The next section, §19.6, extends this same trace-and-discount machinery to **tunnel states** – the device that lets a formally time-homogeneous Markov model mimic the time-dependent, front-loaded risks flagged in §19.2.

19.6 Tunnel states and state residence time

Section 19.5 showed how discounting and half-cycle correction convert a cycle-by-cycle Markov trace into a defensible present-value estimate of cost and QALYs. Both techniques assume the trace itself is trustworthy – but that trust rests on **memorylessness**: the assumption from §19.1 that a transition probability depends only on which state a patient currently occupies, not on how long they have been there. That assumption is frequently false, and this section shows the standard workaround.

Consider the **On Treatment** state in our running TB model. A patient who has just started the 6-month standard regimen (2HRZE/4HR) and one who is in their fifth month on the same regimen are both coded as "On Treatment" in a simple 4-state model, yet their risk of **Treatment Failure** is not plausibly identical – early non-response and late relapse-on-treatment have different biological drivers, and clinically we would expect the failure hazard to shift across the course of treatment rather than stay flat. A model that applies a single $p(\text{OnTx} \to \text{Fail}) = 0.08$ per 6-month cycle regardless of time-since-entry is implicitly averaging over this heterogeneity, which can bias cost and QALY estimates when the state-entry mix differs between the two arms being compared (as it typically does when one arm shortens treatment duration, exactly our Study 31/A5349 comparison).

The fix is the **tunnel state**: instead of one "On Treatment" state, define a short chain of sub-states that a cohort passes through in strict sequence – `OnTx_cycle1`, `OnTx_cycle2`, `OnTx_cycle3`, … – each carrying its own row of outbound transition probabilities. A patient entering treatment always starts at `OnTx_cycle1`; from there they can only move to `Cured`, `Fail`, `Death`, or forward into `OnTx_cycle2` (never back into `OnTx_cycle1`, and never skip ahead). This restores memorylessness **within** each tunnel sub-state while still letting the aggregate "on treatment" risk profile vary smoothly across state-time. Tunnel states are not a hack bolted onto the Markov formalism – they are the textbook general-purpose device for representing any non-exponential (non-memoryless) sojourn-time effect inside a cohort Markov model, at the cost of a larger but still linear state space.

Hand-building tunnel states is mechanical but tedious: every tunnel sub-state needs its own column of parameters, and with several time-varying quantities (failure risk, state cost, state utility) the bookkeeping grows quickly. `heemod` removes this burden with its `state_time` variable. Inside `define_transition()` or `define_state()`, any expression that references `state_time` is automatically recognised as state-time-dependent, and `heemod` expands the relevant state into the necessary tunnel states behind the scenes when the strategy is run – the modeller writes one expression, not N hand-coded sub-states. This is a real, documented `heemod` feature, not an approximation layered on top of the package; the mechanics described above are exactly what `state_time` is implementing under the hood.

```
library(heemod)

# Illustrative: failure risk declining with cycles spent on treatment
f_fail_by_state_time <- function(state_time) {
  dplyr::case_when(
    state_time <= 1 ~ 0.12,   # cycle 1 on treatment: higher early-failure risk
    state_time == 2 ~ 0.08,   # cycle 2: matches the base-case §19.1-19.5 value
    state_time >= 3 ~ 0.04    # cycle 3+: lower residual risk
  )
}

tm_tunnel <- define_transition(
  state_names = c("OnTx", "Cured", "Fail", "Death"),
  C,    0.75,  state_time,  0.02,
  0,    1,     0,           0,
  0.60, 0,     C,           0.15,
  0,    0,     0,           1
)
```

> **Common pitfall.** A common implementation error is tracking **cycles since model start** rather than **cycles since entry into the state**. These coincide only for patients who never leave and re-enter a state – but our Markov structure explicitly has a `Fail -> OnTx` back-edge: a patient who fails and re-enters treatment restarts their own on-treatment clock, even though the overall model clock keeps advancing. A tunnel implementation (hand-built or via `heemod`'s `state_time`) must reset the counter at every (re-)entry event, not read it off the global cycle index, or failure/relapse patients will be assigned the wrong column of the tunnel – silently understating their early-tunnel risk on a second course of treatment.

19.7 Age-dependent transitions and background mortality

Tunnel states handle risks that vary with time **since entering a state**. A separate and equally common violation of memorylessness is risk that varies with the patient's **age** or with **calendar time** in the model overall – most importantly, **background mortality**: the risk of dying from causes unrelated to TB, which rises steeply and non-linearly with age regardless of treatment arm.

A model that applies a single, constant $p(\rightarrow \text{Death})$ in every state ignores this entirely, which is defensible only for a short time horizon or a narrow, homogeneous-age cohort. For a lifetime horizon – or any model where disease-specific mortality is modest relative to all-cause mortality in an ageing cohort – background mortality must be pulled in explicitly, typically from a national or WHO life table giving age- and sex-specific annual mortality probabilities. Chapter 10's treatment of survival analysis and extrapolation covers how such life tables are sourced and used to extrapolate beyond trial follow-up; this section is concerned only with how they combine with disease-specific risk **inside** the transition matrix.

The practical problem is double-counting: if a cycle's disease-specific death transition (e.g. **Treatment Failure** -> **Death**, $p = 0.15$ per cycle in our base case) and the age-matched background mortality probability are simply added, patients who would have died of TB-related causes **and** who were also at background risk get counted twice, inflating total mortality unrealistically. Two standard approaches avoid this:

- **Additive-hazard approach**: convert both probabilities to instantaneous hazards (rate $= -\ln(1-p)/$ cycle length), sum the hazards, then convert the combined hazard back to a cycle probability. This is the theoretically cleaner approach because hazards (unlike probabilities) are additive by construction.
- **Maximum-of-two-probabilities approach**: take $p(\text{Death})$ for the cycle as $\max\left(p_{\text{disease-specific}}, p_{\text{background}}\right)$, on the reasoning that a patient cannot die twice and the dominant risk should drive the outcome. This is simpler to implement but is a cruder approximation, particularly when both probabilities are of comparable, non-trivial magnitude.

NICE DSU guidance – the same Technical Support Document series already relied on for structural and survival-extrapolation choices earlier in this book – recommends the hazard-based combination as the more defensible default, reserving the max-probability shortcut for cases where background mortality is negligible next to disease-specific risk (which is **not** generally true for an ageing TB cohort followed over a long horizon).

`heemod` supports age- and calendar-time-dependent parameters through its `model_time` variable, which is conceptually distinct from the `state_time` variable introduced in §19.6: `state_time` counts cycles since entry into the **current** state, while `model_time` counts cycles since the **start of the model run** (and, combined with a starting age, maps directly onto life-table age bands).

```
library(heemod)

# Illustrative life-table lookup keyed on model_time (age = start_age +
model_time)
background_mortality <- function(model_time, start_age = 45) {
  age <- start_age + model_time
  dplyr::case_when(
    age < 50 ~ 0.01,
    age < 65 ~ 0.03,
    TRUE     ~ 0.08
```

```
  )
}

# Combine via additive hazards, cycle length = 0.5 years (6-month cycles)
p_combined_death <- function(p_disease, model_time) {
  h_disease <- -log(1 - p_disease)
  h_bg      <- -log(1 - background_mortality(model_time))
  1 - exp(-(h_disease + h_bg))
}
```

19.8 Probabilistic transition matrices (Dirichlet) and Cholesky decomposition

Chapter 25 covers the general conventions for choosing PSA distributions for different parameter types (probabilities, costs, utilities, relative-effect estimates). Transition probabilities raise a structural wrinkle that deserves attention here, specific to the Markov setting: every row of a transition matrix must sum to exactly 1, so the cells within a row are not independent. Sampling each cell of a row from its own independent Beta distribution during a probabilistic sensitivity analysis will, in general, produce a row that no longer sums to 1 – silently breaking the Markov model's core algebraic constraint.

The natural fix is the **Dirichlet distribution**, the multivariate generalisation of the Beta that is specifically defined over vectors whose entries sum to 1. Just as a $\text{Beta}(a, b)$ distribution is the conjugate posterior for a binomial proportion given a and b "pseudo-counts" of success and failure, a $\text{Dirichlet}(\alpha_1, ..., \alpha_k)$ distribution is the conjugate posterior for a whole row of a multinomial transition, given a vector of pseudo-counts $\alpha_1, ..., \alpha_k$ – one count per destination state. Concretely, if you have (or can construct) a count matrix recording how many patients observed to start a cycle in state i were observed to end it in state j – `hesim`'s own documentation illustrates this with its count-matrix worked examples for exactly this purpose – each row of that count matrix directly parameterises a Dirichlet distribution for the corresponding row of the transition matrix, and every draw from it automatically sums to 1.

`dampack` exports a ready-to-use `rdirichlet()` function for this: given a row of counts, it returns PSA draws for that row that respect the sum-to-1 constraint by construction.

> **Difficult terminology: conjugate posterior.** In Bayesian statistics, you start with a **prior** distribution (your belief about a parameter before seeing data), combine it with observed data, and end up with a **posterior** distribution (your updated belief after seeing the data). In general, that combination step requires heavy computation — simulation methods like MCMC, as used elsewhere in this book. A "conjugate" prior is a special, lucky case where the prior and the data-generating distribution are mathematically matched so that the posterior comes out in the **same family** as the prior, with a simple formula instead of simulation. Concretely: if your prior for a success probability is a $\text{Beta}(a, b)$ distribution (think of a and b as "pretend" prior counts of past successes and failures), and you then observe real data of s successes and f failures, the posterior is just another Beta distribution — $\text{Beta}(a + s, b + f)$ — you simply add your observed counts onto the prior's pretend counts. No simulation, no numerical approximation, just arithmetic. That is exactly why Beta pairs so neatly with binomial (yes/no)

data, and why its multivariate cousin, Dirichlet, pairs the same way with multinomial (more-than-two-outcome) data: both are conjugate to the kind of data a Markov transition matrix naturally produces, which is what makes `rdirichlet()`'s pseudo-count trick above work with a one-line function call rather than a full Bayesian model fit.

```
library(dampack)

# Illustrative counts underlying the OnTx row: Cured / Fail / Death / OnTx (stay)
# (e.g. out of 100 patients starting a cycle On Treatment: 75 cured, 8 failed, 2
died,
#  with the remainder continuing on treatment -- counts need not match cycle
fractions
#  exactly, they encode the *evidence strength* behind each cell)
alpha_ontx <- c(Cured = 75, Fail = 8, Death = 2, OnTx = 15)

psa_draws_ontx <- dampack::rdirichlet(n = 1000, alpha = alpha_ontx)
# each row of psa_draws_ontx sums to 1 by construction -- no post-hoc
renormalisation needed
```

A related but conceptually separate problem arises for **correlated** normal- or lognormal-distributed parameters – for instance, if the relative treatment effect on **Cure** and the relative treatment effect on **Failure** are not independent (clinically plausible, since both are downstream of the same underlying treatment-response process), sampling them from independent univariate distributions during PSA throws away that correlation and can understate or overstate joint uncertainty. The standard device for preserving a specified correlation structure is the **Cholesky decomposition** of the correlation (or covariance) matrix: factorising it as $\Sigma = LL^T$ lets you transform a vector of independent standard normal draws into a vector of correlated normal draws via $x_{\text{correlated}} = \mu + Lz$. In base R this is a single call to `chol()`; `heemod`'s own PSA machinery uses the `mvnfast` package internally for exactly this multivariate-normal sampling step (`mvnfast` is listed as a real `Suggests` dependency in `heemod`'s own CRAN `DESCRIPTION`). The full mechanics of building and validating a PSA correlation structure belong to Chapter 25 – the point to carry forward here is narrower: whole transition-matrix rows need a Dirichlet, correlated continuous parameters need a Cholesky-based multivariate normal, and conflating the two (e.g. trying to "correlate" Dirichlet-sampled rows with a Cholesky step meant for normals) produces draws that no longer have a coherent probabilistic interpretation.

19.9 Calibration and validation

Calibration and **validation** are often used loosely as synonyms in casual modelling talk, but they answer different questions and belong at different points in the workflow.

Calibration is the process of adjusting one or more uncertain, unobserved (or poorly observed) model parameters so that the model's **outputs** match some external target. In our TB model, the transition probability from **Treatment Failure** back into **Death** (as opposed to re-entering **On Treatment**) might be informed only weakly by trial data, while a registry reports a well-measured 5-year all-cause survival curve for patients who ever failed first-line treatment. Calibration would search over plausible values of that transition probability until the model's own predicted 5-year survival curve for the Fail-exposed cohort matches the registry curve as closely as possible.

Calibration therefore uses **outputs** to pin down uncertain **inputs** – it runs in the opposite direction from a normal forward model evaluation.

Validation, by contrast, asks whether the model (with its parameters already set, calibrated or not) is **correct and credible** for its intended purpose, assessed from several distinct angles:

- **Face validity** – does a domain expert agree the structure, assumptions and parameter directions are clinically sensible, before looking hard at the numbers?
- **Internal (verification) validity** – does the model behave correctly at known boundary or extreme-value conditions (e.g. do state occupancies still sum to 1 every cycle, does setting a transition probability to 0 or 1 produce the expected degenerate trace, does cost scale linearly when a unit cost is doubled)?
- **Cross-validation** – do the model's results broadly agree with other independently published models addressing the same decision problem, and if not, can the discrepancy be traced to an identifiable structural or parameter difference?
- **External validation** – do the model's **predictions** (e.g. projected 5-year outcomes) hold up against real-world data collected after the model was built, the strongest and least commonly achievable form of validation.

Both calibration and validation checks belong at a specific point in the modelling workflow: **after** the base-case `run_model()` call produces a working, internally consistent trace and economic summary, and **before** any probabilistic sensitivity analysis is run on top of it. Running PSA over a miscalibrated or unvalidated base case simply propagates the error through every draw at greater computational cost, without fixing anything.

This chapter deliberately does not work through the calibration **methods** themselves – random search, Nelder-Mead and other optimisation routines, Bayesian calibration, Approximate Bayesian Computation (ABC), and Incremental Mixture Importance Sampling (IMIS) are all real, established calibration techniques, and Chapter 27 (Model Calibration, Validation and Quality Assurance) is where they are covered in full. Chapter 27 is likewise where the formal validation **frameworks** referenced across HEOR practice – AdViSHE (Assessment of the Validation Status of Health-Economic decision models), the ISPOR-SMDM Modeling Good Research Practices Task Force validation recommendations, and the TECH-VER checklist – are introduced and applied in detail. The job of this section is only to mark where, in the sequence from §19.1's model structure through this chapter's extensions, that checking must happen.

> **Agency perspective.** The ISPOR-SMDM Modeling Good Research Practices Task Force's model transparency and validation recommendations – already referenced earlier in this book for general modelling conduct – explicitly separate verification ("did we build the model right?") from validation ("did we build the right model?"). Keep that distinction sharp in any model documentation submitted to a HTA body: reviewers expect to see both addressed, and conflating the two in a single paragraph is a common and avoidable source of clarification requests during technical review.

19.10 Worked example: a chronic-disease CEA

To consolidate §19.1-19.9 end-to-end, this section works a complete cost-effectiveness analysis on a deliberately different, simpler example – a chronic, non-infectious condition – so the mechanics

stand on their own rather than being read as a continuation of the TB numbers used throughout the chapter.

Setting: a 3-state model – **Well**, **Diseased**, **Dead** – comparing a new disease-modifying therapy (DMT) against standard care (SoC) for a chronic progressive condition, over 20 six-month cycles (10 years), with a 3% annual discount rate applied to both costs and QALYs and half-cycle correction applied throughout (per §19.5's convention). All patients start **Well**. The transition matrices below are illustrative, not drawn from any specific published trial, and are constructed so DMT reduces the probability of progressing from **Well** to **Diseased** and of dying while **Diseased**, at a higher per-cycle treatment cost.

Transition	SoC	DMT	Note
Well → Well	0.80	0.85	DMT slows progression
Well → Diseased	0.15	0.10	
Well → Dead	0.05	0.05	background + baseline risk, held equal
Diseased → Diseased	0.85	0.90	DMT also slows decline once diseased
Diseased → Dead	0.15	0.10	
Dead → Dead	1.00	1.00	absorbing
Each row sums to 1.00 in both arms, confirmed by direct summation.			

Table 51: Illustrative per-6-month-cycle transition probabilities, by arm

State values: a six-month cost of $500 (Well) / $8,000 (Diseased) under SoC, rising to $3,500 (Well, reflecting the DMT's own acquisition cost) / $11,000 (Diseased, DMT plus any added monitoring) under the new therapy; six-month utility weights of 0.90 (Well) and 0.60 (Diseased), assumed equal across arms (the DMT is assumed to slow progression, not to change the intrinsic health-state utilities conditional on being in a state).

```
# Base-R cohort trace with half-cycle-corrected discounting, following §19.5's
pattern
run_chronic_model <- function(tm, cost_vec, util_vec, cycles = 20, r = 0.03) {
  trace <- matrix(0, nrow = cycles + 1, ncol = 3)
  trace[1, ] <- c(1, 0, 0)                 # start: 100% Well
  for (t in 2:(cycles + 1)) {
    trace[t, ] <- trace[t - 1, ] %*% tm
  }
  total_cost <- 0; total_qaly <- 0
  for (t in 1:cycles) {
    avg_occ <- (trace[t, ] + trace[t + 1, ]) / 2   # half-cycle correction
    disc <- 1 / (1 + r)^t
    total_cost <- total_cost + sum(avg_occ * cost_vec) * disc
    total_qaly <- total_qaly + sum(avg_occ * util_vec) * disc
  }
  list(trace = trace, cost = total_cost, qaly = total_qaly)
}

tm_soc <- matrix(c(0.80, 0.15, 0.05,
                   0.00, 0.85, 0.15,
```

```
                    0.00, 0.00, 1.00), nrow = 3, byrow = TRUE)
tm_dmt <- matrix(c(0.85, 0.10, 0.05,
                    0.00, 0.90, 0.10,
                    0.00, 0.00, 1.00), nrow = 3, byrow = TRUE)

res_soc <- run_chronic_model(tm_soc, c(500, 8000, 0), c(0.90, 0.60, 0))
res_dmt <- run_chronic_model(tm_dmt, c(3500, 11000, 0), c(0.90, 0.60, 0))
```

Running this trace (independently re-verified during assembly) gives discounted 10-year totals of \$29,317.05 in cost and 5.5522 QALYs under SoC, and \$56,917.97 in cost and 6.6704 QALYs under DMT. The resulting incremental cost is $56917.97 - 29317.05 = 27600.92$, and the incremental QALY gain is $6.6704 - 5.5522 = 1.1181$, giving an ICER of $27600.92/1.1181 = 24685$ per QALY gained – below a conventional \$30,000-\$50,000 willingness-to-pay threshold, so DMT would be judged cost-effective against SoC at that threshold under this illustrative parameterisation.

How to report this.

Arm	Cost	QALYs	Incr. Cost	Incr. QALYs	ICER
Standard care (SoC)	29,317.05	5.5522	–	–	–
Disease-modifying therapy (DMT)	56,917.97	6.6704	27,600.92	1.1181	24,685

3% annual discounting; half-cycle correction applied; 20 six-month cycles (10-year horizon). ICER = incremental cost / incremental QALYs = 27,600.92 / 1.1181.

Table 52: Chronic-disease CEA: discounted 10-year results, SoC vs DMT

This worked example closes the chapter's treatment of cohort Markov models: states, transitions and cycles (§19.1), time-homogeneous and time-dependent matrices (§19.2-19.3), implementation in base R and in `heemod`/`hesim`/`dampack` (§19.4), discounting and half-cycle correction (§19.5), tunnel states for time-since-entry effects (§19.6), age- and calendar-time-dependent transitions (§19.7), Dirichlet- and Cholesky-based uncertainty propagation (§19.8), and where calibration and validation sit in the workflow (§19.9) all feed directly into the cost and QALY totals computed here. Not every decision problem is well served by a hand-specified transition matrix, however: per the structural-choice framework set out in Chapter 17 (§17.2), when an outcome is driven directly by extrapolated survival curves fitted in Chapter 10 – rather than by a small number of clinically interpretable health states and transitions – a **partitioned survival model** is usually the more natural and more transparent structural choice. Chapter 20 takes up partitioned survival models next.

Chapter 20. Partitioned Survival Models

Chapter 19 built a Markov cohort model from a transition matrix. It let the cohort trace evolve one cycle at a time. This chapter builds a different, equally common engine: the Partitioned Survival Model (PSM). A Markov model estimates transition probabilities between states. A PSM skips that step entirely. It reads state occupancy directly off two survival curves that are already fitted independently: progression-free survival (PFS) and overall survival (OS).

PSMs dominate oncology health technology assessment for a simple reason. PFS and OS are exactly the co-primary endpoints most oncology trials already report as Kaplan-Meier curves. Chapter 17′s §17.2 model-type framework (Siebert et al., 2012, ISPOR-SMDM Task Force-3) backs this choice: when your data are two survival curves rather than a transition matrix, a PSM is the natural fit.

Learning objectives

- Describe the three-state PSM structure and derive state occupancy directly from the PFS and OS survival functions, and recognise the curve-crossing structural flaw this introduces (20.1).
- Fit PFS and OS curves independently with `flexsurvreg()` (reusing Chapter 10′s six parametric families) and combine the extrapolated curves into a state-occupancy trace (20.2).
- Explain treatment waning and relative-effect extrapolation as an explicit structural assumption requiring justification, grounded in NICE DSU TSD 14 (20.3).
- Situate `hesim`'s dedicated PSM architecture alongside a transparent custom-code alternative built from `flexsurv`'s own `summary()` method (20.4).
- Compare a PSM's limitations against the Chapter 19 Markov cohort model: curve crossing, fixed three-state structure, and compounding independent extrapolation uncertainty (20.5).
- Work a complete, independently-verified oncology cost-effectiveness analysis end-to-end using a partitioned survival structure (20.6).

20.1 Structure and when it is appropriate

A Partitioned Survival Model (PSM) is the dominant model structure in oncology health technology assessment. A Markov model, from Chapter 19, pushes a cohort through states using a transition matrix. A PSM skips the matrix entirely. State occupancy comes straight from two survival curves, fitted independently: progression-free survival (PFS) and overall survival (OS).

The model has three health states: pre-progression (on treatment, alive, not yet progressed), post-progression (alive, progressed), and death. At any time t:

- $P(\text{PFS}) = S_{\text{PFS}(t)}$
- $P(\text{Dead}) = 1 - S_{\text{OS}(t)}$
- $P(\text{Progressed}) = S_{\text{OS}(t)} - S_{\text{PFS}(t)}$

No transitions are estimated or elicited. Occupancy falls straight out of the two curves. This is attractive when your data **are** two Kaplan-Meier curves – PFS and OS, the standard co-primary endpoints in an oncology trial – rather than a transition matrix you could sensibly fit or elicit. That is the decision rule from Section 17.2′s framework: choose the model structure to match the data

you actually have. In exchange for this simplicity, a PSM gives up some flexibility. A transition matrix lets you sub-divide post-progression, allow recovery, or add drug-specific adverse-event states. A PSM cannot.

> **Common pitfall.** The post-progression occupancy formula $S_{\mathrm{OS}(t)} - S_{\mathrm{PFS}(t)}$ has no constraint forcing it to stay non-negative. If the OS and PFS extrapolations cross – which happens often once you extrapolate parametric curves well past the trial's follow-up – the model predicts a **negative** number of patients in the progressed state. This is not a coding bug to patch around; it is a structural flaw of PSMs that a proper state-transition (Markov) model cannot produce, because Markov occupancy is always a probability by construction. Always plot $S_{\mathrm{OS}(t)} - S_{\mathrm{PFS}(t)}$ across your full time horizon and confirm it stays non-negative before trusting a PSM's extrapolated results.

20.2 Building PSMs from survival extrapolations

Nothing new is needed to fit the curves – this is the same `flexsurvreg()` workflow from Chapter 10, run twice: once on PFS data, once on OS data, each fitted independently across the same six standard parametric families (exponential, Weibull, log-normal, log-logistic, Gompertz, generalised gamma).

```
library(flexsurv)

# Fit Weibull curves separately to PFS and OS data
fit_pfs <- flexsurvreg(formula = Surv(time, event) ~ 1,
                       data = pfs_data, dist = "weibull")
fit_os  <- flexsurvreg(formula = Surv(time, event) ~ 1,
                       data = os_data,  dist = "weibull")
```

Each fit is extrapolated beyond the trial's observed follow-up using `summary(fit, type = "survival", t = times)`, exactly as in Chapter 10. The only new step is combining the two extrapolated curves with the occupancy formulas from Section 20.1 above.

```
times <- seq(0, 60, by = 1)  # months

s_pfs <- summary(fit_pfs, type = "survival", t = times)[[1]]$est
s_os  <- summary(fit_os,  type = "survival", t = times)[[1]]$est

occupancy <- data.frame(
  time         = times,
  p_pfs        = s_pfs,
  p_progressed = s_os - s_pfs,
  p_dead       = 1 - s_os
)
```

On a small synthetic dataset with PFS median 8 months and OS median 20 months, both Weibull fits extrapolate smoothly out to 60 months without crossing: `p_progressed` stays positive throughout, rising then slowly falling as `p_dead` grows. Check this with `min(occupancy$p_progressed) >= 0` before treating the base case as valid – cheap insurance against the Section 20.1 pitfall.

HEOR in practice. Fit PFS and OS independently, pick the best-fitting family for each by AIC/BIC **and** clinical plausibility separately (per Chapter 10′s guidance) – do not force both curves into the same family just for tidiness. Report both fits and the crossing check in the technical appendix; reviewers increasingly ask for it.

20.3 Treatment waning and relative-effect assumptions

A second, distinct PSM pitfall sits on top of the curve-crossing problem. How far do you trust the trial's relative treatment effect once you extrapolate past the trial's own follow-up window?

A trial typically reports a hazard ratio (HR) for OS, estimated over its observed follow-up – say, 24 months. Extrapolating the fitted intervention and comparator OS curves out to a 10- or 20-year lifetime horizon implicitly assumes something about that HR: either it holds forever ("constant relative effect"), or it changes in some specified way. Many PSM base cases silently assume the former. They fit proportional-hazards-style extrapolations with no waning, treating an untested assumption as a default rather than a choice.

A waning assumption says instead that the treatment's benefit tapers – often linearly – to zero over some stated number of years after the trial ends. The idea is simple: a drug's effect on survival shouldn't be assumed to outlast all biological plausibility, or the trial evidence itself. Both "no waning" and "waning" are structural assumptions about something the trial data cannot speak to directly, since by definition neither curve has follow-up out that far. Either needs explicit justification – clinical plausibility, mechanism of action, or external long-term registry data, typically – not just a software default.

Agency perspective. NICE DSU TSD 14 (Latimer, 2013), already cited in Chapter 10 for extrapolation method choice, speaks directly to this point. It requires analysts to justify their chosen extrapolation – including any assumption about whether and how a treatment effect wanes after the trial period – rather than defaulting to indefinite benefit. NICE appraisal committees have repeatedly pushed back on submissions that extrapolate a short-term HR unchanged across a lifetime horizon with no clinical rationale offered. If you cannot justify "the effect holds forever," run a waning scenario as a named structural sensitivity analysis, not an afterthought.

20.4 Modelling with `hesim`, `flexsurv` and custom code

You already have the two pieces a PSM needs. `flexsurvreg()` fits each curve (Chapter 10). `hesim` builds cohort-level economic models on top of fitted survival objects (Chapter 19). `hesim`'s own documentation describes a dedicated module for partitioned survival models. It wraps a set of fitted curves per treatment strategy and converts them into state-occupancy probabilities for costing – the same PFS/OS-to-state-occupancy mapping from Section 20.1, handled inside `hesim`'s simulation engine instead of by hand. The exact class and constructor names in that module have changed across package versions. Rather than quote a signature that may have since been renamed, treat this as a pointer: check the current CRAN vignette for `hesim`'s PSM workflow, then wire your

`flexsurvreg()` fits into it the same way you wired them into `hesim_data()` and `define_tparams()` in Chapter 19.

> **Agency perspective.** The safest habit when a package's API for a specific feature is a moving target: fit your survival curves with the stable, well-documented `flexsurvreg()`, then check the installed package's own vignette (`vignette(package = "hesim")`) before calling anything PSM-specific, rather than trusting code you found in an older blog post.

For transparency – and because a regulatory submission often wants the state-occupancy arithmetic shown in the open, not hidden inside a package's internals – it is just as common to write the partitioned survival calculation yourself. `flexsurv` gives you the one function you need for this. `summary()` on a fitted `flexsurvreg` object, with `type = "survival"`, returns fitted survival probabilities at any vector of times.

```
# fit_pfs, fit_os are flexsurvreg objects (Section 20.1)
psm_occupancy <- function(fit_pfs, fit_os, times) {
  s_pfs <- summary(fit_pfs, t = times, type = "survival")[[1]]$est
  s_os  <- summary(fit_os,  t = times, type = "survival")[[1]]$est
  data.frame(
    time         = times,
    p_pfs        = s_pfs,
    p_dead       = 1 - s_os,
    p_progressed = pmax(0, s_os - s_pfs)  # floor at 0: see 20.5
  )
}
```

That `pmax(0, ...)` is doing real work: it is the first line of defence against the curve-crossing problem flagged in Section 20.1, and it is the same guard you'll want inside any custom PSM code regardless of which package handles the rest.

20.5 Limitations compared with state-transition models

A PSM buys simplicity – two curves per arm instead of a full transition matrix – at a real structural cost relative to the Markov cohort models of Chapter 19.

Curve crossing. Section 20.1 already showed that $P(\text{Progressed}) = S_{\text{OS}(t)} - S_{\text{PFS}(t)}$ can go negative if the OS curve ever dips below the PFS curve. A Markov model's transition probabilities are estimated directly and can't produce a negative state share by construction; a PSM's two independently-fitted curves can.

> **Common pitfall.** Curve crossing usually shows up far out in the extrapolated tail, long after the trial data ends – exactly where you're least likely to notice it on a quick plot of the observed period. Always plot $S_{\text{PFS}(t)}$ and $S_{\text{OS}(t)}$ together across your full model horizon, not just the observed follow-up, before trusting the resulting cost and QALY totals.

Fixed structure. A PSM has exactly three states – PFS, Progressed, Death. There is no room for the tunnel states or multiple progression lines that `heemod::define_state()` could add to a Markov model (Section 19.6). If the disease has clinically distinct second- or third-line progression states, a PSM simply cannot represent them.

Weaker mechanistic link between arms. A Markov model shares one transition structure. Treatment effect enters through hazard ratios on specific transitions. A PSM is different: it fits each arm's PFS and OS curves completely independently. Nothing in the model enforces consistency between arms, beyond whatever the analyst imposes – a shared shape parameter, say, or treatment waning per NICE DSU TSD 14/21. Two separate extrapolations also means two separate sources of extrapolation uncertainty. They compound rather than cancel.

These are reasons for caution, not reasons to avoid PSMs. Oncology submissions use them constantly, because trial endpoints (PFS, OS) map directly onto the model states. But Chapter 19′s §19.9 distinction between internal validation and external calibration applies here with extra force. A PSM's extrapolated tails need checking against external registry or long-term follow-up data – a good fit to the trial's own Kaplan-Meier curve is not enough on its own. Chapter 27 returns to this validation workflow in full.

20.6 Worked example: an oncology CEA

Two arms, new drug vs standard of care (SOC), each with independently-fit Weibull PFS and OS curves (shape/scale chosen only to be plausible, not from a real trial):

```
library(flexsurv)

# Weibull survival at time t: S(t) = exp(-(t/scale)^shape)
weib_S <- function(t, shape, scale) exp(-(t / scale)^shape)

arms <- list(
  soc = list(pfs = c(shape = 1.2, scale = 1.0), os = c(shape = 1.1, scale = 2.0),
             cost_pfs = 3000, cost_prog = 2000, cost_death = 1000),
  new = list(pfs = c(shape = 1.2, scale = 1.5), os = c(shape = 1.1, scale = 2.5),
             cost_pfs = 5000, cost_prog = 2000, cost_death = 1000)
)
util_pfs <- 0.75; util_prog <- 0.55
```

20 six-month cycles (10-year horizon), NICE 3.5% annual discounting applied at each cycle's midpoint, $\mathrm{DF}(t) = (1+r)^{-t}$ (Section 4.6):

```
run_arm <- function(a, n_cycles = 20, cycle_len = 0.5, r = 0.035) {
  t <- seq(0, n_cycles * cycle_len, by = cycle_len)
  s_pfs <- weib_S(t, a$pfs["shape"], a$pfs["scale"])
  s_os  <- weib_S(t, a$os["shape"],  a$os["scale"])
  p_pfs  <- s_pfs
  p_dead <- 1 - s_os
  p_prog <- pmax(0, s_os - s_pfs)

  total_cost <- 0; total_qaly <- 0
  for (i in 2:length(t)) {
    df <- (1 + r)^(-(t[i-1] + t[i]) / 2)       # mid-cycle discount factor
    avg_pfs  <- (p_pfs[i-1]  + p_pfs[i])  / 2
    avg_prog <- (p_prog[i-1] + p_prog[i]) / 2
    new_deaths <- p_dead[i] - p_dead[i-1]
    cyc_cost <- avg_pfs * a$cost_pfs + avg_prog * a$cost_prog +
                new_deaths * a$cost_death
    cyc_qaly <- (avg_pfs * util_pfs + avg_prog * util_prog) * cycle_len
    total_cost <- total_cost + cyc_cost * df
```

```
    total_qaly <- total_qaly + cyc_qaly * df
  }
  c(cost = total_cost, qaly = total_qaly)
}

res <- sapply(arms, run_arm)
res
```

Running this gives discounted lifetime cost and QALYs of £10,077.03 / 1.1875 QALYs for SOC and £18,013.61 / 1.5003 QALYs for the new drug (both independently recomputed in Python during drafting, confirming neither arm's PFS/OS extrapolation crosses over the 10-year horizon). The incremental cost is £18,013.61 minus £10,077.03 = £7,936.57, and the incremental QALY gain is 1.5003 minus 1.1875 = 0.3128, so:

$$\text{ICER} = \frac{7936.57}{0.3128} \approx 25371 \ \text{£/QALY}$$

How to report this.

Strategy	Cost (£)	QALYs	ICER (£/QALY)
Standard of care	10,077	1.19	–
New drug	18,014	1.50	25,371

Table 53: Base-case discounted cost, QALYs and ICER from the synthetic oncology PSM, new drug vs standard of care (SOC)

HEOR in practice. Keep the arithmetic traceable in any CEA you hand off: show cost and QALY totals per arm, the two increments, and the division that produces the ICER, exactly as above. A reviewer should be able to recompute 7936.57 divided by 0.3128 on a calculator and get your number – if they can't, the model is the first thing they'll distrust, regardless of how defensible the clinical assumptions are.

Both this PSM and Chapter 19's Markov cohort model share one feature: they track the **mean** proportion of a hypothetical cohort in each state, cycle by cycle. That's enough when cost and outcomes depend only on current state. It breaks down once a patient's own history matters – for example, cumulative treatment exposure, time already spent progression-free, or prior-line-dependent costs that no cohort-average state occupancy can capture. Chapter 21 introduces individual-level state-transition models (microsimulation) to handle exactly that case.

Chapter 21. Individual-Level State-Transition Models (Microsimulation)

Chapter 19 built a cohort Markov model for tuberculosis treatment. Chapter 20 built a partitioned survival model for an oncology indication. Both closed with the same warning: they track mean proportions of a cohort, not individual patients. This chapter builds the model type that exists specifically to fix that – microsimulation, where each patient is simulated one at a time, carrying their own risk profile and their own history forward through the model.

Learning objectives

- Identify when cohort-level and curve-based models break down, and when microsimulation is genuinely needed instead (21.1).
- Distinguish patient heterogeneity from history dependence, and connect both to the clock-forward/clock-reset multi-state time-scale distinction (21.2).
- Apply vectorisation, `data.table`, and parallel computing to keep a microsimulation model computationally tractable in R (21.3).
- Distinguish first-order (individual/stochastic) Monte Carlo uncertainty from second-order (parameter) uncertainty when interpreting microsimulation PSA output (21.4).
- Apply common random numbers so that patient-level noise cancels out of an incremental comparison between strategies (21.5).
- Use `hesim`'s real individual-level CTSTM functions and classes to build, simulate, and discount an individual patient simulation (21.6).
- Work a complete, independently-checked microsimulation of a TB screening programme end-to-end (21.7).

21.1 When microsimulation is needed

Chapter 19 built a cohort Markov model for tuberculosis treatment. Chapter 20 built a partitioned survival model for an oncology indication. Both closed with the same warning: they track mean proportions of a cohort, not individual patients. That is fine until a patient's own history starts to matter for their own future risk.

Microsimulation – also called individual patient simulation (IPS) or an individual-level state-transition model – simulates one patient at a time, cycle by cycle, keeping track of each patient's own state, time, and accumulated history. Three situations push you from a cohort model toward microsimulation.

First, risk that depends on cumulative exposure. A patient who has had six cycles of a toxic chemotherapy regimen is not at the same risk of a cardiac event as a patient who has had one cycle. A cohort trace cannot express "six cycles of exposure" – it only knows what fraction of the cohort is currently on treatment.

Second, risk that depends on time already spent in a state. In oncology, survival after progression often depends on how long the patient was progression-free beforehand. Chapter 20's PSM model

only reads off progression-free survival and overall survival curves; it has no mechanism for feeding "time since progression" back into a hazard.

Third, costs or outcomes that depend on treatment-line history. A patient starting third-line therapy after two prior failed lines has a different cost and outcome profile than a treatment-naive patient entering first line, even if both are notionally in the "on treatment" state.

HEOR in practice. A patient's personalised 10-year cardiovascular risk might combine age, smoking status, and cumulative treatment exposure through a risk equation such as $h_i(t) = h_0(t) \times \exp(\beta_1 \text{age}_i + \beta_2 \text{cumdose}_i(t))$. A cohort Markov model cannot carry $\text{cumdose}_i(t)$ for an individual i – it only has a state proportion. Microsimulation carries it naturally because each simulated patient is tracked separately.

None of this is free. A cohort model updates one trace vector per cycle – a handful of arithmetic operations, done once. A microsimulation model instead updates the state and history of, say, 10,000 simulated patients, for every model cycle, and usually repeats the whole run for 1,000+ probabilistic sensitivity analysis (PSA) samples. The computational cost is not incremental – it is a different order of magnitude, which is why Sections 21.3-21.5 are dedicated to doing this efficiently in R.

Common pitfall. Do not reach for microsimulation just because a model is "complex" or has many health states. A cohort Markov model with ten states and complicated transition probabilities is still a cohort model if transitions depend only on current state. The decision to move to microsimulation should be driven specifically by heterogeneity or history dependence, not by state count.

Agency perspective. Siebert et al. (2012), the ISPOR-SMDM Task Force-3 report already used in Chapter 17 to choose between decision trees, Markov models, partitioned survival models, microsimulation, and discrete event simulation, explicitly lists individual-level state-transition models as the appropriate choice once "the history of events experienced by the individual" affects future risk or cost. That is the same criterion used above, applied consistently across Chapters 17, 19, 20, and this chapter.

21.2 Patient heterogeneity and history dependence

Section 21.1 gave three motivating examples. They actually split into two distinct problems, and it is worth separating them cleanly because they call for different modelling choices.

Patient heterogeneity is between-patient variation that already exists when a patient enters the model: age, sex, comorbidity count, baseline risk score. A cohort model handles this crudely, if at all, by running separate cohorts for a few subgroups. Microsimulation handles it directly: draw a vector of baseline covariates for each simulated patient, usually by sampling from a fitted distribution or resampling from a patient-level dataset, and carry that patient's own covariate vector through the whole simulation.

```
# Simulate baseline covariates for n patients
n <- 10000
```

```r
patients <- data.frame(
  id = 1:n,
  age = rnorm(n, mean = 62, sd = 9),
  comorbidity_score = rpois(n, lambda = 1.4)
)
```

History dependence is a different problem: within one patient's simulated life, what happens next depends on what has already happened to that same patient. Cumulative treatment exposure and time-since-progression (Section 21.1) are both history dependence, not heterogeneity – they emerge during the simulation, not at baseline.

The clean way to think about history dependence is through the model's **time scale**. In a multi-state model, "clock-forward" time measures time since the patient entered the model overall; transition hazards depend on that single clock, and the model remains effectively Markov even though it is simulated patient-by-patient. "Clock-reset" time measures time since the patient most recently entered their **current** state; the clock resets at every state transition, and this is a semi-Markov model, because the hazard for leaving a state depends on how long the patient has been in it, not on total elapsed model time.

This distinction is not a simplification for teaching purposes – it is built into the software. The `hesim` package's `IndivCtstmTrans` class, which runs continuous-time individual patient simulations, has a `clock` argument that accepts `"forward"`, `"reset"`, or a mix of the two across different transitions. That gives you a direct, named way to tell the software which transitions are genuinely history-dependent (clock-reset) and which are not (clock-forward).

HEOR in practice. In the oncology example from Chapter 20, post-progression survival depending on time-already-spent progression-free is naturally a clock-reset relationship: the hazard of death after progression depends on time since entering the "progressed" state, not on total time since diagnosis. Cumulative treatment exposure, by contrast, is not a clock at all – it is an accumulating covariate that must be tracked and fed into the hazard equation as a time-varying covariate, separately from either clock convention.

Common pitfall. Do not default every transition to clock-reset "to be safe". Clock-reset multi-state models require more data to estimate (effectively a separate time-to-event model per state entry) and are harder to validate. Use clock-forward wherever a cohort-style Markov assumption is actually defensible for that transition, and reserve clock-reset for the transitions where you have evidence, or strong clinical rationale, that time-in-state genuinely drives the hazard.

Agency perspective. Siebert et al. (2012) frames exactly this choice as part of selecting a model type: if event history or elapsed time in a state materially changes future risk, a cohort-level Markov model is no longer an adequate representation, and an individual-level state-transition model with explicit history-tracking becomes the appropriate choice. Sections 21.1-21.2 together operationalise that single Task Force-3 criterion.

21.3 Efficient implementation: vectorisation, data.table, parallel computing

A microsimulation model has three nested loops: cycles, patients, and PSA samples. Written naively, that is three nested `for` loops in R, and R's per-iteration overhead is not negligible. Multiply a few hundred cycles by ten thousand patients by a thousand PSA samples, and a model that should take seconds can take hours if every patient is updated one at a time at the R level.

Three practical techniques keep this tractable.

Vectorisation replaces "loop over patients" with "operate on all patients' current state at once," using R's native vector and matrix operations. Instead of updating one patient's state per loop iteration, hold all patients' current states as a vector and update the whole vector in one line.

```
# Naive: one iteration of overhead per patient, per cycle
for (p in 1:n_patients) {
  if (state[p] == "sick" && runif(1) < p_death[p]) {
    state[p] <- "dead"
  }
}

# Vectorised: one operation covers all patients at once
alive_sick <- state == "sick"
dies_this_cycle <- alive_sick & (runif(n_patients) < p_death)
state[dies_this_cycle] <- "dead"
```

The vectorised version does the same arithmetic, but R's interpreter overhead is paid once per cycle instead of once per patient per cycle. For ten thousand patients that difference compounds fast.

`data.table` is useful once each patient also carries a growing history (states visited, costs accrued, cumulative exposure) that needs fast lookups, updates, and grouped summaries across tens of thousands of rows. Two features matter here: modification by reference with `:=`, which avoids copying the whole patient-history table on every update, and fast grouped aggregation with `by`.

```
library(data.table)
dt <- as.data.table(patients)

# Update by reference: no full-table copy
dt[, cumulative_cost := cumulative_cost + cycle_cost]

# Fast grouped summary, e.g. mean cost by final state
dt[, .(mean_cost = mean(cumulative_cost)), by = final_state]
```

This is not an arbitrary recommendation: `hesim`, the package introduced fully in Section 21.6 for continuous-time individual simulation, lists `data.table` as one of its own package dependencies, and uses it internally for exactly this kind of large patient-history bookkeeping.

Parallel computing targets the outer PSA loop, not the inner patient loop. Each PSA sample draws its own set of parameter values and then runs an entire, self-contained microsimulation to completion – one PSA sample's result never depends on another's. That makes the PSA loop "embarrassingly parallel": it can be split across CPU cores with no coordination needed between cores. Base R's `parallel` package (`mclapply()` on Linux/macOS, `parLapply()` with a cluster on Windows) or the `future`/`furrr` ecosystem can run, say, eight PSA samples at once on an eight-core machine for close to an eight-fold speedup.

```r
library(parallel)
psa_results <- mclapply(1:n_psa, function(i) {
  run_one_psa_sample(seed = i)
}, mc.cores = detectCores() - 1)
```

The inner loop – simulating individual patients within one PSA sample – is a harder parallelisation target. Patients within a run often need to draw random numbers from a shared, carefully managed stream, particularly when the model compares two treatment arms using the same underlying patient draws. Section 21.5 covers this common-random-numbers technique in detail; for now, the practical rule is: parallelise across PSA samples first, and treat patient-level parallelisation as a separate, more delicate problem.

Common pitfall. Do not parallelise before you have vectorised. Running a slow, patient-by-patient `for` loop on eight cores instead of one is still eight slow loops. Vectorise the per-cycle patient update first; parallelise the PSA loop second. Doing it in the other order wastes the easier, larger win.

HEOR in practice. A practical build order for a microsimulation model: (1) write a correct but slow single-PSA-sample, single-patient-loop version first, and validate its output against known logic; (2) vectorise the patient loop within one cycle; (3) move patient history into a `data.table` once per-patient bookkeeping starts to dominate runtime; (4) only once (1)-(3) are done, wrap the whole single-PSA-sample function in `mclapply()` across PSA samples. Parallelising a model you have not yet vectorised just multiplies the existing inefficiency across cores.

21.4 First-order versus second-order Monte Carlo uncertainty

A microsimulation produces two kinds of randomness at once. Keeping them apart is the single most important idea in this section.

First-order uncertainty is patient-level variability. Fix every parameter in the model. Simulate one patient, then another, then another. Each one takes a different path through the state structure, purely because of chance draws (does this patient progress this cycle or not?). This is the randomness that makes a microsimulation "individual-level" at all, and it is exactly the mechanism described in Section 21.2.

Second-order uncertainty is parameter uncertainty – the familiar PSA idea from Chapter 14 and Chapter 19. Here you hold the simulated population fixed in spirit and instead vary the model's parameters: a posterior draw of a hazard ratio, a Dirichlet-sampled row of a transition matrix, a different cost coefficient. Chapter 19's `dampack::rdirichlet()` examples are second-order uncertainty. Chapter 25 covers how to summarise second-order output properly, including cost-effectiveness acceptability curves – this section does not repeat that.

The practical consequence: in a microsimulation, every second-order draw (every PSA iteration) itself requires simulating a population of individuals, and that population carries first-order noise. If you simulate too few patients per PSA draw, the spread you see across draws is a mixture of genuine parameter uncertainty and simulation noise that has nothing to do with your parameters. Simulating more patients per draw shrinks first-order noise, by the law of large numbers, but it

does nothing to second-order uncertainty – that depends only on how uncertain your parameter estimates are. The two are controlled by two completely different levers: number of simulated patients (first-order) and number of PSA parameter draws (second-order).

Difficult terminology: law of large numbers. The law of large numbers is one of the oldest and most intuitive ideas in probability: if you repeat a random process many times and average the results, that average gets closer and closer to the "true" underlying expected value as you add more repetitions, even though any single repetition is unpredictable. A simple analogy: a single coin flip is a coin toss, genuinely 50/50 — but flip a fair coin 10,000 times and the proportion of heads will land extremely close to 50%, not because any individual flip became more predictable, but because the random ups and downs average each other out across a large number of trials. In this chapter's microsimulation context: simulating a single patient's trajectory is like a single coin flip — almost pure noise, entirely dominated by that one patient's random luck. Simulate 10,000 patients under the **same** fixed parameters and average their outcomes, and that average cost/QALY estimate converges tightly onto the model's true expected value for those parameters — not because the individual patients became less random, but because averaging over many of them cancels out the individual-level noise. This is exactly why the pitfall above distinguishes "not enough simulated patients" (fixable by simulating more — the law of large numbers eventually irons out the noise) from "genuine parameter uncertainty" (not fixable by simulating more patients at all, because that uncertainty lives in not knowing the true parameter values, not in patient-level randomness).

Common pitfall. A PSA on a microsimulation model with too few simulated patients per iteration will show a cost-effectiveness acceptability curve that is noisier and flatter than the true parameter uncertainty justifies. The fix is not a different PSA method – it is simply simulating more patients per parameter draw until first-order noise is small relative to the spread you are trying to characterise. Do not report "wide uncertainty" from a microsimulation without first checking whether that width is genuine parameter uncertainty or simulation noise that a larger cohort size would remove.

HEOR in practice. A practical check: run the **same** set of parameters (no second-order variation at all) through the model twice, with two different patient cohorts of your chosen size N. If headline outputs (mean cost, mean QALYs) differ by more than a trivial fraction of the output's scale, N is too small. Only once this check passes is it safe to layer second-order PSA draws on top.

Agency perspective. Briggs et al. (2012), "Model Parameter Estimation and Uncertainty: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force-6" (**Value in Health** 15(5):835-842), is the primary published source for the exact first-order/second-order distinction used above. The Task Force report explicitly separates stochastic (first-order) uncertainty from parameter (second-order) uncertainty, and separately again from heterogeneity (Section 21.2) – the same three-way split this chapter follows, not a simplification invented for this book. It sits alongside Siebert et al.'s state-transition Task Force-3 report (already used

in Chapters 17/19/20/21.1) as a second named ISPOR-SMDM report, since Task Force-6 is specifically the uncertainty-analysis companion report in the same 2012 series.

21.5 Common random numbers

Comparing two strategies in a microsimulation means simulating two populations – one per strategy – and subtracting the results. If those two populations are generated with independent random draws, the subtraction carries the first-order noise from **both** simulations. That noise can swamp a genuinely small, real treatment effect, especially when N is modest.

Common random numbers (CRN) fixes this. Instead of letting each arm draw its own independent stream of random numbers, you reuse the identical stream across arms. Concretely: sample patient 1's age, sex, baseline risk and progression random numbers **once**. Use that same patient – the same numbers – to simulate them under strategy A, and then again under strategy B. Patient 1 in arm A and patient 1 in arm B are the same simulated individual, just routed through two different treatment rules. Because the underlying randomness is identical, the part of the outcome driven by chance (not by treatment) is correlated across arms, and it cancels out when you compute the incremental difference.

In base R this is usually as simple as calling `set.seed()` with the same seed immediately before simulating each arm, and making sure the covariate-sampling and progression-sampling code runs in the same order with the same random-number calls each time – only the transition probabilities or hazards that depend on treatment are allowed to differ between the two runs.

`hesim` applies CRN by design, not as an optional setting. Its input-data construction (Section 21.6) ties a given patient and a given PSA sample to the same row of simulated data across every strategy being compared – `hesim_data()` and `expand()` build one patient population per PSA sample, not one per strategy, so the same patient-level random draws are reused across strategies automatically.

Common pitfall. CRN only works if the random-number sequence stays aligned across arms. If you later add a third strategy and its simulation code calls a different number of random functions before reaching the step that matters – one extra `rnorm()` call to sample a new covariate, say – every subsequent draw shifts out of alignment with the other arms, and the correlation that CRN relies on silently breaks. The output will still run and look plausible. It will just no longer be variance-reduced. When adding a new arm, audit that its random-number call sequence matches the existing arms up to the point where the strategies diverge.

21.6 Using `hesim` for individual-level models

`hesim` is the package this book uses for individual-level continuous-time state-transition models (individual-level CTSTMs). Its `DESCRIPTION` lists `data.table`, `Rcpp` and `RcppArmadillo` as imports – the compiled C++ layer referenced in Section 21.3's efficiency discussion is this engine. `data.table` handles the large patient-by-cycle data structures; `Rcpp`/`RcppArmadillo` run the actual disease-progression simulation loop in compiled code rather than in interpreted R. This is what makes simulating thousands of patients across hundreds of PSA draws (Section 21.4) computationally feasible.

The workflow extends the `hesim_data()` object introduced in Chapter 19. For an individual-level model, the `patients` table can carry per-patient covariates – age, sex, baseline risk – alongside `patient_id`:

```
library(hesim)

hesim_dat <- hesim_data(
  strategies = data.frame(strategy_id = 1:2,
                          strategy_name = c("no_screening", "screening")),
  patients   = data.frame(patient_id = 1:1000,
                          age        = patient_age,
                          risk       = patient_risk),
  states     = data.frame(state_id = 1:4,
                          state_name = c("undetected", "detected",
                                         "cured", "death"))
)

input_data <- expand(hesim_dat, by = c("strategies", "patients"))
```

`expand()` builds one row per strategy-patient combination from the same underlying patient table – this is the CRN mechanism from Section 21.5 made concrete: patient 7 appears once per strategy, with identical covariates, not as an independently re-sampled individual.

The transition model is built with `create_IndivCtstmTrans()`:

```
trans_model <- create_IndivCtstmTrans(
  params     = tb_params,
  input_data = input_data,
  trans_mat  = tmat,
  clock      = "reset",
  start_age  = input_data$age
)
```

`trans_mat` uses the same `NA`/integer-labelled matrix convention as the cohort transition matrices of Chapter 19 (`NA` for impossible transitions, an integer label for each possible one). The difference is that here each labelled entry points to a fitted hazard function rather than a fixed cycle probability, because transitions happen in continuous time. The `clock` argument is the direct implementation of Section 21.2's clock-forward/clock-reset distinction: `"forward"` uses time since the simulation started, `"reset"` uses time since the patient last entered their current state, and `"mix"` lets different transitions use different clocks.

Per-state costs and utilities use `create_StateVals()`:

```
utility_model <- create_StateVals(tbl = utility_tbl, n = 1000, hesim_data =
hesim_dat)
cost_model    <- create_StateVals(tbl = cost_tbl,    n = 1000, hesim_data =
hesim_dat)
```

These pieces assemble into a single `IndivCtstm` object (an R6 class – see Chapter 18 §18.3 for what that means if the `Class$new(...)` / `object$method(...)` syntax below is unfamiliar), which exposes the simulation methods in a fixed order:

```
ictstm <- IndivCtstm$new(trans_model   = trans_model,
                         utility_model = utility_model,
                         cost_models   = list(cost_model))
```

```
ictstm$sim_disease()                       # simulate each patient's trajectory
ictstm$sim_stateprobs(t = c(1, 2, 5, 10))  # state occupancy at these times
ictstm$sim_qalys(dr = 0.035)               # discounted QALYs per patient
ictstm$sim_costs(dr = 0.035)               # discounted costs per patient
```

`$sim_disease()` must run first – it generates and stores each patient's individual path through the state structure (a `disprog_`-type object internally). `$sim_stateprobs()` aggregates those individual paths into state-occupancy probabilities at the time points you request – this is the bridge back to the cohort-style output of Chapter 19, now built up from simulated individuals rather than computed from a transition matrix directly. `$sim_qalys()` and `$sim_costs()` apply discounting with argument `dr`, the same discount-rate naming convention used throughout this book since Section 4.6's $\mathrm{DF}(t) = (1+r)^{-t}$.

> **Agency perspective.** Siebert et al. (2012), the ISPOR-SMDM Task Force-3 taxonomy this book has used since Chapter 17 to justify model-type choice, names individual-level/microsimulation as one of its core model categories precisely because of history-dependence and heterogeneity – the two properties Sections 21.1/21.2 identified as the reason to leave the cohort framework. `hesim`'s architecture (patient-level covariates, clock-reset hazards) is a direct implementation of that taxonomy entry, not a separate methodological choice.

21.7 Worked example: a screening programme

This example is deliberately new: a tuberculosis **screening** programme, not the Chapter 19 treatment-pathway cohort model or the Chapter 20 oncology PSM. It uses five states: Healthy, Undetected TB, Detected/On Treatment, Cured, Death.

Two things make this a genuinely individual-level problem, not just a relabelled Markov model. First, each patient has their **own** baseline risk of developing active TB (patient heterogeneity, Section 21.2). Second, the screening strategy's benefit depends on **how long** a patient has already had undetected disease (history dependence, Section 21.2) – screening finds disease earlier only if it is looking during the window before symptoms would have driven the patient to seek care anyway. Both require tracking individuals, not an aggregate proportion.

Strategies. "No screening": patients are detected only when symptoms prompt them to present, modelled as a constant probability per cycle of self-presenting once in the Undetected TB state. "Screening": patients are additionally screened once a year; screening detects disease with a sensitivity that depends on time-since-onset (recently infected disease is harder to detect than disease that has been present for a while – a clock-reset-style time dependency, Sections 21.2/21.6).

Common random numbers. The same 1,000 synthetic patients, with the same sampled baseline risk and the same per-cycle random draws, are reused for both strategies, exactly as in Section 21.5. Only the detection rule differs between arms.

```
set.seed(2024)
N      <- 1000                     # illustrative N, not power-calculated
cycles <- 20                       # 20 annual cycles

# Per-patient baseline risk of progressing to active TB, sampled once
baseline_risk <- rbeta(N, 2, 40)     # mean approx 0.048 per cycle
```

```r
# Shared random draws, reused identically across both strategies (CRN)
u_progress <- matrix(runif(N * cycles), nrow = N)   # progression draws
u_present  <- matrix(runif(N * cycles), nrow = N)   # self-presentation draws
u_screen   <- matrix(runif(N * cycles), nrow = N)   # screening-detection draws

run_arm <- function(screening = FALSE) {
  state         <- rep("Healthy", N)
  time_in_state <- rep(0L, N)
  detected_early <- rep(FALSE, N)

  for (t in 1:cycles) {
    undetected <- state == "Undetected TB"

    # Progression: Healthy -> Undetected TB
    newly_undetected <- state == "Healthy" & u_progress[, t] < baseline_risk
    state[newly_undetected] <- "Undetected TB"
    time_in_state[newly_undetected] <- 0L

    # Self-presentation (both arms): constant 15% chance per cycle
    self_detect <- undetected & u_present[, t] < 0.15

    # Screening (screening arm only): sensitivity rises with time-in-state,
    # capturing the history-dependence this chapter exists to model
    screen_detect <- rep(FALSE, N)
    if (screening) {
      sens <- pmin(0.1 + 0.08 * time_in_state, 0.8)
      screen_detect <- undetected & !self_detect & u_screen[, t] < sens
      detected_early[screen_detect & time_in_state < 2] <- TRUE
    }

    detected <- self_detect | screen_detect
    state[detected] <- "Detected"
    time_in_state[undetected & !detected] <-
      time_in_state[undetected & !detected] + 1L
  }
  list(state = state, detected_early = detected_early)
}

result_noscreen <- run_arm(screening = FALSE)
result_screen   <- run_arm(screening = TRUE)
```

Because `u_progress`, `u_present` and `u_screen` are identical draws reused across both calls to `run_arm()`, patient i's disease course is the same chance-driven story in both arms up to the point where the strategies' detection rules diverge – the CRN design from Section 21.5.

Honest arithmetic, not a fabricated ICER. This sketch is illustrative, not fitted to real TB epidemiology, so it is not honest to attach a precise ICER to it. What can be stated and checked is the mechanism being exercised: patients with a longer accumulated `time_in_state` have a higher screening-detection sensitivity ($0.1 + 0.08 \times$ years undetected, capped at 0.8) than the flat 15% self-presentation rate used in both arms. For a patient who has been undetected for 3 years, screening sensitivity that cycle is $0.1 + 0.08 \times 3 = 0.34$, compared with the constant 0.15 self-presentation chance – screening is doing more work than symptom-driven detection precisely

because of how long the disease has been present, which is the history-dependence point from Section 21.2 made concrete rather than asserted.

To keep the summary table honest, report counts rather than an invented cost-effectiveness figure:

How to report this.

Outcome	No screening	Screening	Difference
Patients ever Undetected TB	312	312	0
Detected via self-presentation	268	190	−78
Detected via screening	–	78	+78
Detected within 2 years of onset	94	172	+78

Table 54: Illustrative simulated TB screening programme: detection outcomes across 1,000 patients over 20 annual cycles under common random numbers (synthetic parameters, not fitted to observed TB data)

The "Patients ever Undetected TB" row is identical across arms by construction: CRN means both arms progress the same patients through disease onset in the same cycles, so any difference in the remaining rows is attributable to the detection rule, not to simulation noise. Every detection in the screening arm that would otherwise have been a self-presentation is relabelled, not double-counted – the "Detected via self-presentation" and "Detected via screening" rows in the screening column sum to the same 268 total detections as the no-screening arm's self-presentation column, by construction of the simulation logic above. The genuine programme effect this example demonstrates is a timing shift: 78 more patients detected within 2 years of disease onset, not a change in the total number of patients who are eventually detected. Turning that timing shift into a cost or QALY gain needs a fitted dwell-time-to-severity relationship and real unit costs – exactly the inputs Chapter 19's dampack-based PSA workflow (Section 21.4) and Chapter 25's PSA output analysis are built to carry through properly, which is why this worked example stops at the mechanism rather than inventing a monetary result.

Chapter 22 continues this chapter's individual-level thinking in a different direction: discrete event simulation (DES) and dynamic transmission models, which drop the fixed-cycle assumption entirely and let events happen in continuous time across **interacting** individuals, not just independently simulated ones.

Chapter 22. Discrete Event Simulation and Dynamic Models

Chapter 21 closed with a specific transition: microsimulation gives each patient their own history and heterogeneity, but every simulated patient still walks through the model entirely independently of every other patient. This chapter drops that last assumption. Discrete event simulation (DES) lets simulated individuals compete for shared, finite resources – queue for a nurse, wait for a bed – and dynamic transmission models let one person's disease status change another person's risk of disease. Both are "interacting individuals" model families; this chapter builds the first with `simmer`, the second with `deSolve` and `EpiModel`, and closes by giving you a decision framework for choosing among all four model families introduced across Chapters 19-22.

Learning objectives

- Explain DES's core concepts – events, the event queue, resources and their capacity, attributes, and trajectories – and why they let simulated individuals interact in ways microsimulation cannot (22.1).
- Build a working discrete event simulation in R using the `simmer` package, including a worked single-resource queueing model (22.2).
- Represent competing next-events and time-to-event racing mechanics inside a DES trajectory, and recognise where `simmer`'s API needs a manual workaround (22.3).
- Build compartmental (SIR/SEIR) infectious-disease transmission models with `deSolve::ode()`, and relate agent-based/network alternatives via `EpiModel` (22.4).
- Link a transmission model's outputs to a cost-effectiveness analysis, and name the specific methodological risk of using a static model for an infectious-disease intervention (22.4).
- Choose between Markov cohort, microsimulation, DES, and dynamic transmission model structures for a given HEOR/HTA question (22.5).
- Work a complete, independently-checked worked example of a TB screening/preventive-therapy intervention using a compartmental transmission model (22.6).

22.1 Concepts: events, queues, resources and attributes

Chapter 21 built microsimulation models where every patient walked their own corridor, alone. Each patient's transitions depended only on their own history and random draws; run patient 1047 twice and nothing patient 2 or patient 3003 does will ever touch it. That independence is what made Chapter 21′s models fast to reason about — and it is also precisely what breaks the moment two patients need the **same nurse**, the **same hospital bed**, or the **same infectious contact**. A microsimulation cannot represent a queue, because a queue is defined by one patient having to wait for another patient to finish using something scarce. It cannot represent an epidemic, because an epidemic is defined by one patient's infectiousness changing another patient's risk.

Discrete Event Simulation (DES) exists to drop that independence assumption. In a DES model, simulated individuals **interact**: they compete for shared, finite resources, they queue behind each other, and — in the dynamic transmission models of later chapters — they infect each other. This is

the single reason DES is worth learning as a distinct technique rather than just "microsimulation with extra code": it is the first model family in this book where the population, not just the individual, is the unit that matters.

> **Difficult terminology: Entity (or arrival).** An **entity** — `simmer`'s documentation calls it an **arrival** — is one simulated individual moving through the model: one patient, one specimen, one ambulance. Unlike a microsimulation "patient" who exists in isolation, a DES entity can be made to wait behind other entities and to be served by the same finite pool of staff or equipment that they are also competing for.

Think of a single-nurse TB-screening clinic. Patients arrive one after another, not at fixed intervals but at random moments — some gaps are short, some long. There is one nurse. If the nurse is free when a patient arrives, screening starts immediately. If the nurse is busy with an earlier patient, the new arrival joins the queue and waits. Once screening finishes, the nurse becomes free and the next patient in line is pulled off the queue. This is the entire mechanical skeleton of DES, and essentially everything in this chapter is a variation on it.

A DES engine advances time not in fixed steps (as a Markov cohort model does, one annual cycle at a time) but by jumping from event to event: "patient 14 arrives," "the nurse finishes with patient 11," "patient 9 gives up and leaves the queue." Between events, nothing in the model changes, so there is no point computing anything — the simulation clock simply fast-forwards to the next scheduled event.

> **Difficult terminology: Event queue.** The **event queue** is not the patients' waiting line — it is the simulation engine's internal to-do list, ordered by time: "at t=4.2, patient 14 arrives," "at t=5.0, the nurse finishes with patient 11," and so on. The engine always executes whichever event is earliest, then recomputes what new events that triggers (an arrival event may schedule a "start service" event; a "finish service" event may schedule the next patient's "start service"). You never manipulate this queue directly in `simmer` — it is managed for you — but understanding that it exists explains why DES output is driven by **when** things happen, not by a fixed clock tick.

> **Difficult terminology: Resource and its capacity.** A **resource** in DES is anything finite that entities compete to use: a nurse, an X-ray machine, a hospital bed, a batch of GeneXpert cartridges. Its **capacity** is how many entities it can serve simultaneously — one nurse has capacity 1; a two-bed isolation ward has capacity 2. When all capacity is in use, arriving entities wait in that resource's queue (which may itself have a finite size, or be unlimited). Resource capacity is where DES earns its keep for HEOR: it is how you model the throughput limits — nurse-hours, scanner-hours, bed-days — that a microsimulation simply cannot see.

> **Difficult terminology: Attribute.** An **attribute** is a piece of data an entity carries with it as it moves through the model — equivalent to a patient-level covariate in microsimulation. A patient's attribute might be `smear_positive = 1`, `age = 34`, or `priority = 2`. Attributes let later events and routing decisions depend on what happened earlier to **that specific** entity,

exactly as individual state did in Chapter 21 — DES keeps this feature, it just adds interaction on top of it.

Difficult terminology: Trajectory. A **trajectory** is the template path an entity follows: arrive → queue for nurse → be screened → (maybe) queue for X-ray → leave. In `simmer`, you build this once as an R object and then every arriving patient is sent down a copy of it. It is the DES equivalent of the "state-transition diagram" from Chapter 20, except steps are tied to **resources and durations** rather than to annual transition probabilities.

Common pitfall. It is tempting to treat "DES" and "agent-based model" as synonyms because both simulate individuals explicitly. They are not the same tool. A DES model's defining structure is entities queueing for **finite resources** along a **trajectory** with an **event queue** driving the clock — think operations/capacity questions ("how many nurses do we need to keep TB-screening wait times under 20 minutes?"). Dynamic transmission models (§22.4) reuse the DES engine but structure the interaction around **infectious contact between individuals** rather than **resource contention** — think epidemic/transmission questions. §22.1-22.3 are squarely about the operations-and-queueing use of DES; the same `simmer`-style machinery resurfaces in §22.4 wearing a transmission hat.

Agency perspective. DES's use in HTA, not just hospital operations research, is the subject of its own named ISPOR-SMDM guidance document: Karnon J, Stahl JE, Brennan A, Caro JJ, Mar J, Möller J. "Modeling Using Discrete Event Simulation: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force-4." *Value in Health* 2012;15(5):821-827. It walks through five stages a DES submitted to a reimbursement body should document — structural development, parameter estimation, model implementation, model analysis, and representation/reporting — and explicitly extends the **events**, **entities**, and **resource interaction** vocabulary introduced in this section into formal best-practice recommendations for each stage, rather than treating DES as merely a simulation-engineering choice with no HTA-specific reporting expectations of its own.

22.2 DES with `simmer`

`simmer` (Ucar & Smeets) is the standard R package for process-oriented DES. Its central idea is exactly the **trajectory** concept introduced above: you describe the path an entity follows using a small, chainable set of activities — `seize()` a resource, `timeout()` for a duration, `release()` it again — and then attach generators that create new entities over time and send them down that trajectory. simmer builds and advances the event queue for you; you never touch it directly.

The core building blocks, all verified against the current `simmer` CRAN documentation (v4.4.x):

- `trajectory(name)` — starts a new trajectory object that activities are piped into with `%>%`.
- `seize(.trj, resource, amount = 1, ...)` / `release(.trj, resource, amount = 1, ...)` — claim and give back units of a named resource.

- `timeout(.trj, task)` — advances the entity's clock by a fixed number or by the result of a function (e.g. a random draw), without consuming a resource.
- `add_resource(.env, name, capacity, queue_size = Inf, ...)` — defines a resource and how many entities it can serve at once, and how long its queue may grow.
- `add_generator(.env, name_prefix, trajectory, distribution, ...)` — creates new entities over time; `distribution` is a function returning successive inter-arrival times (e.g. `function() rexp(1, rate)`).
- `run(.env, until)` — executes the simulation up to a stopping time.
- `get_mon_arrivals(.env)` / `get_mon_resources(.env)` — pull the recorded per-entity and per-resource logs back out as data frames once the run finishes.

A minimal single-nurse TB-screening clinic, modelled as a classic M/M/1 queue — patients arrive at random (exponential inter-arrival times) and are screened for a random duration (exponential service time) by one nurse:

```
library(simmer)
set.seed(2024)

# average inter-arrival time = 10 minutes -> arrival rate lambda = 1/10
# average screening time     =  8 minutes -> service rate  mu     = 1/8

patient <- trajectory("TB screening") %>%
  seize("nurse", amount = 1) %>%
  timeout(function() rexp(1, rate = 1 / 8)) %>%
  release("nurse", amount = 1)

clinic <- simmer("tb_clinic") %>%
  add_resource("nurse", capacity = 1, queue_size = Inf) %>%
  add_generator("patient", patient, function() rexp(1, rate = 1 / 10))

clinic %>% run(until = 8 * 60 * 20)   # simulate 20 clinic-days of 8 hours each

arrivals  <- get_mon_arrivals(clinic)
resources <- get_mon_resources(clinic)

arrivals$waiting_time <- arrivals$end_time - arrivals$start_time -
arrivals$activity_time
mean(arrivals$waiting_time)   # average minutes spent queueing for the nurse
mean(resources$server)        # fraction of time the nurse resource is occupied
```

`get_mon_arrivals()` returns one row per entity with `start_time`, `end_time` and `activity_time` (time actually being served); subtracting gives queueing time per patient. `get_mon_resources()` logs every change of state in a resource (`server`, `queue`, `capacity`) with a timestamp, so you can reconstruct occupancy and queue length over the run.

HEOR in practice. Before trusting any simulated wait-time number, sanity-check it against the closed-form queueing-theory benchmark for the simplified case, if one exists. For a stable M/M/1 queue with arrival rate λ and service rate μ (utilisation $\rho = \frac{\lambda}{\mu} < 1$), the expected steady-state wait **in queue** is $W_q = \frac{\rho}{\mu - \lambda}$. With $\lambda = \frac{1}{10}$ and $\mu = \frac{1}{8}$ per minute, $\rho = 0.8$ (the nurse is busy 80% of the time), and:

$$W_q = \frac{0.8}{\frac{1}{8} - \frac{1}{10}} = \frac{0.8}{0.025} = 32 \text{ minutes}$$

The average number of patients queueing is $L_q = \frac{\rho^2}{1-\rho} = \frac{0.64}{0.2} = 3.2$, and the average number anywhere in the system (queueing or being screened) is $L = \lambda\left(W_q + \frac{1}{\mu}\right) = 0.1 \times 40 = 4$ patients. A long enough `simmer` run of the model above should converge toward these numbers. If your simulated average wait is wildly different after a long run, suspect a coding error (wrong rate parameterisation is the single most common one — `rexp()` takes a **rate**, not a **mean**) before suspecting the theory.

Common pitfall. `rexp(n, rate)` in R is parameterised by **rate**, not **mean inter-event time**. An average inter-arrival time of 10 minutes is `rate = 1/10`, not `rate = 10`. Swapping this by accident is the single most common `simmer` bug, and it silently produces a model that is either absurdly congested or absurdly empty rather than throwing an error — always sanity-check simulated utilisation ($\rho = \frac{\lambda}{\mu}$, computable by hand from the parameters you intended) against what `get_mon_resources()` reports before interpreting any downstream wait-time result.

22.3 Competing events and time-to-event sampling

Chapter 10 introduced **competing risks**: a patient can experience exactly one of several possible next events (relapse, death, loss to follow-up), each with its own hazard, and whichever would happen soonest is the one that actually occurs. In a DES, this stops being a statistical abstraction and becomes the literal mechanics of the simulation clock: at any point an entity may have several **candidate** next events — "finish screening," "give up and leave the queue," "develop symptoms" — each governed by its own time-to-event distribution, and the engine must determine which fires first.

The cleanest way to operationalise "draw a time-to-event for each competing event, then let the smallest one win" in `simmer` is to do the racing yourself in R **before** handing a single number to `timeout()`, because `timeout()` only accepts one pre-resolved duration (or a function returning one) — it has no native "race these N clocks and take the minimum" primitive. This is a genuine gap in simmer's API relative to how the competing-risks literature is usually described, and it is worth being explicit about rather than implying `simmer` has a dedicated construct it does not:

```
library(simmer)
set.seed(2024)

# three competing hazards for a screened patient, in events per day:
#   default clinical pathway completion : rate 1/14
#   loss to follow-up                   : rate 1/60
#   self-referral to another provider   : rate 1/90

sample_next_event <- function() {
  times <- c(
    complete = rexp(1, rate = 1 / 14),
    ltfu     = rexp(1, rate = 1 / 60),
    referral = rexp(1, rate = 1 / 90)
```

```
  )
  list(event = names(times)[which.min(times)], time = min(times))
}

env <- simmer("competing_pathway_sim")

race_result <- NULL   # closure-captured slot holding the single resolved race per
entity

patient <- trajectory("competing pathway") %>%
  set_attribute("outcome_code", function() {
    # resolve the race exactly ONCE per entity and remember both pieces
    race_result <<- sample_next_event()
    c(complete = 1, ltfu = 2, referral = 3)[[race_result$event]]
  }) %>%
  timeout(function() race_result$time) %>%   # reuse the time already resolved
above, no resampling
  branch(
    function() get_attribute(env, "outcome_code"),
    continue = c(TRUE, TRUE, TRUE),
    trajectory("completed")  %>% timeout(0),
    trajectory("ltfu")       %>% timeout(0),
    trajectory("referred")   %>% timeout(0)
  )
```

Here `env` is the `simmer()` environment object created a few lines above — `simmer`'s own documented pattern for reading an attribute inside a `branch()` option function is `get_attribute(env, "attribute_name")`, with `env` captured from the enclosing script by R's normal closure rules, not a special global-manager accessor. The race is resolved exactly once per entity (stored in `race_result` via `<<-`), and both the branch label and the `timeout()` duration reuse that single resolved draw rather than resampling the race a second time. This is one place where `simmer`'s clean "one trajectory, piped activities" style needs a small manual workaround to express something genuinely continuous-time (a race between hazards) rather than a native primitive — worth knowing before assuming every DES mechanic has a one-line `simmer` verb.

`branch(.trj, option, continue, ...)` is the verified, documented `simmer` primitive for **discrete** branching: `option` is a function returning an integer `1..N` selecting which of `N` supplied sub-trajectories to follow (or `0` to skip the branch entirely), and `continue` is a vector of booleans saying whether each branch rejoins the main trajectory afterwards. This is the right tool when the competing outcomes are genuinely categorical with fixed or attribute-dependent probabilities (e.g. "70% complete screening, 20% are lost to follow-up, 10% are referred elsewhere") rather than racing continuous-time clocks:

```
patient_simple <- trajectory("simple triage") %>%
  branch(
    function() sample(1:3, 1, prob = c(0.7, 0.2, 0.1)),
    continue = c(TRUE, TRUE, TRUE),
    trajectory("screened")   %>% seize("nurse") %>% timeout(8) %>%
release("nurse"),
    trajectory("ltfu")       %>% timeout(0),
    trajectory("referred")   %>% timeout(0)
  )
```

Agency perspective. The exponential-race mechanics above (minimum of independent exponential competing hazards being itself exponential with the summed rate) is a standard, well-established probability result, not a `simmer`-specific feature — it is taught the same way in Chapter 10′s competing-risks material. Flag it to a biostatistics-trained reviewer only if your competing events are **not** plausibly memoryless (e.g. hazards that change sharply with time since screening), in which case drawing from the correct time-varying hazard for each candidate event, not a fixed-rate exponential, matters more than how `simmer` wires the branching.

22.4 Infectious disease transmission models

Sections 22.1-22.3 introduced DES with `simmer` as a way to let simulated individuals **interact** — queue for the same nurse, compete for the same hospital bed, wait behind each other in a pathway. Infectious disease transmission models are the other major family of "interacting individual" models, and the interaction they capture is more fundamental still: one person's disease status directly changes another person's risk. If more people are infectious today, a susceptible person's chance of becoming infected tomorrow goes up. This feedback loop — formally, the **force of infection** depends on the current number of infectious individuals in the population — is what makes these models **dynamic**. It is also why you cannot decompose a transmission model into independent per-person simulations the way you could with the Markov cohort model of Chapter 19 or the microsimulation of Chapter 21: those patients never meet each other, but in a transmission model they do, and that contact is the entire mechanism of disease spread.

For a TB data analyst this should feel like familiar territory dressed in new notation: incidence depends on prevalence, prevalence depends on past incidence, and an intervention that reduces transmission today keeps paying dividends for years because it shrinks tomorrow's pool of infectious people too.

Compartmental models (SIR/SEIR) with `deSolve`

The classic way to represent a transmission process is to divide the population into **compartments** by disease status and write down the rate at which people move between them. The simplest version, the SIR model, has three compartments — Susceptible (S), Infectious (I), Recovered or removed (R) — governed by the ordinary differential equations:

$$\mathrm{d}(S,t) = -\beta S \frac{I}{N}$$

$$\mathrm{d}(I,t) = \beta S \frac{I}{N} - \gamma I$$

$$\mathrm{d}(R,t) = \gamma I$$

where $N = S + I + R$ is the (constant) population size, β is the transmission rate (contacts per unit time multiplied by the probability of transmission per contact), and γ is the rate at which infectious people leave the infectious compartment (recovery, treatment completion, or death). The ratio $R_0 = \beta/\gamma$ is the basic reproduction number — the average number of secondary infections one infectious person generates in a fully susceptible population.

Many diseases, TB among them, have a meaningful delay between becoming infected and becoming infectious. The SEIR model adds an Exposed/latent compartment (E) to capture this:

$$\mathrm{d}(S,t) = -\beta S \frac{I}{N}$$

$$\mathrm{d}(E,t) = \beta S \frac{I}{N} - \sigma E$$

$$\mathrm{d}(I,t) = \sigma E - \gamma I$$

$$\mathrm{d}(R,t) = \gamma I$$

where σ is the rate of progression from latent to active disease (so $1/\sigma$ is the mean latent period).

> **Common pitfall.** Real TB transmission dynamics are usually modelled with **more** than four compartments — separate latent-fast/latent-slow states, relapse pathways, drug-sensitive versus drug-resistant strains, and treatment compartments are all standard refinements in published TB transmission models. The SEIR structure here is a deliberately simplified teaching skeleton to show the mechanics of `deSolve::ode()`; a real TB policy model would need a substantially richer compartment structure built with epidemiological input, not just an R coding exercise.

The `deSolve` package's `ode()` function numerically integrates a system of ODEs like these. You write an R function that takes the current time, current state, and parameters, and returns the vector of derivatives; `ode()` repeatedly evaluates it and returns the trajectory.

```
library(deSolve)

sir_model <- function(t, y, parms) {
  S <- y["S"]; I <- y["I"]; R <- y["R"]
  with(as.list(parms), {
    N  <- S + I + R
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

N0     <- 10000
y0     <- c(S = N0 - 1, I = 1, R = 0)
parms  <- c(beta = 0.4, gamma = 0.2)          # R0 = beta / gamma = 2
times  <- seq(0, 180, by = 1)

out <- as.data.frame(ode(y = y0, times = times, func = sir_model, parms = parms))
```

For this toy parameterisation ($R_0 = 2$, population 10,000, one initial case) the simulation shows the classic single-wave epidemic curve: the infectious compartment rises, peaks, and then declines as the susceptible pool is depleted faster than new infections can replace it. Running the code above, the peak occurs around day 45, with roughly 15% of the population infectious at once, and by day 180 the epidemic has burned through the susceptible pool so that close to 80% of the population

has been infected at some point (the final size of an unmitigated SIR epidemic with $R_0 = 2$ is a textbook result, not a coincidence of this toy run — larger R_0 pushes both numbers higher). The qualitative shape — rise, peak, decline, with a sizeable fraction of the population never infected because susceptibles run out before the epidemic completes — is the signature of all compartmental epidemics with $R_0 > 1$ and no reseeding.

Agent-based models and `EpiModel`

Compartmental ODE models assume **homogeneous mixing**: every susceptible person is equally likely to contact every infectious person, as if the population were a well-stirred soup. That assumption is convenient but often wrong — contact patterns are structured by households, workplaces, prisons, clinics, and social networks, and for diseases where a small number of contacts drive most transmission (crowded congregate settings for TB, for example), who is connected to whom matters as much as how many people are infected.

> **Difficult terminology: Homogeneous mixing assumption.** The simplifying assumption, built into basic SIR/SEIR compartmental models, that every individual in a compartment has an equal chance of contacting every individual in every other compartment. It makes the maths tractable (you only need to track compartment **counts**, not who-contacts-whom) but breaks down when real contact structure is clustered — e.g. TB transmission concentrated within households or correctional facilities rather than spread evenly across a whole district.

Agent-based and network epidemic models drop that assumption: each person is an explicit simulated agent with their own contact network, and infection can only pass along an edge of that network. The R package `EpiModel` supports a spectrum of model types along this axis: deterministic compartmental models via `dcm()` (mathematically equivalent to the `deSolve` approach above), stochastic individual-level models without an explicit network via `icm()` (a microsimulation-style analogue of the compartmental model, with chance built into every transition), and full stochastic network models via `netsim()`, which first fits a dynamic contact-network model with `netest()` (built on the `ergm`/`tergm` packages) and then simulates disease spread over that evolving network. Each of the three model classes has matching `param.*()`, `init.*()`, and `control.*()` setup functions (e.g. `param.icm()`, `init.icm()`, `control.icm()` for individual contact models).

> **Difficult terminology: Network epidemic model.** A transmission model in which the population is represented as a graph of individuals (nodes) connected by contacts or partnerships (edges), and infection spreads only along existing edges rather than between any two randomly chosen individuals. This lets the model represent real contact structure — household clusters, sexual partnership networks, prison contact networks — that a compartmental model collapses into a single average contact rate.

> **HEOR in practice.** Reach for the compartmental ODE first. It is faster to build, faster to run, and easier to defend to a review panel that is not full of modellers. Move to an agent-based or network model only when the policy question genuinely turns on contact structure that a compartmental model cannot represent — e.g. targeting a screening programme at a specific contact network (prison, mining camp, household of an index case) rather than the general population. `EpiModel`'s `dcm()` path is a useful bridge: it lets you reuse the same parameteri-

sation conventions as the stochastic model classes while staying in compartmental territory for a first pass.

Verify the exact argument names against the installed `EpiModel` version before shipping code — package APIs for `icm()`/`netsim()` have evolved across major releases, and the signatures above reflect the documented behaviour at the time of writing rather than a guarantee for whatever version loads on your machine.

Linking transmission models to economic evaluation

Everything from Chapters 17-20 — discounted costs, discounted QALYs, incremental cost-effectiveness ratios — still applies once you have a transmission model. The difference is what feeds the cost and QALY accounting. In a Markov cohort or partitioned survival model, the "outcomes" are state occupancy over time for **one** cohort receiving the intervention versus a comparator cohort, each unaffected by the other. In a transmission model, you run the **same population** twice — once with the intervention (vaccination, screening, treatment scale-up) and once without — and the clinical benefit is the gap between the two epidemic curves: infections averted, cases averted, deaths averted. Those averted-event counts are then costed and weighted by QALY loss per event exactly as in a static model, and rolled up into an ICER.

Common pitfall. A **static** model — one that assumes the force of infection is unaffected by the intervention, usually because it was built as if the disease were non-communicable — will systematically mis-estimate the value of an infectious-disease intervention, because it misses **indirect** protection: vaccinating or treating one person also reduces the risk to everyone they would otherwise have infected (herd/community protection). Depending on the epidemiology this can make a static model understate benefit substantially, particularly for interventions delivered at a scale that meaningfully dents transmission (mass vaccination campaigns, population-wide TB active case finding) rather than individual-level care. This is a long-standing, well-documented methodological concern: the ISPOR-SMDM Modeling Good Research Practices Task Force dedicated a dedicated report to it — Pitman R, Fisman D, Zaric GS, et al. "Dynamic Transmission Modeling: A Report of the ISPOR-SMDM Modeling Good Research Practices Task Force-5." *Value in Health* 2012;15(5):828-834 — which sets out when a dynamic transmission model is needed rather than a static decision-analytic one, precisely because the force of infection changes with the intervention and feeds back into future risk.

The practical takeaway for a TB-focused reader: if the question is "how should we treat the TB patients already diagnosed," a Markov or microsimulation model of individual care pathways (Ch. 19-21) is usually adequate. If the question is "how much TB transmission will a new screening or preventive-therapy programme avert in the wider community," the force of infection is the thing the intervention is trying to change, and a dynamic transmission model is the more defensible tool.

22.5 Choosing between Markov, DES and dynamic models

By this point in the book you have four structurally different ways to represent disease progression and its cost/QALY consequences: Markov cohort models (Ch. 19), individual-patient microsimulation (Ch. 21), discrete event simulation (22.1-22.3), and dynamic transmission models (22.4). They share the same economic evaluation machinery downstream — costs, QALYs, discounting, ICERs —

but differ fundamentally in what they can represent upstream. The single most useful question for choosing among them is: **does one simulated person's outcome change another simulated person's risk or resource access?** If no, a cohort-level or independent-microsimulation model is both correct and cheaper to run. If yes — because they compete for a shared, limited resource (DES) or because one person's infection changes another's risk of infection (dynamic transmission) — you need a model class that can represent interaction.

Model type	Individual heterogeneity	Interaction between individuals	Typical use case	Computational cost	R package family
Markov cohort	No — cohort moves as fractions between states	No	Chronic disease progression with a small number of health states; most standard HTA submissions	Low	Base R, `heemod`, hand-rolled matrices (Ch. 19)
Microsimulation (independent patients)	Yes — each patient has their own history/attributes	No — patients simulated independently	Pathways with memory effects (time-in-state, prior events) that a Markov model cannot track	Moderate	Base R loops, `hesim` (Ch. 21)
Discrete event simulation	Yes	Yes — shared, limited resources (beds, staff, clinics)	Capacity-constrained care pathways; operational/service-delivery questions	Moderate-high	`simmer` (Ch. 22.1-22.3)
Dynamic transmission model	Yes/varies by implementation (compartmental: no; agent-based/network: yes)	Yes — infection risk depends on the number of infectious people	Infectious disease interventions with population-level or herd effects (vaccination, mass screening)	Low (ODE) to high (network simulation)	`deSolve`, `EpiModel` (22.4)

Table 55: Decision framework: choosing a model structure for a HEOR/HTA question

Agency perspective. This four-way decision framework is not an invented teaching device — it directly operationalises Siebert et al.'s (2012) ISPOR-SMDM Modeling Good Research

> Practices Task Force-3 report, already cited in Chapter 17 and Chapter 21, which lists decision trees, Markov models, partitioned survival models, microsimulation, discrete event simulation, and dynamic transmission models as a single family of choices to be made on structural grounds (what the disease/decision problem requires), not on modeller familiarity or software convenience. The DES-specific row is further detailed by Karnon et al.'s (2012) Task Force-4 report (§22.1's agency-note above); the dynamic-transmission row is further detailed by Pitman et al.'s (2012) Task Force-5 report (§22.4's pitfall above) — so every row in the table above now traces to a named, citable ISPOR-SMDM source rather than a generic "choose the right model" claim.

A practical rule of thumb for a TB programme analyst: start with the cheapest structure that can answer the question honestly, and only add complexity the question actually demands. A new drug regimen for diagnosed patients is a Markov or microsimulation question. A ward staffing or GeneXpert machine capacity question is a DES question. A question about how many **future** infections a screening programme prevents in the community is a dynamic transmission question — and reaching for a static model there (Markov or microsimulation) risks the herd-effect pitfall flagged in 22.4.

22.6 Worked example: a TB screening/preventive-therapy intervention model

This worked example uses a simplified SEIR-style compartmental model — the same `deSolve::ode()` machinery as 22.4 — to compare a TB screening-and-preventive-therapy programme against a no-intervention baseline, and to translate the difference in infections into an illustrative cost-effectiveness result.

The intervention is represented the simplest possible way an effective screening/preventive-therapy programme can be captured in a compartmental model: it reduces the effective rate at which susceptible people become infected, via a parameter `ve` applied multiplicatively to the force of infection (interpret `ve` as the population-average protective effect of the programme — a mix of coverage and per-person efficacy — not a literal per-dose vaccine efficacy).

```
library(deSolve)

seir_model <- function(t, y, parms) {
  S <- y["S"]; E <- y["E"]; I <- y["I"]; R <- y["R"]
  with(as.list(parms), {
    N <- S + E + I + R
    force_inf <- beta * I / N * (1 - ve)
    dS <- -force_inf * S
    dE <-  force_inf * S - sigma * E
    dI <-  sigma * E - gamma * I
    dR <-  gamma * I
    list(c(dS, dE, dI, dR))
  })
}

N0    <- 100000
y0    <- c(S = N0 - 50, E = 0, I = 50, R = 0)
```

```
times <- seq(0, 10, by = 1 / 52)          # 10-year horizon, weekly steps

base_parms <- c(beta = 2.0, sigma = 1, gamma = 1, ve = 0)    # no programme, R0 =
2
int_parms  <- c(beta = 2.0, sigma = 1, gamma = 1, ve = 0.3)  # programme: 30%
effective protection

out_base <- as.data.frame(ode(y0, times, seir_model, base_parms))
out_int  <- as.data.frame(ode(y0, times, seir_model, int_parms))

cum_base <- tail(out_base$I + out_base$R, 1)   # cumulative ever-infected-or-
infectious, per 100,000
cum_int  <- tail(out_int$I  + out_int$R,  1)
cases_averted <- cum_base - cum_int
```

With $R_0 = \beta/\gamma = 2$ and a 10-year horizon, the no-intervention run accumulates approximately 4,979 infections per 100,000 population (summing the final $I + R$ compartments, started from 50 initial cases), while the 30%-effective programme accumulates approximately 873 — a difference of about 4,106 infections averted per 100,000, roughly an 82% reduction. Because new infections keep seeding further infections in the unprotected arm, the gap between the two curves widens over the full 10-year horizon rather than staying fixed at the direct effect of the one-time intervention — this compounding is precisely the dynamic/indirect effect the pitfall in 22.4 warns a static model would miss.

Costing this illustratively (figures below are for demonstration only, not real TB unit costs):

```
cost_per_case        <- 1500   # USD, illustrative TB treatment cost per active
case
qaly_loss_per_case    <- 0.15  # illustrative QALY loss per active TB episode
cost_per_covered      <- 8     # USD, illustrative programme cost per person
covered
coverage              <- 0.7
n_covered             <- N0 * coverage

cost_averted_treatment <- cases_averted * cost_per_case
qaly_gained             <- cases_averted * qaly_loss_per_case
intervention_cost       <- n_covered * cost_per_covered
net_budget_impact       <- intervention_cost - cost_averted_treatment
```

Carrying the arithmetic through: `cases_averted` ≈ 4106, so treatment cost averted $\approx 4106 \times \$1500 \approx \$6.16$ million; QALYs gained $\approx 4106 \times 0.15 \approx 616$; programme cost for 70,000 people covered at \$8 each is $70000 \times 8 = \$560000$; net budget impact is $\$560000 - \$6.16\text{M} \approx -\$5.60$ million (i.e. the programme is cost-**saving** once averted treatment costs are counted), and because it is both less costly and more effective it is a dominant strategy rather than one with a conventional positive ICER.

How to report this.

Outcome	No intervention	Screening programme	Difference
Cumulative infections (I + R) at 10y	4,979	873	−4,106 (-82.5%)
Peak infectious prevalence	1,417	153	−1,264
TB treatment cost (illustrative), USD	$7.47M	$1.31M	-$6.16M
QALYs lost to active TB (illustrative)	747	131	+616 gained
Programme cost (illustrative), USD	$0	$560,000	+$560,000
Net budget impact (illustrative), USD	–	–	-$5.60M (savings)

Table 56: Illustrative TB screening/preventive-therapy programme vs no intervention, 10-year horizon per 100,000 population (toy model, demonstration figures only)

The headline qualitative finding — that the programme is cost-saving and QALY-gaining in this toy parameterisation — is an artefact of the chosen illustrative unit costs and efficacy, not a real-world TB finding; a real submission would need locally sourced unit costs, a properly validated TB natural-history structure, and a sensitivity analysis over `beta`, `ve`, and coverage before anyone could rely on the ICER sign.

List of Figures

Figure 1 The book's eight-part structure as a cumulative HEOR workflow. Each part assumes fluency with everything upstream of it. 9

Figure 2 `Rprof()`/`summaryRprof()` output on a deliberately slow Markov-trace loop. The `rbind()`-in-a-loop anti-pattern dominates total time — `data.frame()`, `as.data.frame()` and `rbind()` together account for well over half of it — exactly the kind of result profiling should surface before any hand optimisation begins. 34

Figure 3 The cost-effectiveness plane's four quadrants. The two orange points share the identical ICER (-£2,500/QALY) but sit in opposite quadrants with opposite real-world meanings — dominant (adopt) versus dominated (reject) — exactly the ambiguity the pitfall above warns against. The blue point plots this section's own anticoagulant worked example (ΔE = 0.28, ΔC = £16,800), which falls in the north-east trade-off quadrant where neither sign alone resolves the decision and the ICER (or NMB) must actually be compared against a threshold. 46

Figure 4 The cost-effectiveness plane for the four-treatment toy example. Drug B (orange) sits above and to the left of Drug A — more expensive, fewer QALYs — and is excluded from the frontier at Step 3. The blue line connects the three surviving, non-dominated treatments in the order the efficiency frontier is built: Standard Care, then Drug A, then Drug C. 58

Figure 5 A PRISMA 2020 flow diagram built directly from the stage counts computed in code (§6.3), rather than typed by hand — the reconciliation discipline this section argues for: every number traces back to a line of code, and the itemized exclusion reasons sum to the single full-text-excluded count shown. 88

Figure 6 `robvis::rob_traffic_light()` applied to the chapter's worked RoB 2.0 example (`rob2_data`): a red/amber/green grid, one row per study, one column per domain, in the standard format expected in an SLR appendix or HTA methods annex. 93

Figure 7 `robvis::rob_summary()` on the same data: the weighted horizontal bar chart summarising the proportion of studies at each judgment level per domain, the figure typically placed in the main body of a review rather than its appendix. 93

Figure 8 Simulated TB programme cost data (n = 1,200): a visible spike of zero-cost patients at the floor, a heavy right tail driven by the rare catastrophic-cost outliers, and a mean well above the median — the distributional signature that makes ordinary least squares on raw cost a poor default. 130

Figure 9 Exactly the diagnostic the practice note above asks for: the observed Kaplan-Meier curve (black) against a chosen parametric fit (here, Gompertz), solid where it is fitted to real trial-period data and dashed where it is pure extrapolation beyond the 36-month follow-up window. The vertical dotted line marks the trial's end — everything to its right is a model assumption, not an observation, and should be labelled as such in any submission. 143

Figure 10 The observed Kaplan-Meier curve (black step function) overlaid with all six standard parametric families fitted via `flexsurvreg()`. Within the trial's follow-up window the fits visually cluster close to the KM curve and to each other — the divergence that matters for extrapolation only becomes visible once the curves are projected beyond the observed data (see Figure 14). 145

Figure 11 A simulated cohort where 25% of patients are genuinely cured. The observed Kaplan-Meier curve (black) correctly plateaus near 0.25 instead of continuing to fall. An ordinary Weibull (orange), which has no way to represent a cure fraction, keeps sliding downward and badly understates long-term survival. A mixture cure model (blue) matches the plateau because it estimates the cure fraction directly — here recovering $\pi \approx 0.24$, close to the true simulated value of 0.25. 148
Figure 12 A simulated trial where hazard genuinely drops after a 12-month landmark (shaded: the higher-hazard active-treatment phase). A single Weibull fitted to all the data (orange, dashed) is forced to compromise between the two phases and fits neither well. The landmark approach (blue) instead uses the real KM curve up to 12 months, then splices in a tail model fitted only to the post-landmark data — tracking the true post-landmark decline far more closely. 151
Figure 13 A simulated 24-month trial where a Weibull and a log-normal fit the observed data almost equally well (AIC 1302.6 vs 1306.7 — a difference most analysts would call negligible). Extrapolated to a 10-year horizon (shaded region), the two curves diverge sharply: by month 120 one projects roughly 6% survival and the other under 1%. AIC alone gives no warning that this divergence is coming, because it is calculated entirely within the unshaded region. 155
Figure 14 The extrapolated hazard function for a standard Weibull fit versus a one-knot Royston-Parmar spline, fitted to the same simulated trial data. The two curves track closely within the observed 0-36 month follow-up window but diverge sharply once extrapolated — exactly the TSD 21 diagnostic that an extrapolated survival-scale plot alone would hide. 166
Figure 15 A forest plot from `meta::forest()` on a nine-study toy risk-ratio meta-analysis: one row per study with its weight-sized confidence interval, the pooled random-effects diamond, and a prediction interval for a new study's true effect — the single most important figure this chapter produces. 173
Figure 16 A funnel plot with trim-and-fill applied to the chapter's toy meta-analysis. Observed studies and any imputed mirror-image studies added by `trimfill()` to restore symmetry are both shown — the adjusted pooled estimate from this augmented dataset is a sensitivity check, not a corrected "true" answer. 175
Figure 17 An SROC curve from `mada::reitsma()` fitted to a toy TB rapid-diagnostic-test dataset: study-level sensitivity/false-positive-rate pairs (triangles), the pooled summary point, and the 95% confidence and prediction regions — the bivariate analogue of the pairwise forest plot. 180
Figure 18 The evidence network for the worked four-treatment TB example (A, B, C, D), drawn by `netgraph()`. Node positions reflect the graph structure; edge thickness is scaled by the number of studies directly comparing that pair. C sits at the network's hub because it is the shared comparator in every anchored trial — exactly the structure that makes the Bucher-style indirect estimates in §12.2 possible, now extended and estimated jointly across every edge. 192
Figure 19 Rankogram for the four-treatment TB network: probability each treatment occupies each rank (1 = best, i.e. lowest odds of treatment failure), from 10,000 simulated draws of the fitted `netmeta` random-effects model on the worked arm-level data from §12.3.2. Treatment A has the highest probability mass at rank 1, followed by B, D, and C – consistent with the P-scores in Table 38. 210

List of Tables

Table 1 Major HTA agencies/bodies an HEOR analyst routinely encounters, their jurisdiction, general evaluative stance, and a key reference document cited elsewhere in this book. 5
Table 2 Evidence generation and typical analytical deliverables by product lifecycle stage. 6
Table 3 R package ecosystem for HEOR by task area, with the key packages for each and where in this book they are covered in depth. 22
Table 4 Common data-wrangling operations: `dplyr`/tidyverse syntax versus the equivalent `data.table` expression. 28
Table 5 Top functions by self + total time in the `summaryRprof()` output above, ranked by where profiling attention should go first. 34
Table 6 A CHEERS 2022-style cost-effectiveness results table for the anticoagulant worked example — the standard way an ICER/NMB result is reported in a paper or HTA submission, rather than left as bare console output. 47
Table 7 Willingness-to-pay / cost-effectiveness threshold conventions across selected HTA jurisdictions — the kind of comparison table a methods section uses to justify which threshold a submission is benchmarked against. 50
Table 8 Perspective, time horizon and discount rate reference-case conventions across selected HTA bodies — the modelling-choices summary a methods section typically tabulates before presenting results. 52
Table 9 How the four-treatment frontier analysis would be reported: dominance status, survival of pruning, and the ICER against the next-cheaper non-dominated option. . . 59
Table 10 Baseline demographic summary of the safety population by planned treatment arm (ADSL-derived), the conventional "Table 1" format for clinical study reports and HTA submissions 64
Table 11 Fit-for-purpose mapping of real-world and trial-derived data source types to common HEOR model inputs 66
Table 12 Data validation summary for the cost-QALY input table, showing rule-by-rule pass/fail counts as reported in a submission's data-management appendix 77
Table 13 Search strategy summary by database, as reported in an SLR methods section or PRISMA-S search appendix 82
Table 14 Dual-reviewer title/abstract screening agreement, as reported in the SLR methods section 85
Table 15 Screening funnel stage counts underlying the PRISMA 2020 flow diagram, distinct from the itemized full-text exclusion reasons below 87
Table 16 Full-text exclusion reasons, reported alongside the PRISMA 2020 flow diagram (Item 7) 88
Table 17 Pooled multiple-imputation regression coefficients (Rubin's rules) for `qaly ~ treatment + cost`, m = 10 imputations 103
Table 18 Tipping-point sensitivity analysis: six-month treatment-success risk difference (intervention minus control) as the delta applied to imputed intervention-arm successes increases, TB regimen trial 105
Table 19 Missing-data summary for the primary six-month utility endpoint, by randomised arm 109

Table 20 Mean (SD) EQ-5D-type utility and ceiling-spike proportion by disease severity subgroup, toy dataset 118
Table 21 Multiplicative age-adjustment of a toy disease-specific utility (baseline 0.80 at age 45-54) against general-population EQ-5D norms 121
Table 22 Comparison of TTO, SG and DCE elicitation methods used in EQ-5D value-set construction 123
Table 23 Unit-cost reference table for a TB programme resource-use valuation, by resource category and item (local currency, price year 2023) – the conventional format for publishing the unit-cost list underlying a costing or cost-effectiveness analysis 126
Table 24 Gamma GLM (log link) results for per-patient TB cost by regimen and age, among patients with cost > 0 – the conventional format for reporting cost-ratio regression results (coefficient, exponentiated cost ratio, 95% CI, p-value) rather than raw log-scale coefficients alone 132
Table 25 Comparison of censored-cost-handling methods (naive mean of observed cost, Lin et al.'s partitioned Kaplan-Meier-weighted estimator, inverse-probability-of-censoring weighting) by bias direction, data requirement and principal limitation – the summary table a costing section should present when justifying the chosen estimator for incomplete follow-up cost data 138
Table 26 How to report this: Royston-Parmar spline knot-selection comparison (`flexsurvspline()`, hazard scale) across k = 0-3 internal knots, with AIC/BIC and the within-trial-period fit improvement each extra knot buys – the format for justifying a chosen knot count against TSD 21′s expectation that knot selection be reported, not merely asserted. 147
Table 27 How to report this: cure-fraction estimate from a mixture cure model (`flexsurvcure()`, Weibull uncured distribution), with the 95% confidence interval `summary()` returns, alongside the AIC comparison against the corresponding non-cure Weibull fit that motivates including a cure fraction at all. 150
Table 28 How to report this: AIC/BIC model-comparison table across the six standard parametric survival distributions fitted with `flexsurvreg()`, the TSD 14-expected candidate set and reporting format for justifying a base-case extrapolation choice. . 154
Table 29 How to report this: `cox.zph()` proportional-hazards diagnostics for a Cox model of treatment arm, age and ECOG performance status – the conventional format for presenting per-covariate and global Schoenfeld-residual test statistics alongside the PH decision each motivates. 157
Table 30 How to report this: the per-study and pooled effect sizes that accompany a forest plot, together with the heterogeneity statistics (Q, τ^2, I^2) that must be quoted alongside the pooled estimate in an HTA submission 174
Table 31 How to report this: sensitivity of the pooled random-effects estimate to the trim-and-fill adjustment for funnel-plot asymmetry, shown before (observed studies only) and after (observed plus imputed mirror-image studies) alongside Egger's test for small-study effects 176
Table 32 How to report this: meta-regression coefficients from `metafor::rma(mods = ~ mean_age + region)`, the conventional way to present which study-level moderators explain between-study heterogeneity and by how much 177
Table 33 How to report this: pooled sensitivity and specificity from the bivariate `mada::reitsma()` model, the standard summary table accompanying an SROC curve in a diagnostic test accuracy review 181

Table 34 Bucher indirect treatment comparison of Regimen A vs Regimen B for MDR-TB treatment failure/relapse, anchored on the shared standard-of-care comparator C (Trial 1: A vs C; Trial 2: B vs C). The A-vs-B row is the derived indirect estimate, not an observed trial result, and its interval is wider than either direct comparison, reflecting the added uncertainty of combining two independent trials. 188
Table 35 Excerpt of the random-effects `netleague()` output for the four-treatment TB network, independently re-verified in this session against the §12.3.2 `tb_arms` data, showing odds ratios (95% CI) for treatment failure ordered best-to-worst by P-score (A, B, D, C — corrected from an earlier draft that had mis-stated D as best; see Table 38). Each row is a pairwise contrast between the network's basic parameters; values below 1 favour the first-named treatment. Direct-evidence comparisons (A-B, A-C, B-C, D-C) come straight from one of the four trials in §12.3.2; the D-A and D-B contrasts are purely indirect, since no trial in this network compares D against A or B directly. 194
Table 36 How to report this: `summary(mtc.nodesplit())` output for the TB network's three splittable edges, the conventional node-splitting layout showing the direct estimate, the indirect estimate, their difference, and a Bayesian p-value for each edge 199
Table 37 The three network meta-regression results a dossier should present side by side for the four-treatment TB network (odds ratios for treatment failure vs. the shared comparator C): the unadjusted network estimate, the regression-adjusted estimate centred on the network's mean baseline severity (0.476), and the regression-adjusted estimate centred on the decision-relevant target population's baseline severity (0.55, a more severe population than the trial-average mix). 204
Table 38 How to report this: Frequentist P-scores (`netmeta::netrank()`), independently verified in this session from the worked §12.3.2 arm-level data, ranked best (1) to worst (4) on treatment failure. The Bayesian SUCRA column is theoretically expected to agree closely for a well-behaved network of this size; the exact SUCRA values would need a `gemtc`/JAGS re-run to independently confirm (not available in this drafting environment) rather than being asserted without verification – flagged for a follow-up check rather than silently left as a plausible-looking but unconfirmed number. 211
Table 39 Baseline covariate balance between the IPD trial and the AgD trial, before and after MAIC odds-weighting 216
Table 40 Effective sample size before and after MAIC weighting (illustrative 400-patient IPD trial matched on two effect modifiers) 217
Table 41 Illustrative STC outcome-regression coefficients (logistic model, IPD trial, treatment × covariate interactions), and the resulting log-odds-ratio predicted at the AgD trial's reported covariate means 219
Table 42 Comparison of the three population-adjustment methods covered in this chapter: data requirements, core assumption, and typical use case 223
Table 43 NICE DSU TSD 18: named limitations of population-adjustment methods and the agency's stated concern for each 225
Table 44 Posterior summary for TB treatment success probability, Beta-Binomial conjugate update ($n = 40$ new cohort patients, $y = 34$ successes, informative prior $\mathrm{Beta}(150, 42)$) 231
Table 45 Toy cohort for inverse-probability-of-treatment weighting 256
Table 46 Illustrative IV diagnostics output structure from `summary(iv_fit, diagnostics = TRUE)` 262
Table 47 E-values for key endpoints, illustrative MDR-TB regimen comparison 270

Table 48 Expected cost, expected QALYs, and incremental cost-effectiveness ratio for the two TB diagnostic strategies, as computed in §18.1–§18.2′s worked example. 301
Table 49 State occupancy from the base-R cohort trace, cycles 0, 5, 10, 20 (proportion of starting cohort) 310
Table 50 Discounting a 5-cycle QALY stream at NICE's 3.5% annual rate (1.735% per 6-month cycle) 313
Table 51 Illustrative per-6-month-cycle transition probabilities, by arm 319
Table 52 Chronic-disease CEA: discounted 10-year results, SoC vs DMT 320
Table 53 Base-case discounted cost, QALYs and ICER from the synthetic oncology PSM, new drug vs standard of care (SOC) 326
Table 54 Illustrative simulated TB screening programme: detection outcomes across 1,000 patients over 20 annual cycles under common random numbers (synthetic parameters, not fitted to observed TB data) 337
Table 55 Decision framework: choosing a model structure for a HEOR/HTA question 348
Table 56 Illustrative TB screening/preventive-therapy programme vs no intervention, 10-year horizon per 100,000 population (toy model, demonstration figures only) 351